S4D400

Grundlegende ABAP-Programmierung

TEILNEHMERHANDBUCH
PRÄSENZSCHULUNG

Version der Schulung: 2601
Dauer der Schulung: 4 Tage

Beachten Sie, dass diese Schulung maschinell übersetzt wurde und die Genauigkeit der Übersetzung daher möglicherweise nicht mit der einer menschlichen Übersetzung vergleichbar ist. Greifen Sie im Zweifelsfall auf die Sprachversion für US-Englisch zurück.

SAP-Urheberrechte, Marken und Haftungsausschlüsse

© 2026 SAP SE oder ein SAP-Konzernunternehmen. Alle Rechte vorbehalten.

Weitergabe und Vervielfältigung dieser Publikation oder von Teilen daraus sind, zu welchem Zweck und in welcher Form auch immer, ohne die ausdrückliche schriftliche Genehmigung durch SAP SE oder ein SAP-Konzernunternehmen nicht gestattet.

SAP und andere in diesem Dokument erwähnte Produkte und Dienstleistungen von SAP sowie die dazugehörigen Logos sind Marken oder eingetragene Marken der SAP SE (oder von einem SAP-Konzernunternehmen) in Deutschland und verschiedenen anderen Ländern weltweit. Weitere Hinweise und Informationen zum Markenrecht finden Sie unter https://www.sap.com/corporate/en/legal/copyright.html

Die von SAP SE oder deren Vertriebsfirmen angebotenen Softwareprodukte können Softwarekomponenten auch anderer Softwarehersteller enthalten.

Produkte können länderspezifische Unterschiede aufweisen.

Diese Materialien wurden unter Umständen maschinell übersetzt und können grammatikalische Fehler oder Ungenauigkeiten enthalten.

Die vorliegenden Unterlagen werden von der SAP SE oder einem SAP-Konzernunternehmen bereitgestellt und dienen ausschließlich zu Informations-zwecken. Die SAP SE oder ihre Konzernunternehmen übernehmen keinerlei Haftung oder Gewährleistung für Fehler oder Unvollständigkeiten in dieser Publikation. Die SAP SE oder ein SAP-Konzernunternehmen steht lediglich für Produkte und Dienstleistungen nach der Maßgabe ein, die in der Vereinbarung über die jeweiligen Produkte und Dienstleistungen ausdrücklich geregelt ist. Keine der hierin enthaltenen Informationen ist als zusätzliche Garantie zu interpretieren.

Insbesondere sind die SAP SE oder ihre Konzernunternehmen in keiner Weise verpflichtet, in dieser Publikation oder einer zugehörigen Präsentation dargestellte Geschäftsabläufe zu verfolgen oder hierin wiedergegebene Funktionen zu entwickeln oder zu veröffentlichen. Diese Publikation oder eine zugehörige Präsentation, die Strategie und etwaige künftige Entwicklungen, Produkte und/oder Plattformen der SAP SE oder ihrer Konzern- unternehmen können von der SAP SE oder ihren Konzernunternehmen jederzeit und ohne Angabe von Gründen unangekündigt geändert werden. Die in dieser Publikation enthaltenen Informationen stellen keine Zusage, kein Versprechen und keine rechtliche Verpflichtung zur Lieferung von Material, Code oder Funktionen dar. Sämtliche vorausschauenden Aussagen unterliegen unterschiedlichen Risiken und Unsicherheiten, durch die die tatsächlichen Ergebnisse von den Erwartungen abweichen können. Die vorausschauenden Aussagen geben die Sicht zu dem Zeitpunkt wieder, zu dem sie getätigt wurden. Dem Leser wird empfohlen, diesen Aussagen kein übertriebenes Vertrauen zu schenken und sich bei Kaufentscheidungen nicht auf sie zu stützen.

Typografische Konventionen

Dieses Handbuch wurde vom Amerikanischen Englisch ins Deutsche übersetzt.

Die folgenden typografischen Konventionen werden in diesem Handbuch verwendet:

Diese Informationen werden in der Präsentation des Schulungsreferenten angezeigt.	
Demonstration	
Vorgehensweise	
Warnung oder Achtung	
Hinweis	
Zugehörige oder zusätzliche Informationen	
Moderierte Diskussion	
Steuerung der Benutzungsoberfläche	*Beispieltext*
Fenstertitel	*Beispieltext*

© Copyright. Alle Rechte vorbehalten.

Inhaltsverzeichnis

vii **Überblick über die Schulung**

1 **Kapitel 1 : Erste Schritte**

3 Lektion: Die Entwicklungsumgebung vorbereiten
8 Lektion: Einführung in ABAP
11 Übung 1 : Ein ABAP-Cloud-Projekt anlegen und das ABAP-Coding untersuchen
17 Lektion: Die Softwarestruktur und -logistik verstehen
21 Übung 2 : Ein ABAP-Paket anlegen
25 Lektion: Erste ABAP-Anwendung entwickeln
31 Übung 3 : Anwendung „Hello World“ erstellen
35 Lektion: ABAP-Entwicklung mit KI verstärken

47 **Kapitel 2 : Grundlegende Methoden und Konzepte anwenden**

49 Lektion: Grundlagen von ABAP verstehen
55 Lektion: Mit grundlegenden Datenobjekten und Datentypen arbeiten
66 Lektion: Daten verarbeiten
71 Übung 4 : Variablen deklarieren und Daten verarbeiten
77 Lektion: Kontrollstrukturen in ABAP verwenden
85 Übung 5 : Bedingte Verzweigungen implementieren
93 Lektion: Mit einfachen internen Tabellen arbeiten
99 Übung 6 : Mit einfachen internen Tabellen arbeiten
107 Lektion: Debugging eines ABAP-Programms
117 Übung 7 : ABAP-Code debuggen

131 **Kapitel 3 : Mit lokalen Klassen arbeiten**

132 Lektion: Eine lokale Klasse definieren
137 Übung 8 : Eine lokale Klasse definieren
141 Lektion: Instanzen für eine Klasse anlegen
147 Übung 9 : Instanzen anlegen und verwalten
155 Lektion: Methoden definieren und aufrufen
165 Übung 10 : Methoden definieren und aufrufen
176 Lektion: Kapselung zur Sicherstellung der Konsistenz verwenden
183 Übung 11 : Private Attribute und einen Konstruktor verwenden

197 **Kapitel 4 : Daten aus der Datenbank lesen**

199 Lektion: Eine Tabellendefinition untersuchen
203 Lektion: Grundlegende SELECT-Anweisungen implementieren
211 Übung 12 : Daten aus einer Datenbanktabelle auslesen
219 Lektion: Mit CDS-Views arbeiten
225 Übung 13 : Eine CDS-View-Entität analysieren und verwenden

© Copyright. Alle Rechte vorbehalten.

239 **Kapitel 5 : Mit strukturierten Datenobjekten arbeiten**

241 Lektion: Ein strukturiertes Datenobjekt deklarieren
247 Lektion: Mit strukturierten Datenobjekten arbeiten
257 Übung 14 : Ein strukturiertes Datenobjekt verwenden

265 **Kapitel 6 : Mit komplexen internen Tabellen arbeiten**

267 Lektion: Eine komplexe interne Tabelle deklarieren
274 Lektion: Mit komplexen internen Tabellen arbeiten
287 Übung 15 : Komplexe interne Tabelle verwenden

299 **Kapitel 7 : Datenbankänderungen mit Business Objects implementieren**

300 Lektion: Analysieren eines Business-Objekts
307 Übung 16 : Ein Business-Objekt analysieren
315 Lektion: Verwendung der Entity Manipulation Language
321 Übung 17 : Daten über EML modifizieren

329 **Kapitel 8 : Das ABAP-RESTful-Anwendungsprogrammiermodell beschreiben**

331 Lektion: Einführung in das Programmiermodell
335 Lektion: Die Architektur erkunden
343 Übung 18 : Datenbanktabelle kopieren
357 Übung 19 : OData-UI-Service generieren und Vorschau anzeigen
366 Lektion: ABAP-Logik hinzufügen
381 Übung 20 : Flugpreis validieren
389 Lektion: Verbesserung der Benutzerfreundlichkeit
395 Übung 21 : Benutzungsoberfläche anpassen

© Copyright. Alle Rechte vorbehalten.

Überblick über die Schulung

ZIELGRUPPE

Diese Schulung richtet sich an die folgenden Zielgruppen:

- Entwicklungsberater
- Entwickler

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

KAPITEL 1 Erste Schritte

Lektion 1

Die Entwicklungsumgebung vorbereiten 3

Lektion 2

Einführung in ABAP 8

Übung 1 : Ein ABAP-Cloud-Projekt anlegen und das ABAP-Coding untersuchen 11

Lektion 3

Die Softwarestruktur und -logistik verstehen 17

Übung 2 : Ein ABAP-Paket anlegen 21

Lektion 4

Erste ABAP-Anwendung entwickeln 25

Übung 3 : Anwendung „Hello World“ erstellen 31

Lektion 5

ABAP-Entwicklung mit KI verstärken 35

LERNZIELE

- Legen Sie ein ABAP-Cloud-Projekt an.
- Arbeiten Sie mit einem Entwicklungsobjekt.
- Legen Sie ein ABAP-Paket an.
- Legen Sie eine Anwendung „Hello World“ an.
- Erläutern Sie, wie ABAP-KI-Funktionen ABAP-Entwickler bei der Anwendungsentwicklung unterstützen können.
- Beschreiben Sie kurz, wie das ABAP AI SDK verwendet werden kann, um Geschäftsanwendungen KI-Funktionen hinzuzufügen.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 1
Lektion 1

Die Entwicklungsumgebung vorbereiten

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Legen Sie ein ABAP-Cloud-Projekt an.

ABAP-Cloud-Projekt

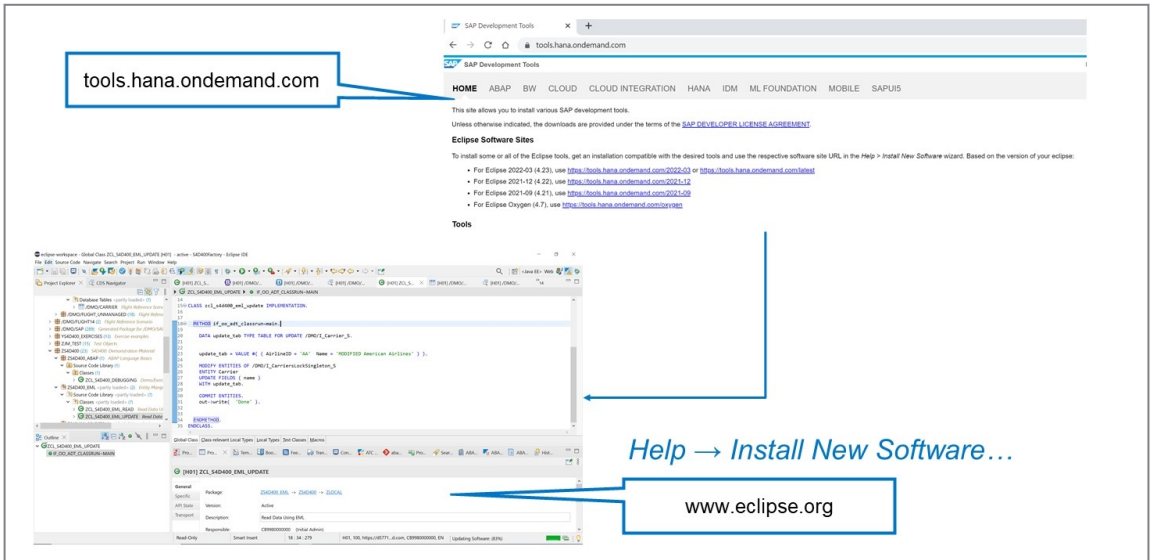


Abbildung 1: Eclipse und ABAP Development Tools installieren

Um ABAP-Anwendungen für SAP BTP zu entwickeln, benötigen Sie die Eclipse-Entwicklungsumgebung. Sie können sie kostenlos unter „eclipse.org“ herunterladen. Die standardmäßige Eclipse-Installation enthält nicht die ABAP Development Tools (ADT). Diese müssen Sie separat herunterladen. Öffnen Sie hierzu Eclipse, und wählen Sie *Help -> Install New Software*. Sie gelangen auf ein Dialogfenster, in dem Sie die URL https://tools.hana.ondemand.com/latest eingeben. Dadurch werden die ABAP-Werkzeuge heruntergeladen und in Eclipse installiert.

Sie sollten regelmäßig prüfen, ob sowohl für Eclipse als auch für die ABAP-Werkzeuge Aktualisierungen vorhanden sind. Wählen Sie hierzu in Eclipse *Help -> Check for Updates*.

Notiz:
In den meisten Präsenzschulungen verwenden wir einen Windows Terminal Server (WTS) mit einer vorkonfigurierten Eclipse-IDE-Installation.

© Copyright. Alle Rechte vorbehalten.

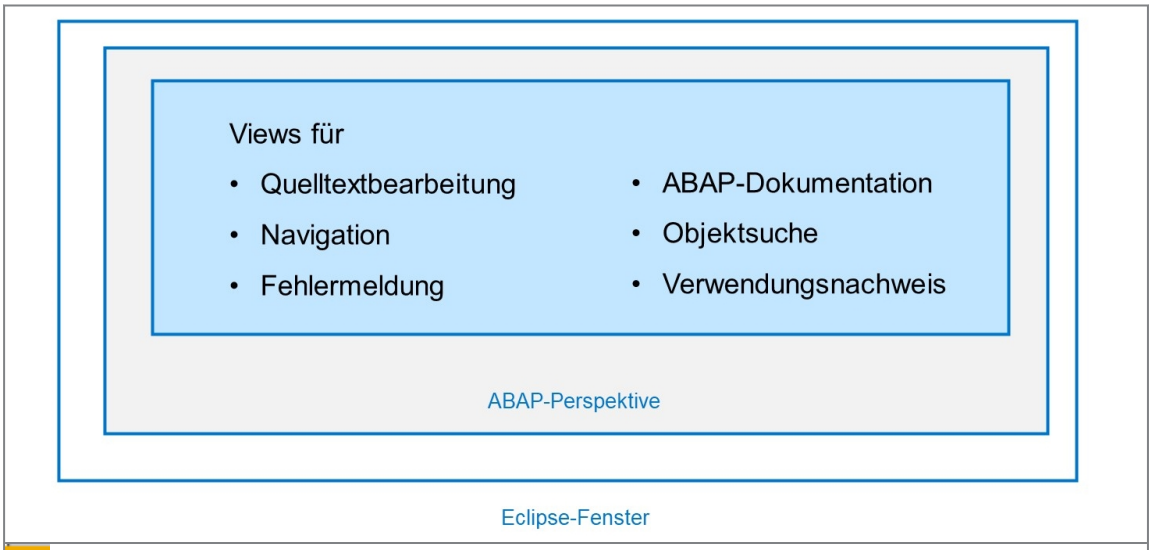


Abbildung 2: Die ABAP-Perspektive

Wenn Sie Eclipse und ADT installiert haben, müssen Sie zur ABAP-Perspektive wechseln. Eine Perspektive in Eclipse ist ein Fenster, in dem Sie eine bestimmte Aufgabe ausführen können. Die Standardperspektive in einer neuen Eclipse-Installation ist die Java-Entwicklungsperspektive. Um ABAP-Anwendungen zu entwickeln, wählen Sie *Window → Perspective → Open Perspective → Other*. Ein Dialogfenster mit einer Liste aller verfügbaren Perspektiven wird angezeigt. Doppelklicken Sie auf *ABAP*.

Eine Perspektive besteht aus einer Reihe von Views, die im Eclipse-Fenster als Registerkarten angezeigt werden. Die wichtigsten Views sind der Quelltext-Editor und der Project Explorer, mit denen Sie zwischen verschiedenen Entwicklungsobjekten navigieren. Sie benötigen zudem eine Vielzahl weiterer Views, z.B. die View „Problems“, um Fehlermeldungen anzuzeigen, die View „Documentation“ sowie Views für die Suche nach Objekten oder zur Anzeige, wo im System ein bestimmtes Objekt verwendet wird.

Verschiedene Arten von ABAP-Projekten

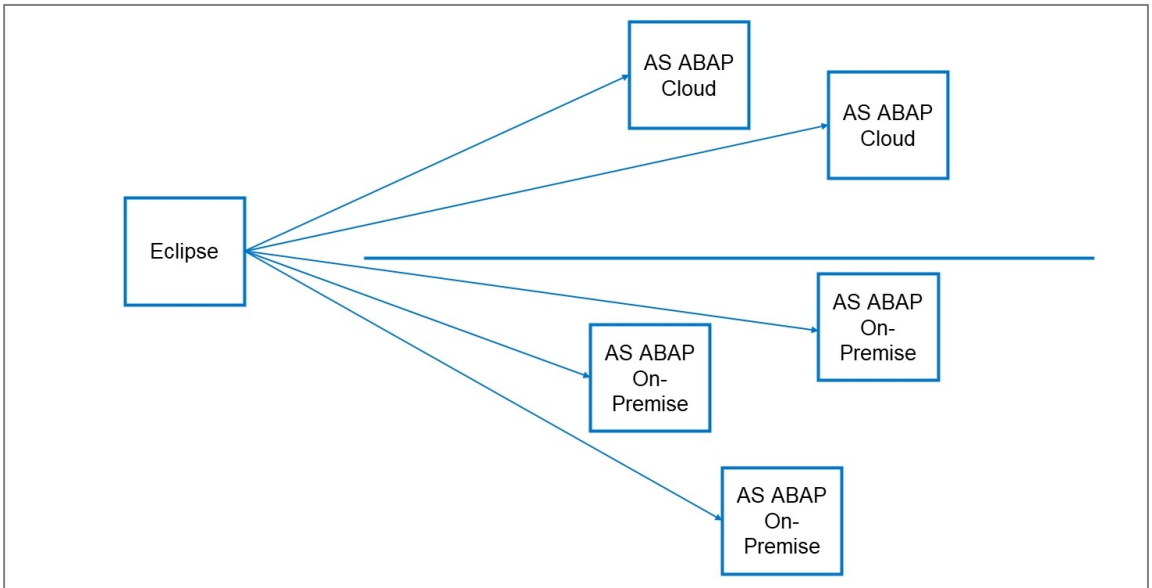


Abbildung 3: Verschiedene Arten von ABAP-Projekten

© Copyright. Alle Rechte vorbehalten.

Eclipse als Entwicklungsumgebung ist nicht in das ABAP-System integriert. Stattdessen müssen Sie eine Verbindung zu jedem ABAP-System herstellen, in dem Sie arbeiten möchten. Jede Verbindung wird in Eclipse durch ein Projekt dargestellt. Es gibt zwei Arten von Projekten in ADT: *ABAP-Projekte*, die Sie verwenden, um eine Verbindung zu einem On-Premise-ABAP-System oder zu SAP S/4HANA Private Cloud herzustellen, und *ABAP-Cloud-Projekte*, die Sie verwenden, um entweder eine Verbindung zu SAP BTP ABAP oder zu SAP S/4HANA Public Cloud herzustellen.

Notiz:
Im Folgenden wird nur der Zugriff auf ein Cloud-System besprochen.

Die ABAP-Umgebung suchen

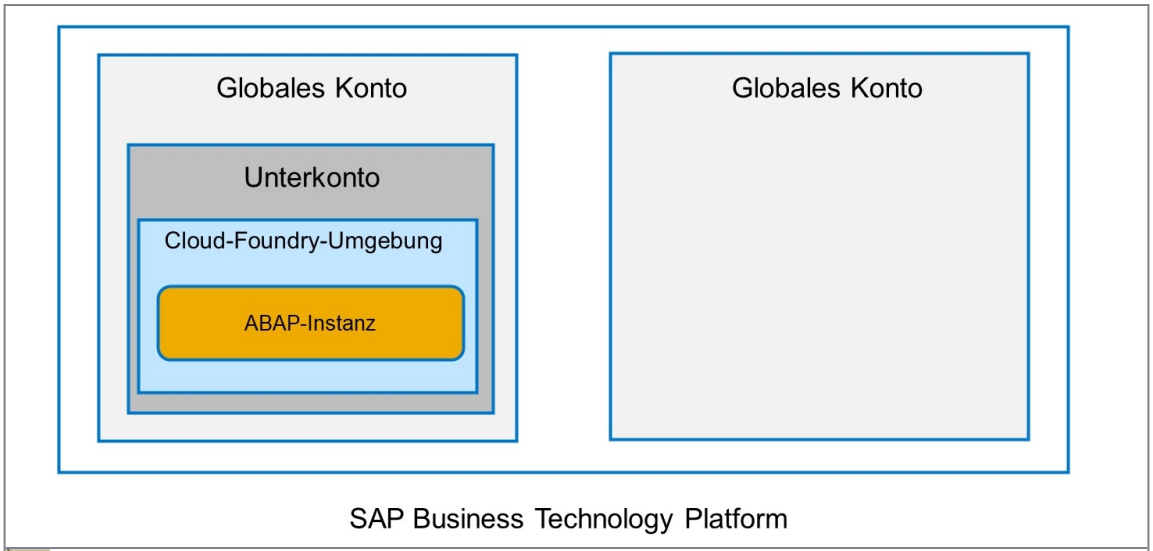


Abbildung 4: Die ABAP-Umgebung suchen

Die SAP Business Technology Platform ist die Platform as a Service (PAAS) der SAP. Um darauf zugreifen zu können, müssen Sie ein globales Konto anlegen. Es stehen verschiedene Abonnementmodelle zur Verfügung, je nachdem, ob Sie große Produktivumgebungen betreiben müssen oder nur eine Einzelbenutzerumgebung für eigene kontinuierliche professionelle Entwicklungsprojekte benötigen.

Innerhalb eines globalen Kontos gibt es mehrere Unterkonten. Jedes Unterkonto kann unterschiedlich konfiguriert werden, sodass ein einzelnes Unternehmen mehrere verschiedene Plattformen ausführen kann. Das Abonnement kann jedoch über das einzelne globale Konto verwaltet werden. Innerhalb des Unterkontos deployen Sie eine Laufzeit wie Cloud Foundry oder Kyma. Danach können Sie eine ABAP-Instanz deployen.

Notiz:
In diesem Kurs verwenden wir eine ABAP-Instanz, die auf der SAP Business Technology Platform (SAP BTP) implementiert ist. Das Material ist jedoch auch für andere ABAP-Deployments relevant, z.B. für ein SAP-S/4HANA-On-Premise-System oder ein SAP-S/4HANA-Cloud-System.

© Copyright. Alle Rechte vorbehalten.

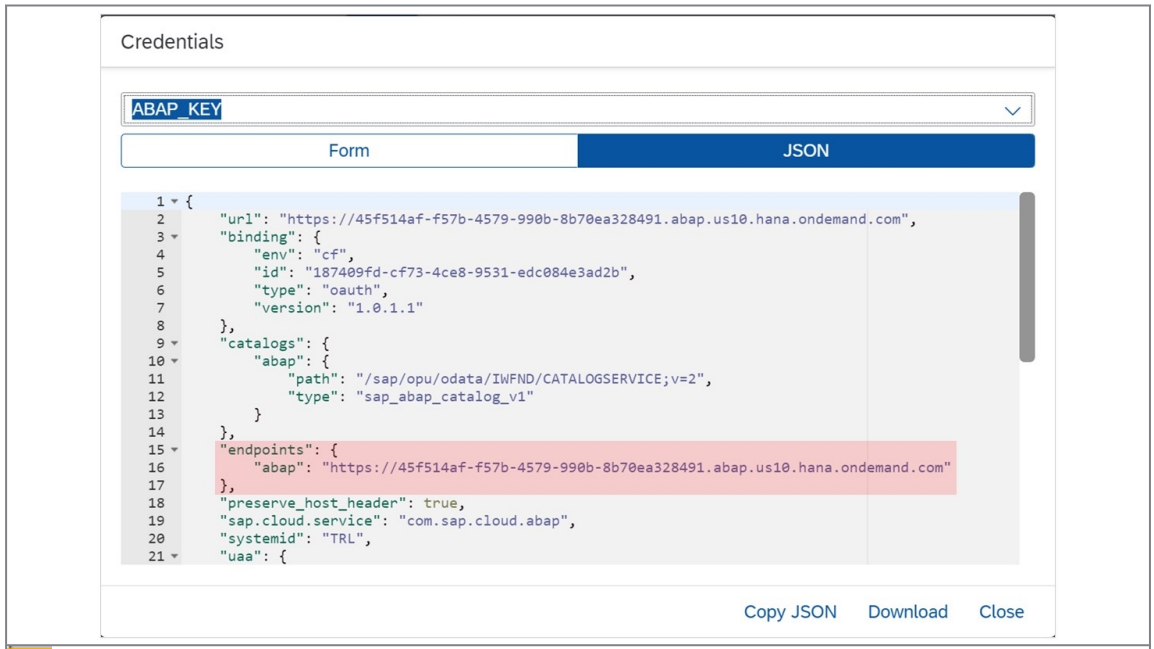


Abbildung 5: Die ABAP-Serviceinstanz-URL innerhalb des Serviceschlüssels

Um sich an einer ABAP-Instanz auf SAP BTP anzumelden, benötigen Sie die *ABAP-Serviceinstanz-URL* sowie einen Benutzer und ein Kennwort. Die *ABAP-Serviceinstanz-URL* wird im *Serviceschlüssel* angegeben. Der *Serviceschlüssel* ist spezifisch für die ABAP-Instanz und wird in SAP BTP angelegt, wo er in eine Textdatei heruntergeladen werden kann.

Notiz:
Für diese Schulung ist es nicht erforderlich, einen *Serviceschlüssel* anzulegen oder herunterzuladen. Ihr Referent stellt Ihnen die *ABAP-Serviceinstanz-URL*, einen Benutzernamen und ein Kennwort zur Verfügung.

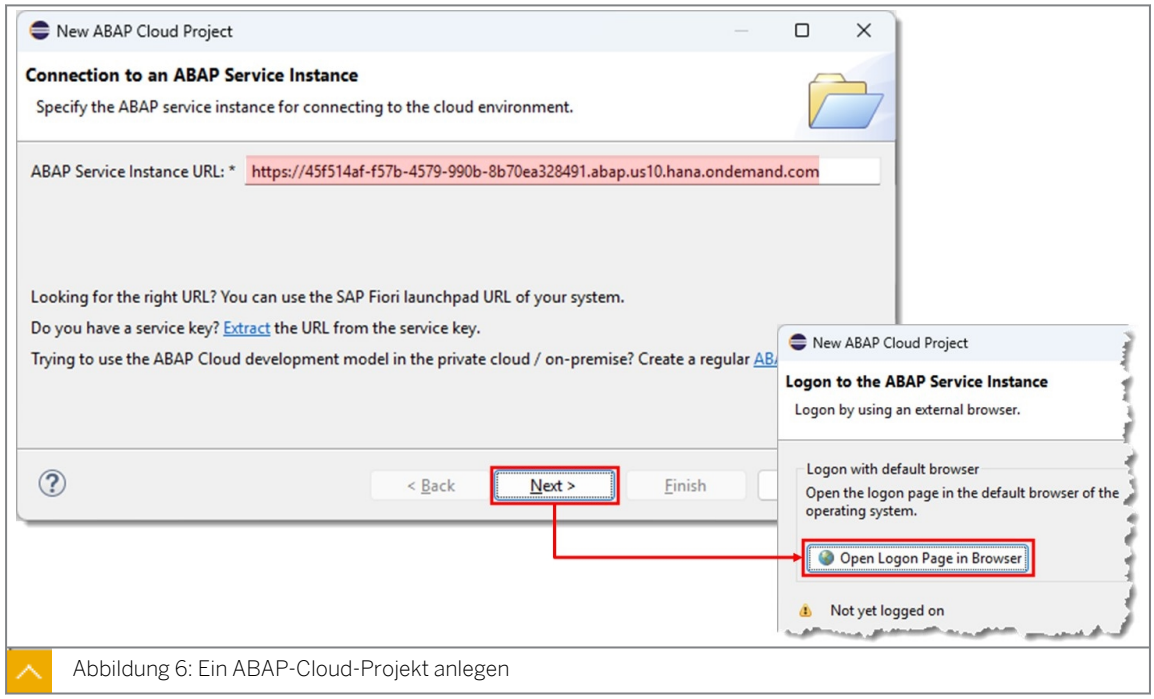


Abbildung 6: Ein ABAP-Cloud-Projekt anlegen

© Copyright. Alle Rechte vorbehalten.

Um ABAP zu entwickeln, müssen Sie sich an Ihrem ABAP-Konto anmelden. Gehen Sie dabei folgendermaßen vor:

1. Wählen Sie in Eclipse *File* → *New* → *ABAP Cloud Project*.
2. Sie gelangen auf ein Dialogfenster, in dem Sie die URL der *ABAP-Serviceinstanz* eingeben.
3. Wählen Sie *Weiter* und anschließend *Anmeldeseite im Browser öffnen*.
4. Melden Sie sich mit Benutzer und Kennwort an.
5. Schließen Sie das Browserfenster, und kehren Sie zu Eclipse zurück. Legen Sie die Sprache fest, die Sie verwenden möchten, und wählen Sie *Finish*.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Legen Sie ein ABAP-Cloud-Projekt an.

© Copyright. Alle Rechte vorbehalten.

Kapitel 1
Lektion 2

Einführung in ABAP

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Arbeiten Sie mit einem Entwicklungsobjekt.

Untersuchung von ABAP-Code

Im Project Explorer arbeiten

Um mit ABAP-Entwicklungsobjekten zu arbeiten, zeigen Sie in der Regel den Inhalt eines Pakets im Project Explorer an. Pakete sind Container für logisch zusammengehörende Entwicklungsobjekte.

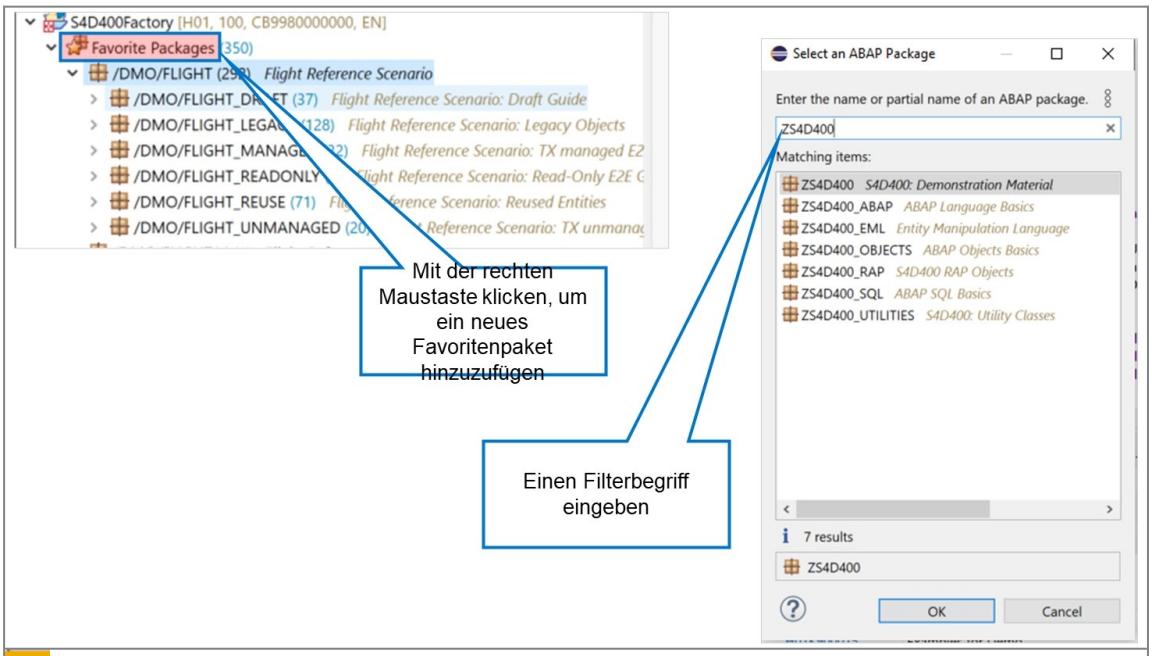


Abbildung 7: Der Project Explorer

Um ein Paket zu Ihren Favoriten hinzuzufügen, klicken Sie mit der rechten Maustaste im Project Explorer auf den Knoten *Favorite Packages* und wählen *Add Package*. Ein Dialogfenster wird geöffnet, in dem Sie einen Filterbegriff eingeben können. Das System zeigt dann nur die Pakete an, die diesen Begriff enthalten. Doppelklicken Sie auf das Paket, das Sie zu Ihren Favoriten hinzufügen möchten.

Entwicklungsobjekte öffnen

Es gibt zwei Möglichkeiten, ein Entwicklungsobjekt in ADT zu öffnen.

© Copyright. Alle Rechte vorbehalten.

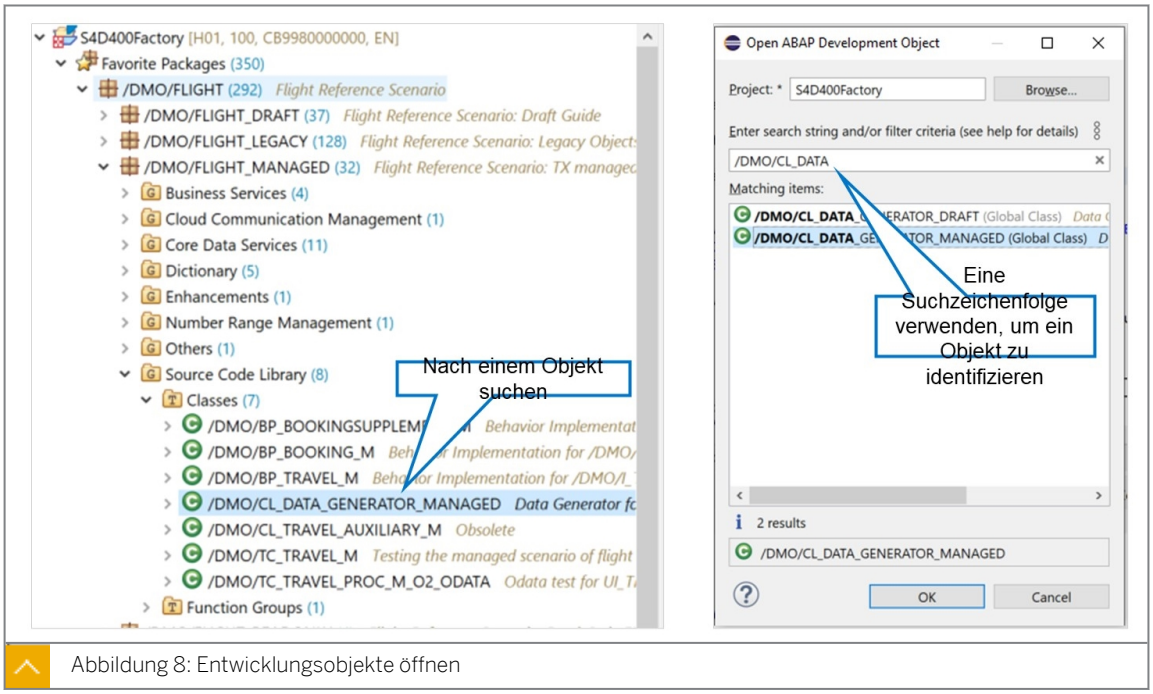


Abbildung 8: Entwicklungsobjekte öffnen

Bei der ersten Möglichkeit suchen Sie den Objektnamen im Project Explorer und doppelklicken darauf. Die andere Möglichkeit besteht darin, die Tastenkombination „Strg + Umschalt + A“ zu verwenden. Daraufhin wird ein Dialogfenster geöffnet, in dem Sie einen Teil des Namens eines Objekts eingeben können.

Ein Entwicklungsobjekt untersuchen

Wenn Sie mit ABAP-Code arbeiten, stehen Ihnen in ADT bestimmte Funktionstasten zur Verfügung.

Schauen wir uns diese Funktionstasten an.

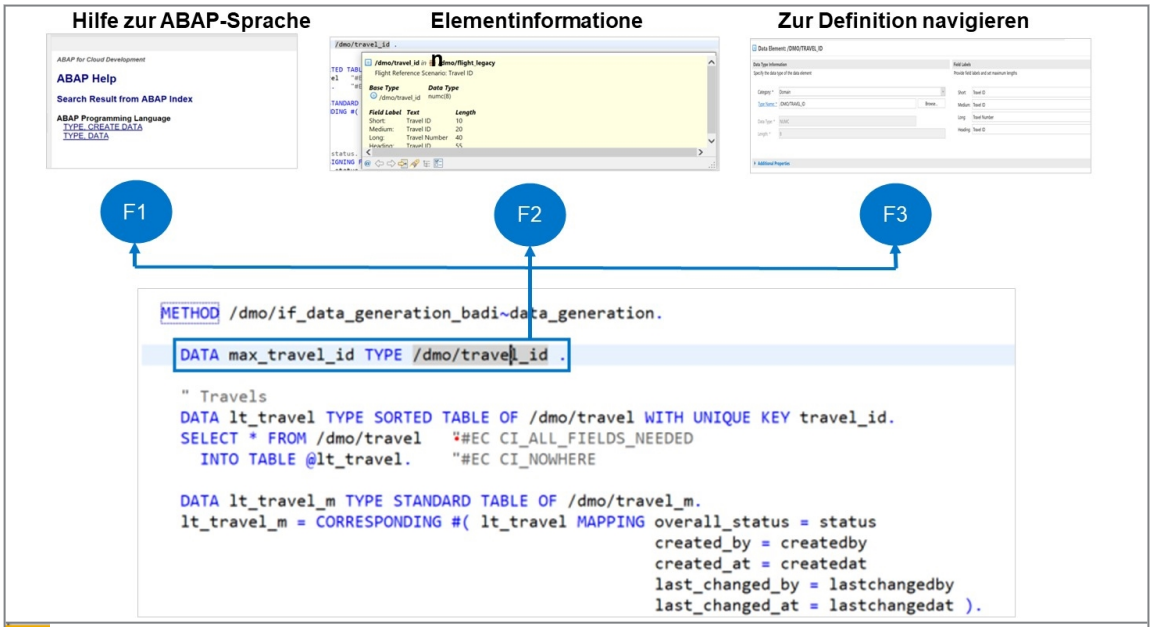


Abbildung 9: Ein Entwicklungsobjekt untersuchen

F1

© Copyright. Alle Rechte vorbehalten.

Die F1-Taste zeigt die ABAP-Sprachhilfe für die aktuelle Anweisung an. Eine Anweisung ist der Name für einen Befehl in ABAP.

F2

Die F2-Taste zeigt Informationen über das Element an, auf dem der Cursor steht.

F3

Die F3-Taste navigiert zur Definition des Objekts, auf dem der Cursor steht. Von dort aus können Sie mit der Tastenkombination `ALT + PFEIL NACH LINKS` auf Ihrer Tastatur zum ursprünglichen Objekt zurückkehren.

© Copyright. Alle Rechte vorbehalten.

Kapitel 1
Übung 1

Ein ABAP-Cloud-Projekt anlegen und das ABAP-Coding untersuchen

In dieser Übung starten Sie Ihr ABAP-Cloud-Projekt und analysieren vorhandenen ABAP-Code, um sich mit einigen grundlegenden Funktionen der ABAP-Entwicklungsumgebung vertraut zu machen.

Aufgabe 1: Ein ABAP-Cloud-Projekt anlegen
Öffnen Sie Eclipse, wechseln Sie zur ABAP-Perspektive, und legen Sie ein ABAP-Cloud-Projekt an.

1. Öffnen Sie Eclipse, und wechseln Sie zur ABAP-Perspektive.
2. Legen Sie ein *ABAP-Cloud-Projekt* an.

Aufgabe 2: Die ABAP-Klasse /DMO/CL_FLIGHT_LEGACY analysieren
Verwenden Sie einige Funktionen der *ABAP Development Tools*, um den Quelltext der ABAP-Klasse **`/DMO/CL_FLIGHT_LEGACY`**zu analysieren.

> Notiz:
> Sie müssen den Quelltext in dieser Übung nicht lesen. Konzentrieren Sie sich stattdessen auf die Navigations- und Anzeigefunktionen in der Entwicklungsumgebung.

1. Fügen Sie das Paket **`/DMO/FLIGHT`** zu Ihren *Favoritenpaketen* hinzu.
2. Öffnen Sie die ABAP-Klasse **`/DMO/CL_FLIGHT_LEGACY`** in der Editor-View.
3. Öffnen Sie die Registerkarte *Properties*, um die Verwaltungsdaten des Entwicklungsobjekts **`/DMO/CL_FLIGHT_LEGACY`** anzuzeigen.
4. Suchen Sie die ABAP-Klasse **`/DMO/CL_FLIGHT_LEGACY`** in der View *Project Explorer* auf der linken Seite der Anzeige.
5. Suchen Sie die Zeichenfolge **`get_instance( )->get`** im Quelltext der ABAP-Klasse **`/DMO/CL_FLIGHT_LEGACY`**.

   > Notiz:
   > Zwischen den Klammern muss genau ein Leerzeichen stehen.

6. Navigieren Sie in der aktuellen Quelltextzeile zur Implementierung des Quelltextelements `get`.

© Copyright. Alle Rechte vorbehalten.

7. Einige Codezeilen weiter unten befindet sich eine Zeile, die mit `SELECT` beginnt. Zeigen Sie einige Informationen zum Code-Element `/dmo/travel` nach dem Schlüsselwort `FROM` an, ohne von dort weg zu navigieren.
8. Zeigen Sie die ABAP-Sprachhilfe für das Schlüsselwort `SELECT` an.

© Copyright. Alle Rechte vorbehalten.

Kapitel 1
Lösung 1

Ein ABAP-Cloud-Projekt anlegen und das ABAP-Coding untersuchen

In dieser Übung starten Sie Ihr ABAP-Cloud-Projekt und analysieren vorhandenen ABAP-Code, um sich mit einigen grundlegenden Funktionen der ABAP-Entwicklungsumgebung vertraut zu machen.

Aufgabe 1: Ein ABAP-Cloud-Projekt anlegen
Öffnen Sie Eclipse, wechseln Sie zur ABAP-Perspektive, und legen Sie ein ABAP-Cloud-Projekt an.

1. Öffnen Sie Eclipse, und wechseln Sie zur ABAP-Perspektive.
   a) Öffnen Sie Eclipse.
   b) Wählen Sie *Window* → *Perspective* → *Open Perspective* → *Other...*
   c) Doppelklicken Sie im Dialogfenster auf *ABAP*.
2. Legen Sie ein *ABAP-Cloud-Projekt* an.
   a) Wählen Sie *File* → *New* → *ABAP Cloud Project*.
   b) Gehen Sie im daraufhin angezeigten Dialogfenster *Neues ABAP-Cloud-Projekt* wie folgt vor:
      Wenn Ihnen die *ABAP-Serviceinstanz-URL* übergeben wurde:
      i. Kopieren Sie diese URL, und fügen Sie sie in das entsprechende Feld im Dialogfenster ein.
      ii. Wählen Sie *Next*.
      Wenn Sie eine Datei erhalten haben, die den *Serviceschlüssel* enthält:
      i. Wählen Sie im Dialogfeld den Link *Extrahieren*.
      ii. Wählen Sie *Importieren...*, und wählen Sie die Datei mit dem *Serviceschlüssel* aus.
      iii. Klicken Sie nach dem Import auf *In Zwischenablage* kopieren, um die *Serviceinstanz-URL* zu kopieren.
      iv. Wählen Sie *Schließen*.
      v. Fügen Sie oben im Dialogfenster im Feld *ABAP-Serviceinstanz-URL:** die *ABAP-Serviceinstanz-URL* mit der Tastenkombination *Strg* + *V* ein.
      vi. Wählen Sie *Next*.
   c) Wählen Sie *Open Logon Page in Browser*.
   d) Sie werden aufgefordert, sich am System anzumelden.

© Copyright. Alle Rechte vorbehalten.

Geben Sie den Benutzer und das Kennwort ein, die Sie von Ihrem Referenten erhalten haben.

e) Wenn die Meldung *You have been successfully logged on* angezeigt wird, schließen Sie das Browserfenster, und kehren Sie zu Eclipse zurück.

f) Um das Anlegen des Projekts abzuschließen, wählen Sie *Finish*.

Aufgabe 2: Die ABAP-Klasse /DMO/CL_FLIGHT_LEGACY analysieren
Verwenden Sie einige Funktionen der *ABAP Development Tools*, um den Quelltext der ABAP-Klasse **`/DMO/CL_FLIGHT_LEGACY`**zu analysieren.

Notiz:
Sie müssen den Quelltext in dieser Übung nicht lesen. Konzentrieren Sie sich stattdessen auf die Navigations- und Anzeigefunktionen in der Entwicklungsumgebung.

1. Fügen Sie das Paket **`/DMO/FLIGHT`** zu Ihren *Favoritenpaketen* hinzu.
   a) Klicken Sie in der Ansicht *Project Explorer* mit der rechten Maustaste auf den Ordner *Favorite Packages* unter Ihrem Projektordner.
   b) Wählen Sie im Kontextmenü *Paket hinzufügen...*.
   c) Geben Sie im Eingabefeld */***`DMO/FLIGHT`** als Suchstring ein.
   d) Klicken Sie in der Liste der Matching-Positionen auf den Eintrag */DMO/FLIGHT*, und wählen Sie *OK*.
2. Öffnen Sie die ABAP-Klasse **`/DMO/CL_FLIGHT_LEGACY`** in der Editor-View.
   a) Wählen Sie im Eclipse-Menü *Navigate* → *Open ABAP Development Object...*oder drücken Sie **`Strg + Umschalt + A`**.
   b) Geben Sie im Eingabefeld als Suchzeichenfolge **`/DMO/CL_FLIGHT`** ein.
   c) Klicken Sie in der Liste der Matching-Positionen auf */DMO/CL_FLIGHT_LEGACY (Klasse)*, und wählen Sie *OK*.
3. Öffnen Sie die Registerkarte *Properties*, um die Verwaltungsdaten des Entwicklungsobjekts **`/DMO/CL_FLIGHT_LEGACY`** anzuzeigen.
   a) Positionieren Sie den Cursor an einer beliebigen Stelle im Quelltext der Klasse **`/DMO/CL_FLIGHT_LEGACY`**.
   b) Navigieren Sie auf der Registerkarte unter der Editorsicht zur Registerkarte *"Eigenschaften"*.
   c) Analysieren Sie die dort angezeigten Verwaltungsdaten, z.B. die Originalsprache oder den Zeitstempel der letzten Änderung.
4. Suchen Sie die ABAP-Klasse **`/DMO/CL_FLIGHT_LEGACY`** in der View *Project Explorer* auf der linken Seite der Anzeige.
   a) Positionieren Sie den Cursor an einer beliebigen Stelle im Quelltext der Klasse **`/DMO/CL_FLIGHT_LEGACY`**.

© Copyright. Alle Rechte vorbehalten.

b) Wählen Sie in der Symbolleiste des *Projekt-ExplorersLink with Editor*. Dadurch sollte ein Teil des Baums unter Ihrem ABAP-Projekt mit der ABAP-Klasse **`/DMO/CL_FLIGHT_LEGACY`** als Endpunkt expandiert werden.

5. Suchen Sie die Zeichenfolge **`get_instance( )->get`** im Quelltext der ABAP-Klasse **`/DMO/CL_FLIGHT_LEGACY`**.

> Notiz:
> Zwischen den Klammern muss genau ein Leerzeichen stehen.

a) Positionieren Sie den Cursor in der ersten Zeile des Quelltexts der Klasse **`/DMO/CL_FLIGHT_LEGACY`**.

b) Drücken Sie **`Strg + F`**, um das Dialogfenster *Find/Replace* zu öffnen.

c) Geben Sie im daraufhin angezeigten Eingabefeld *Find***`get_instance( )->get`** ein.

6. Navigieren Sie in der aktuellen Quelltextzeile zur Implementierung des Quelltextelements `get`.

a) Drücken Sie die *Strg* -Taste, und klicken Sie auf *"Abrufen"*. Wählen Sie im angezeigten Menü *Zur Implementierung navigieren*. Alternativ können Sie den Cursor in das Wort *get* setzen und **`F3`** drücken.

7. Einige Codezeilen weiter unten befindet sich eine Zeile, die mit `SELECT` beginnt. Zeigen Sie einige Informationen zum Code-Element `/dmo/travel` nach dem Schlüsselwort `FROM` an, ohne von dort weg zu navigieren.

a) Positionieren Sie in der Quelltextzeile, die mit `SELECT` beginnt, den Cursor auf `/dmo/travel`, und drücken Sie **`F2`**.

8. Zeigen Sie die ABAP-Sprachhilfe für das Schlüsselwort `SELECT` an.

a) Positionieren Sie den Cursor am Anfang der aktuellen Quelltextzeile auf `SELECT`, und drücken Sie **`F1`**.

> Hinweis:
> Wenn die geöffnete Sicht ABAP-Sprachhilfe anzeigt, dass die Ergebnismenge der Suchanfrage in der aktuellen ABAP-Sprachversion nicht unterstützt wird, geben Sie **`SELECT`** in das *Suchbegrifffeld* der Sicht ABAP-Sprachhilfe ein, und drücken *Sie die Eingabetaste*. Diese Suche sollte die gewünschte Hilfe liefern.

> Notiz:
> An dieser Stelle ist es nicht erforderlich, die Dokumentation zu lesen oder zu verstehen.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Arbeiten Sie mit einem Entwicklungsobjekt.

© Copyright. Alle Rechte vorbehalten.

Die Softwarestruktur und -logistik verstehen

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Legen Sie ein ABAP-Paket an.

ABAP-Paket

Die Entwicklung organisieren

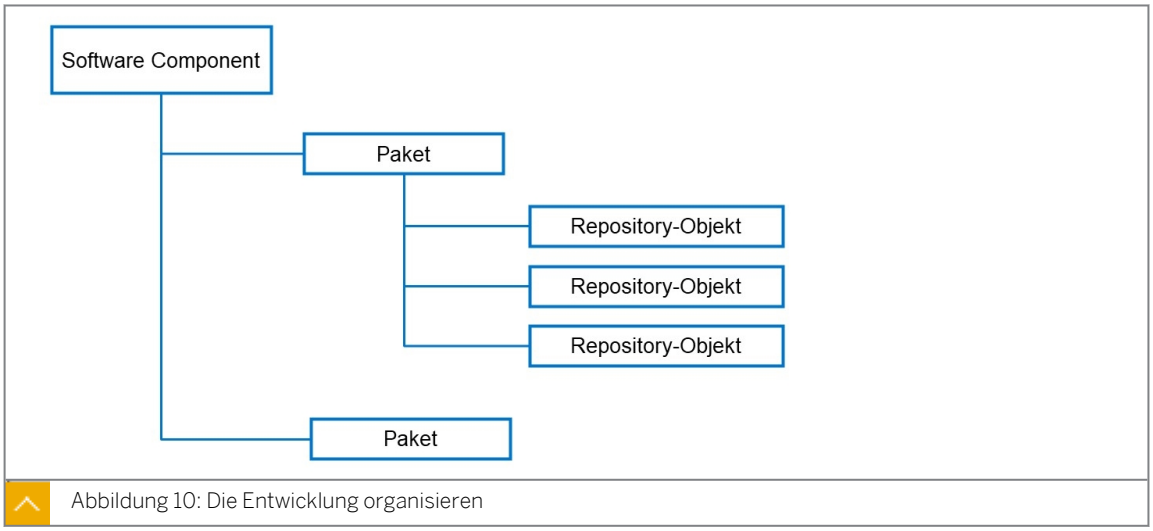


Abbildung 10: Die Entwicklung organisieren

Wenn Sie ein Entwicklungsobjekt in der ABAP-Umgebung anlegen, müssen Sie es einem Paket zuordnen. Pakete sind Entwicklungsobjekte, die als Container für alle Entwicklungsobjekte dienen, die logisch zusammengehören. Jedes Paket ist zudem einer Softwarekomponente zugeordnet. Der komplette Satz an Entwicklungsobjekten im System wird als ABAP Repository bezeichnet. Daher werden Entwicklungsobjekte oft auch Repository-Objekte genannt.

© Copyright. Alle Rechte vorbehalten.

Notiz:
Entwicklungsobjekte wie Pakete werden über Namensräume organisiert. Namensräume gewährleisten eine klare Trennung zwischen SAP-Standardobjekten und Kunden- oder Partnerentwicklungen. Kundenentwicklungen müssen im Kundennamensraum angelegt werden, der mit Z oder Y beginnt. Das bedeutet, dass der erste Buchstabe des Namens eines Entwicklungsobjekts, z.B. ein Paketname, Z oder Y sein muss. Kunden und Partner können auch eigene Namensräume bei SAP anfordern. Den Namen der Entwicklungsobjekte wird dann /<Namensraum>/ vorangestellt. Das Namensraumkonzept verhindert Namenskonflikte und schützt Kundencode bei Upgrades und Updates.

Sie entwickeln Ihre Anwendungen in einer Entwicklungsumgebung, müssen dann aber sicherstellen, dass sie in einer geeigneten Testumgebung getestet werden können, bevor sie in die Produktivumgebung verschoben werden. In der Regel verfügen Sie über ein einzelnes globales Konto sowie ein Unterkonto für jede Entwicklungs-, Test- und Produktivumgebung. Mit Softwarekomponenten können Sie Ihre Objekte transportieren.

Transportauftrag

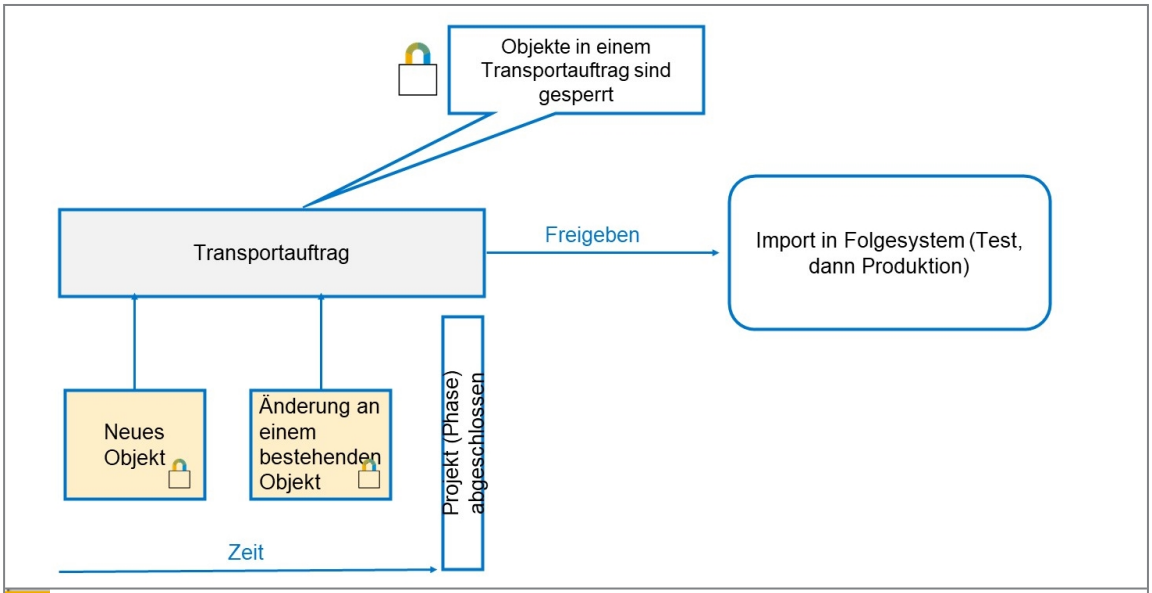


Abbildung 11: Transportauftrag

Wenn Sie ein neues Entwicklungsobjekt anlegen oder ein vorhandenes ändern, müssen Sie es einem Transportauftrag zuordnen. Transportaufträge stellen sicher, dass alle logisch zusammengehörenden Entwicklungsobjekte zusammen in das Test- und anschließend in das Produktivsystem transportiert werden.

Jeder Transportauftrag hat einen Eigentümer, und der Eigentümer kann dem Auftrag andere Benutzer zuordnen. Auf diese Weise unterstützen Transportaufträge die Entwicklung im Team.

Wenn ein Objekt in einen Transportauftrag aufgenommen wird, wird es gesperrt. Das bedeutet, dass es nur von einem Benutzer bearbeitet werden kann, der demselben Auftrag zugeordnet ist.

© Copyright. Alle Rechte vorbehalten.

Wenn die Arbeit an allen Objekten im Auftrag abgeschlossen ist, müssen alle ihm zugeordneten Entwickler ihre Arbeit freigeben. Danach kann der Eigentümer des Auftrags den gesamten Auftrag freigeben. Der Systemadministrator kann ihn dann zu Testzwecken in das Testsystem importieren.

Wenn Sie eine beliebige Art von Transportauftrag freigeben, gibt das System die Sperren für die Objekte im Auftrag frei, sodass jeder Entwickler wieder auf sie zugreifen kann.

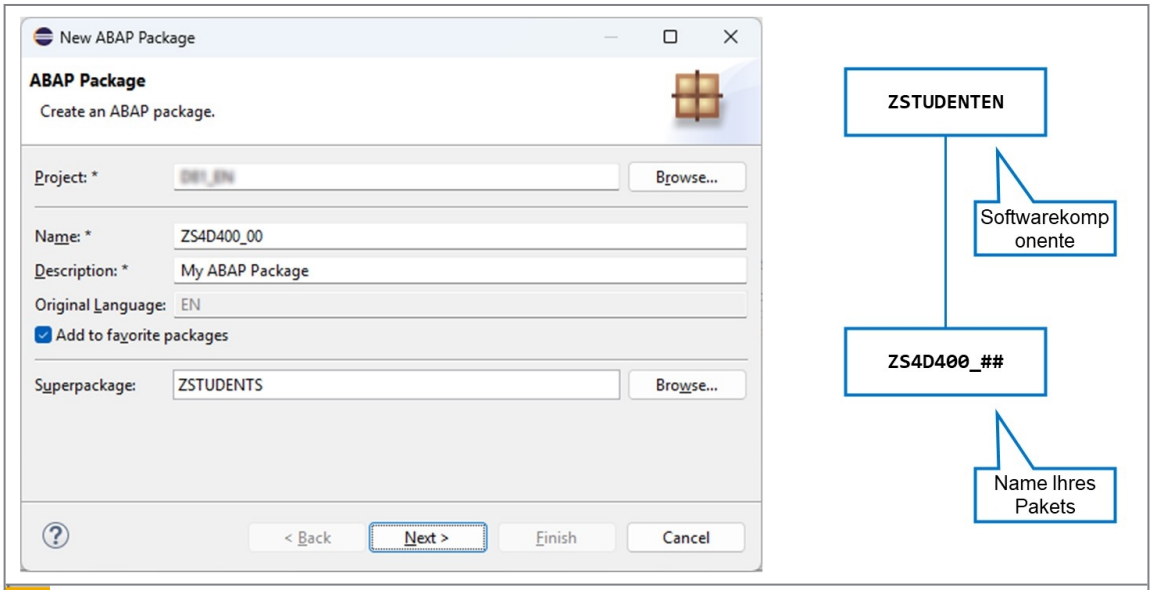


Abbildung 12: Paket anlegen

Um ein neues Paket anzulegen, wählen Sie *File* → *New* → *ABAP Package*. Geben Sie den Namen des Pakets ein, der mit dem Buchstaben Z oder Y beginnen muss. Dies ist eine Voraussetzung für die gesamte ABAP-Entwicklung im Kundennamensraum und dient dazu, Namenskonflikte zu vermeiden, da von SAP ausgelieferte Objekte nie Z oder Y als erstes Zeichen des Namens verwenden.

Geben Sie eine Beschreibung für das Paket und ein Oberpaket ein. In dieser Klasse verwenden wir ZSTUDENTS als Oberpaket. ZSTUDENTS ist ein generiertes Paket, das zur gleichnamigen Softwarekomponente gehört. Das heißt, indem Sie ZSTUDENTS als Oberpaket angeben, ordnen Sie der Softwarekomponente ZSTUDENTS ein Paket zu.

© Copyright. Alle Rechte vorbehalten.

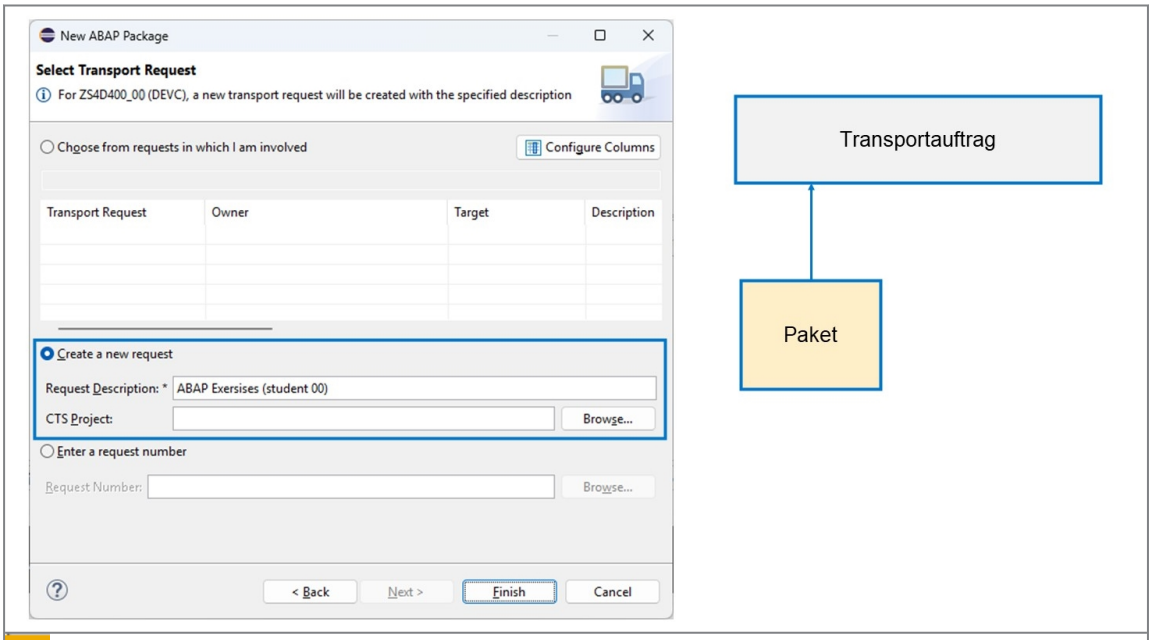


Abbildung 13: Paket einem Transportauftrag zuordnen

Sie müssen Ihr Paket einem Transportauftrag zuordnen. Da Sie noch nicht mit Transportaufträgen gearbeitet haben, wählen Sie *Create a new request*. Geben Sie eine Beschreibung für den Transportauftrag ein, und wählen Sie *Finish*.

Sie sind nun bereit, mit der Entwicklung zu beginnen. Sie ordnen alle weiteren Objekte, die Sie anlegen, Ihrem Paket und demselben Transportauftrag zu.

© Copyright. Alle Rechte vorbehalten.

Ein ABAP-Paket anlegen

In dieser Übung fügen Sie zunächst das Paket **`/LRN/S4D400_EXERCISE`** zur Liste der *Favoritenpakete* hinzu. Anschließend legen Sie ein neues Paket unter dem Oberpaket **`ZSTUDENTS`** an (Namensvorschlag: **`ZS4D400_##`**, wobei ## für Ihre Gruppennummer steht).

Notiz:
Dieses Paket ist die Startseite für alle Entwicklungsobjekte, die Sie in diesem Kurs anlegen.

Ihr neues Paket sollte die folgenden Attribute haben:

Attribut	Wert
Name	**`ZS4D400_##`**, wobei **`##`** für Ihre Gruppennummer steht
Description	**`Mein ABAP-Paket`**
Add to favorite packages	Markiert
Superpackage	**`ZSTUDENTEN`**

1. Fügen Sie das Paket **`/LRN/S4D400_EXERCISE`** zur Liste der *Favoritenpakete* hinzu.
2. Legen Sie in Ihrem ABAP-Cloud-Projekt ein neues Paket mit den oben aufgeführten Attributen an. Wenn Sie einen Transport angeben sollen, wählen Sie den Transportauftrag aus, an dem Sie beteiligt sind. Wenn kein Transportauftrag aufgeführt ist, legen Sie einen neuen Auftrag an.
3. Stellen Sie sicher, dass Ihr neues Paket zu den *Favoritenpaketen* hinzugefügt wurde.

© Copyright. Alle Rechte vorbehalten.

Kapitel 1
Lösung 2

Ein ABAP-Paket anlegen

In dieser Übung fügen Sie zunächst das Paket **/LRN/S4D400_EXERCISE** zur Liste der *Favoritenpakete* hinzu. Anschließend legen Sie ein neues Paket unter dem Oberpaket **ZSTUDENTS** an (Namensvorschlag: **ZS4D400_##**, wobei ## für Ihre Gruppennummer steht).

Notiz:
Dieses Paket ist die Startseite für alle Entwicklungsobjekte, die Sie in diesem Kurs anlegen.

Ihr neues Paket sollte die folgenden Attribute haben:

Attribut	Wert
Name	**ZS4D400_##**, wobei **##** für Ihre Gruppennummer steht
Description	**Mein ABAP-Paket**
Add to favorite packages	Markiert
Superpackage	**ZSTUDENTEN**

1. Fügen Sie das Paket **/LRN/S4D400_EXERCISE** zur Liste der *Favoritenpakete* hinzu.
   a) Erweitern Sie im *Projekt-Explorer* auf der linken Seite Ihr ABAP-Cloud-Projekt.
   b) Klicken Sie mit der rechten Maustaste auf *Favorite Packages*, und wählen Sie *Add Package...*.
   c) Geben Sie im Suchfeld **/LRN/S4D400** ein.
   d) Wählen Sie aus der Liste der Matching-Positionen */LRN/S4D400_EXERCISE* aus, und wählen Sie dann *OK*.
2. Legen Sie in Ihrem ABAP-Cloud-Projekt ein neues Paket mit den oben aufgeführten Attributen an. Wenn Sie einen Transport angeben sollen, wählen Sie den Transportauftrag aus, an dem Sie beteiligt sind. Wenn kein Transportauftrag aufgeführt ist, legen Sie einen neuen Auftrag an.
   a) Klicken Sie im *Project Explorer* mit der rechten Maustaste auf Ihr ABAP-Cloud-Projekt, und wählen Sie *New* → *ABAP Package*.
   b) Geben Sie den Paketnamen **ZS4D400_##** ein, wobei **##** für Ihre Gruppennummer steht.

© Copyright. Alle Rechte vorbehalten.

c) Geben Sie als Beschreibung **`Mein ABAP-Paket`** ein.

d) Markieren Sie das Ankreuzfeld *Zu Favoritenpaketen hinzufügen*.

e) Geben Sie das Oberpaket **`ZSTUDENTS`** ein.

f) Wählen Sie *Next*.

g) Stellen Sie sicher, dass die *Softwarekomponente* auf **`ZSTUDENTS`** gesetzt ist.

h) Wählen Sie *Next*.

i) Prüfen Sie, ob unter der Option *Aus Aufträgen auswählen, an denen ich beteiligt bin* ein Transportauftrag aufgeführt ist. Wenn dies der Fall ist, wählen Sie diese Option. Wenn die Liste leer ist, wählen Sie die Option *Neuen Auftrag anlegen*, und geben Sie eine Auftragsbeschreibung ein, z.B. **`ABAP-Übungen`**.

j) Wählen Sie *Finish*.

3. Stellen Sie sicher, dass Ihr neues Paket zu den *Favoritenpaketen* hinzugefügt wurde.

a) Expandieren Sie im *Project Explorer* auf der linken Seite Ihr ABAP-Cloud-Projekt und dann den Unterknoten *Favorite Packages*.

b) Stellen Sie sicher, dass Ihr eigenes Paket **`ZS4D400_##`** hier aufgeführt ist. Wenn nicht, fügen Sie es so hinzu, als hätten Sie zuvor das Paket **`/LRN/S4D400_EXERCISE`** hinzugefügt.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Legen Sie ein ABAP-Paket an.

© Copyright. Alle Rechte vorbehalten.

Erste ABAP-Anwendung entwickeln

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Legen Sie eine Anwendung „Hello World“ an.

„Hello World“-App in ABAP

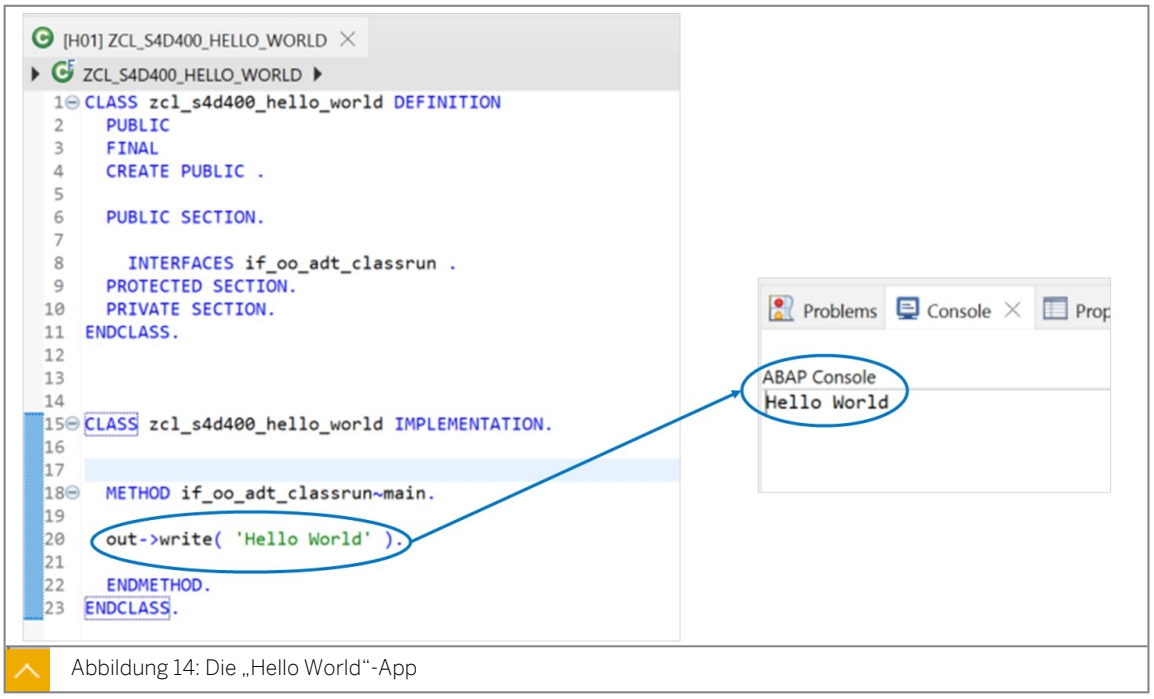


Abbildung 14: Die „Hello World“-App

Wie bei allen anderen Programmiersprachen sollten Sie sich auch in ABAP zunächst mit der Entwicklungsumgebung und den grundlegenden Aspekten der Sprache vertraut machen, indem Sie eine kurze „Hello World”-App schreiben.

Die wichtigste Benutzungsoberflächentechnologie, die Sie in der modernen ABAP-Programmierung verwenden werden, ist SAP Fiori Elements. ADT bietet jedoch eine Konsole, mit der Sie schnell und einfach Ausgaben in Testanwendungen erstellen können.

© Copyright. Alle Rechte vorbehalten.

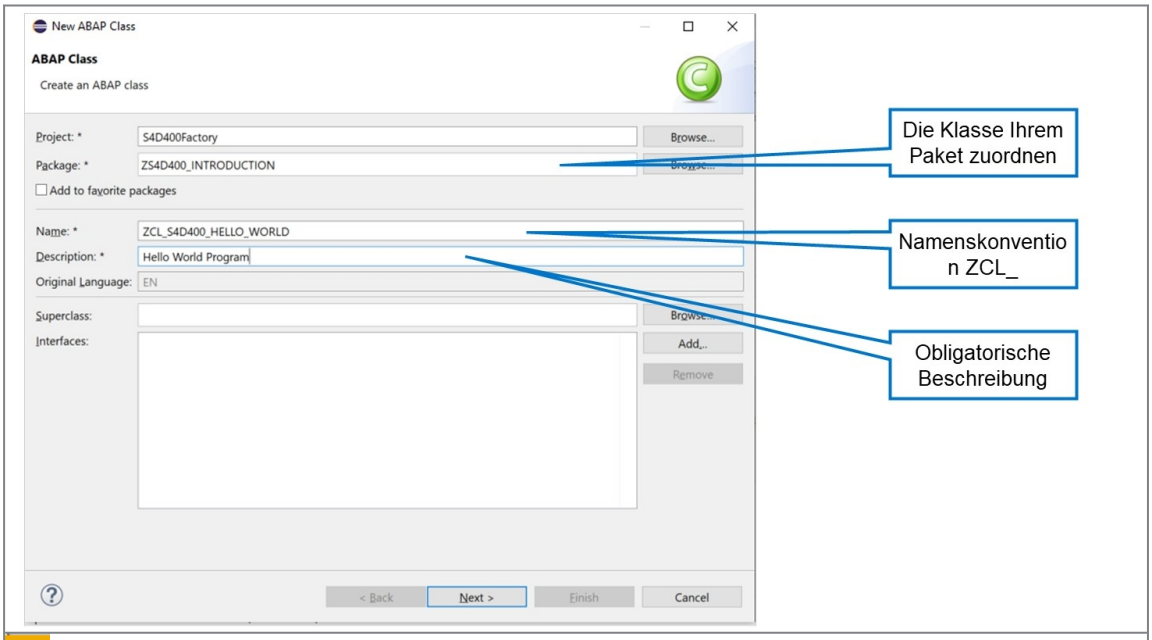


Abbildung 15: Klasse anlegen

Sie schreiben Ihren ABAP-Code in einer Klasse. Um eine Klasse anzulegen, wählen Sie *File → New → ABAP Class*. Ein Dialogfenster wird angezeigt. Stellen Sie sicher, dass das Projekt korrekt ist, und geben Sie den Namen des Pakets ein, das Sie bereits angelegt haben.

Als Nächstes müssen Sie Ihre Klasse gemäß den ABAP-Namenskonventionen benennen. Der Name der Klasse muss mit Z oder Y beginnen, um Namenskollisionen mit SAP-eigenen Objekten zu vermeiden. Namenskonventionen erfordern auch, dass dem Namen der Klasse für class das Präfix *CL* vorangestellt wird.

Geben Sie nach dem Präfix *ZCL* den tatsächlichen Namen der Klasse ein. Dieser sollte möglichst leicht verständlich sein. Der Name der Klasse (einschließlich Präfix) darf bis zu 30 Zeichen lang sein und aus den Buchstaben A-Z, den Ziffern 0-9 und dem Unterstrichsymbol _ bestehen. Mit dem Unterstrich trennen Sie die einzelnen Wörter, die Sie im Namen der Klasse verwenden.

© Copyright. Alle Rechte vorbehalten.

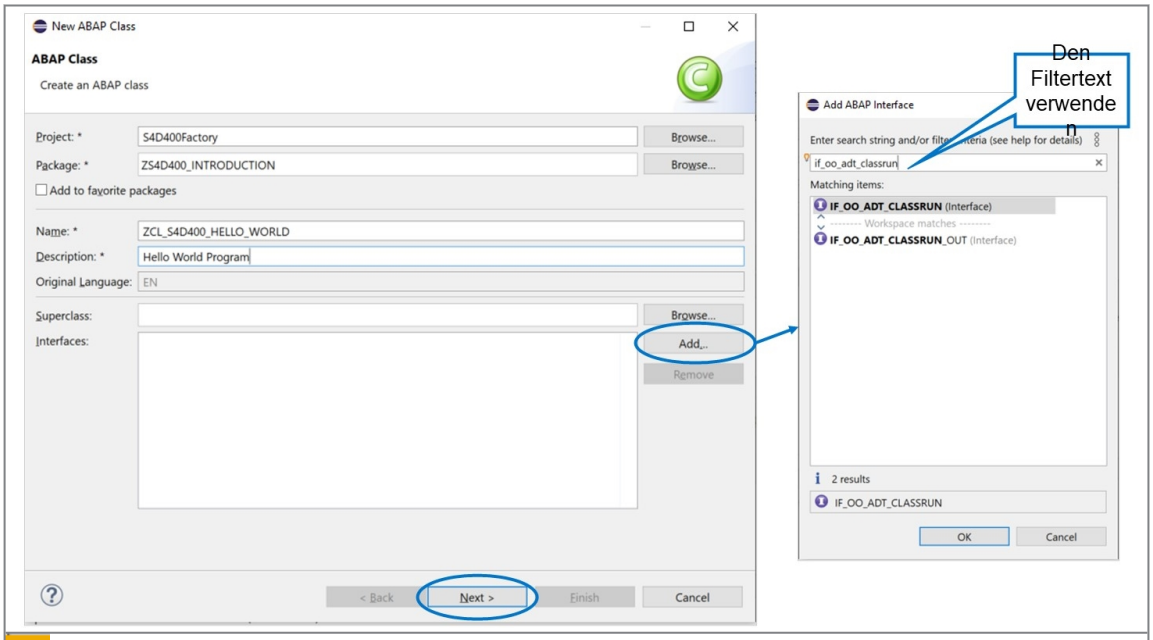


Abbildung 16: Hinzufügen des Interface IF_OO_ADT_CLASSRUN

Nachdem Sie den Namen Ihrer Klasse eingegeben haben, wählen Sie *Hinzufügen...*, um der Klasse ein Interface hinzuzufügen. Interfaces stellen sicher, dass ein Benutzer die Klasse in besonderer Weise verwenden kann. Das Interface IF_OO_ADT_CLASSRUN stellt sicher, dass die Entwicklungsumgebung die Klasse ausführen und die Ausgabe in der Eclipse-Konsole anzeigen kann. Verwenden Sie im Dialogfenster *ABAP-Schnittstelle hinzufügen* das Filterfeld, um die Anzahl der Einträge in der Trefferliste einzuschränken. Wenn das Interface IF_OO_ADT_CLASSRUN angezeigt wird, doppelklicken Sie darauf. Wenn Sie zum Dialogfenster *Neue ABAP-Klasse* zurückkehren, wird das Interface in der Liste der Interfaces angezeigt.

Wählen Sie *Next*.

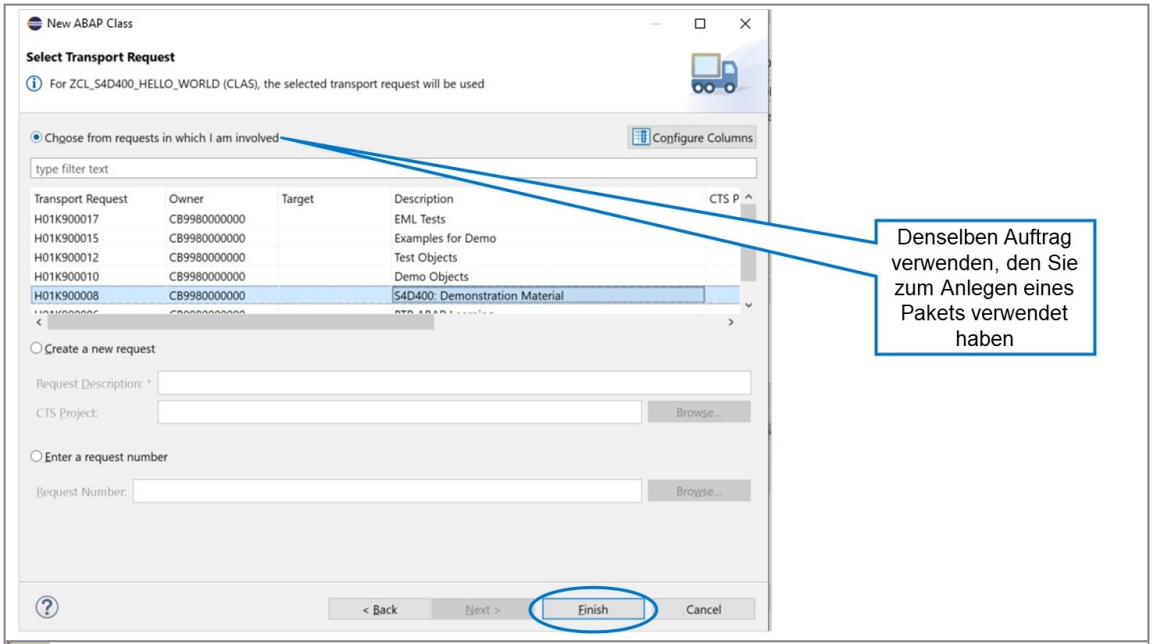


Abbildung 17: Klasse einem Transportauftrag zuordnen

© Copyright. Alle Rechte vorbehalten.

Auf dem letzten Bild haben Sie die Klasse einem Paket zugeordnet. Nun müssen Sie sie einem Transportauftrag zuordnen. Markieren Sie unter *Choose from requests in which I am involved* den Auftrag, den Sie zum Anlegen Ihres Pakets verwendet haben, und wählen Sie *Finish*.

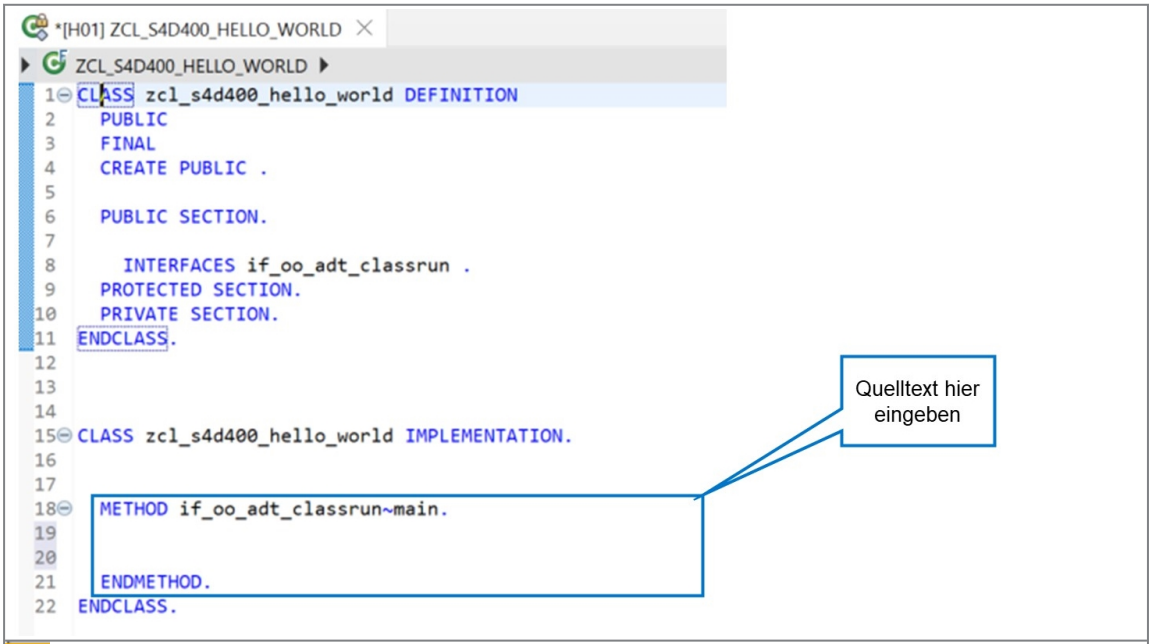


Abbildung 18: Klasse bearbeiten

Mit dem Interface IF_OO_ADT_CLASSRUN können Sie eine Klasse in ADT mit dem *F9-Schlüssel* ausführen. Dabei führt das System den Code zwischen `METHOD if_oo_adt_classrun~main` und `ENDMETHOD` aus. In diesem Quelltextblock können Sie Informationen in der ADT-Konsole ausgeben.

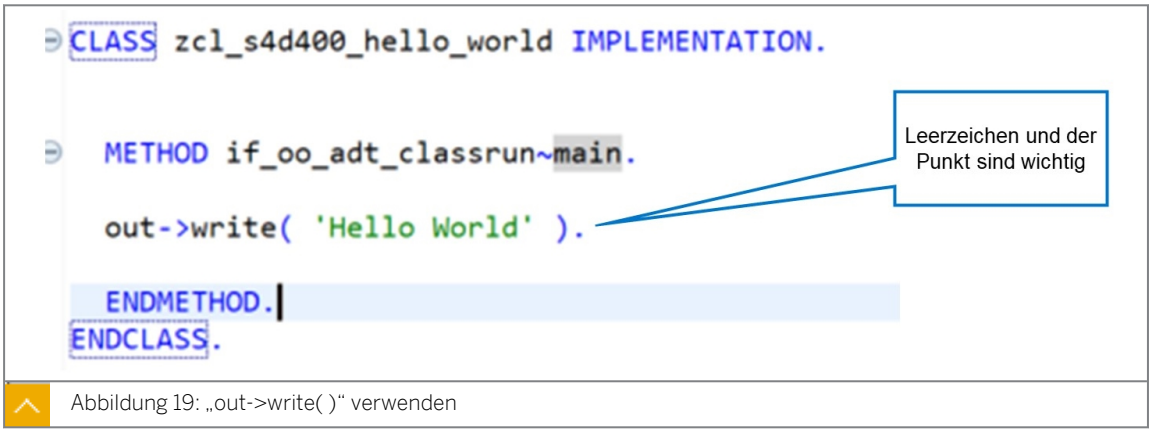


Abbildung 19: „out->write( )“ verwenden

In Ihrem Quelltextblock können Sie `out->write( )` verwenden, um Informationen in der Konsole anzuzeigen. Die Linie `out->write( 'Hello World' ).` druckt *Hello World* auf die Konsole. Entscheidend ist, dass Sie an dieser Stelle nicht wissen müssen, wie es funktioniert, Sie müssen nur den Code eingeben und Folgendes sicherstellen:

- Zwischen `write` und der öffnenden Klammer ist kein Leerzeichen vorhanden.
- Nach der öffnenden Klammer befindet sich mindestens ein Leerzeichen.
- Vor der schließenden Klammer befindet sich mindestens ein Leerzeichen.

© Copyright. Alle Rechte vorbehalten.

- Am Ende der Zeile befindet sich ein Punkt.

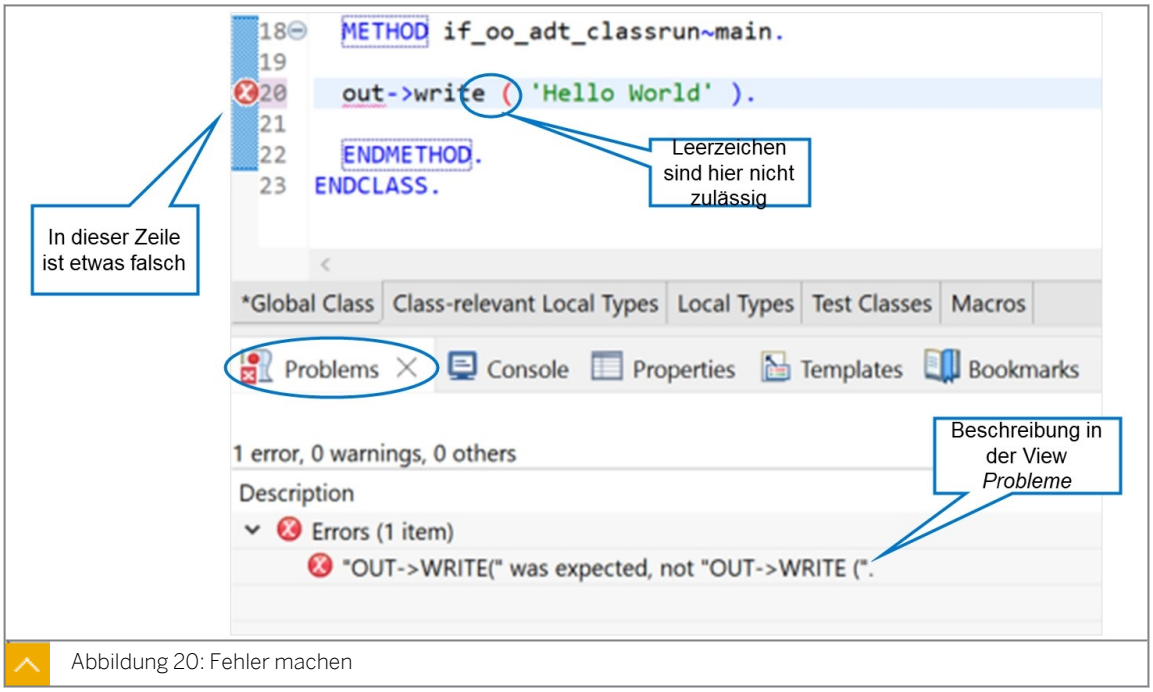


Abbildung 20: Fehler machen

Wenn Sie ABAP-Code schreiben, werden Sie unweigerlich Fehler machen. ADT prüft Ihren Code bei der Entstehung und kennzeichnet Fehler am linken Rand des Editors mit einem weißen Kreuz auf einem roten Hintergrund.

Die entsprechenden Fehlermeldungen werden in der View *Problems* unterhalb des Editors angezeigt. ADT zeigt dieselbe Meldung auch als Dialogfenster an, wenn Sie mit der Maus auf das Fehlersymbol im Editor zeigen.

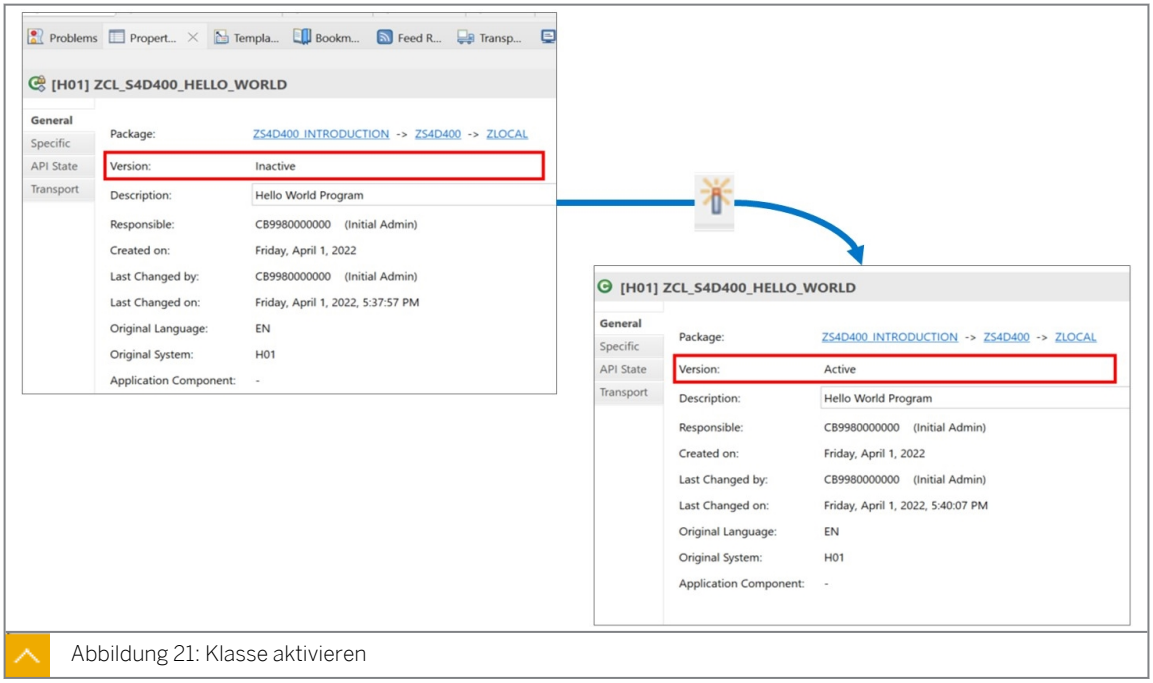


Abbildung 21: Klasse aktivieren

Um ein ABAP-Objekt auszuführen, müssen Sie es aktivieren. Wählen Sie dazu in der Symbolleiste das Symbol *Aktivieren*, oder verwenden Sie die Tastenkombination **`Strg + F3`**.

© Copyright. Alle Rechte vorbehalten.

Sie können sehen, ob ein Objekt aktiv ist oder nicht, indem Sie in der Sicht *Eigenschaften* suchen, die sich in der Regel auf der Registerkarte unterhalb des ABAP-Editors befindet.

Während der Aktivierung kompiliert das System das Objekt in eine Form, die das ABAP-Laufzeitsystem verstehen kann.

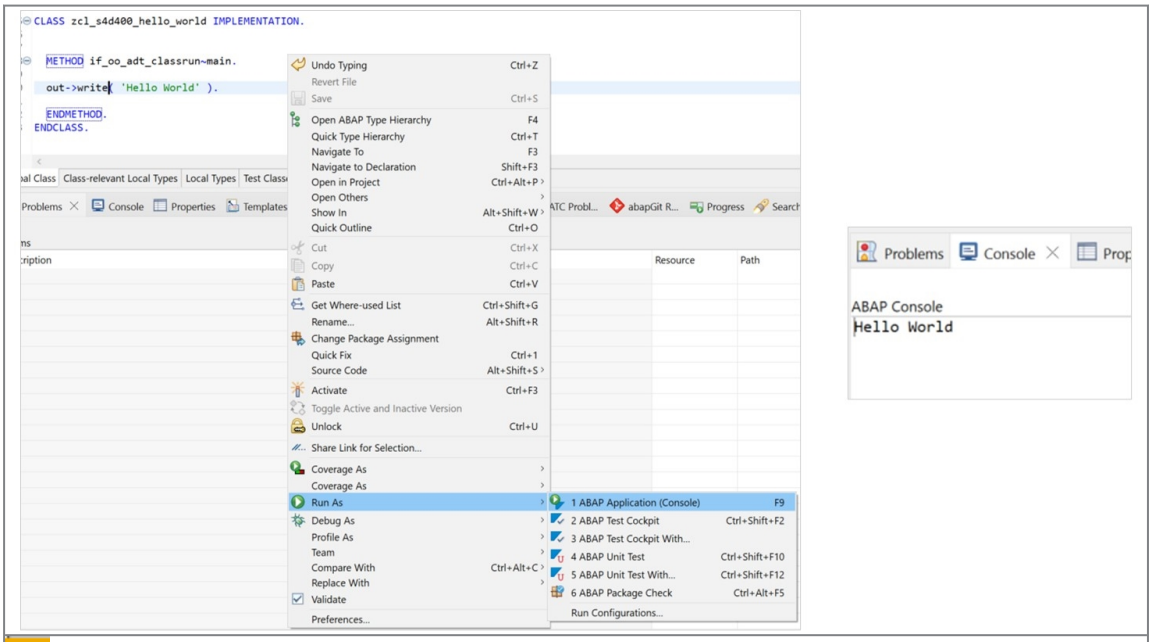


Abbildung 22: Klasse ausführen

Um die Klasse auszuführen, drücken Sie **F9**, oder klicken Sie mit der rechten Maustaste in den Editor, und wählen Sie *Run As → ABAP Application (Console)*. Die Ausgabe *Hello World* wird in der Konsole angezeigt. Wenn die Sicht *Console* nicht angezeigt wird, wählen Sie *Window → Show View → Other...*und wählen Sie die Sicht *Konsole*.

© Copyright. Alle Rechte vorbehalten.

Kapitel 1
Übung 3

Anwendung „Hello World“ erstellen

Anwendung „Hello World“ erstellen
Legen Sie in Ihrem Paket eine neue ABAP-Klasse an. Lassen Sie die Klasse das Interface **`IF_OO_ADT_CLASSRUN`** implementieren, damit Sie die Klasse als Hauptprogramm für eine Eclipse-Konsolen-App verwenden können.

1. Legen Sie in Ihrem Paket eine neue ABAP-Klasse mit dem Namen **`ZCL_##_HELLO_WORLD`** an. Stellen Sie sicher, dass sie das Interface **`IF_OO_ADT_CLASSRUN`** verwendet. Wenn Sie aufgefordert werden, die Klasse einem Transportauftrag zuzuordnen, verwenden Sie den Transportauftrag, den Sie in der vorherigen Aufgabe angelegt haben.
2. Verwenden Sie in der Methode **`if_oo_adt_classrun~main( )`** `out->write( )`, um die Phrase *Hello World* auszugeben.
3. Aktivieren und testen Sie die Klasse.
4. Überprüfen Sie die Ausgabe in der View *Console* von Eclipse.

© Copyright. Alle Rechte vorbehalten.

Kapitel 1
Lösung 3

Anwendung „Hello World“ erstellen

Anwendung „Hello World“ erstellen
Legen Sie in Ihrem Paket eine neue ABAP-Klasse an. Lassen Sie die Klasse das Interface **`IF_OO_ADT_CLASSRUN`** implementieren, damit Sie die Klasse als Hauptprogramm für eine Eclipse-Konsolen-App verwenden können.

1. Legen Sie in Ihrem Paket eine neue ABAP-Klasse mit dem Namen **`ZCL_##_HELLO_WORLD`** an. Stellen Sie sicher, dass sie das Interface **`IF_OO_ADT_CLASSRUN`** verwendet. Wenn Sie aufgefordert werden, die Klasse einem Transportauftrag zuzuordnen, verwenden Sie den Transportauftrag, den Sie in der vorherigen Aufgabe angelegt haben.
   a) Wählen Sie *File* → *New* → *ABAP Class*.
   b) Geben Sie Ihr Paket **`ZS4D400_##`** ein, wobei **`##`** für Ihre Gruppennummer steht.
   c) Geben Sie den Namen **`ZCL_##_HELLO_WORLD`** ein, wobei **`##`** für Ihre Gruppennummer steht, und geben Sie eine Beschreibung für Ihre Klasse ein.
   d) Wählen Sie *Hinzufügen...* (neben dem Gruppenrahmen *Schnittstellen*).
   e) Geben Sie den Filtertext **`IF_OO_ADT_CLASSRUN`** ein. Doppelklicken Sie auf den passenden Eintrag in der Trefferliste.
   f) Wählen Sie *Next*.
   g) Wählen Sie *Choose from requests in which I am involved* und Ihren eigenen Transportauftrag aus.
   h) Wählen Sie *Finish*.
2. Verwenden Sie in der Methode **`if_oo_adt_classrun~main( )`** `out->write( )`, um die Phrase *Hello World* auszugeben.
   a) Geben Sie im Editor folgenden Quelltext zwischen `METHOD if_oo_adt_classrun~main` und `ENDMETHOD.` ein:

```
out->write( 'Hello World' ).
```

3. Aktivieren und testen Sie die Klasse.
   a) Aktivieren Sie die Klasse mit der Tastenkombination **`Strg + F3`**.
   b) Führen Sie die Klasse mit der Taste **`F9`** aus.
4. Überprüfen Sie die Ausgabe in der View *Console* von Eclipse.
   a) Überprüfen Sie die View *Console*, die planmäßig als neue Registerkarte unterhalb der Editor-View geöffnet wird.
   b) Wenn die View *Console* nicht angezeigt wird, öffnen Sie sie, indem Sie *Window* → *Show View* → *Other* wählen. Doppelklicken Sie in der Trefferliste auf *Console*.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Legen Sie eine Anwendung „Hello World“ an.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 1
Lektion 5

ABAP-Entwicklung mit KI verstärken

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Erläutern Sie, wie ABAP-KI-Funktionen ABAP-Entwickler bei der Anwendungsentwicklung unterstützen können.
- Beschreiben Sie kurz, wie das ABAP AI SDK verwendet werden kann, um Geschäftsanwendungen KI-Funktionen hinzuzufügen.

Joule für Entwickler: Ihr KI-Assistent

Einführung

Stellen Sie sich vor, Sie sind ein neuer ABAP-Entwickler für ein Projekt. Sie müssen vorhandenen Code verstehen, initiale Versionen von ABAP-Code generieren oder neue Funktionen implementieren. Anstatt viel Zeit damit zu verbringen, Code manuell zu analysieren oder alles von Grund auf neu zu schreiben, können Sie ABAP-KI-Funktionen verwenden, um Ihre Entwicklung zu beschleunigen und die Qualität Ihres Codes von Anfang an zu verbessern.

Generative KI verändert die ABAP-Entwicklung durch die Bereitstellung von Tools, mit denen Entwickler produktiver arbeiten und intelligentere Geschäftsanwendungen erstellen können. In dieser Lektion erhalten Sie eine allgemeine Vorstellung davon, wie Sie diese KI-Funktionen in Ihren ABAP-Projekten verwenden können.

KI-Codierungshilfe – Hauptfunktionen von Joule

Joule for Developers ist eine Reihe KI-gestützter Werkzeuge, die in die ABAP-Entwicklungswerkzeuge in Eclipse integriert sind. Es wirkt wie Ihr Assistent und hilft Ihnen bei verschiedenen Entwicklungsaufgaben, um Sie effizienter zu machen.

© Copyright. Alle Rechte vorbehalten.

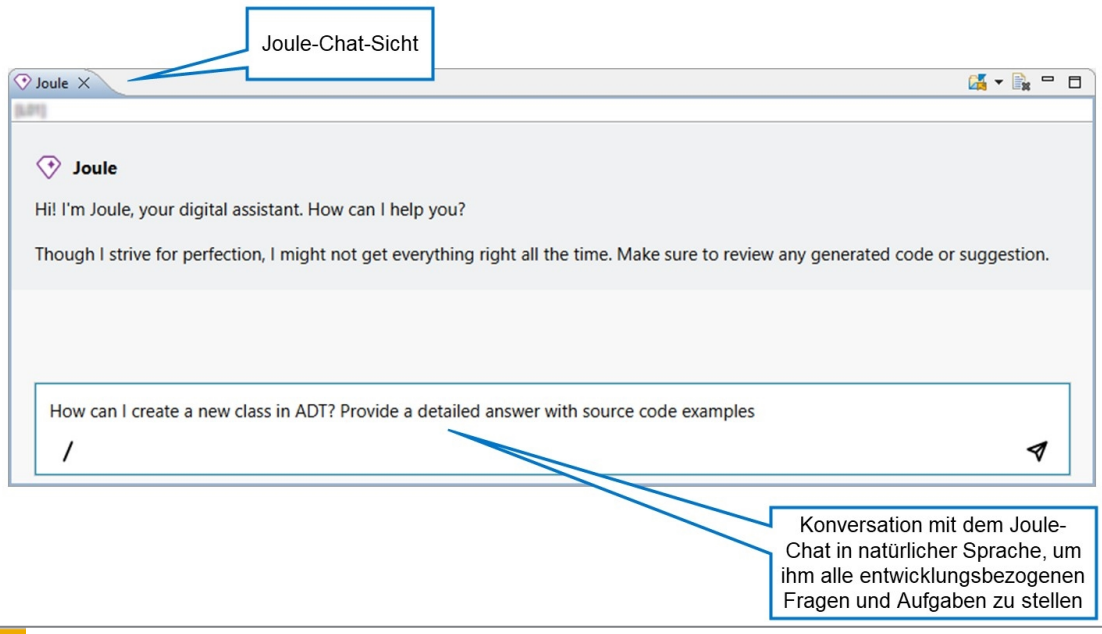


Abbildung 23: Joule-Chat

Sie können mit einer KI chatten, um Hilfe bei entwicklungsbezogenen Fragen zu erhalten. Stellen Sie Fragen, fordern Sie Erläuterungen an, oder generieren Sie Codebeispiele in einfacher Sprache. Um die Joule-Chat-Sicht zu öffnen, wählen Sie *Fenster → Sicht anzeigen → Andere... → Joule*

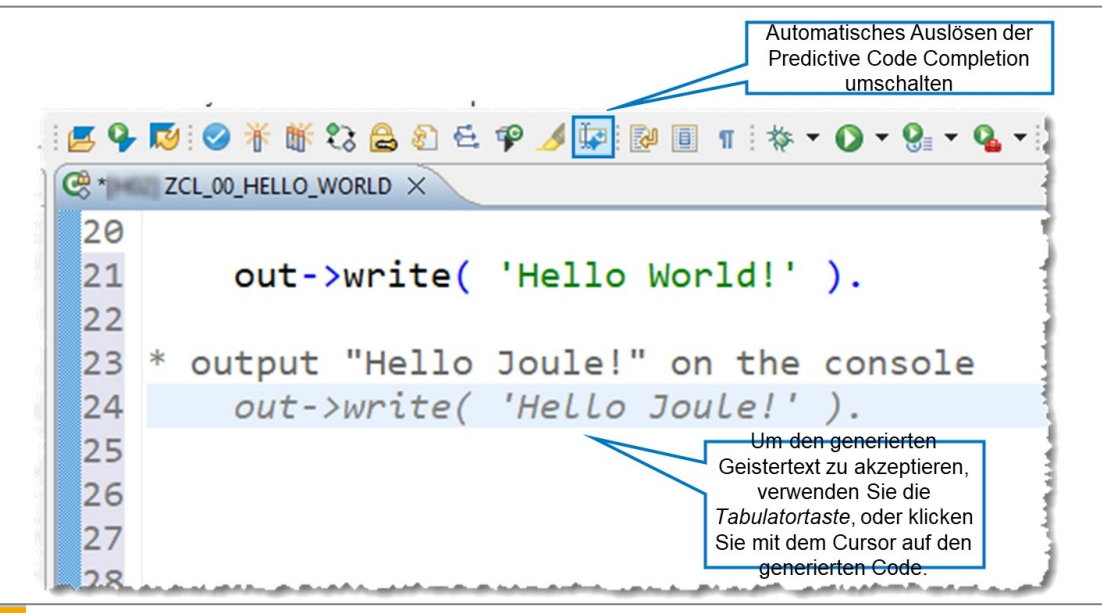


Abbildung 24: Predictive Code Completion

Sie können Geistertext während Ihres Schreibvorgangs in Ihren Quelltext einbinden. Geistertexte werden basierend auf Ihrer Cursorposition und dem Inhalt im ABAP-Quelltexteditor vorhergesagt. Die vorausschauende Code-Vervollständigung unterstützt derzeit die folgenden Objekttypen:

- Klassen
- Schnittstellen

© Copyright. Alle Rechte vorbehalten.

- Programme

Die vorausschauende Quelltextvervollständigung ist standardmäßig aktiviert, aber Sie können sie bei Bedarf manuell deaktivieren. Wenn diese Option aktiviert ist, wird Geistertext während Ihres Quelltextprozesses kontinuierlich angezeigt und im ABAP-Quelltexteditor grau angezeigt.

Wählen Sie in der Symbolleiste in ABAP Development Tools für Eclipse *die Option Automatisches Auslösen der Predictive Code Completion umschalten*, um die vorausschauende Quelltextvervollständigung zu aktivieren oder zu deaktivieren.

Beginnen Sie mit der Eingabe im Quelltexteditor und machen Sie eine kurze Pause. Wenn Predictive Code Completion aktiviert ist, wird Geistertext im ABAP-Quelltext-Editor an der Stelle angezeigt, an der Sie angehalten haben. Sie können sie dann in Ihren Code integrieren.

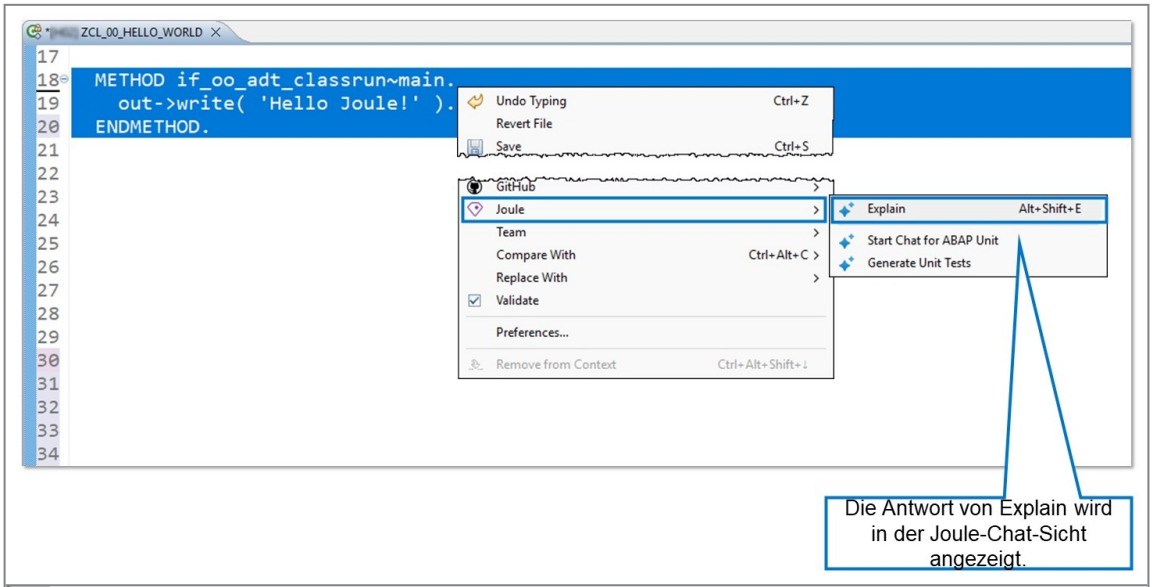


Abbildung 25: Funktion erläutern

Mit der *Explain-Funktion* erhalten Sie detaillierte Erläuterungen zu ausgewählten Entwicklungsobjekten oder Quelltextzeilen.

Um die *Explain-Funktion* zu öffnen, wählen Sie ein Entwicklungsobjekt im *Projekt-Explorer*, im Quellcode-Editor (durch Auswahl von Codezeilen) oder in anderen Sichten wie der *Schemaansicht* aus.

Wählen Sie dann *Joule* → *Explain* aus dem Kontextmenü, oder öffnen Sie die Joule-Chat-Sicht, und wählen Sie im Eingabefeld unten in der Sicht **`/Explain`** aus.

Die Verwendung der *Explain-Funktion* ist ein kollaborativer Prozess. Sie können für die *Explain-Fähigkeit* in natürlicher Sprache angeben, wie die Erklärung generiert werden soll, z.B. ob sie detailliert oder prägnant sein soll, mit oder ohne Beispiele.

© Copyright. Alle Rechte vorbehalten.

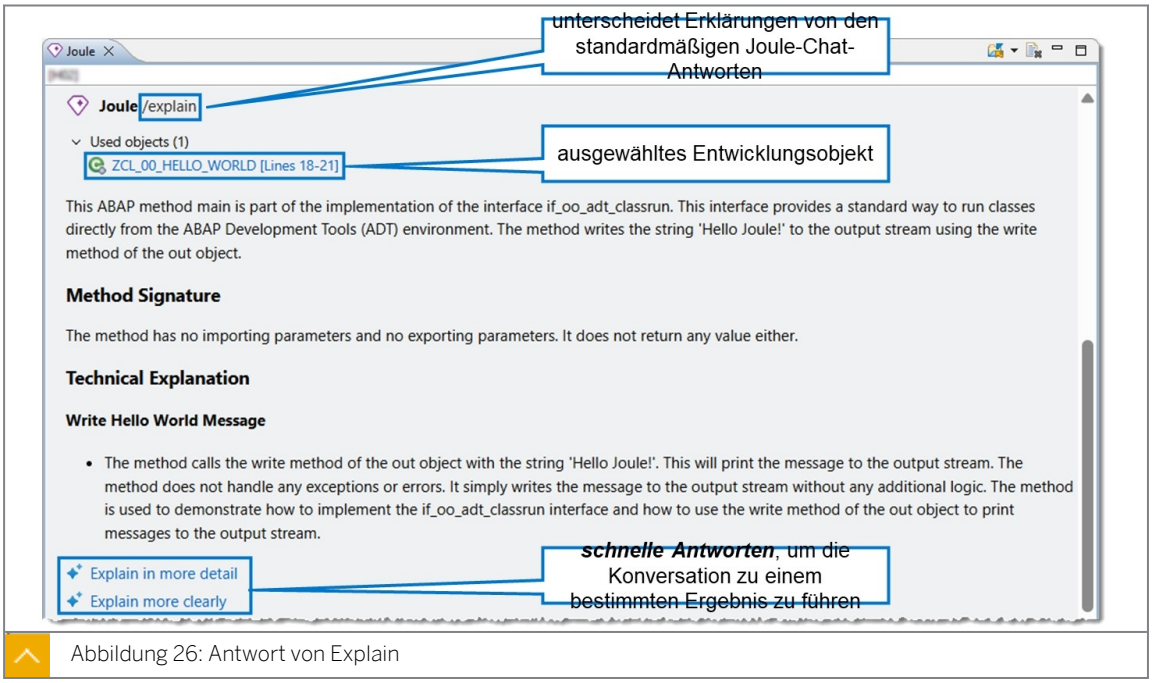


Abbildung 26: Antwort von Explain

Die Antwort von *Explain* wird in der Joule-Chat-Sicht mit dem ausgewählten Entwicklungsobjekt unter *Verwendete Objekte* angezeigt. Die Erläuterungen sind mit */Explain* gekennzeichnet, um sie von den standardmäßigen Joule-Chat-Antworten zu unterscheiden. Über die Joule-Chat-Sicht haben Sie die Möglichkeit, Folgefragen zu stellen. Für einige Entwicklungsobjekttypen erhalten Sie sogar die Möglichkeit, die Konversation mit *schnellen Antworten* zu einem bestimmten Ergebnis zu führen. Diese *schnellen Antworten* werden am Ende der Antwort von *Explain* in der Joule-Chat-Sicht angezeigt. Sie haben beispielsweise die Möglichkeit, durch die Verwendung solcher *schnellen Antworten* eine detailliertere oder verständlichere Erklärung zu generieren.

Darüber hinaus bietet SAP Joule für Entwickler die folgenden ABAP-KI-Funktionen:

- ABAP-Unit-Test-Generierung

  Die KI kann Ihnen beim Generieren von ABAP-Unit-Tests helfen. Auf diese Weise können Sie Tests anlegen, die Ihren Code einfacher prüfen.

- CDS-Testgenerierung

  Die KI kann Ihnen beim Anlegen von Testklassen für CDS-Entitäten helfen. Dadurch wird die Qualität und Korrektheit Ihrer CDS-Views sichergestellt.

- OData-UI-Service von Grund auf neu

  Legen Sie alle RAP-servicebezogenen Repository-Objekte mithilfe des Joule-Chats von Grund auf neu an.

- RAP-Business-Logikprognose

  Implementieren Sie Determinierungen oder Validierungen in der Implementierungsklasse Ihrer Verhaltensdefinition.

- Nutzen

  Generieren Sie einen OData-Request und ABAP-Code für die Verwendung eines entsprechenden OData-Aufrufs über den OData-Client-Proxy.

- GenKI-gesteuerter Erweiterbarkeitsassistent

© Copyright. Alle Rechte vorbehalten.

Der Erweiterbarkeits-Assistent kann Fragen zur Felderweiterbarkeit und Feldlogik beantworten. Sie kann Sie bei verschiedenen Aufgaben im Zusammenhang mit der Felderweiterbarkeit unterstützen, einschließlich dem Anlegen benutzerdefinierter Felder und der Suche nach Wertehilfesichten.

Achtung:
Wichtige Punkte, die Sie beachten sollten:

- KI-Ergebnisse können variieren.
- Prüfen Sie KI-generierten Code immer auf Richtigkeit.
- Stellen Sie sicher, dass KI-generierter Code Ihren Anforderungen entspricht.

Eigene KI-gestützte Anwendungen erstellen

ABAP AI SDK

Während Joule KI-Hilfe in der Entwicklungsumgebung bereitstellt, können Sie mit dem ABAP AI SDK (Software Development Kit) KI-Funktionen direkt zu Ihren eigenen Geschäftsanwendungen hinzufügen.

Das ABAP AI SDK stellt eine ABAP-Bibliothek bereit, mit der Sie große Sprachmodelle (Large Language Models, LLMs) verwenden können.

Mit dem ABAP AI SDK können Sie:

- Verwenden Sie LLMs aus Ihrem ABAP-Code.
- Fügen Sie Ihren Apps oder Services GenAI-Funktionen hinzu.
- Verwenden Sie Funktionen wie die Vervollständigung von Eingabeaufforderungen und Funktionsaufrufe.

Mit dem ABAP AI SDK können Sie die folgenden Funktionen verwenden:

- Fertigstellungs-API

  Dieses API bietet Zugriff auf die Fertigstellungs-API-Funktion von LLMs, d.h., es verwendet ein LLM, um eine Textantwort aus einer Eingabeaufforderung zu generieren.

- Prompt-Library-API

  Mit der Prompt-Library-API können Sie vordefinierte Prompt-Vorlagen verwenden, um neue Prompts zu generieren.

- Tracing

  Wenn beim Aufrufen des APIs Probleme auftreten, können Sie das ABAP-Cross-Trace initiieren. Sie können dies z.B. mit dem Fertigstellungs-API tun, um den Aufrufstack zu prüfen und die Fehlerursache zu ermitteln.

Einrichten des ABAP AI SDK

Um das ABAP AI SDK verwenden zu können, muss Ihr Systemadministrator Systemverbindungen mit dem Hub für generative KI einrichten. Als Entwickler definieren Sie das zu verwendende LLM und legen intelligente Szenarios und Modelle an.

© Copyright. Alle Rechte vorbehalten.

Zusammenfassung

- ABAP-KI-Funktionen helfen Entwicklern, produktiver zu sein, indem sie KI-Unterstützung in der ABAP-Entwicklungsumgebung bereitstellen.
- Zu den wichtigsten Funktionen von Joule gehören Joule-Chat, Predictive Code Completion und Explain, die Entwicklungsaufgaben vereinfachen.
- Mit dem ABAP AI SDK können Entwickler benutzerdefinierte KI-fähige Geschäftsanwendungen erstellen, indem sie große Sprachmodelle verwenden.

Weitere Lektüre

- SAP Joule für Entwickler, ABAP-KI-Funktionen | SAP Help Portal
- Eigene KI-fähige Anwendungen entwickeln | SAP Help Portal
- Workshop RAP120 – SAP-Fiori-Apps mit ABAP Cloud und SAP Joule für Entwickler erstellen

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Erläutern Sie, wie ABAP-KI-Funktionen ABAP-Entwickler bei der Anwendungsentwicklung unterstützen können.
- Beschreiben Sie kurz, wie das ABAP AI SDK verwendet werden kann, um Geschäftsanwendungen KI-Funktionen hinzuzufügen.

© Copyright. Alle Rechte vorbehalten.

Kapitel 1

Überprüfung des Lernerfolgs

1. Mit einem ABAP-Cloud-Projekt in ABAP Development Tools (ADT) können Sie eine Verbindung zu welchen Systemarten herstellen?

   Wählen Sie die richtigen Antworten.

   - ☐ **A** ABAP-Instanz von SAP BTP
   - ☐ **B** On-Premise-ABAP-System
   - ☐ **C** Integrierte ABAP-Instanz in SAP S/4HANA Cloud

2. Wie navigieren Sie im ABAP-Quelltext zur Definition eines Entwicklungsobjekts?

   Wählen Sie die richtige Antwort.

   - ☐ **A** Doppelklicken Sie auf den Namen des Objekts.
   - ☐ **B** Positionieren Sie den Cursor auf den Namen des Objekts, und drücken Sie F2.
   - ☐ **C** Positionieren Sie den Cursor auf den Namen des Objekts, und drücken Sie F3.

3. Was müssen Sie tun, wenn Sie ein ABAP-Paket anlegen?

   Wählen Sie die richtigen Antworten.

   - ☐ **A** Verwenden Sie den Buchstaben Z oder Y als erstes Zeichen im Namen.
   - ☐ **B** Ordnen Sie das Paket einer Anwendungskomponente zu.
   - ☐ **C** Ordnen Sie das Paket einer Softwarekomponente zu.
   - ☐ **D** Geben Sie an, welche anderen Pakete den Inhalt des Pakets verwenden können.

4. Welche der folgenden Zeichen sind im Namen einer ABAP-Klasse nicht zulässig?

   Wählen Sie die richtige Antwort.

   - ☐ **A** 8
   - ☐ **B** _
   - ☐ **C** $

© Copyright. Alle Rechte vorbehalten.

5. Welche der folgenden Funktionen ist eine Schlüsselfunktion der ABAP-KI-Funktionen?

   Wählen Sie die richtige Antwort.

   - ☐ A Automatisierte Bereitstellung in SAP BTP
   - ☐ B Predictive Code Completion
   - ☐ C Echtzeitüberwachung der Anwendungsperformance
   - ☐ D Automatische Übersetzung von ABAP-Code in andere Sprachen

6. Welche der folgenden Funktionen kann die Explain-Funktion in Joule bereitstellen?

   Wählen Sie die richtige Antwort.

   - ☐ A Einfache Erläuterungen des ABAP-Codes
   - ☐ B Automatische Generierung von UIs
   - ☐ C Performanceanalyse von ABAP-Code
   - ☐ D Vorschläge zur Datenbankoptimierung

7. Was ist der Hauptzweck des ABAP AI SDK?

   Wählen Sie die richtige Antwort.

   - ☐ A Bereitstellung einer Entwicklungsumgebung zum Anlegen von ABAP-Code
   - ☐ B Verwalten von Benutzerberechtigungen in ABAP-Systemen
   - ☐ C Um die Entwicklung benutzerdefinierter KI-gestützter Geschäftsanwendungen zu ermöglichen
   - ☐ D Automatisieren des Deployments von ABAP-Anwendungen in der Cloud

© Copyright. Alle Rechte vorbehalten.

Überprüfung des Lernerfolgs - Antworten

1. Mit einem ABAP-Cloud-Projekt in ABAP Development Tools (ADT) können Sie eine Verbindung zu welchen Systemarten herstellen?

   Wählen Sie die richtigen Antworten.

   - [X] A ABAP-Instanz von SAP BTP
   - [ ] B On-Premise-ABAP-System
   - [X] C Integrierte ABAP-Instanz in SAP S/4HANA Cloud

   Das ist richtig.

2. Wie navigieren Sie im ABAP-Quelltext zur Definition eines Entwicklungsobjekts?

   Wählen Sie die richtige Antwort.

   - [ ] A Doppelklicken Sie auf den Namen des Objekts.
   - [ ] B Positionieren Sie den Cursor auf den Namen des Objekts, und drücken Sie F2.
   - [X] C Positionieren Sie den Cursor auf den Namen des Objekts, und drücken Sie F3.

   Das ist richtig.

3. Was müssen Sie tun, wenn Sie ein ABAP-Paket anlegen?

   Wählen Sie die richtigen Antworten.

   - [X] A Verwenden Sie den Buchstaben Z oder Y als erstes Zeichen im Namen.
   - [ ] B Ordnen Sie das Paket einer Anwendungskomponente zu.
   - [X] C Ordnen Sie das Paket einer Softwarekomponente zu.
   - [ ] D Geben Sie an, welche anderen Pakete den Inhalt des Pakets verwenden können.

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

4. Welche der folgenden Zeichen sind im Namen einer ABAP-Klasse nicht zulässig?

   Wählen Sie die richtige Antwort.

   - [ ] A 8
   - [ ] B _
   - [x] C $

   Das ist richtig.

5. Welche der folgenden Funktionen ist eine Schlüsselfunktion der ABAP-KI-Funktionen?

   Wählen Sie die richtige Antwort.

   - [ ] A Automatisierte Bereitstellung in SAP BTP
   - [x] B Predictive Code Completion
   - [ ] C Echtzeitüberwachung der Anwendungsperformance
   - [ ] D Automatische Übersetzung von ABAP-Code in andere Sprachen

   Richtig! Die vorausschauende Code-Vervollständigung ist eine Schlüsselfunktion der ABAP-KI-Funktionen.

6. Welche der folgenden Funktionen kann die Explain-Funktion in Joule bereitstellen?

   Wählen Sie die richtige Antwort.

   - [x] A Einfache Erläuterungen des ABAP-Codes
   - [ ] B Automatische Generierung von UIs
   - [ ] C Performanceanalyse von ABAP-Code
   - [ ] D Vorschläge zur Datenbankoptimierung

   Richtig! Die Explain-Funktion in Joule kann einfache Erläuterungen des ABAP-Codes bereitstellen.

© Copyright. Alle Rechte vorbehalten.

7. Was ist der Hauptzweck des ABAP AI SDK?

   Wählen Sie die richtige Antwort.

   - [ ] A Bereitstellung einer Entwicklungsumgebung zum Anlegen von ABAP-Code
   - [ ] B Verwalten von Benutzerberechtigungen in ABAP-Systemen
   - [x] C Um die Entwicklung benutzerdefinierter KI-gestützter Geschäftsanwendungen zu ermöglichen
   - [ ] D Automatisieren des Deployments von ABAP-Anwendungen in der Cloud

   Richtig! Der Hauptzweck des ABAP AI SDK besteht darin, die Entwicklung benutzerdefinierter KI-fähiger Geschäftsanwendungen zu ermöglichen.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

KAPITEL 2 Grundlegende Methoden und Konzepte anwenden

Lektion 1

Grundlagen von ABAP verstehen 49

Lektion 2

Mit grundlegenden Datenobjekten und Datentypen arbeiten 55

Lektion 3

Daten verarbeiten 66

Übung 4 : Variablen deklarieren und Daten verarbeiten 71

Lektion 4

Kontrollstrukturen in ABAP verwenden 77

Übung 5 : Bedingte Verzweigungen implementieren 85

Lektion 5

Mit einfachen internen Tabellen arbeiten 93

Übung 6 : Mit einfachen internen Tabellen arbeiten 99

Lektion 6

Debugging eines ABAP-Programms 107

Übung 7 : ABAP-Code debuggen 117

LERNZIELE

- Beschreiben Sie die Entwicklung von ABAP.
- Beschreiben Sie die Grundlagen der ABAP-Syntax.
- Deklarieren Sie Datenobjekte.
- Ordnen Sie Werte zu.
- Führen Sie arithmetische Berechnungen durch.
- Zeichenkettenverarbeitung anwenden.

© Copyright. Alle Rechte vorbehalten.

- bedingte Verzweigung implementieren
- Ausnahmen behandeln.
- Iterationen implementieren.
- Definieren Sie einfache interne Tabellen.
- Daten mit einfachen internen Tabellen verarbeiten.
- Wechseln Sie in den Debugging-Modus.
- Steuern Sie die Ausführung von Code.
- Analysieren Sie den Inhalt von Datenobjekten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Lektion 1

Grundlagen von ABAP verstehen

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Beschreiben Sie die Entwicklung von ABAP.
- Beschreiben Sie die Grundlagen der ABAP-Syntax.

ABAP früher und heute

Ursprung und Evolution von ABAP

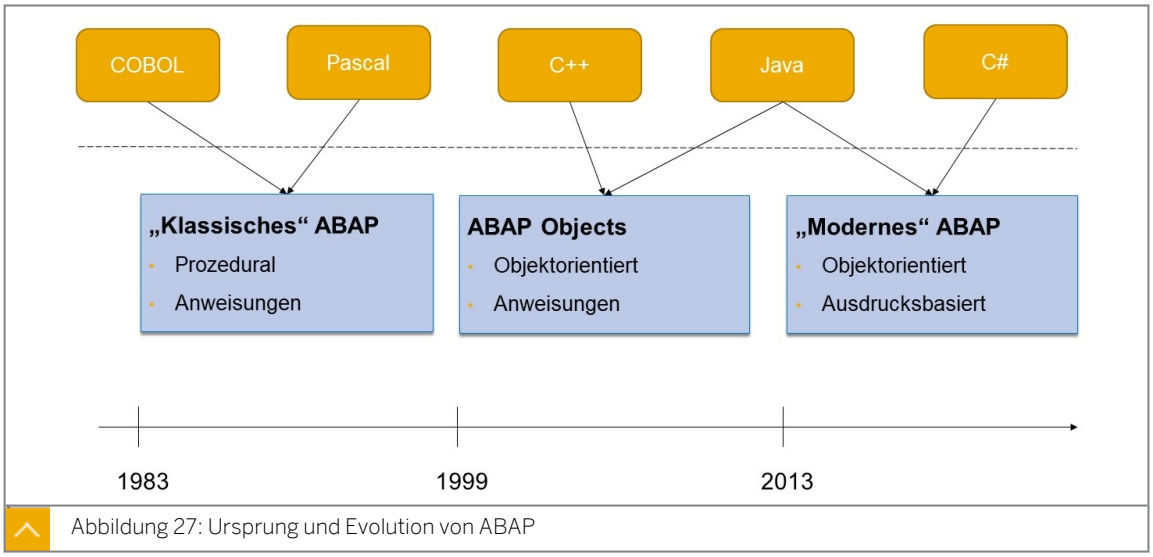


Abbildung 27: Ursprung und Evolution von ABAP

ABAP ist eine von SAP entwickelte Programmiersprache zur Entwicklung von Unternehmensanwendungen in der SAP-Umgebung.

Als ABAP 1983 erstmals veröffentlicht wurde, war es als prozedurale Programmiersprache auf der Grundlage von Anweisungen konzipiert. Obwohl es relativ unabhängig war, wurde es von anderen populären Programmiersprachen der Zeit beeinflusst, hauptsächlich von COBOL und Pascal.

Bis Ende der 1990er Jahre wurde die Sprache ABAP erweitert, um eine objektorientierte Programmierung zu ermöglichen. Wichtige Konzepte gängiger objektorientierter Programmiersprachen wie C++ und Java wurden in die ABAP-Sprache integriert. ABAP Objects, wie die Programmiersprache inzwischen genannt wurde, kombinierte prozedurale und objektorientierte Teile in einer Sprache.

Obwohl ABAP die objektorientierte Programmierung ermöglichte, blieb die Syntax von Anweisungen dominiert. Im Vergleich zu anderen modernen Programmiersprachen wie Java und C# erschien ABAP-Code langwierig, kompliziert und beinhaltete viele Hilfsvariablen, um

© Copyright. Alle Rechte vorbehalten.

die verschiedenen Anweisungen zu verknüpfen. Um dies zu ändern, wurde ABAP um moderne Ausdrücke erweitert, um die bestehenden Anweisungen zu ersetzen oder zu erweitern. Auch jetzt kombiniert ABAP traditionelle anweisungsbasierte Syntaxelemente mit modernen, ausdrucksbasierten Syntaxelemente in einer Programmiersprache.

Für neue ABAP-Entwicklungen wird empfohlen, wo immer möglich objektorientierte und ausdrucksbasierte Syntaxelemente zu verwenden.

ABAP-Sprachversionen

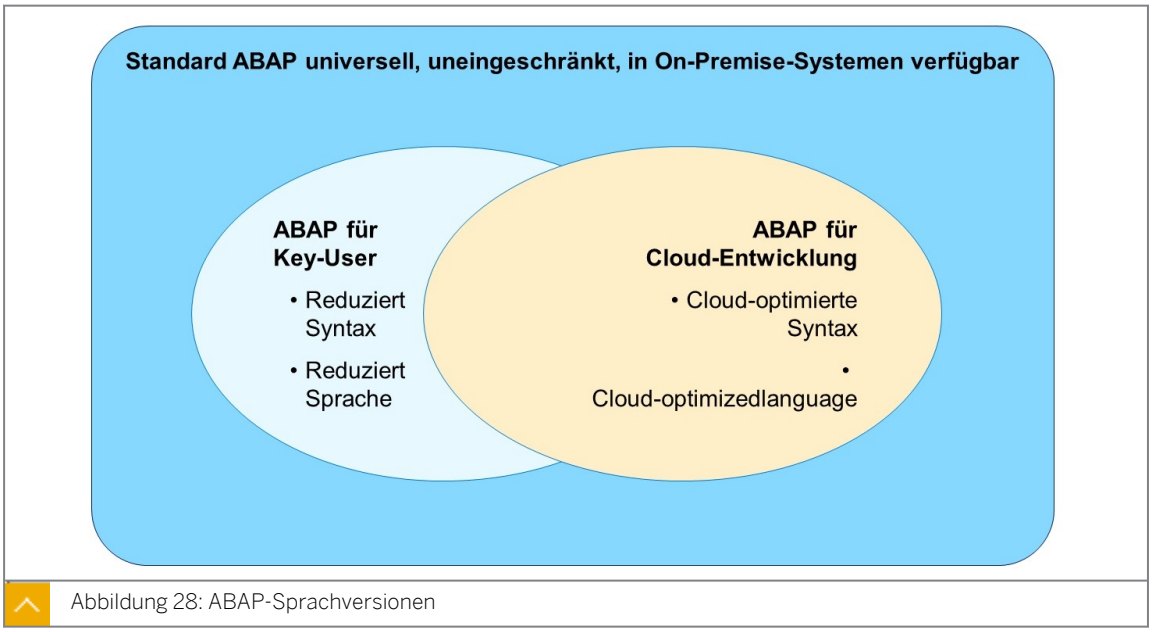


Abbildung 28: ABAP-Sprachversionen

Jedes ABAP-Programm weist die Programmeigenschaft „ABAP-Sprachversion" auf, die intern durch eine Versionskennung definiert ist. Die Version eines Programms bestimmt, welche Sprachelemente und Repository-Objekte im Programm verwendet werden können und welche Syntaxregeln gelten. Folgende Versionen sind derzeit verfügbar:

Standard ABAP

Diese ABAP-Version ist die universelle Grundversion. Dabei handelt es sich um eine uneingeschränkte ABAP-Sprachversion, die den gesamten in Unicode-Systemen verwendbaren Sprachumfang von ABAP abdeckt und, abgesehen vom statischen Paketkonzept, den Zugriff auf alle anderen Repository-Objekte zulässt. Die Syntaxprüfung für Standard ABAP wird als Unicode-Prüfung durchgeführt. Dies ist die Mindestanforderung für ein Unicode-System.

ABAP für Key-User

Diese ABAP-Version ist für die sichere Implementierung von Erweiterungen durch Key-User im Rahmen der von SAP bereitgestellten Erweiterungsoptionen vorgesehen. ABAP für Key-User ist eine eingeschränkte ABAP-Sprachversion, in der die allgemeinen Regeln für Standard ABAP gelten, aber nur ein sehr eingeschränkter Satz an Sprachelementen unterstützt wird und der Zugriff auf Repository-Objekte eingeschränkt ist.

ABAP für Cloud-Entwicklung

Diese ABAP-Version ist für Entwicklungen in der ABAP-Umgebung von SAP BTP sowie in der ABAP-Umgebung von S/4HANA Cloud vorgesehen. ABAP für Cloud-Entwicklung ist eine eingeschränkte ABAP-Sprachversion, in der die allgemeinen Regeln für Standard ABAP gelten, aber nur ein eingeschränkter Satz an Sprachelementen unterstützt wird und der Zugriff auf Repository-Objekte eingeschränkt ist.

© Copyright. Alle Rechte vorbehalten.

Notiz:
Dieser Kurs bietet eine allgemeine Einführung in die ABAP-Entwicklung. Diese beschränkt sich auf Syntaxelemente und Sprachfunktionen, die in allen drei Sprachversionen verfügbar sind. Eine Ausnahme bildet die Einführung in das *ABAP-RESTful-Anwendungsprogrammiermodell*, das kein Bestandteil von *ABAP für Key-User* ist.

ABAP Cloud

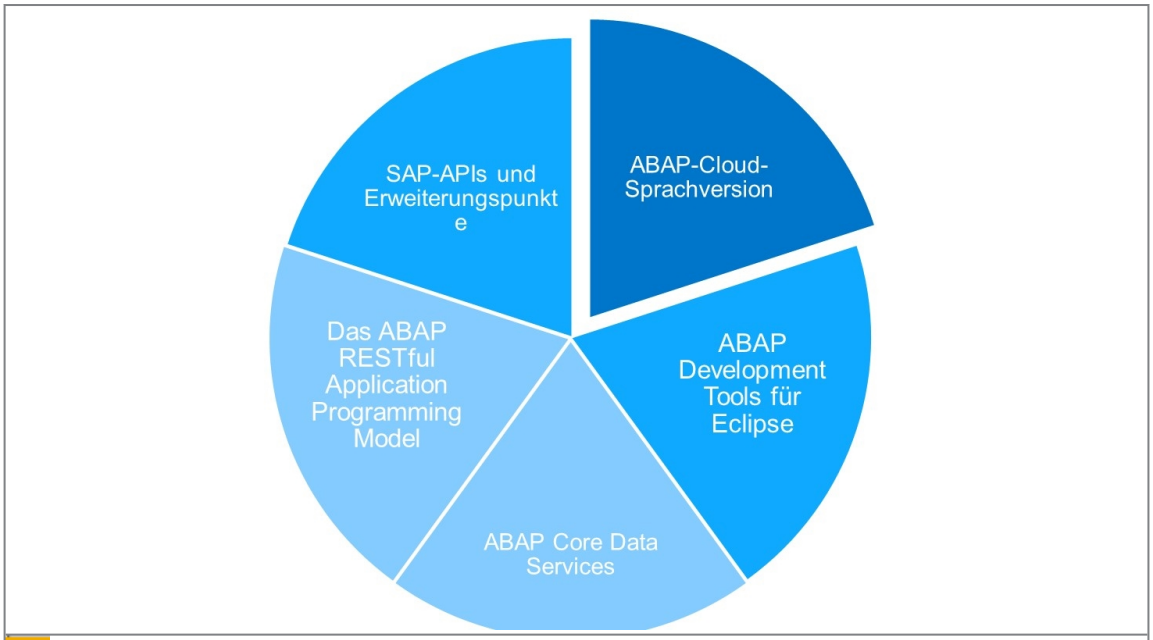


Abbildung 29: ABAP-Cloud-Elemente

Die Sprachversion ABAP für Cloud-Entwicklung ist ein Schlüsselelement von **ABAP Cloud**. Der technologische Kern von ABAP Cloud definiert die Designzeit- und Laufzeitarchitektur aller Erweiterungen, Services und Anwendungen. Die wichtigsten ABAP-Cloud-Elemente sind:

- **ABAP für Cloud-Entwicklung** als Cloud-optimierte ABAP-Sprache für die Geschäftslogik
- **ABAP Development Tools** für Eclipse als integrierte ABAP-Entwicklungsumgebung
- **ABAP Core Data Services (CDS)** für das Datenmodell und für integrierte Analysefunktionen
- Das **ABAP-RESTful-Anwendungsprogrammiermodell** zum Erstellen von Apps und Services
- **Obligatorische öffentliche SAP-APIs und Erweiterungspunkte**, um automatisierte Cloud-Vorgänge und eine lebenszyklusstabile Erweiterbarkeit zu ermöglichen.

© Copyright. Alle Rechte vorbehalten.

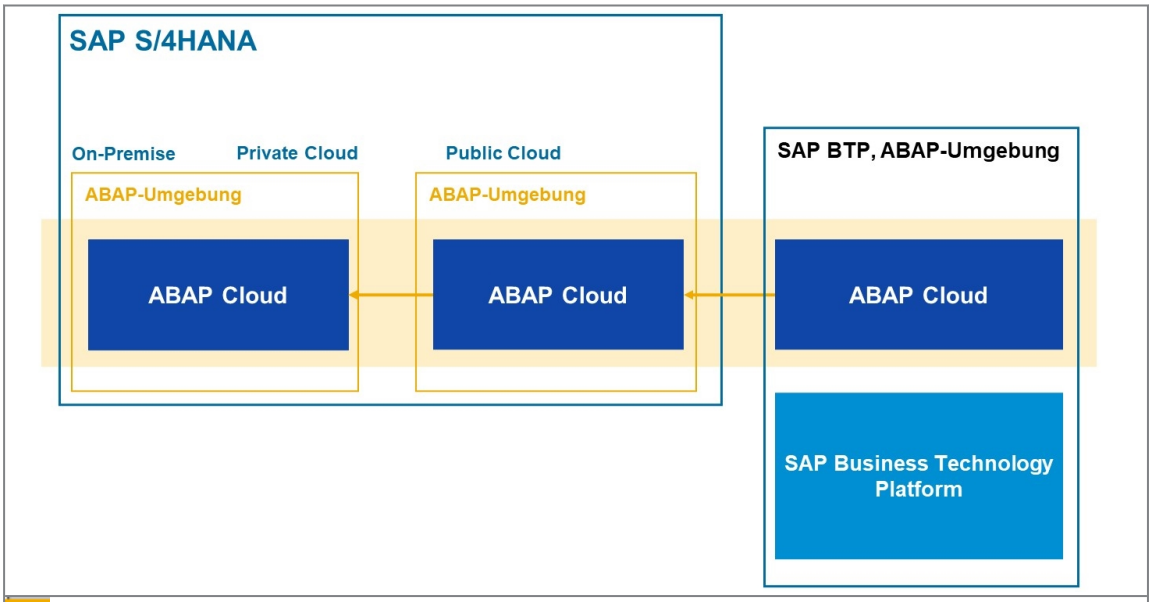


Abbildung 30: ABAP-Cloud-Verfügbarkeit

Die Verwendung von ABAP Cloud ist auf der *SAP Business Technology Platform* und in *SAP S/4HANA Cloud, Public Edition* obligatorisch. Es ist nicht obligatorisch, wird jedoch dringend empfohlen, die Prinzipien von ABAP Cloud in *SAP S/4HANA, Private Edition* und in On-Premise-Systemen zu befolgen. Wenn Sie Ihre benutzerdefinierte Entwicklung mit ABAP Cloud implementieren, sind Ihre Entwicklungen standardmäßig cloudfähig, und Sie ändern nicht die Zusammensetzung des Kernsystems. Dies ist ein wichtiges Prinzip für die Systempflege in der Cloud-Umgebung (Clean-Core-Ansatz).

Grundlagen der ABAP-Syntax

Grundlegende Funktionen der ABAP-Sprache

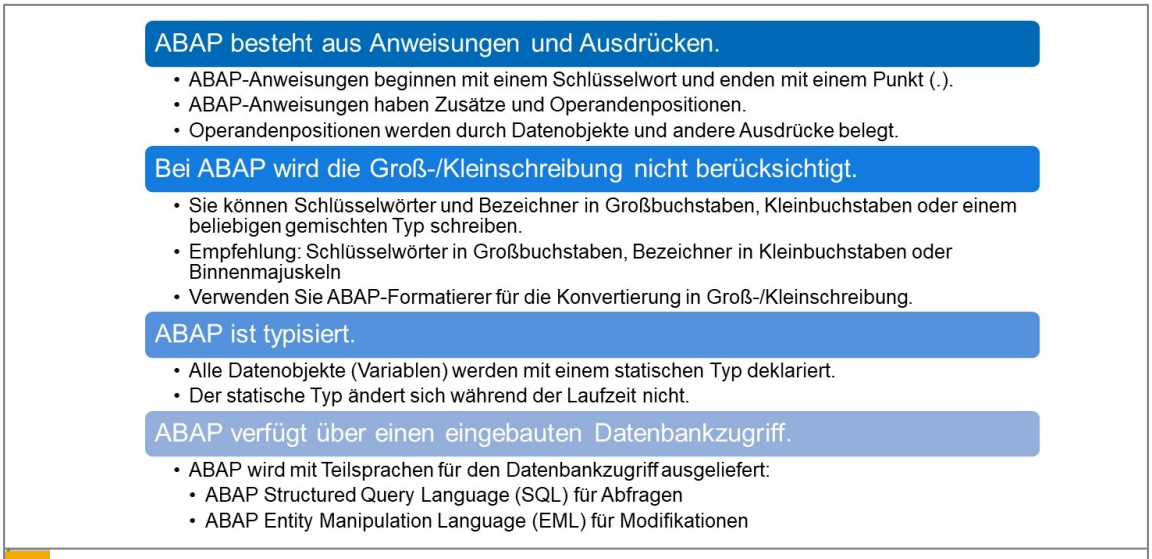


Abbildung 31: Grundlegende Funktionen der ABAP-Sprache

© Copyright. Alle Rechte vorbehalten.

Beispiele für ABAP-Syntax

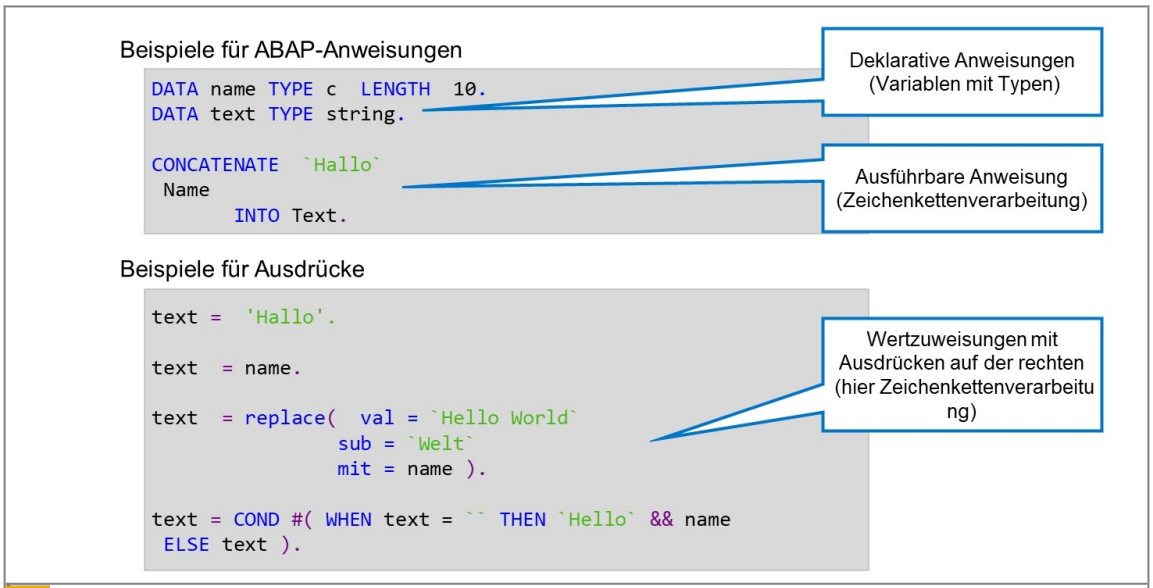


Abbildung 32: Beispiele für ABAP-Syntax

Diese Codebeispiele veranschaulichen einige der grundlegenden Funktionen der Programmiersprache ABAP. Der obere Code besteht aus drei Anweisungen, von denen jede mit einem Punkt (.) endet.

Die DATA-Anweisung ist eine deklarative Anweisung zur Definition von Variablen. Die Anweisung CONCATENATE ist eine Anweisung zur Datenverarbeitung, um zeichenartige Daten zu kombinieren.

Das untere Codebeispiel besteht aus vier Anweisungen.

Der untere Code besteht aus vier Wertzuweisungen. Auf den Namen der Zielvariable (text) folgt das Gleichheitszeichen (=), das als Zuweisungsoperator dient. Die Operandenposition auf der rechten Seite wird durch unterschiedlich komplexe Ausdrücke belegt: ein Literal, eine Variable, eine Zeichenkettenverarbeitungsfunktion (replace) und ein Ausdruck für eine bedingte Zuweisung (COND).

© Copyright. Alle Rechte vorbehalten.

Kommentare in ABAP

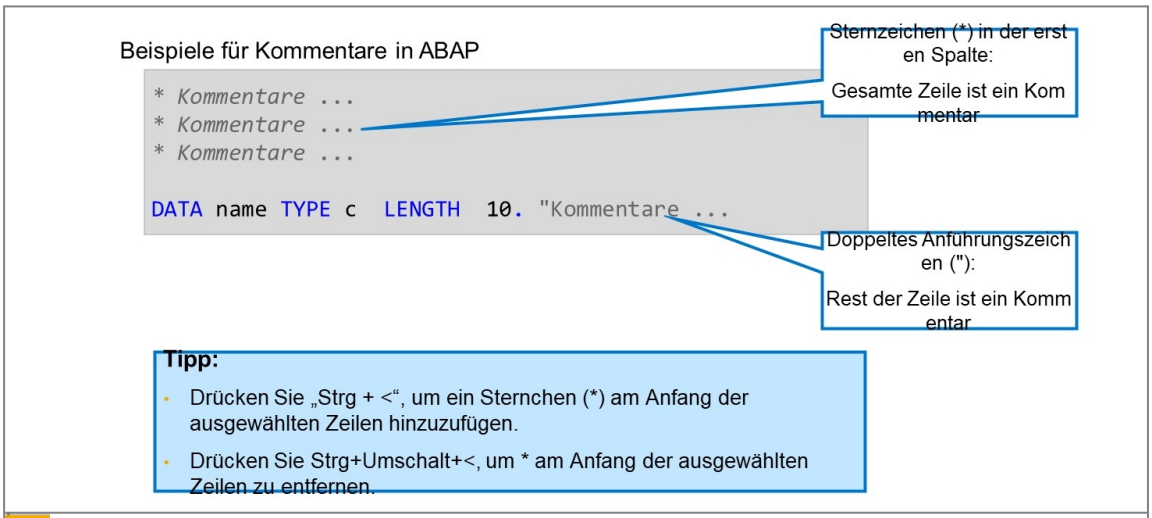


Abbildung 33: Kommentare in ABAP

Ein Kommentar in einer Programmiersprache ist ein Quelltext, der vom Compiler oder Interpreter ignoriert wird.

In ABAP gibt es zwei Möglichkeiten, Quelltext als Kommentar zu definieren:

- Ein Sternchen (*) in der ersten Spalte kennzeichnet die gesamte Zeile als Kommentar.
- Ein doppeltes Anführungszeichen (") kennzeichnet den Rest der Zeile, d.h. den Quelltext rechts daneben, als Kommentar.

Notiz:
Sie können das Anführungszeichen (") in einer beliebigen Spalte platzieren. Das Sternchen (*) funktioniert nur, wenn es in der ersten Spalte platziert ist. An anderen Stellen führt dies zu einem Syntaxfehler, es sei denn, es handelt sich um einen syntaktisch korrekten Teil einer ABAP-Anweisung.

Hinweis:
Um eine oder mehrere Codezeilen zu kommentieren, d.h., um in der ersten Spalte ein Sternchen (*) hinzuzufügen, markieren Sie die Codezeilen, und drücken Sie **`Strg + <`**. Alternativ können Sie über das Kontextmenü *Source Code → Add Comment* wählen. Um die ausgewählten Codezeilen zu entkommentieren, d.h., um das Sternchen (*) aus der ersten Spalte zu entfernen, drücken Sie **`Strg + >`**, oder wählen Sie im Kontextmenü *Source Code → Remove Comment*.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Beschreiben Sie die Entwicklung von ABAP.
- Beschreiben Sie die Grundlagen der ABAP-Syntax.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Lektion 2

Mit grundlegenden Datenobjekten und Datentypen arbeiten

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Deklarieren Sie Datenobjekte.
- Ordnen Sie Werte zu.

Datenobjekte in ABAP

Ein Datenobjekt in einem ABAP-Programm bildet einen reservierten Bereich des Programmspeichers ab.

ABAP kennt drei Arten von Datenobjekten: Variablen, Konstanten und Literale.

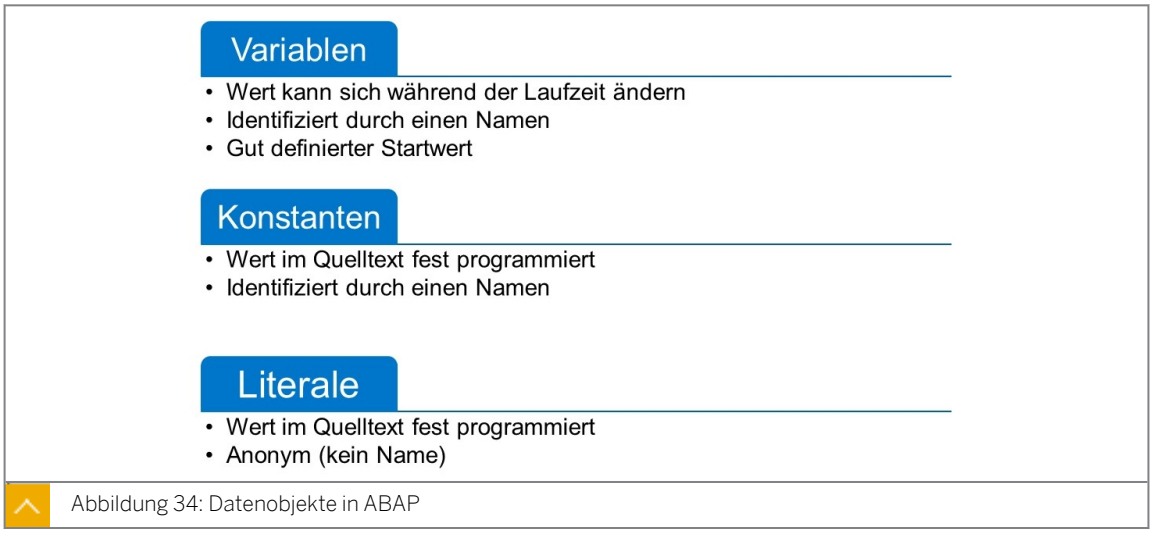


Abbildung 34: Datenobjekte in ABAP

Variablen

Eine Variable ist ein Datenobjekt mit einem Inhalt, der sich während der Laufzeit ändern kann. Eine Variable ist durch einen Namen gekennzeichnet. Der Name wird auch verwendet, um das Datenobjekt zur Laufzeit zu adressieren. Der Startwert einer ABAP-Variable ist immer wohldefiniert.

Konstanten

Konstanten ähneln Variablen. Im Gegensatz zu Variablen ist der Wert jedoch im Quelltext fest programmiert und darf sich während der Laufzeit nicht ändern. Konstanten haben wie Variablen einen Namen, über den sie wiederverwendet werden können.

Literale

© Copyright. Alle Rechte vorbehalten.

Der Wert von Literalen ist ebenfalls im Quelltext fest programmiert. Im Gegensatz zu Konstanten haben Literale keinen Namen. Aus diesem Grund können Sie ein Literal nicht wiederverwenden. Verwenden Sie Literale nur, um die Werte für Konstanten und die Startwerte für Variablen anzugeben.

ABAP-Datenobjekte sind immer typisiert: Jedes Datenobjekt basiert auf einem Datentyp, der bestimmt, welche Art von Informationen es enthalten kann. Der Datentyp eines ABAP-Datenobjekts bleibt während der Programmausführung unverändert.

Deklaration von Variablen

Eine Variable in einem ABAP-Programm wird mit dem Schlüsselwort DATA deklariert.

Eine DATA-Anweisung besteht aus drei Teilen. Sehen wir uns die einzelnen Teile genauer an.

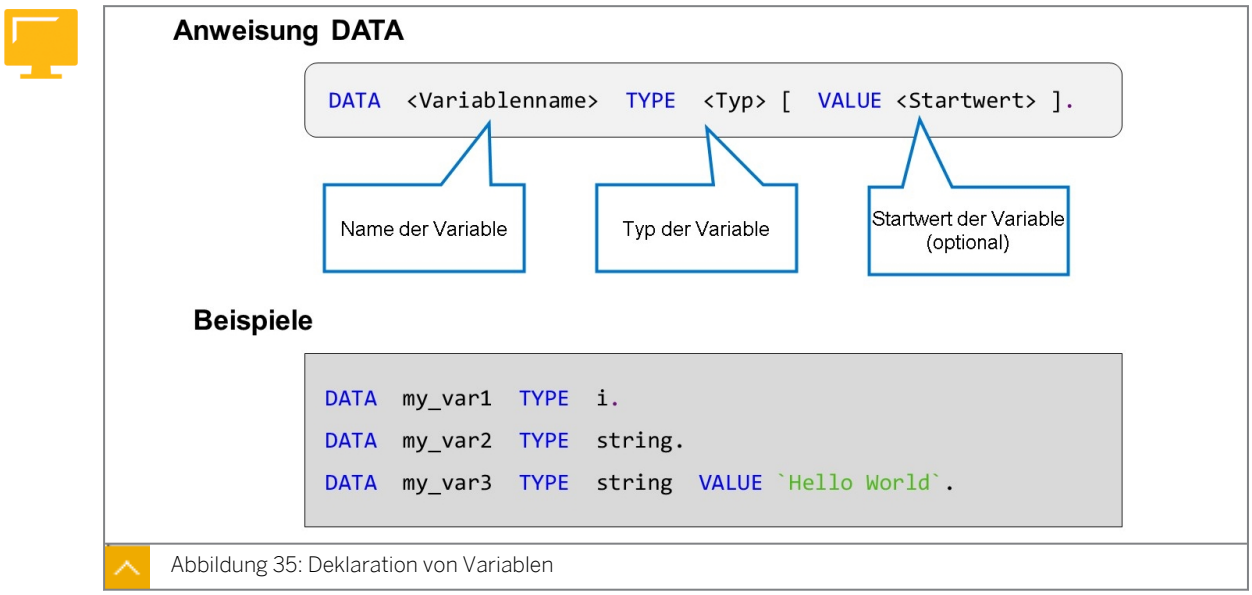


Abbildung 35: Deklaration von Variablen

DATEN

Auf das Schlüsselwort DATA folgt der Name der Variablen. Der Name einer Variablen kann maximal 30 Zeichen umfassen. Er kann die Zeichen A-Z, die Ziffern 0-9 und einen Unterstrich enthalten. Der Name muss mit einem Buchstaben oder einem Unterstrich beginnen.

TYPE

Nach dem Zusatz TYPE wird der Typ der Variablen angegeben. Im Beispiel werden die eingebauten Typen I (für Integerzahlen) und STRING (Zeichenfolge mit variabler Länge) verwendet.

VALUE

Der Zusatz VALUE ist optional und kann verwendet werden, um einen Startwert für die Variable anzugeben. Wenn VALUE fehlt, wird die Variable mit einem Initialwert angelegt, der vom technischen Typ der Variable abhängt.

© Copyright. Alle Rechte vorbehalten.

Quellen für ABAP-Datentypen

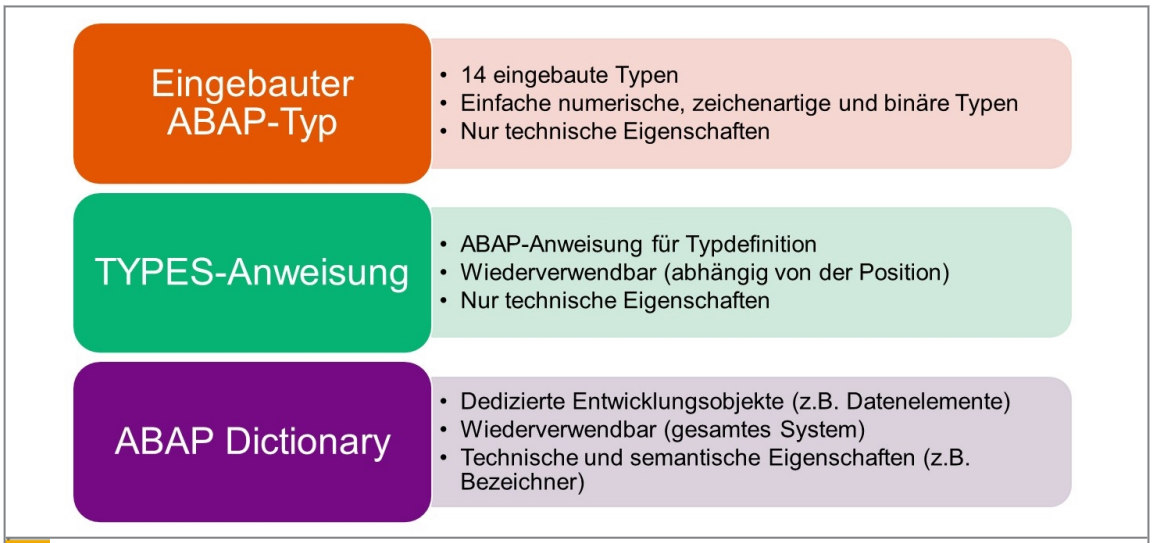


Abbildung 36: Quellen für ABAP-Datentypen

ABAP bietet folgende Quellen für Datentypen:

Eingebauter ABAP-Typ

ABAP weist einen Satz mit 14 eingebauten Datentypen für einfache numerische, zeichenartige und binäre Datenobjekte auf.

TYPES-Anweisung

Die Anweisung TYPES ermöglicht die Definition von Datentypen und deren Wiederverwendung an verschiedenen Stellen, je nach Ort der Definition.

ABAP Dictionary

Das ABAP Dictionary ist Bestandteil des ABAP Repository. Es verwaltet unter anderem globale Datentypen, die systemweit verfügbar sind. ABAP-Dictionary-Typen definieren nicht nur technische Eigenschaften, sondern fügen semantische Informationen, z.B. Bezeichner, hinzu. ABAP-Dictionary-Typen sind insbesondere bei der Implementierung von Benutzungsoberflächen nützlich.

© Copyright. Alle Rechte vorbehalten.

Eingebaute ABAP-Typen

```
TYPES  <Variable>  TYPE  <Typ>.
```

Typ	Wertebereich	Initialwert
STRING	Beliebige Zeichen, beliebige Länge	leer
I	-2.147.483.648 bis +2.147.483.647	0
D	Datum im Format JJJJMMTT	'00000000'
T	Uhrzeit im Format HHMMSS	'000000'

```
TYPES  <Variable>  TYPE  <Typ>  LENGTH <len>  DECIMALS  <dec>.
```

Typ	Werte für <Länge>	Werte für <Dezimalstellen>	Wertebereich	Initialwert
C	1 – 262.143	n.a.	Beliebige Zeichen	Abfolge von Leerzeichen
N	1 – 262.143	n.a.	Nur Ziffern	Abfolge des Zeichens '0'
P	1 – 16	0 – 14	Abhängig von der Länge	0

Abbildung 37: Eingebaute ABAP-Typen

Einige wichtige ABAP-Typen sind in der Abbildung „Eingebaute ABAP-Typen“ aufgeführt. Für einige dieser Typen können Sie den Zusatz LENGTH hinzufügen, um eine feste Länge anzugeben. Diese Typen werden als unvollständige Typen bezeichnet. Im Fall des Typs P können Sie auch die Anzahl der Dezimalstellen festlegen. Die ABAP-Typen, für die die Zusätze LENGTH und DECIMALS nicht unterstützt werden, werden als vollständige Typen bezeichnet.

Vollständige ABAP-Typen

TYP STRING

Ein Feld vom Typ STRING ist ein zeichenartiges Feld variabler Länge. Im ABAP-Laufzeitsystem wird Speicher zugewiesen und freigegeben, um die Verwaltung von STRING-Variablen zu optimieren. Dieses Verhalten können Sie nicht direkt beeinflussen. Die maximale Länge einer Zeichenfolge wird durch Einstellungen gesteuert, die der Systemadministrator vornimmt. In der Praxis reden wir aber von einer unbegrenzten Länge von Zeichenfolgen.

TYP I

Ein Feld vom Typ I ist ein numerisches Feld, das eine ganze Zahl enthält. Das System weist 4 Byte (32 Bit) für ein solches Feld zu, das Werte zwischen - 2^31 und + 2^31 zulässt.

TYP D

Ein Feld vom Typ D stellt ein Datum dar. In ABAP hat das Datum immer das Format JJJJMMTT (ohne Trennzeichen). Das System konvertiert dieses Format gemäß dem aktuellen Gebietsschema, bevor der Wert auf der Benutzungsoberfläche angezeigt wird. Ebenso wird ein Datum bei der Eingabe vom System in das ABAP-Format umgewandelt, bevor es verarbeitet wird.

TYP T

Ein Feld vom Typ T stellt eine Uhrzeit dar. In ABAP hat dieser Typ das Format HHMMSS (ohne Trennzeichen, im 24-Stunden-Format). Wenn im aktuellen Gebietsschema ein 12-Stunden-Format verwendet wird, wandelt das System die Werte automatisch um.

© Copyright. Alle Rechte vorbehalten.

Unvollständige ABAP-Typen

Typ C

Ein Feld vom Typ C ist ein zeichenartiges Feld bestimmter Länge. Sie geben die Länge in Zeichen an; das Laufzeitsystem weist dann für das Feld die doppelte Anzahl an Byte zu. Sie verwenden diesen Typ, wenn eine feste Länge wichtig ist.

Typ N

Ein Feld vom Typ N ist ein Zeichenfeld bestimmter Länge, das nur Ziffern enthält. Dieses Feld muss eine Folge von Ziffern enthalten, die nicht als Zahl betrachtet werden soll und mit der Sie keine Berechnungen durchführen möchten. Hierbei kann es sich z.B. um eine Personalnummer oder eine Kostenstelle handeln.

Typ P

Ein Feld vom Typ P (P für „Packed Number = gepackte Zahl") ist ein Feld, das einen numerischen Wert mit einer bestimmten Anzahl von Ziffern und Dezimalstellen enthält. Verwenden Sie diesen Typ für Zahlen mit Dezimalstellen oder bei denen der Wertebereichstyp I nicht ausreicht.

Lokale Typen und globale Typen

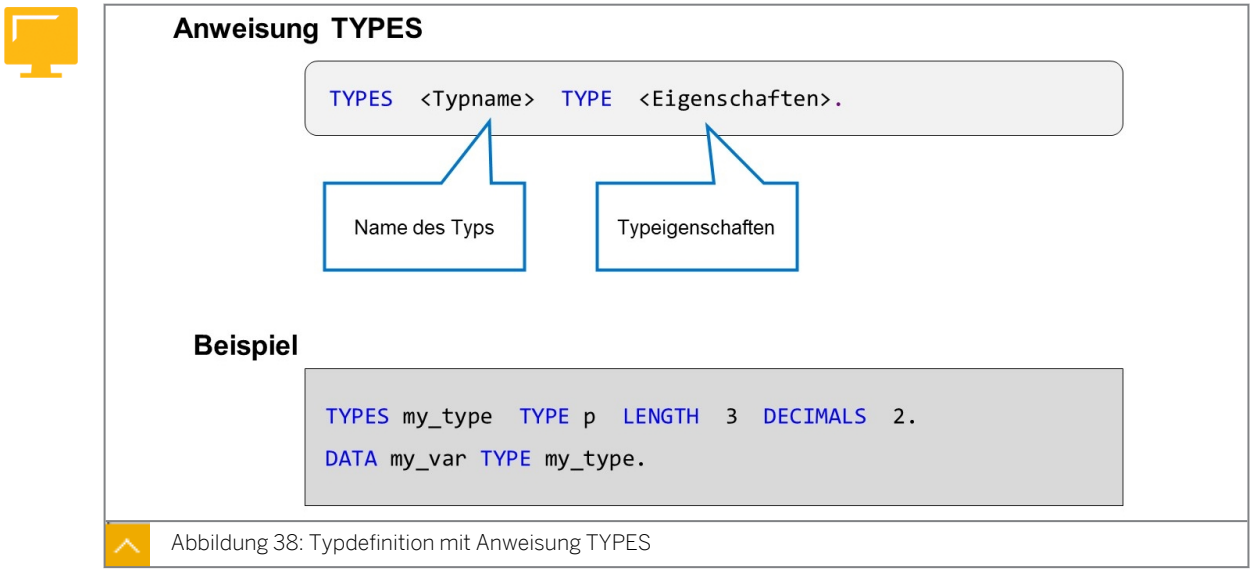


Abbildung 38: Typdefinition mit Anweisung TYPES

Anstatt eingebaute Typen direkt in der DATA-Anweisung zu verwenden, kann der Typ zunächst mit der Anweisung TYPES definiert werden. Der Typ kann dann in einer DATA-Anweisung hinter dem TYPE-Zusatz verwendet werden.

© Copyright. Alle Rechte vorbehalten.

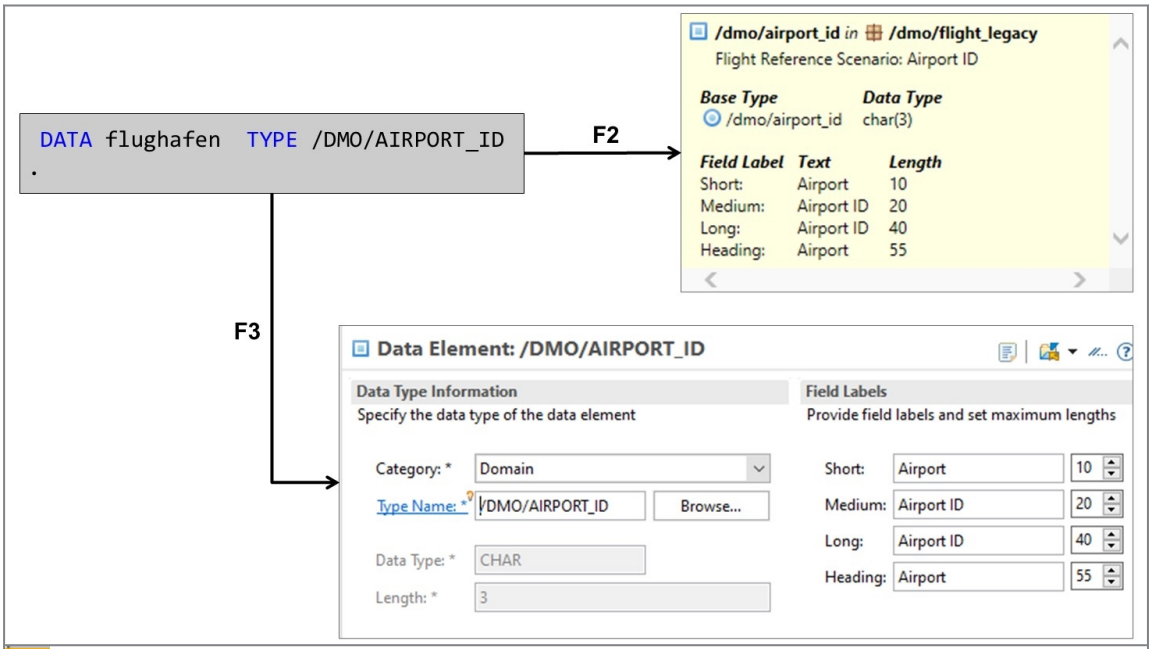


Abbildung 39: Datenelemente im ABAP Dictionary

In einem SAP-System gibt es Tausende von Business-Entitäten, wie Ländercode, Werk, Materialnummer, Geschäftsjahr, Kostenstelle usw. Technisch ist es möglich, diese Einheiten über die gerade erläuterten eingebauten ABAP-Typen in jedem einzelnen Programm zu definieren. Dies wäre aber sehr arbeitsintensiv und fehleranfällig. Stattdessen bietet SAP das ABAP Dictionary, das eine zentrale Ablage für wichtige Datentypen und auch das Werkzeug zum Anlegen von Datenbanktabellen ist.

Im ABAP Dictionary werden einzelne Business-Entitäten durch Datenelemente beschrieben. Im Beispiel ist die Variable „Airport“ mit dem Datenelement /DMO/AIRPORT_ID typisiert.

Wenn Sie die F2-Taste drücken, um die Details dieses Datentyps anzuzeigen, sehen Sie, dass dieser Typ technisch gesehen ein Zeichen der Länge 3 ist. Darüber hinaus stellt das Datenelement die Beschreibung „Flugreferenzszenario: Flughafen-ID” und vier Feldbezeichner unterschiedlicher Länge bereit.

Wenn Sie die Taste F3 drücken, um zur Definition des Typs zu navigieren, wird mit dem Editor für Datenelemente eine neue View geöffnet.

© Copyright. Alle Rechte vorbehalten.

Konstanten und Literale

Deklaration von Konstanten

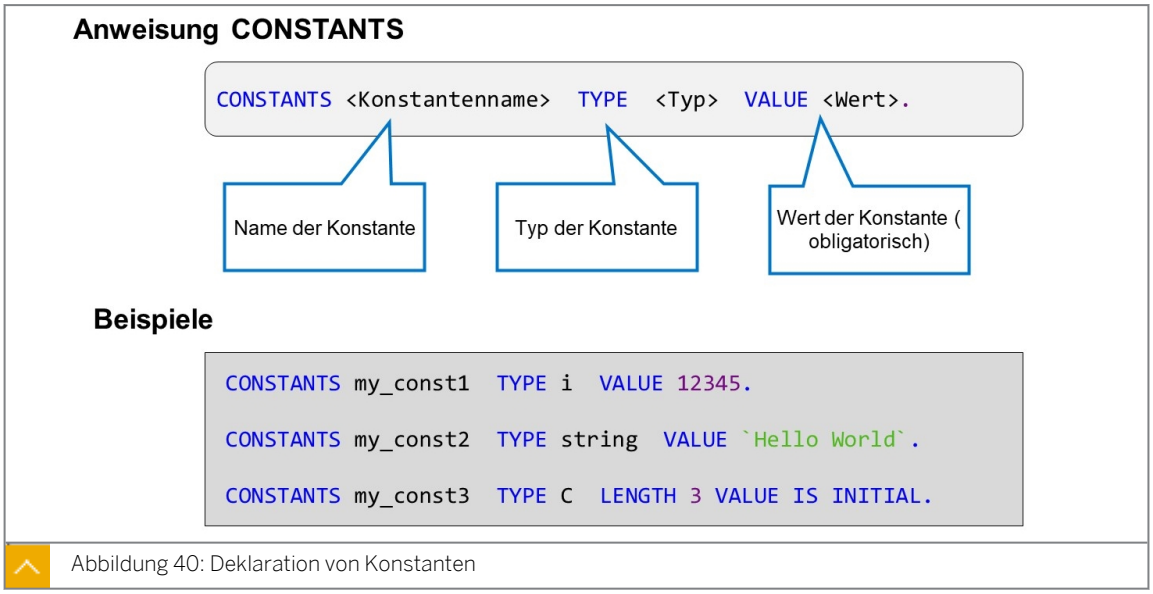


Abbildung 40: Deklaration von Konstanten

Eine Konstante ist ein Datenobjekt mit einem fest programmierten Wert, der während der Laufzeit nicht geändert werden darf. Ein Schreibzugriff auf eine Konstante führt zu einem Syntaxfehler.

In ABAP deklarieren Sie eine Konstante mit dem Schlüsselwort CONSTANTS. Eine CONSTANT-Anweisung besteht aus den gleichen Bestandteilen wie eine DATA-Anweisung. Der einzige Unterschied besteht darin, dass der Zusatz VALUE obligatorisch ist.

Der Zusatz VALUE kann in der Spezialform VALUE IS INITIAL verwendet werden, wenn der Wert der Konstante typgerecht initial sein soll.

Literale in ABAP

Literale sind anonyme Datenobjekte mit einem fest programmierten Wert. Literale werden häufig verwendet, um nicht-initiale Werte für Konstanten und nicht-initiale Startwerte für Variablen zu definieren.

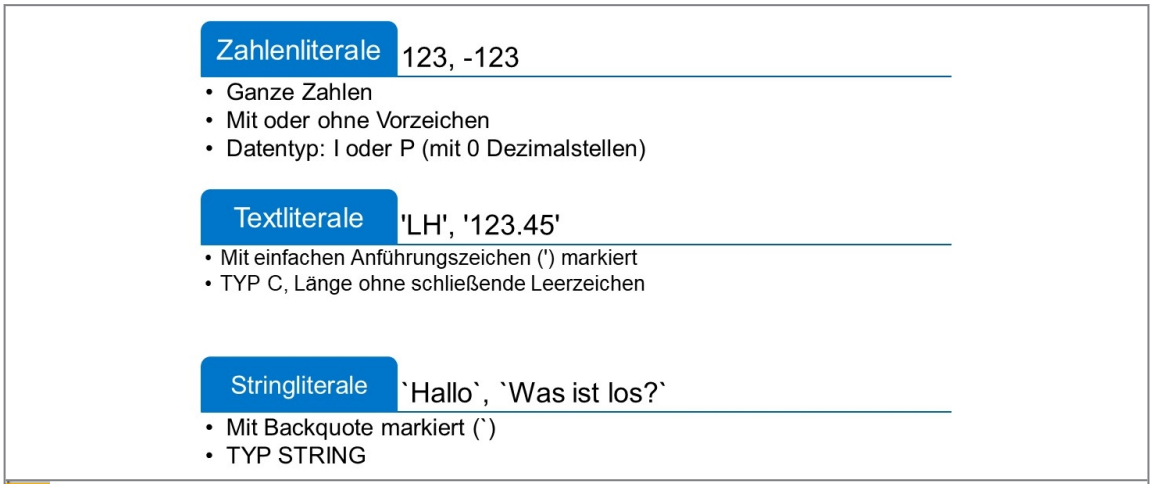


Abbildung 41: Literale in ABAP

© Copyright. Alle Rechte vorbehalten.

Technisch gesehen können Sie überall in Ihrem Quelltext Literale verwenden. Um die Lesbarkeit und Wartbarkeit zu unterstützen, wird empfohlen, stattdessen Konstanten zu definieren und zu verwenden.

ABAP kennt drei Arten von Literalen:

- Zahlenliterale sind ganze Zahlen mit oder ohne Vorzeichen. Zahlenliterale weisen in der Regel den Datentyp I auf. Nur wenn der Wert zu groß für den Datentyp I ist, wird Typ P mit ausreichender Länge und ohne Dezimalstellen verwendet.
- Textliterale sind Zeichenketten in einfachen Anführungszeichen. Textliterale weisen den Typ C auf. Die Länge wird aus dem Inhalt in Anführungszeichen abgeleitet. Nachstehende Leerzeichen werden ignoriert.
- Stringliterale sind Zeichenketten in einem Paar Backquotes (`). Stringliterale weisen den Typ STRING auf. Sie sollten verwendet werden, um Werte für Datenobjekte vom Typ String bereitzustellen.

Wertzuweisungen zu Datenobjekten

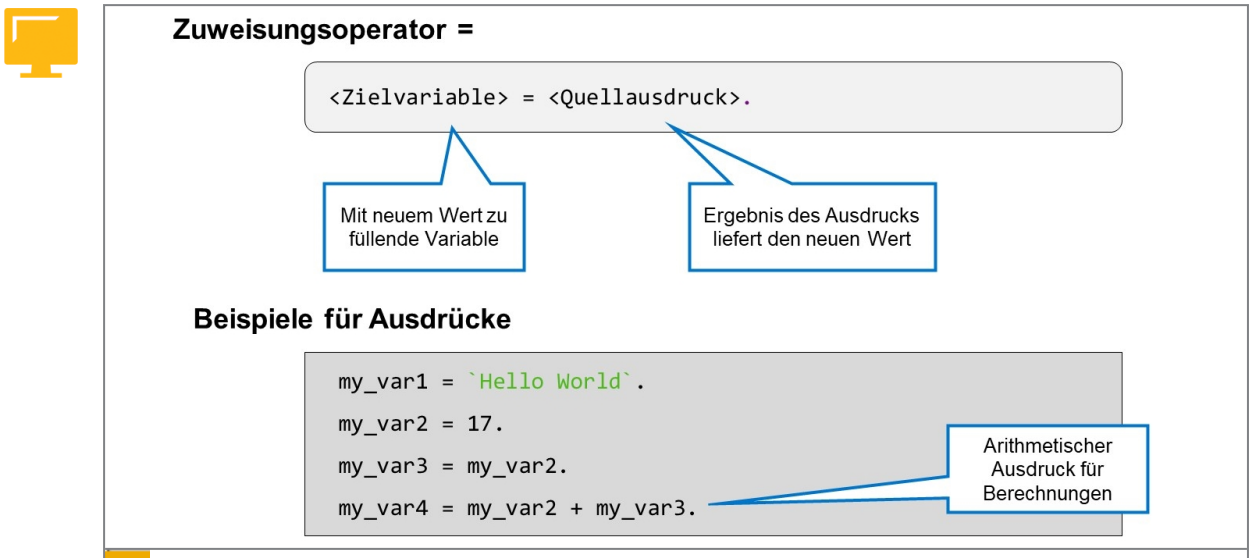


Abbildung 42: Wertzuweisungen und Ausdrücke

Verwenden Sie Wertzuweisungen, um den Wert von Variablen zu ändern.

In ABAP ist der Operator für Wertzuweisungen ein einfaches Gleichheitszeichen (=). Die Variable auf der linken Seite wird mit dem Wert des Ausdrucks auf der rechten Seite gefüllt.

Der einfachste mögliche Ausdruck ist ein einzelnes Datenobjekt, z.B. ein Literal oder eine andere Variable. ABAP unterstützt jedoch auch komplexere Ausdrücke wie den arithmetischen Ausdruck im Beispiel.

© Copyright. Alle Rechte vorbehalten.

Implizite Typkonvertierungen

```
   DATA my_int TYPE i.
 DATA my_pack  TYPE p LENGTH 8 DECIMALS 2.
 DATA my_char10  TYPE c LENGTH 10  VALUE 'abcdefghij'.
 DATA my_char4  TYPE c LENGTH 4.
DATA my_string  TYPE string.


* Implizite Typkonvertierungen
 my_int  = '1234'. " C -> I
    my_pack  = '-273.15'. " C -> P
    my_int =  my_char. " C -> I Potentieller Laufzeitfehler
    my_char4 = my_char10 " C -> C Möglicher Informationsverlust
    my_string  = 'Hallo '. " C -> STRING Ignoriert schließende Leerzeiche
n
```

Abbildung 43: Implizite Typkonvertierungen

Wie in der Abbildung dargestellt, sollten Sie aus folgenden Gründen Typkonvertierungen nach Möglichkeit vermeiden:

- Zusätzlicher Laufzeitverbrauch: Werte mit Typkonvertierungen benötigen mehr Laufzeit als Bewertungen mit identischen Typen.
- Potenzielle Laufzeitfehler: Einige Kombinationen aus Quelltyp und Zieltyp können zu Laufzeitfehlern führen. Wenn beispielsweise die Zielvariable einen numerischen Typ I und der Quellausdruck einen zeichenartigen Typ aufweist, gibt die Laufzeit einen Fehler aus, wenn der Text nicht in eine Zahl übersetzt werden kann.
- Möglicher Informationsverlust: Einige Kombinationen aus Quelltyp und Zieltyp führen nicht zu Laufzeitfehlern, sondern zu Datenverlust. Beispiel: Quelle und Ziel sind beide vom Typ C, aber der Typ der Zielvariable ist kürzer. Dann schneidet die Laufzeit den Wert der Quelle einfach ab.

© Copyright. Alle Rechte vorbehalten.

Variablen zurücksetzen

Anweisung CLEAR

```
CLEAR  <Variablenname>.
```

Beispiele

```
DATA  my_var1  TYPE  i.
DATA  my_var2  TYPE i VALUE 1234.
DATA  my_var3  TYPE  string.

my_var1 = my_var2.
my_var3 = `Hello World`.

CLEAR my_var1. " Wert wird auf 0 zurückgesetzt.
CLEAR my_var2. " Wert wird auf 0 zurückgesetzt!
CLEAR my_var3.      " String is now empty again
```

Abbildung 44: Variablen zurücksetzen

> Notiz:
> Die CLEAR-Anweisung ignoriert den Startwert aus dem VALUE-Zusatz. Nach CLEAR enthält die Variable immer den typgerechten Initialwert.

Inline-Deklarationen in Wertzuweisungen

Explizite Deklaration mit DATA-Anweisung

```
DATA my_var1  TYPE  string.
DATA my_var2  TYPE  i.
DATA my_var3  TYPE  i.
DATA my_var4 TYPE i

my_var1 = `Hello World`.
my_var2 = 17.
my_var3 = my_var2.
my_var4 = my_var2 + my_var3.
```

Inline-Deklaration mit DATA( )

```
 DATA(my_var1) = `Hello World`.
DATA(Mein_Var2) = 17.
DATA(Mein_Var3) = Mein_Var2.
DATA(Mein_Var4) = Mein_Var2 + Mein_Var3.
```

Typ der Variablen (links) abgeleitet vom Typ des Ausdrucks (rechts)

Abbildung 45: Inline-Deklarationen in Wertzuweisungen

Bisher haben wir die DATA-Anweisung verwendet, um unsere Variablen explizit zu deklarieren, bevor wir sie im ausführbaren Quelltext verwendet haben.

Alternativ können Sie eine Variable an der Stelle deklarieren, an der Sie sie als Erstes mit Daten füllen. Wir bezeichnen dies als Inline-Deklaration. Bei einer Inline-Deklaration umschließen Sie den Namen der neuen Variablen mit Klammern und fügen das Schlüsselwort DATA vor der öffnenden Klammer hinzu.

© Copyright. Alle Rechte vorbehalten.

Notiz:
Innerhalb der Klammern bzw. zwischen DATA und der öffnenden Klammer sind keine Leerzeichen erlaubt.

Nach der Inline-Deklaration kann die Variable wie jede explizit deklarierte Variable verwendet werden.

Der ABAP-Compiler leitet den Typ für die inline deklarierte Variable aus dem Kontext der Operandenposition ab. Beispiel: Wenn auf der linken Seite einer Wertzuweisung eine Inline-Deklaration verwendet wird, ist der Typ der neuen Variablen gleich dem Typ des Ausdrucks auf der rechten Seite.

Im Beispiel in der Abbildung „Inline-Deklarationen in Wertzuweisungen“ ist „my_var1“ vom Typ „string“, da dies der Typ des Stringliterals auf der rechten Seite ist. Die anderen drei Variablen sind vom Typ „integer“.

ZUSAMMENFASSUNG DER LEKTION
Nun können Sie

- Deklarieren Sie Datenobjekte.
- Ordnen Sie Werte zu.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Lektion 3

Daten verarbeiten

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Führen Sie arithmetische Berechnungen durch.
- Zeichenkettenverarbeitung anwenden.

Arithmetische Berechnungen

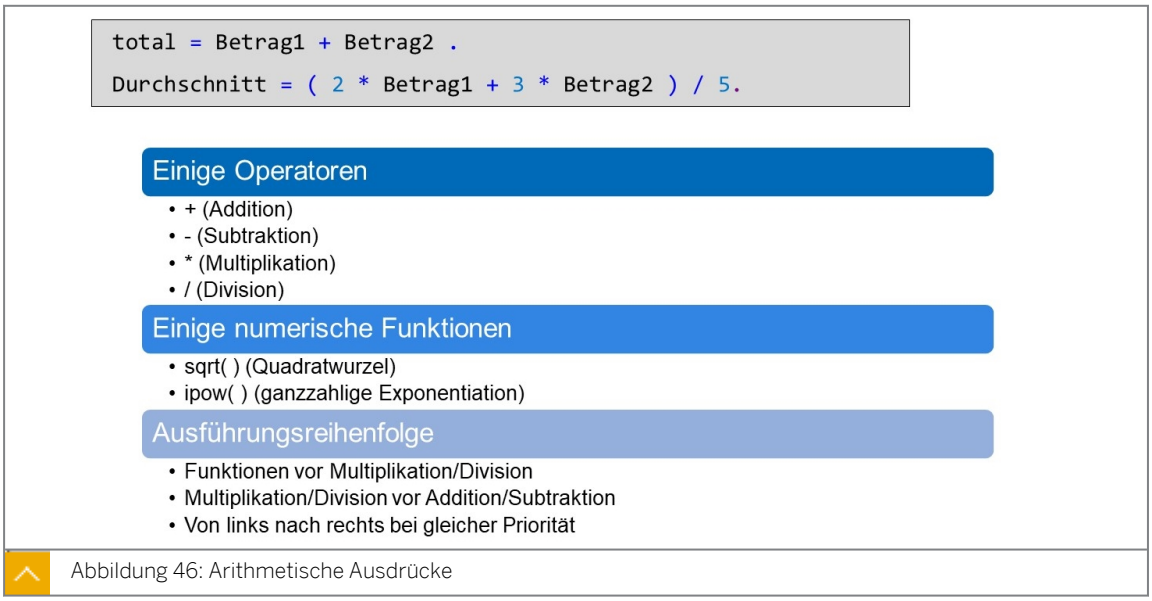


Abbildung 46: Arithmetische Ausdrücke

Arithmetische Ausdrücke sind ABAP-Ausdrücke mit einer Kombination aus Werten, Operatoren und Funktionen, die vom Laufzeitsystem verarbeitet werden, um ein Ergebnis zu erzeugen. Bei arithmetischen Ausdrücken hängt der Ergebnistyp vom Typ der Operanden ab, die als Eingabe in den Ausdruck verwendet werden.

Sie können einen arithmetischen Ausdruck an jeder beliebigen Position eines Leseoperanden verwenden, z.B. auf der rechten Seite einer Wertzuweisung.

Das erste Beispiel ist eine einfache Addition. Die Inhalte von „amount1“ und „amount2“ werden addiert, und das Ergebnis wird in der Summenvariable abgelegt.

Das zweite Beispiel ist etwas komplizierter. Vor dem Hinzufügen werden die Inhalte von amount1 und amount2 mit den Faktoren 2 und 3 gewogen. Das Ergebnis dieser Addition wird dann durch 5 geteilt, um einen gewichteten Durchschnitt zu berechnen.

Für grundlegende Arithmetik bietet ABAP die Operatoren + für Addition, - für Subtraktion, * für Multiplikation und / für Division. Darüber hinaus können Sie den Operator DIV für die

© Copyright. Alle Rechte vorbehalten.

Ganzzahldivision und MOD für den ganzzahligen Rest einer Division verwenden. Somit ist 6 DIV 4 gleich 1 und 6 MOD 4 gleich 2.

ABAP verfügt über eine Reihe integrierter Funktionen für verschiedene Aufgaben. Viele dieser Funktionen dienen der Stringverarbeitung. Hier sehen Sie jedoch einige Beispiele für numerische Funktionen. Sie verwenden die Funktion „sqrt( )“, um die Quadratwurzel zu ziehen, und die Funktion „ipow( )“, um eine Zahl auf eine ganzzahlige Potenz zu erhöhen.

In komplexen Ausdrücken mit mehr als einem Operator haben Multiplikation und Division Vorrang vor Addition und Subtraktion. Ausdrücke mit gleicher Rangfolge werden von links nach rechts verarbeitet.

Notiz:
Die ABAP-Syntax erfordert mindestens ein Leerzeichen zwischen Operatoren und Operanden. 1 + 1 ist korrekt. 1+1 führt zu einem Syntaxfehler.

Leerzeichen sind auch nach öffnenden Klammern und vor schließenden Klammern erforderlich.

Stringverarbeitung

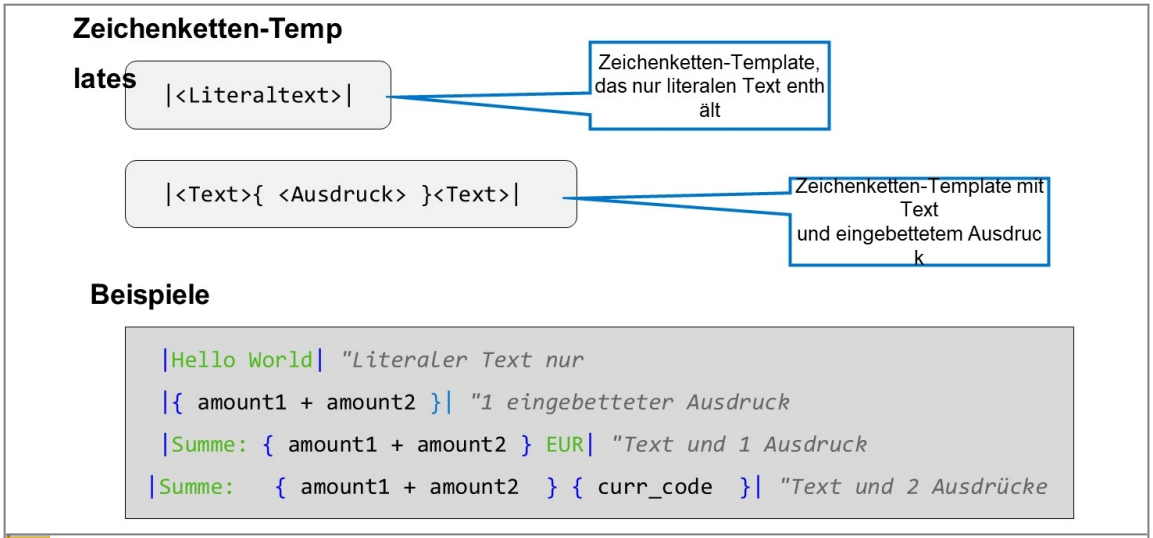


Abbildung 47: Zeichenketten-Templates

Zeichenketten-Templates sind ABAP-Ausdrücke vom Ergebnistyp „string“. Sie können Zeichenketten-Templates an jeder beliebigen Position eines Leseoperanden verwenden, z.B. auf der rechten Seite einer Wertzuweisung.

Ein Zeichenketten-Template beginnt und endet mit einem Pipe-Symbol (|). Das einfachste Zeichenketten-Template enthält lediglich literalen Text. In dieser Form unterscheidet sich ein Zeichenketten-Template nicht wirklich von einem Stringliteral.

Was ein Zeichenketten-Template von einem Stringliteral unterscheidet, ist die Fähigkeit, Ausdrücke einzubetten. Ein eingebetteter Ausdruck ist ein ABAP-Ausdruck, der von einem Paar geschweifter Klammern ({ und }) umgeben ist. Zur Laufzeit wertet ABAP den eingebetteten Ausdruck aus und übersetzt das Ergebnis in eine Zeichenkette. Im Endergebnis ersetzt diese Zeichenkette den eingebetteten Ausdruck (zusammen mit den umgebenden geschweiften Klammern).

© Copyright. Alle Rechte vorbehalten.

Notiz:
Die ABAP-Syntax erfordert mindestens ein Leerzeichen nach der öffnenden Klammer und mindestens ein Leerzeichen vor der schließenden Klammer.

Natürlich kann ein Zeichenketten-Template mehr als einen eingebetteten Ausdruck enthalten.

Innerhalb der geschweiften Klammern können beliebige ABAP-Ausdrücke platziert werden: arithmetische Ausdrücke, wie im obigen Beispiel, aber einzelne Variablen oder sogar Literale können als eingebettete Ausdrücke dienen.

Formatoption DATE

```
DATA the_date  TYPE d VALUE '19891130'.

|{ the_date   }| "=> '19891130', internes Format
|{ the_date   DATE = ISO  }| "=> '1989-11-30', Ausgabe im ISO-Format
|{ the_date   DATE = USER }| "=> '11/30/1989', '30.11.1989', …
```

Formatierungsoptionen für Zahlen

```
 DATA my_number TYPE p LENGTH 3 DECIMALS 2 VALUE '-273.15'.

 |{ my_number }|
|{ my_number NUMBER = USER }| "=> '-273.15', or '-273,15', …
|{ my_number SIGN = RIGHT }| "=> '273.15-'
|{ my_number STYLE = SCIENTIFIC }| "=> '-2.7315E+02'
```

Abbildung 48: Formatoptionen in Zeichenketten-Templates

Ein wichtiger Anwendungsfall für Zeichenketten-Templates ist die kontrollierte Formatierung der Daten für die Ausgabe.

Im ersten Beispiel ist die Variable „the_date“ vom Typ D und enthält ein Datum im internen Format (Rohformat) JJJJMMTT (wobei JJJJ für das Jahr, MM für den zweistelligen Monat und TT für das zweistellige Datum steht). Wenn Sie die Variable „the_date“ als eingebetteten Ausdruck in einem Zeichenketten-Template verwenden, entspricht das Ergebnis dem internen Format. Wenn Sie jedoch die Formatoption „DATE = <Datumsformat>“ innerhalb der geschweiften Klammern hinzufügen, formatiert das System den Wert als Datum. Wenn Sie „DATE = ISO“ hinzufügen, erfolgt die Ausgabe im ISO-Format. Mit „DATE = USER“ hängt das Ausgabeformat von den Benutzereinstellungen des aktuellen Benutzers ab.

Das zweite Beispiel veranschaulicht einige der Optionen, die Sie für die Formatierung von Zahlen verwenden können. Mit NUMBER steuern Sie die allgemeine Formatierung von Zahlen, z.B., ob ein Dezimalpunkt oder ein Dezimalkomma verwendet wird. Mit SIGN steuern Sie die Position des Vorzeichens und ob ein Pluszeichen (+) angezeigt wird oder nicht. Mit STYLE können Sie aus mehreren vordefinierten Formaten wählen, z.B. einem wissenschaftlichen Format oder einem technischen Format.

© Copyright. Alle Rechte vorbehalten.

Operator &&

```
 DATA result  TYPE string.
DATA part1  TYPE string  VALUE `Hello` .
 DATA part2  TYPE string  VALUE `World`  .
 result = part1 && part2.
 result = part1 && | | && part2.
```

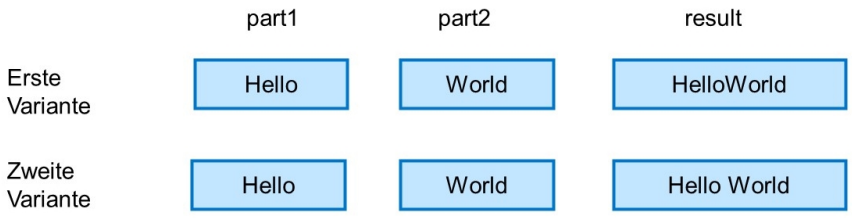


Abbildung 49: Zeichenketten verbinden

Sie können Felder mit dem Verkettungsoperator && verbinden. Sie können eine beliebige Kombination aus Datenobjekten und Zeichenkettenausdrücken verbinden.

Die Bestandteile des Ausdrucks werden ohne Leerzeichen oder Trennzeichen verbunden. Falls Sie Leerzeichen oder ein anderes Trennzeichen benötigen, müssen Sie es selbst, wie im zweiten Beispiel dargestellt, als Teil des Ausdrucks einfügen.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Übung 4

Variablen deklarieren und Daten verarbeiten

In dieser Übung erstellen Sie ein Programm, das eine Berechnung basierend auf zwei Zahlen durchführt, dann das Ergebnis formatiert und ausgibt.

Vorlage:

keine

Lösung:

/LRN/CL_S4D400_BTS_COMPUTE (globale Klasse)

Aufgabe 1: Variablendeklaration
Legen Sie in Ihrem eigenen Paket eine neue globale Klasse an, und definieren Sie numerische Variablen für die Eingabe.

1. Legen Sie eine neue globale Klasse an, die das Interface **`IF_OO_ADT_CLASSRUN`** implementiert (Namensvorschlag: **`ZCL_##_COMPUTE`**, wobei ## für Ihre Gruppennummer steht).
2. Deklarieren Sie in der Methode *IF_OO_ADT_CLASSRUN~MAIN* zwei Variablen für Ganzzahlwerte (Namensvorschläge: **`number1`** und **`number2`**).
3. Implementieren Sie für jede der beiden numerischen Variablen eine Wertzuweisung. Weisen Sie der ersten Variable einen negativen Wert und der zweiten Variable einen positiven Wert zu.

Aufgabe 2: Datenverarbeitung
Berechnen Sie ein Verhältnis, und schreiben Sie das Ergebnis in die Konsole.

1. Implementieren Sie einen arithmetischen Ausdruck, der das Verhältnis der beiden Zahlen berechnet. Speichern Sie das Ergebnis in einer Variablen, die Sie mit einer Inline-Deklaration deklarieren (Namensvorschlag: **`result`**).
2. Implementieren Sie ein Zeichenketten-Template, das einen Text wie diesen zusammensetzt:„6 / 3 = 2”, mit den tatsächlichen Werten der Variablen anstelle von 6, 3 und 2. Speichern Sie das Ergebnis in einer Variablen, die Sie mit einer Inline-Deklaration deklarieren (Namensvorschlag: **`output`**).
3. Rufen Sie die Methode **`out->write( … )`** auf, um den Text in die Konsole zu schreiben.

Aufgabe 3: Aktivieren und testen
Aktivieren und testen Sie die Konsolen-App.

1. Aktivieren Sie die Klasse.
2. Führen Sie die Konsolen-App aus.
3. Testen Sie die App mit verschiedenen Eingabewerten.

© Copyright. Alle Rechte vorbehalten.

Was ist das Ergebnis von -8 / 3?

Wie können Sie dieses Ergebnis erklären?

Aufgabe 4: Genauigkeit des Ergebnisses kontrollieren
Stellen Sie sicher, dass das Ergebnis auf einen Wert mit 2 Dezimalstellen und nicht auf einen ganzzahligen Wert gerundet wird.

1. Anstelle einer Inline-Deklaration verwenden Sie eine explizite Deklaration der Variable **`result`**. Verwenden Sie einen numerischen Typ, der es ermöglicht, die Anzahl der Dezimalstellen festzulegen.
2. Aktivieren Sie die Klasse, und testen Sie sie erneut.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Lösung 4

Variablen deklarieren und Daten verarbeiten

In dieser Übung erstellen Sie ein Programm, das eine Berechnung basierend auf zwei Zahlen durchführt, dann das Ergebnis formatiert und ausgibt.

Vorlage:

keine

Lösung:

/LRN/CL_S4D400_BTS_COMPUTE (globale Klasse)

Aufgabe 1: Variablendeklaration
Legen Sie in Ihrem eigenen Paket eine neue globale Klasse an, und definieren Sie numerische Variablen für die Eingabe.

1. Legen Sie eine neue globale Klasse an, die das Interface **`IF_OO_ADT_CLASSRUN`** implementiert (Namensvorschlag: **`ZCL_##_COMPUTE`**, wobei ## für Ihre Gruppennummer steht).
   a) Wählen Sie *File* → *New* → *ABAP Class*.
   b) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_COMPUTE`** ein, wobei ## für Ihre Gruppennummer steht.
   c) Geben Sie eine Beschreibung ein.
   d) Wählen Sie im Gruppenrahmen *Interfaces* die Option *Add*.
   e) Geben Sie **`IF_OO_ADT_CLASSRUN`** ein. Wenn das Interface in der Trefferliste angezeigt wird, fügen Sie es mit Doppelklick zur Klassendefinition hinzu.
   f) Wählen Sie *Next*.
   g) Wählen Sie Ihren Transportauftrag aus, und wählen Sie *Finish*.
2. Deklarieren Sie in der Methode *IF_OO_ADT_CLASSRUN~MAIN* zwei Variablen für Ganzzahlwerte (Namensvorschläge: **`number1`** und **`number2`**).
   a) Fügen Sie folgenden Code hinzu:

```
DATA number1 TYPE i.
DATA number2 TYPE i.
```

3. Implementieren Sie für jede der beiden numerischen Variablen eine Wertzuweisung. Weisen Sie der ersten Variable einen negativen Wert und der zweiten Variable einen positiven Wert zu.

© Copyright. Alle Rechte vorbehalten.

a) Fügen Sie folgenden Code hinzu:

```
number1 = -8.
number2 =  3.
```

Aufgabe 2: Datenverarbeitung
Berechnen Sie ein Verhältnis, und schreiben Sie das Ergebnis in die Konsole.

1. Implementieren Sie einen arithmetischen Ausdruck, der das Verhältnis der beiden Zahlen berechnet. Speichern Sie das Ergebnis in einer Variablen, die Sie mit einer Inline-Deklaration deklarieren (Namensvorschlag: **`result`**).
   a) Fügen Sie folgenden Code hinzu:

```
DATA(result) = number1 / number2.
```

2. Implementieren Sie ein Zeichenketten-Template, das einen Text wie diesen zusammensetzt:„6 / 3 = 2", mit den tatsächlichen Werten der Variablen anstelle von 6, 3 und 2. Speichern Sie das Ergebnis in einer Variablen, die Sie mit einer Inline-Deklaration deklarieren (Namensvorschlag: **`output`**).
   a) Fügen Sie folgenden Code hinzu:

```
DATA(output) = |{ number1 } / { number2 } = { result }|.
```

3. Rufen Sie die Methode **`out->write( … )`** auf, um den Text in die Konsole zu schreiben.
   a) Fahren Sie wie in der vorangegangenen Übung fort.

Aufgabe 3: Aktivieren und testen
Aktivieren und testen Sie die Konsolen-App.

1. Aktivieren Sie die Klasse.
   a) Drücken Sie **`Strg + F3`**, um die Klasse zu aktivieren.
2. Führen Sie die Konsolen-App aus.
   a) Drücken Sie **`F9`**, um die Konsolen-App auszuführen.
3. Testen Sie die App mit verschiedenen Eingabewerten.
   a) Bearbeiten Sie die Wertzuweisungen, aktivieren Sie sie, und führen Sie sie wie zuvor aus.

   Was ist das Ergebnis von -8 / 3?

   Das Ergebnis ist -3.

   Wie können Sie dieses Ergebnis erklären?

   Aufgrund der Inline-Deklaration ist die Variable **`result`** vom Typ „Integer", und ABAP rundet das Ergebnis über die Rundungsregel „half-up".

Aufgabe 4: Genauigkeit des Ergebnisses kontrollieren
Stellen Sie sicher, dass das Ergebnis auf einen Wert mit 2 Dezimalstellen und nicht auf einen ganzzahligen Wert gerundet wird.

© Copyright. Alle Rechte vorbehalten.

1. Anstelle einer Inline-Deklaration verwenden Sie eine explizite Deklaration der Variable **`result`**. Verwenden Sie einen numerischen Typ, der es ermöglicht, die Anzahl der Dezimalstellen festzulegen.
   a) Fügen Sie am Anfang der Methode **`if_oo_adt_classrun~main`** folgenden Code hinzu:

```
    DATA result TYPE p LENGTH 8 DECIMALS 2.
```

   b) Ersetzen Sie in der Codezeile mit der Inline-Deklaration `DATA(result)` mit `result`.

2. Aktivieren Sie die Klasse, und testen Sie sie erneut.
   a) Fahren Sie wie zuvor fort.
   b) Vergleichen Sie Ihren Quelltext mit dem folgenden Auszug aus der Musterlösung:

```
 METHOD if_oo_adt_classrun~main.

* Declarations
**************************

    DATA number1 TYPE i.
    DATA number2 TYPE i.

    DATA result TYPE p LENGTH 8 DECIMALS 2.


* Input Values
**************************

    number1 = -8.
    number2 =  3.

* Calculation
**************************

*    DATA(result) = number1 / number2.   "with inline declaration

   result = number1 / number2.

    DATA(output) = |{ number1 } / { number2 } = { result }|.

* Output
**************************

    out->write( output ).

  ENDMETHOD.
```

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Führen Sie arithmetische Berechnungen durch.
- Zeichenkettenverarbeitung anwenden.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Lektion 4

Kontrollstrukturen in ABAP verwenden

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- bedingte Verzweigung implementieren
- Ausnahmen behandeln.
- Iterationen implementieren.

Bedingte Verzweigungen

IF ... ELSEIF ... ELSE. ... ENDIF.

```
IF <logische_Bedingung_1>.
    <Quelltextblock_1>
ELSEIF <logische_Bedingung_2>.
    <Quelltextblock_2>
ELSEIF <logische_Bedingung_3>.
    <Quelltextblock_3>
ELSE.
    <ELSE_Block>
ENDIF.
```

Optional Teil

- IF und ENDIF sind obligatorisch
- ELSEIF und ELSE sind optional
- Erster wahrer Ausdruck hat Vorrang
- Wenn alle Ausdrücke falsch sind ELSE-Block wird ausgeführt
- Einer oder kein Quelltextblock ausgeführt, nie zwei davon

Achtung: Trennzeichen (.) nach jedem logischen Ausdruck und nach ELSE erforderlich!

Abbildung 50: Verzweigungen mit frei definierten Bedingungen

Eine bedingte Verzweigung ist eine Kontrollstruktur, mit der Sie die Ausführung von Quelltext von logischen Bedingungen abhängig machen können.

Die häufigste bedingte Verzweigung besteht aus einem Paar der Schlüsselwörter IF und ENDIF. Der Quelltextblock zwischen IF und ENDIF wird nur ausgeführt, wenn die Bedingung nach IF erfüllt ist.

Sie können weitere Quelltextblöcke hinzufügen, indem Sie die Struktur IF ... ENDIF um maximal ein Schlüsselwort ELSE und eine beliebige Anzahl von Schlüsselwörtern ELSEIF erweitern. Durch das Hinzufügen des Schlüsselworts ELSE stellen Sie sicher, dass immer genau einer der Quelltextblöcke ausgeführt wird. Wenn ELSE fehlt, kann es vorkommen, dass keiner der Quelltextblöcke ausgeführt wird.

Der auszuführende Quelltextblock wird wie folgt ermittelt:

- Zuerst wird die IF-Bedingung ausgewertet. Wenn diese erfüllt ist, wird der zugehörige Quelltextblock ausgeführt, und das Programm wird nach ENDIF fortgesetzt.

© Copyright. Alle Rechte vorbehalten.

- Nur wenn die IF-Bedingung nicht erfüllt ist, wird die Bedingung nach dem ersten ELSEIF ausgewertet. Wenn diese erfüllt ist, wird der zugehörige Quelltextblock ausgeführt, und das Programm wird nach ENDIF fortgesetzt.
- Dies geschieht nacheinander für alle ELSEIF-Bedingungen. Wenn keine der Bedingungen erfüllt ist und die Struktur ELSE enthält, wird der Quelltextblock nach ELSE ausgeführt. Andernfalls wird keiner der Quelltextblöcke ausgeführt.

Hinweis:
Im Gegensatz zu vielen anderen Programmiersprachen benötigt ABAP ein Trennzeichen (.) nach jeder der logischen Bedingungen und selbst nach dem Schlüsselwort ELSE.

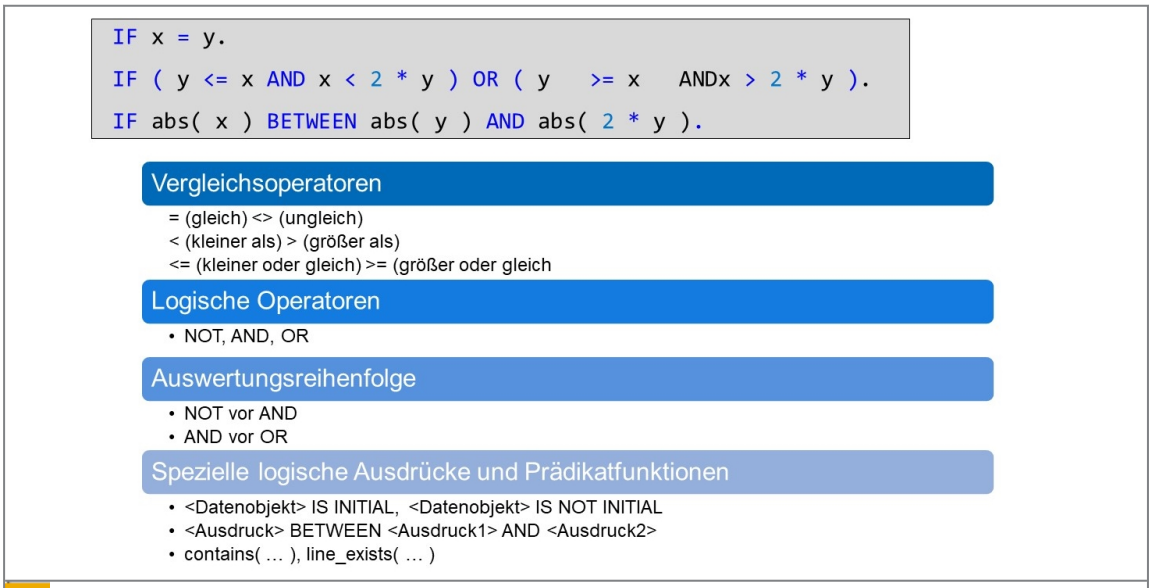


Abbildung 51: Logische Bedingungen

Logische Bedingungen sind eine Kombination aus Vergleichen, logischen Operationen, Ausdrücken und Funktionen, die das Laufzeitsystem auswertet, um zu entscheiden, ob die Bedingung wahr oder falsch ist.

Der häufigste Anwendungsfall für logische Bedingungen ist nach den Schlüsselwörtern IF oder ELSEIF in einer „IF ... ENDIF.“-Struktur.

Das erste Beispiel ist ein einfacher Vergleich: Die Bedingung ist wahr, wenn die beiden Datenobjekte x und y den gleichen Wert haben.

Das zweite Beispiel ist etwas differenzierter: Entweder ist der Wert von x größer oder gleich y und kleiner als das Doppelte des Werts von y oder er ist kleiner oder gleich y und größer als der doppelte Wert von y.

Das dritte Beispiel verwendet die arithmetische Funktion „abs( )“ und den logischen Ausdruck „BETWEEN <Ausdruck1> AND <Ausdruck2>“. Die Bedingung ist wahr, wenn der absolute Wert von x zwischen dem absoluten Wert von y und dem absoluten Wert von zweimal y liegt.

Für einfache Wertevergleiche können Sie die Operatoren =, <>, >, <, >= und <= verwenden. Sie können nicht nur die Werte von Datenobjekten vergleichen, sondern auch die Werte vieler anderer Ausdrücke, wie beim arithmetischen Ausdruck „2 * y“ im Beispiel.

© Copyright. Alle Rechte vorbehalten.

Notiz:
ABAP verwendet das gleiche Symbol (=) für Wertzuweisungen und für Wertevergleiche. Die Unterscheidung erfolgt anhand der Position.

Sie können die Operatoren AND und OR verwenden, um logische Ausdrücke zu kombinieren, und den Operator NOT, um einen Ausdruck zu negieren. Ohne Klammern bindet NOT stärker als AND und AND stärker als OR.

ABAP kennt einige spezielle logische Ausdrücke:

- <Datenobjekt> IS INITIAL ist wahr, wenn <Datenobjekt> seinen typgerechten Initialwert enthält
- <Datenobjekt> IS NOT INITIAL ist wahr, wenn <Datenobjekt> einen Wert enthält, der vom typgerechten Initialwert abweicht
- <Datenobjekt> BETWEEN <Ausdruck1> AND <Ausdruck2>

Einige spezielle ABAP-Funktionen sind Prädikatfunktionen. Dies bedeutet, dass sie selbst logische Bedingungen sind. „Contains( )“ ist eine Funktion, die zeichenartige Werte vergleicht und „line_exists( )“ führt eine Existenzprüfung für eine Zeile einer internen Tabelle durch.

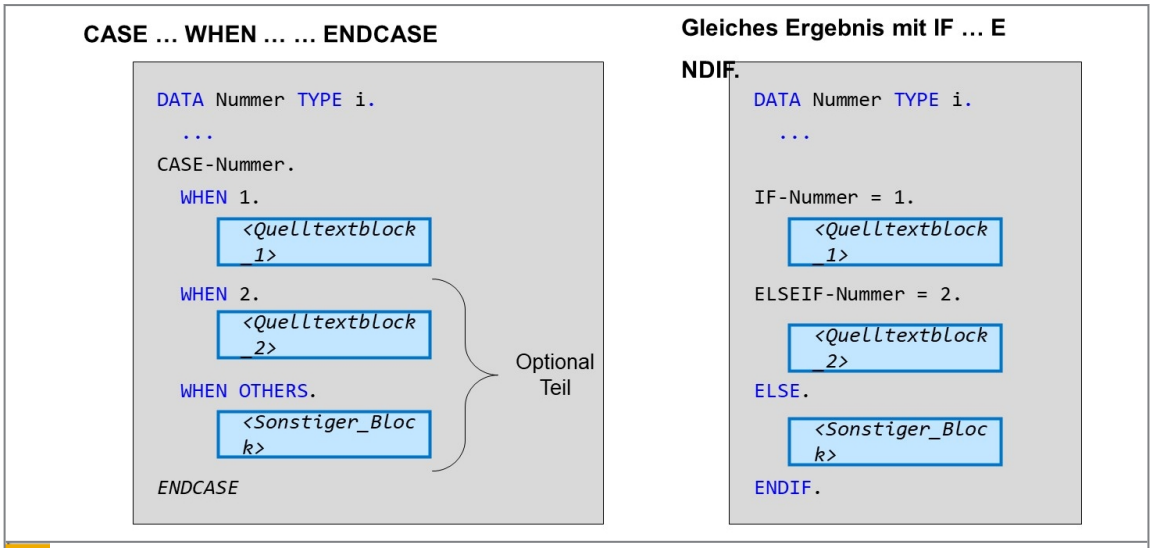


Abbildung 52: Verzweigung mit „Ist-gleich“-Vergleichen

Eine zweite Technik für bedingte Verzweigungen ist das CASE ... WHEN .. ENDCASE Kontrollstruktur.

Bedingte Verzweigung mit CASE .. ENDCASE ist ein Spezialfall der allgemeineren Verzweigung mit IF ... ENDIF. Sie können CASE in Situationen verwenden, in denen die Verzweigung vom Wert eines einzelnen Datenobjekts abhängt, das Sie nacheinander mit einer Reihe möglicher Werte vergleichen, indem Sie jedes Mal einen „Ist-gleich“-Vergleich verwenden.

Im Beispiel wird der Wert des Datenobjekts number mit den Werten 1 und 2 verglichen. Wenn der Wert gleich 1 ist, wird <code_block_1> ausgeführt, und wenn der Wert euqals 2 ist, wird stattdessen <code_blocl_2> ausgeführt. Bei jedem anderen Wert wird der Quelltextblock nach WHEN OTHERS ausgeführt.

© Copyright. Alle Rechte vorbehalten.

Alle bedingten Verzweigungen mit CASE ... ENDCASE könnten auch mit einer IF ... ENDIF-Struktur implementiert werden. Dies wird durch das Beispiel auf der rechten Seite veranschaulicht.

Hinweis:
Sie sollten CASE ... ENDCASE verwenden, wenn Sie sich mit dem Sonderfall befassen, um die Lesbarkeit Ihres Quelltexts zu erhöhen.

Ausnahmebehandlung

Exceptions

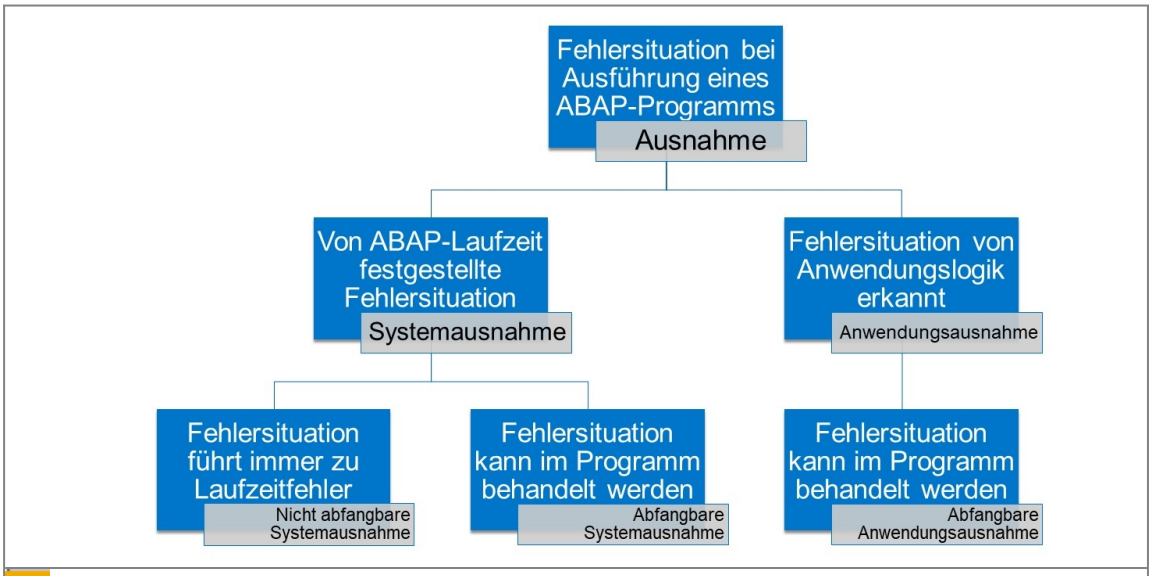


Abbildung 53: Exceptions

In ABAP ist eine Ausnahme eine Fehlersituation bei der Ausführung eines ABAP-Programms. Der Quelltext, der die Fehlersituation erkennt, löst eine Ausnahme aus.

Je nachdem, wodurch die Ausnahme ausgelöst wird, wird zwischen Systemausnahmen und Anwendungsausnahmen unterschieden.

Ohne weitere Maßnahmen führen Ausnahmen zu Laufzeitfehlern. Ein Laufzeitfehler bricht das Programm ab. Er wird standardmäßig in einem Kurzdump dokumentiert.

Sie können den Laufzeitfehler vermeiden, wenn die betreffende Ausnahme behandelbar ist. Eine behandelbare Ausnahme kann im Programm mit den Anweisungen TRY ... CATCH ... ENDTRY behandelt werden.

Alle Anwendungsausnahmen und viele Systemausnahmen sind behandelbar. Später in diesem Kurs erfahren Sie, wie Sie Anwendungsausnahmen auslösen. An dieser Stelle konzentrieren wir uns auf die Behandlung behandelbarer Systemausnahmen.

© Copyright. Alle Rechte vorbehalten.

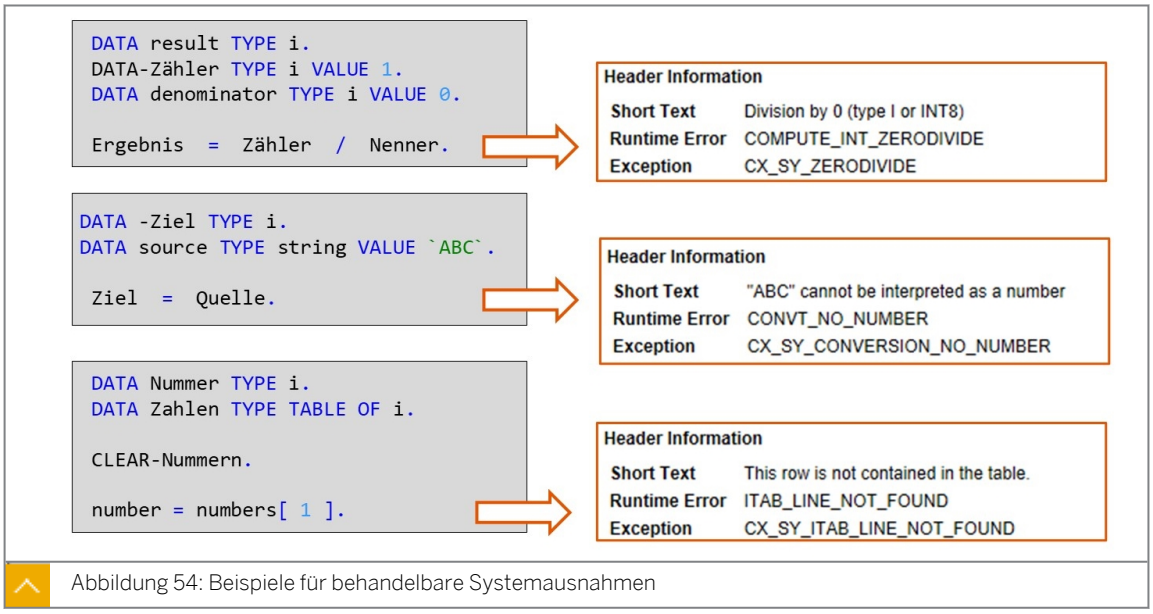


Abbildung 54: Beispiele für behandelbare Systemausnahmen

In der Abbildung „Beispiele für behandelbare Systemausnahmen“ sehen Sie einige Beispiele für Laufzeitfehler:

- Das erste Codebeispiel versucht eine Division mit Null als Nenner. Dieser Vorgang ist nicht definiert.
- Das zweite Codebeispiel möchte eine Zeichenkette in eine ganze Zahl konvertieren, aber die Zeichenkette kann nicht als Zahl interpretiert werden.
- Das dritte Codebeispiel versucht, die erste Zeile aus einer internen Tabelle zu lesen. Die interne Tabelle ist jedoch initial, sodass keine erste Zeile gefunden werden kann.

Wenn Sie die Codebeispiele auf der linken Seite ausführen, löst die ABAP-Laufzeit behandelbare Systemausnahmen aus. Diese Ausnahmen werden noch nicht vom Programm behandelt, was zu Laufzeitfehlern führt.

Bei den Screenshots auf der rechten Seite handelt es sich um Snippets aus den Kurzdumps, die durch diese Laufzeitfehler erzeugt werden. Neben einem Kurztext zur Beschreibung der Situation sehen Sie die ID des Laufzeitfehlers sowie die ID der nicht behandelten Ausnahme.

© Copyright. Alle Rechte vorbehalten.

Ausnahmebehandlung

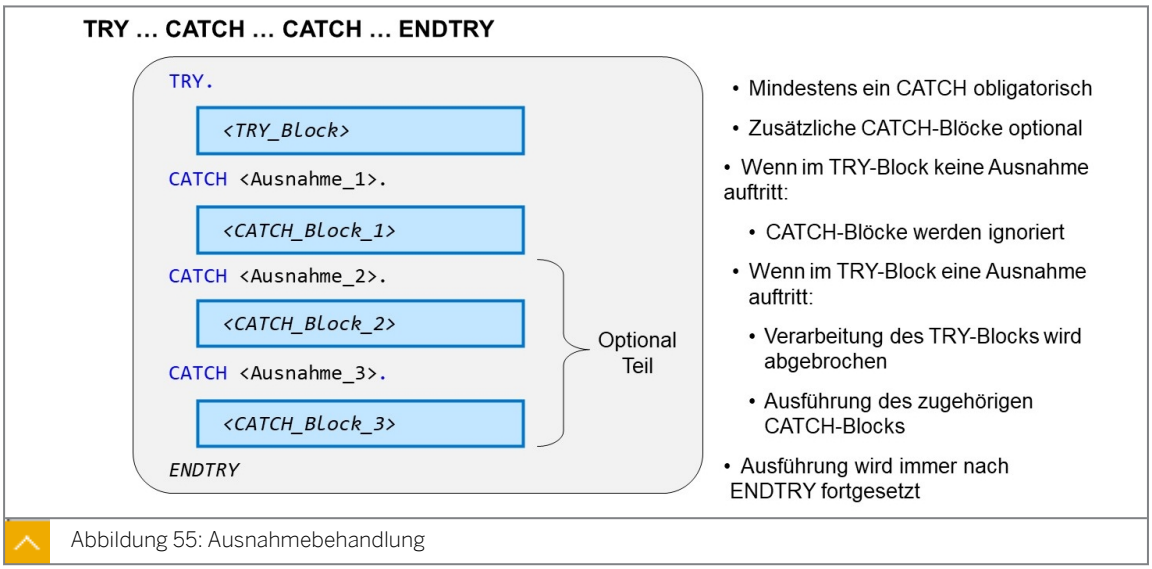


Abbildung 55: Ausnahmebehandlung

Um zu verhindern, dass ein Programm aufgrund einer behandelbaren Ausnahme abgebrochen wird, muss der Quelltext, aus dem die Ausnahme stammt, mit den Anweisungen TRY und ENDTRY umgeben werden. Dadurch wird der Quelltext Teil des TRY-Blocks der TRY ... ENDTRY-Struktur.

Vor der Anweisung ENDTRY müssen Sie eine CATCH-Anweisung hinzufügen, gefolgt von der ID der Ausnahme, die Sie behandeln möchten. Optional können Sie mehrere CATCH-Anweisungen hinzufügen, um mehrere verschiedene Ausnahmen zu behandeln. Auf jede CATCH-Anweisung muss Quelltext folgen, um diese Ausnahme zu behandeln. Dieser Quelltext wird als CATCH-Block bezeichnet.

Wenn die Programmausführung die TRY-Anweisung erreicht, wird mit dem Quelltext im TRY-Block fortgefahren. Dann können drei Dinge passieren:

1. Wenn während des TRY-Blocks keine Ausnahme ausgelöst wird, werden die CATCH-Blöcke ignoriert. Die Ausführung wird nach der Anweisung ENDTRY fortgesetzt.
2. Wenn während des TRY-Blocks eine Ausnahme ausgelöst wird, für die es ein passendes CATCH gibt, wird die Ausführung des TRY-Blocks beendet und der CATCH-Block für diese Ausnahme ausgeführt. Danach wird die Ausführung nach der Anweisung ENDTRY fortgesetzt.
3. Wenn während des TRY-Blocks eine Ausnahme ausgelöst wird, für die kein passendes CATCH vorhanden ist, bricht das Programm mit einem Laufzeitfehler ab.

© Copyright. Alle Rechte vorbehalten.

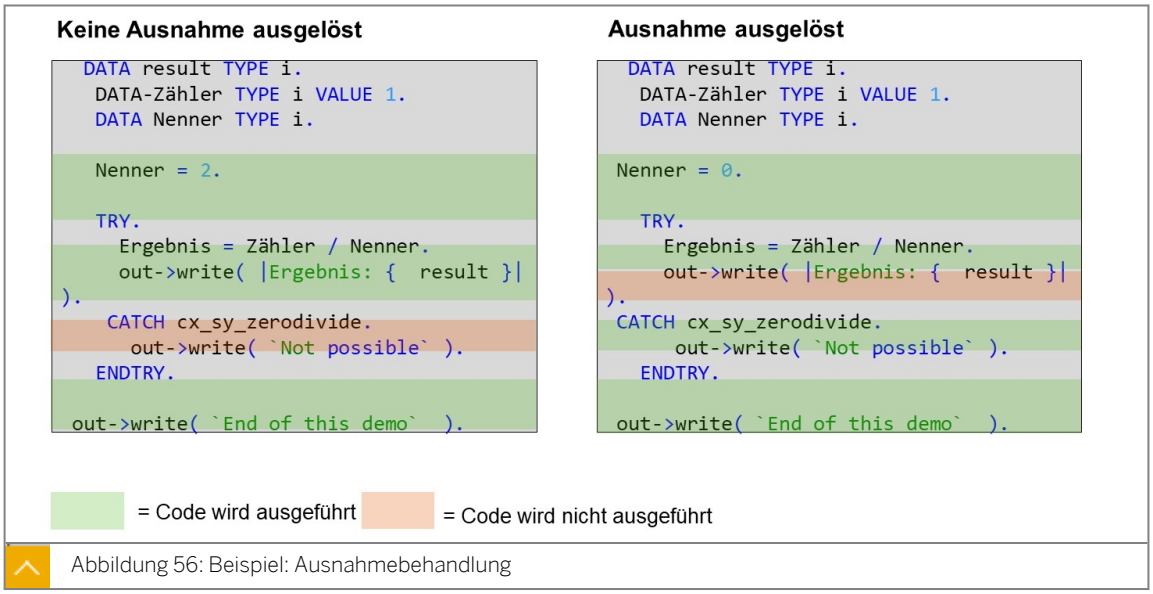


Abbildung 56: Beispiel: Ausnahmebehandlung

Das Beispiel veranschaulicht die Behandlung der Ausnahme CX_SY_ZERODIVIDE, die ausgelöst wird, wenn in einer Division der Nenner Null ist.

Der Quelltext auf der linken und rechten Seite ist identisch – mit Ausnahme des Werts, der der Variable für den Nenner zugeordnet ist. Rote und grüne Hintergrundfarben werden verwendet, um zwischen ausgeführtem und nicht ausgeführtem Quelltext zu unterscheiden.

Auf der linken Seite wird der gesamte TRY-Block ausgeführt und der CATCH-Block ignoriert. Auf der rechten Seite wird der TRY-Block ausgeführt, jedoch nur bis zur Quelltextzeile, in der das System die Ausnahme auslöst. Der Rest des TRY-Blocks wird nicht ausgeführt. Stattdessen wird der CATCH-Block ausgeführt.

Der Quelltext nach ENDTRY wird in beiden Fällen ausgeführt. Er ist unabhängig von der erfolgreichen oder nicht erfolgreichen Ausführung des TRY-Blocks.

Iterationen

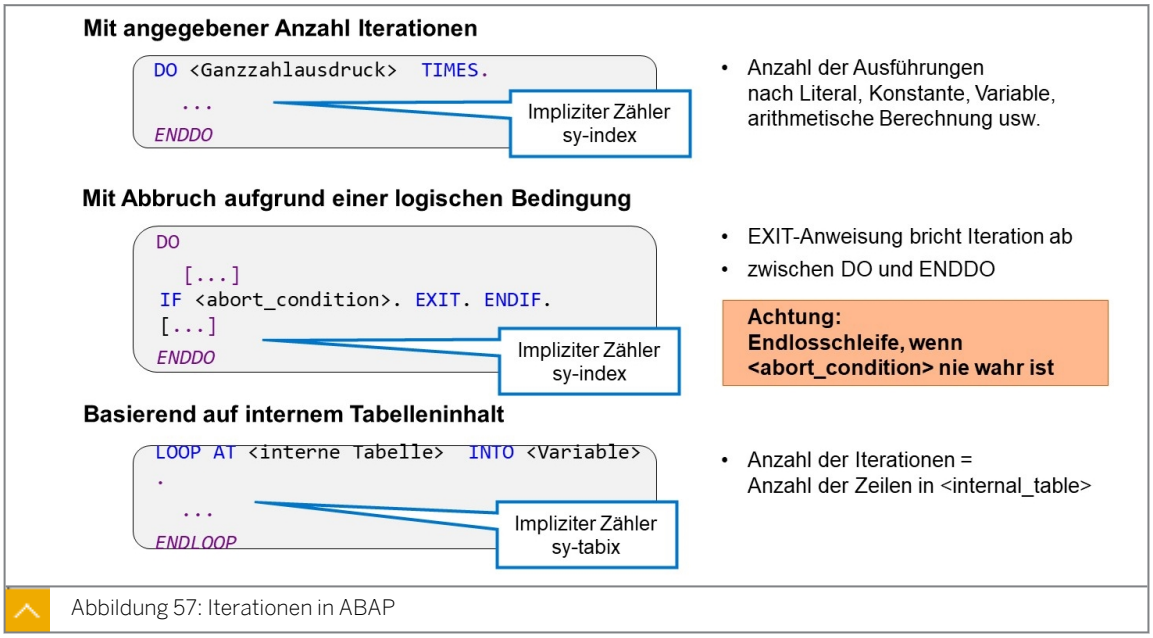


Abbildung 57: Iterationen in ABAP

© Copyright. Alle Rechte vorbehalten.

Iterationen sind Kontrollstrukturen, die einen mehrfach ausgeführten Quelltextblock definieren.

Die einfachste Form der Iteration besteht aus einem Quelltextblock, der von den beiden Anweisungen DO und ENDDO umgeben ist. Ohne weitere Maßnahmen entsteht eine Endlosschleife, die durch eine der folgenden Möglichkeiten vermieden werden muss:

Angegebene Anzahl von Iterationen

Durch Erweiterung der DO-Anweisung mit einem Integer-Ausdruck gefolgt vom Schlüsselwort TIMES kann explizit angegeben werden, wie oft der Quelltextblock iteriert werden soll. Der Integer-Ausdruck kann genauso einfach wie ein Zahlenliteral sein, aber auch arithmetische Berechnungen sind möglich. Wenn der Wert des Ausdrucks gleich 0 ist, wird der Quelltextblock zwischen DO und ENDDO überhaupt nicht ausgeführt, und das Programm fährt sofort mit dem Quelltext nach ENDDO fort.

Abbruch aufgrund einer logischen Bedingung

Sie können eine Iteration jederzeit mit der Anweisung EXIT abbrechen. Das Programm wird dann mit dem Quelltext nach ENDDO fortgesetzt. Beachten Sie, dass außerhalb von Iterationen EXIT eine andere Wirkung hat. Dort bricht die Anweisung die Verarbeitung des aktuellen Verarbeitungsblocks ab, z.B. der aktuellen Methode.

In der Regel ist EXIT von IF und ENDIF umgeben, um die Iteration abhängig von einer Abbruchbedingung abzubrechen. Beachten Sie, dass solche Iterationen in Endlosschleifen umgewandelt werden können, wenn die Abbruchbedingung nie wahr wird.

Natürlich ist es möglich, die beiden Methoden zu kombinieren, d.h., die Anzahl der Iterationen explizit anzugeben und dann die Iteration mit EXIT zu verlassen. Dadurch wird die Anzahl der Iterationen zu einer maximalen Anzahl, die zur Laufzeit möglicherweise nicht erreicht wird.

Basierend auf einer internen Tabelle

Eine dritte Art der Iteration ist die Struktur LOOP ... ENDLOOP, mit der die Zeilen einer internen Tabelle nacheinander gelesen werden. Hier wird die Anzahl der Iterationen durch die Anzahl der Zeilen in der internen Tabelle bestimmt.

Im Quelltextblock zwischen DO und ENDDO können Lesezugriffe auf das eingebaute ABAP-Datenobjekt „sy-index“ implementiert werden. Diese Integer-Variable dient als Iterationszähler, d.h., die ABAP-Laufzeit erhöht die Variable zu Beginn jeder neuen Iteration um eins.

Notiz:
Im Gegensatz zu anderen Programmiersprachen beginnt „sy-index“ bei der ersten Iteration mit dem Wert 1.

Die eingebaute ABAP-Variable „sy-tabix“ kann einen ähnlichen Zweck für Iterationen mit LOOP erfüllen. Beachten Sie jedoch, dass „sy-tabix“ eigentlich kein Zähler ist, sondern die Position der Tabellenzeile identifiziert, die in der aktuellen Iteration verarbeitet wird. Später erfahren Sie mehr über den Unterschied, wenn nicht alle Zeilen einer internen Tabelle in einer „LOOP ... ENDLOOP“-Struktur verarbeitet werden.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Übung 5

Bedingte Verzweigungen implementieren

In dieser Übung aktivieren Sie das Programm für die vier Basisberechnungsoperationen und behandeln Fehlersituationen.

Vorlage:

/LRN/CL_S4D400_BTS_COMPUTE (globale Klasse)

Lösung:

/LRN/CL_S4D400_BTS_BRANCH (globale Klasse)

Aufgabe 1: Vorlage kopieren (optional)
Kopieren Sie die Vorlageklasse. Wenn Sie die vorherige Übung abgeschlossen haben, können Sie diese Aufgabe überspringen und mit der Bearbeitung Ihrer Klasse **`ZCL_##_COMPUTE`** fortfahren.

1. Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_BTS_COMPUTE`** im ABAP Editor.
2. Verknüpfen Sie die View *Project Explorer* mit dem Editor.
3. Kopieren Sie die Klasse **`/LRN/CL_S4D400_BTS_COMPUTE`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_BRANCH`**, wobei ## für Ihre Gruppennummer steht).

Aufgabe 2: Variable für Operator
Deklarieren Sie eine Variable für den Berechnungsoperator, und machen Sie die Berechnung von dieser Variable abhängig.

1. Deklarieren Sie eine Variable für den Berechnungsoperator (Namensvorschlag: **`op`**). Die Variable benötigt einen Typ fester Länge, um einen der folgenden Werte zu enthalten: „+", „-", „*" oder „/".
2. Fügen Sie direkt nach der Wertzuweisung für die Variablen **`number1`** und **`number2`** eine Wertzuweisung hinzu, um die neue Variable **`op`** auf den Wert „/" zu setzen.
3. Verwenden Sie eine geeignete Kontrollstruktur, um die Variable **`result`** abhängig vom Wert der Variable **`op`** mit unterschiedlichen arithmetischen Ausdrücken zu füllen.
4. Passen Sie das Zeichenketten-Template so an, dass statt des Literalzeichens „/" der Inhalt von **`op`** verwendet wird.
5. Aktivieren und testen Sie die Anwendung mit unterschiedlichen Werten für **`number1`**, **`number2`** und **`op`**.

© Copyright. Alle Rechte vorbehalten.

Was geschieht, wenn Sie **op** auf den Wert „%” setzen?

Aufgabe 3: Ungültigen Operator behandeln

Behandeln Sie einen ungültigen Wert von **op**. Wenn **op** einen anderen Wert als „+”, „-”, „*” oder „/” enthält, schreiben Sie anstelle der Berechnung und des Ergebniswerts eine Fehlermeldung in die Konsole.

1. Fügen Sie in der CASE-Kontrollstruktur eine Verzweigung hinzu, die für alle anderen Werte von **op** ausgeführt wird.
2. Füllen Sie in dieser Verzweigung **output** mit einem geeigneten Text, um die Fehlersituation zu beschreiben.

Notiz:
Wir empfehlen, an dieser Stelle die Inline-Deklaration der Variable **output** durch eine explizite Deklaration am Anfang der Methode zu ersetzen.

3. Verwenden Sie eine geeignete Kontrollstruktur, um sicherzustellen, dass die Variable **output** nur dann mit dem Berechnungsergebnis gefüllt wird, wenn sie nicht bereits mit einem Fehlertext gefüllt ist.
4. Aktivieren und testen Sie die Anwendung mit unterschiedlichen Werten für **number1**, **number2** und **op**.

   Was geschieht, wenn Sie **number1** auf **1**, **number2** auf **0** und **op** auf **/** setzen?

Aufgabe 4: Division durch Null behandeln

Verhindern Sie den Programmabbruch, indem Sie die Ausnahme behandeln.

1. Umschließen Sie innerhalb der Verzweigung für die Division (**op** hat den Wert „/”) die Berechnung mit einem Paar aus TRY- und ENDTRY-Anweisungen.
2. Fügen Sie vor der Anweisung ENDTRY einen CATCH-Block für die Ausnahme hinzu, der bei einer Division durch Null ausgelöst wird.
3. Füllen Sie im CATCH-Block die Variable **output** mit einem passenden Fehlertext.
4. Aktivieren und testen Sie die Anwendung mit unterschiedlichen Werten für **number1**, **number2** und **op**. Testen Sie insbesondere die Behandlung der Fehlersituationen: ungültiger Operator und Division durch Null.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Lösung 5

Bedingte Verzweigungen implementieren

In dieser Übung aktivieren Sie das Programm für die vier Basisberechnungsoperationen und behandeln Fehlersituationen.

Vorlage:

/LRN/CL_S4D400_BTS_COMPUTE (globale Klasse)

Lösung:

/LRN/CL_S4D400_BTS_BRANCH (globale Klasse)

Aufgabe 1: Vorlage kopieren (optional)
Kopieren Sie die Vorlageklasse. Wenn Sie die vorherige Übung abgeschlossen haben, können Sie diese Aufgabe überspringen und mit der Bearbeitung Ihrer Klasse **`ZCL_##_COMPUTE`** fortfahren.

1. Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_BTS_COMPUTE`** im ABAP Editor.
   a) Wählen Sie in der *Eclipse*-Symbolleiste *Open ABAP Development Object*. Drücken Sie alternativ **`Strg + Umschalt + A`**.
   b) Geben Sie als Suchzeichenfolge **`/LRN/CL_S4D400_BTS`** ein.
   c) Wählen Sie aus der Liste der Entwicklungsobjekte */LRN/CL_S4D400_BTS_COMPUTE* aus, und wählen Sie dann *OK*.
2. Verknüpfen Sie die View *Project Explorer* mit dem Editor.
   a) Suchen Sie in der Symbolleiste *Project Explorer* die Drucktaste *Link with Editor*. Wählen Sie die Drucktaste, falls diese noch nicht markiert ist. Dadurch sollte das Entwicklungsobjekt, das im Editor geöffnet ist, im *Project Explorer* hervorgehoben sein.
3. Kopieren Sie die Klasse **`/LRN/CL_S4D400_BTS_COMPUTE`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_BRANCH`**, wobei ## für Ihre Gruppennummer steht).
   a) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Klasse **`/LRN/CL_S4D400_BTS_COMPUTE`**, um das Kontextmenü zu öffnen.
   b) Wählen Sie im Kontextmenü *Duplicate ...*
   c) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_BRANCH`** ein, wobei ## für Ihre Gruppennummer steht.
   d) Passen Sie die Beschreibung an, und wählen Sie *Next*.
   e) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

Aufgabe 2: Variable für Operator
Deklarieren Sie eine Variable für den Berechnungsoperator, und machen Sie die Berechnung von dieser Variable abhängig.

© Copyright. Alle Rechte vorbehalten.

1. Deklarieren Sie eine Variable für den Berechnungsoperator (Namensvorschlag: **op**). Die Variable benötigt einen Typ fester Länge, um einen der folgenden Werte zu enthalten: „+", „-", „*" oder „/".
   a) Fügen Sie folgenden Code hinzu:

```
DATA op      TYPE c LENGTH 1.
```

2. Fügen Sie direkt nach der Wertzuweisung für die Variablen **number1** und **number2** eine Wertzuweisung hinzu, um die neue Variable **op** auf den Wert „/" zu setzen.
   a) Passen Sie den Quelltext wie folgt an:

```
 number1 = 123.
 number2 = 0.
 op      = '/'.
```

3. Verwenden Sie eine geeignete Kontrollstruktur, um die Variable **result** abhängig vom Wert der Variable **op** mit unterschiedlichen arithmetischen Ausdrücken zu füllen.
   a) Passen Sie den Quelltext wie folgt an:

```
CASE op.
   WHEN '+'.
     result = number1 + number2.
   WHEN '-'.
     result = number1 - number2.
   WHEN '*'.
     result = number1 * number2.
   WHEN '/'.
     result = number1 / number2.
ENDCASE.
```

4. Passen Sie das Zeichenketten-Template so an, dass statt des Literalzeichens „/" der Inhalt von **op** verwendet wird.
   a) Passen Sie das Zeichenketten-Template wie folgt an:

```
output = |{ number1 } { op } { number2 } = { result }|.
```

5. Aktivieren und testen Sie die Anwendung mit unterschiedlichen Werten für **number1**, **number2** und **op**.

   Was geschieht, wenn Sie **op** auf den Wert „%" setzen?

   Das Ergebnis ist immer Null (0,00), da keiner der vier WHEN-Verzweigungen ausgeführt wird.

   a) Fahren Sie wie in den vorangegangenen Übungen fort.

Aufgabe 3: Ungültigen Operator behandeln

Behandeln Sie einen ungültigen Wert von **op**. Wenn **op** einen anderen Wert als „+", „-", „*" oder „/" enthält, schreiben Sie anstelle der Berechnung und des Ergebniswerts eine Fehlermeldung in die Konsole.

1. Fügen Sie in der CASE-Kontrollstruktur eine Verzweigung hinzu, die für alle anderen Werte von **op** ausgeführt wird.

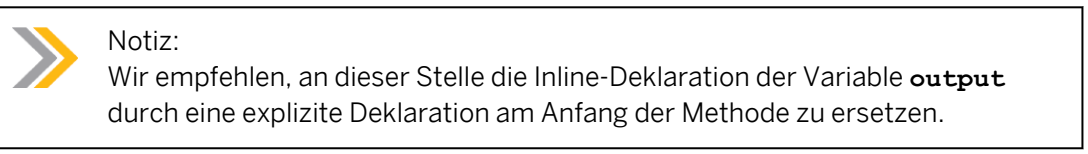
© Copyright. Alle Rechte vorbehalten.

a) Passen Sie den Quelltext wie folgt an:

```
CASE op.
      WHEN '+'.
        result = number1 + number2.
      WHEN '-'.
        result = number1 - number2.
      WHEN '*'.
        result = number1 * number2.
      WHEN '/'.
        result = number1 / number2.
      WHEN OTHERS.

    ENDCASE.
```

2. Füllen Sie in dieser Verzweigung **output** mit einem geeigneten Text, um die Fehlersituation zu beschreiben.



a) Fügen Sie am Anfang der Methode den folgenden Code hinzu:

```
    DATA output TYPE string.
```

b) Ersetzen Sie die Inline-Deklaration `DATA(output)` mit der explizit deklarierten Variable **output**.

c) Geben Sie nach der `WHEN OTHERS.`-Anweisung folgenden Quelltext ein:

```
    output = |'{ op }' is not a valid operator!|.
```

3. Verwenden Sie eine geeignete Kontrollstruktur, um sicherzustellen, dass die Variable **output** nur dann mit dem Berechnungsergebnis gefüllt wird, wenn sie nicht bereits mit einem Fehlertext gefüllt ist.

a) Passen Sie den Quelltext wie folgt an:

```
    IF output IS INITIAL.  "no error so far
      output = |{ number1 } { op } { number2 } = { result }|.
    ENDIF.
```

4. Aktivieren und testen Sie die Anwendung mit unterschiedlichen Werten für **number1**, **number2** und **op**.

   Was geschieht, wenn Sie **number1** auf **1**, **number2** auf **0** und **op** auf **/** setzen?

   Die Anwendung bricht mit der Ausnahme CX_SY_ZERODIVIDE ab.

a) Fahren Sie wie in den vorangegangenen Übungen fort.

Aufgabe 4: Division durch Null behandeln

Verhindern Sie den Programmabbruch, indem Sie die Ausnahme behandeln.

© Copyright. Alle Rechte vorbehalten.

1. Umschließen Sie innerhalb der Verzweigung für die Division (**op** hat den Wert „/") die Berechnung mit einem Paar aus TRY- und ENDTRY-Anweisungen.
   a) Passen Sie den Quelltext wie folgt an:

```
    WHEN '/'.

       TRY.
           result = number1 / number2.

       ENDTRY.

     WHEN OTHERS.
```

2. Fügen Sie vor der Anweisung ENDTRY einen CATCH-Block für die Ausnahme hinzu, der bei einer Division durch Null ausgelöst wird.
   a) Passen Sie den Quelltext wie folgt an:

```
       TRY.
           result = number1 / number2.

         CATCH cx_sy_zerodivide.
        ENDTRY.
```

3. Füllen Sie im CATCH-Block die Variable **output** mit einem passenden Fehlertext.
   a) Passen Sie den Quelltext wie folgt an:

```
        TRY.
           result = number1 / number2.

         CATCH cx_sy_zerodivide.
           output = |Division by zero is not defined|.
       ENDTRY.
```

4. Aktivieren und testen Sie die Anwendung mit unterschiedlichen Werten für **number1**, **number2** und **op**. Testen Sie insbesondere die Behandlung der Fehlersituationen: ungültiger Operator und Division durch Null.
   a) Fahren Sie wie zuvor fort.
   b) Vergleichen Sie Ihren Quelltext mit dem folgenden Auszug aus der Musterlösung:

```
METHOD if_oo_adt_classrun~main.

* Declarations
*************************

    DATA number1 TYPE i.
    DATA number2 TYPE i.

    DATA result  TYPE p LENGTH 8 DECIMALS 2.

    DATA op      TYPE c LENGTH 1.

    DATA output TYPE string.

* Input Values
*************************

    number1 = 123.
```


© Copyright. Alle Rechte vorbehalten.

```
    number2 = 0.
    op      = '/'.

* Calculation
**************************
    CASE op.
      WHEN '+'.
        result = number1 + number2.
      WHEN '-'.
        result = number1 - number2.
      WHEN '*'.
        result = number1 * number2.
      WHEN '/'.

        TRY.
            result = number1 / number2.
          CATCH cx_sy_zerodivide.
            output = |Division by zero is not defined|.
        ENDTRY.

      WHEN OTHERS.

        output = |'{ op }' is not a valid operator!|.

    ENDCASE.

    IF output IS INITIAL.  "no error so far

      output = |{ number1 } { op } { number2 } = { result }|.

    ENDIF.

* Output
**************************

    out->write( output ).

  ENDMETHOD.
```

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- bedingte Verzweigung implementieren
- Ausnahmen behandeln.
- Iterationen implementieren.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Lektion 5

Mit einfachen internen Tabellen arbeiten

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Definieren Sie einfache interne Tabellen.
- Daten mit einfachen internen Tabellen verarbeiten.

Definition von einfachen internen Tabellen

Interne Tabellen

Interne Tabellen sind variable Datenobjekte, in denen mehrere typgleiche Werte gespeichert werden können. Dieser Typ muss in der Deklaration angegeben werden und wird als Zeilentyp der internen Tabelle bezeichnet.

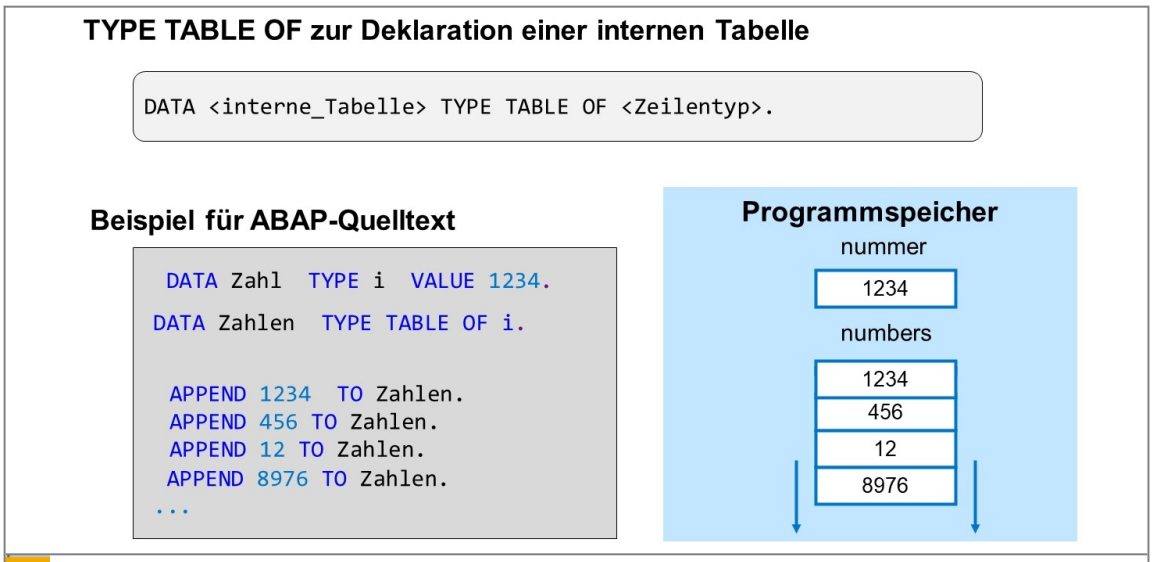


Abbildung 58: Interne Tabellen

Jeder Wert belegt eine Zeile der internen Tabelle. Die Anzahl der Zeilen ist nicht beschränkt. Theoretisch können Sie beliebig viele Werte in einer internen Tabelle speichern. Einschränkungen ergeben sich nur aus technischen Grenzen wie verfügbarem Speicher oder der Systemkonfiguration.

Der Initialwert einer internen Tabelle ist eine leere Tabelle, d.h. eine Tabelle mit 0 Zeilen. Es gibt verschiedene Methoden zum Füllen einer internen Tabelle. Im Beispiel wird mit der Anweisung APPEND am Ende der internen Tabelle eine neue Zeile hinzugefügt und diese mit einem Wert gefüllt.

© Copyright. Alle Rechte vorbehalten.

Tabellentypen

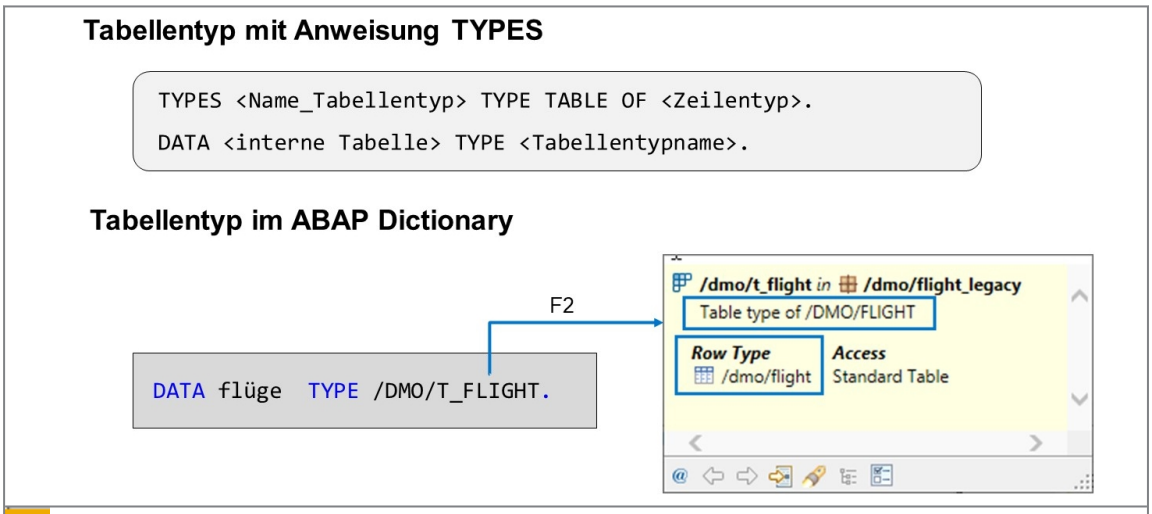


Abbildung 59: Tabellentypen

Es gibt auch Tabellentypen im ABAP Dictionary. Diese Tabellentypen werden mit einem dedizierten Editor gepflegt. Sie werden als globale Tabellentypen bezeichnet, da sie überall im System sichtbar sind.

Datenverarbeitung mit einfachen internen Tabellen

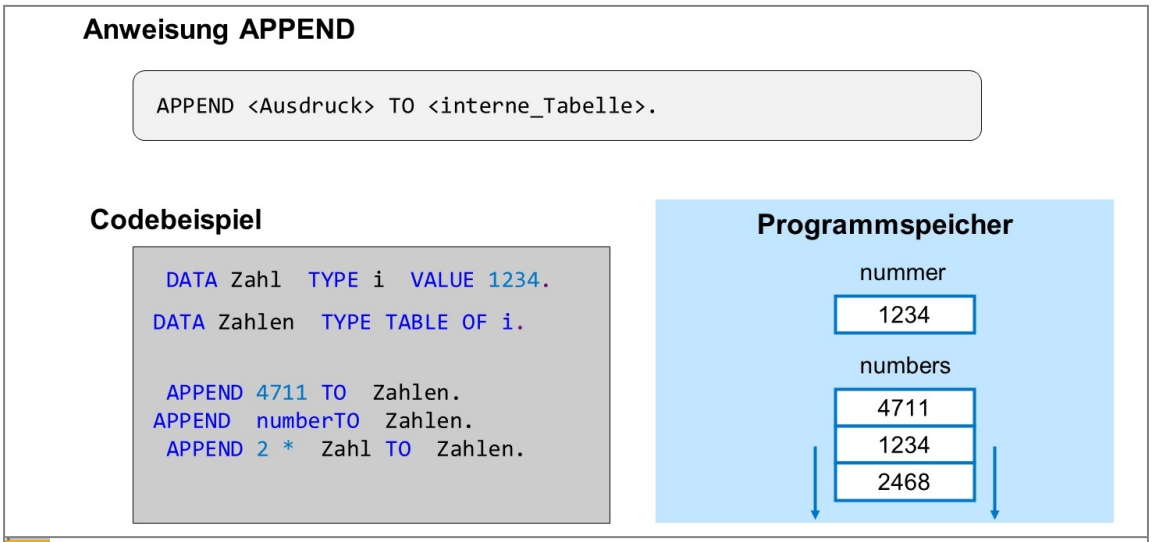


Abbildung 60: Eine interne Tabelle mit APPEND füllen

Eine Möglichkeit, eine interne Tabelle mit Daten zu füllen, ist die APPEND-Anweisung. Mit jeder APPEND-Anweisung wird der internen Tabelle eine neue Zeile hinzugefügt. Diese neue Zeile wird dann mit dem Wert des Ausdrucks nach dem Schlüsselwort APPEND gefüllt.

Hinweis:

Der Typ des Ausdrucks muss mit dem Zeilentyp der internen Tabelle kompatibel sein. Wenn er nicht identisch ist, versucht das System eine implizite Typkonvertierung.

© Copyright. Alle Rechte vorbehalten.

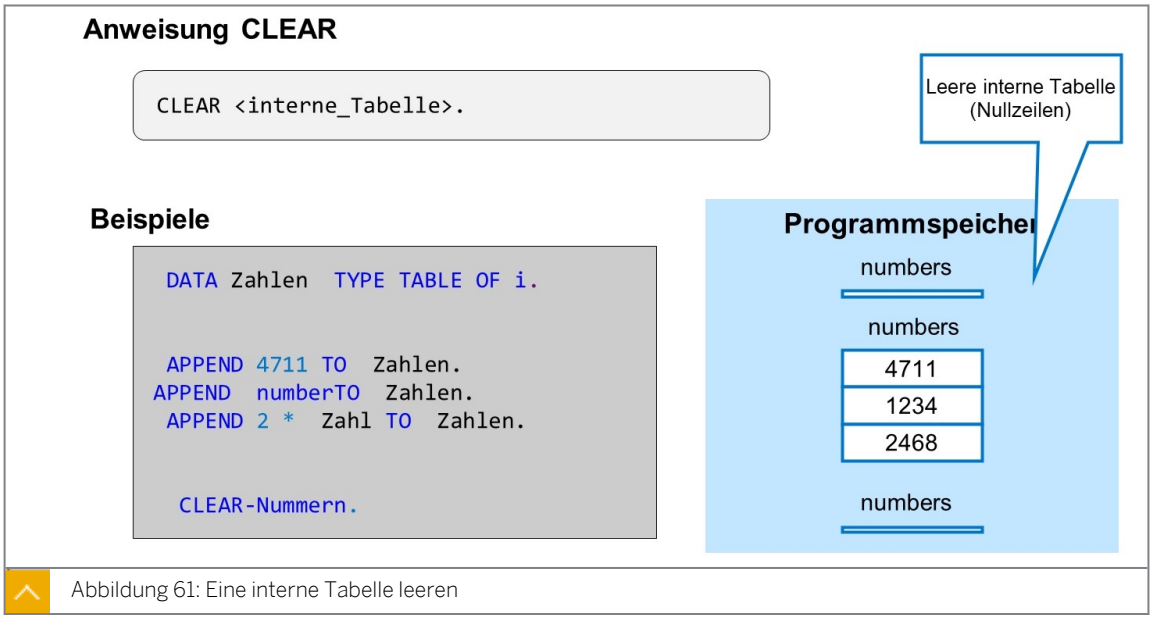


Abbildung 61: Eine interne Tabelle leeren

Der Initialwert einer internen Tabelle ist eine leere Tabelle, d.h. eine interne Tabelle mit null Zeilen.

Sie haben bereits gelernt, dass Sie mit der Anweisung CLEAR eine ABAP-Variable auf ihren typgerechten Initialwert zurücksetzen können.

Wenn Sie CLEAR für eine interne Tabelle verwenden, löschen Sie ihren gesamten Inhalt und setzen die Anzahl der Zeilen auf null.

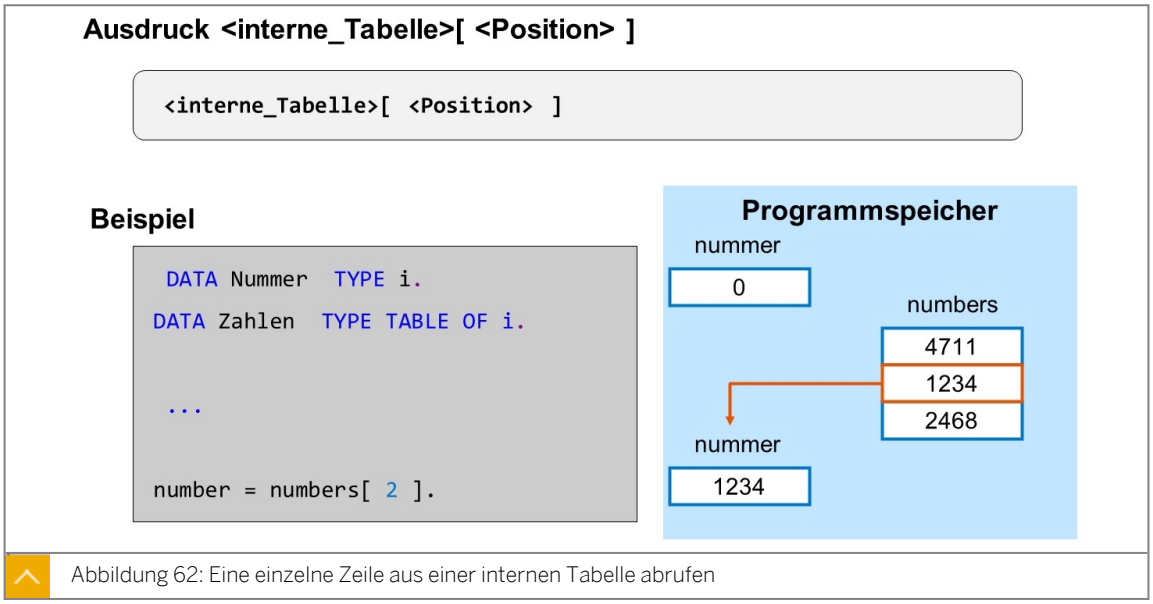


Abbildung 62: Eine einzelne Zeile aus einer internen Tabelle abrufen

Es gibt verschiedene Möglichkeiten, Daten aus einer internen Tabelle abzurufen. In diesem Beispiel wird der Inhalt einer einzelnen Zeile über einen internen Tabellenausdruck abgerufen. Im Tabellenausdruck folgt unmittelbar auf den Namen der internen Tabelle ein Paar eckiger Klammern. Ein Integer-Ausdruck innerhalb der Klammern gibt die Position der zu lesenden Zeile an.

Hinweis:

Die korrekte Syntax erfordert mindestens ein Leerzeichen nach der öffnenden Klammer und vor der schließenden Klammer.

© Copyright. Alle Rechte vorbehalten.

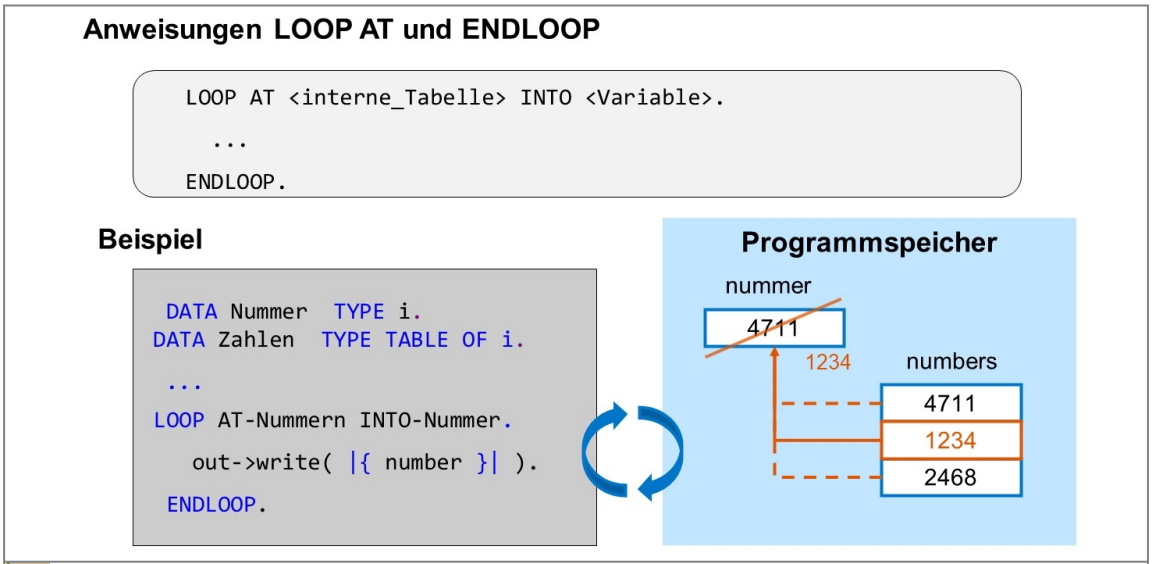


Abbildung 63: In einer Schleife aus einer internen Tabelle lesen

Mit einer Kombination der Anweisungen LOOP AT und ENDLOOP können alle Zeilen einer internen Tabelle gelesen und nacheinander verarbeitet werden. LOOP AT und ENDLOOP kommen immer paarweise vor, d.h., für jede LOOP AT-Anweisung muss es genau eine entsprechende ENDLOOP-Anweisung geben.

Zwischen LOOP AT und ENDLOOP kann eine beliebige Menge ABAP-Quelltext platziert werden. Der Quelltext, den Sie dort platzieren, wird mehrmals verarbeitet, einmal für jede Zeile der internen Tabelle.

Notiz:
Wenn die interne Tabelle leer ist, wird der Quelltext zwischen LOOP AT und ENDLOOP überhaupt nicht verarbeitet.

Das nach dem Zusatz INTO aufgeführte Datenobjekt wird als Arbeitsbereich für die interne Tabelle bezeichnet. Der Typ dieses Datenobjekts muss mit dem Zeilentyp der internen Tabelle identisch oder zumindest kompatibel sein.

Zu Beginn jeder Runde wird der Arbeitsbereich mit den Daten aus der nächsten Tabellenzeile gefüllt.

Im Beispiel enthält die interne Tabelle `numbers` drei Zeilen. Daher wird der Quelltext zwischen LOOP AT und ENDLOOP 3 mal verarbeitet. Während der ersten Runde enthält die Variable `number` 4711, den Wert der ersten Zeile. Dieser Wert wird in die Konsole geschrieben. Zu Beginn der zweiten Runde wird der Wert in der Variablen `number` durch den Wert 1234 aus der zweiten Zeile ersetzt usw.

© Copyright. Alle Rechte vorbehalten.

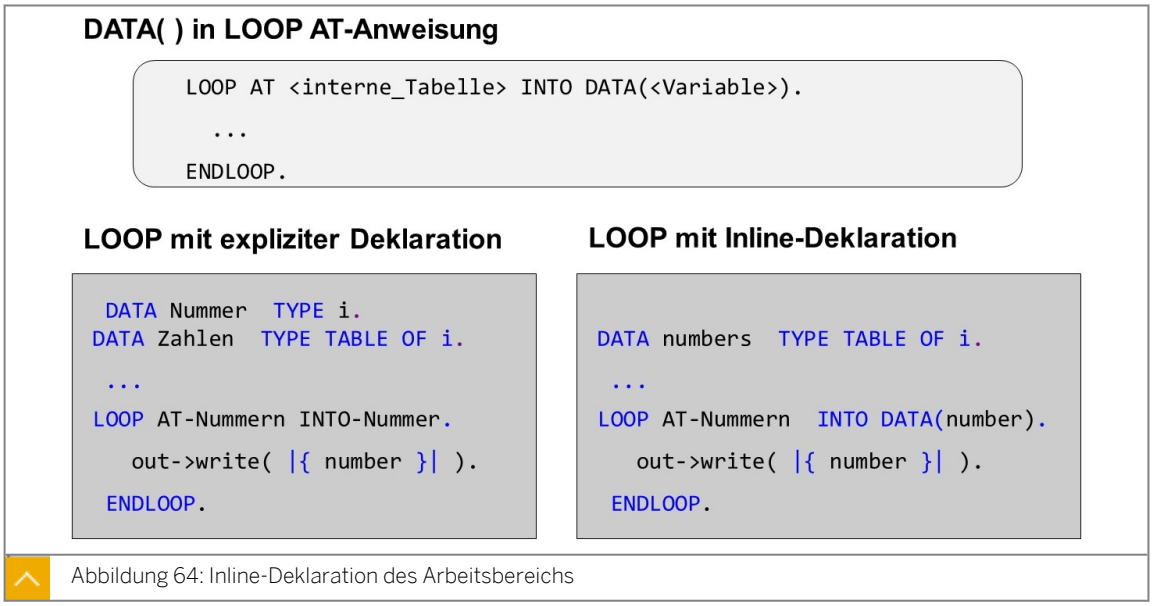


Abbildung 64: Inline-Deklaration des Arbeitsbereichs

Bei der Implementierung einer Schleife über eine interne Tabelle kann anstelle der expliziten Deklaration des Arbeitsbereichs mit einer DATA-Anweisung eine Inline-Deklaration nach dem Zusatz INTO verwendet werden.

Dadurch reduzieren Sie nicht nur die Menge des zu typisierenden Quelltexts, sondern stellen auch sicher, dass der Typ des Arbeitsbereichs zum Zeilentyp der internen Tabelle passt, da der Typ des inline deklarierten Datenobjekts aus dem Kontext abgeleitet wird, der in diesem Fall die interne Tabelle ist.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Übung 6

Mit einfachen internen Tabellen arbeiten

In dieser Übung verwenden Sie Iterationen und einfache interne Tabellen, um die Fibonacci-Zahlen zu berechnen und anzuzeigen.

Notiz:
Die Fibonacci-Zahlen sind eine Zahlenfolge, in der jede Zahl die Summe der beiden vorhergehenden Zahlen ist. Die Folge beginnt mit 0 und 1, und jede nachfolgende Zahl ist die Summe der beiden vorherigen Zahlen. Die Folge hat verschiedene mathematische Eigenschaften, Muster und Anwendungen in Bereichen wie Mathematik, Informatik, Natur und Kunst.

Vorlage:

keine

Lösung:

/LRN/CL_S4D400_BTS_ITERATE (globale Klasse)

Aufgabe 1: Die Zahlen berechnen
Legen Sie in Ihrem eigenen Paket eine neue globale Klasse an. Definieren Sie eine Konstante für die Anzahl der Iterationen und eine interne Tabelle für die Fibonacci-Zahlen. Berechnen Sie dann die Fibonacci-Zahlen in einer Iteration, und speichern Sie sie in der internen Tabelle.

1. Legen Sie eine neue globale Klasse an, die das Interface **`IF_OO_ADT_CLASSRUN`** implementiert (Namensvorschlag: **`ZCL_##_ITERATE`**, wobei ## für Ihre Gruppennummer steht).
2. Definieren Sie in der Methode *IF_OO_ADT_CLASSRUN~MAIN* eine Konstante für die Anzahl der Iterationen (Namensvorschlag: **`max_count`**) mit einem kleinen Wert, z.B. **`20`**.
3. Deklarieren Sie eine einfache interne Tabelle zum Speichern der Fibonacci-Nummern (Namensvorschlag: **`Zahlen`**).
4. Implementieren Sie eine Iteration, die **`max_count`** mal ausgeführt wird.
5. Implementieren Sie innerhalb der Iteration eine Fallunterscheidung basierend auf dem eingebauten Iterationszähler: In der ersten Iteration fügen Sie der internen Tabelle **`numbers`** den Wert **`0`** hinzu. Fügen Sie in der zweiten Iteration den Wert **`1`** am Ende der internen Tabelle **`numbers`** hinzu.

Hinweis:
Der eingebaute Iterationszähler ist das Systemfeld **`sy-index`**.

© Copyright. Alle Rechte vorbehalten.

6. Fügen Sie eine Verzweigung hinzu, die für alle anderen Werte ausgeführt wird. Berechnen Sie in dieser Verzweigung den neuen Eintrag als Summe der beiden vorherigen Einträge in **`numbers`**.

Hinweis:
Greifen Sie über den Index **`sy-index - 1`** und **`sy-index - 2`** auf die vorherigen Einträge zu.

Aufgabe 2: Eine formatierte Ausgabe vorbereiten
Bereiten Sie in einer Schleife über der internen Tabelle eine formatierte Ausgabe vor, in der jede Fibonacci-Zahl mit der jeweiligen Folgenummer aufgeführt ist. Bereiten Sie die Ausgabe in einer anderen internen Tabelle mit dem Zeilentyp *string* vor.

1. Deklarieren Sie am Anfang der Methode **`if_oo_adt_classrun~main`** eine interne Tabelle vom Zeilentyp *string* (Namensvorschlag: **`output`**).
2. Implementieren Sie eine Schleife über die interne Tabelle **`numbers`**, um die Fibonacci-Zahlen nacheinander in eine Variable **`number`** zu lesen.

Hinweis:
Verwenden Sie eine Inline-Deklaration für **`number`**.

3. Deklarieren Sie eine Ganzzahlvariable (vorgeschlagener Name: **`counter`**), die Sie vor der Schleife auf null setzen und in jeder Iteration um 1 erhöhen.

Hinweis:
Sie können eine Inline-Deklaration für **`counter`** verwenden.

4. Fügen Sie in jeder Iteration der internen Tabelle **`output`** eine neue Zeile hinzu, die den Inhalt von **`counter`** (4 Zeichen breit, linksbündig), einen Doppelpunkt (:) und den Inhalt von **`number`** (10 Zeichen breit, rechtsbündig) enthält.

Hinweis:
Verwenden Sie ein Zeichenketten-Template mit geeigneten Formatoptionen, um das Format anzupassen.

Welche Formatoptionen werden verwendet, um die Breite und Ausrichtung festzulegen?

Aufgabe 3: Ausgabe und Test
Schreiben Sie das Ergebnis in die Konsole. Anschließend aktivieren und testen Sie Ihre Klasse als Konsolen-App.

© Copyright. Alle Rechte vorbehalten.

1. Rufen Sie am Ende der Methode die Methode **`out->write`** auf, um den Inhalt von **`output`** in die Konsole zu schreiben. Geben Sie dem Importing-Parameter **`name`** eine geeignete Beschriftung.
2. Aktivieren Sie die Klasse.
3. Führen Sie Ihre Klasse als Konsolen-App aus.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Lösung 6

Mit einfachen internen Tabellen arbeiten

In dieser Übung verwenden Sie Iterationen und einfache interne Tabellen, um die Fibonacci-Zahlen zu berechnen und anzuzeigen.

Notiz:
Die Fibonacci-Zahlen sind eine Zahlenfolge, in der jede Zahl die Summe der beiden vorhergehenden Zahlen ist. Die Folge beginnt mit 0 und 1, und jede nachfolgende Zahl ist die Summe der beiden vorherigen Zahlen. Die Folge hat verschiedene mathematische Eigenschaften, Muster und Anwendungen in Bereichen wie Mathematik, Informatik, Natur und Kunst.

Vorlage:

keine

Lösung:

/LRN/CL_S4D400_BTS_ITERATE (globale Klasse)

Aufgabe 1: Die Zahlen berechnen
Legen Sie in Ihrem eigenen Paket eine neue globale Klasse an. Definieren Sie eine Konstante für die Anzahl der Iterationen und eine interne Tabelle für die Fibonacci-Zahlen. Berechnen Sie dann die Fibonacci-Zahlen in einer Iteration, und speichern Sie sie in der internen Tabelle.

1. Legen Sie eine neue globale Klasse an, die das Interface **`IF_OO_ADT_CLASSRUN`** implementiert (Namensvorschlag: **`ZCL_##_ITERATE`**, wobei ## für Ihre Gruppennummer steht).
   a) Wählen Sie *File* → *New* → *ABAP Class*.
   b) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_ITERATE`** ein, wobei ## für Ihre Gruppennummer steht.
   c) Geben Sie eine Beschreibung ein.
   d) Wählen Sie im Gruppenrahmen *Interfaces* die Option *Add*.
   e) Geben Sie **`IF_OO_ADT_CLASSRUN`** ein. Wenn das Interface in der Trefferliste angezeigt wird, fügen Sie es mit Doppelklick zur Klassendefinition hinzu.
   f) Wählen Sie *Next*.
   g) Wählen Sie Ihren Transportauftrag aus, und wählen Sie *Finish*.
2. Definieren Sie in der Methode *IF_OO_ADT_CLASSRUN~MAIN* eine Konstante für die Anzahl der Iterationen (Namensvorschlag: **`max_count`**) mit einem kleinen Wert, z.B. **`20`**.

© Copyright. Alle Rechte vorbehalten.

a) Fügen Sie folgenden Code hinzu:

```
CONSTANTS max_count TYPE i VALUE 20.
```

3. Deklarieren Sie eine einfache interne Tabelle zum Speichern der Fibonacci-Nummern (Namensvorschlag: **`Zahlen`**).

   a) Fügen Sie folgenden Code hinzu:

```
DATA numbers TYPE TABLE OF i.
```

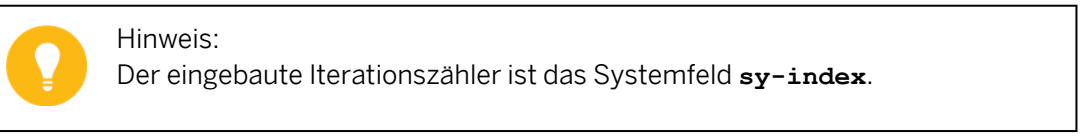
4. Implementieren Sie eine Iteration, die **`max_count`** mal ausgeführt wird.

   a) Fügen Sie nach den Deklarationen den folgenden Quelltext hinzu:

```
DO max_count TIMES.

ENDDO.
```

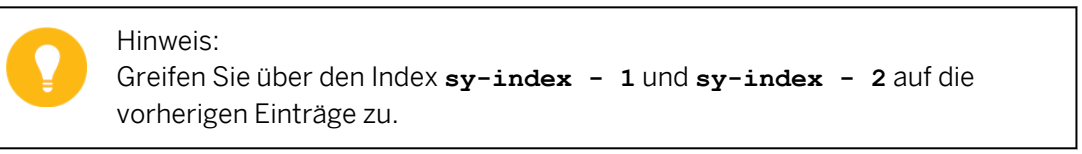
5. Implementieren Sie innerhalb der Iteration eine Fallunterscheidung basierend auf dem eingebauten Iterationszähler: In der ersten Iteration fügen Sie der internen Tabelle **`numbers`** den Wert **`0`** hinzu. Fügen Sie in der zweiten Iteration den Wert **`1`** am Ende der internen Tabelle **`numbers`** hinzu.



   a) Fügen Sie zwischen `DO` und `ENDDO` folgenden Code hinzu:

```
CASE sy-index.
   WHEN 1.
     APPEND 0 TO numbers.
   WHEN 2.
     APPEND 1 TO numbers.
ENDCASE.
```

6. Fügen Sie eine Verzweigung hinzu, die für alle anderen Werte ausgeführt wird. Berechnen Sie in dieser Verzweigung den neuen Eintrag als Summe der beiden vorherigen Einträge in **`numbers`**.



   a) Passen Sie den Quelltext wie folgt an:

```
CASE sy-index.
   WHEN 1.
     APPEND 0 TO numbers.
   WHEN 2.
     APPEND 1 TO numbers.
   WHEN OTHERS.
       APPEND numbers[  sy-index - 2 ]
            + numbers[  sy-index - 1 ]
```

© Copyright. Alle Rechte vorbehalten.

```
              TO numbers.
  ENDCASE.
```

Aufgabe 2: Eine formatierte Ausgabe vorbereiten
Bereiten Sie in einer Schleife über der internen Tabelle eine formatierte Ausgabe vor, in der jede Fibonacci-Zahl mit der jeweiligen Folgenummer aufgeführt ist. Bereiten Sie die Ausgabe in einer anderen internen Tabelle mit dem Zeilentyp *string* vor.

1. Deklarieren Sie am Anfang der Methode **`if_oo_adt_classrun~main`** eine interne Tabelle vom Zeilentyp *string* (Namensvorschlag: **`output`**).

2. Implementieren Sie eine Schleife über die interne Tabelle **`numbers`**, um die Fibonacci-Zahlen nacheinander in eine Variable **`number`** zu lesen.

Hinweis:
Verwenden Sie eine Inline-Deklaration für **`number`**.

a) Fügen Sie am Ende der Methode den folgenden Code hinzu:

```
LOOP AT numbers INTO DATA(number).

ENDLOOP.
```

3. Deklarieren Sie eine Ganzzahlvariable (vorgeschlagener Name: **`counter`**), die Sie vor der Schleife auf null setzen und in jeder Iteration um 1 erhöhen.

Hinweis:
Sie können eine Inline-Deklaration für **`counter`** verwenden.

a) Passen Sie den Quelltext wie folgt an:

```
DATA(counter) = 0.
LOOP AT numbers INTO DATA(number).

   counter = counter + 1.

ENDLOOP.
```

4. Fügen Sie in jeder Iteration der internen Tabelle **`output`** eine neue Zeile hinzu, die den Inhalt von **`counter`** (4 Zeichen breit, linksbündig), einen Doppelpunkt (:) und den Inhalt von **`number`** (10 Zeichen breit, rechtsbündig) enthält.

Hinweis:
Verwenden Sie ein Zeichenketten-Template mit geeigneten Formatoptionen, um das Format anzupassen.

© Copyright. Alle Rechte vorbehalten.

Welche Formatoptionen werden verwendet, um die Breite und Ausrichtung festzulegen?

Formatoptionen WIDTH und ALIGN.

a) Passen Sie den Quelltext wie folgt an:

```
  DATA(counter) = 0.
  LOOP AT numbers INTO DATA(number).

     counter = counter + 1.

     APPEND |{ counter WIDTH = 4 ALIGN = LEFT }: { number WIDTH = 10
ALIGN = RIGHT }|
          TO output.

  ENDLOOP.
```

Aufgabe 3: Ausgabe und Test
Schreiben Sie das Ergebnis in die Konsole. Anschließend aktivieren und testen Sie Ihre Klasse als Konsolen-App.

1. Rufen Sie am Ende der Methode die Methode **`out->write`** auf, um den Inhalt von **`output`** in die Konsole zu schreiben. Geben Sie dem Importing-Parameter **`name`** eine geeignete Beschriftung.
   a) Fügen Sie nach `ENDLOOP.` den folgenden Code hinzu:

```
  out->write(
         data   = output
         name   = |The first { max_count } Fibonacci Numbers|
                ) .
```

2. Aktivieren Sie die Klasse.
   a) Drücken Sie **`Strg + F3`**, um die Klasse zu aktivieren.
3. Führen Sie Ihre Klasse als Konsolen-App aus.
   a) Drücken Sie **`F9`**, um die Klasse als Konsolen-App auszuführen.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Definieren Sie einfache interne Tabellen.
- Daten mit einfachen internen Tabellen verarbeiten.

© Copyright. Alle Rechte vorbehalten.

Debugging eines ABAP-Programms

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Wechseln Sie in den Debugging-Modus.
- Steuern Sie die Ausführung von Code.
- Analysieren Sie den Inhalt von Datenobjekten.

Der Debugging-Modus

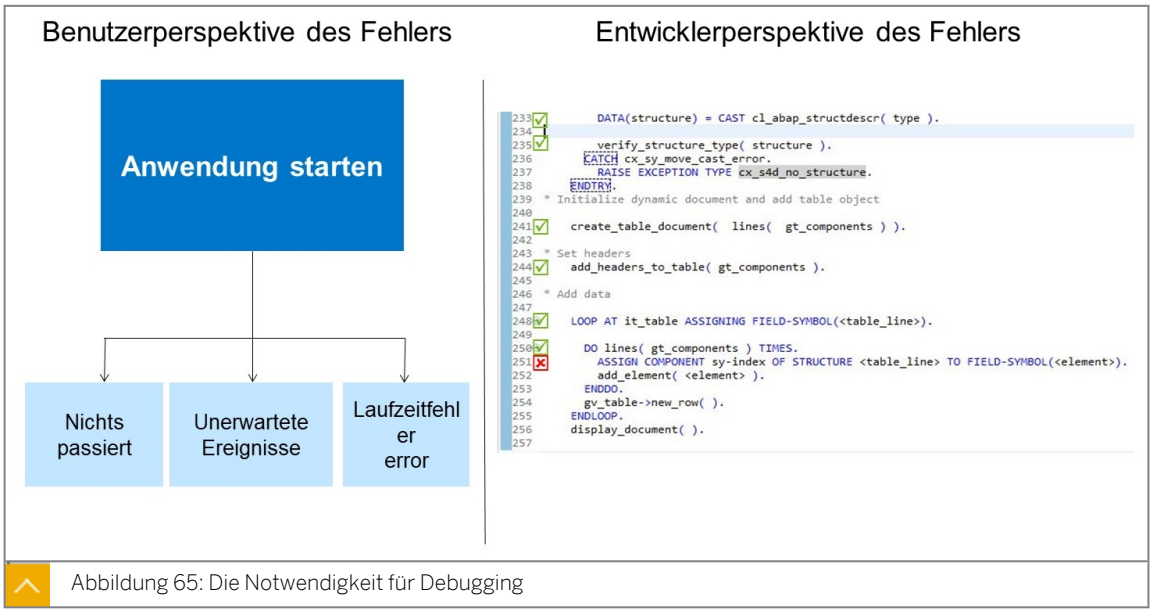


Abbildung 65: Die Notwendigkeit für Debugging

Es lässt sich nicht vermeiden, dass in Programmen Fehler auftreten. Diese Fehler können sich aber auf unterschiedliche Weise bemerkbar machen. Wenn ein Benutzer eine fehlerhafte Anwendung startet, kann sie abstürzen, es können unerwartete Ereignisse auftreten oder es kann überhaupt nichts passieren. Aus der Sicht der Benutzer, also auf der Ebene der Benutzungsoberfläche, ist es unmöglich zu erkennen, wie und warum dieser Fehler aufgetreten ist.

Als Entwickler müssen Sie jetzt das Programm näher untersuchen (d.h. Zeile für Zeile), um herauszufinden, welche Anweisungen und Kombinationen von Werten in den verschiedenen Programmvariablen den Fehler verursacht haben. Hier kommt der Debugger ins Spiel.

© Copyright. Alle Rechte vorbehalten.

Den Debugger starten

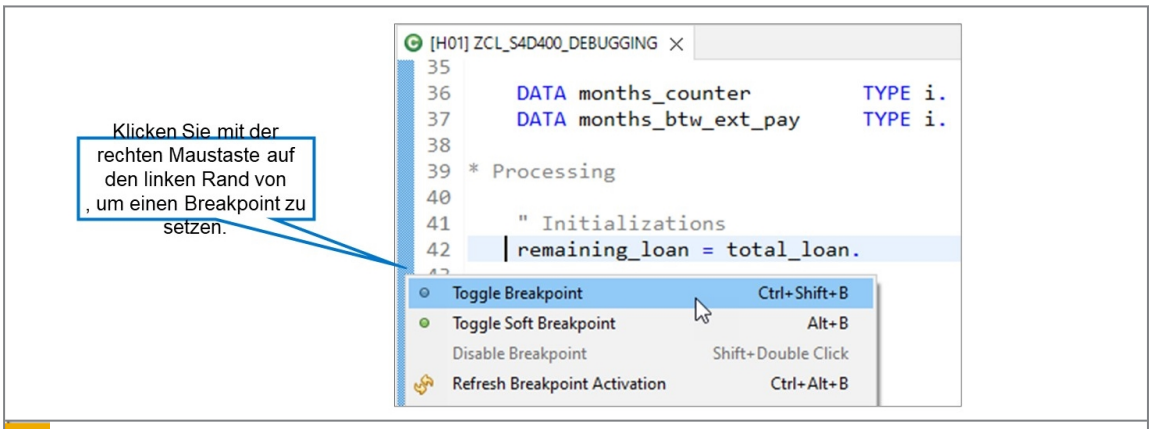


Abbildung 66: Den Debugger starten

Zum Debuggen eines ABAP-Programms setzen Sie einen Breakpoint und führen dann das Programm normal aus. Wenn das Programm den Breakpoint erreicht, wird es vom System unterbrochen und in ADT wird die ABAP-Debug-Perspektive geöffnet. Anschließend können Sie jede nachfolgende Anweisung einzeln ausführen, um zu sehen, welche Auswirkung sie auf das Programm hat. Sie können auch den Inhalt aller Variablen im Programm prüfen, um festzustellen, ob unerwartete Werte auftreten.

Um einen Breakpoint zu setzen oder zu entfernen, klicken Sie mit der rechten Maustaste auf den linken Rand des Editors, und wählen Sie „Toggle Breakpoint“. Alternativ können Sie auf den linken Rand doppelklicken. Beachten Sie, dass das Programm aktiviert werden muss, bevor Sie Breakpoints setzen können.

Breakpoints sind benutzerspezifisch und persistent, d.h., dass sie auch nach Ihrer Abmeldung von ADT und einer erneuten Anmeldung aktiv bleiben. Um zu verhindern, dass der Debugger an einem Breakpoint startet, müssen Sie ihn entweder löschen (mit der Funktion „Toggle Breakpoint“) oder mit der entsprechenden Funktion im Kontextmenü deaktivieren.

> Notiz:
> Abhängig von Ihren Personalisierungseinstellungen fordert ADT zuerst eine Bestätigung an, bevor die Debug-Perspektive automatisch geöffnet wird.

© Copyright. Alle Rechte vorbehalten.

Die Debug-Perspektive in ADT

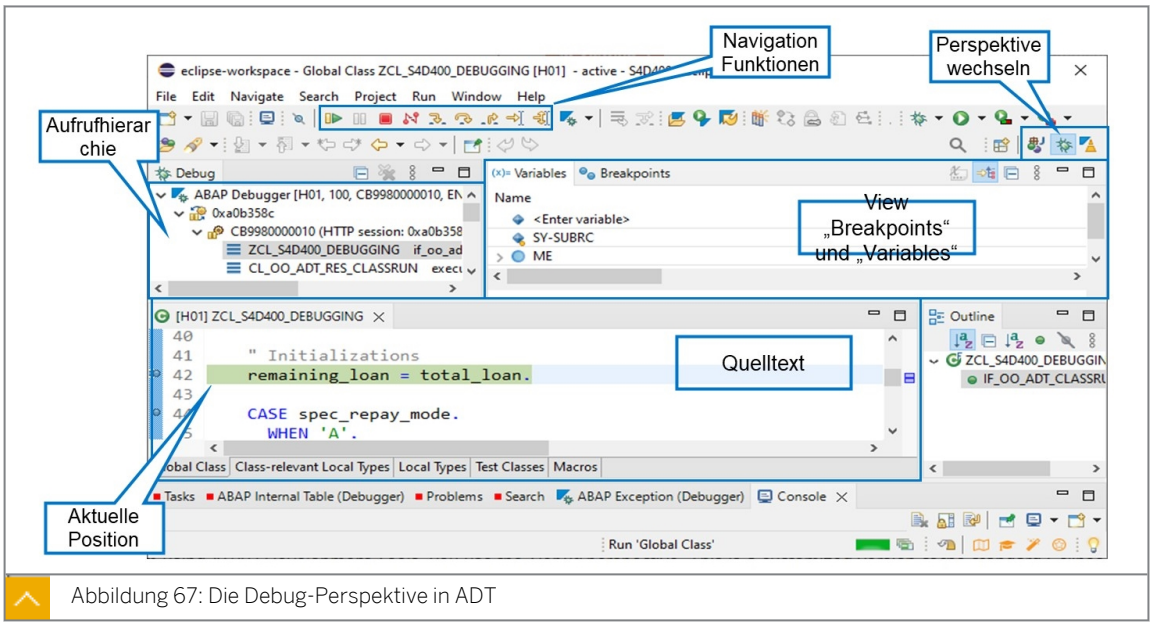


Abbildung 67: Die Debug-Perspektive in ADT

Beim Debuggen eines ABAP-Programms mit den ABAP Development Tools verwenden Sie die Debug-Perspektive. Hierbei handelt es sich um eine standardmäßige Eclipse-Debug-Perspektive, die Views und Funktionen enthält, die besonders für das Debugging wichtig sind.

Wichtige Elemente der Debug-Perspektive:

View „Source Code“

Die View *Source Code* ist der zentrale Teil der Debug-Perspektive. Hier wird der Quelltext angezeigt und die aktuelle Position im Programm markiert.

View „Variables“

Die View *Variables* ist ebenfalls sehr wichtig. In dieser View können Sie die aktuellen Werte von Variablen anzeigen.

View „Breakpoints“

Die View *Breakpoints* wird neben der View *Variables* angezeigt (in der obigen Abbildung nicht ausgewählt). In dieser View können Sie Breakpoints anzeigen, löschen oder anlegen. Breakpoints sind Stellen im Programm, an denen die normale Verarbeitung unterbrochen wird, und das System Ihnen den Debugger anzeigt, damit Sie den Zustand des Programms in genau diesem Moment analysieren können.

Navigationsfunktionen

Beim Debuggen eines Programms verwenden Sie die Navigationsfunktionen, um die Ausführung des Quelltexts zu steuern.

View „Debug“

Die View *Debug* links oben zeigt die Debugging-Sitzung und die Aufrufhierarchie. Diese benötigen Sie später, wenn Sie Aufrufe von Modularisierungseinheiten, z.B. Methoden, debuggen.

Perspektivenauswahl

Mit den Drucktasten für die Perspektivenauswahl (*Perspective Selector*) in der rechten oberen Ecke können Sie zurück zur ABAP-Perspektive wechseln.

© Copyright. Alle Rechte vorbehalten.

Hinweis:
Da viele wichtige Editorwerkzeuge in der Debugging-Perspektive fehlen, wird empfohlen, nach Abschluss des Debuggings zurück zur ABAP-Perspektive zu wechseln.

Codeausführung steuern

Navigationsfunktionen

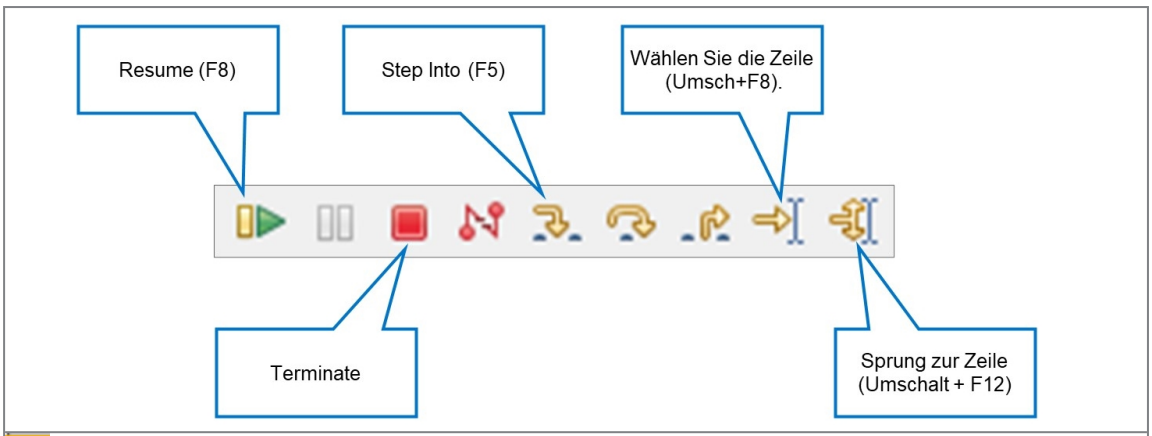


Abbildung 68: Navigationsfunktionen

Sobald Sie mit dem Debugging begonnen haben, verwenden Sie die Navigationsfunktionen, um die Ausführung des Quelltexts zu steuern.

Wichtige Navigationsfunktionen:

Step Into (`F5`)

Wählen Sie *Step Into*, oder drücken Sie **`F5`**, um einen einzelnen Schritt auszuführen. Verwenden Sie diese Funktion für eine schrittweise Analyse. Beispiel: Sie möchten feststellen, welcher Quelltextblock einer Kontrollstruktur tatsächlich ausgeführt wird.

Resume (`F8`)

Wählen Sie *Resume*, oder drücken Sie **`F8`**, um das Programm bis zum nächsten Breakpoint auszuführen. Wenn der Debugger keine weiteren Breakpoints findet, wird das Programm bis zum Ende ausgeführt und die Debugging-Sitzung beendet.

Run to Line (`Umschalt + F8`)

Wählen Sie *Run to Line*, oder drücken Sie **`Umschalt + F8`**, um das Programm bis zur aktuellen Cursorposition auszuführen. Das Klicken auf eine Quelltextzeile und das Wählen dieser Funktion bietet eine bequeme Alternative zum Setzen eines Breakpoints, Wählen von „Resume“ und Entfernen des Breakpoints.

Jump to Line (`Umschalt + F12`)

Wählen Sie *Jump to Line*, oder drücken Sie **`Umschalt + F12`**, um einige Quelltextzeilen zu überspringen oder zu einem bereits ausgeführten Quelltext zurückzukehren. Diese Funktion kann hilfreich sein, um zu simulieren, was passieren würde, wenn ein bestimmter Quelltextteil entfernt würde, oder um ein Stück Quelltext zu debuggen, bei dem die Analyse beim ersten Mal versäumt wurde. Beachten Sie, dass es sich hierbei

© Copyright. Alle Rechte vorbehalten.

tatsächlich um einen Sprung handelt und nicht um die Ausführung des Quelltexts. Wenn Sie rückwärts springen, werden Änderungen an Datenobjekten nicht rückgängig gemacht.

Terminate

Wählen Sie *Terminate*, wenn Sie mit dem Debugging fertig sind und das verbleibende Programm nicht ausführen möchten. Die Debugging-Sitzung wird sofort beendet.

Spezielle Breakpoints

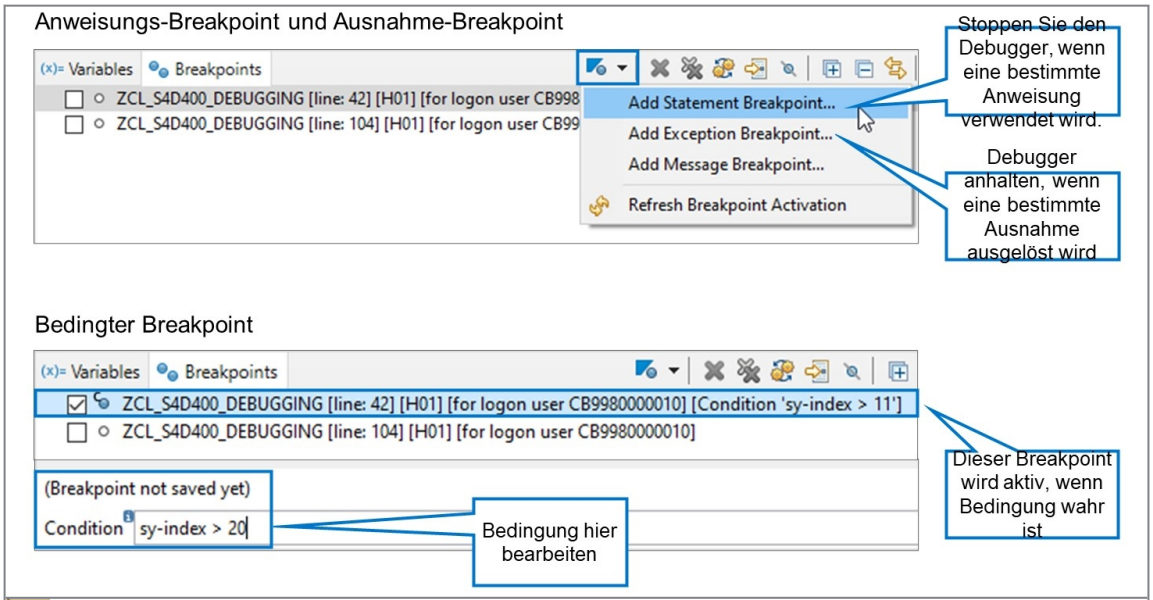


Abbildung 69: Spezielle Breakpoints anlegen

Sie haben gelernt, dass Sie Breakpoints anlegen und verwalten können, indem Sie auf den linken Rand der View „ABAP Editor“ klicken. Dies funktioniert auch mit der View „ABAP Editor“ in der Debug-Perspektive.

Darüber hinaus können Sie in die View „Breakpoints“ wechseln und dort Ihre Breakpoints verwalten.

In der View „Breakpoints“ können Sie auch spezielle Breakpoints anlegen:

Anweisungs-Breakpoint

Ein Anweisungs-Breakpoint ist nicht an eine bestimmte Quelltextzeile, sondern an eine bestimmte ABAP-Anweisung angehängt. Ein Anweisungs-Breakpoint für die Anweisung CLEAR bewirkt z.B., dass das Programm im Debugger gestoppt wird, wenn eine CLEAR-Anweisung ausgeführt wird, unabhängig davon, wo sich diese Anweisung befindet.

Um einen Anweisungs-Breakpoint anzulegen, öffnen Sie die Dropdown-Liste in der Symbolleiste der View „Breakpoints“, und wählen Sie *Add Statement Breakpoint ...*

Ausnahme-Breakpoint

Ein Ausnahme-Breakpoint wird an eine bestimmte Ausnahme angehängt. Dies bewirkt, dass das Programm im Debugger gestoppt wird, wenn diese bestimmte Ausnahme ausgelöst wird, und zwar unabhängig davon, ob diese Ausnahme vom Programm behandelt wird oder zu einem Laufzeitfehler führt. Um einen Ausnahme-Breakpoint anzulegen, öffnen Sie die Dropdown-Liste in der Symbolleiste der View „Breakpoints“, und wählen Sie „Add Exception Breakpoint ...“.

Bedingter Breakpoint

© Copyright. Alle Rechte vorbehalten.

Sie wandeln einen Breakpoint in einen bedingten Breakpoint um, indem Sie eine Bedingung hinzufügen. Wenn die Programmausführung auf einen bedingten Breakpoint trifft, stoppt das Programm im Debugger nur, wenn die Bedingung erfüllt ist. Wenn sich beispielsweise ein Breakpoint zwischen DO und ENDDO befindet, wird das Programm bei jeder Iteration im Debugger angehalten. Wenn Sie jedoch eine Bedingung „sy-index > 20“ hinzufügen, ignoriert der Debugger diesen Breakpoint während der ersten 20 Iterationen und stoppt nur in den folgenden Iterationen.

Um einem Breakpoint eine Bedingung hinzuzufügen, wählen Sie den Breakpoint in der Liste der Breakpoints aus, und geben Sie die Bedingung im Feld „Condition“ ein. Drücken Sie die Eingabetaste, um den Breakpoint mit der Bedingung zu sichern.

Watchpoints

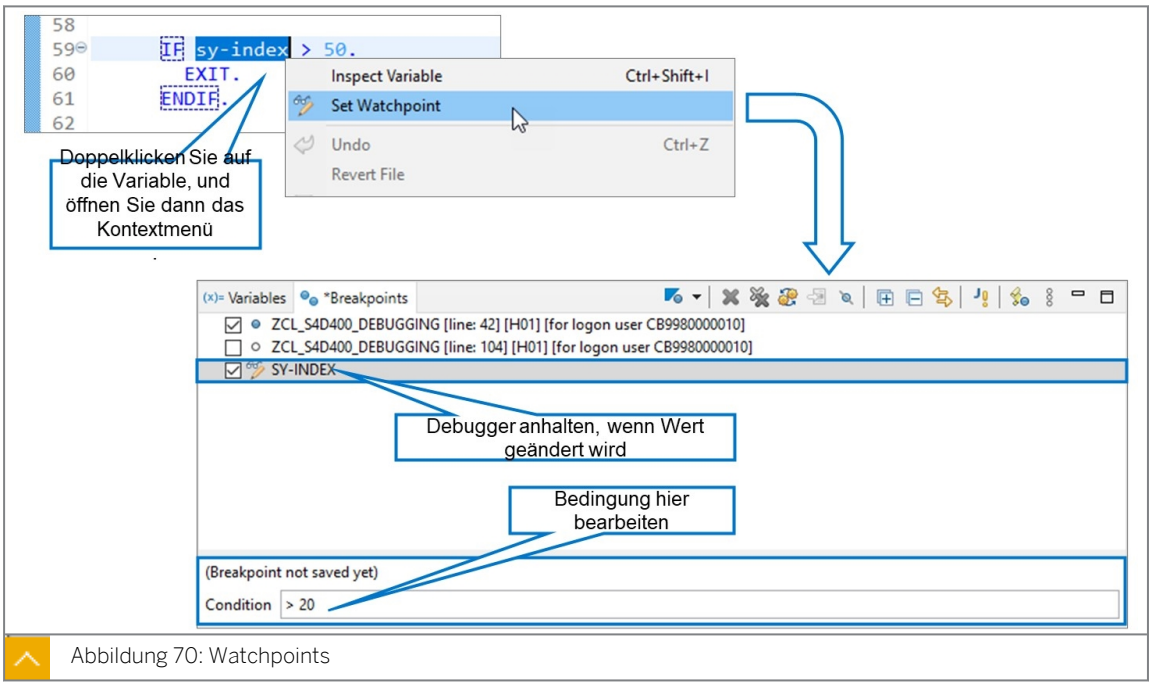


Abbildung 70: Watchpoints

Wenn Ihnen ein unerwarteter Wert einer Variable Probleme bereitet, können Sie diesen Wert im Laufe des Programms mit einem Watchpoint verfolgen.

Ein einfacher Watchpoint für eine Variable bewirkt, dass das Programm im Debugger anhält, wenn sich der Wert dieser Variablen ändert. Durch das Hinzufügen einer Bedingung können Sie erreichen, dass das Programm nicht bei jeder Wertänderung der Variablen stoppt, sondern nur in den Fällen, in denen auch die Bedingung erfüllt ist.

Um einen Watchpoint für eine Variable zu setzen, doppelklicken Sie auf die Variable im Quelltext, klicken Sie mit der rechten Maustaste darauf, und wählen Sie *Watchpoint setzen*. Dadurch wird ein Watchpoint für diese Variable angelegt, den Sie dann in der *View Breakpoints* sehen können.

Um eine Bedingung zu einem Watchpoint hinzuzufügen, wählen Sie sie in der Liste der Breakpoints aus, und geben Sie die Bedingung in das Feld *Bedingung* ein. Wählen Sie *Sichern*, um den Watchpoint mit der Bedingung zu sichern.

© Copyright. Alle Rechte vorbehalten.

Analyse von Datenobjekten

Inhalt von Datenobjekten anzeigen

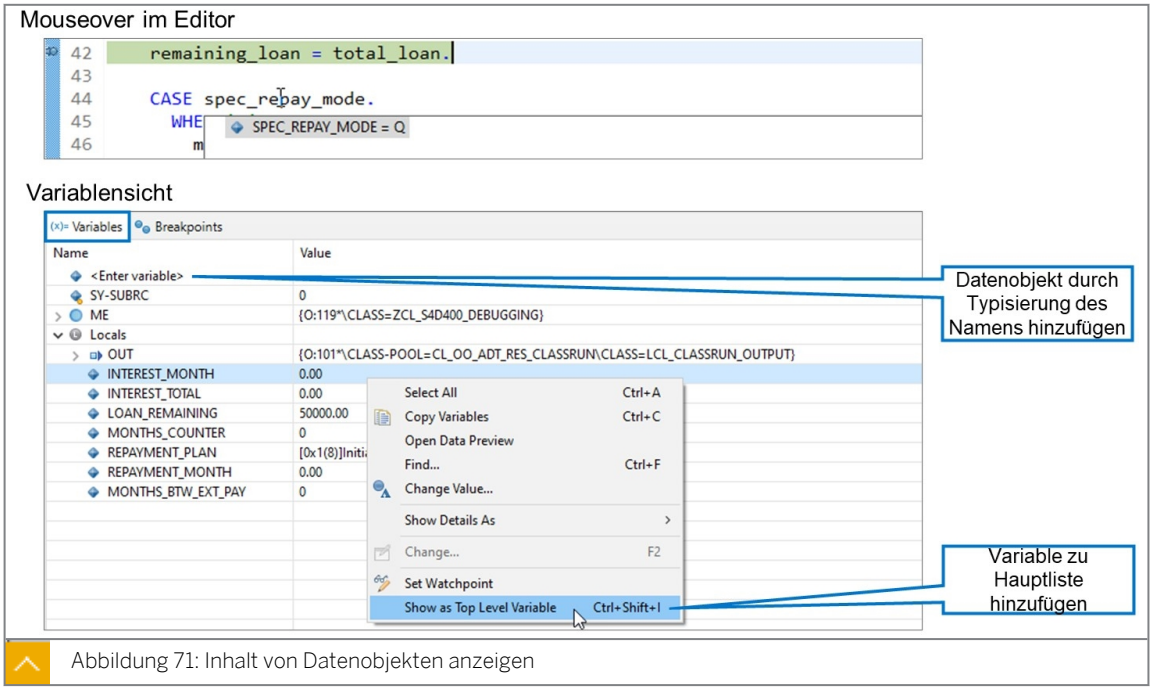


Abbildung 71: Inhalt von Datenobjekten anzeigen

Eine Möglichkeit, den Inhalt von Datenobjekten im Debugger zu analysieren, ist die Mouseover-Funktionalität des ABAP Editors. Positionieren Sie im Debugging-Modus den Cursor auf den Namen eines Datenobjekts und warten Sie einen Moment. Ein Dialogfenster mit dem Inhalt des Datenobjekts wird geöffnet.

Eine weitere Möglichkeit, den Inhalt von Datenobjekten im Debugger zu analysieren, ist die View „Variables“. Diese View zeigt eine Liste der Datenobjekte und ihre aktuellen Werte an. Die Hauptliste mit den sogenannten Variablen der obersten Ebene enthält standardmäßig einige eingebaute Datenobjekte. Im Beispiel sind dies SY-SUBRC und ME. Expandieren Sie den Knoten „Locals", um eine Liste aller variablen Datenobjekte anzuzeigen, die im aktuellen Verarbeitungsblock definiert sind.

Es gibt drei Möglichkeiten, der Hauptliste in der View „Variables“ Datenobjekte hinzuzufügen:

- Doppelklicken Sie im Editor auf den Namen eines Datenobjekts.
- Klicken Sie in der Variablenliste mit der linken Maustaste auf den Platzhalter <Variable eingeben>, und geben Sie den Namen des Datenobjekts ein.
- Klicken Sie mit der rechten Maustaste auf eine Variable im Knoten „Locals“, und wählen Sie „Show as Top Level Variable“.

Hinweis:
Um ein Datenobjekt aus der Liste zu entfernen, klicken Sie mit der rechten Maustaste darauf, und wählen Sie *Remove*.

© Copyright. Alle Rechte vorbehalten.

Inhalt interner Tabellen anzeigen

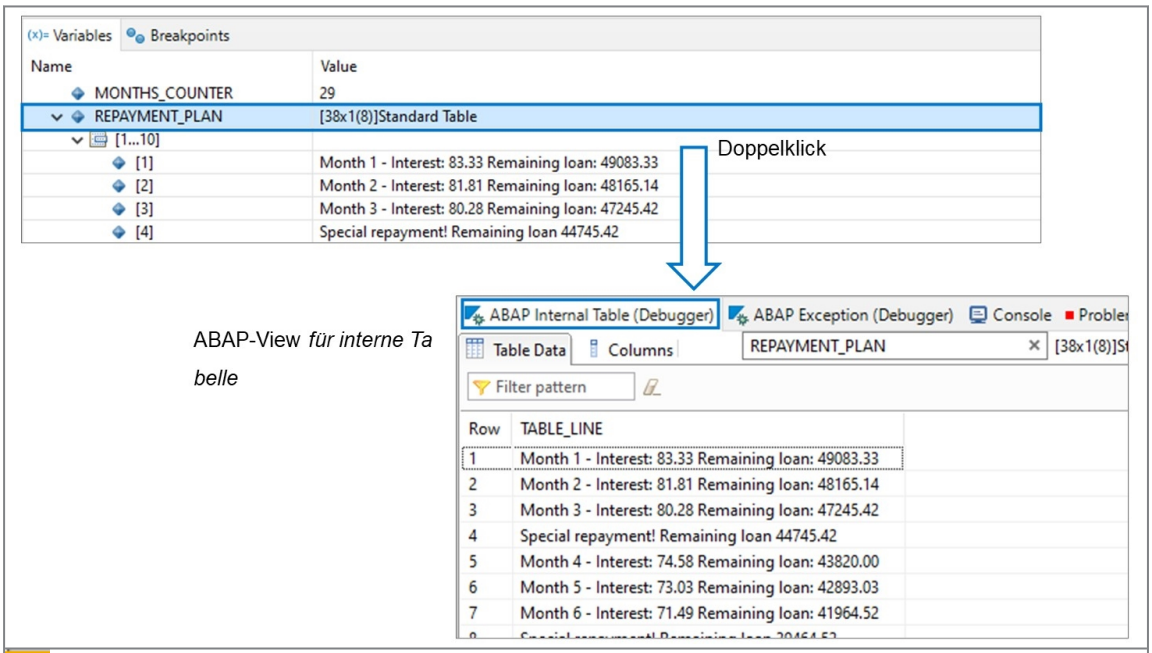


Abbildung 72: Inhalt interner Tabellen anzeigen

Bei internen Tabellen zeigt die *View Variables* eine Übersicht mit der Anzahl der Zeilen und der Anzahl der Spalten an. Expandieren Sie die Hierarchie unterhalb des Namens der internen Tabelle, um den Inhalt der Zeilen anzuzeigen. Doppelklicken Sie auf den Namen der internen Tabelle, um sie in der Sicht *ABAP Internal Table* zu analysieren.

Die View *ABAP Internal Table* ist ein Debugger-Werkzeug zur Analyse des Inhalts interner Tabellen. In der Standardkonfiguration der *Debugger-Perspektive* ist die View *ABAP Internal Table* eine der Views unterhalb des Quelltexts. Sie können den Tabelleninhalt filtern, indem Sie ein Filtermuster eingeben und die *Eingabetaste* drücken.

Variablenwerte ändern

Abhängig von Ihren Berechtigungen können Sie den Wert von Variablen während des Debuggings ändern.

© Copyright. Alle Rechte vorbehalten.

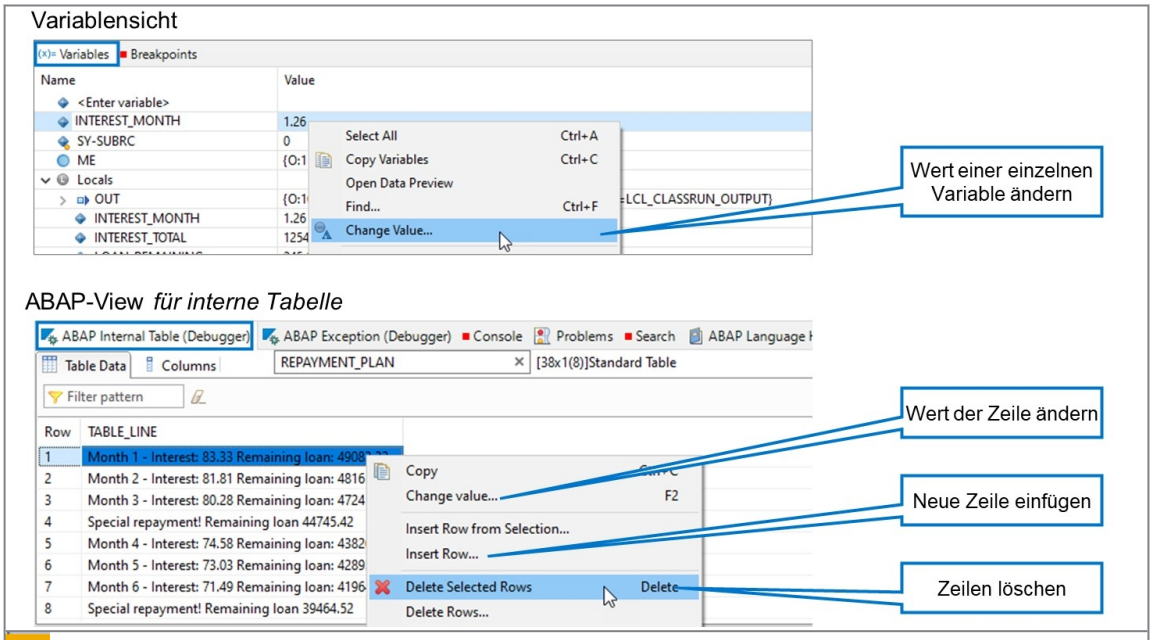


Abbildung 73: Werte von Variablen ändern

Bei einfachen Variablen suchen Sie die Variable in der Ansicht *Variablen*, klicken mit der rechten Maustaste darauf und wählen *Wert ändern...*.

Um den Inhalt einer internen Tabelle zu ändern, muss zwischen dem Ändern des Werts einer vorhandenen Zeile und dem Hinzufügen oder Löschen von Zeilen unterschieden werden.

Wie in der Abbildung dargestellt, sind die folgenden Funktionen verfügbar, wenn Sie mit der rechten Maustaste auf die Sicht *ABAP Internal Table* klicken:

Wert ändern...

Wählen Sie *Wert ändern...*, um den Inhalt einer vorhandenen Zeile zu ändern.

Zeile einfügen...

Wählen Sie *Zeile einfügen...*, um eine neue Zeile hinzuzufügen. Sie können entscheiden, ob Sie die neue Zeile anhängen oder an der gewählten Position einfügen möchten.

Ausgewählte Zeilen löschen

Wählen Sie *Ausgewählte Zeilen löschen*, um die ausgewählten Zeilen zu entfernen, bevor Sie mit der rechten Maustaste darauf klicken. Um eine Zeile auszuwählen, klicken Sie mit der linken Maustaste darauf. Um mehr als eine Zeile auszuwählen, halten Sie die *Strg* -Taste oder die *Umschalt* -Taste gedrückt, wenn Sie mit der linken Maustaste auf zusätzliche Zeilen klicken.

Zeilen löschen...

Wählen Sie *Zeilen löschen...*, um einen größeren Bereich von Zeilen oder sogar alle Zeilen zu entfernen. Sie werden nach der Nummer der Startzeile und der Endzeile gefragt, die Sie löschen möchten.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Übung 7

ABAP-Code debuggen

In dieser Übung analysieren Sie ABAP-Code mit dem ABAP Debugger.

Vorlage:

/LRN/CL_S4D400_BTT_DEBUG (globale Klasse)

Lösung:

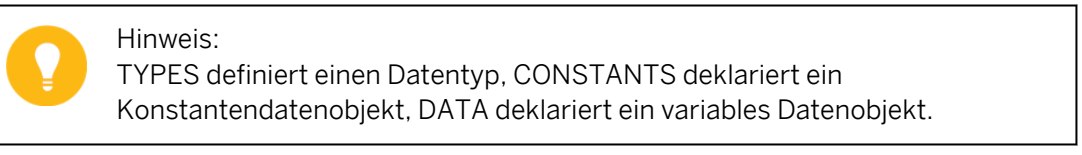
Keine, die Klasse bleibt unverändert.

Aufgabe 1: Vorbereitung
Kopieren Sie die Klasse, die Sie debuggen möchten.

1. Öffnen Sie den Quelltext der globalen Klasse /LRN/CL_S4D400_BTT_DEBUG im ABAP Editor.
2. Verknüpfen Sie die View *Project Explorer* mit dem Editor.
3. Kopieren Sie die Klasse /LRN/CL_S4D400_BTT_DEBUG in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: ZCL_##_DEBUG, wobei ## für Ihre Gruppennummer steht).
4. Aktivieren und testen Sie die Klasse.
5. Überprüfen Sie die Ausgabe in der View *Console*.

Aufgabe 2: Startwerte analysieren
Starten Sie den Debugger bei der ersten ausführbaren Anweisung in der Methode **`if_oo_adt_classrun~main( )`**, und analysieren Sie die Werte einiger Variablen- und Konstantendatenobjekte.

1. Setzen Sie bei der ersten Anweisung einen Breakpoint, der keinen Typ definiert bzw. kein Datenobjekt deklariert.



2. Führen Sie die Klasse als Konsolen-App aus, und rufen Sie den Debugger auf.
3. Zeigen Sie den Wert des Datenobjekts **`loan_remaining`** und **`loan_total`** in der View *Variables* an.

Aufgabe 3: Programmausführung steuern
Setzen Sie Breakpoints und Watchpoints. Führen Sie einzelne Schritte aus, oder setzen Sie die Ausführung fort, bis der nächste Breakpoint oder Watchpoint erreicht ist. Überwachen Sie die Wertänderungen der Datenobjekte.

© Copyright. Alle Rechte vorbehalten.

1. Führen Sie einen einzelnen Schritt aus, um die Wertzuweisung in der aktuellen Zeile zu debuggen.
2. Zeigen Sie den Inhalt des Datenobjekts **`spec_repay_mode`** an. Führen Sie anschließend einen weiteren Einzelschritt aus, um zu sehen, welche WHEN-Verzweigung der CASE-Kontrollstruktur ausgeführt wird.
3. Setzen Sie einen Watchpoint für die Variable **`loan_remaining`**, und setzen Sie die Programmausführung fort. Wo stoppt die Programmausführung wieder?
4. Zeigen Sie den Inhalt des Datenobjekts **`repayment_plan`** an. Führen Sie dann einen weiteren Einzelschritt aus, um zu sehen, wie es mit der APPEND-Anweisung gefüllt wird.

Notiz:
Da **`repayment_plan`** eine interne Tabelle ist, wird sie nicht nur in der View *Variables*, sondern auch in der View *ABAP Internal Table (Debugger)* unterhalb des Editors angezeigt.

5. Untersuchen Sie das Zeichenketten-Template in der APPEND-Anweisung und beziehen Sie es auf die resultierende erste Zeile in der internen Tabelle **`repayment_plan`**. Zeigen Sie den Inhalt der Datenobjekte an, die in den eingebetteten Ausdrücken vorkommen.
6. Setzen Sie einen weiteren Watchpoint für die Variable **`repayment_plan`**, und setzen Sie die Programmausführung einige Male fort. Immer wenn die Ausführung einen der Watchpoints erreicht, analysieren Sie den Wert von **`loan_remaining`**. Neue Zeilen werden zu **`repayment_plan`** hinzugefügt.
7. Setzen Sie nach einer Weile einen Anweisungs-Breakpoint für alle EXIT-Anweisungen, und löschen Sie die beiden Watchpoints.
8. Setzen Sie die Programmausführung fort, bis Sie die erste EXIT-Anweisung erreicht haben. Deaktivieren Sie den Anweisungs-Breakpoint für die **`EXIT`**-Anweisung. Führen Sie dann Einzelschritte aus, bis Sie den Ausgabeteil des Programms erreichen.
9. Öffnen Sie die View *Console*. Führen Sie das verbleibende Programm aus, und verfolgen Sie die Ausgabe in der View *Console*.

Notiz:
Drücken Sie nicht **`F5`**, um die Ausgabe zu debuggen. Verwenden Sie stattdessen **`F6`**.

Notiz:
Wie Sie später im Kurs lernen werden, handelt es sich bei `out->write( ... )` nicht um eine ABAP-Anweisung, sondern um einen wiederverwendbaren Quelltextblock, der aus vielen ABAP-Anweisungen besteht. Wenn Sie **`F5`** drücken, gelangen Sie über *Step Into* in diesen Quelltext, um ihn im Detail zu analysieren. Mit **`F6`** (*Step Over*) überspringen Sie den Quelltextblock und behandeln ihn wie eine einzelne Anweisung.

© Copyright. Alle Rechte vorbehalten.

10. Vergessen Sie beim Beenden der Anwendung nicht, wieder in die *ABAP-Perspektive* zu wechseln.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2
Lösung 7

ABAP-Code debuggen

In dieser Übung analysieren Sie ABAP-Code mit dem ABAP Debugger.

Vorlage:

/LRN/CL_S4D400_BTT_DEBUG (globale Klasse)

Lösung:

Keine, die Klasse bleibt unverändert.

Aufgabe 1: Vorbereitung
Kopieren Sie die Klasse, die Sie debuggen möchten.

1. Öffnen Sie den Quelltext der globalen Klasse /LRN/CL_S4D400_BTT_DEBUG im ABAP Editor.
   a) Wählen Sie in der *Eclipse*-Symbolleiste *Open ABAP Development Object*. Drücken Sie alternativ **`Strg + Umschalt + A`**.
   b) Geben Sie als Suchzeichenfolge **`/LRN/CL_S4D400_BTT`** ein.
   c) Wählen Sie in der Liste der Entwicklungsobjekte */LRN/CL_S4D400_BTT_DEBUG* aus, und wählen Sie dann *OK*.
2. Verknüpfen Sie die View *Project Explorer* mit dem Editor.
   a) Suchen Sie in der Symbolleiste *Project Explorer* die Drucktaste *Link with Editor*. Wählen Sie die Drucktaste, falls diese noch nicht markiert ist. Dadurch sollte das Entwicklungsobjekt, das im Editor geöffnet ist, im *Project Explorer* hervorgehoben sein.
3. Kopieren Sie die Klasse /LRN/CL_S4D400_BTT_DEBUG in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: ZCL_##_DEBUG, wobei ## für Ihre Gruppennummer steht).
   a) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Klasse /LRN/CL_S4D400_BTT_DEBUG, um das Kontextmenü zu öffnen.
   b) Wählen Sie im Kontextmenü *Duplicate ...*
   c) Geben Sie den Namen Ihres Pakets (**`ZS4D400_##`**, wobei ## für Ihre Gruppennummer steht) und den Namen für die neue Klasse (**`ZCL_##_DEBUG`**) ein, und wählen Sie *Next*.
   d) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.
4. Aktivieren und testen Sie die Klasse.
   a) Drücken Sie **`Strg + F3`**, um die Klasse zu aktivieren.
   b) Drücken Sie **`F9`**, um die Klasse auszuführen.
5. Überprüfen Sie die Ausgabe in der View *Console*.

© Copyright. Alle Rechte vorbehalten.

a) Überprüfen Sie die View *Console*, die planmäßig als neue Registerkarte unterhalb der Editor-View geöffnet wird.

b) Wenn die View *Console* nicht angezeigt wird, öffnen Sie sie, indem Sie *Window* → *Show View* → *Other* wählen. Doppelklicken Sie in der Trefferliste auf *Console*.

Aufgabe 2: Startwerte analysieren

Starten Sie den Debugger bei der ersten ausführbaren Anweisung in der Methode **`if_oo_adt_classrun~main( )`**, und analysieren Sie die Werte einiger Variablen- und Konstantendatenobjekte.

1. Setzen Sie bei der ersten Anweisung einen Breakpoint, der keinen Typ definiert bzw. kein Datenobjekt deklariert.

Hinweis:
TYPES definiert einen Datentyp, CONSTANTS deklariert ein Konstantendatenobjekt, DATA deklariert ein variables Datenobjekt.

   a) Doppelklicken Sie auf den linken Rand des Editors neben der Zeile `loan_remaining = loan_total.`, um einen Breakpoint zu setzen.

2. Führen Sie die Klasse als Konsolen-App aus, und rufen Sie den Debugger auf.
   a) Drücken Sie **`F9`**, um die Klasse auszuführen.
   b) Wenn Sie gefragt werden, ob Sie zur Debug-Perspektive wechseln möchten, markieren Sie *Remember my decision*, und wählen Sie *OK*.

3. Zeigen Sie den Wert des Datenobjekts **`loan_remaining`** und **`loan_total`** in der View *Variables* an.
   a) Doppelklicken Sie in der aktuellen Quelltextzeile (die Zeile mit grünem Hintergrund) auf `loan_remaining`.
   b) Doppelklicken Sie in derselben Quelltextzeile auf **`loan_total`**.

Aufgabe 3: Programmausführung steuern

Setzen Sie Breakpoints und Watchpoints. Führen Sie einzelne Schritte aus, oder setzen Sie die Ausführung fort, bis der nächste Breakpoint oder Watchpoint erreicht ist. Überwachen Sie die Wertänderungen der Datenobjekte.

1. Führen Sie einen einzelnen Schritt aus, um die Wertzuweisung in der aktuellen Zeile zu debuggen.
   a) Wählen Sie in der Symbolleiste *Step Into (F5)*, oder drücken Sie **`F5`**.
   b) Prüfen Sie, ob sich der Wert von **`loan_remaining`** von **`0..00`** in **`5000,00`** geändert hat.

2. Zeigen Sie den Inhalt des Datenobjekts **`spec_repay_mode`** an. Führen Sie anschließend einen weiteren Einzelschritt aus, um zu sehen, welche WHEN-Verzweigung der CASE-Kontrollstruktur ausgeführt wird.
   a) Doppelklicken Sie in der nächsten Quelltextzeile auf **`spec_repay_mode`**.
   b) Drücken Sie **`F5`**, um zu sehen, dass das Programm zur Quelltextzeile `WHEN 'Q'.` springt.

© Copyright. Alle Rechte vorbehalten.

3. Setzen Sie einen Watchpoint für die Variable **`loan_remaining`**, und setzen Sie die Programmausführung fort. Wo stoppt die Programmausführung wieder?
   a) Klicken Sie in der View *Variables* mit der rechten Maustaste auf *LOAN_REMAINING*, und wählen Sie *Set Watchpoint*.
   b) Wählen Sie in der Symbolleiste *Resume (F8)*, oder drücken Sie **`F8`**.
   c) Die Programmausführung stoppt unmittelbar nach der Quelltextzeile `loan_remaining = loan_remaining - repayment_month.`, genauer gesagt, in der nächsten Quelltextzeile mit ausführbarem Code.
4. Zeigen Sie den Inhalt des Datenobjekts **`repayment_plan`** an. Führen Sie dann einen weiteren Einzelschritt aus, um zu sehen, wie es mit der APPEND-Anweisung gefüllt wird.

   Notiz:
   Da **`repayment_plan`** eine interne Tabelle ist, wird sie nicht nur in der View *Variables*, sondern auch in der View *ABAP Internal Table (Debugger)* unterhalb des Editors angezeigt.

   a) Doppelklicken Sie auf **`repayment_plan`** am Ende der APPEND-Anweisung.
   b) Drücken Sie **`F5`**, um zu sehen, wie **`repayment_plan`** mit einer ersten Zeile gefüllt wird.
5. Untersuchen Sie das Zeichenketten-Template in der APPEND-Anweisung und beziehen Sie es auf die resultierende erste Zeile in der internen Tabelle **`repayment_plan`**. Zeigen Sie den Inhalt der Datenobjekte an, die in den eingebetteten Ausdrücken vorkommen.
   a) Doppelklicken Sie auf die Datenobjekte, die zwischen den geschweiften Klammern stehen, um deren Inhalt anzuzeigen.
6. Setzen Sie einen weiteren Watchpoint für die Variable **`repayment_plan`**, und setzen Sie die Programmausführung einige Male fort. Immer wenn die Ausführung einen der Watchpoints erreicht, analysieren Sie den Wert von **`loan_remaining`**. Neue Zeilen werden zu **`repayment_plan`** hinzugefügt.
   a) Klicken Sie in der View *Variables* mit der rechten Maustaste auf *REPAYMENT_PLAN*, und wählen Sie *Set Watchpoint*.
   b) Drücken Sie **`F8`**, um die Programmausführung fortzusetzen.
   c) Analysieren Sie die Datenobjekte in den Views *Variables* und *ABAP Internal Table (Debugger)*.
7. Setzen Sie nach einer Weile einen Anweisungs-Breakpoint für alle EXIT-Anweisungen, und löschen Sie die beiden Watchpoints.
   a) Navigieren Sie zur View *Breakpoints*.

      Hinweis:
      Sie finden die View *Breakpoints* neben der View *Variables*.

   b) Expandieren Sie in der Symbolleiste der View *Breakpoints* die Dropdown-Drucktaste ganz links, und wählen Sie *Add Statement Breakpoint ...*.

© Copyright. Alle Rechte vorbehalten.

c) Geben Sie im daraufhin angezeigten Dialogfenster **`EXIT`** als Suchzeichenfolge ein, klicken Sie auf den Wert *EXIT* in der Trefferliste, und wählen Sie *OK*.

d) Klicken Sie in der View *Breakpoints* mit der rechten Maustaste auf den Watchpoint *LOAN_REMAINING*, und wählen Sie *Remove*.

e) Klicken Sie in der View *Breakpoints* mit der rechten Maustaste auf den Watchpoint *REPAYMENT_PLAN*, und wählen Sie *Remove*.

8. Setzen Sie die Programmausführung fort, bis Sie die erste EXIT-Anweisung erreicht haben. Deaktivieren Sie den Anweisungs-Breakpoint für die **`EXIT`**-Anweisung. Führen Sie dann Einzelschritte aus, bis Sie den Ausgabeteil des Programms erreichen.

a) Drücken Sie **`F8`**, um die Programmausführung fortzusetzen.

b) Wechseln Sie zur View *Breakpoints*, und entmarkieren Sie die Zeile mit **`EXIT [Statement]`**.

c) Drücken Sie **`F5`**, bis Sie die erste Quelltextzeile erreichen, die mit `out->write(` beginnt.

9. Öffnen Sie die View *Console*. Führen Sie das verbleibende Programm aus, und verfolgen Sie die Ausgabe in der View *Console*.

Notiz:
Drücken Sie nicht **`F5`**, um die Ausgabe zu debuggen. Verwenden Sie stattdessen **`F6`**.

Notiz:
Wie Sie später im Kurs lernen werden, handelt es sich bei `out->write( ... )` nicht um eine ABAP-Anweisung, sondern um einen wiederverwendbaren Quelltextblock, der aus vielen ABAP-Anweisungen besteht. Wenn Sie **`F5`** drücken, gelangen Sie über *Step Into* in diesen Quelltext, um ihn im Detail zu analysieren. Mit **`F6`** (*Step Over*) überspringen Sie den Quelltextblock und behandeln ihn wie eine einzelne Anweisung.

a) Drücken Sie mehrmals **`F6`**, bis Sie das Ende der Anwendung erreicht haben.

b) Analysieren Sie die zusätzliche Ausgabe in der View *Console*, und vergleichen Sie sie mit dem Zeichenketten-Template in der vorherigen Quelltextzeile.

10. Vergessen Sie beim Beenden der Anwendung nicht, wieder in die *ABAP-Perspektive* zu wechseln.

a) Wählen Sie *ABAP* ganz rechts in der Eclipse-Symbolleiste.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Wechseln Sie in den Debugging-Modus.
- Steuern Sie die Ausführung von Code.
- Analysieren Sie den Inhalt von Datenobjekten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 2

Überprüfung des Lernerfolgs

1. Welche der folgenden Elemente können Sie verwenden, um einen Kommentar in ABAP zu kennzeichnen?

   Wählen Sie die richtigen Antworten.

   - ☐ **A** // am Anfang einer Zeile
   - ☐ **B** * am Anfang einer Zeile
   - ☐ **C** -- an beliebiger Stelle in der Zeile
   - ☐ **D** " an beliebiger Stelle in der Zeile

2. Welche der folgenden eingebauten ABAP-Typen sind vollständig?

   Wählen Sie die richtigen Antworten.

   - ☐ **A** C
   - ☐ **B** D
   - ☐ **C** P
   - ☐ **D** I

3. Eine Variable deklarieren Sie wie folgt: DATA var TYPE I VALUE 100. Anschließend verwenden Sie die Anweisung „CLEAR var“. Wie lautet der Wert von „var“ nach der CLEAR-Anweisung?

   Wählen Sie die richtige Antwort.

   - ☐ **A** 0
   - ☐ **B** 100

4. Das Ergebnis des Ausdrucks result = var MOD 2. ist 1. Was sagt Ihnen das über den Wert der Variablen var aus?

   Wählen Sie die richtige Antwort.

   - ☐ **A** „var“ ist eine gerade Zahl.
   - ☐ **B** „var“ ist eine ungerade Zahl.

© Copyright. Alle Rechte vorbehalten.

5. Sie möchten zwei Zeichenfolgen verketten. Welcher Operator ist der richtige?

   Wählen Sie die richtige Antwort.

   - ☐ A +
   - ☐ B &
   - ☐ C ++
   - ☐ D &&

6. Auf die IF-Bedingung „IF a > 10.“ folgt die ELSEIF-Bedingung „ELSEIF a = 25.“ Die Variable a hat den Wert 25. Welche Codeverzweigungen werden ausgeführt?

   Wählen Sie die richtige Antwort.

   - ☐ A Die durch „IF a > 10“ eingeführte Verzweigung.
   - ☐ B Die durch „ELSEIF a = 25“ eingeführte Verzweigung.
   - ☐ C Beide Verzweigungen.

7. Welche der folgenden Aussagen zu Ausnahmen sind zutreffend?

   Wählen Sie die richtigen Antworten.

   - ☐ A Alle nicht behandelten Ausnahmen führen zu Laufzeitfehlern.
   - ☐ B Alle Ausführungen sind behandelbar.
   - ☐ C Manche Ausnahmen sind behandelbar, andere nicht.
   - ☐ D Ein „TRY...ENDTRY“-Block muss mindestens zwei CATCH-Anweisungen enthalten.

8. Wie können Sie eine Schleife in ABAP beenden?

   Wählen Sie die richtigen Antworten.

   - ☐ A Nach einer bestimmten Anzahl von Wiederholungen
   - ☐ B Nach einer bestimmten Zeit
   - ☐ C Wenn eine bestimmte Bedingung erfüllt ist

9. Was ist der Arbeitsbereich einer internen Tabelle?

   Wählen Sie die richtige Antwort.

   - ☐ A Die erste Zeile der internen Tabelle
   - ☐ B Die aktuelle Zeile der internen Tabelle
   - ☐ C Eine Variable vom gleichen Typ wie der Zeilentyp der internen Tabelle

© Copyright. Alle Rechte vorbehalten.

10. Welche Informationen sehen Sie, wenn Sie den Mauszeiger auf einer Variable im Debugger positionieren?

    Wählen Sie die richtige Antwort.

    - ☐ A Den Datentyp der Variable
    - ☐ B Den aktuellen Inhalt der Variable
    - ☐ C Den aktuellen Speicherverbrauch der Variable

11. Wann wählen Sie im Debugger *Fortsetzen* (F8)?

    Wählen Sie die richtige Antwort.

    - ☐ A Zum Ausführen des Programms bis zum nächsten Breakpoint
    - ☐ B Ausführen eines einzelnen Schritts
    - ☐ C Debug-Sitzung beenden
    - ☐ D Ausführen des Programms bis zur aktuellen Cursorposition

© Copyright. Alle Rechte vorbehalten.

Kapitel 2

Überprüfung des Lernerfolgs - Antworten

1. Welche der folgenden Elemente können Sie verwenden, um einen Kommentar in ABAP zu kennzeichnen?

   Wählen Sie die richtigen Antworten.

   - [ ] A // am Anfang einer Zeile
   - [X] B * am Anfang einer Zeile
   - [ ] C -- an beliebiger Stelle in der Zeile
   - [X] D " an beliebiger Stelle in der Zeile

   Das ist richtig.

2. Welche der folgenden eingebauten ABAP-Typen sind vollständig?

   Wählen Sie die richtigen Antworten.

   - [ ] A C
   - [X] B D
   - [ ] C P
   - [X] D I

   Das ist richtig.

3. Eine Variable deklarieren Sie wie folgt: DATA var TYPE I VALUE 100. Anschließend verwenden Sie die Anweisung „CLEAR var“. Wie lautet der Wert von „var“ nach der CLEAR-Anweisung?

   Wählen Sie die richtige Antwort.

   - [X] A 0
   - [ ] B 100

   Ja, die CLEAR-Anweisung ignoriert den VALUE-Zusatz in der DATA-Anweisung und setzt die Variable auf den richtigen Initialwert für ihren Typ zurück.

© Copyright. Alle Rechte vorbehalten.

4. Das Ergebnis des Ausdrucks result = var MOD 2. ist 1. Was sagt Ihnen das über den Wert der Variablen var aus?

   Wählen Sie die richtige Antwort.

   - [ ] A „var“ ist eine gerade Zahl.
   - [X] B „var“ ist eine ungerade Zahl.

   Das ist richtig.

5. Sie möchten zwei Zeichenfolgen verketten. Welcher Operator ist der richtige?

   Wählen Sie die richtige Antwort.

   - [ ] A +
   - [ ] B &
   - [ ] C ++
   - [X] D &&

   Das ist richtig.

6. Auf die IF-Bedingung „IF a > 10.“ folgt die ELSEIF-Bedingung „ELSEIF a = 25.“ Die Variable a hat den Wert 25. Welche Codeverzweigungen werden ausgeführt?

   Wählen Sie die richtige Antwort.

   - [X] A Die durch „IF a > 10“ eingeführte Verzweigung.
   - [ ] B Die durch „ELSEIF a = 25“ eingeführte Verzweigung.
   - [ ] C Beide Verzweigungen.

   Das ist richtig.

7. Welche der folgenden Aussagen zu Ausnahmen sind zutreffend?

   Wählen Sie die richtigen Antworten.

   - [X] A Alle nicht behandelten Ausnahmen führen zu Laufzeitfehlern.
   - [ ] B Alle Ausführungen sind behandelbar.
   - [X] C Manche Ausnahmen sind behandelbar, andere nicht.
   - [ ] D Ein „TRY...ENDTRY“-Block muss mindestens zwei CATCH-Anweisungen enthalten.

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

8. Wie können Sie eine Schleife in ABAP beenden?

   Wählen Sie die richtigen Antworten.

   - [X] A Nach einer bestimmten Anzahl von Wiederholungen
   - [ ] B Nach einer bestimmten Zeit
   - [X] C Wenn eine bestimmte Bedingung erfüllt ist

   Das ist richtig.

9. Was ist der Arbeitsbereich einer internen Tabelle?

   Wählen Sie die richtige Antwort.

   - [ ] A Die erste Zeile der internen Tabelle
   - [ ] B Die aktuelle Zeile der internen Tabelle
   - [X] C Eine Variable vom gleichen Typ wie der Zeilentyp der internen Tabelle

   Das ist richtig.

10. Welche Informationen sehen Sie, wenn Sie den Mauszeiger auf einer Variable im Debugger positionieren?

    Wählen Sie die richtige Antwort.

    - [ ] A Den Datentyp der Variable
    - [X] B Den aktuellen Inhalt der Variable
    - [ ] C Den aktuellen Speicherverbrauch der Variable

    Das ist richtig.

11. Wann wählen Sie im Debugger *Fortsetzen* (F8)?

    Wählen Sie die richtige Antwort.

    - [X] A Zum Ausführen des Programms bis zum nächsten Breakpoint
    - [ ] B Ausführen eines einzelnen Schritts
    - [ ] C Debug-Sitzung beenden
    - [ ] D Ausführen des Programms bis zur aktuellen Cursorposition

    Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

KAPITEL 3 Mit lokalen Klassen arbeiten

Lektion 1

Eine lokale Klasse definieren 132
Übung 8 : Eine lokale Klasse definieren 137

Lektion 2

Instanzen für eine Klasse anlegen 141
Übung 9 : Instanzen anlegen und verwalten 147

Lektion 3

Methoden definieren und aufrufen 155
Übung 10 : Methoden definieren und aufrufen 165

Lektion 4

Kapselung zur Sicherstellung der Konsistenz verwenden 176
Übung 11 : Private Attribute und einen Konstruktor verwenden 183

LERNZIELE

- Definieren Sie eine lokale Klasse innerhalb einer globalen Klasse.
- Instanzen einer ABAP-Klasse anlegen.
- Methoden definieren und aufrufen.
- Erläutern Sie die Kapselung.
- Definieren und verwenden Sie Konstruktoren.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3
Lektion 1

Eine lokale Klasse definieren

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Definieren Sie eine lokale Klasse innerhalb einer globalen Klasse.

Lokale Klassen

Lokale und globale Klassen

Klassen in ABAP können entweder lokal oder global sein.

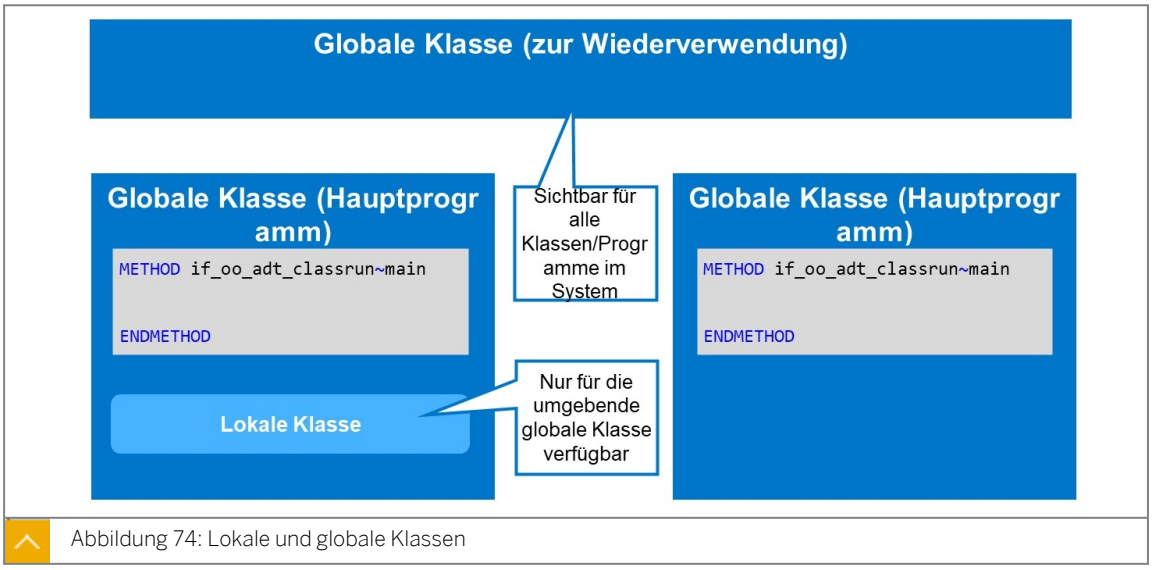


Abbildung 74: Lokale und globale Klassen

Globale Klassen werden zentral gespeichert sind in einem eigenen Repository-Objekt enthalten, dem so genannten Class-Pool. Eine globale Klasse kann als Hauptprogramm dienen. Globale Klassen können auch Logik enthalten, die von anderen ABAP-Programmen, einschließlich anderer globaler Klassen, wiederverwendet werden kann.

Lokale Klassen werden als Teil eines ABAP-Programms, z.B. einer globalen Klasse, definiert. Sie können sie nur in dem Programm oder der Klasse verwenden, in dem bzw. der sie definiert sind. Lokale Klassen eignen sich für Entitäten oder Funktionen, die Sie nur in einem einzigen Programm benötigen.

Die ABAP-Syntax lokaler und globaler Klassen ist fast identisch. In diesem Kurs arbeiten Sie mit lokalen Klassen in Ihrer globalen Klasse. Die globale Klasse mit der Methode „if_oo_adt_classrun~main“ dient nur als eine Art Hauptprogramm.

© Copyright. Alle Rechte vorbehalten.

Lokale Klassen in globalen Klassen

Wie in der nachstehenden Abbildung dargestellt, liegt der Fokus beim Öffnen einer globalen Klasse in ADT zuerst auf der Registerkarte „///Globale Klasse“. Hier finden Sie den Quelltext der globalen Klasse selbst. Um den Quelltext lokaler Klassen anzuzeigen oder einzugeben, müssen Sie zur Registerkarte „///Lokale Typen“ navigieren.

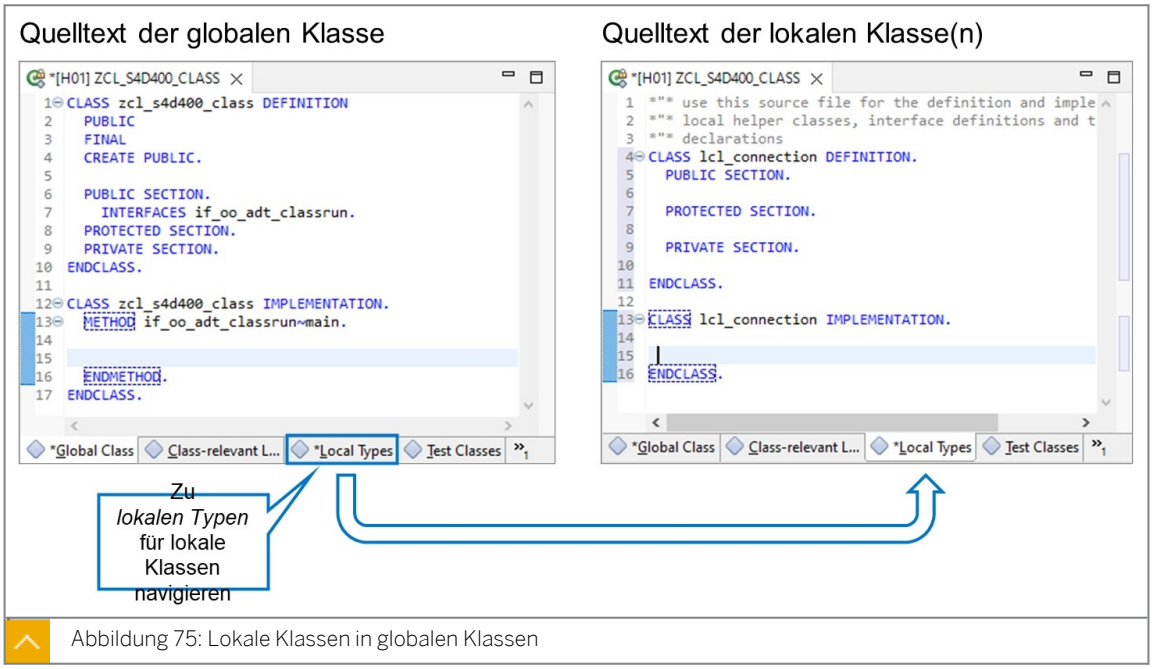


Abbildung 75: Lokale Klassen in globalen Klassen

ADT stellt eine Quelltextvorlage für die lokale Klasse bereit. Um diese Vorlage zu verwenden, gehen Sie wie folgt vor:

1. Geben Sie im ABAP-Editor **`lcl`** ein, und drücken Sie **`Strg + Leertaste`**.
2. Wählen Sie in der angezeigten Liste *///lcl - Lokale Klasse*, und drücken Sie die **`Eingabetaste`**.
3. Passen Sie den Namen der neuen lokalen Klasse an.

Für die Klassen in diesem Kurs müssen Sie den Zusatz „///create private“ aus der Anweisung CLASS ... DEFINITION entfernen.

© Copyright. Alle Rechte vorbehalten.

Attribute

Quelltext einer Klasse in ABAP

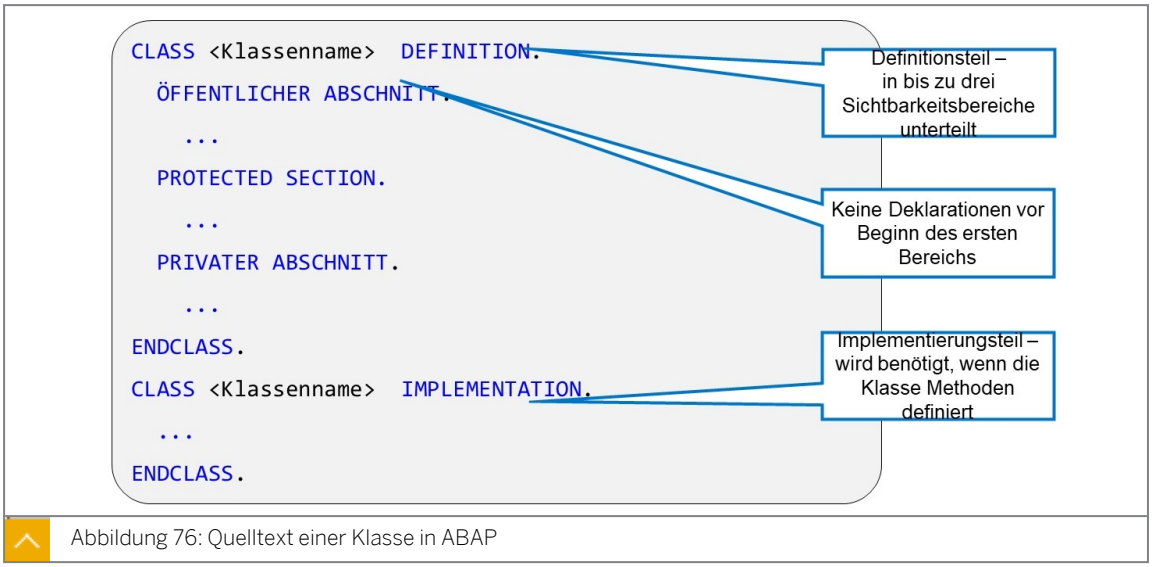


Abbildung 76: Quelltext einer Klasse in ABAP

In ABAP besteht der Quelltext einer Klasse aus zwei Teilen – der Definition und der Implementierung. Der Definitionsteil einer Klasse ist in bis zu drei Bereiche unterteilt, die Sichtbarkeitsbereiche der Klasse.

Definition

Der Definitionsteil einer Klasse enthält die Definition und die Deklaration aller Elemente in der Klasse, d.h. die Typen, die Konstanten, die Attribute und die Methoden. Er beginnt mit CLASS <class_name> DEFINITION. und endet mit ENDCLASS.

Implementierung

Der Implementierungsteil einer Klasse enthält den ausführbaren Code der Klasse, d.h. die Implementierung ihrer Methoden. Er beginnt mit CLASS <class_name> IMPLEMENTATION. und endet mit ENDCLASS. Der Implementierungsteil einer Klasse ist optional. Er wird obligatorisch, sobald die Klassendefinition ausführbare Methoden enthält.

Sichtbarkeitsbereiche

Jeder Sichtbarkeitsbereich einer Klasse beginnt mit einer der Anweisungen PUBLIC SECTION, PROTECTED SECTION, PRIVATE SECTION und endet implizit mit Beginn des nächsten Bereichs. Der letzte Bereich endet mit der Anweisung ENDCLASS. Alle Deklarationen einer Klasse müssen sich in einem der Bereiche befinden. Mit anderen Worten: Zwischen dem Beginn der Klassendefinition und dem Beginn des ersten Abschnitts sind keine Deklarationen erlaubt.

Der Bereich, in dem sich eine Deklaration befindet, definiert die Sichtbarkeit des deklarierten Elements der Klasse.

Die drei Sichtbarkeitsbereiche einer Klasse sind optional. Wenn Sie einen bestimmten Bereich nicht benötigen, müssen Sie ihn auch nicht deklarieren. Wenn eine Klassendefinition jedoch aus mehreren Bereichen besteht, müssen diese die Reihenfolge PUBLIC SECTION – PROTECTED SECTION – PRIVATE SECTION beachten.

© Copyright. Alle Rechte vorbehalten.

Definitionsteil einer Klasse

```
CLASS <Klassenname> DEFINITION.
 <beliebig> ABSCHNITT.

TYPES  <Typ> TYPE  <Typ>.
 DATA  <instance_attribute> TYPE  <type> [VALUE  <v> ].
 CLASS-DATA  <static_attribute> TYPE  <type> [VALUE  <v> ].
 CONSTANTS  <Konstante> TYPE  <Typ> VALUE  <v>.

 METHODS  <Instanzmethode> <Signatur>.
CLASS-METHODS  <static_method> <Signatur>.

ENDCLASS.
```

Abbildung 77: Definitionsteil einer Klasse

Die Abbildung zeigt die verschiedenen Arten von Komponenten, die Sie in einer Klasse definieren können. Sie können all diese Komponenten in einem beliebigen Sichtbarkeitsbereich der Klasse definieren.

TYPES

Über die Anweisung TYPES definieren Sie Typen in Ihrer Klasse. Sie tun dies genauso wie außerhalb einer Klasse. Wenn Typen öffentlich sind, kann ein Programm, das die Klasse verwendet, mit dem Typ (oder den Typen) eigene Variablen deklarieren.

DATA und CLASS-DATA

Mit den Anweisungen DATA und CLASS-DATA können Sie die Attribute der Klasse deklarieren. Über DATA wird ein Instanzattribut und über CLASS-DATA ein statisches Attribut deklariert.

CONSTANTS

Klassen können auch Konstanten enthalten, die mit CONSTANTS deklariert werden. Konstanten und Typen sind statische Komponenten der Klasse.

METHODS und CLASS-METHODS

Mit METHODS und CLASS-METHODS definieren Sie die Methoden einer Klasse. METHODS definiert eine **Instanzmethode**, CLASS-METHODS eine **statische Methode**. Auf den Namen der Methode folgt ihre Signatur, d.h. die Menge von Werten, die mit dem Aufrufer der Methode ausgetauscht werden, sowie die Ausnahmen, die für die Methode auftreten können.

© Copyright. Alle Rechte vorbehalten.

Beispiel: Attribute der Klasse Flugverbindung

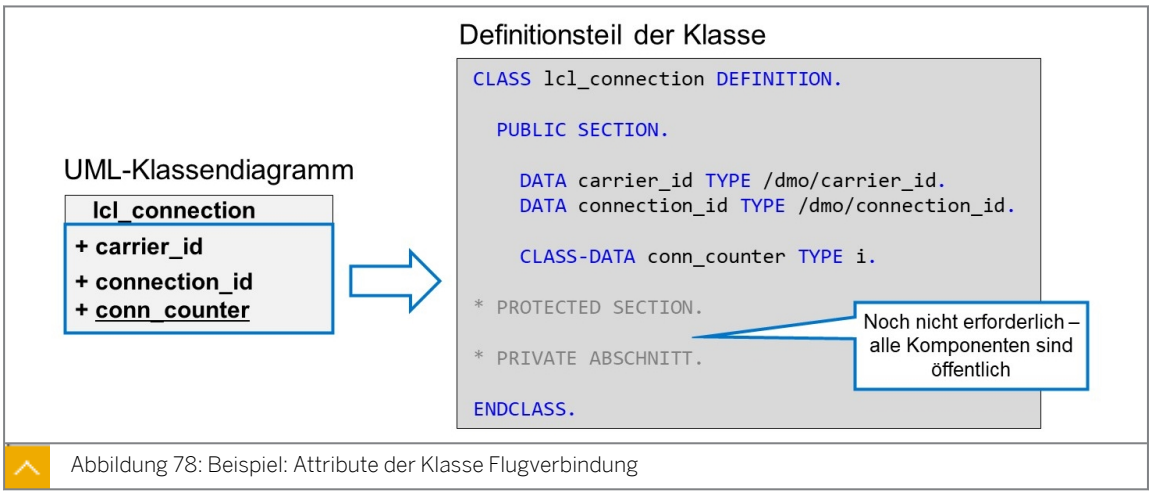


Abbildung 78: Beispiel: Attribute der Klasse Flugverbindung

Jetzt soll das UML-Modell in ABAP implementiert werden, und zwar als Erstes die Attribute der Verbindungsklasse. Sie deklarieren das Attribut dann mit der Anweisung DATA im entsprechenden Sichtbarkeitsbereich.

Das Attribut „conn_counter“ ist unterstrichen, d.h., es handelt sich um ein statisches Attribut. Statische Attribute deklarieren Sie mit der Anweisung CLASS-DATA. Die Syntax der Anweisung CLASS-DATA entspricht der der Anweisung DATA.

Notiz:
Wir beginnen hier mit öffentlichen Attributen. Wenn wir später im Kurs über Kapselung sprechen, wandeln wir sie in private Attribute um.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3
Übung 8

Eine lokale Klasse definieren

In dieser Übung definieren Sie eine lokale Klasse innerhalb einer globalen Klasse.

Vorlage:

keine

Lösung:

/LRN/CL_S4D400_CLS_LOCAL_CLASS (globale Klasse)

Aufgabe 1: Globale Klasse anlegen
Legen Sie in Ihrem eigenen Paket eine neue ABAP-Klasse an.

1. Legen Sie eine neue globale Klasse an, die das Interface *IF_OO_ADT_CLASSRUN* implementiert (Namensvorschlag: **`ZCL_##_LOCAL_CLASS`**, wobei ## für Ihre Gruppennummer steht).

Aufgabe 2: Eine lokale Klasse definieren
Definieren Sie eine lokale Klasse **`lcl_connection`**, und deklarieren Sie einige öffentliche Attribute.

1. Legen Sie innerhalb der globalen Klasse eine neue lokale Klasse **`lcl_connection`** an. Verwenden Sie die Quelltextvervollständigung, um den Code zu generieren. Stellen Sie jedoch sicher, dass Sie den Zusatz `create private` aus dem generierten Code entfernen.
2. Deklarieren Sie in der lokalen Klasse **`lcl_connection`** die folgenden **öffentlichen** Attribute:

   Tabelle 1: Attribute

Attributname	Umfang	Datentyp
`carrier_id`	Instanz	/DMO/CARRIER_ID
`connection_id`	Instanz	/DMO/CONNECTION_ID
`conn_counter`	statisch	I

3. Aktivieren Sie die Klasse.

Notiz:
Da die Methode **`if_oo_adt_classrun~main`** keinen ausführbaren Code enthält, gibt es an dieser Stelle noch nichts zu testen oder debuggen.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3
Lösung 8

Eine lokale Klasse definieren

In dieser Übung definieren Sie eine lokale Klasse innerhalb einer globalen Klasse.

Vorlage:

keine

Lösung:

/LRN/CL_S4D400_CLS_LOCAL_CLASS (globale Klasse)

Aufgabe 1: Globale Klasse anlegen
Legen Sie in Ihrem eigenen Paket eine neue ABAP-Klasse an.

1. Legen Sie eine neue globale Klasse an, die das Interface *IF_OO_ADT_CLASSRUN* implementiert (Namensvorschlag: **`ZCL_##_LOCAL_CLASS`**, wobei ## für Ihre Gruppennummer steht).
   a) Wählen Sie *File* → *New* → *ABAP Class*.
   b) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_LOCAL_CLASS`** ein, wobei **`##`** für Ihre Gruppennummer steht. Geben Sie eine Beschreibung ein.
   c) Wählen Sie im Gruppenrahmen *Interfaces* die Option *Add*.
   d) Geben Sie **`IF_OO_ADT_CLASSRUN`** ein. Wenn das Interface in der Trefferliste angezeigt wird, fügen Sie es mit Doppelklick zur Klassendefinition hinzu.
   e) Wählen Sie *Next*.
   f) Wählen Sie Ihren Transportauftrag aus, und wählen Sie *Finish*.

Aufgabe 2: Eine lokale Klasse definieren
Definieren Sie eine lokale Klasse **`lcl_connection`**, und deklarieren Sie einige öffentliche Attribute.

1. Legen Sie innerhalb der globalen Klasse eine neue lokale Klasse **`lcl_connection`** an. Verwenden Sie die Quelltextvervollständigung, um den Code zu generieren. Stellen Sie jedoch sicher, dass Sie den Zusatz `create private` aus dem generierten Code entfernen.
   a) Wechseln Sie zur Registerkarte *///Lokale Typen*.
   b) Geben Sie **`lcl`** in den Editor ein, und drücken Sie **`Strg + Leertaste`**.
   c) Doppelklicken Sie im Dialogfenster auf *lcl - class*.
   d) Solange noch ein Rahmen um **`lcl`** in der Zeile `class lcl definition create private.` angezeigt wird, vervollständigen Sie den Namen der Klasse zu **`lcl_connection`**.

© Copyright. Alle Rechte vorbehalten.

e) Löschen Sie die Wörter `create private` am Ende der Quelltextzeile `class lcl_connection definition.`.

2. Deklarieren Sie in der lokalen Klasse **`lcl_connection`** die folgenden **öffentlichen** Attribute:

Tabelle 1: Attribute

Attributname	Umfang	Datentyp
`carrier_id`	Instanz	/DMO/CARRIER_ID
`connection_id`	Instanz	/DMO/CONNECTION_ID
`conn_counter`	statisch	I

a) Fügen Sie nach der Zeile `PUBLIC SECTION.` und vor der Zeile `PROTECTED SECTION.` folgenden Code hinzu:

```
DATA carrier_id    TYPE /dmo/carrier_id.
DATA connection_id TYPE /dmo/connection_id.

CLASS-DATA conn_counter TYPE i.
```

3. Aktivieren Sie die Klasse.

> Notiz:
> Da die Methode **`if_oo_adt_classrun~main`** keinen ausführbaren Code enthält, gibt es an dieser Stelle noch nichts zu testen oder debuggen.

a) Drücken Sie **`Strg + F3`**, um die Klasse zu aktivieren.

b) Vergleichen Sie Ihren Quelltext auf der Registerkarte *///Local Types* mit dem folgenden Auszug aus der Musterlösung:

```
*"* use this source file for the definition and implementation of
*"* local helper classes, interface definitions and type
*"* declarations

CLASS lcl_connection DEFINITION.

  PUBLIC SECTION.

    DATA carrier_id    TYPE /dmo/carrier_id.
    DATA connection_id TYPE /dmo/connection_id.

    CLASS-DATA conn_counter TYPE i.

  PROTECTED SECTION.
  PRIVATE SECTION.

ENDCLASS.

CLASS lcl_connection IMPLEMENTATION.

ENDCLASS.
```

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Definieren Sie eine lokale Klasse innerhalb einer globalen Klasse.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3
Lektion 2

Instanzen für eine Klasse anlegen

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Instanzen einer ABAP-Klasse anlegen.

Anlegen von Instanzen

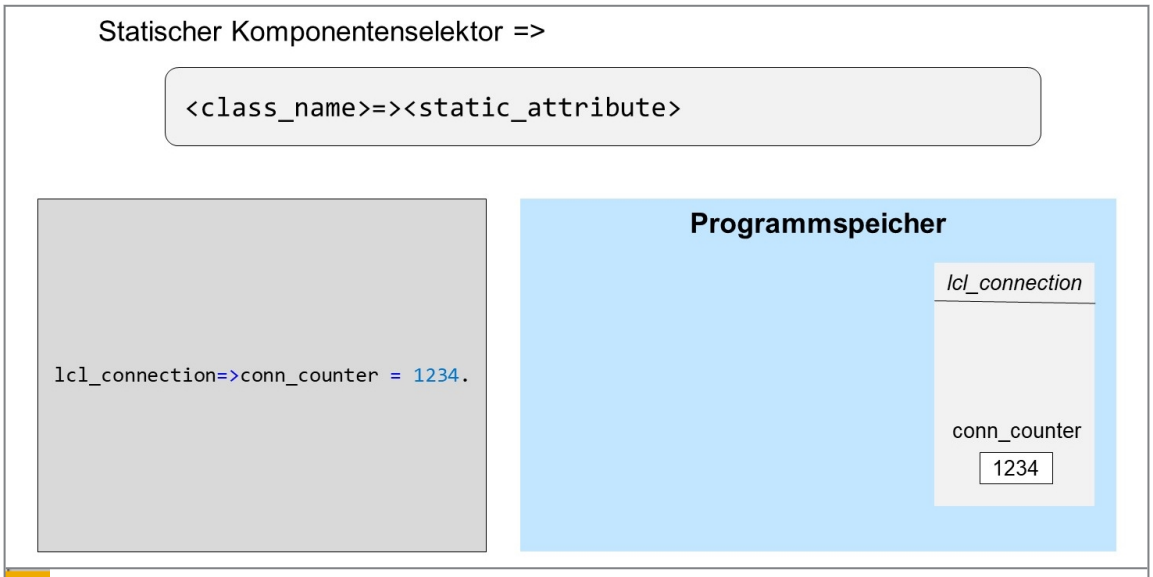


Abbildung 79: Zugriff auf statische Attribute

Sie arbeiten mit Attributen wie mit normalen Variablen desselben Typs. Außerhalb der Klasse reicht der Attributname jedoch nicht aus, um das Attribut eindeutig zu identifizieren. Um ein statisches Attribut außerhalb der Klasse zu adressieren, geben Sie zuerst den Klassennamen, dann den statischen Komponentenselektor (=>) und erst dann den Attributnamen ein. Der statische Komponentenselektor ist ein Doppelpfeil, der aus dem Gleichheits- und dem Größer-als-Zeichen besteht.

Hinweis:
Vor oder nach dem Komponentenselektor sind keine Leerzeichen erlaubt.

© Copyright. Alle Rechte vorbehalten.

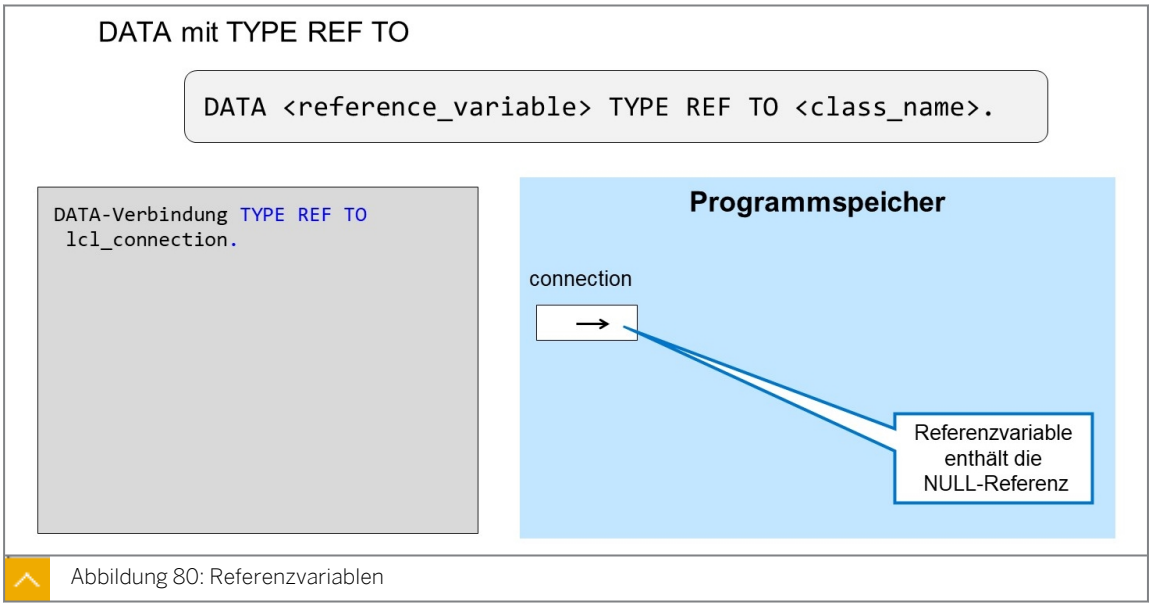


Abbildung 80: Referenzvariablen

Bei Instanzattributen ist die Situation noch komplizierter: Um auf eine Instanzkomponente zugreifen zu können, benötigen Sie eine Referenzvariable.

Eine Referenzvariable ist eine spezielle Variable, mit der Sie Objekte anlegen, adressieren und verwalten können. Eine Referenzvariable wird verwendet, um auf die Instanz einer Klasse im Programmspeicher zu zeigen. Sie deklarieren Referenzvariablen über die Anweisung DATA mit dem Zusatz TYPE REF TO gefolgt vom Namen einer Klasse.

Der Initialwert einer Referenzvariablen wird als NULL-Referenz bezeichnet, die Referenz zeigt nirgendwo hin.

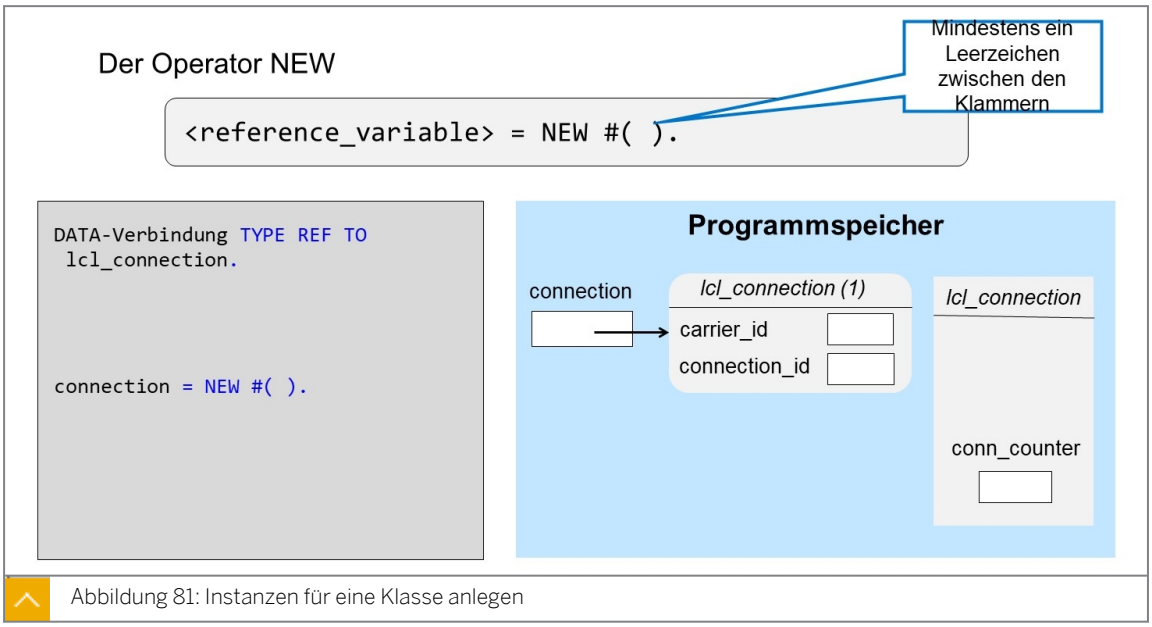


Abbildung 81: Instanzen für eine Klasse anlegen

Zum Anlegen einer neuen Instanz für eine Klasse verwenden Sie den Operator NEW. Im obigen Beispiel wird ein Ausdruck NEW #( ) auf der rechten Seite einer Wertzuweisung verwendet. Das Ergebnis des Ausdrucks ist die Speicheradresse der neu angelegten Instanz. Diese Referenz wird dann in der Referenzvariable auf der linken Seite der Zuweisung gespeichert.

© Copyright. Alle Rechte vorbehalten.

Möglicherweise haben Sie bemerkt, dass der Name der Klasse, die instanziiert werden soll, nicht im Ausdruck angezeigt wird. Aufgrund des Ortes des Ausdrucks NEW #( ) weiß das System jedoch bereits, dass die Zielvariable „connection“ vom Typ „REF TO lcl_connection“ ist und daher eine Instanz der Klasse „lcl_connection“ erzeugt werden muss. Das Rautenzeichen nach dem Operator NEW bedeutet: „Den Typ der Variable vor dem Gleichheitszeichen verwenden“. In komplexeren Szenarien können Sie hier den Namen der Klasse anstelle des Rautenzeichens angeben.

Hinweis:
Zwischen den Klammern muss mindestens ein Leerzeichen stehen.

Wenn Sie eine Klasse zum ersten Mal adressieren (dies könnte beim Zugriff auf eine statische Komponente oder beim Anlegen einer Instanz für die Klasse der Fall sein), lädt das Laufzeitsystem auch die Klassendefinition in den Programmspeicher. Diese Klassendefinition enthält alle statischen Attribute, die nur einmal in der Klasse vorhanden sind, anstatt einmal in jeder Instanz.

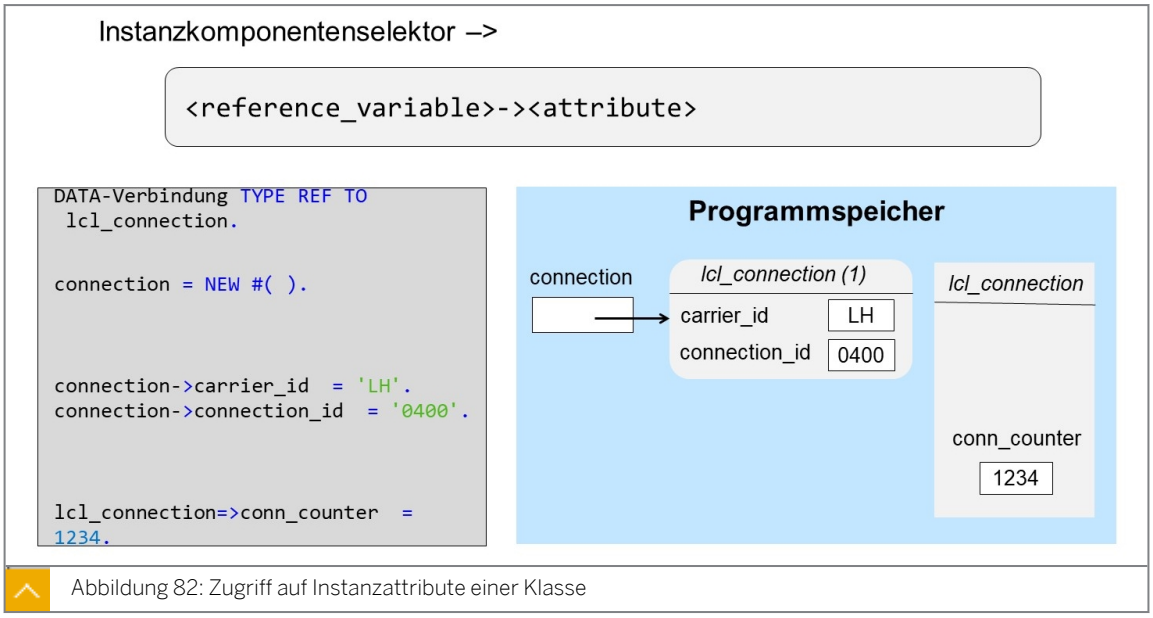


Abbildung 82: Zugriff auf Instanzattribute einer Klasse

Statische Komponenten werden über den Klassennamen und den statischen Komponentenselektor adressiert. Dies funktioniert nicht für Instanzkomponenten, da Sie die Instanz angeben müssen, auf die Sie zugreifen möchten.

Um ein Instanzattribut außerhalb der Klasse zu adressieren, geben Sie zuerst die Referenzvariable, dann den Selektor für die Instanzkomponente (->) und erst dann den Attributnamen ein. Der Selektor für die Instanzkomponente ist ein Pfeil, der aus dem Gleichheits- und dem Größer-als-Zeichen besteht.

Notiz:
Im Gegensatz zu vielen anderen Programmiersprachen verwendet ABAP unterschiedliche Zeichen für den Instanzkomponentenselektor und den statischen Komponentenselektor.

© Copyright. Alle Rechte vorbehalten.

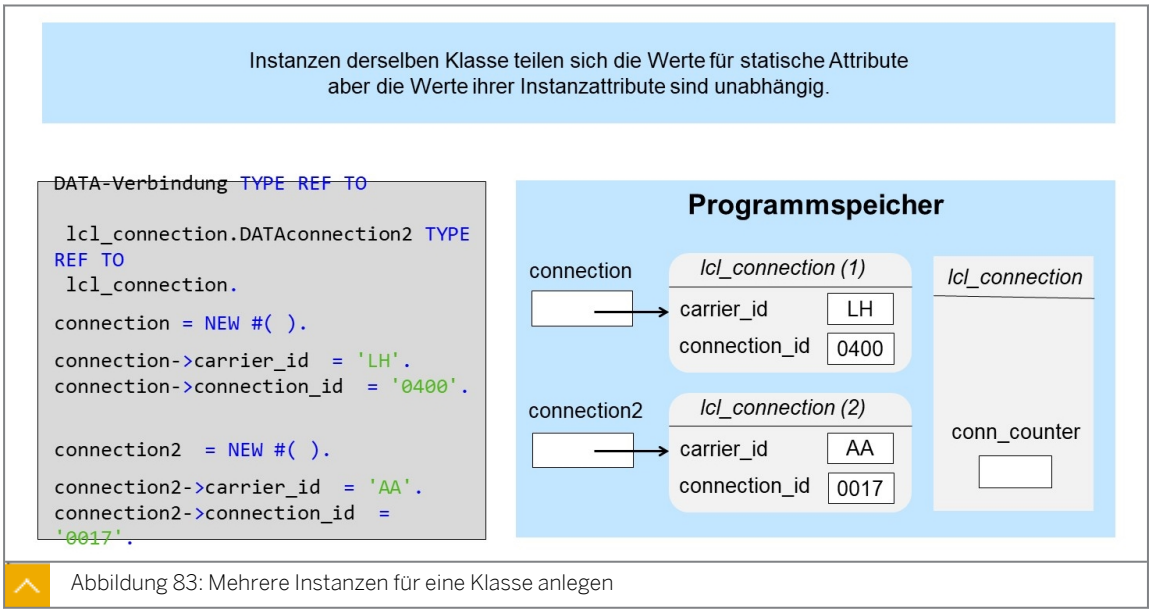


Abbildung 83: Mehrere Instanzen für eine Klasse anlegen

Eines der Hauptmerkmale der objektorientierten Programmierung ist die Tatsache, dass Sie mehrere Instanzen für dieselbe Klasse anlegen können. Jede Instanz wird an einer anderen Stelle im Programmspeicher erzeugt, und die Werte von Instanzattributen in einer Instanz sind unabhängig von den Werten in anderen Instanzen. Wie die Grafik veranschaulicht, teilen sich Instanzen derselben Klasse den Wert für die statischen Attribute.

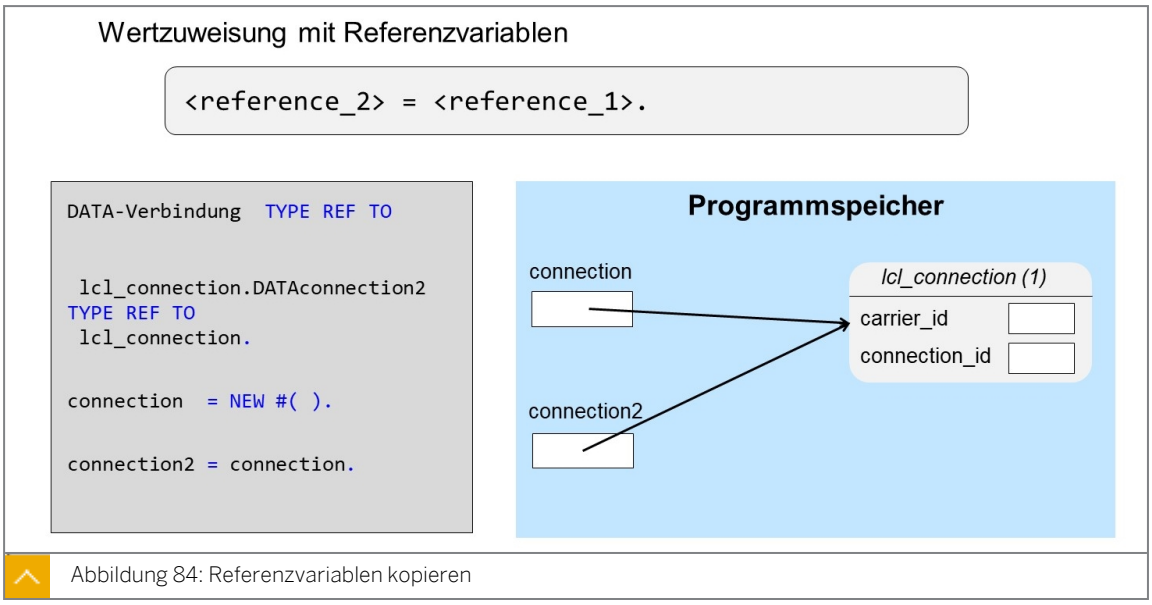


Abbildung 84: Referenzvariablen kopieren

Durch die Zuweisung einer Referenzvariablen zu einer anderen, kopiert das System die Adresse des Objekts, auf das die erste Variable zeigt, in die zweite Referenzvariable. Dadurch zeigen beide Referenzvariablen auf das gleiche Objekt.

© Copyright. Alle Rechte vorbehalten.

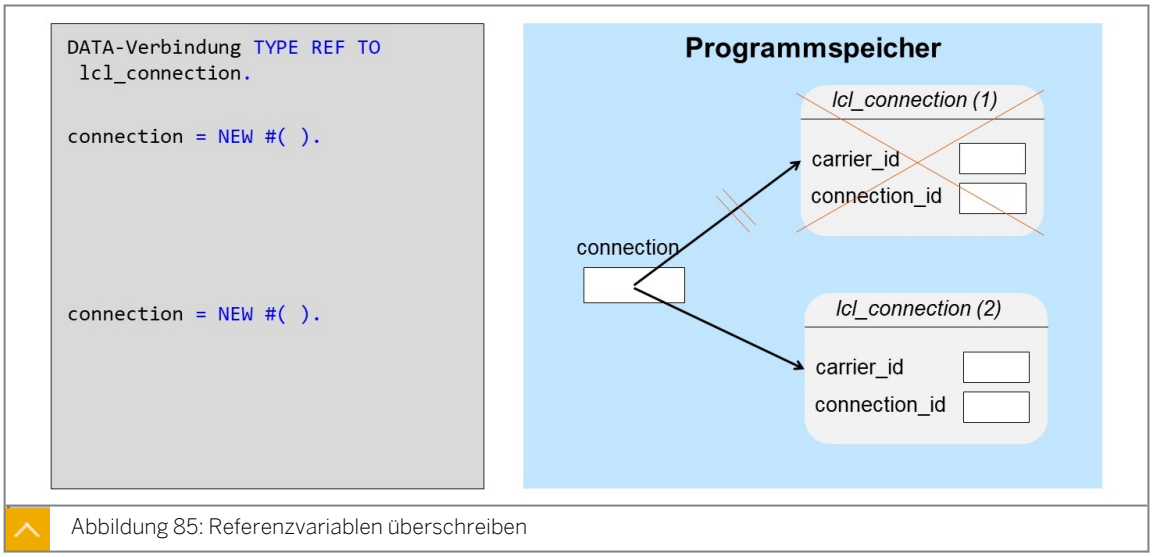


Abbildung 85: Referenzvariablen überschreiben

Sie können mit derselben Referenzvariable mehrere Instanzen für eine Klasse anlegen. Jedes Mal, wenn Sie den Ausdruck NEW #( ) verwenden, legt das System eine neue Instanz für die Klasse an und hinterlegt die Adresse der neuen Instanz in der Referenzvariablen. Die Adresse der neuen Instanz überschreibt jedoch die Adresse der vorherigen Instanz.

Im Beispiel oben überschreibt die Adresse von „lcl_connection (2)“ die Adresse von „lcl_connection (1)“. Folglich gibt es im Programm keine Referenzvariable mehr, die auf „lcl_connection (1)“ zeigt. Wenn dies für eine Instanz auftritt, kann sie nicht mehr aus dem Programm adressiert werden.

Um zu verhindern, dass der Programmspeicher mit Objekten gefüllt wird, die nicht mehr adressiert werden und zu einem Überlauf führen können, verfügt das Laufzeitsystem über die Komponente „Garbage Collector“. Der Garbage Collector ist ein Programm, das regelmäßig nach Objekten sucht, auf die keine Referenzen mehr zeigen, und diese dann löscht. Wenn Sie bei einem Programm die letzte Referenz auf ein Objekt löschen, indem Sie es überschreiben oder die Anweisung CLEAR verwenden, löscht der Garbage Collector das Objekt bei seinem nächsten Lauf.

Notiz:
Am Ende eines Programms, wenn alle Referenzvariablen freigegeben werden, löscht der Garbage Collector alle Instanzen, auf die diese zeigten. Sie selbst müssen sich keine Gedanken über das Ressourcenmanagement im Programm machen.

© Copyright. Alle Rechte vorbehalten.

Instanzverwaltung in einer internen Tabelle

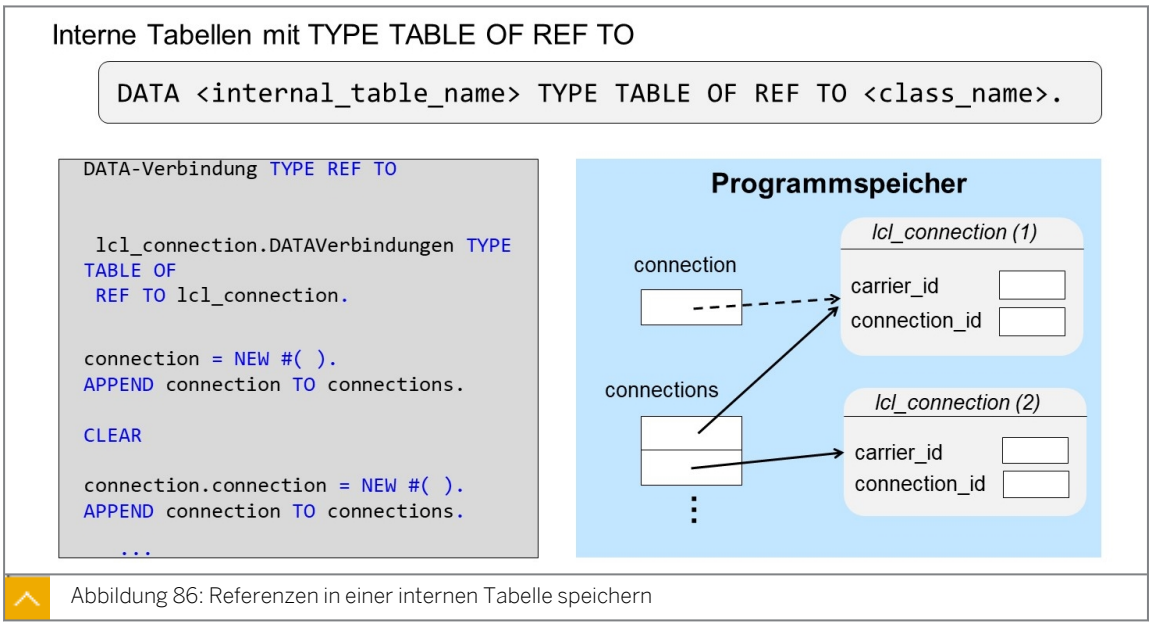


Abbildung 86: Referenzen in einer internen Tabelle speichern

Wenn Sie die Objekte behalten möchten, können Sie die Referenzen in einer internen Tabelle hinterlegen. Diese Technik eignet sich, wenn Sie eine ganze Reihe von Objekten anlegen. Sie ermöglicht die Verwendung einer einzelnen Referenzvariable, um viele Objekte anzulegen. Obwohl die Referenzvariable mit der Adresse des nächsten Objekts überschrieben wird, sind die vorhandenen Objekte sicher, da die interne Tabelle eine Referenz auf sie enthält. Löschen Sie deshalb nie die „letzte“ Referenz auf die Objekte.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3
Übung 9

Instanzen anlegen und verwalten

In dieser Übung legen Sie Instanzen Ihrer lokalen Klasse an und verwalten diese.

Vorlage:

/LRN/CL_S4D400_CLS_LOCAL_CLASS (globale Klasse)

Lösung:

/LRN/CL_S4D400_CLS_INSTANCES (globale Klasse)

Aufgabe 1: Vorlage kopieren (optional)
Kopieren Sie die Vorlageklasse. Wenn Sie die vorherige Übung abgeschlossen haben, können Sie diese Aufgabe überspringen und mit der Bearbeitung Ihrer Klasse **`ZCL_##_LOCAL_CLASS`** fortfahren.

1. Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_CLS_LOCAL_CLASS`** im ABAP Editor.
2. Verknüpfen Sie die View *Project Explorer* mit dem Editor.
3. Kopieren Sie die Klasse **`/LRN/CL_S4D400_CLS_LOCAL_CLASS`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_INSTANCES`**, wobei ## für Ihre Gruppennummer steht).

Aufgabe 2: Eine Instanz anlegen
Deklarieren Sie in der Methode **`main`** Ihrer globalen Klasse eine Referenzvariable, und legen Sie eine Instanz Ihrer lokalen Klasse an.

1. Deklarieren Sie in der Methode **`if_oo_adt_classrun~main`** Ihrer globalen Klasse eine Referenzvariable (Namensvorschlag: **`connection`**), und geben Sie sie mit Ihrer lokalen Klasse **`lcl_connection ein`**.
2. Erzeugen Sie eine Instanz der Klasse und setzen Sie die Attribute **`carrier_id`** und **`connection_id`** (Vorschlagswerte: *LH* für Attribut **`carrier_id`** und *0400* für Attribut **`connection_id`**).
3. Aktivieren Sie die Klasse, und analysieren Sie mit dem Debugger Schritt für Schritt, was passiert.

Aufgabe 3: Mehrere Instanzen verwalten
Legen Sie weitere Instanzen Ihrer lokalen Klasse an, und speichern Sie die Referenzen auf diese Instanzen in einer internen Tabelle.

1. Deklarieren Sie in der Methode **`if_oo_adt_classrun~main( )`** Ihrer globalen Klasse eine interne Tabelle mit dem Zeilentyp `REF TO lcl_connection` (Namensvorschlag: **`connections`**).

© Copyright. Alle Rechte vorbehalten.

2. Hängen Sie nach dem Instanziieren der Klasse und dem Setzen der Attribute die Objektreferenz an die interne Tabelle an.
3. Legen Sie zwei weitere Instanzen der Klasse **`lcl_connection`** an, setzen Sie deren Instanzattribute auf andere Werte als in der ersten Instanz, und hängen Sie die Referenzen auf die neuen Instanzen an die interne Tabelle **`connections`** an. Verwenden Sie für alle drei Instanzen dieselbe Referenzvariable **`connection`**.
4. Aktivieren Sie die Klasse, und analysieren Sie mit dem Debugger Schritt für Schritt, was passiert.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3
Lösung 9

Instanzen anlegen und verwalten

In dieser Übung legen Sie Instanzen Ihrer lokalen Klasse an und verwalten diese.

Vorlage:

/LRN/CL_S4D400_CLS_LOCAL_CLASS (globale Klasse)

Lösung:

/LRN/CL_S4D400_CLS_INSTANCES (globale Klasse)

Aufgabe 1: Vorlage kopieren (optional)
Kopieren Sie die Vorlageklasse. Wenn Sie die vorherige Übung abgeschlossen haben, können Sie diese Aufgabe überspringen und mit der Bearbeitung Ihrer Klasse **`ZCL_##_LOCAL_CLASS`** fortfahren.

1. Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_CLS_LOCAL_CLASS`** im ABAP Editor.
   a) Wählen Sie in der *Eclipse*-Symbolleiste *Open ABAP Development Object*. Drücken Sie alternativ **`Strg + Umschalt + A`**.
   b) Geben Sie als Suchzeichenfolge **`/LRN/CL_S4D400_CLS`** ein.
   c) Wählen Sie aus der Liste der Entwicklungsobjekte */LRN/CL_S4D400_CLS_LOCAL_CLASS* aus, und wählen Sie dann *OK*.
2. Verknüpfen Sie die View *Project Explorer* mit dem Editor.
   a) Suchen Sie in der Symbolleiste *Project Explorer* die Drucktaste *Link with Editor*. Wählen Sie die Drucktaste, falls diese noch nicht markiert ist. Dadurch sollte das Entwicklungsobjekt, das im Editor geöffnet ist, im *Project Explorer* hervorgehoben sein.
3. Kopieren Sie die Klasse **`/LRN/CL_S4D400_CLS_LOCAL_CLASS`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_INSTANCES`**, wobei ## für Ihre Gruppennummer steht).
   a) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Klasse **`/LRN/CL_S4D400_CLS_LOCAL_CLASS`**, um das Kontextmenü zu öffnen.
   b) Wählen Sie im Kontextmenü *Duplicate ...*
   c) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_INSTANCES`** ein, wobei ## für Ihre Gruppennummer steht.
   d) Passen Sie die Beschreibung an, und wählen Sie *Next*.
   e) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

Aufgabe 2: Eine Instanz anlegen
Deklarieren Sie in der Methode **`main`** Ihrer globalen Klasse eine Referenzvariable, und legen Sie eine Instanz Ihrer lokalen Klasse an.

© Copyright. Alle Rechte vorbehalten.

1. Deklarieren Sie in der Methode **`if_oo_adt_classrun~main`** Ihrer globalen Klasse eine Referenzvariable (Namensvorschlag: **`connection`**), und geben Sie sie mit Ihrer lokalen Klasse **`lcl_connection ein`**.
   a) Wechseln Sie zur Registerkarte *///Globale Klasse*.
   b) Passen Sie den Quelltext wie folgt an:

```
METHOD if_oo_adt_classrun~main.

  DATA connection TYPE REF TO lcl_connection.

ENDMETHOD.
```

2. Erzeugen Sie eine Instanz der Klasse und setzen Sie die Attribute **`carrier_id`** und **`connection_id`** (Vorschlagswerte: *LH* für Attribut **`carrier_id`** und *0400* für Attribut **`connection_id`**).
   a) Passen Sie den Quelltext wie folgt an:

```
DATA connection TYPE REF TO lcl_connection.

connection = new #( ).

connection->carrier_id    = 'LH'.
connection->connection_id = '0400'.
```

Hinweis:
Nachdem Sie den Komponentenselektor (->) typisiert haben, drücken Sie **`Strg + Leertaste`**, um die Attributnamen aus einer Vorschlagsliste auszuwählen.

3. Aktivieren Sie die Klasse, und analysieren Sie mit dem Debugger Schritt für Schritt, was passiert.
   a) Drücken Sie **`Strg + F3`**, um die Klasse zu aktivieren.
   b) Doppelklicken Sie auf den linken Rand des Editors neben der Zeile `connection = new #( ).`, um einen Breakpoint zu setzen.
   c) Drücken Sie **`F9`**, um die Klasse auszuführen.
   d) Doppelklicken Sie auf das Wort **`connection`**, um den Inhalt der Referenzvariablen **`connection`** anzuzeigen.
   e) Drücken Sie **`F5`**, um im Debugger einen Schritt weiter zu gehen. Prüfen Sie, ob sich der Wert von **`connection`** geändert hat.
   f) Expandieren Sie in der View *Variables* die Verzweigung **`connection`**, um die Initialattribute der Klasse anzuzeigen.
   g) Drücken Sie **`F5`**, um im Debugger einen Schritt weiter zu gehen. Prüfen Sie, ob sich der Wert von **`carrier_id`** geändert hat.
   h) Drücken Sie erneut *F5*. Prüfen Sie, ob sich der Wert von **`Connection_ID`** geändert hat.

© Copyright. Alle Rechte vorbehalten.

Aufgabe 3: Mehrere Instanzen verwalten

Legen Sie weitere Instanzen Ihrer lokalen Klasse an, und speichern Sie die Referenzen auf diese Instanzen in einer internen Tabelle.

1. Deklarieren Sie in der Methode **`if_oo_adt_classrun~main( )`** Ihrer globalen Klasse eine interne Tabelle mit dem Zeilentyp `REF TO lcl_connection` (Namensvorschlag: **`connections`**).

   a) Passen Sie den Quelltext wie folgt an:

```
DATA connection TYPE REF TO lcl_connection.
DATA connections TYPE TABLE OF REF TO lcl_connection.

connection = new #( ).

connection->carrier_id    = 'LH'.
connection->connection_id = '0400'.
```

2. Hängen Sie nach dem Instanziieren der Klasse und dem Setzen der Attribute die Objektreferenz an die interne Tabelle an.

   a) Passen Sie den Quelltext wie folgt an:

```
connection = NEW #( ).

connection->carrier_id = 'LH'.
connection->connection_id = '0400'.

APPEND connection TO connections.
```

3. Legen Sie zwei weitere Instanzen der Klasse **`lcl_connection`** an, setzen Sie deren Instanzattribute auf andere Werte als in der ersten Instanz, und hängen Sie die Referenzen auf die neuen Instanzen an die interne Tabelle **`connections`** an. Verwenden Sie für alle drei Instanzen dieselbe Referenzvariable **`connection`**.

   a) Passen Sie den Quelltext wie folgt an:

```
connection = NEW #(  ).

connection->carrier_id    = 'LH'.
connection->connection_id = '0400'.

APPEND connection TO connections.

connection = NEW #(  ).

connection->carrier_id    = 'AA'.
connection->connection_id = '0017'.

APPEND connection TO connections.

connection = NEW #(  ).

connection->carrier_id    = 'SQ'.
connection->connection_id = '0001'.

APPEND connection TO connections.
```

© Copyright. Alle Rechte vorbehalten.

4. Aktivieren Sie die Klasse, und analysieren Sie mit dem Debugger Schritt für Schritt, was passiert.
   a) Drücken Sie **Strg + F3**, um die Klasse zu aktivieren.
   b) Doppelklicken Sie auf den linken Rand des Editors neben der ersten Zeile `connection = new #( ).`, um den Breakpoint zu entfernen.
   c) Doppelklicken Sie auf den linken Rand des Editors neben der ersten Zeile `APPEND connection TO connections.`, um einen neuen Breakpoint zu setzen.
   d) Drücken Sie **F9**, um die Klasse auszuführen.
   e) Doppelklicken Sie auf das Wort **connection**, um den Inhalt der Referenzvariablen **connection** anzuzeigen.
   f) Doppelklicken Sie auf das Wort **connections**, um den Inhalt der internen Tabelle **connections** anzuzeigen.
   g) Drücken Sie **F5**, um im Debugger einen Schritt weiter zu gehen. Prüfen Sie, ob sich der Inhalt von **connections** geändert hat.
   h) Expandieren Sie in der View *Variables* die Verzweigung **connections**, um den Inhalt der internen Tabelle anzuzeigen.
   i) Drücken Sie mehrmals **F5**, bis Sie das Ende des Programms erreicht haben. Überprüfen Sie dabei den Inhalt der internen Tabelle.
   j) Vergleichen Sie Ihren Quelltext auf der Registerkarte *///Global Class* mit dem folgenden Auszug aus der Musterlösung:

```
METHOD if_oo_adt_classrun~main.

    DATA connection TYPE REF TO lcl_connection.
    DATA connections TYPE TABLE OF REF TO lcl_connection.

* First Instance
**********************************************************************

    connection = NEW #(  ).

    connection->carrier_id    = 'LH'.
    connection->connection_id = '0400'.

    APPEND connection TO connections.

* Second Instance
**********************************************************************
    connection = NEW #(  ).

    connection->carrier_id    = 'AA'.
    connection->connection_id = '0017'.

    APPEND connection TO connections.

* Third Instance
**********************************************************************
    connection = NEW #(  ).
```


© Copyright. Alle Rechte vorbehalten.

```
    connection->carrier_id    = 'SQ'.
    connection->connection_id = '0001'.

    APPEND connection TO connections.

  ENDMETHOD.
```

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Instanzen einer ABAP-Klasse anlegen.

© Copyright. Alle Rechte vorbehalten.

Methoden definieren und aufrufen

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Methoden definieren und aufrufen.

Methodendefinition

```
CLASS <Klassenname> DEFINITION.
   <beliebig> SECTION.

 METHODS <method_name>
         IMPORTING <Eingabe_1> TYPE <Typ>
 <Eingabe_2> TYPE <Typ>  DEFAULT  <Wert>
 ...
         EXPORTING <Ausgabe_1> TYPE <Typ>
 ...
         CHANGING <inout_1> TYPE <Typ>  OPTIONAL ...
          RETURNING VALUE(<Ergebnis>) TYPE <Typ>
 RAISING  <Ausnahme1>
 <Ausnahme2>
 ... .
ENDCLASS.
```

Abbildung 87: Methodendefinition

Im Definitionsteil einer Klasse definieren Sie mit METHODS eine Instanzmethode und mit CLASS-METHODS eine statische Methode. Auf den Namen der Methode folgt ihre Signatur, d.h. die Menge von Werten, die mit dem Aufrufer der Methode ausgetauscht werden, sowie die Ausnahmen, die für die Methode auftreten können.

Die Signatur einer Methode besteht aus Parametern und Ausnahmen. Jeder Parameter hat einen Namen und einen Typ.

ABAP kennt folgende Arten von Parametern:

Importing-Parameter

Importing-Parameter sind Werte, die die Methode vom Aufrufer erhält. Eine Methode kann eine beliebige Anzahl von Importing-Parametern haben.

Standardmäßig sind Importing-Parameter obligatorisch. Es gibt aber zwei Möglichkeiten, sie als optional zu definieren:

© Copyright. Alle Rechte vorbehalten.

- Mit dem Zusatz OPTIONAL. Der Parameter ist optional und sein Standardwert ist der entsprechende Initialwert für den Typen des Parameters.
- Mit dem Zusatz DEFAULT <val>. Der Parameter ist optional, und Sie geben seinen Defaultwert über <val> an.

Innerhalb einer Methode dürfen Sie Importing-Parameter nicht ändern. Wenn Sie dies versuchen, verursachen Sie einen Syntaxfehler.

Exporting-Parameter

Exporting-Parameter sind Ergebnisse, die von der Methode zurückgegeben werden. Eine Methode kann eine beliebige Anzahl von Exporting-Parametern haben. Alle Exporting-Parameter sind optional – ein aufrufendes Programm verwendet nur die Werte, die es wirklich benötigt.

Changing-Parameter

Changing-Parameter sind Werte, die die Methode vom Aufrufer erhält. Im Gegensatz zu Importing-Parametern kann die Methode die Werte dieser Parameter ändern. Sie werden dann mit demselben Namen an den Aufrufer zurückgegeben. Eine Methode kann eine beliebige Anzahl von Changing-Parametern haben. Changing-Parameter sind standardmäßig obligatorisch. Sie können sie auf dieselbe Weise wie Importing-Parameter als optional definieren.

Returning-Parameter

Ein Returning-Parameter ist ein Methodenergebnis, das direkt in einem Ausdruck verwendet werden kann. Eine Methode kann nur einen Returning-Parameter haben. Returning-Parameter müssen eine spezielle Form der Parameterübergabe verwenden, die als Wertübergabe bezeichnet wird. Diese Form der Parameterübergabe wird durch das Umschließen des Parameternamens mit Klammern (keine Leerzeichen!) definiert. Außerdem wird das Schlüsselwort VALUE vorangestellt.

Mit dem Schlüsselwort RAISING werden die Ausnahmen aufgelistet, die die Methode auslösen kann, um auf eine Fehlersituation hinzuweisen. Das aufrufende Programm kann dann auf den Fehler reagieren.

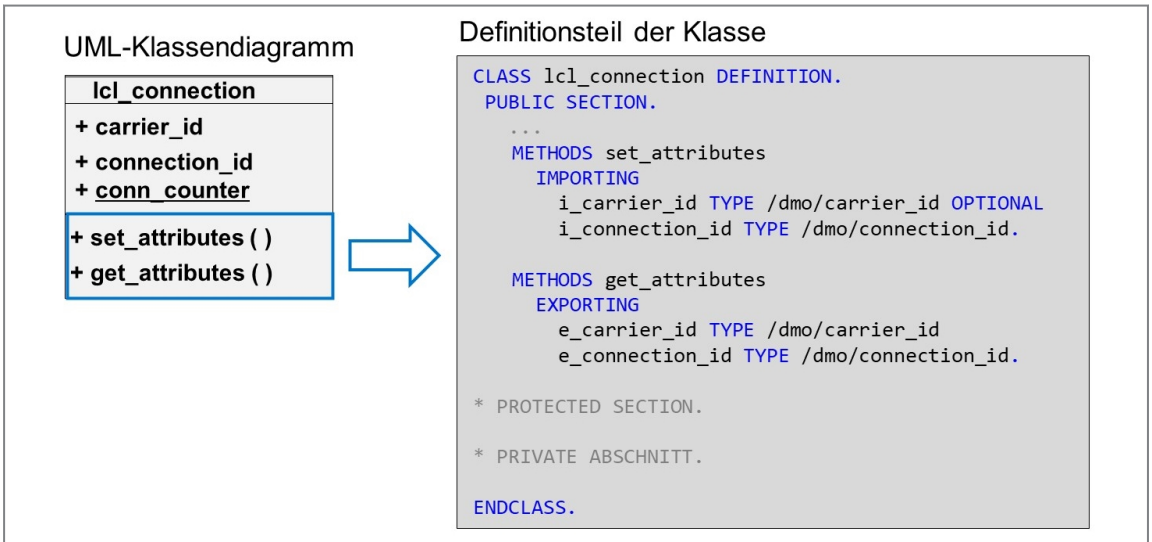


Abbildung 88: Beispiel: Methoden der Klasse Flugverbindung

Fügen Sie als Beispiel unserer Klasse „lcl_connection“ zwei Methoden hinzu.

© Copyright. Alle Rechte vorbehalten.

Eine `set_attributes( )` Methode zum Festlegen der Werte für die Attribute `carrier_id` und `connection_id` und eine `get_attributes( )` Methode, um die Werte dieser Attribute zurückzugeben. Wenn `set_attributes( )` zwei Importparameter benötigt, einen für jedes Attribut, benötigt die `get_attributes( )` Methode zwei Exportparameter.

Für jede Methode, die Sie definieren, müssen Sie auch eine Implementierung im Implementierungsteil der Klasse anlegen. Bis Sie diesen Vorgang durchgeführt haben, werden Syntaxfehler in Ihrer Klasse angezeigt. Die Syntaxprüfung meldet, dass die Implementierung der Methode fehlt.

Hinweis:
ADT bietet eine Schnellkorrektur zum Hinzufügen der fehlenden Implementierung. Um diese Schnellkorrektur zu verwenden, gehen Sie wie folgt vor:

1. Positionieren Sie den Cursor in einer METHODS-Anweisung mit einem Fehler, und drücken Sie *Strg + 1*.
2. Wählen Sie aus der Liste der möglichen Quickfixes die Option *Implementierung für ... hinzufügen*, und drücken *Sie die Eingabetaste*. Wenn die Implementierung für mehrere Methoden fehlt, lautet der Quickfix-Titel *Nicht implementierte Methoden hinzufügen*.

Methodenimplementierung

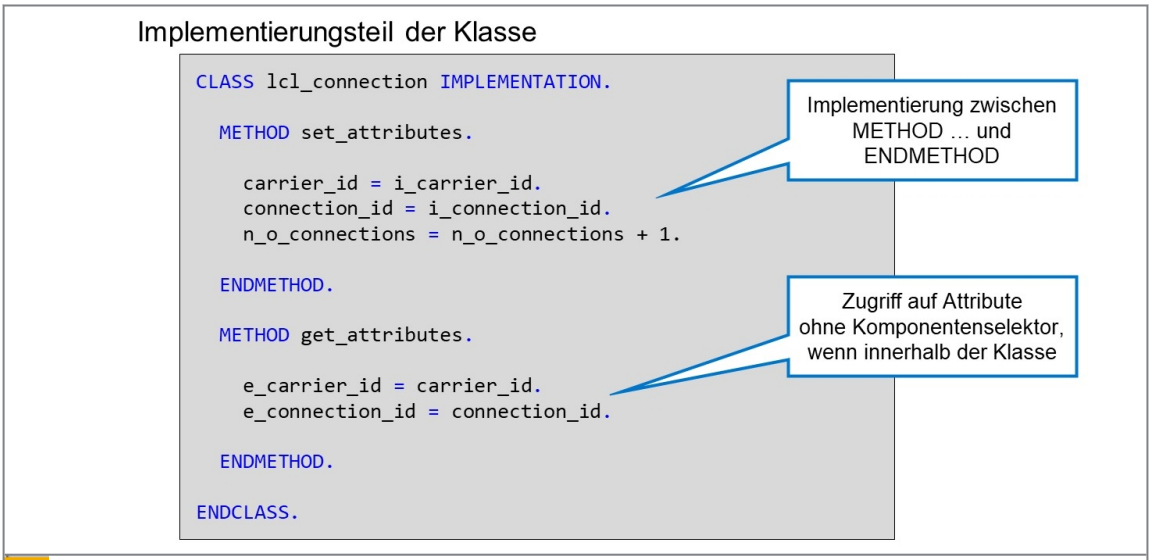


Abbildung 89: Methoden implementieren

Sie müssen jede definierte Methode implementieren. Dies geschieht im Implementierungsteil der Klasse, indem Sie das Anweisungspaar METHOD <method_name> und ENDMETHOD hinzufügen.

Die Methodenimplementierung enthält ABAP-Anweisungen, die auf die Parameter der Methode (es ist nicht zulässig, Importing-Parameter zu ändern) und alle Typen, Attribute sowie Konstanten zugreifen, die Sie unabhängig von ihrer Sichtbarkeit in der Klasse deklariert haben. Instanzmethoden können sowohl auf Instanz- als auch auf statische Attribute zugreifen. Statische Methoden können nur auf statische Komponenten zugreifen.

© Copyright. Alle Rechte vorbehalten.

Innerhalb des Implementierungsteils einer Klasse können Sie auf die Attribute dieser Klasse ohne Referenzvariable und '->' (oder bei statischen Attributen auf den Klassennamen und '=>') zugreifen.

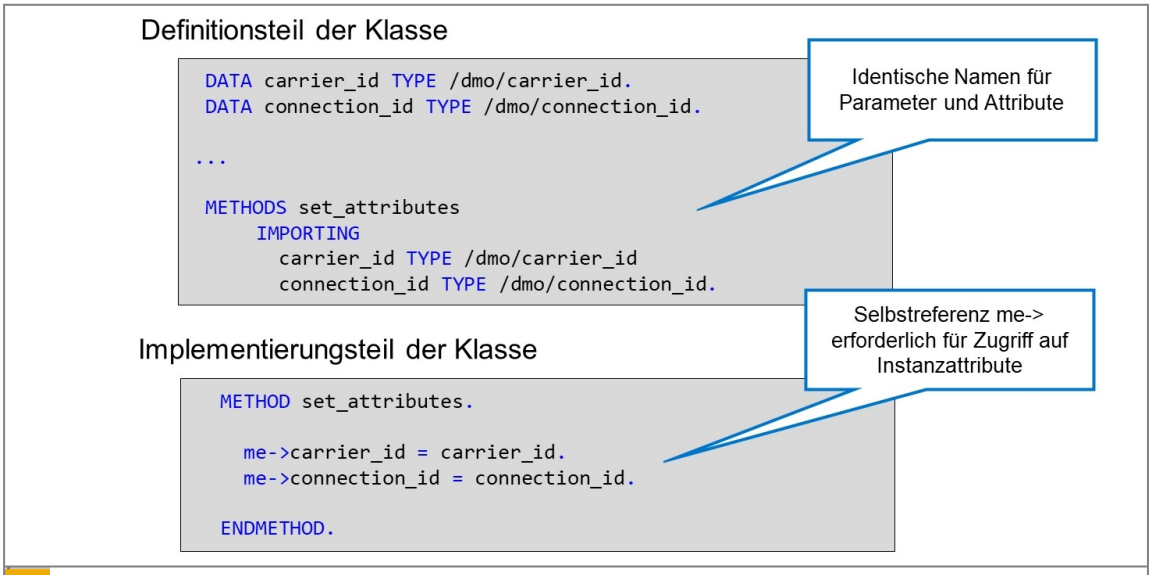


Abbildung 90: Die Notwendigkeit einer Selbstreferenz ME->

Nur bei der Implementierung von Instanzmethoden bietet ABAP die eingebaute Variable ME. ME ist eine Referenzvariable, die mit der aktuellen Klasse typisiert und zur Laufzeit mit der Adresse der aktuellen Instanz gefüllt wird. Die Verwendung von ME ist optional und sollte vermieden werden, es sei denn, der Name eines Attributs kollidiert mit dem Namen eines anderen Datenobjekts, z.B. mit einem Parameternamen.

Im Beispiel haben die Importing-Parameter der Methode „set_attributes( )“ den gleichen Namen wie die entsprechenden Attribute. In einer solchen Situation bezeichnet der Name allein den Parameter. Erst durch das Voranstellen von „me->“ wird deutlich, dass auf ein Attribut der aktuellen Instanz zugegriffen werden soll.

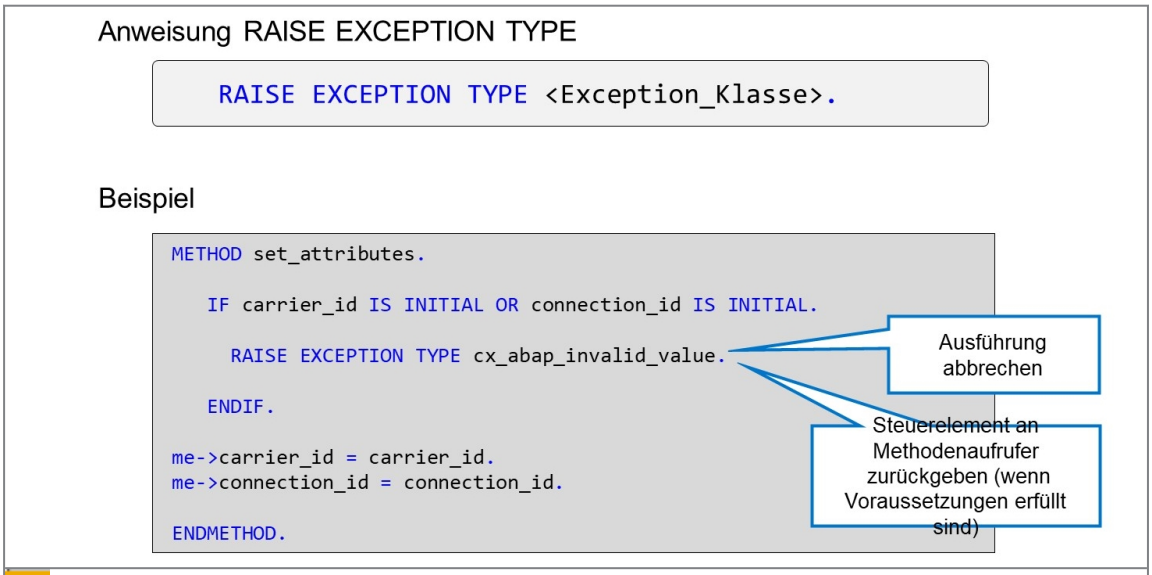


Abbildung 91: Ausnahmen auslösen

© Copyright. Alle Rechte vorbehalten.

Wenn bei der Ausführung einer Methode ein Fehler auftritt, kann die Methode mit der Anweisung RAISE EXCEPTION TYPE eine Ausnahme auslösen, gefolgt vom Namen der Ausnahme.

Notiz:
Technisch gesehen sind Ausnahmenamen die Namen spezieller ABAP-Klassen, und die Anweisung RAISE EXCEPTION TYPE erzeugt eine Instanz der referenzierten Klasse. Die Instanz wird als Ausnahmeobjekt und die spezielle Art von ABAP-Klasse als Ausnahmeklasse bezeichnet.

Sobald eine Ausnahme ausgelöst wird, wird die Methodenausführung abgebrochen.

Die Kontrolle wird an das aufrufende Programm zurückgegeben, wenn alle Voraussetzungen erfüllt sind. Andernfalls bricht das gesamte Programm mit einem Laufzeitfehler ab.

Die Voraussetzungen zum Fortsetzen des Programms lauten wie folgt:

- Die ausgelöste Ausnahme wird in der RAISING-Klausel der Methodensignatur deklariert.
- Der Methodenaufruf ist von einer TRY ... ENDTRY-Kontrollstruktur umgeben.
- Die TRY ... ENDTRY-Kontrollstruktur enthält einen CATCH-Block für die Ausnahme

Notiz:
Für einige Ausnahmen zeigt der ABAP Editor eine Syntaxwarnung mit Quickfix an, wenn Sie eine Ausnahme auslösen, die noch nicht in der RAISING-Klausel der Methode deklariert ist.

Methodenaufrufe

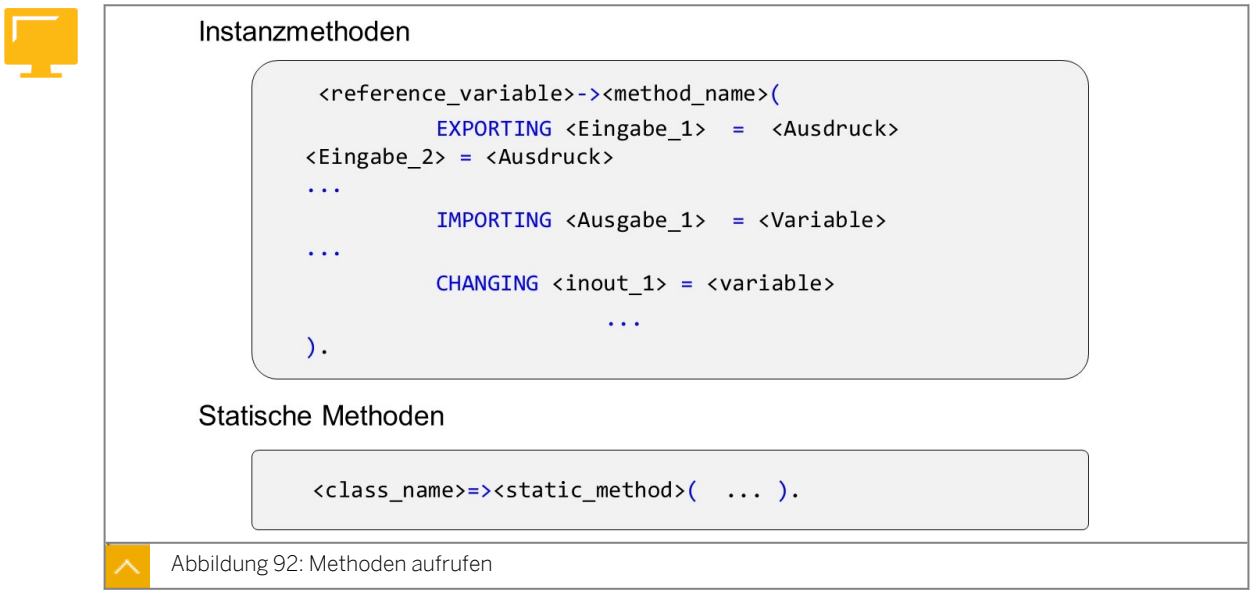


Abbildung 92: Methoden aufrufen

Der Aufruf einer Instanzmethode erfolgt über eine Referenzvariable und den Selektor für die Instanzkomponenten (->). Nach dem Komponentenselektor folgt der Name der aufzurufenden Methode. Für statische Methoden verwenden Sie den Klassennamen und den statischen Komponentenselektor (=>). In beiden Fällen erfolgt die Parameterübergabe in

© Copyright. Alle Rechte vorbehalten.

einem Klammerpaar. Die Klammern werden auf jeden Fall benötigt. Sie bleiben leer, wenn keine Parameter übergeben werden sollen.

> Notiz:
> Zwischen dem Methodennamen und der öffnenden Klammer ist kein Leerzeichen erlaubt. Andererseits ist mindestens ein Leerzeichen nach der öffnenden Klammer und vor der schließenden Klammer erforderlich.

Importing-Parameter werden nach dem Schlüsselwort EXPORTING aufgeführt, da die Werte, die von der aufgerufenen Methode importiert werden, vom aufrufenden Programm exportiert werden. Die Parameternamen werden auf der linken Seite eines Gleichheitszeichens (=) aufgeführt, auf der rechten Seite wird ein Ausdruck aufgeführt. Dieser Ausdruck kann ein einfaches Literal, eine Konstante oder eine Variable sein, aber auch alle anderen Ausdrücke sind erlaubt, solange der Typ des Ausdrucks mit dem Typ des Parameters übereinstimmt.

Exporting-Parameter werden nach dem Schlüsselwort IMPORTING aufgeführt. Beachten Sie, dass für den Exporting-Parameter der Parametername ebenfalls auf der linken Seite des Gleichheitszeichens aufgeführt wird. Die Variable auf der rechten Seite muss denselben Typ wie der Parameter aufweisen.

Changing-Parameter werden nach dem Schlüsselwort CHANGING aufgeführt. Der Typ der Variablen auf der rechten Seite muss mit dem Typ des Parameters auf der linken Seite übereinstimmen.

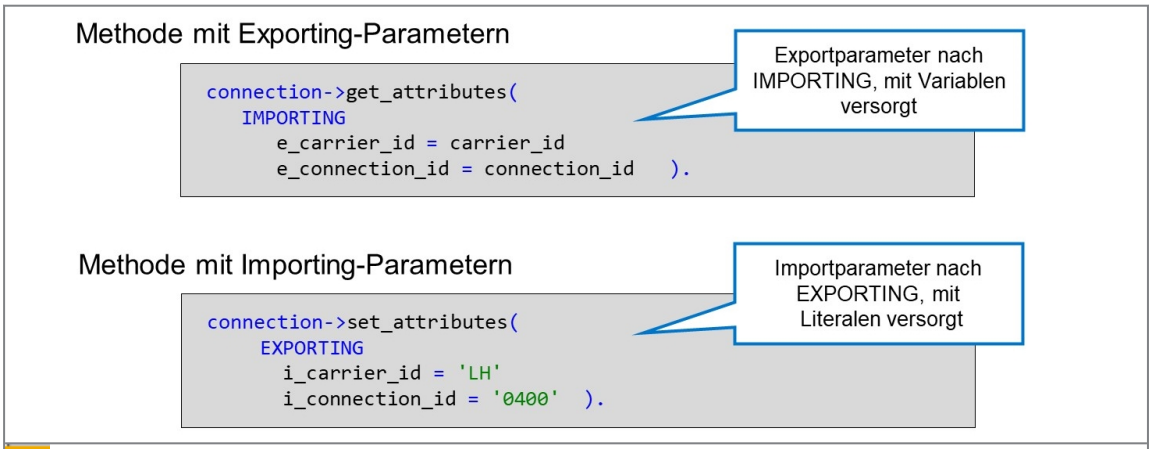


Abbildung 93: Beispiele für Methodenaufrufe

Das Beispiel zeigt den Aufruf der Instanzmethoden `set_attributes( )` und `get_attributes( )`. Die Parameternamen stehen immer auf der linken Seite des Gleichheitszeichens (=).

© Copyright. Alle Rechte vorbehalten.

Hinweis:
Sie können die Code Completion in ADT verwenden, um den Methodenaufruf einschließlich der Parameterübergabe zu generieren. Dazu gehen Sie wie folgt vor :

1. Geben Sie die Referenzvariable (bzw. den Klassennamen) und den Komponentenselektor ein.
2. Nachdem Sie den Komponentenselektor eingegeben haben, drücken Sie **`Strg + Leertaste`**, um eine Liste der Komponenten anzuzeigen, die Sie adressieren können.
3. Wählen Sie eine Methode aus, und drücken **`Sie Umschalt + Enter`**, um den Namen der Methode und ihre vollständige Signatur in Ihren Code einzufügen. Optionale Parameter werden in Kommentarzeilen aufgeführt.

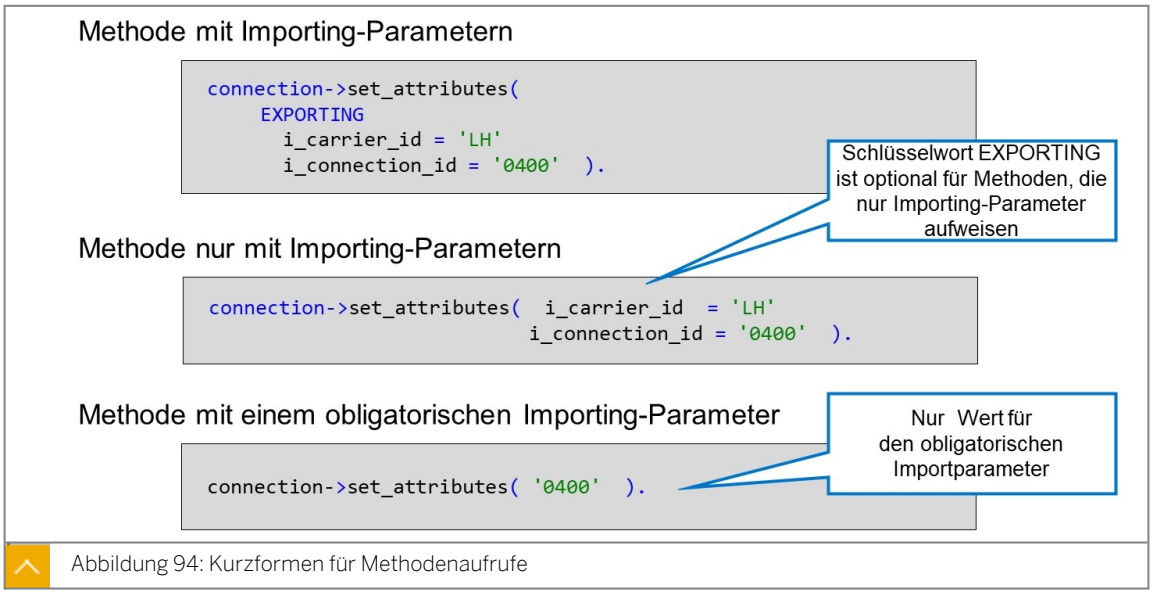


Abbildung 94: Kurzformen für Methodenaufrufe

Beim Aufruf einer Methode wird immer das Schlüsselwort IMPORTING benötigt, um den Wert von Exporting-Parametern zu empfangen. Das Schlüsselwort EXPORTING wird jedoch optional, wenn Sie nur Importing-Parameter bereitstellen möchten.

Bei Methoden mit nur einem obligatorischen Importing-Parameter können Sie sogar die explizite Parameterzuordnung weglassen. Im Beispiel ist der Parameter „i_carrier_id“ optional mit dem Standardwert 'LH'. Daher wird der Wert zwischen den Klammern dem verbleibenden obligatorischen Importing-Parameter „i_connection_id“ zugeordnet.

© Copyright. Alle Rechte vorbehalten.

Ausnahmebehandlung

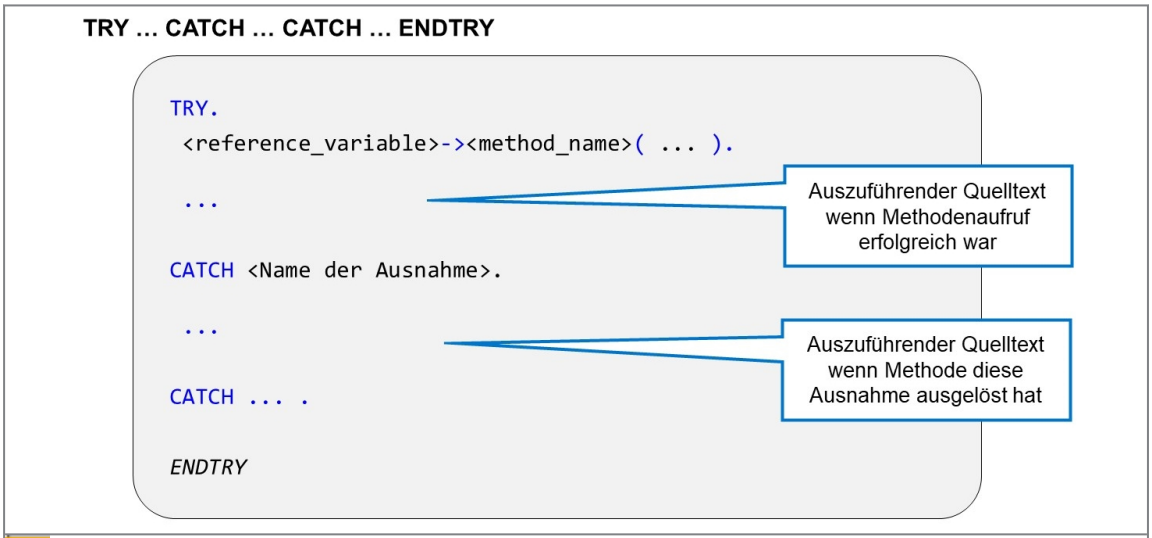


Abbildung 95: Behandlung von Ausnahmen, die von Methoden ausgelöst werden

In einem vorherigen Abschnitt dieses Kurses haben Sie gelernt, wie Sie Laufzeitfehler mit TRY ... ENDTRY-Kontrollstrukturen behandeln.

Ausnahmen, die in der RAISING-Klausel einer Methodendefinition deklariert sind, werden genauso behandelt:

- Platzieren Sie den Methodenaufruf im TRY -Block der TRY ... ENDTRY -Kontrollstruktur.
- Definieren Sie einen CATCH-Block mit dem Namen der Ausnahme, die Sie behandeln möchten.

Notiz:
Wenn Quelltext vorhanden ist, den Sie überspringen möchten, wenn der Methodenaufruf fehlgeschlagen ist, können Sie diesen Quelltext nach dem Methodenaufruf im TRY-Block platzieren.

© Copyright. Alle Rechte vorbehalten.

Funktionale Methoden

Definitionsteil der Klasse „lcl_connection“

```
METHODS get_output
      RETURNING VALUE(r_output) TYPE string_table.
```

Verwendung in der Wertzuweisung (mit Inline-Deklaration)

```
DATA(result) = connection->get_output( ).
```

Verwendung als Eingabe für eine andere Methode

```
out->write( data = connection->get_output( )
            name = ` ` ).
```

Abbildung 96: Funktionale Methoden

Methoden, die einen Returning-Parameter aufweisen, werden als funktionale Methoden bezeichnet. Es ist nicht möglich, mehr als einen Returning-Parameter für dieselbe Methode zu definieren. Sie müssen den Returning-Parameter in der Form VALUE(<Parametername>) definieren. Neben dem Returning-Parameter kann eine funktionale Methode eine beliebige Kombination anderer Parameter aufweisen. Am häufigsten werden jedoch nur Importing-Parameter hinzugefügt. Der Grund dafür ist, dass Sie mit zusätzlichen Exporting- oder Changing-Parametern den größten Vorteil funktionaler Methoden verlieren, nämlich dass Sie das Ergebnis einer funktionalen Methode direkt in anderen ABAP-Ausdrücken verwenden können.

Die Abbildung zeigt zwei Beispiele, wie Sie funktionale Methoden in Ausdrücken verwenden können. Im ersten einfachen Beispiel ist der Returning-Parameter direkt einer Variable zugewiesen. Das zweite Beispiel zeigt, wie der Aufruf der funktionalen Methode als Eingabe für eine andere Methode verwendet wird. Das System führt die funktionale Methode aus und verwendet dann den Wert des Rückgabeparameters als Eingabe für die Methode `write( )`.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Methoden definieren und aufrufen

In dieser Übung definieren und implementieren Sie Methoden in Ihrer lokalen Klasse lcl_connection. Diese Methoden rufen Sie dann aus der `main( )` Methode Ihrer globalen Klasse auf.

Vorlage:

/LRN/CL_S4D400_CLS_INSTANCES (globale Klasse)

Lösung:

/LRN/CL_S4D400_CLS_METHODS (globale Klasse)

Aufgabe 1: Vorlage kopieren (optional)
Kopieren Sie die Vorlageklasse. Wenn Sie die vorherige Übung abgeschlossen haben, können Sie diese Aufgabe überspringen und mit der Bearbeitung Ihrer Klasse **`ZCL_##_LOCAL_CLASS`** oder Ihrer Klasse **`ZCL_##_INSTANCES`** fortfahren.

1. Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_CLS_INSTANCES`** im ABAP Editor.
2. Verknüpfen Sie die View *Project Explorer* mit dem Editor.
3. Kopieren Sie die Klasse **`/LRN/CL_S4D400_CLS_INSTANCES`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_METHODS`**, wobei ## für Ihre Gruppennummer steht).

Aufgabe 2: Methoden definieren
Definieren Sie in Ihrer lokalen Klasse **`lcl_connection`** die Instanzmethoden **`get_output`** und **`set_attributes`**.

1. Navigieren Sie zur Definition der lokalen Klasse **`lcl_connection`**.
2. Fügen Sie dem öffentlichen Bereich der Klasse die Methode **`get_output`** hinzu. Sie sollte einen Rückgabeparameter **`r_output`** des globalen Tabellentyps **`STRING_TABLE`** haben.

Notiz:
Denken Sie daran, den Namen des Returning-Parameters mit `VALUE( )` zu umschließen.

3. Fügen Sie dem öffentlichen Bereich der Klasse eine Methode **`set_attributes`** hinzu. Diese sollte einen Importing-Parameter für jedes Instanzattribut der Klasse aufweisen. Verwenden Sie dieselben Typen für die Parameter, mit denen Sie die Attribute typisiert haben. Um die Parameter von den Attributen zu unterscheiden, fügen Sie dem Parameternamen das Präfix **`i_`** hinzu. Zusätzlich sollte die Methode die Ausnahme **`CX_ABAP_INVALID_VALUE`** auslösen.

© Copyright. Alle Rechte vorbehalten.

Aufgabe 3: Methoden implementieren

1. Verwenden Sie einen Quickfix, um die Methodenimplementierungen zur Klasse hinzuzufügen.
2. Implementieren Sie die Methode **`get_output`**. Hängen Sie einige Zeichenketten-Templates an den Returning-Parameter **`r_output`** an. Verwenden Sie in den Zeichenketten-Templates eingebettete Ausdrücke, um die Werte der Attribute **`carrier_id`** und **`connection_id`** hinzuzufügen.
3. Implementieren Sie die Methode **`set_attributes`**. Wenn einer der beiden Importing-Parameter leer ist (`IS INITIAL`), lösen Sie die Ausnahme **`CX_ABAP_INVALID_VALUE`** aus. Andernfalls füllen Sie die beiden Instanzattribute mit den Werten der Importing-Parameter.
4. Aktivieren Sie Ihre Klasse.

Aufgabe 4: Funktionale Methoden aufrufen

Rufen Sie in der Methode **`main`** Ihrer globalen Klasse die funktionale Methode **`get_output`** auf.

1. Wechseln Sie zur Implementierung der Methode **`if_oo_adt_classrun~main`** in der globalen Klasse.
2. Fügen Sie am Ende der Methode eine Schleife über die interne Tabelle **`connections`** mit der Referenzvariable **`connection`** als Arbeitsbereich hinzu.
3. Rufen Sie in der Schleife nacheinander für jede Instanz die funktionale Methode **`get_output`** auf, und schreiben Sie das Ergebnis in die Konsole.

Hinweis:
Sie können den Aufruf der Methode **`get_output`** direkt als Eingabe für **`out->write( ).`** verwenden.

4. Aktivieren Sie die Klasse. Führen Sie sie als Konsolen-App aus, und analysieren Sie die Konsolenausgabe.

Aufgabe 5: Code Completion verwenden und Ausnahmen behandeln

Verwenden Sie die Code Completion, um die Methode **`set_attributes`** aufzurufen und die von der Methode ausgelöste Ausnahme zu behandeln.

1. Ersetzen Sie für die erste Instanz der Klasse **`lcl_connection`** den direkten Zugriff auf die Attribute **`carrier_id`** und **`connection_id`** durch einen Aufruf der Instanzmethode **`set_attributes( )`**. Verwenden Sie die Code Completion, um die vollständige Signatur der Methode einzufügen.

Hinweis:
Drücken Sie **`Strg + Leertaste`**, nachdem Sie die Komponentenauswahl (->) typisiert haben, um eine Liste der verfügbaren Attribute und Methoden anzuzeigen. Wählen Sie die Methode, und drücken Sie dann **`Umschalt + Eingabetaste`**, um die vollständige Signatur der Methode einzufügen.

© Copyright. Alle Rechte vorbehalten.

2. Behandeln Sie die Ausnahme. Umschließen Sie den Methodenaufruf mit `TRY.` und `ENDTRY.`. Fügen Sie einen CATCH-Block hinzu, um die Ausnahme **`CX_ABAP_INVALID_VALUE`** zu behandeln. Stellen Sie sicher, dass die neue Instanz nur zur internen Tabelle **`connections`** hinzugefügt wird, wenn der Methodenaufruf erfolgreich war.
3. Wiederholen Sie den vorherigen Schritt für die anderen beiden Instanzen der Klasse **`lcl_connection`**.
4. Aktivieren Sie die Klasse. Führen Sie sie aus, und debuggen Sie die Methodenaufrufe.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3
Lösung 10

Methoden definieren und aufrufen

In dieser Übung definieren und implementieren Sie Methoden in Ihrer lokalen Klasse lcl_connection. Diese Methoden rufen Sie dann aus der `main( )` Methode Ihrer globalen Klasse auf.

Vorlage:

/LRN/CL_S4D400_CLS_INSTANCES (globale Klasse)

Lösung:

/LRN/CL_S4D400_CLS_METHODS (globale Klasse)

Aufgabe 1: Vorlage kopieren (optional)
Kopieren Sie die Vorlageklasse. Wenn Sie die vorherige Übung abgeschlossen haben, können Sie diese Aufgabe überspringen und mit der Bearbeitung Ihrer Klasse **`ZCL_##_LOCAL_CLASS`** oder Ihrer Klasse **`ZCL_##_INSTANCES`** fortfahren.

1. Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_CLS_INSTANCES`** im ABAP Editor.
   a) Wählen Sie in der *Eclipse*-Symbolleiste *Open ABAP Development Object*. Drücken Sie alternativ **`Strg + Umschalt + A`**.
   b) Geben Sie als Suchzeichenfolge **`/LRN/CL_S4D400_CLS`** ein.
   c) Wählen Sie aus der Liste der Entwicklungsobjekte */LRN/CL_S4D400_CLS_INSTANCES* aus, und wählen Sie dann *OK*.
2. Verknüpfen Sie die View *Project Explorer* mit dem Editor.
   a) Suchen Sie in der Symbolleiste *Project Explorer* die Drucktaste *Link with Editor*. Wählen Sie die Drucktaste, falls diese noch nicht markiert ist. Dadurch sollte das Entwicklungsobjekt, das im Editor geöffnet ist, im *Project Explorer* hervorgehoben sein.
3. Kopieren Sie die Klasse **`/LRN/CL_S4D400_CLS_INSTANCES`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_METHODS`**, wobei ## für Ihre Gruppennummer steht).
   a) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Klasse **`/LRN/CL_S4D400_CLS_INSTANCES`**, um das Kontextmenü zu öffnen.
   b) Wählen Sie im Kontextmenü *Duplicate ...*
   c) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_METHODS`** ein, wobei ## für Ihre Gruppennummer steht.
   d) Passen Sie die Beschreibung an, und wählen Sie *Next*.
   e) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

© Copyright. Alle Rechte vorbehalten.

Aufgabe 2: Methoden definieren

Definieren Sie in Ihrer lokalen Klasse **`lcl_connection`** die Instanzmethoden **`get_output`** und **`set_attributes`**.

1. Navigieren Sie zur Definition der lokalen Klasse **`lcl_connection`**.
   a) Wählen Sie in der globalen Klasse *Local Types*.
2. Fügen Sie dem öffentlichen Bereich der Klasse die Methode **`get_output`** hinzu. Sie sollte einen Rückgabeparameter **`r_output`** des globalen Tabellentyps **`STRING_TABLE`** haben.

Notiz:
Denken Sie daran, den Namen des Returning-Parameters mit `VALUE( )` zu umschließen.

   a) Passen Sie den Quelltext wie folgt an:

```
PUBLIC SECTION.

   DATA carrier_id    TYPE /dmo/carrier_id.
   DATA connection_id TYPE /DMO/Connection_id.

   CLASS-DATA conn_counter TYPE i.

   METHODS get_output
     returning
       value(r_output) type string_table.

 PROTECTED SECTION.
```

Notiz:
Ignorieren Sie für den Moment den Syntaxfehler *///Implementierung fehlt für Methode "GET_OUTPUT"*.

3. Fügen Sie dem öffentlichen Bereich der Klasse eine Methode **`set_attributes`** hinzu. Diese sollte einen Importing-Parameter für jedes Instanzattribut der Klasse aufweisen. Verwenden Sie dieselben Typen für die Parameter, mit denen Sie die Attribute typisiert haben. Um die Parameter von den Attributen zu unterscheiden, fügen Sie dem Parameternamen das Präfix **`i_`** hinzu. Zusätzlich sollte die Methode die Ausnahme **`CX_ABAP_INVALID_VALUE`** auslösen.
   a) Passen Sie den Quelltext wie folgt an:

```
PUBLIC SECTION.

   DATA carrier_id    TYPE /dmo/carrier_id.
   DATA connection_id TYPE /DMO/Connection_id.

   CLASS-DATA conn_counter TYPE i.

   METHODS set_attributes
     IMPORTING
       i_carrier_id    TYPE /dmo/carrier_id
```

© Copyright. Alle Rechte vorbehalten.

```
        i_connection_id TYPE /dmo/connection_id
      RAISING
        cx_abap_invalid_value.

    METHODS get_output
      returning
        value(r_output) type string_table.

  PROTECTED SECTION.
```

Notiz:
Ignorieren Sie für den Moment den Syntaxfehler *///Implementierung fehlt für Methode "SET_ATTRIBUTES"*.

Aufgabe 3: Methoden implementieren

1. Verwenden Sie einen Quickfix, um die Methodenimplementierungen zur Klasse hinzuzufügen.
   a) Positionieren Sie den Cursor auf den Namen einer der Methoden im Editor, und drücken Sie **`Strg + 1`**.
   b) Doppelklicken Sie auf den Vorschlag *///2 nicht implementierte Methoden hinzufügen*.
2. Implementieren Sie die Methode **`get_output`**. Hängen Sie einige Zeichenketten-Templates an den Returning-Parameter **`r_output`** an. Verwenden Sie in den Zeichenketten-Templates eingebettete Ausdrücke, um die Werte der Attribute **`carrier_id`** und **`connection_id`** hinzuzufügen.
   a) Fügen Sie in der lokalen Klasse den folgenden Quelltext zwischen den Anweisungen `METHOD get_output.` und `ENDMETHOD.` hinzu:

```
    APPEND |------------------------------| TO r_output.
    APPEND |Carrier:     { carrier_id    }| TO r_output.
    APPEND |Connection:  { connection_id }| TO r_output.
```

3. Implementieren Sie die Methode **`set_attributes`**. Wenn einer der beiden Importing-Parameter leer ist (`IS INITIAL`), lösen Sie die Ausnahme **`CX_ABAP_INVALID_VALUE`** aus. Andernfalls füllen Sie die beiden Instanzattribute mit den Werten der Importing-Parameter.
   a) Passen Sie den Quelltext wie folgt an:

```
METHOD set_attributes.

  IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
    RAISE EXCEPTION TYPE cx_abap_invalid_value.
  ENDIF.

  carrier_id    = i_carrier_id.
  connection_id = i_connection_id.

ENDMETHOD.
```

4. Aktivieren Sie Ihre Klasse.
   a) Wählen Sie **`Strg + F3`**, um die Klasse zu aktivieren.

© Copyright. Alle Rechte vorbehalten.

Aufgabe 4: Funktionale Methoden aufrufen
Rufen Sie in der Methode **`main`** Ihrer globalen Klasse die funktionale Methode **`get_output`** auf.

1. Wechseln Sie zur Implementierung der Methode **`if_oo_adt_classrun~main`** in der globalen Klasse.
   a) Wählen Sie in der globalen Klasse *///Global Class*, blättern Sie nach unten zur Implementierung der Methode **`if_oo_adt_classrun~main`**.
2. Fügen Sie am Ende der Methode eine Schleife über die interne Tabelle **`connections`** mit der Referenzvariable **`connection`** als Arbeitsbereich hinzu.
   a) Fügen Sie folgenden Code vor `ENDMETHOD.` hinzu:

```
LOOP AT connections INTO connection.

ENDLOOP.
```

3. Rufen Sie in der Schleife nacheinander für jede Instanz die funktionale Methode **`get_output`** auf, und schreiben Sie das Ergebnis in die Konsole.

Hinweis:
Sie können den Aufruf der Methode **`get_output`** direkt als Eingabe für **`out->write( ).`** verwenden.

   a) Passen Sie den Quelltext wie folgt an:

```
LOOP AT connections INTO connection.

   out->write( connection->get_output( ) ).

ENDLOOP.
```

4. Aktivieren Sie die Klasse. Führen Sie sie als Konsolen-App aus, und analysieren Sie die Konsolenausgabe.
   a) Drücken Sie **`Strg + F3`**, um die Klasse zu aktivieren.
   b) Drücken Sie **`F9`**, um die Klasse auszuführen.
   c) Analysieren Sie die Ausgabe im Fenster *Console*.

Aufgabe 5: Code Completion verwenden und Ausnahmen behandeln
Verwenden Sie die Code Completion, um die Methode **`set_attributes`** aufzurufen und die von der Methode ausgelöste Ausnahme zu behandeln.

1. Ersetzen Sie für die erste Instanz der Klasse **`lcl_connection`** den direkten Zugriff auf die Attribute **`carrier_id`** und **`connection_id`** durch einen Aufruf der Instanzmethode **`set_attributes( )`**. Verwenden Sie die Code Completion, um die vollständige Signatur der Methode einzufügen.

© Copyright. Alle Rechte vorbehalten.

Hinweis:
Drücken Sie **Strg + Leertaste**, nachdem Sie die Komponentenauswahl (->) typisiert haben, um eine Liste der verfügbaren Attribute und Methoden anzuzeigen. Wählen Sie die Methode, und drücken Sie dann **Umschalt + Eingabetaste**, um die vollständige Signatur der Methode einzufügen.

a) Positionieren Sie den Cursor in der nächsten Zeile nach der ersten `connection = NEW #( ).`.

b) Geben Sie `connection->` ein, und drücken Sie **Strg + Leertaste**.

c) Wählen Sie *set_attributes*, und drücken Sie **Umschalt + Eingabetaste**.

d) Übergeben Sie Werte an die Importing-Parameter. Verwenden Sie dieselben Literale, die Sie zuvor zum Setzen der Attribute dieser Instanz verwendet haben.

e) Entfernen oder kommentieren Sie den direkten Zugriff auf die Instanzattribute für diese Instanz.

2. Behandeln Sie die Ausnahme. Umschließen Sie den Methodenaufruf mit `TRY.` und `ENDTRY.`. Fügen Sie einen CATCH-Block hinzu, um die Ausnahme **CX_ABAP_INVALID_VALUE** zu behandeln. Stellen Sie sicher, dass die neue Instanz nur zur internen Tabelle **connections** hinzugefügt wird, wenn der Methodenaufruf erfolgreich war.

a) Kommentieren Sie die generierte Quelltextzeile `CATCH cx_abap_invalid_value.` aus.

b) Fügen Sie `TRY.` vor dem Methodenaufruf hinzu.

c) Fügen Sie `ENDTRY.` nach der CATCH-Anweisung hinzu.

d) Fügen Sie zwischen der CATCH-Anweisung und der ENDTRY-Anweisung Folgendes hinzu: `out->write( `Method call failed` ).`

e) Verschieben Sie die APPEND-Anweisung nach oben zwischen den Methodenaufruf und die CATCH-Anweisung.

f) Der vollständige Methodenaufruf sollte nun wie folgt aussehen:

```
    TRY.
        connection->set_attributes(
          EXPORTING
            i_carrier_id    = 'LH'
            i_connection_id = '0400'
        ).

*       connection->carrier_id    = 'LH'.
*       connection->connection_id = '0400'.

        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.
```


© Copyright. Alle Rechte vorbehalten.

3. Wiederholen Sie den vorherigen Schritt für die anderen beiden Instanzen der Klasse **`lcl_connection`**.

   a) Nach diesem Schritt sollte die Implementierung der Methode **`if_oo_adt_classrun~main`** wie folgt aussehen:

```
METHOD if_oo_adt_classrun~main.

    DATA connection TYPE REF TO lcl_connection.
    DATA connections TYPE TABLE OF REF TO lcl_connection.

* First Instance
**********************************************************************
    connection = NEW #(  ).

    TRY.
        connection->set_attributes(
          EXPORTING
            i_carrier_id    = 'LH'
            i_connection_id = '0400'
        ).

*        connection->carrier_id    = 'LH'.
*        connection->connection_id = '0400'.

        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

* Second instance
**********************************************************************

    connection = NEW #(  ).

    TRY.
        connection->set_attributes(
          EXPORTING
            i_carrier_id    = 'AA'
            i_connection_id = '0017'
        ).

*        connection->carrier_id    = 'AA'.
*        connection->connection_id = '0017'.

        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

* Third instance
**********************************************************************
    connection = NEW #(  ).

    TRY.
        connection->set_attributes(
          EXPORTING
            i_carrier_id    = 'SQ'
            i_connection_id = '0001'
        ).
```

© Copyright. Alle Rechte vorbehalten.

```
*         connection->carrier_id    = 'SQ'.
*         connection->connection_id = '0001'.

          APPEND connection TO connections.

        CATCH cx_abap_invalid_value.
          out->write( `Method call failed` ).
      ENDTRY.


* Output
**********************************************************************

      LOOP AT connections INTO connection.

        out->write( connection->get_output( ) ).

      ENDLOOP.

  ENDMETHOD.
```

4. Aktivieren Sie die Klasse. Führen Sie sie aus, und debuggen Sie die Methodenaufrufe.
   a) Drücken Sie **Strg + F3**, um die Klasse zu aktivieren.
   b) Drücken Sie **F9**, um die Klasse auszuführen.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Methoden definieren und aufrufen.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3
Lektion 4

Kapselung zur Sicherstellung der Konsistenz verwenden

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Erläutern Sie die Kapselung.
- Definieren und verwenden Sie Konstruktoren.

Datenkapselung

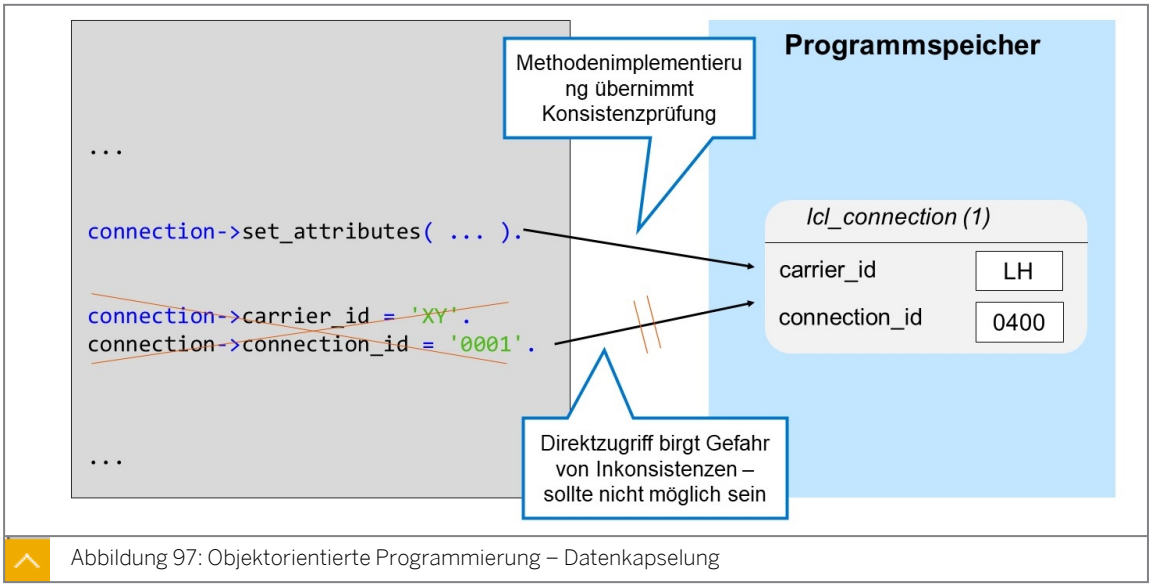


Abbildung 97: Objektorientierte Programmierung – Datenkapselung

Bei der objektorientierten Programmierung entspricht ein Objekt einem realen Objekt (z.B. einem Beschäftigten, einem Fahrzeug oder, wie im vorliegenden Fall, einer Flugverbindung). Es weist Attribute auf, die es beschreiben. Im Fall einer Flugverbindung sind dies die Fluggesellschafts-ID und die Flugnummer.

Betrachten wir nun, was passiert, wenn ein Programm mit diesem Objekt versucht, Werte für diese Attribute bereitzustellen. Es ist offensichtlich, dass nur die Kombinationen aus Fluggesellschafts-ID und Flugnummer akzeptiert werden dürfen, die einer tatsächlich vorhandenen Flugverbindung entsprechen.

Bei der Objektorientierung darf das Client-Programm nicht in der Lage sein, die Attributwerte direkt zu ändern. Stattdessen sollte es eine Methode aufrufen müssen, um diese Aufgabe auszuführen. Die Methode, die mit dem Objekt bereitgestellt wird, kann dann prüfen, ob die Kombination aus Fluggesellschafts-ID und Flugnummer gültig ist, und, wenn dies nicht der Fall ist, die Änderung ablehnen.

© Copyright. Alle Rechte vorbehalten.

Als Entwickler der Klasse „lcl_connection“ können wir die Verwendung der Methode „set_attributes( )“ sicherstellen, indem wir die Attribute „carrier_id“ und „connection_id“ privat oder zumindest schreibgeschützt machen.

Dieses Konzept wird als Datenkapselung bezeichnet. Die Informationen über die Flugverbindung werden vom Verbindungsobjekt selbst verwaltet und können nicht von anderen Personen bearbeitet werden. Dadurch wird sichergestellt, dass die Prüfung der Konsistenz nicht umgangen werden kann - ob wissentlich oder unwissentlich. Dies ist einer der wichtigsten Vorteile der Objektorientierung.

Notiz:
Bei der objektorientierten Programmierung wird empfohlen, die Datenkapselung so weit wie möglich zu verwenden!

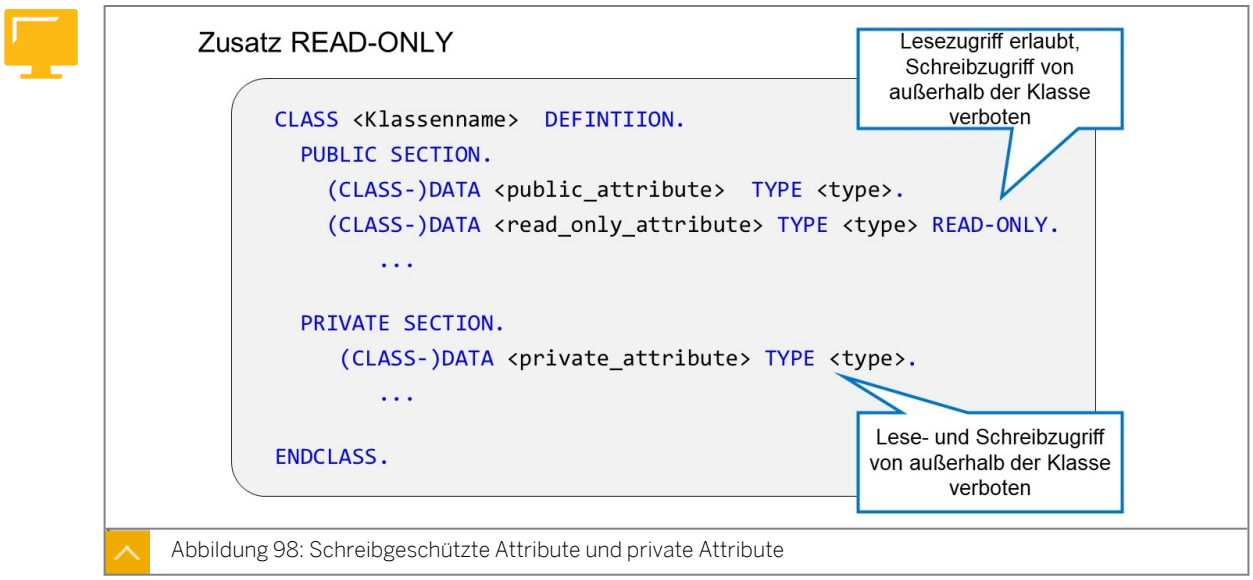


Abbildung 98: Schreibgeschützte Attribute und private Attribute

Mit den öffentlichen Attributen, die wir bisher definiert haben, war es möglich, die Werte der Attribute überall innerhalb und außerhalb der Klasse zu lesen und zu ändern.

Um den Zugriff auf die Attribute einzuschränken, gibt es zwei Möglichkeiten:

1. Behalten Sie die DATA- oder CLASS-DATA-Anweisung im öffentlichen Bereich bei, und fügen Sie den Zusatz READ-ONLY hinzu. Damit ist der Schreibzugriff auf das Attribut außerhalb der Klasse verboten, ein Lesezugriff ist aber weiterhin möglich.

   Notiz:
   Der Zusatz READ-ONLY ist nur im öffentlichen Bereich einer Klasse zulässig.

2. Verschieben Sie die Anweisung DATA oder CLASS-DATA aus dem öffentlichen Bereich in einen der anderen Sichtbarkeitsbereiche, z.B. den privaten Bereich. Damit ist der Lese- und Schreibzugriff auf das Attribut außerhalb der Klasse verboten.

© Copyright. Alle Rechte vorbehalten.

Hinweis:
ADT bietet einen Quickfix zum Ändern der Sichtbarkeit eines Attributs. Um diesen zu verwenden, positionieren Sie den Cursor im Attributnamen, drücken Sie **Strg + 1**, und wählen Sie „///Als privat kennzeichnen“.

Instanzkonstruktor

```
DATA-Verbindung TYPE REF TO

 lcl_connection.DATAconnection2 TYPE
REF TO
 lcl_connection.



connection = NEW #( ).



connection2 = NEW #( ).

connection2->set_attributes(
 i_carrier_id  = 'LH'.
i_connection_id  = '0400'.

connection2->set_attributes(
 i_carrier_id  = 'AA'.
i_connection_id  = '0017'.
```

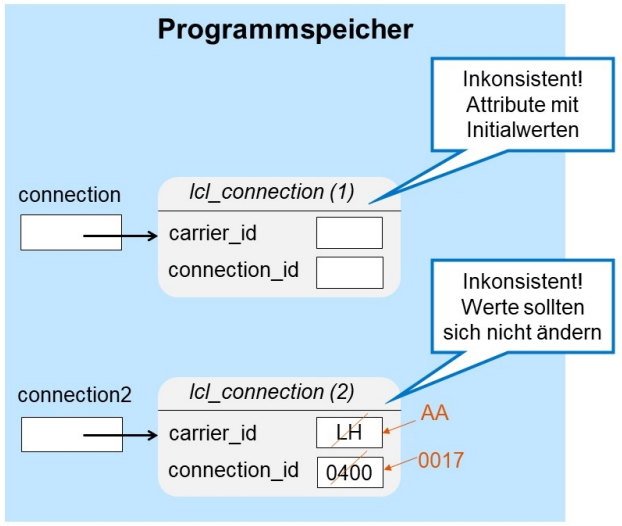


Abbildung 99: Motivation für Konstruktormethoden

Indem Sie Ihre Attribute privat oder zumindest schreibgeschützt machen, können Sie sicherstellen, dass das Client-Programm die verfügbare Methode „set_attributes( )“ verwendet, um die Werte für die Attribute „carrier_id“ und „connection_id“ zu setzen.

Dennoch können weiterhin Inkonsistenzen auftreten:

- Sie können nicht erzwingen, dass das Programm „set_attributes( )“ für jede neue Instanz aufruft. Dadurch kann es Instanzen mit Initialwerten für „carrier_id“ und „connection_id“ geben.
- Das Client-Programm kann „set_attributes( )“ mehrmals für dieselbe Instanz aufrufen. Dies sollte auch nicht möglich sein.

Sie benötigen eine Methode, um nicht initiale Werte während der Instanziierung zu erzwingen und spätere Änderungen zu verhindern.

Um diese Probleme zu lösen, verwenden objektorientierte Programmiersprachen Konstruktormethoden.

© Copyright. Alle Rechte vorbehalten.

```
CLASS <Klassenname> DEFINITION.
   PUBLIC SECTION.

 METHODS constructor
         IMPORTING <Eingabe_1> TYPE <Typ>
 <Eingabe_2> TYPE <Typ>  DEFAULT  <Wert>
 ...
 RAISING  <Ausnahme1>
 <Ausnahme2>
 ... .
ENDCLASS.
```

Abbildung 100: Konstruktormethodendefinition

Der Konstruktor wird vom Laufzeitsystem automatisch beim Anlegen einer neuen Instanz der Klasse aufgerufen, kann jedoch nicht explizit aufgerufen werden. Es wird vom System gewährleistet, dass der Konstruktor für jede angelegte Instanz genau einmal ausgeführt werden kann.

Aus Syntaxsicht hat eine Konstruktormethode folgende Eigenschaften:

- Sie ist eine öffentliche Instanzmethode mit dem reservierten Namen „constructor“.
- Sie kann Importing-Parameter aufweisen, z.B. zum Abrufen der Startwerte für die Attribute der neuen Instanz.
- Sie kann Ausnahmen auslösen.

Notiz:
In ABAP ist es nicht möglich, mehr als eine Konstruktormethode in derselben Klasse zu definieren.

Hinweis:
In ADT können Sie einen Quickfix verwenden, um eine Konstruktormethode zu generieren. Um diesen Quickfix zu verwenden, gehen Sie wie folgt vor:

1. Positionieren Sie den Cursor entweder im Definitions- oder im Implementierungsteil im Klassennamen, und drücken Sie Strg + 1.
2. Wählen Sie aus der Liste der verfügbaren Quickfixes „///Konstruktor generieren“.
3. Wählen Sie im daraufhin angezeigten Dialogfenster die Attribute aus, die Sie mit dem Konstruktor initialisieren möchten, und wählen Sie ///Finish“.

Nachdem Sie den Konstruktor generiert haben, können Sie seine Definition und Implementierung an Ihre Anforderungen anpassen.

© Copyright. Alle Rechte vorbehalten.

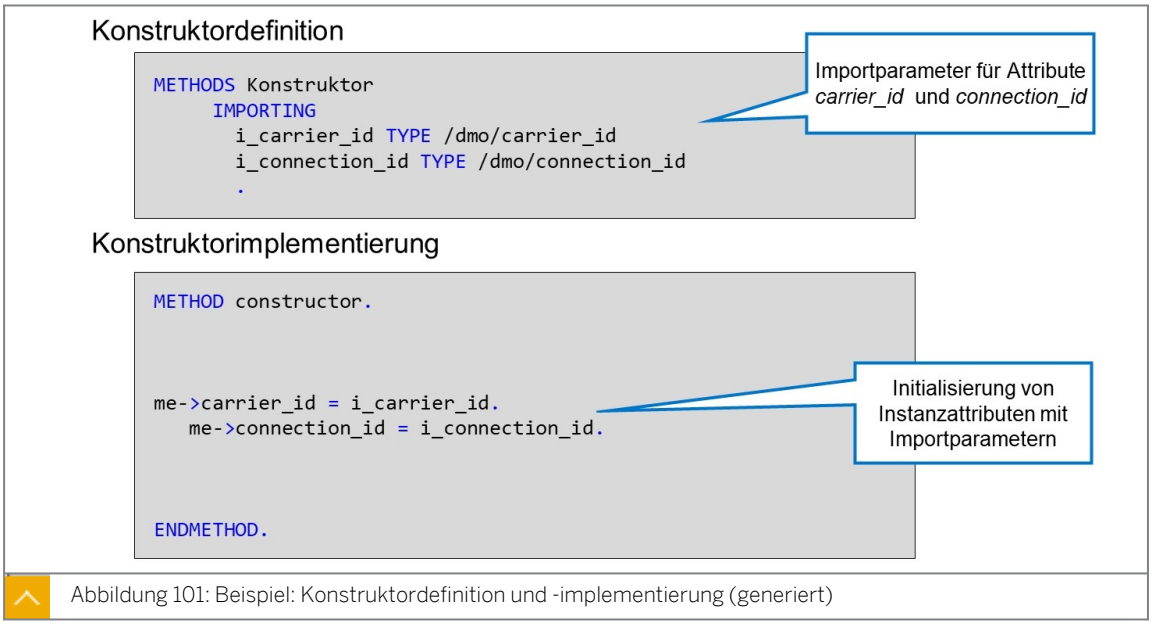


Abbildung 101: Beispiel: Konstruktordefinition und -implementierung (generiert)

Das Beispiel zeigt den Konstruktor der Klasse „lcl_connection“. Die generierte Definition enthält einen Importparameter für jedes der Attribute `carrier_id` und `connection_id`.

Der generierte Teil der Implementierung enthält eine Wertzuweisung für jedes der Attribute mit dem zugehörigen Importing-Parameter auf der rechten Seite.

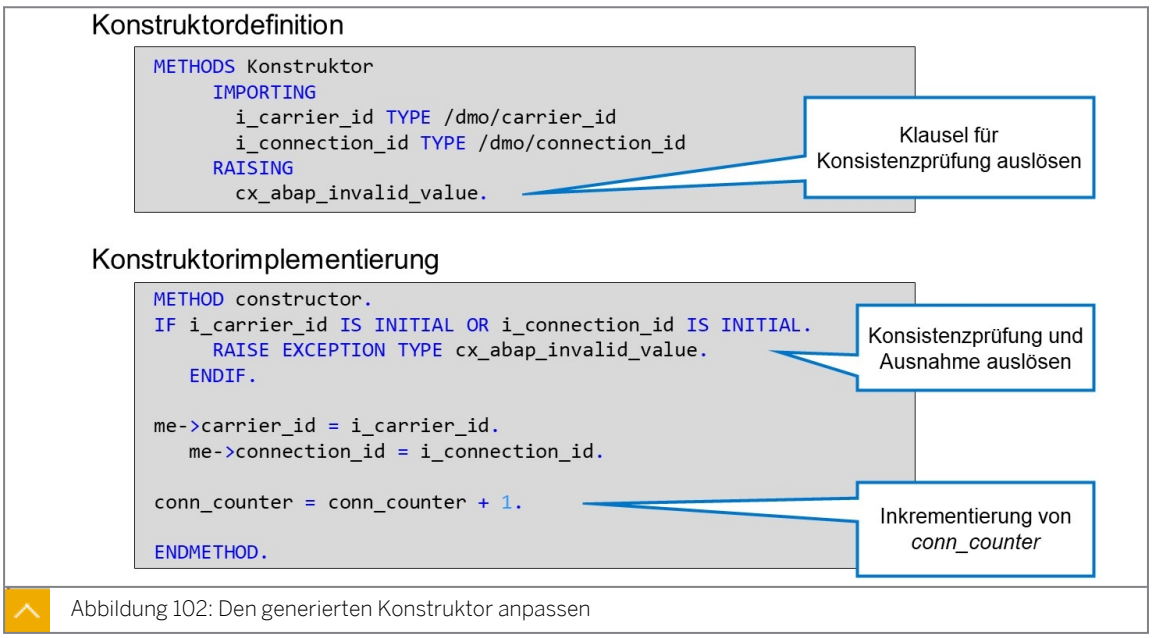


Abbildung 102: Den generierten Konstruktor anpassen

Das Beispiel zeigt einige manuelle Zusätze zum generierten Konstruktor der Klasse „lcl_connection“.

Um das Anlegen von Instanzen mit Initialwerten abzulehnen, wurde die Implementierung um eine Konsistenzprüfung und die Definition des Konstruktors um eine RAISING-Klausel erweitert.

Da der Konstruktor für jede neue Instanz der Klasse „lcl_connection“ nur genau einmal ausgeführt wird, ist die Konstruktorimplementierung die perfekte Stelle zum Inkrementieren des statischen Attributs „conn_counter“.

© Copyright. Alle Rechte vorbehalten.

Konstruktordefinition

```
METHODS Konstruktor
    IMPORTING
      i_carrier_id TYPE /dmo/carrier_id
      i_connection_id TYPE /dmo/connection_id
    RAISING
      cx_abap_invalid_value.
```

Instanziierung mit NEW #( )

```
DATA-Verbindung TYPE REF TO lcl_connection.

TRY.
  connection = NEW #( i_carrier_id = 'LH'
                      i_connection_id = '0400' ).

CATCH cx_abap_invalid_value.
    ...
ENDTRY.
```

Importparameter der Methode *constructor( )* angeben

Abbildung 103: Klassen mit Konstruktor instanziieren

Wenn Sie eine Klasse mit Konstruktor instanziieren, ruft das System die Konstruktormethode automatisch auf. Wenn der Konstruktor Importing-Parameter hat, übergeben Sie sie an den NEW-Ausdruck. Gehen Sie dabei so vor, wie bei der Übergabe von Parametern an eine normale Methode.

Notiz:
Ein Konstruktor kann nur Importing-Parameter enthalten. Das Schlüsselwort EXPORTING ist im NEW-Ausdruck weder erforderlich noch zulässig.

Notiz:
Falls der Konstruktor Ausnahmen hat, müssen Sie sicherstellen, dass Sie diese abfangen, indem Sie die Instanziierung in einen Block TRY... CATCH...ENDTRY einschließen. Falls ein Konstruktor eine Ausnahme auslöst, wird die Steuerung direkt zum Programm mit dem NEW-Ausdruck zurückgegeben. In diesem Fall erhalten Sie keine neue Instanz der Klasse.

Statische Konstruktoren

```
CLASS <Klassenname> DEFINITION.
   ÖFFENTLICHER ABSCHNITT.

 CLASS-METHODS class_constructor.ENDCLASS
.
```

Abbildung 104: Statische Konstruktordefinition

Der Instanzkonstruktor wird beim Anlegen einer Instanz für eine Klasse jeweils einmal aufgerufen. Manchmal müssen Sie Aktionen nur einmal für die gesamte Klasse durchführen. Zu diesem Zweck ermöglicht ABAP die Definition eines statischen Konstruktors, auch Klassenkonstruktor genannt.

© Copyright. Alle Rechte vorbehalten.

Das Laufzeitsystem ruft den statischen Konstruktor nur einmal auf, wenn die Klasse während der Ausführung eines Programms zum ersten Mal adressiert wird.

Die erste Adressierung einer Klasse könnte wie folgt aussehen:

- Erste Instanziierung der Klasse
- Erster Aufruf einer statischen Methode
- Erster Zugriff auf ein öffentliches statisches Attribut

Notiz:
Diese Liste ist nicht vollständig. Es gibt weitere Aktionen (z.B. im Rahmen der Vererbung), die dazu führen können, dass eine Klasse zum ersten Mal adressiert wird.

Ein typischer Anwendungsfall für den statischen Konstruktor ist die dynamische Initialisierung von statischen Attributen mit nicht initialen Werten. Daher ist es wichtig, dass die Laufzeit den statischen Konstruktor vor dem Anlegen der Instanz, dem Aufrufen der statischen Methode oder dem Adressieren eines statischen Attributs aufruft.

Aus Syntaxsicht hat eine Konstruktormethode folgende Eigenschaften:

- Sie ist eine öffentliche statische Methode mit dem reservierten Namen „class_constructor“.
- Sie weist keine Parameter oder Ausnahmen auf.

Notiz:
Ein statischer Konstruktor muss keine Signatur aufweisen, da es unmöglich ist, vorauszusagen, wann eine Klasse zum ersten Mal adressiert wird.

Hinweis:
In ADT können Sie einen Quickfix verwenden, um eine Konstruktormethode für die Klasse zu generieren. Um diesen Quickfix zu verwenden, gehen Sie wie folgt vor:

1. Positionieren Sie den Cursor entweder im Definitions- oder im Implementierungsteil im Klassennamen, und drücken Sie **`Strg + 1`**.
2. Wählen Sie aus der Liste der verfügbaren Quickfixes „///Generate class constructor“.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3
Übung 11

Private Attribute und einen Konstruktor verwenden

In dieser Übung machen Sie das statische Attribut Ihrer Klasse schreibgeschützt und die Instanzattribute privat. Sie verwenden einen Konstruktor, um deren Werte zu setzen.

Vorlage:

/LRN/CL_S4D400_CLS_METHODS (globale Klasse)

Lösung:

/LRN/CL_S4D400_CLS_CONSTRUCTOR (globale Klasse)

Aufgabe 1: Vorlage kopieren (optional)
Kopieren Sie die Vorlageklasse. Wenn Sie die vorherige Übung abgeschlossen haben, können Sie diese Aufgabe überspringen und mit der Bearbeitung Ihrer Klasse (**`ZCL_##_METHODS`**, **`ZCL_##_LOCAL_CLASS`** oder **`ZCL_##_INSTANCES`**) fortfahren.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_CLS_METHODS`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_CONSTRUCTOR`**, wobei ## für Ihre Gruppennummer steht).

Aufgabe 2: Attribute privat machen
Ändern Sie die Sichtbarkeit der Attribute **`carrier_id`** und **`connection_id`**, um die Verwendung der Methoden **`set_attributes( )`** und **`get_output( )`** zu erzwingen.

1. Setzen Sie die Sichtbarkeit des Attributs **`carrier_id`** mit einem Quickfix auf privat.
2. Setzen Sie die Sichtbarkeit des Attributs **`connection_id`** mit einem Quickfix auf privat.

Aufgabe 3: Instanzkonstruktor anlegen
Ersetzen Sie die öffentliche Methode **`set_attributes`** durch einen Instanzkonstruktor, um sicherzustellen, dass die Attribute beim Anlegen einer neuen Instanz gesetzt und danach nicht mehr geändert werden.

1. Kommentieren Sie die Definition und Implementierung der Methode **`set_attributes`** aus.
2. Fügen Sie der lokalen Klasse **`lcl_connection`** per Quickfix einen Instanzkonstruktor hinzu. Stellen Sie sicher, dass der Konstruktor Importing-Parameter aufweist, die den Attributen **`carrier_id`** und **`connection_id`** entsprechen.
3. Erweitern Sie die generierte Konstruktordefinition. Fügen Sie der Definition des **`Methodenkonstruktors`** die Ausnahme **`CX_ABAP_INVALID_VALUE`** hinzu.
4. Erweitern Sie die generierte Konstruktorimplementierung. Wenn einer der Importing-Parameter initial ist, lösen Sie die Ausnahme **`CX_ABAP_INVALID_VALUE`** aus.

© Copyright. Alle Rechte vorbehalten.

5. Fügen Sie in der Konstruktorimplementierung eine Anweisung hinzu, um den Wert des statischen Attributs **`conn_counter`** um eins zu erhöhen. Stellen Sie sicher, dass die Anweisung nur ausgeführt wird, wenn keine Ausnahme ausgelöst wurde.
6. Stellen Sie sicher, dass es nicht möglich ist, den Wert von **`conn_counter`** von außerhalb der Klasse zu ändern.

Aufgabe 4: Den Konstruktor verwenden
Passen Sie die Instanziierung der Klasse **`lcl_connection`** an, um die Parameter des Instanzkonstruktors bereitzustellen und die Ausnahme zu behandeln.

1. Gehen Sie in der Methode **`if_oo_adt_classrun~main`** zum Ausdruck NEW #( ) für die erste Instanz von **`lcl_connection`**.
2. Geben Sie im NEW-Ausdruck die Importing-Parameter von **`constructor`** an.

Hinweis:
Sie können die Parameterübergabe aus dem Aufruf der Methode **`set_attributes`** für diese Instanz übernehmen.

3. Entfernen oder kommentieren Sie alle Aufrufe der Methode **`set_attributes`** aus.
4. Verschieben Sie das Anlegen der Instanz in den TRY-Block der Ausnahmebehandlung.
5. Wiederholen Sie die vorherigen Schritte für die anderen Instanzen Ihrer lokalen Klasse.
6. Aktivieren Sie die Klasse. Führen Sie sie aus, und debuggen Sie die Instantiierung.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3
Lösung 11

Private Attribute und einen Konstruktor verwenden

In dieser Übung machen Sie das statische Attribut Ihrer Klasse schreibgeschützt und die Instanzattribute privat. Sie verwenden einen Konstruktor, um deren Werte zu setzen.

Vorlage:

/LRN/CL_S4D400_CLS_METHODS (globale Klasse)

Lösung:

/LRN/CL_S4D400_CLS_CONSTRUCTOR (globale Klasse)

Aufgabe 1: Vorlage kopieren (optional)
Kopieren Sie die Vorlageklasse. Wenn Sie die vorherige Übung abgeschlossen haben, können Sie diese Aufgabe überspringen und mit der Bearbeitung Ihrer Klasse (**`ZCL_##_METHODS`**, **`ZCL_##_LOCAL_CLASS`** oder **`ZCL_##_INSTANCES`**) fortfahren.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_CLS_METHODS`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_CONSTRUCTOR`**, wobei ## für Ihre Gruppennummer steht).
   a) Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_CLS_METHODS`**.
   b) Verknüpfen Sie die View *Project Explorer* mit dem Editor.
   c) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Klasse **`/LRN/CL_S4D400_CLS_METHODS`**, um das Kontextmenü zu öffnen.
   d) Wählen Sie im Kontextmenü *Duplicate ...*
   e) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_CONSTRUCTOR`** ein, wobei ## für Ihre Gruppennummer steht.
   f) Passen Sie die Beschreibung an, und wählen Sie *Next*.
   g) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

Aufgabe 2: Attribute privat machen
Ändern Sie die Sichtbarkeit der Attribute **`carrier_id`** und **`connection_id`**, um die Verwendung der Methoden **`set_attributes( )`** und **`get_output( )`** zu erzwingen.

1. Setzen Sie die Sichtbarkeit des Attributs **`carrier_id`** mit einem Quickfix auf privat.
   a) Wechseln Sie zur Registerkarte *///Lokale Typen*.
   b) Gehen Sie zur Deklaration des Attributs **`carrier_id`** in der lokalen Klasse **`lcl_connection`**.
   c) Positionieren Sie den Cursor auf **`carrier_id`**, und drücken Sie **`Strg + 1`**.

© Copyright. Alle Rechte vorbehalten.

d) Doppelklicken Sie auf den Vorschlag */// Carrier_id privat machen* .

e) Prüfen Sie, ob die Deklaration des Attributs **carrier_id** in den privaten Bereich verschoben wurde.

2. Setzen Sie die Sichtbarkeit des Attributs **connection_id** mit einem Quickfix auf privat.

a) Gehen Sie zur Deklaration des Attributs **connection_id** in der lokalen Klasse **lcl_connection**.

b) Positionieren Sie den Cursor auf **connection_id**, und drücken Sie **Strg + 1**.

c) Doppelklicken Sie auf den Vorschlag */// Make connection_id private*.

d) Prüfen Sie, ob die Deklaration des Attributs **connection_id** in den privaten Bereich verschoben wurde.

Aufgabe 3: Instanzkonstruktor anlegen

Ersetzen Sie die öffentliche Methode **set_attributes** durch einen Instanzkonstruktor, um sicherzustellen, dass die Attribute beim Anlegen einer neuen Instanz gesetzt und danach nicht mehr geändert werden.

1. Kommentieren Sie die Definition und Implementierung der Methode **set_attributes** aus.

a) Markieren Sie alle Zeilen, die zur Anweisung `METHODS set_attributes` gehören, einschließlich der Zeile mit dem Punkt (.).

b) Drücken Sie **Strg + <**, um vor jeder ausgewählten Zeile ein Sternchen (*) hinzuzufügen.

c) Markieren Sie alle Zeilen, die zur Implementierung der Methode **set_attributes** gehören, einschließlich der entsprechenden Anweisung `ENDMETHOD.`, und drücken Sie erneut **Strg + <**.

2. Fügen Sie der lokalen Klasse **lcl_connection** per Quickfix einen Instanzkonstruktor hinzu. Stellen Sie sicher, dass der Konstruktor Importing-Parameter aufweist, die den Attributen **carrier_id** und **connection_id** entsprechen.

a) Setzen Sie den Cursor auf den Namen der Klasse, und drücken Sie **Strg + 1**.

b) Doppelklicken Sie auf */// Generate constructor*.

c) Stellen Sie im Dialogfenster sicher, dass **carrier_id** und **connection_id** ausgewählt sind, und wählen Sie */// Finish*.

3. Erweitern Sie die generierte Konstruktordefinition. Fügen Sie der Definition des **Methodenkonstruktors** die Ausnahme **CX_ABAP_INVALID_VALUE** hinzu.

a) Navigieren Sie zur generierten Definition des **Methodenkonstruktors**.

b) Passen Sie den Quelltext wie folgt an:

```
METHODS constructor
      IMPORTING
        i_connection_id TYPE /dmo/connection_id
        i_carrier_id    TYPE /dmo/carrier_id
      RAISING
        cx_ABAP_INVALID_VALUE.
```


© Copyright. Alle Rechte vorbehalten.

4. Erweitern Sie die generierte Konstruktorimplementierung. Wenn einer der Importing-Parameter initial ist, lösen Sie die Ausnahme **CX_ABAP_INVALID_VALUE** aus.
   a) Navigieren Sie von der Definition zur Implementierung des **Methodenkonstruktors**, z.B., indem Sie den Cursor auf **constructor** positionieren und **F3** drücken.
   b) Passen Sie den Quelltext wie folgt an:

```
METHOD constructor.

    IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.

    me->connection_id = i_connection_id.
    me->carrier_id    = i_carrier_id.

  ENDMETHOD.
```

5. Fügen Sie in der Konstruktorimplementierung eine Anweisung hinzu, um den Wert des statischen Attributs **conn_counter** um eins zu erhöhen. Stellen Sie sicher, dass die Anweisung nur ausgeführt wird, wenn keine Ausnahme ausgelöst wurde.
   a) Passen Sie den Quelltext wie folgt an:

```
    me->connection_id = i_connection_id.
    me->carrier_id    = i_carrier_id.

    conn_counter = conn_counter + 1.

  ENDMETHOD.
```

6. Stellen Sie sicher, dass es nicht möglich ist, den Wert von **conn_counter** von außerhalb der Klasse zu ändern.
   a) Fügen Sie `READ-ONLY` zur Deklaration des statischen Attributs **conn_counter** hinzu.
   b) Passen Sie den Quelltext wie folgt an:

```
  CLASS-DATA conn_counter TYPE i READ-ONLY.
```

Aufgabe 4: Den Konstruktor verwenden

Passen Sie die Instanziierung der Klasse **lcl_connection** an, um die Parameter des Instanzkonstruktors bereitzustellen und die Ausnahme zu behandeln.

1. Gehen Sie in der Methode **if_oo_adt_classrun~main** zum Ausdruck NEW #( ) für die erste Instanz von **lcl_connection**.
   a) Wechseln Sie zur Registerkarte *///Globale Klasse*.
   b) Suchen Sie die erste Zeile mit `connection = NEW #( ).`.
2. Geben Sie im NEW-Ausdruck die Importing-Parameter von **constructor** an.

Hinweis:
Sie können die Parameterübergabe aus dem Aufruf der Methode **set_attributes** für diese Instanz übernehmen.

© Copyright. Alle Rechte vorbehalten.

a) Passen Sie den Ausdruck NEW wie folgt an:

```
connection = NEW #(
                    i_carrier_id    = 'LH'
                    i_connection_id = '0400'
                  ).
```

3. Entfernen oder kommentieren Sie alle Aufrufe der Methode **`set_attributes`** aus.
   a) Markieren Sie alle Zeilen, die zur Anweisung `connection->set_attributes( ... ).` gehören, einschließlich der Zeile mit der schließenden Klammer und dem Punkt (.).
   b) Drücken Sie **`Strg + <`**, um vor jeder ausgewählten Zeile ein Sternchen (*) hinzuzufügen.
4. Verschieben Sie das Anlegen der Instanz in den TRY-Block der Ausnahmebehandlung.
   a) Verschieben Sie die Anweisung `TRY.` nach oben an die Stelle vor dem Anlegen der Instanz.
   b) Ihre erste Instanzerstellung sollte nun wie folgt aussehen:

```
TRY.
    connection = NEW #(
                        i_carrier_id    = 'LH'
                        i_connection_id = '0400'
                      ).

*        connection->set_attributes(
*          EXPORTING
*            i_carrier_id    = 'LH'
*            i_connection_id = '0400'
*        ).

         APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.
```

5. Wiederholen Sie die vorherigen Schritte für die anderen Instanzen Ihrer lokalen Klasse.
   a) Nach diesem Schritt sollte die Implementierung der Methode **`if_oo_adt_classrun~main`** wie folgt aussehen:

```
METHOD if_oo_adt_classrun~main.

    DATA connection TYPE REF TO lcl_connection.
    DATA connections TYPE TABLE OF REF TO lcl_connection.

* First Instance
**********************************************************************
    TRY.
    connection = NEW #(
                        i_carrier_id    = 'LH'
                        i_connection_id = '0400'
                      ).

*        connection->set_attributes(
*          EXPORTING
```


© Copyright. Alle Rechte vorbehalten.

```
*             i_carrier_id    = 'LH'
*             i_connection_id = '0400'
*         ).

        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

* Second instance
**********************************************************************
    TRY.
    connection = NEW #(
                        i_carrier_id    = 'AA'
                        i_connection_id = '0017'
                      ).
        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

* Third instance
**********************************************************************
    TRY.
    connection = NEW #(
                        i_carrier_id    = 'SQ'
                        i_connection_id = '0001'
                      ).
        APPEND connection TO connections.

      CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

* Output
**********************************************************************
    LOOP AT connections INTO connection.
      out->write( connection->get_output( ) ).
    ENDLOOP.
  ENDMETHOD.
```

6. Aktivieren Sie die Klasse. Führen Sie sie aus, und debuggen Sie die Instantiierung.
   a) Drücken Sie **Strg + F3**, um die Klasse zu aktivieren.
   b) Drücken Sie **F9**, um die Klasse auszuführen.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Erläutern Sie die Kapselung.
- Definieren und verwenden Sie Konstruktoren.

© Copyright. Alle Rechte vorbehalten.

Kapitel 3

Überprüfung des Lernerfolgs

1. In welcher Reihenfolge definieren Sie die Sichtbarkeitsbereiche einer Klasse?

   Wählen Sie die richtige Antwort.

   - ☐ **A** PUBLIC SECTION. PROTECTED SECTION. PRIVATE SECTION.
   - ☐ **B** PRIVATE SECTION. PROTECTED SECTION. PUBLIC SECTION.
   - ☐ **C** Es spielt keine Rolle.

2. Sie haben eine Klasse definiert, die Instanzattribute und statische Attribute enthält. Sie haben zudem eine Referenzvariable deklariert, aber noch keine Instanz der Klasse angelegt. Auf welche Komponenten der Klasse können Sie an dieser Stelle wie zugreifen?

   Wählen Sie die richtige Antwort.

   - ☐ **A** Instanzkomponenten über die Referenzvariable
   - ☐ **B** Instanzkomponenten über den Namen der Klasse
   - ☐ **C** Statische Komponenten über die Referenzvariable
   - ☐ **D** Statische Komponenten über den Namen der Klasse.

3. Sie haben zwei Referenzvariablen „ref1“ und „ref2“ deklariert. „ref1“ zeigt auf ein Objekt. Nun führen Sie die Anweisung „ref2 = ref1“ aus. Was passiert?

   Wählen Sie die richtige Antwort.

   - ☐ **A** Das ABAP-System legt eine Kopie des Objekts an, auf das „ref1“ zeigt, und ordnet seine Adresse „ref2“ zu. Es gibt nun zwei Objekte.
   - ☐ **B** Das ABAP-System ordnet die Adresse des Objekts zu, auf das „ref1“ zeigt, der Referenzvariable „ref2“ zu. Es gibt nur ein Objekt.

© Copyright. Alle Rechte vorbehalten.

4. Eine Klasse „my_class“ enthält die öffentliche statische Methode „my_method“. Wie lautet der korrekte Quelltext zum Aufrufen dieser Methode?

   Wählen Sie die richtige Antwort.

   - ☐ A my_class->my_method().
   - ☐ B my_class=>my_method().
   - ☐ C my_class->my_method( ).
   - ☐ D my_class=>my_method( ).

5. Eine funktionale Methode muss Folgendes aufweisen:

   Wählen Sie die richtige Antwort.

   - ☐ A Genau einen Importing-Parameter
   - ☐ B Genau einen Exporting-Parameter
   - ☐ C Keine Changing-Parameter
   - ☐ D Genau einen Returning-Parameter

6. Ihre Klasse enthält ein öffentliches Instanzattribut „attr“. Wie können Sie sicherstellen, dass dessen Wert nur innerhalb der Klasse geändert werden kann?

   Wählen Sie die richtigen Antworten.

   - ☐ A Machen Sie es zu einem privaten Attribut.
   - ☐ B Konvertieren Sie es in eine Konstante.
   - ☐ C Belassen Sie es im öffentlichen Bereich, verwenden Sie jedoch den Zusatz READ-ONLY.
   - ☐ D Verschieben Sie sie in eine andere lokale Klasse.

7. Welche der folgenden Signaturelemente darf ein Instanzkonstruktor enthalten?

   Wählen Sie die richtigen Antworten.

   - ☐ A Importing-Parameter
   - ☐ B Exporting-Parameter
   - ☐ C Changing-Parameter
   - ☐ D Ausnahmen

© Copyright. Alle Rechte vorbehalten.

8. Der statische Konstruktor wird bei der ersten Adressierung der Klasse ausgeführt. Wann könnte das sein?

   Wählen Sie die richtigen Antworten.

   - ☐ A Am Anfang des Programms
   - ☐ B Wenn Sie eine statische Methode der Klasse aufrufen
   - ☐ C Wenn Sie die Klasse instanziieren

© Copyright. Alle Rechte vorbehalten.

Kapitel 3

Überprüfung des Lernerfolgs - Antworten

1. In welcher Reihenfolge definieren Sie die Sichtbarkeitsbereiche einer Klasse?

   Wählen Sie die richtige Antwort.

   - [X] A PUBLIC SECTION. PROTECTED SECTION. PRIVATE SECTION.
   - [ ] B PRIVATE SECTION. PROTECTED SECTION. PUBLIC SECTION.
   - [ ] C Es spielt keine Rolle.

   Das ist richtig.

2. Sie haben eine Klasse definiert, die Instanzattribute und statische Attribute enthält. Sie haben zudem eine Referenzvariable deklariert, aber noch keine Instanz der Klasse angelegt. Auf welche Komponenten der Klasse können Sie an dieser Stelle wie zugreifen?

   Wählen Sie die richtige Antwort.

   - [ ] A Instanzkomponenten über die Referenzvariable
   - [ ] B Instanzkomponenten über den Namen der Klasse
   - [ ] C Statische Komponenten über die Referenzvariable
   - [X] D Statische Komponenten über den Namen der Klasse.

   Das ist richtig.

3. Sie haben zwei Referenzvariablen „ref1“ und „ref2“ deklariert. „ref1“ zeigt auf ein Objekt. Nun führen Sie die Anweisung „ref2 = ref1“ aus. Was passiert?

   Wählen Sie die richtige Antwort.

   - [ ] A Das ABAP-System legt eine Kopie des Objekts an, auf das „ref1“ zeigt, und ordnet seine Adresse „ref2“ zu. Es gibt nun zwei Objekte.
   - [X] B Das ABAP-System ordnet die Adresse des Objekts zu, auf das „ref1“ zeigt, der Referenzvariable „ref2“ zu. Es gibt nur ein Objekt.

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

4. Eine Klasse „my_class“ enthält die öffentliche statische Methode „my_method“. Wie lautet der korrekte Quelltext zum Aufrufen dieser Methode?

   Wählen Sie die richtige Antwort.

   - [ ] **A** my_class->my_method().
   - [ ] **B** my_class=>my_method().
   - [ ] **C** my_class->my_method( ).
   - [X] **D** my_class=>my_method( ).

   Richtig!

5. Eine funktionale Methode muss Folgendes aufweisen:

   Wählen Sie die richtige Antwort.

   - [ ] **A** Genau einen Importing-Parameter
   - [ ] **B** Genau einen Exporting-Parameter
   - [ ] **C** Keine Changing-Parameter
   - [X] **D** Genau einen Returning-Parameter

   Das ist richtig.

6. Ihre Klasse enthält ein öffentliches Instanzattribut „attr“. Wie können Sie sicherstellen, dass dessen Wert nur innerhalb der Klasse geändert werden kann?

   Wählen Sie die richtigen Antworten.

   - [X] **A** Machen Sie es zu einem privaten Attribut.
   - [ ] **B** Konvertieren Sie es in eine Konstante.
   - [X] **C** Belassen Sie es im öffentlichen Bereich, verwenden Sie jedoch den Zusatz READ-ONLY.
   - [ ] **D** Verschieben Sie sie in eine andere lokale Klasse.

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

7. Welche der folgenden Signaturelemente darf ein Instanzkonstruktor enthalten?

   Wählen Sie die richtigen Antworten.

   - [X] A Importing-Parameter
   - [ ] B Exporting-Parameter
   - [ ] C Changing-Parameter
   - [X] D Ausnahmen

   Das ist richtig.

8. Der statische Konstruktor wird bei der ersten Adressierung der Klasse ausgeführt. Wann könnte das sein?

   Wählen Sie die richtigen Antworten.

   - [ ] A Am Anfang des Programms
   - [X] B Wenn Sie eine statische Methode der Klasse aufrufen
   - [X] C Wenn Sie die Klasse instanziieren

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

KAPITEL 4 Daten aus der Datenbank lesen

Lektion 1

Eine Tabellendefinition untersuchen 199

Lektion 2

Grundlegende SELECT-Anweisungen implementieren 203

Übung 12 : Daten aus einer Datenbanktabelle auslesen 211

Lektion 3

Mit CDS-Views arbeiten 219

Übung 13 : Eine CDS-View-Entität analysieren und verwenden 225

LERNZIELE

- Untersuchen Sie eine Tabellendefinition.
- Die grundlegenden Funktionen von ABAP SQL beschreiben.
- Einzelwerte von der Datenbank lesen.
- Analysieren Sie eine CDS-View-Definition.
- Daten über eine CDS-View lesen.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 4
Lektion 1

Eine Tabellendefinition untersuchen

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Untersuchen Sie eine Tabellendefinition.

Datenbanktabellendefinitionen

Jedes ABAP-System wird auf einem relationalen Datenbankmanagementsystem (DBMS) ausgeführt. Ursprünglich unterstützte ABAP verschiedene Datenbankmanagementsysteme verschiedener Anbieter. Daher erfolgt die Definition von Datenbankobjekten wie Tabellen und Views datenbankunabhängig.

> Notiz:
> Neuere ABAP-Releases unterstützen lediglich SAP HANA als Datenbank. Die ABAP-Umgebung auf SAP BTP wird immer auf SAP HANA ausgeführt.

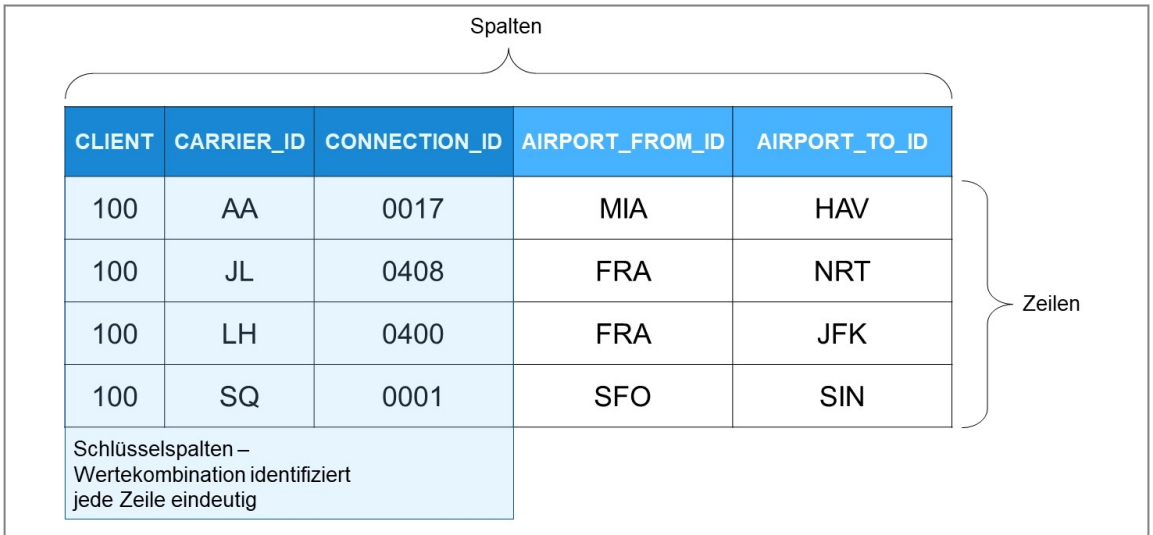


Abbildung 105: Datenbanktabellendefinition

In einer relationalen Datenbank werden die Informationen in zweidimensionalen Tabellen abgelegt. Dabei steht jede Zeile für einen Datensatz, dessen Inhalt auf die Spalten aufgeteilt ist. Die Datenbank wird als relationale Datenbank bezeichnet, weil auch die Tabellen zueinander in Beziehung stehen. Die Tabelle in der Abbildung enthält beispielsweise ein Fluggesellschaftskürzel. In der Datenbanktabelle bestünde somit eine Beziehung zu einer weiteren Tabelle, welche die Information enthält, dass AA für American Airlines steht, JL für Japanese Airline, LH für Lufthansa usw.

© Copyright. Alle Rechte vorbehalten.

Eine Folge von Spalten am Anfang jeder Datenbanktabelle bildet ihren Schlüssel. Die Wertekombination des Schlüssels gewährleistet die eindeutige Identifizierung jeder Tabellenzeile.

In SAP-Systemen sind Datenbanktabellendefinitionen Entwicklungsobjekte und somit mandantenunabhängig. Die große Mehrheit der Tabellen enthält jedoch Geschäftsdaten, die ihrerseits mandantenabhängig sind. Um die Daten getrennt zu halten, verfügen mandantenabhängige Tabellen über ein Mandantenfeld (oft CLIENT oder MANDT genannt) als erstes Schlüsselfeld. Die Datenbank der SAP greift mit ABAP SQL auf Anweisungen zu, um sicherzustellen, dass eine Anweisung nur Daten aus dem aktuellen Mandanten ändert.

Datenbanktabellendefinitionen

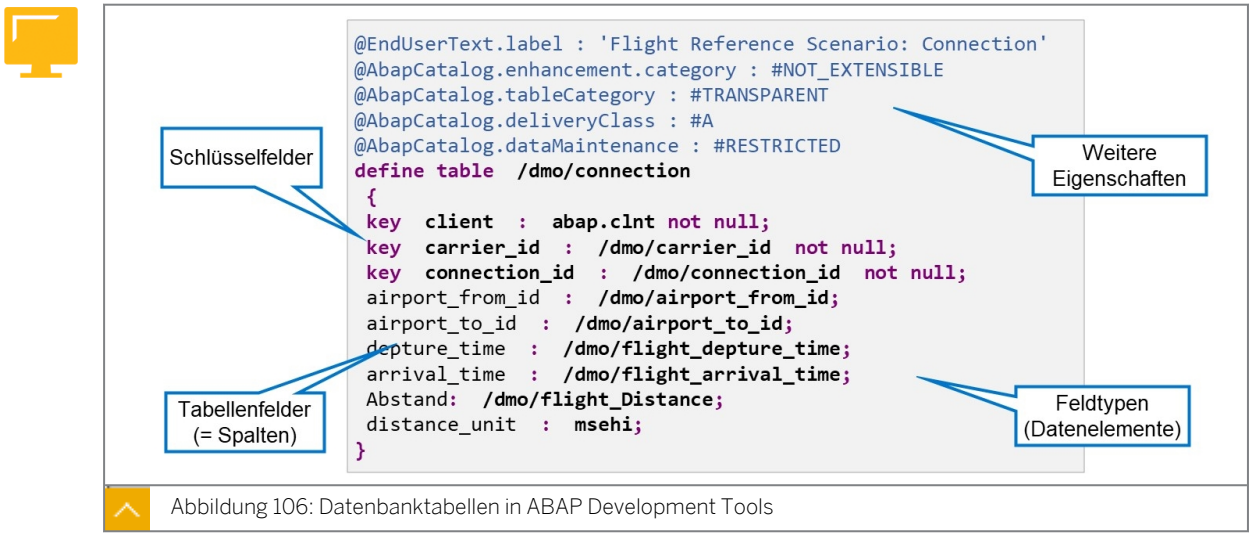


Abbildung 106: Datenbanktabellen in ABAP Development Tools

ADT stellt einen dedizierten Editor für Datenbanktabellendefinitionen bereit.

Sehen wir uns die verschiedenen Teile der Tabellendefinition an.

Der Hauptteil der Definition besteht aus einer DEFINE TABLE-Anweisung mit dem Namen der Tabelle. Danach folgt die Liste der Tabellenfelder in einem Paar geschweifter Klammern ({ , }); jedes Feld mit einem Typ, der durch ein Datenelement beschrieben wird. Der Schlüssel, der zur eindeutigen Identifizierung jedes einzelnen Datensatzes dient, wird durch eine Untermenge von Feldern am Anfang der Liste definiert.

Mehrere zusätzliche Quelltextzeilen vor der Anweisung DEFINE TABLE geben zusätzliche Eigenschaften der Datenbanktabelle an, darunter einen Bezeichner.

© Copyright. Alle Rechte vorbehalten.

Datenvorschau für Datenbanktabellen

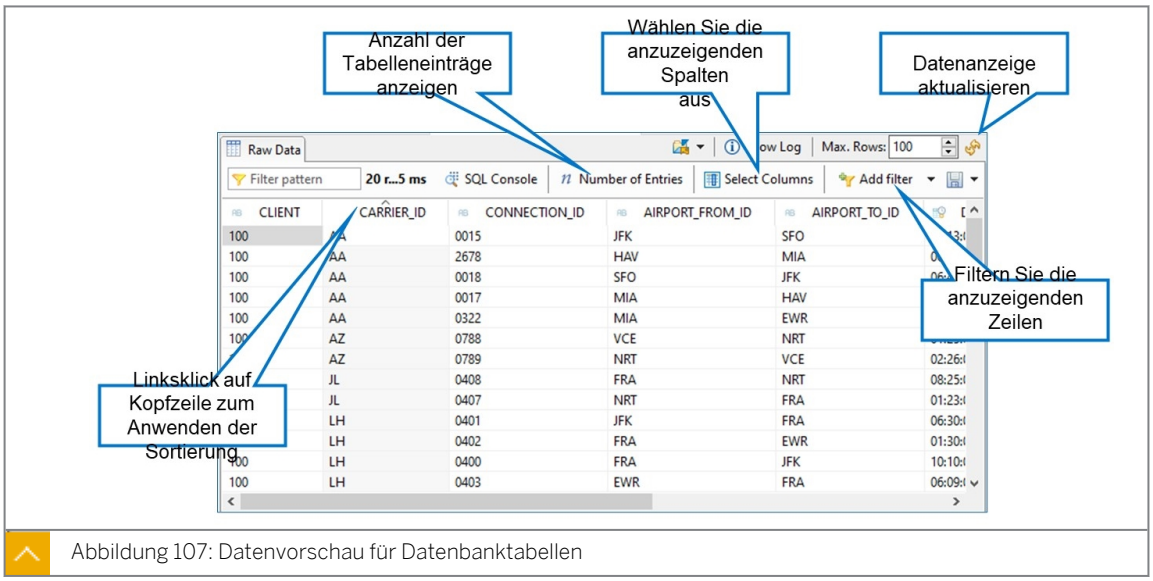


Abbildung 107: Datenvorschau für Datenbanktabellen

Mit dem Werkzeug Data Preview für die Datenvorschau können Sie den Inhalt einer Datenbanktabelle anzeigen und analysieren. Um die Datenvorschau für eine bestimmte Tabelle zu öffnen, klicken Sie mit der rechten Maustaste auf eine beliebige Stelle in der Tabellendefinition, und wählen Sie *///Open With → Data Preview*. Alternativ können Sie den Cursor an einer beliebigen Stelle in der Definition der Datenbanktabelle positionieren und `F8` drücken.

Das Werkzeug zeigt die in der Datenbanktabelle gespeicherten Daten an. Einige der wichtigsten Funktionen lauten wie folgt:

///Select Columns

Bei Tabellen mit vielen Spalten können Sie die Spalten anpassen, die Sie anzeigen möchten.

///Change Column Sequence

Sie können die Spaltenreihenfolge ändern, indem Sie eine Spaltenüberschrift mit Drag&Drop an eine andere Position ziehen.

///Apply Sorting

Sie können eine Sortierung anwenden, indem Sie mit der linken Maustaste auf eine Spaltenüberschrift klicken.

Add Filter

Bei Tabellen mit vielen Zeilen können Sie die Daten filtern, indem Sie *///Add Filter* wählen.

///Refresh Display

Wählen Sie die Drucktaste *///Refresh*, um die Daten erneut zu laden.

///Number of Entries

Wählen Sie *///Number of Entries*, wenn Sie nur an der Anzahl der Zeilen interessiert sind, die Ihre Filterbedingungen erfüllen. Ohne Filter wird die Gesamtzahl der Einträge angezeigt.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Untersuchen Sie eine Tabellendefinition.

© Copyright. Alle Rechte vorbehalten.

Kapitel 4
Lektion 2

Grundlegende SELECT-Anweisungen implementieren

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Die grundlegenden Funktionen von ABAP SQL beschreiben.
- Einzelwerte von der Datenbank lesen.

Grundlegende ABAP-SQL-Syntax

Structured Query Language (SQL)

Structured Query Language (SQL)
Besteht aus drei Teilsprachen:

Data Manipulation Language (DML)
Anweisungen zum Lesen, Anlegen, Aktualisieren oder Löschen von Datensätzen in Datenbanktabellen

Data Definition Language (DDL)
Anweisungen zum Anlegen, Anpassen oder Löschen von Definitionen von Datenbanktabellen und anderen Datenbankartefakten

Data Control Language (DCL)
Anweisungen zum Regulieren von Zugriff auf Datenbanktabellen auf Ebene der Datenbank

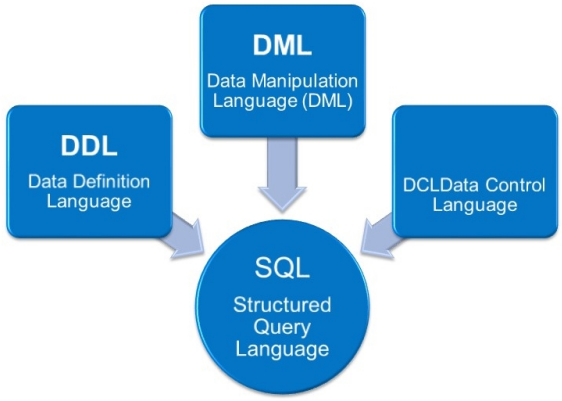


Abbildung 108: Structured Query Language (SQL)

Alle relationalen Datenbanksysteme nutzen eine Variante der Structured Query Language (SQL), um Ihnen zu ermöglichen, mit ihnen zu arbeiten. Standard-SQL besteht aus drei Hauptkomponenten:

Data Manipulation Language (DML)

DML enthält die Anweisungen, die Sie für das Arbeiten mit den Informationen in Datenbanktabellen benötigen. Dabei handelt es sich im Einzelnen um die Anweisungen INSERT zum Hinzufügen neuer Datensätze, SELECT zum Lesen von Daten, UPDATE zum Ändern vorhandener Datensätze und DELETE zum Löschen von Tabelleninhalten. Diese Anweisungen werden in ABAP in Form von ABAP SQL abgebildet.

Data Definition Language (DDL)

© Copyright. Alle Rechte vorbehalten.

Die Data Definition Language stellt Befehle zum Anlegen und Löschen von Datenbanktabellen und weiteren Datenbankartefakten wie Schemata, Tabellenindizes und Views bereit. In einem SAP-System führen Sie diese Aufgaben mit dedizierten ADT-Editoren und nicht mit ABAP-Programmen durch. Ein Beispiel ist der Editor für Datenbanktabellendefinitionen, den wir zuvor gesehen haben.

Data Control Language (DCL)

Mit der Data Control Language schränken Sie in SQL den Datenbankzugriff eines bestimmten Benutzers ein. Sie wird in ABAP nicht in ihrer klassischen Form eingesetzt, da die Benutzer auf Datenbankebene nicht eins zu eins den Endbenutzern entsprechen. Folglich verfügt ABAP über ein eigenes Berechtigungskonzept.

ABAP-SQL-Architektur

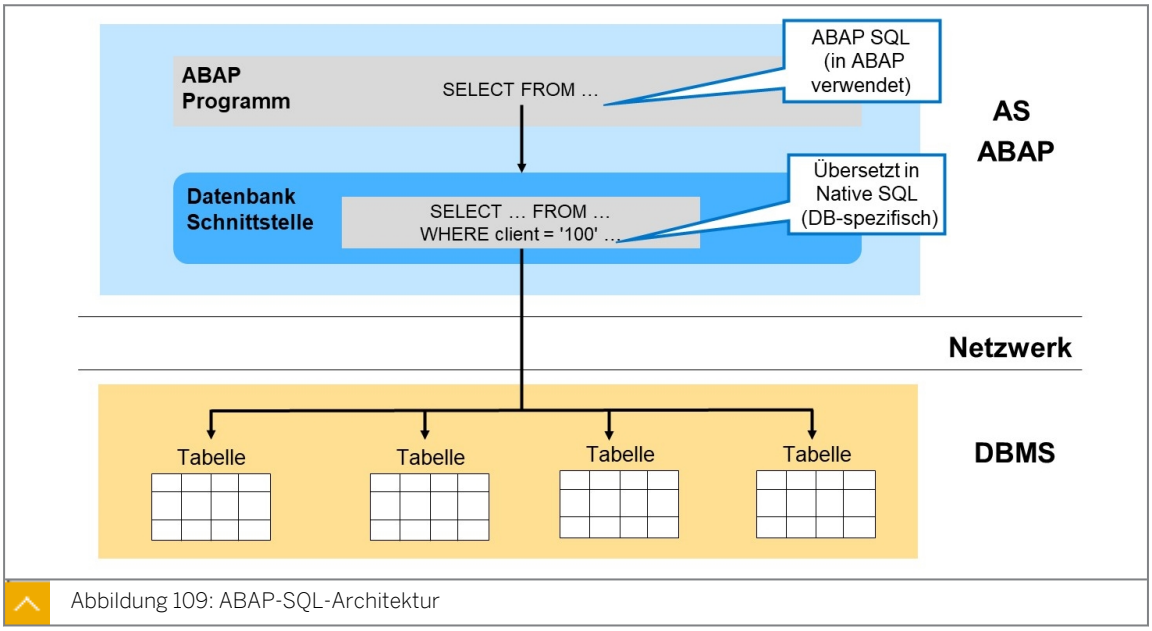


Abbildung 109: ABAP-SQL-Architektur

Früher mussten SAP-Systeme eine ganze Reihe von Datenbankplattformen unterstützen, und jede Plattform implementierte den SQL-Standard leicht unterschiedlich. Das bedeutete, dass jede Plattform leicht unterschiedliche Befehle für eine bestimmte Aufgabe benötigte. Um zu vermeiden, dass der ABAP-Quelltext datenbankspezifisch ist, hat die SAP ABAP SQL (oder Open SQL, wie es ursprünglich genannt wurde) erfunden.

> Notiz:
> Die Namensänderung von Open SQL zu ABAP SQL zeigt auch, dass ABAP ab Release 7.53 nur SAP HANA als DBMS unterstützt.

ABAP SQL ist eine abstrakte Menge von SQL-Befehlen, die auf ABAP-Ebene implementiert und in die ABAP-Sprache integriert sind. Zur Laufzeit wird ABAP SQL in eine Variante von SQL übersetzt, die die Datenbank versteht. Diese Variante wird Native SQL genannt, um sie von ABAP SQL, der in ABAP integrierten SQL-Variante, zu unterscheiden. Die Übersetzung von ABAP SQL in Native SQL findet in der Datenbankschnittstelle statt, einer Komponente des ABAP-Systems, die aus einem allgemeinen Teil und einer datenbankspezifischen Bibliothek besteht.

Obwohl neuere ABAP-Releases nur SAP HANA als DBMS unterstützen, hat SAP das Konzept von ABAP SQL und der Datenbankschnittstelle beibehalten. Dafür gibt es folgende Gründe:

© Copyright. Alle Rechte vorbehalten.

Kompatibilität der Architektur

ABAP SQL und die Datenbankschnittstelle sind integraler Bestandteil der Systemarchitektur.

Kompatibilität des Quelltextes

ABAP-SQL-Coding aus früheren SAP-Produkten (einschließlich kundenspezifischer Entwicklungen) sollte in den modernen, nur auf SAP HANA basierenden ABAP-Umgebungen ohne Nebeneffekte ausgeführt werden.

Aufgaben der Datenbankschnittstelle

Die Datenbankschnittstelle übersetzt nicht nur Anweisungen, sondern ist auch für ABAP-spezifische Aufgaben wie z.B. die automatische Mandantenbehandlung zuständig.

Einzelsatz aus der Datenbank auslesen

Die SELECT-Anweisung in ABAP

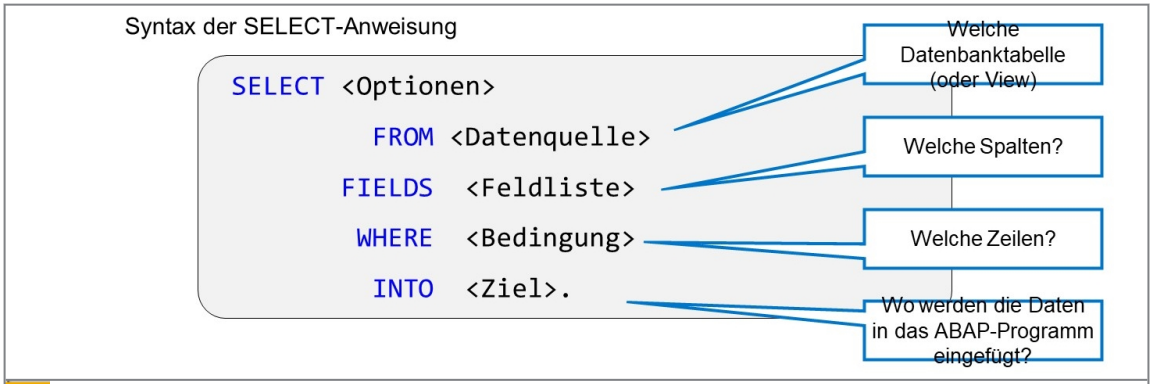


Abbildung 110: Die SELECT-Anweisung in ABAP

Um Daten aus der Datenbank zu lesen, verwenden Sie die SELECT-Anweisung.

Wenn Sie in ABAP SQL eine SELECT-Anweisung schreiben, vergleicht die Syntaxprüfung das, was Sie geschrieben haben, mit der Definition der Tabellen und Views. Wenn Sie versuchen, nicht vorhandene Tabellen, Views oder Felder zu adressieren, führt dies zu einem Syntaxfehler.

Die Grundsyntax der SELECT-Anweisung enthält mehrere Abschnitte – so genannte Klauseln – und folgt stets dem in der Abbildung „Die SELECT-Anweisung in ABAP“ gezeigten Muster. Die wichtigsten Klauseln einer SELECT-Anweisung lauten wie folgt:

FROM

In der FROM-Klausel der SELECT-Anweisung geben Sie die Datenquelle an, aus der Sie lesen möchten. Dabei kann es sich entweder um eine Datenbanktabelle oder um eine View handeln. Spezielle SQL-Methoden ermöglichen es Ihnen, Daten aus mehreren Quellen in derselben SELECT-Anweisung zu kombinieren.

FELDER

In der FIELDS-Klausel der SELECT-Anweisung listen Sie die auszulesenden Spalten der Datenbanktabelle auf. Die Spalten in der Liste müssen durch Kommata getrennt sein. Wenn Sie die komplette Tabellenzeile lesen möchten, können Sie statt einer Spaltenliste auch „FIELDS *“ angeben. Beachten Sie jedoch, dass dies zu einer erheblich größere Arbeitsbelastung für die Datenbank führen könnte, als nur die benötigten Spalten zu lesen.

© Copyright. Alle Rechte vorbehalten.

WHERE

In der WHERE-Klausel können Sie eine Bedingung angeben, die beschreibt, welche Zeilen der Tabelle gelesen werden sollen. Die Bedingung „WHERE carrier_id = 'LH'“ bedeutet beispielsweise, dass nur solche Zeilen gelesen werden, deren Spalte CARRIER_ID den Wert LH enthält.

Die WHERE-Klausel kann mehrere Bedingungen enthalten, die über die Operatoren AND und OR miteinander verknüpft werden. Beispielsweise würden „WHERE carrier_id = 'LH'“ und „connection_id = '0400'“ die Daten der Flugverbindung LH 0400 zurückgeben. Über den Operator NOT können Sie Bedingungen auch negieren. Die WHERE-Klausel ist die einzige optionale Klausel. Beachten Sie jedoch, dass Sie ohne WHERE-Klausel alle Daten aus der Tabelle auslesen bzw. – wenn die Tabelle ein Mandantenfeld aufweist – alle Daten, die zum Anmeldemandanten des Benutzers gehören. SELECT-Anweisungen ohne WHERE-Klausel können zu schwerwiegenden Performanceproblemen führen und sollten vermieden werden.

INTO

Mit der INTO-Klausel werden die Variablen im ABAP-Programm festgelegt, in die die Daten abgelegt werden sollen. Hierbei handelt es sich normalerweise um eine Struktur oder um eine interne Tabelle. Die Abfolge der Komponenten sollte mit der Reihenfolge der Spaltenliste in der FIELDS-Klausel identisch sein.

Notiz:
In ABAP werden Sie auch anderen Formen von SQL-Syntax begegnen. Diese sind älter und wurden aus Kompatibilitätsgründen beibehalten. Sie sollten sich angewöhnen, die moderne Syntax zu verwenden, da sie weitaus mehr Funktionen und Features bietet als die alte Form.

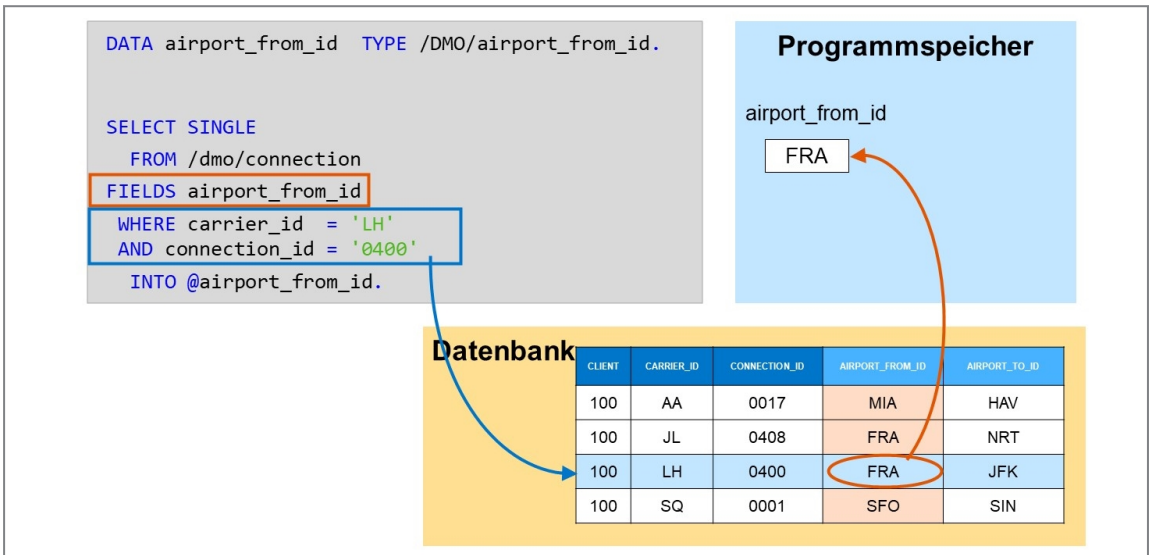


Abbildung 111: Beispiel 1: Einzelfeld eines Einzelsatzes lesen

Die Abbildung „Beispiel 1: Einzelfeld eines Einzelsatzes auslesen“ veranschaulicht eine SELECT-Anweisung, die einen Einzelwert aus der Datenbank liest. Die FROM-Klausel besagt, dass die Anweisung aus der Datenbanktabelle /DMO/CONNECTION liest. Die Option SINGLE hinter dem Schlüsselwort SELECT zeigt an, dass nur eine Zeile (ein Einzelsatz) gelesen wird. Diese Zeile wird in der WHERE-Klausel identifiziert, indem Schlüsselfilterwerte für die

© Copyright. Alle Rechte vorbehalten.

Schlüsselfelder „carrier_id“ und „connection_id“ angegeben werden. Beachten Sie, dass die Datenbankschnittstelle einen Filter für den verbleibenden Schlüsselfeldmandanten hinzufügt.

Die FIELDS-Klausel listet nur eine Spalte der Tabelle auf: Spalte AIRPORT_FROM_ID.

Die INTO-Klausel muss mit dem Rest der Anweisung übereinstimmen. In unserem Beispiel wird hiermit die Variable „airport_from_id“ als Zielobjekt angegeben. Es handelt sich um ein skalares Datenobjekt vom gleichen Typ wie das Tabellenfeld „airport_from_id“.

Notiz:
Das At-Zeichen (@) identifiziert „airport_from_id“ als Name eines ABAP-Datenobjekts. Es ist für alle Variablen und Konstanten obligatorisch, die Sie in einer ABAP-SQL-Anweisung verwenden. Es wird benötigt, um Mehrdeutigkeiten zu vermeiden, wenn z.B. ein Datenobjekt und ein Tabellenfeld den gleichen Namen aufweisen.

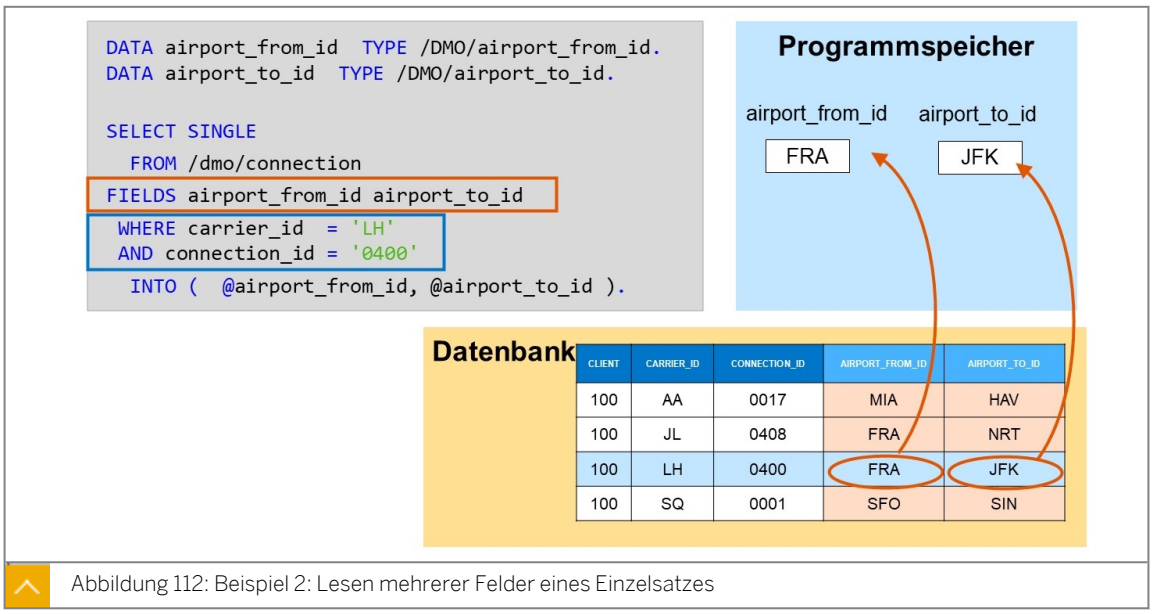


Abbildung 112: Beispiel 2: Lesen mehrerer Felder eines Einzelsatzes

Die Abbildung „Beispiel 2: Mehrere Felder eines Einzelsatzes auslesen“ zeigt eine SELECT-Anweisung, die zwei Werte aus demselben Satz der Datenbank liest.

Dieses Mal listet die FIELDS-Klausel zwei Spalten der Tabelle auf: die Spalte AIRPORT_FROM_ID und die Spalte AIRPORT_TO_ID.

Für eine Übereinstimmung gibt die INTO-Klausel die Variablen „airport_from_id“ und „airport_to_id“ als Zielobjekte an. Sie werden durch ein Komma getrennt und in Klammern eingeschlossen, um deutlich zu machen, dass sie zusammen das Ziel der SELECT-Anweisung bilden.

© Copyright. Alle Rechte vorbehalten.

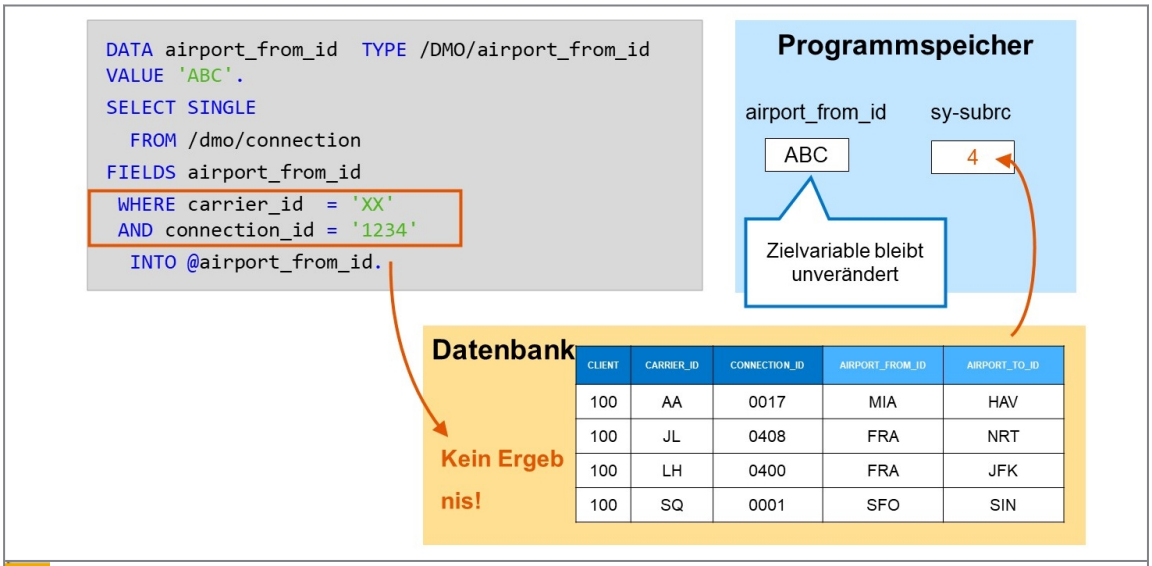


Abbildung 113: Was passiert, wenn es kein Ergebnis gibt?

Bei der Implementierung einer SELECT-Anweisung muss immer berücksichtigt werden, dass es zu keinem Ergebnis kommen kann, entweder weil die Datenbanktabelle gar keine Daten enthält oder weil sie keine Zeilen enthält, die die Bedingungen in der WHERE-Klausel erfüllen. Im Beispiel aus der obigen Abbildung enthält die Datenbanktabelle keine Zeile mit „carrier_id = 'XX'“ und „connection_id = '1234'“.

ABAP SQL verwendet das Systemfeld SY-SUBRC, um eine erfolgreiche oder nicht erfolgreiche Ausführung einer Anweisung anzuzeigen. Das Systemfeld SY-SUBRC ist vom Typ Integer. Der Initialwert 0 zeigt immer eine erfolgreiche Ausführung an. Wenn SY-SUBRC nach einer SELECT-Anweisung den Wert 4 enthält, bedeutet dies, dass die Datenbank ein leeres Ergebnis zurückgegeben hat.

Notiz:
Wenn die Datenbank ein leeres Ergebnis zurückgibt, fasst ABAP SQL die Zielvariable hinter INTO NICHT an!

Insbesondere wird die Zielvariable beim Auftreten eines Fehlers nicht initialisiert.

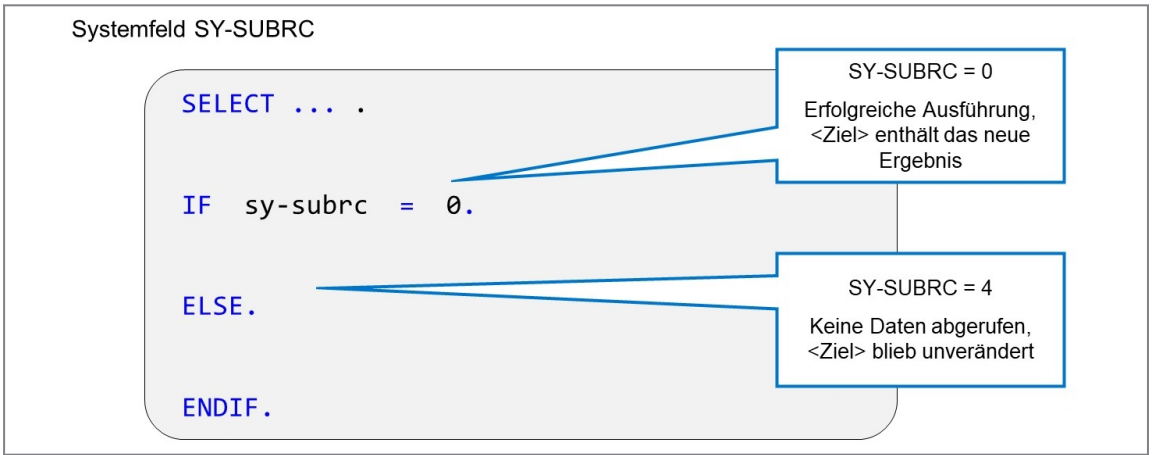


Abbildung 114: Behandlung leerer Ergebnisse

© Copyright. Alle Rechte vorbehalten.

Es wird empfohlen, den Inhalt des Systemfelds „sy-subrc“ unmittelbar nach jeder SELECT-Anweisung auszuwerten.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 4
Übung 12

Daten aus einer Datenbanktabelle auslesen

In dieser Übung erweitern Sie Ihre lokale Klasse um Attribute für den Abflughafen und den Zielflughafen und lesen die Werte für diese Attribute aus einer Datenbanktabelle aus.

Vorlage:

/LRN/CL_S4D400_CLS_CONSTRUCTOR (globale Klasse)

Lösung:

/LRN/CL_S4D400_DBS_SELECT (globale Klasse)

Aufgabe 1: Vorlage kopieren
Kopieren Sie die Vorlageklasse. Alternativ können Sie Ihre Lösung aus der vorherigen Übung kopieren.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_CLS_CONSTRUCTOR`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_SELECT`**, wobei ## für Ihre Gruppennummer steht).

Aufgabe 2: Zusätzliche Attribute deklarieren
Erweitern Sie die lokale Klasse **`lcl_connection`** um die privaten Instanzattribute **`airport_from_id`** und **`airport_to_id`**. Fügen Sie der Implementierung der Methode **`get_output`** Ausgabedaten für die neuen Attribute hinzu.

Notiz:
Die neuen Attribute sind noch nicht gefüllt. Um sie zu füllen, fügen wir dem **`Konstruktor`** in einer der nächsten Aufgaben dieser Übung eine SELECT-Anweisung hinzu.

1. Wechseln Sie in die lokale Klasse **`lcl_connection`**.
2. Fügen Sie der Klassendefinition folgende **private** Attribute hinzu:

   Tabelle 2: Attribute

Attributname	Umfang	Datentyp
`airport_from_id`	Instanz	/DMO/AIRPORT_FROM_ID
`airport_to_id`	Instanz	/DMO/AIRPORT_TO_ID

3. Erweitern Sie die Implementierung der Methode **`get_output`**. Hängen Sie weitere Zeichenketten-Templates an den Returning-Parameter **`r_output`** an. Betten Sie die neuen Attribute als Ausdrücke in die Zeichenketten-Templates ein.
4. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe.

© Copyright. Alle Rechte vorbehalten.

Aufgabe 3: Die Datenbanktabelle analysieren
Analysieren Sie die Definition der Datenbanktabelle **`/DMO/CONNECTION`**.

1. Öffnen Sie das Entwicklungsobjekt, das die Definition der Datenbanktabelle **`/DMO/CONNECTION`** enthält.
2. Öffnen Sie die *Elementinformationen* (F2) für die Datenbanktabelle /DMO/CONNECTION.

   Welche Felder weisen die Beschreibung *///Flight Reference Scenario: From Airport* und *///Flight Reference Scenario: To Airport* auf?

Aufgabe 4: Daten aus der Datenbank lesen
Implementieren Sie im **`Methodenkonstruktor`** der lokalen Klasse **`lcl_connection`** eine SELECT-Anweisung, die Werte für die neuen Attribute aus der Datenbanktabelle **`/DMO/CONNECTION`** liest.

1. Kehren Sie zur lokalen Klasse **`lcl_connection`** in Ihrer globalen Klasse zurück.
2. Navigieren Sie zur Implementierung der Methode **`constructor`**.
3. Fügen Sie nach der Anweisung `ENDIF.` eine SELECT-Anweisung hinzu, die einen Einzelsatz aus der Datenbanktabelle **`/DMO/CONNECTION`** liest.
4. Implementieren Sie die FIELDS-Klausel. Lesen Sie die Tabellenfelder **`airport_from_id`** und **`airport_to_id`**.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**`Strg + Leertaste`**), um die Feldnamen einzugeben.

5. Implementieren Sie die WHERE-Bedingung. Schränken Sie alle Schlüsselfelder der Datenbanktabelle (mit Ausnahme des Mandantenfeldes) mit den Werten der Importing-Parameter **`i_carrier_id`** und **`i_connection_id`** ein. Vergessen Sie nicht, als Escape-Zeichen für die Parameter das Präfix @ voranzustellen.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**`Strg + Leertaste`**), um die Element- und Parameternamen einzugeben.

Warum ist es nicht notwendig, das Mandantenfeld in der WHERE -Bedingung anzugeben?

© Copyright. Alle Rechte vorbehalten.

6. Implementieren Sie die INTO-Klausel. Speichern Sie das SELECT-Ergebnis in den Attributen **`airport_from_id`** und **`airport_to_id`**. Vergessen Sie nicht, als Escape-Zeichen für die Attribute das Präfix @ voranzustellen.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**`Strg + Leertaste`**), um die Attributnamen einzugeben.

7. Implementieren Sie die Fehlerbehandlung nach der SELECT-Anweisung. Überprüfen Sie den Inhalt des Systemfeldes **`sy-subrc`**. Wenn er ungleich Null ist, muss die Ausnahme **`CX_ABAP_INVALID_VALUE`** ausgelöst werden.

8. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe. Prüfen Sie, ob die Ausgabe für die neuen Attribute Daten anzeigt.

© Copyright. Alle Rechte vorbehalten.

Kapitel 4
Lösung 12

Daten aus einer Datenbanktabelle auslesen

In dieser Übung erweitern Sie Ihre lokale Klasse um Attribute für den Abflughafen und den Zielflughafen und lesen die Werte für diese Attribute aus einer Datenbanktabelle aus.

Vorlage:

/LRN/CL_S4D400_CLS_CONSTRUCTOR (globale Klasse)

Lösung:

/LRN/CL_S4D400_DBS_SELECT (globale Klasse)

Aufgabe 1: Vorlage kopieren
Kopieren Sie die Vorlageklasse. Alternativ können Sie Ihre Lösung aus der vorherigen Übung kopieren.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_CLS_CONSTRUCTOR`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_SELECT`**, wobei ## für Ihre Gruppennummer steht).
   a) Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_CLS_CONSTRUCTOR`**.
   b) Verknüpfen Sie die View *Project Explorer* mit dem Editor.
   c) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Klasse **`/LRN/CL_S4D400_CLS_CONSTRUCTOR`**, um das Kontextmenü zu öffnen.
   d) Wählen Sie im Kontextmenü *Duplicate ...*
   e) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_SELECT`** ein, wobei ## für Ihre Gruppennummer steht.
   f) Passen Sie die Beschreibung an, und wählen Sie *Next*.
   g) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

Aufgabe 2: Zusätzliche Attribute deklarieren
Erweitern Sie die lokale Klasse **`lcl_connection`** um die privaten Instanzattribute **`airport_from_id`** und **`airport_to_id`**. Fügen Sie der Implementierung der Methode **`get_output`** Ausgabedaten für die neuen Attribute hinzu.

Notiz:
Die neuen Attribute sind noch nicht gefüllt. Um sie zu füllen, fügen wir dem **`Konstruktor`** in einer der nächsten Aufgaben dieser Übung eine SELECT-Anweisung hinzu.

1. Wechseln Sie in die lokale Klasse **`lcl_connection`**.
   a) Wählen Sie in der globalen Klasse *Local Types*.

© Copyright. Alle Rechte vorbehalten.

2. Fügen Sie der Klassendefinition folgende **private** Attribute hinzu:

   Tabelle 2: Attribute

Attributname	Umfang	Datentyp
airport_from_id	Instanz	/DMO/AIRPORT_FROM_ID
airport_to_id	Instanz	/DMO/AIRPORT_TO_ID

   a) Passen Sie den Quelltext wie folgt an:

```
  PRIVATE SECTION.
    DATA carrier_id    TYPE /dmo/carrier_id.
    DATA connection_id TYPE /dmo/connection_id.

    DATA airport_from_id TYPE /dmo/airport_from_id.
    DATA airport_to_id   TYPE /dmo/airport_to_id.

ENDCLASS.
```

3. Erweitern Sie die Implementierung der Methode **get_output**. Hängen Sie weitere Zeichenketten-Templates an den Returning-Parameter **r_output** an. Betten Sie die neuen Attribute als Ausdrücke in die Zeichenketten-Templates ein.

   a) Navigieren Sie zur Implementierung der Methode **get_output**.

   b) Passen Sie den Quelltext wie folgt an:

```
    APPEND |--------------------------------|             TO r_output.
    APPEND |Carrier:     { carrier_id      }|             TO r_output.
    APPEND |Connection:  { connection_id   }|             TO r_output.
    APPEND |Departure:   { airport_from_id }|             TO r_output.
    APPEND |Destination: { airport_to_id   }|             TO r_output.
```

4. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe.

   a) Drücken Sie **Strg + F3**, um die Klasse zu aktivieren.

   b) Drücken Sie **F9**, um die Klasse auszuführen.

Aufgabe 3: Die Datenbanktabelle analysieren

Analysieren Sie die Definition der Datenbanktabelle **/DMO/CONNECTION**.

1. Öffnen Sie das Entwicklungsobjekt, das die Definition der Datenbanktabelle **/DMO/CONNECTION** enthält.

   a) Wählen Sie in der Eclipse-Symbolleiste *///Open ABAP Development Object*, oder drücken Sie **Strg + Umschalt + A**.

   b) Geben Sie im Eingabefeld als Suchzeichenfolge **/dmo/con** ein.

   c) Klicken Sie in der Liste der passenden Einträge auf */DMO/CONNECTION (Datenbanktabelle)*, und wählen Sie *OK*.

2. Öffnen Sie die *Elementinformationen* (F2) für die Datenbanktabelle /DMO/CONNECTION.

   a) Klicken Sie auf `/dmo/connection` nach dem Schlüsselwort `define table`, und drücken Sie **F2**, um die Elementinformationen anzuzeigen.

© Copyright. Alle Rechte vorbehalten.

Welche Felder weisen die Beschreibung *///Flight Reference Scenario: From Airport* und *///Flight Reference Scenario: To Airport* auf?

Die Feldnamen lauten **airport_from_id** und **airport_to_id**.

Aufgabe 4: Daten aus der Datenbank lesen

Implementieren Sie im **Methodenkonstruktor** der lokalen Klasse **lcl_connection** eine SELECT-Anweisung, die Werte für die neuen Attribute aus der Datenbanktabelle **/DMO/CONNECTION** liest.

1. Kehren Sie zur lokalen Klasse **lcl_connection** in Ihrer globalen Klasse zurück.
   a) Öffnen Sie in der Editor-View von Eclipse die Registerkarte *ZCL_##_SELECT*.
   b) Wählen Sie in der globalen Klasse *Local Types*.
2. Navigieren Sie zur Implementierung der Methode **constructor**.
   a) Suchen Sie nach der Quelltextzeile `METHOD constructor.`.
3. Fügen Sie nach der Anweisung `ENDIF.` eine SELECT-Anweisung hinzu, die einen Einzelsatz aus der Datenbanktabelle **/DMO/CONNECTION** liest.
   a) Passen Sie den Quelltext wie folgt an:

```
IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
  RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.

SELECT SINGLE
  FROM /dmo/connection
```

4. Implementieren Sie die FIELDS-Klausel. Lesen Sie die Tabellenfelder **airport_from_id** und **airport_to_id**.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**Strg + Leertaste**), um die Feldnamen einzugeben.

   a) Geben Sie nach `FROM /DMO/CONNECTION FIELDS` ein.
   b) Drücken Sie nach einem Leerzeichen **Strg + Leertaste**, und wählen Sie *airport_from_id*.
   c) Drücken Sie nach einem Komma und einem Leerzeichen erneut **Strg + Leertaste**, und wählen Sie *airport_to_id*.
   d) Die vollständige FIELDS-Klausel sollte wie folgt aussehen:

```
    FIELDS airport_from_id, airport_to_id
```

5. Implementieren Sie die WHERE-Bedingung. Schränken Sie alle Schlüsselfelder der Datenbanktabelle (mit Ausnahme des Mandantenfeldes) mit den Werten der Importing-Parameter **i_carrier_id** und **i_connection_id** ein. Vergessen Sie nicht, als Escape-Zeichen für die Parameter das Präfix @ voranzustellen.

© Copyright. Alle Rechte vorbehalten.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**Strg + Leertaste**), um die Element- und Parameternamen einzugeben.

a) Fügen Sie den folgenden Quelltext nach der FIELDS-Klausel hinzu:

```
  WHERE carrier_id    = @i_carrier_id
    AND connection_id = @i_connection_id
```

Warum ist es nicht notwendig, das Mandantenfeld in der WHERE -Bedingung anzugeben?

Die Mandantenbehandlung wird vom Compiler durchgeführt.

6. Implementieren Sie die INTO-Klausel. Speichern Sie das SELECT-Ergebnis in den Attributen **airport_from_id** und **airport_to_id**. Vergessen Sie nicht, als Escape-Zeichen für die Attribute das Präfix @ voranzustellen.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**Strg + Leertaste**), um die Attributnamen einzugeben.

a) Passen Sie den Quelltext wie folgt an:

```
  WHERE carrier_id    = @i_carrier_id
    AND connection_id = @i_connection_id
  INTO ( @airport_from_id, @airport_to_id ).
```

b) Die vollständige SELECT-Anweisung sollte wie folgt aussehen:

```
SELECT SINGLE
    FROM /dmo/connection
  FIELDS airport_from_id, airport_to_id
   WHERE carrier_id    = @i_carrier_id
     AND connection_id = @i_connection_id
    INTO ( @airport_from_id, @airport_to_id ).
```

7. Implementieren Sie die Fehlerbehandlung nach der SELECT-Anweisung. Überprüfen Sie den Inhalt des Systemfeldes **sy-subrc**. Wenn er ungleich Null ist, muss die Ausnahme **CX_ABAP_INVALID_VALUE** ausgelöst werden.

a) Fügen Sie den folgenden Quelltext nach der SELECT-Anweisung hinzu:

```
IF sy-subrc <> 0.
   RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.
```

8. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe. Prüfen Sie, ob die Ausgabe für die neuen Attribute Daten anzeigt.

a) Drücken Sie **Strg + F3**, um die Klasse zu aktivieren.

b) Drücken Sie **F9**, um die Klasse auszuführen.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Die grundlegenden Funktionen von ABAP SQL beschreiben.
- Einzelwerte von der Datenbank lesen.

© Copyright. Alle Rechte vorbehalten.

Kapitel 4
Lektion 3

Mit CDS-Views arbeiten

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Analysieren Sie eine CDS-View-Definition.
- Daten über eine CDS-View lesen.

CDS-Views

View-Definitionen in ABAP Core Data Services (ABAP CDS)

ABAP Core Data Services, kurz ABAP CDS, sind eine Infrastruktur zum Definieren und Nutzen semantisch reicher Datenmodelle in ABAP. Sie verwenden eine CDS DDL, um CDS-Entitäten zu definieren, die ein semantisches Datenmodell implementieren. Die wichtigsten CDS-Modellierungsentitäten sind CDS-View-Entitäten.

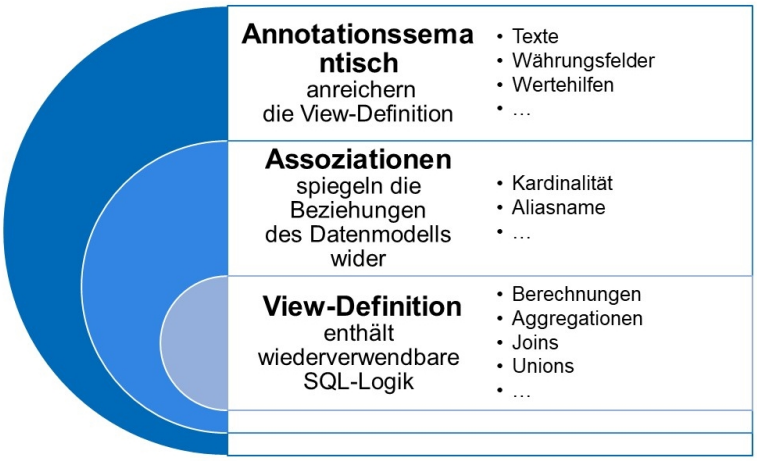


Abbildung 115: View-Definitionen in ABAP Core Data Services (ABAP CDS)

Die CDS-View-Definition enthält wiederverwendbare SQL-Logik: manchmal so einfach wie eine Projektion von Tabellenfeldern und manchmal komplexer mit Berechnungen, Aggregationen, Joins, Unions usw.

Eine CDS-View-Definition kann Assoziationen enthalten, um die Beziehungen des Datenmodells widerzuspiegeln. Consumer der View können die Assoziationen verwenden, um zugehörige Daten abzurufen.

Schließlich werden Annotationen verwendet, um die View-Definition semantisch anzureichern. Diese Metadaten werden von Frameworks ausgewertet, die auf CDS-View-Definitionen aufbauen. Ein solches Framework ist das ABAP-RESTful-Anwendungsprogrammiermodell, für das wir später in diesem Kurs ein Beispiel besprechen.

© Copyright. Alle Rechte vorbehalten.

Schauen wir uns einige Beispiele für CDS-View-Definitionen an.

Datendefinition /DMO/I_CONNECTION

```
define view entity  /DMO/I_Connection
    Wählen Sie /dmo/connection als Verbindung aus.



{

Schlüssel  Connection.carrier_id als  AirlineID,
Schlüssel  Connection.connection_id als  ConnectionID,
 Connection.airport_from_idals DepartureAirport,
 Connection.airport_to_idals DestinationAirport,
    ...


}
```

Name der View-Entität

Datenquelle mit Aliasnamen

Felder mit Aliasnamen

Abbildung 116: Beispiel: CDS-View-Definition - SQL-Logik

CDS-View-Definitionen sind in Repository-Objekten vom Typ Datendefinition enthalten. Sehen wir uns nun den Quelltext der Datendefinition /DMO/I_CONNECTION an.

Der Hauptteil ist die Anweisung DEFINE VIEW ENTITY. Sie enthält den Namen der CDS-View-Entität und nach dem Schlüsselwort FROM die Datenquelle. In unserem Beispiel lautet der Name der View-Entität „/DMO/I_Connection“, und die Datenquelle ist die Datenbanktabelle „/dmo/connection“. Der optionale Zusatz AS definiert einen Aliasnamen „Connection“, um die Datenquelle innerhalb der View-Definition zu adressieren.

Notiz:
Die Quelle einer CDS-View-Entität kann auch eine andere CDS-View sein.

Ein Paar geschweifter Klammern enthält die Liste der View-Elemente. In unserem Beispiel sind die View-Elemente Felder der Datenbanktabelle „/dmo/connection“. Das Schlüsselwort „Key“ vor den ersten beiden Elementen definiert sie als Schlüsselfelder der CDS-View-Entität. Der optionale Zusatz AS definiert einen Aliasnamen für jedes View-Element.

© Copyright. Alle Rechte vorbehalten.

Datendefinition /DMO/I_CONNECTION

```
define view entity  /DMO/I_Connection
    Wählen Sie /dmo/connection als Verbindung aus.

    Assoziation [1..1] zu /DMO/I_Carrier als _Airline
 auf $projektion.AirlineID = _Fluggesellschaft.AirlineID

{

Schlüssel  Connection.carrier_id als  AirlineID,
Schlüssel  Connection.connection_id als  ConnectionID,
 Connection.airport_from_idals DepartureAirport,
 Connection.airport_to_idals DestinationAirport,
    ...

    _Airline
}
```

Beziehung zur Entität /DMO/I_Carrier

Verfügbar für Verbraucher unter name _Airline

Abbildung 117: Beispiel: CDS-View-Definition - Assoziation

Der Zusatz `association` definiert eine Beziehung zu einer anderen CDS-View-Entität. In unserem Beispiel ist die zugehörige View die CDS-View-Entität /DMO/I_Carrier und der Name der Assoziation ist _Airline.

Diese Assoziation steht für Consumer der View zur Verfügung, nachdem sie zur Elementliste hinzugefügt wurde. Dieser Vorgang wird als Exponieren der Assoziation bezeichnet.

Datendefinition /DMO/I_CONNECTION

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Connection View - CDS Data Model'
define view entity  /DMO/I_Connection
    Wählen Sie /dmo/connection als Verbindung aus.

    Assoziation [1..1] zu /DMO/I_Carrier als _Airline
 auf $projektion.AirlineID = _Fluggesellschaft.AirlineID

{
    @ObjectModel.text.association: '_Airline'
Schlüssel  Connection.carrier_id als  AirlineID,
Schlüssel  Connection.connection_id als  ConnectionID,
 Connection.airport_from_idals DepartureAirport,
 Connection.airport_to_idals DestinationAirport,
    ...

     _Airline
}
```

Metadaten für die CDS-View-Entität

Metadaten für ein View-Element

Abbildung 118: Beispiel: CDS-View-Definition - Annotationen

Annotationen beginnen mit dem At-Zeichen (@) und werden verwendet, um die View-Definition für Consumer semantisch anzureichern. Annotationen vor der View-Definition werden als Entitätsannotationen bezeichnet. Entitätsannotationen werden verwendet, um Metadaten für die View-Entität als Ganzes zu definieren. Annotationen zwischen den geschweiften Klammern werden als Elementannotationen bezeichnet. Elementannotationen werden verwendet, um Metadaten für die verschiedenen Elemente der View zu definieren.

© Copyright. Alle Rechte vorbehalten.

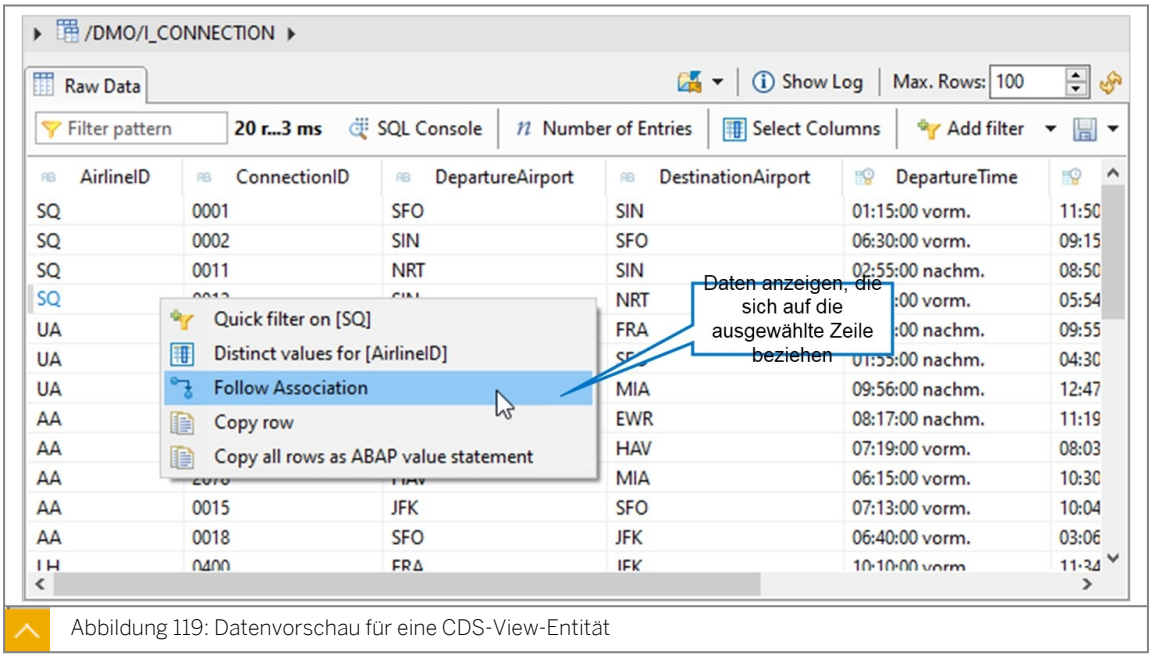


Abbildung 119: Datenvorschau für eine CDS-View-Entität

Sie haben bereits gelernt, wie Sie das Werkzeug *Datenvorschau* verwenden, um den Inhalt einer Datenbanktabelle anzuzeigen und zu analysieren. Dieses Werkzeug ist auch für CDS-View-Entitäten verfügbar.

Um die *Datenvorschau* für eine bestimmte CDS-Entität zu öffnen, klicken Sie mit der rechten Maustaste auf eine beliebige Stelle in der Datendefinition, und wählen Sie *Öffnen Mit → Datenvorschau*. Alternativ können Sie den Cursor an eine beliebige Stelle in der Datendefinition setzen und *F8* drücken.

Das Werkzeug zeigt die von der CDS-Entität zurückgegebenen Daten an. Dieselben Funktionen stehen zur Verfügung, um die Daten zu sortieren oder zu filtern und die Anzeige anzupassen.

Wenn die View-Definition eine oder mehrere Assoziationen enthält, können Sie diese verwenden, um zugehörige Daten anzuzeigen. Gehen Sie dazu wie folgt vor:

1. Klicken Sie mit der rechten Maustaste auf eine Zeile in der Anzeige.
2. Wählen Sie im Kontextmenü */// Follow Association*.
3. Wählen Sie aus der Liste der verfügbaren Assoziationen die gewünschte Assoziation aus.

© Copyright. Alle Rechte vorbehalten.

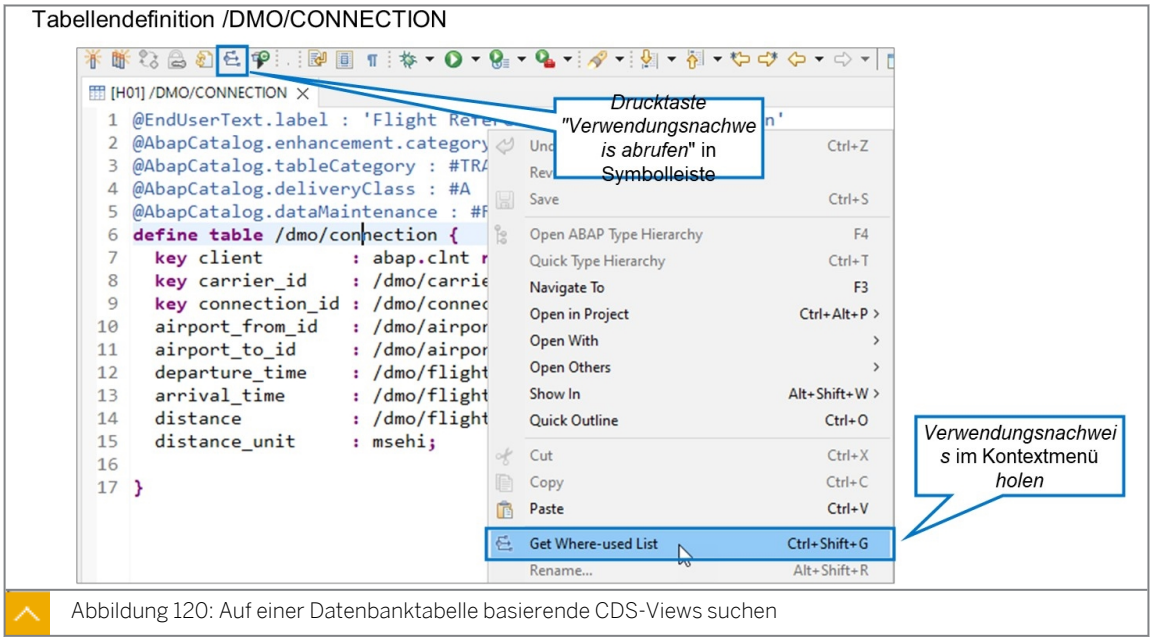


Abbildung 120: Auf einer Datenbanktabelle basierende CDS-Views suchen

Wenn Sie alle CDS-Views mit einer bestimmten Datenbanktabelle als Quelle finden möchten, können Sie das Werkzeug „///Verwendungsnachweis“ von ADT verwenden. Um dieses Werkzeug zu verwenden, gehen Sie wie folgt vor:

1. Öffnen Sie die Definition der Datenbanktabelle.
2. Klicken Sie mit der rechten Maustaste auf eine beliebige Stelle im Quelltext, und wählen Sie im Kontextmenü *///Verwendungsnachweis holen*. Alternativ können Sie „Strg + Umschalt + G“ drücken oder die Drucktaste mit demselben Symbol in der Symbolleiste wählen.
3. Die View *///Suche* zeigt eine Liste aller Entwicklungsobjekte an, die die Datenbanktabelle direkt verwenden.

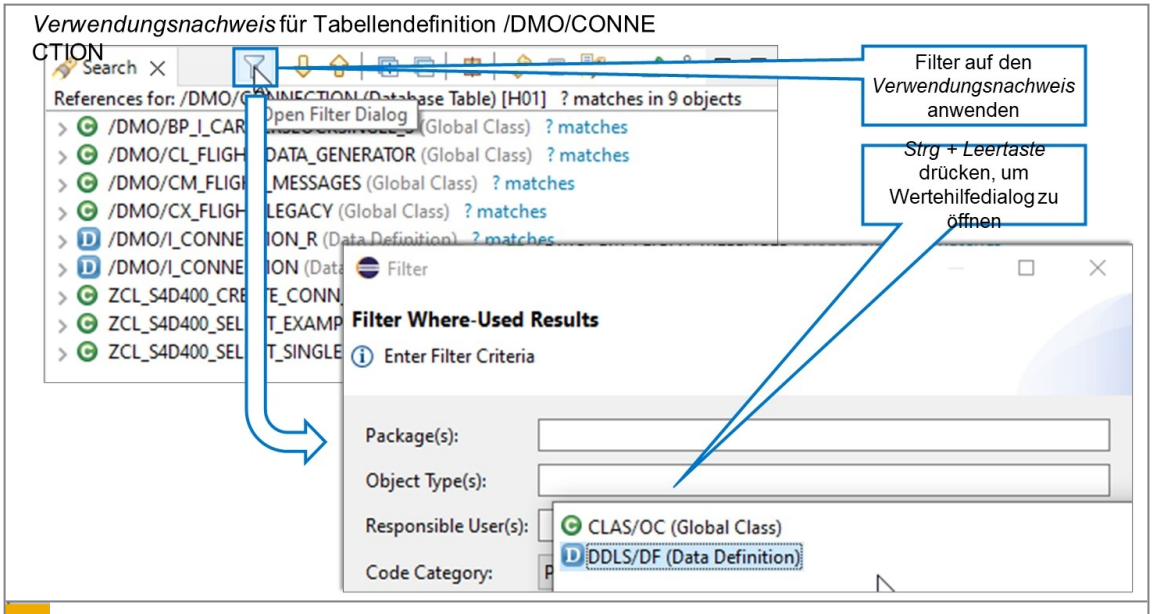


Abbildung 121: Filter auf den Verwendungsnachweis anwenden

© Copyright. Alle Rechte vorbehalten.

Sie können Filter auf den Verwendungsnachweis anwenden, wenn Sie z.B. nur Objekte aus bestimmten Paketen oder Objekten des jeweiligen Objekttyps benötigen. Das Beispiel zeigt, wie Sie nach CDS-Views filtern, die die Tabelle verwenden.

CDS-Views in ABAP SQL

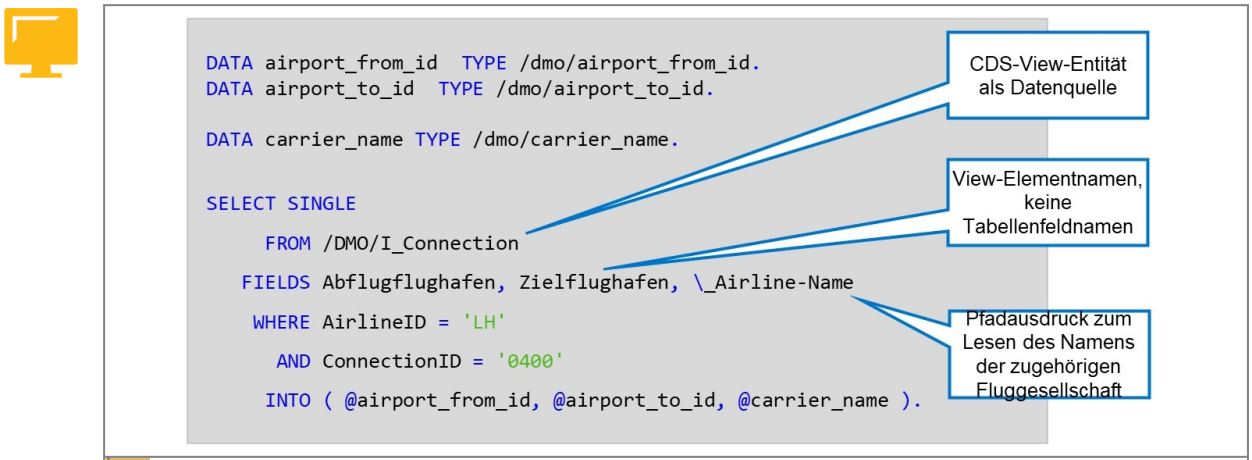


Abbildung 122: Beispiel: Lesen von Daten aus einer CDS-View-Entität

Wenn Sie eine SELECT-Anweisung in ABAP implementieren, können Sie eine CDS-View-Entität als Datenquelle verwenden, anstatt direkt aus der Datenbanktabelle zu lesen. Dies hat verschiedene Vorteile:

- Wiederverwendung der in der CDS-View enthaltenen SQL-Logik
- Übersichtliches und einfaches Lesen zugehöriger Daten über die Assoziationen
- Manchmal sind die Namen von Views und View-Elementen besser lesbar als die technischen Namen von Datenbanktabellen und Tabellenfeldern.

Die SELECT-Anweisung im Beispiel verwendet die CDS-View-Entität „/DMO/I_Connection“ als Datenquelle.

Notiz:
Beachten Sie, dass die in der FIELDS-Klausel und in den WHERE-Bedingungen verwendeten Namen die Aliasnamen der View-Elemente sind.

Das dritte Element in der FIELDS-Klausel verwendet die exponierte Assoziation „_Airline“. Sie liest das Element `Name` aus der zugehörigen CDS-View-Entität /DMO/I_Carrier. Ein solches Element wird Pfadausdruck genannt. Der umgekehrte Schrägstrich (\) ist ein obligatorisches Präfix für Assoziationsnamen.

© Copyright. Alle Rechte vorbehalten.

Kapitel 4
Übung 13

Eine CDS-View-Entität analysieren und verwenden

In dieser Übung lesen Sie aus einer CDS-View-Entität und nicht direkt aus der Datenbanktabelle. Dadurch haben Sie auch einen komfortablen Zugriff auf den Namen der Fluggesellschaft.

Vorlage:

/LRN/CL_S4D400_DBS_SELECT (globale Klasse)

Lösung:

/LRN/CL_S4D400_DBS_CDS (globale Klasse)

Aufgabe 1: Vorlage kopieren (optional)
Kopieren Sie die Vorlageklasse. Wenn Sie die vorherige Übung abgeschlossen haben, können Sie diese Aufgabe überspringen und mit der Bearbeitung Ihrer Klasse **`ZCL_##_SELECT`** fortfahren.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_DBS_SELECT`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_CDS`**, wobei ## für Ihre Gruppennummer steht).

Aufgabe 2: CDS-View-Entitäten analysieren
Analysieren Sie die Definition der CDS-View-Entität **`/DMO/I_Connection`**.

1. Öffnen Sie das Entwicklungsobjekt, das die Definition der CDS-View-Entität **`/DMO/I_Connection`** enthält.
2. Analysieren Sie die Elementliste der CDS-View-Entität.

   Wie lauten die Aliasnamen für die Felder **`airport_from_id`** und **`airport_to_id`**?

   _____
   _____
   _____
3. Öffnen Sie die *Elementinformationen* (F2) für das Ziel der Assoziation **`_Airline`**.

   Welches Element wird als *///Flight Reference Scenario: Carrier Name* beschrieben? Wie lautet der Elementtyp?

   _____
   _____
   _____

© Copyright. Alle Rechte vorbehalten.

Aufgabe 3: Zusätzliche Attribute deklarieren
Erweitern Sie die lokale Klasse **`lcl_connection`** um ein privates Instanzattribut **`carrier_name`**. Fügen Sie der Implementierung der Methode **`get_output( )`** Ausgabedaten für das neue Attribut hinzu.

Notiz:
Das neue Attribut ist noch nicht gefüllt. Um es zu füllen, passen wir die SELECT-Anweisung in der nächsten Aufgabe dieser Übung an.

1. Kehren Sie zur lokalen Klasse **`lcl_connection`** in Ihrer globalen Klasse zurück.
2. Fügen Sie der Klassendefinition das folgende private Attribut hinzu:

   Tabelle 3: Attribute

Attributname	Umfang	Datentyp
`carrier_name`	Instanz	/DMO/CARRIER_NAME

3. Erweitern Sie die Implementierung der Methode **`get_output`**. Fügen Sie den Namen der Fluggesellschaf dem Zeichenketten-Template mit der Fluggesellschafts-ID hinzu.
4. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe.

Aufgabe 4: CDS-View-Entitäten verwenden
Ersetzen Sie in der Implementierung der Methode **`constructor`** die SELECT-Anweisung, die aus der Datenbanktabelle **`/dmo/connection`** liest, durch eine SELECT-Anweisung, die aus der CDS-View-Entität **`/DMO/I_Connection`** liest.

1. Kommentieren Sie in der Implementierung der Methode **`constructor`** die SELECT-Anweisung aus.
2. Fügen Sie nach dem kommentierten Quelltext eine SELECT-Anweisung hinzu, die einen einzelnen Datensatz aus der CDS-View **`/DMO/I_Connection`** liest.
3. Implementieren Sie die FIELDS-Klausel. Lesen Sie die View-Elemente **`DepartureAirport`** und **`DestinationAirport`**. Lesen Sie zusätzlich das View-Element **`Name`** aus dem Ziel der Assoziation **`_Airline`**.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**`Strg + Leertaste`**), um die Namen einzugeben.

4. Implementieren Sie die WHERE-Bedingung. Schränken Sie die Schlüsselelemente der CDS-View-Entität mit den Werten der Importing-Parameter **`i_carrier_id`** und **`i_connection_id`** ein. Vergessen Sie nicht, als Escape-Zeichen für die Parameter das Präfix @ voranzustellen.

© Copyright. Alle Rechte vorbehalten.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**`Strg + Leertaste`**), um die Element- und Parameternamen einzugeben.

Ist es möglich, das Mandantenfeld in die WHERE-Bedingung aufzunehmen?

5. Implementieren Sie die INTO-Klausel. Speichern Sie das SELECT-Ergebnis in den Attributen **`airport_from_id`**, **`airport_to_id`** und **`carrier_name`**. Vergessen Sie nicht, als Escape-Zeichen für die Attribute das Präfix @ voranzustellen.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**`Strg + Leertaste`**), um die Attributnamen einzugeben.

6. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe. Prüfen Sie, ob die Ausgabe für das neue Attribut Daten anzeigt.

© Copyright. Alle Rechte vorbehalten.

Kapitel 4
Lösung 13

Eine CDS-View-Entität analysieren und verwenden

In dieser Übung lesen Sie aus einer CDS-View-Entität und nicht direkt aus der Datenbanktabelle. Dadurch haben Sie auch einen komfortablen Zugriff auf den Namen der Fluggesellschaft.

Vorlage:

/LRN/CL_S4D400_DBS_SELECT (globale Klasse)

Lösung:

/LRN/CL_S4D400_DBS_CDS (globale Klasse)

Aufgabe 1: Vorlage kopieren (optional)
Kopieren Sie die Vorlageklasse. Wenn Sie die vorherige Übung abgeschlossen haben, können Sie diese Aufgabe überspringen und mit der Bearbeitung Ihrer Klasse **`ZCL_##_SELECT`** fortfahren.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_DBS_SELECT`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_CDS`**, wobei ## für Ihre Gruppennummer steht).
   a) Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_DBS_SELECT`**.
   b) Verknüpfen Sie die View *Project Explorer* mit dem Editor.
   c) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Klasse **`/LRN/CL_S4D400_DBS_SELECT`**, um das Kontextmenü zu öffnen.
   d) Wählen Sie im Kontextmenü *Duplicate ...*
   e) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_CDS`** ein, wobei ## für Ihre Gruppennummer steht.
   f) Passen Sie die Beschreibung an, und wählen Sie *Next*.
   g) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

Aufgabe 2: CDS-View-Entitäten analysieren
Analysieren Sie die Definition der CDS-View-Entität **`/DMO/I_Connection`**.

1. Öffnen Sie das Entwicklungsobjekt, das die Definition der CDS-View-Entität **`/DMO/I_Connection`** enthält.
   a) Wählen Sie in der Eclipse-Symbolleiste *///Open ABAP Development Object*, oder drücken Sie **`Strg + Umschalt + A`**.
   b) Geben Sie im Eingabefeld als Suchzeichenfolge **`/DMO/I_Con`** ein.
   c) Klicken Sie in der Liste der passenden Einträge auf */DMO/I_CONNECTION (Datendefinition)*, und wählen Sie *OK*.

© Copyright. Alle Rechte vorbehalten.

2. Analysieren Sie die Elementliste der CDS-View-Entität.

   Wie lauten die Aliasnamen für die Felder **airport_from_id** und **airport_to_id**?

   Die Aliasnamen lauten **DepartureAirport** und **DestinationAirport**.

   a) Navigieren Sie zur durch Kommata getrennten Listen zwischen dem Paar geschweifter Klammern ({ }).

   b) Sie finden die Aliasnamen nach dem Schlüsselwort `AS`.

3. Öffnen Sie die *Elementinformationen* (F2) für das Ziel der Assoziation **_Airline**.

   a) Klicken Sie auf `/DMO/I_Carrier` vor `as _Airline`, und drücken Sie **F2**, um die Elementinformationen anzuzeigen.

   Welches Element wird als *///Flight Reference Scenario: Carrier Name* beschrieben? Wie lautet der Elementtyp?

   Der Elementname lautet **Name** und der Elementtyp **/dmo/carrier_name**.

Aufgabe 3: Zusätzliche Attribute deklarieren

Erweitern Sie die lokale Klasse **lcl_connection** um ein privates Instanzattribut **carrier_name**. Fügen Sie der Implementierung der Methode **get_output( )** Ausgabedaten für das neue Attribut hinzu.

Notiz:
Das neue Attribut ist noch nicht gefüllt. Um es zu füllen, passen wir die SELECT-Anweisung in der nächsten Aufgabe dieser Übung an.

1. Kehren Sie zur lokalen Klasse **lcl_connection** in Ihrer globalen Klasse zurück.

   a) Wählen Sie in der globalen Klasse *Local Types*.

2. Fügen Sie der Klassendefinition das folgende **private** Attribut hinzu:

   Tabelle 3: Attribute

Attributname	Umfang	Datentyp
carrier_name	Instanz	/DMO/CARRIER_NAME

   a) Passen Sie den Quelltext wie folgt an:

```
  PRIVATE SECTION.
    DATA carrier_id      TYPE /dmo/carrier_id.
    DATA connection_id   TYPE /dmo/connection_id.

    DATA airport_from_id TYPE /dmo/airport_from_id.
    DATA airport_to_id   TYPE /dmo/airport_to_id.

    DATA carrier_name    TYPE /dmo/carrier_name.
ENDCLASS.
```

© Copyright. Alle Rechte vorbehalten.

3. Erweitern Sie die Implementierung der Methode **get_output**. Fügen Sie den Namen der Fluggesellschaf dem Zeichenketten-Template mit der Fluggesellschafts-ID hinzu.
   a) Navigieren Sie zur Implementierung der Methode **get_output**.
   b) Ersetzen Sie die Anweisung `APPEND |Carrier: { carrier_id }| TO r_output.` mit der folgenden Anweisung:

```
APPEND |Carrier: { carrier_id } { carrier_name }| TO r_output.
```

4. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe.
   a) Drücken Sie **Strg + F3**, um die Klasse zu aktivieren.
   b) Drücken Sie **F9**, um die Klasse auszuführen.

Aufgabe 4: CDS-View-Entitäten verwenden

Ersetzen Sie in der Implementierung der Methode **constructor** die SELECT-Anweisung, die aus der Datenbanktabelle **/dmo/connection** liest, durch eine SELECT-Anweisung, die aus der CDS-View-Entität **/DMO/I_Connection** liest.

1. Kommentieren Sie in der Implementierung der Methode **constructor** die SELECT-Anweisung aus.
   a) Markieren Sie alle Zeilen, die zu `SELECT SINGLE ... .` gehören.
   b) Drücken Sie **Strg + <**, um vor jeder ausgewählten Zeile ein Sternchen (*) hinzuzufügen.
2. Fügen Sie nach dem kommentierten Quelltext eine SELECT-Anweisung hinzu, die einen einzelnen Datensatz aus der CDS-View **/DMO/I_Connection** liest.
   a) Passen Sie den Quelltext wie folgt an:

```
*    SELECT SINGLE
*      FROM /dmo/connection
*    FIELDS airport_from_id, airport_to_id
*     WHERE carrier_id    = @i_carrier_id
*       AND connection_id = @i_connection_id
*      INTO ( @airport_from_id, @airport_to_id ).

  SELECT SINGLE
    FROM /DMO/I_Connection
```

3. Implementieren Sie die FIELDS-Klausel. Lesen Sie die View-Elemente **DepartureAirport** und **DestinationAirport**. Lesen Sie zusätzlich das View-Element **Name** aus dem Ziel der Assoziation **_Airline**.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**Strg + Leertaste**), um die Namen einzugeben.

   a) Geben Sie nach `FROM /DMO/I_Connection` `FIELDS` ein.
   b) Drücken Sie nach einem Leerzeichen **Strg + Leertaste**, und wählen Sie *DepartureAirport*.

© Copyright. Alle Rechte vorbehalten.

c) Drücken Sie nach einem Komma und einem Leerzeichen erneut **Strg + Leertaste**, und wählen Sie **DestinationAirport**.

d) Geben Sie nach einem Komma und einem Leerzeichen einen umgekehrten Schrägstrich (\) ein, drücken Sie **Strg + Leertaste**, und wählen Sie *_Airline*.

e) Geben Sie unmittelbar nach `_Airline` einen Bindestrich (-) ein, drücken Sie **Strg + Leertaste**, und wählen Sie *Name*.

f) Die vollständige FIELDS-Klausel sollte wie folgt aussehen:

```
    FIELDS DepartureAirport, DestinationAirport, \_Airline-Name
```

4. Implementieren Sie die WHERE-Bedingung. Schränken Sie die Schlüsselelemente der CDS-View-Entität mit den Werten der Importing-Parameter **i_carrier_id** und **i_connection_id** ein. Vergessen Sie nicht, als Escape-Zeichen für die Parameter das Präfix @ voranzustellen.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**Strg + Leertaste**), um die Element- und Parameternamen einzugeben.

a) Fügen Sie den folgenden Quelltext nach der FIELDS-Klausel hinzu:

```
  WHERE AirlineID    = @i_carrier_id
    AND ConnectionID = @i_connection_id
```

Ist es möglich, das Mandantenfeld in die WHERE-Bedingung aufzunehmen?

Nein, die Elementliste der CDS-View-Entität enthält nicht das Mandantenfeld.

5. Implementieren Sie die INTO-Klausel. Speichern Sie das SELECT-Ergebnis in den Attributen **airport_from_id**, **airport_to_id** und **carrier_name**. Vergessen Sie nicht, als Escape-Zeichen für die Attribute das Präfix @ voranzustellen.

Hinweis:
Verwenden Sie die automatische Vervollständigung (**Strg + Leertaste**), um die Attributnamen einzugeben.

a) Passen Sie den Quelltext wie folgt an:

```
  WHERE AirlineID    = @i_carrier_id
    AND ConnectionID = @i_connection_id
   INTO ( @airport_from_id, @airport_to_id, @carrier_name ).
```

b) Die vollständige SELECT-Anweisung sollte wie folgt aussehen:

```
SELECT SINGLE
      FROM /DMO/I_Connection
    FIELDS DepartureAirport, DestinationAirport, \_Airline-Name
     WHERE AirlineID    = @i_carrier_id
```

© Copyright. Alle Rechte vorbehalten.

```
       AND ConnectionID = @i_connection_id
      INTO ( @airport_from_id, @airport_to_id, @carrier_name ).
```

6. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe. Prüfen Sie, ob die Ausgabe für das neue Attribut Daten anzeigt.
   a) Drücken Sie **Strg + F3**, um die Klasse zu aktivieren.
   b) Drücken Sie **F9**, um die Klasse auszuführen.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Analysieren Sie eine CDS-View-Definition.
- Daten über eine CDS-View lesen.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 4

Überprüfung des Lernerfolgs

1. Wie lautet die Verknüpfung zur Vorschau des Inhalts einer Datenbanktabelle?
   Wählen Sie die richtige Antwort.

   - ☐ **A** F1
   - ☐ **B** F2
   - ☐ **C** F3
   - ☐ **D** F8

2. Welcher Teil von SQL wird durch ABAP SQL repräsentiert?
   Wählen Sie die richtige Antwort.

   - ☐ **A** Data Control Language
   - ☐ **B** Data Definition Language
   - ☐ **C** Data Manipulation Language (DML)

3. Wie lautet die richtige Reihenfolge der Klauseln in der SELECT SINGLE-Anweisung?
   Wählen Sie die richtige Antwort.

   - ☐ **A** SELECT SINGLE FIELDS... VON... INTO... WHERE...
   - ☐ **B** SELECT SINGLE FROM... FIELDS... WHERE... INTO
   - ☐ **C** SELECT SINGLE FROM... FIELDS... INTO... WHERE...
   - ☐ **D** Sie können die Klauseln in beliebiger Reihenfolge schreiben.

4. Welche der folgenden Datenquellen sind gültige Datenquellen für eine CDS-View-Entität?
   Wählen Sie die richtigen Antworten.

   - ☐ **A** Eine interne Tabelle
   - ☐ **B** Datenbanktabelle
   - ☐ **C** Eine andere CDS-View-Entität
   - ☐ **D** Eine Struktur

© Copyright. Alle Rechte vorbehalten.

5. Sie schreiben eine SELECT-Anweisung, die Daten mit einer CDS-View-Entität liest. Sie möchten in der Feldliste ein Feld aus einer zugehörigen Tabelle lesen. Was verwenden Sie?

   Wählen Sie die richtige Antwort.

   - ☐ A Einen logischen Ausdruck
   - ☐ B Einen Pfadausdruck
   - ☐ C Einen regulären Ausdruck

© Copyright. Alle Rechte vorbehalten.

Überprüfung des Lernerfolgs - Antworten

1. Wie lautet die Verknüpfung zur Vorschau des Inhalts einer Datenbanktabelle?

   Wählen Sie die richtige Antwort.

   - [ ] A F1
   - [ ] B F2
   - [ ] C F3
   - [x] D F8

   Richtig! Die Tastenkombination für die Vorschau des Inhalts einer Datenbanktabelle ist F8.

2. Welcher Teil von SQL wird durch ABAP SQL repräsentiert?

   Wählen Sie die richtige Antwort.

   - [ ] A Data Control Language
   - [ ] B Data Definition Language
   - [x] C Data Manipulation Language (DML)

   Das ist richtig.

3. Wie lautet die richtige Reihenfolge der Klauseln in der SELECT SINGLE-Anweisung?

   Wählen Sie die richtige Antwort.

   - [ ] A SELECT SINGLE FIELDS... VON... INTO... WHERE...
   - [x] B SELECT SINGLE FROM... FIELDS... WHERE... INTO
   - [ ] C SELECT SINGLE FROM... FIELDS... INTO... WHERE...
   - [ ] D Sie können die Klauseln in beliebiger Reihenfolge schreiben.

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

4. Welche der folgenden Datenquellen sind gültige Datenquellen für eine CDS-View-Entität?

   Wählen Sie die richtigen Antworten.

   - [ ] A Eine interne Tabelle
   - [X] B Datenbanktabelle
   - [X] C Eine andere CDS-View-Entität
   - [ ] D Eine Struktur

   Das ist richtig.

5. Sie schreiben eine SELECT-Anweisung, die Daten mit einer CDS-View-Entität liest. Sie möchten in der Feldliste ein Feld aus einer zugehörigen Tabelle lesen. Was verwenden Sie?

   Wählen Sie die richtige Antwort.

   - [ ] A Einen logischen Ausdruck
   - [X] B Einen Pfadausdruck
   - [ ] C Einen regulären Ausdruck

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

KAPITEL 5 Mit strukturierten Datenobjekten arbeiten

Lektion 1

Ein strukturiertes Datenobjekt deklarieren 241

Lektion 2

Mit strukturierten Datenobjekten arbeiten 247

Übung 14 : Ein strukturiertes Datenobjekt verwenden 257

LERNZIELE

- Deklarieren Sie ein strukturiertes Datenobjekt.
- Arbeiten Sie mit strukturierten Datenobjekten.
- Verwenden Sie strukturierte Datenobjekte in ABAP SQL.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 5
Lektion 1

Ein strukturiertes Datenobjekt deklarieren

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Deklarieren Sie ein strukturiertes Datenobjekt.

Strukturierte Datenobjekte

```
DATA  airport_from_id  TYPE /DMO/airport_from_id.DATA
 airport_to_id  TYPE /DMO/airport_to_id.

SELECT SINGLE
   FROM /dmo/I_Connection
 FIELDS Abflugflughafen, Zielflughafen
  WHERE Spediteur-ID  = 'LH'
  AND Verbindungs-ID = '0400'
   INTO (  @airport_from_id, @airport_to_id ).
```

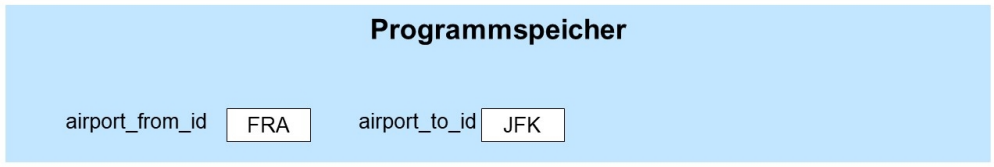


Abbildung 123: Der Stand bisher – einfache Variablen

Bisher haben Sie mit einfachen Variablen gearbeitet, die jeweils eine einzelne Information speichern können. In diesem Beispiel haben wir zwei Variablen: eine für den Abflughafen, eine andere für den Zielflughafen.

Wenn Sie einen Datensatz aus der Datenbank lesen, müssen Sie all diese Informationen zusammenhalten. Die beiden Variablen im Beispiel sind völlig unabhängig voneinander und daher nicht geeignet, unterschiedliche zusammengehörende Informationen zu speichern.

In ABAP besteht die Lösung darin, eine strukturierte Variable (kurz auch als „Struktur“ bezeichnet) zu verwenden.

© Copyright. Alle Rechte vorbehalten.

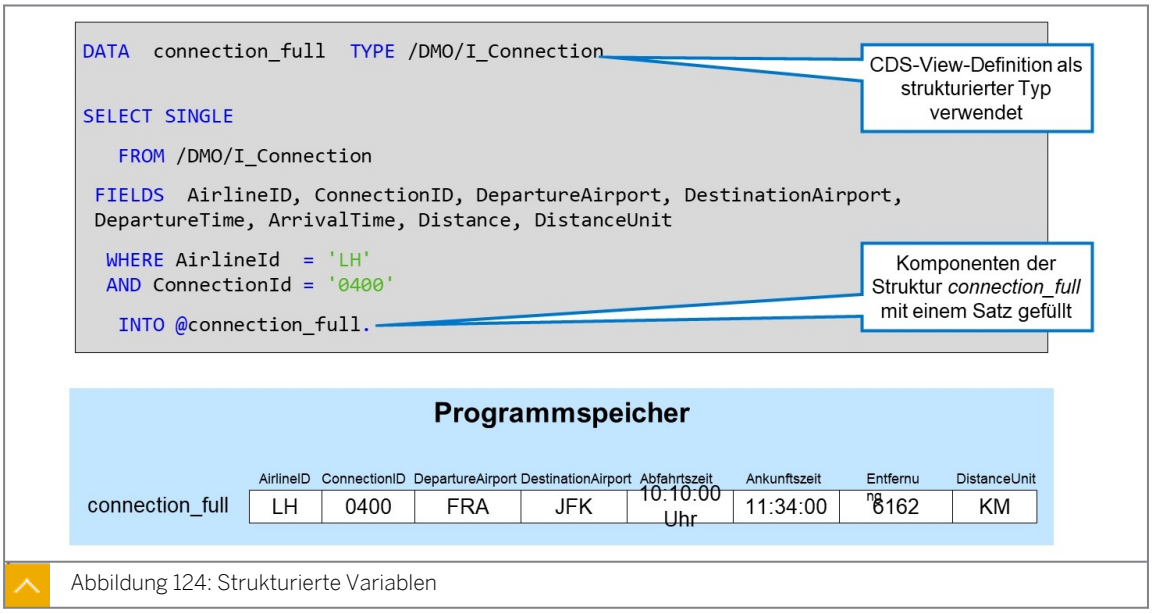


Abbildung 124: Strukturierte Variablen

Eine Struktur ist ein variables ABAP-Datenobjekt mit einem Namen und einem strukturierten Typ.

Im Beispiel stellt das Datenobjekt connection_full eine solche Struktur dar. Sie ist in acht Komponenten unterteilt, von denen jede ebenfalls über einen Namen und einen Typ verfügt. Sie können sowohl die Struktur als Ganzes als auch die einzelnen Komponenten adressieren. Insbesondere können Sie jede Komponente auf genau die gleiche Weise wie eine eigenständige einfache Variable verwenden.

Es gibt verschiedene Möglichkeiten zum Deklarieren einer Struktur. Sie können strukturierte Typen mit dem Schlüsselwort TYPES definieren oder ein Repository-Objekt vom Typ „Struktur“ verwenden. Die Definitionen von Views und Datenbanktabellen können ebenfalls als strukturierte Typen dienen. Im Beispiel wird die CDS-View /DMO/I_Connection als strukturierter Typ verwendet.

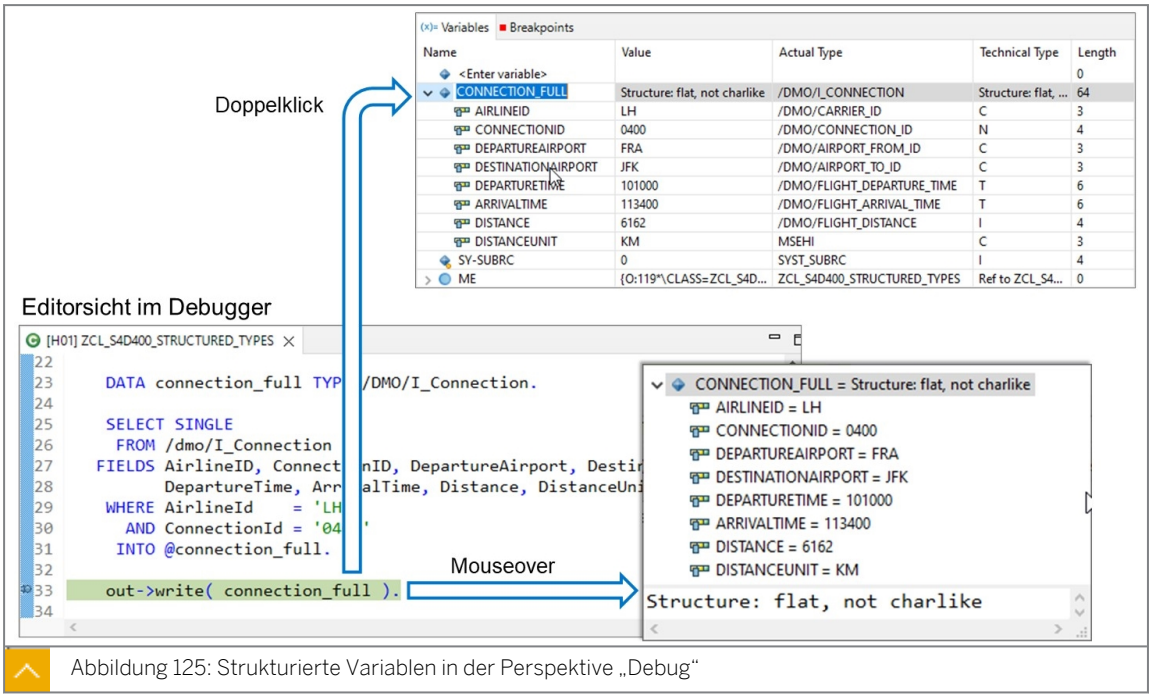


Abbildung 125: Strukturierte Variablen in der Perspektive „Debug“

© Copyright. Alle Rechte vorbehalten.

In der *Debugger-Perspektive* gibt es zwei Möglichkeiten, die Struktur und den Inhalt einer strukturierten Variablen zu analysieren:

Variablenvorschau (Mouseover)

Setzen Sie den Fokus auf den Quelltexteditor, und positionieren Sie den Mauszeiger auf einen Variablennamen. Dann öffnet sich ein Dialogfenster mit Details zur Struktur und zum Inhalt des Datenobjekts.

Anzeige in der *Variablensicht* (Doppelklick)

Um die Variable in der *Variablensicht* anzuzeigen, geben Sie entweder den Variablennamen unter <*Variable eingeben*> ein, oder doppelklicken Sie im Quelltext-Editor auf den Variablennamen. Expandieren Sie den Knoten mit dem Strukturnamen, um die Liste der Komponenten anzuzeigen.

Strukturierte Datentypen

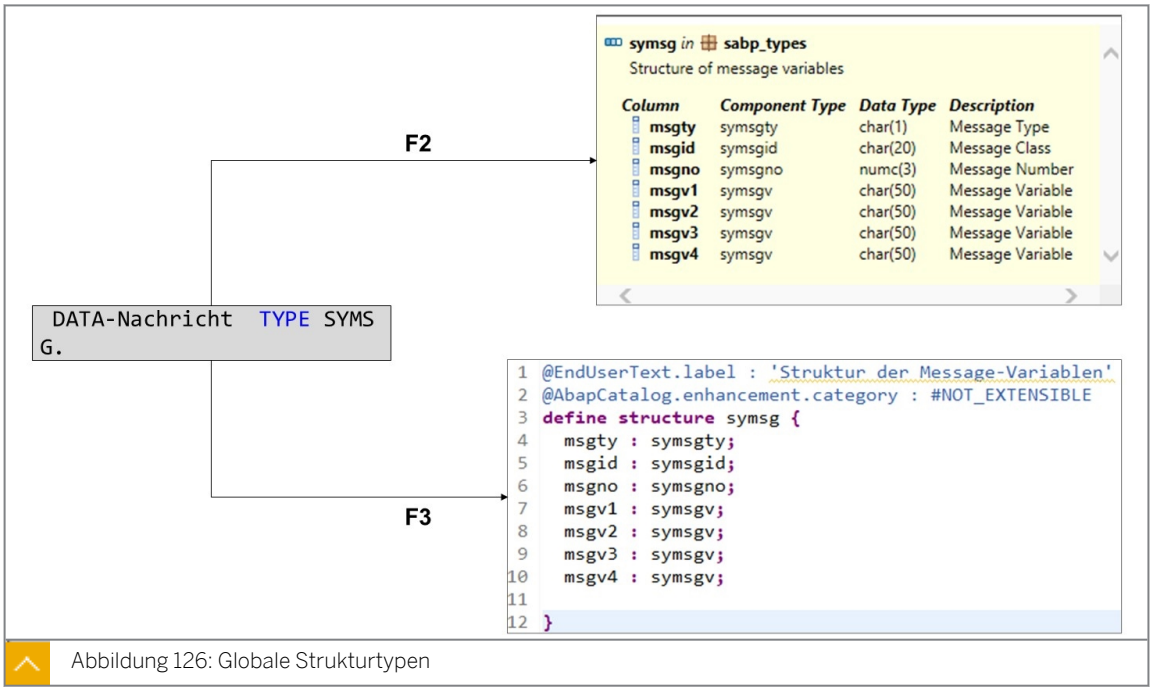


Abbildung 126: Globale Strukturtypen

Ein globaler Strukturtyp ist ein Repository-Objekt, das an beliebiger Stelle im System als Datentyp verwendet werden kann. Im Beispiel wird der Strukturtyp SYMSG verwendet, um variable Nachrichten zu deklarieren.

Wenn Sie die Taste **F2** drücken, um die Details dieses Datentyps anzuzeigen, sehen Sie, dass es sich um einen Strukturtyp mit sieben Komponenten handelt. Sie können auch die Namen, die technischen Typen und die Beschreibungen der Komponenten anzeigen.

Wenn Sie die Taste **F3** drücken, um zur Definition des Typs zu navigieren, wird mit dem ADT-Editor eine neue View für globale Strukturtypen geöffnet.

Die Definition eines globalen Strukturtyps ist der Definition einer Datenbanktabelle sehr ähnlich. Der Hauptteil der Definition besteht aus einer DEFINE-STRUCTURE-Anweisung mit dem Namen des Strukturtyps. Danach folgt die Liste der Strukturkomponenten in geschweiften Klammern ({ , }). Jede Komponente verfügt über einen Komponententyp. Komponententypen werden häufig durch Datenelemente beschrieben, es können aber auch Strukturtypen als Komponententypen verwendet werden. Strukturen mit strukturierten Komponenten werden als „geschachtelte Strukturen“ bezeichnet.

© Copyright. Alle Rechte vorbehalten.

Zusätzliche Quelltextzeilen vor der Anweisung DEFINE STRUCTURE geben zusätzliche Eigenschaften des Strukturtyps an, darunter einen Bezeichner.

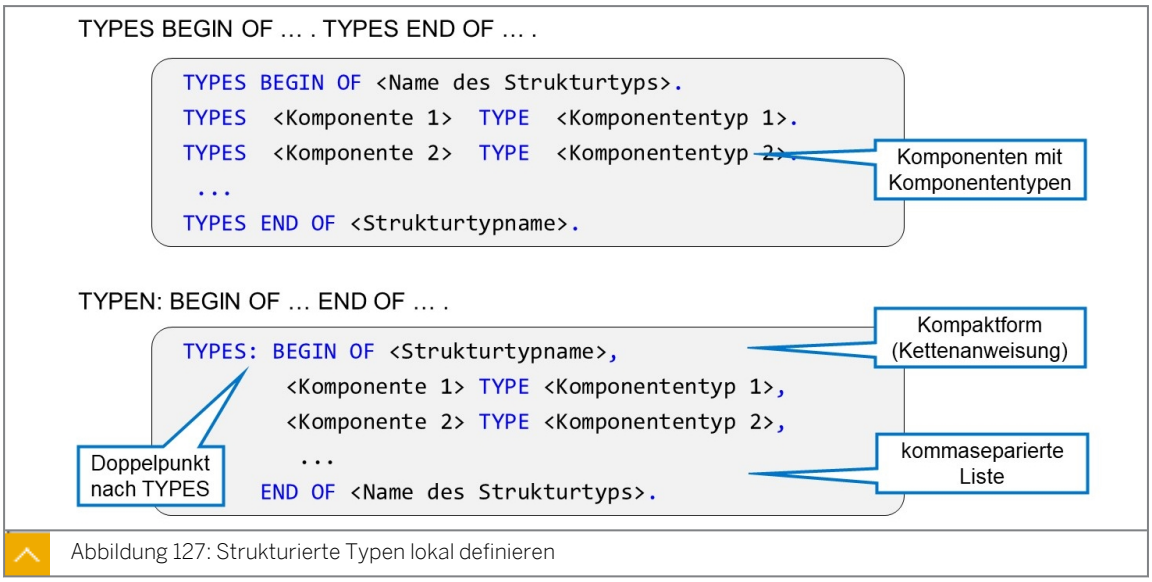


Abbildung 127: Strukturierte Typen lokal definieren

Sie können strukturierte Typen auch in einem ABAP-Programm mit der Anweisung TYPES definieren. Die Strukturdefinition beginnt mit der Anweisung TYPES BEGIN OF <Name des Strukturtyps> und endet mit TYPES END OF <Name des Strukturtyps>. Dazwischen geben Sie den Namen jeder Komponente und deren Typ in einer zusätzlichen TYPES-Anweisung an.

Eine kompakte Form verwendet das Schlüsselwort TYPES nur einmal, gefolgt von einem Doppelpunkt (:). Der Zusatz BEGIN OF, der Zusatz END OF und die dazwischen liegenden Komponentendefinitionen werden dann durch Kommata getrennt.

Dies wird als „Kettenanweisung“ bezeichnet.

Notiz:
In der Vergangenheit wurden Kettenanweisungen in ABAP sehr häufig verwendet. Heutzutage werden sie nur empfohlen, um eng zusammengehörende Anweisungen zusammenzufassen.

© Copyright. Alle Rechte vorbehalten.

```
TYPES: BEGIN OF st_connection,
         airport_from_id TYPE /dmo/airport_from_id,
         airport_to_id  TYPE /dmo/airport_to_id,
         carrier_name TYPE /dmo/carrier_name,
       END OF st_connection


 DATA-Verbindung TYPE st_connection.
```

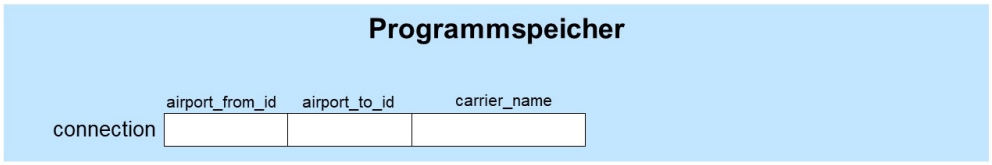


Abbildung 128: Beispiel: Lokaler strukturierter Typ

In diesem Beispiel wird eine TYPES:-Kettenanweisung verwendet, um den lokalen Strukturtyp st_connection zu definieren, der aus den drei Komponenten airport_from_id, airport_to_id und carrier_name besteht. Jede Komponente wird mit einem Datenelement typisiert, das mit /dmo/ beginnt.

Der lokale Strukturtyp st_connection wird dann in einer DATA-Anweisung verwendet, um die strukturierte Variable connection zu typisieren.

```
TYPES: BEGIN OF st_nested,
         airport_from_id  TYPE /dmo/airport_from_id,
         airport_to_id  TYPE /dmo/airport_to_id,
message TYPE symsg,
carrier_name TYPE /dmo/carrier_name,
       END OF st_nested.


 DATA-Verbindung TYPE st_nested.
```

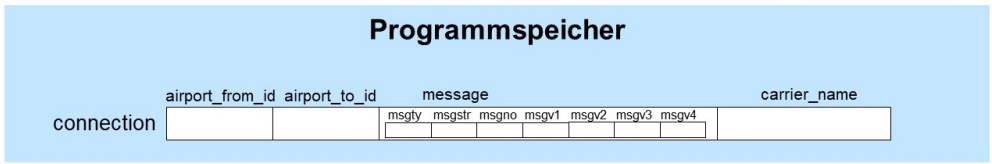


Abbildung 129: Beispiel: Geschachtelter lokaler strukturierter Typ

In diesem Beispiel hat die Variable `connection` einen geschachtelten strukturierten Typ. Der Typ `st_nested` definiert 4 Komponenten. Drei sind mit Datenelementen typisiert, daher handelt es sich um einfache Komponenten. Die Komponente `message` ist aber mit einem strukturierten Typ typisiert. Daher handelt es sich um eine strukturierte Komponente. Dadurch wird die Variable `connection` zu einer geschachtelten Struktur.

© Copyright. Alle Rechte vorbehalten.

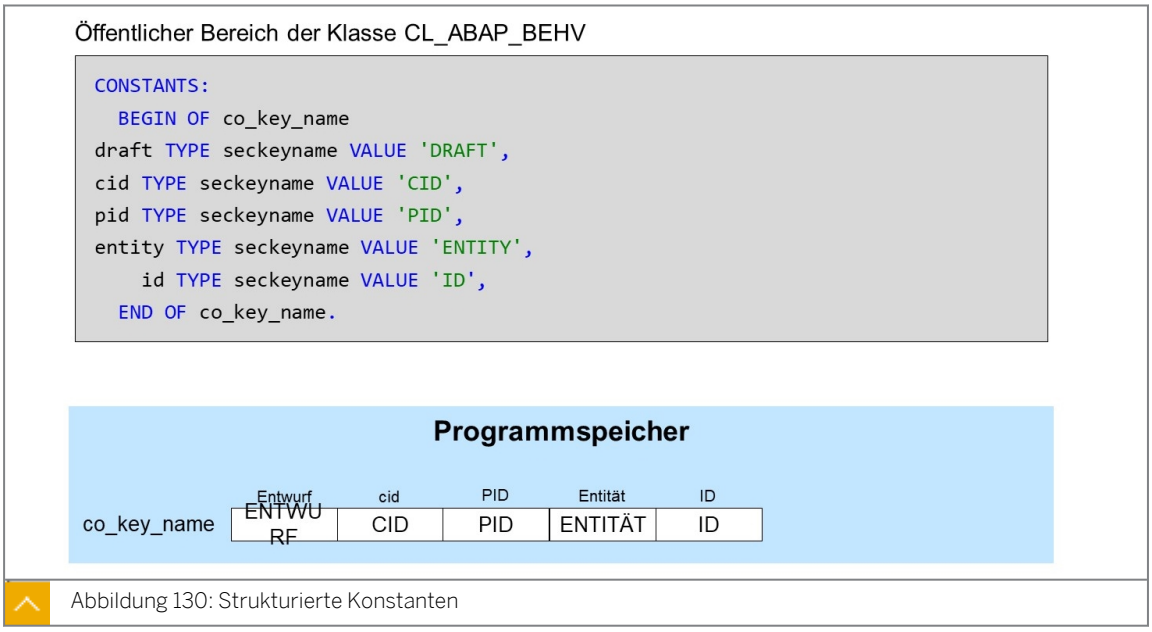


Abbildung 130: Strukturierte Konstanten

ABAP unterstützt nicht nur strukturierte Variablen, sondern auch strukturierte Konstanten. Um eine strukturierte Konstante zu definieren, verwenden Sie BEGIN OF und END OF als Teil einer CONSTANTS-Anweisung. Das Beispiel zeigt eine strukturierte Konstante, die im öffentlichen Bereich der globalen Klasse CL_ABAP_BEHV definiert ist. Alle fünf Komponenten sind mit dem Datenelement SECKEYNAME typisiert. Hinweis: Der Zusatz VALUE ist bei der Definition von Konstanten obligatorisch.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Deklarieren Sie ein strukturiertes Datenobjekt.

© Copyright. Alle Rechte vorbehalten.

Kapitel 5
Lektion 2

Mit strukturierten Datenobjekten arbeiten

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Arbeiten Sie mit strukturierten Datenobjekten.
- Verwenden Sie strukturierte Datenobjekte in ABAP SQL.

Strukturierte Datenobjekte

Strukturkomponentenselektor –

```
<Strukturname>-<Komponentenname>
```

Beispiel: Zugriff auf Strukturkomponenten

```
TYPES: BEGIN OF st_connection,
         airport_from_id TYPE /dmo/airport_from_id,
         airport_to_id TYPE /dmo/airport_to_id,
         carrier_name TYPE /dmo/carrier_name,
       END OF st_connection

DATA-Verbindung TYPE st_connection.
connection-airport_from_id = 'ABC'.
connection-airport_to_id = 'XYZ'.
connection-carrier_name = 'Meine Fluggesellschaft'.
```

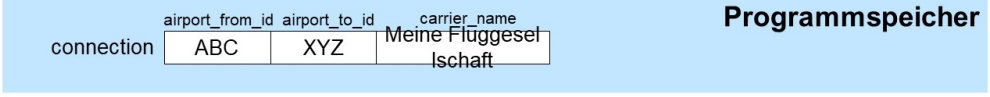


Abbildung 131: Zugriff auf Strukturkomponenten

Um auf eine Komponente einer Struktur zuzugreifen, müssen Sie ein Minuszeichen (-) zwischen den Strukturnamen und den Komponentennamen setzen.

Notiz:
Vor oder nach dem Komponentenselektor sind keine Leerzeichen erlaubt.

Wenn Sie auf diese Weise auf eine Strukturkomponente zugreifen, kann sie an jeder Operandenposition verwendet werden, an der eine Variable des gleichen Typs verwendet werden kann. Die Komponente airport_from_id der Struktur connection im obigen Beispiel hat den Typ /DMO/AIRPORT_FROM_ID. Folglich können Sie diese Komponente an jeder Operandenposition verwenden, an der Sie eine einfache Variable des Typs /DMO/AIRPORT_FROM_ID verwenden können; und zwar nicht nur auf der linken Seite einer Wertzuweisung wie im Beispiel, sondern auch auf der rechten Seite, in der

© Copyright. Alle Rechte vorbehalten.

Parameterübergabe eines Methodenaufrufs, in der INTO- oder WHERE-Klausel einer SELECT-Anweisung usw.

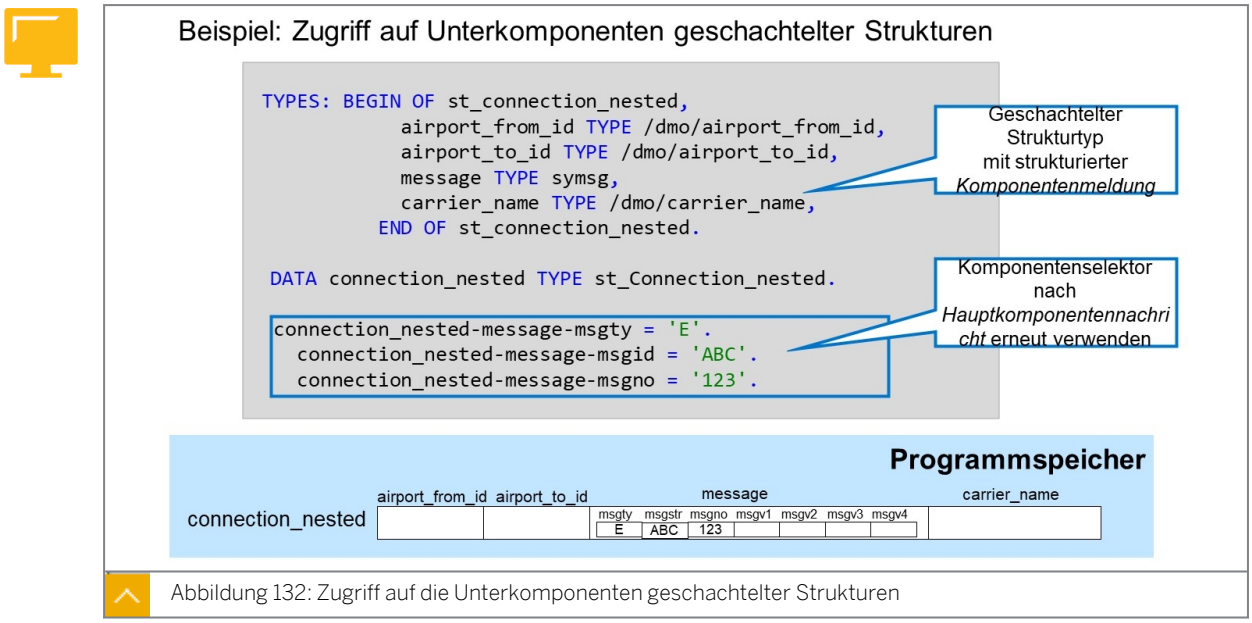


Abbildung 132: Zugriff auf die Unterkomponenten geschachtelter Strukturen

Wenn die Komponente einer Struktur selbst eine Struktur ist, greifen Sie durch erneute Verwendung des Komponentenselektors nach dem Namen der Hauptkomponente auf die Unterkomponenten zu. Die erste Wertzuweisung im Beispiel greift auf die Komponente MSGTY von MESSAGE zu, die selbst eine Komponente der geschachtelten Struktur CONNECTION_NESTED ist.

Hinweis:
Mit der Codevervollständigung können Sie den Zugriff auf Strukturkomponenten implementieren. Setzen Sie den Cursor direkt hinter den Strukturkomponentenselektor, und drücken Sie **STRG + LEERTASTE**, um eine Liste aller verfügbaren Strukturkomponenten anzuzeigen.

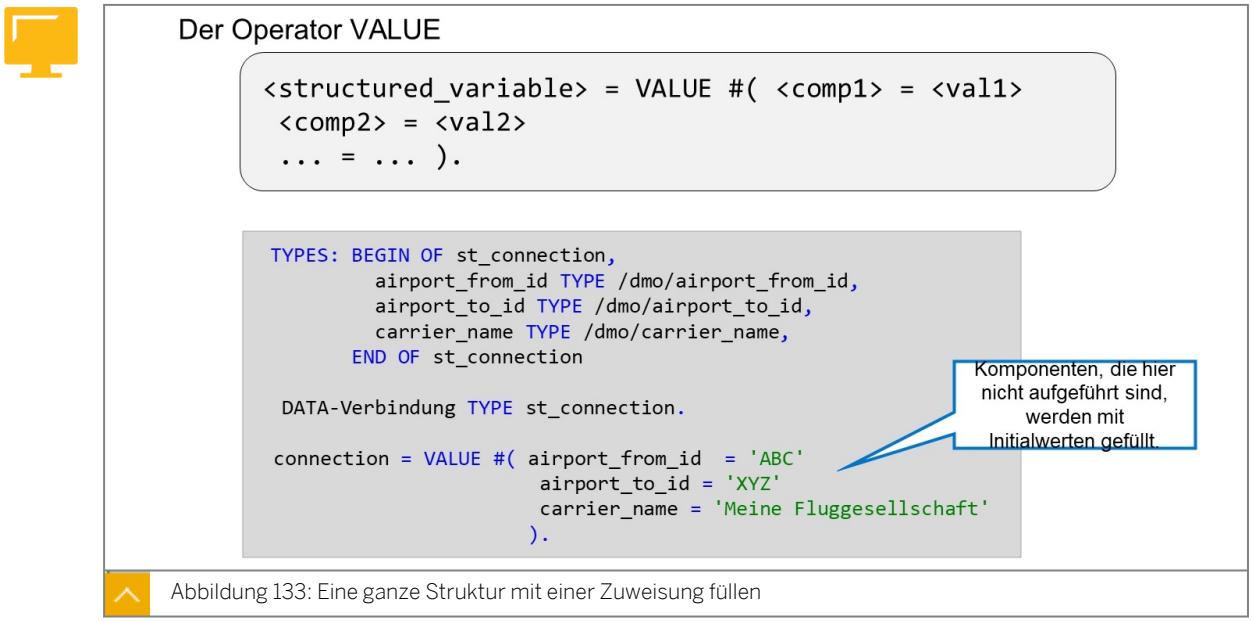


Abbildung 133: Eine ganze Struktur mit einer Zuweisung füllen

© Copyright. Alle Rechte vorbehalten.

Mit dem Ausdruck VALUE #( ) können einem strukturierten Datenobjekt auf elegante Art Werte zugewiesen werden.

Wenn Sie eine gesamte Struktur füllen möchten, können Sie jede Komponente wie im vorherigen Beispiel einzeln adressieren.

Sie können aber auch einen Ausdruck VALUE #( ) zum Füllen der Struktur verwenden. Der Ausdruck konstruiert eine Struktur, füllt sie mit Werten und weist die gefüllte Struktur einer Variablen zu, in diesem Fall `connection`. Das Rautenzeichen (#) weist die ABAP-Laufzeitumgebung an, eine Struktur mit demselben Typ wie die Zielvariable `connection` zu konstruieren. In den Klammern führen Sie die Komponenten der Struktur auf, die gefüllt werden sollen (es müssen nicht alle sein), und weisen ihnen einen Wert zu.

Wenn Sie eine Struktur auf diese Weise füllen, löscht das Laufzeitsystem alle vorhandenen Werte aus der Struktur, bevor sie wieder mit den Werten aus Ihrem Ausdruck gefüllt wird.

> Notiz:
> Eine Zuweisung in der Form `connection` = VALUE #( ). mit nur einem Leerzeichen zwischen den Klammern füllt alle Komponenten der Struktur mit dem typspezifischen Initialwert. Dies hat die gleiche Wirkung wie die Anweisung CLEAR `connection`.

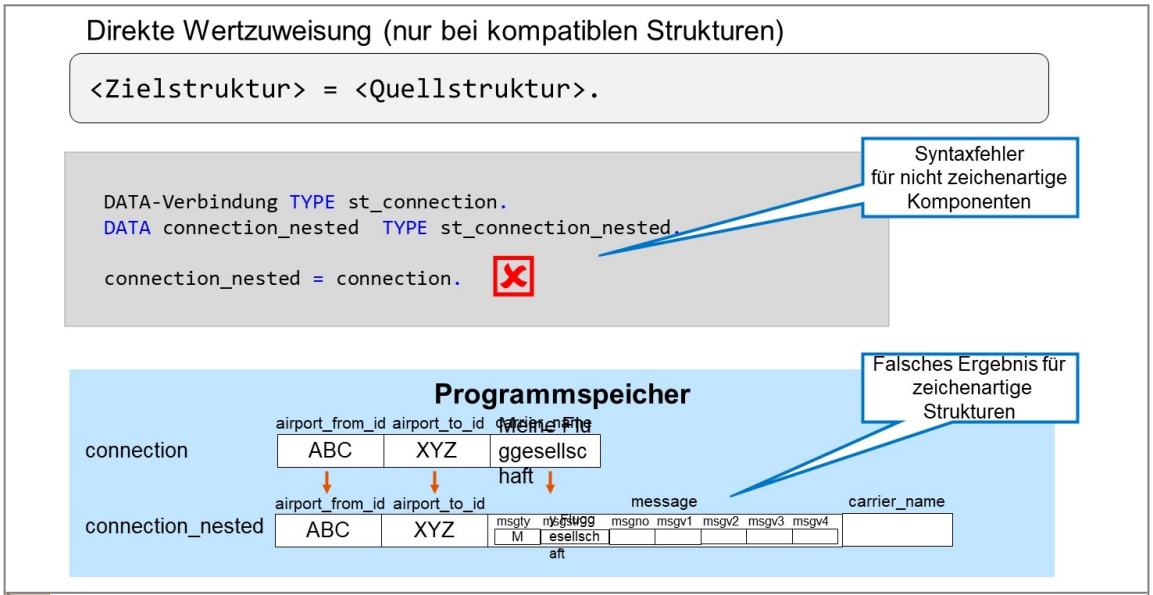


Abbildung 134: Wertzuweisung zwischen Strukturen

In ABAP dürfen Sie den Inhalt einer Struktur nur dann direkt in eine andere Struktur mit der Notation `<target structure> = <source structure>` kopieren, wenn die beiden Strukturtypen kompatibel sind. Dies ist in der Regel nur dann der Fall, wenn beide Strukturen vom selben Typ sind. Wenn die Strukturen unterschiedliche Typen haben, können zwei Fälle eintreten:

- Wenn eine der Strukturen eine nicht-zeichenartige Komponente an einer Stelle hat, an der die andere Struktur eine zeichenartige Komponente aufweist, führt die direkte Zuweisung zu einem Syntaxfehler.

© Copyright. Alle Rechte vorbehalten.

- Wenn beide Strukturen zeichenartig sind, d.h. beide Strukturen nur aus zeichenartigen Komponenten bestehen, ist eine direkte Zuweisung technisch möglich. In der Regel kommt es jedoch zu einem falschen Ergebnis.

Im Beispiel sind Quellstruktur und Zielstruktur zeichenartig. Daher ist eine direkte Zuordnung technisch möglich. Da sie jedoch nicht kompatibel sind, ist das Ergebnis falsch: Der Inhalt der Komponente `carrier_name` wird in die Komponente `message` in der Zielstruktur kopiert.

Notiz:
Da sich kein Syntaxfehler ergibt, müssen Sie besonders vorsichtig sein, wenn Sie mit nicht kompatiblen zeichenartigen Strukturen arbeiten.

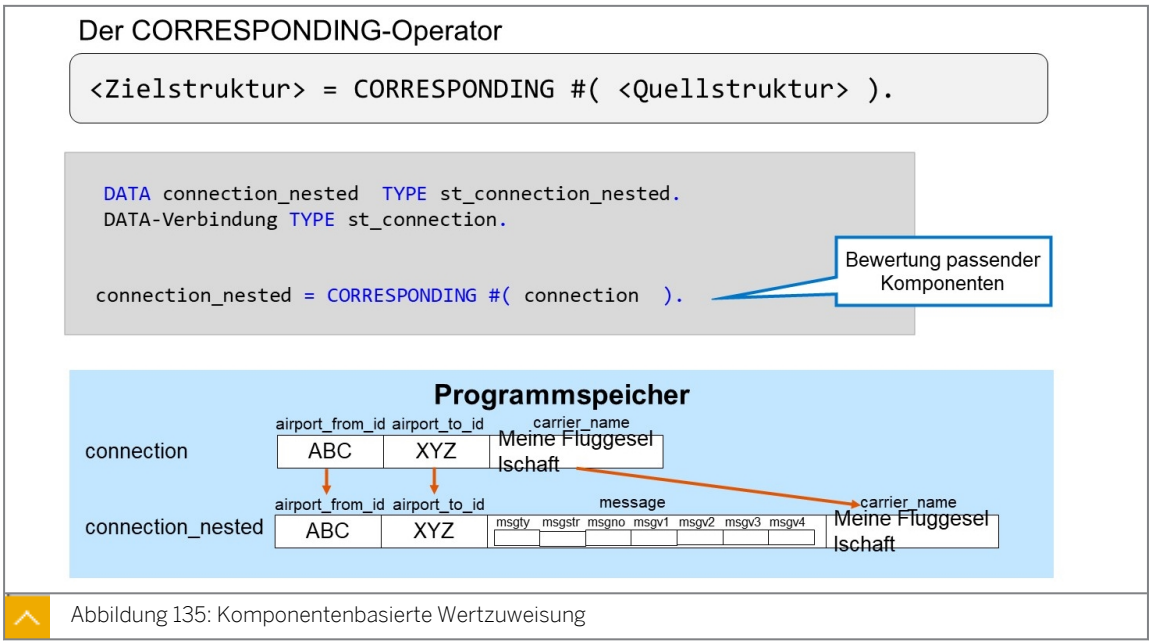


Abbildung 135: Komponentenbasierte Wertzuweisung

Wenn Sie Daten zwischen Strukturen kopieren, möchten Sie in der Regel Informationen aus einem Feld in das entsprechende Feld der Zielstruktur kopieren - `airport_from_id` nach `airport_from_id`, `airport_to_id` nach `airport_to_id` usw. Um dies in ABAP zu erreichen, verwenden Sie den Ausdruck CORRESPONDING. Dadurch werden Werte aus `<source_structure>` den entsprechenden, d.h. namensgleichen Komponenten von `<target_structure>` zugeordnet. Sie müssen die folgenden Punkte beachten:

- Die Felder müssen identische Namen haben.
- Die Komponenten müssen sich nicht in beiden Strukturen in der gleichen Position oder Reihenfolge befinden.
- Wenn die Felder unterschiedliche Typen aufweisen, versucht ABAP eine Typkonvertierung entsprechend dem vordefinierten Satz von Regeln.

Der CORRESPONDING -Ausdruck im Beispiel legt eine Struktur vom Typ `st_connection_nested` an. Der Struktur werden dann die Werte von gleichnamigen Komponenten aus der Struktur `connection` zugewiesen. Alle anderen Komponenten bleiben initial. Die so aufgebaute Struktur wird dann der `connection_nested` Struktur zugeordnet. Dadurch werden der Komponente `message` Initialwerte eingeschränkt zugeordnet.

© Copyright. Alle Rechte vorbehalten.

Strukturierte Datenobjekte in ABAP SQL

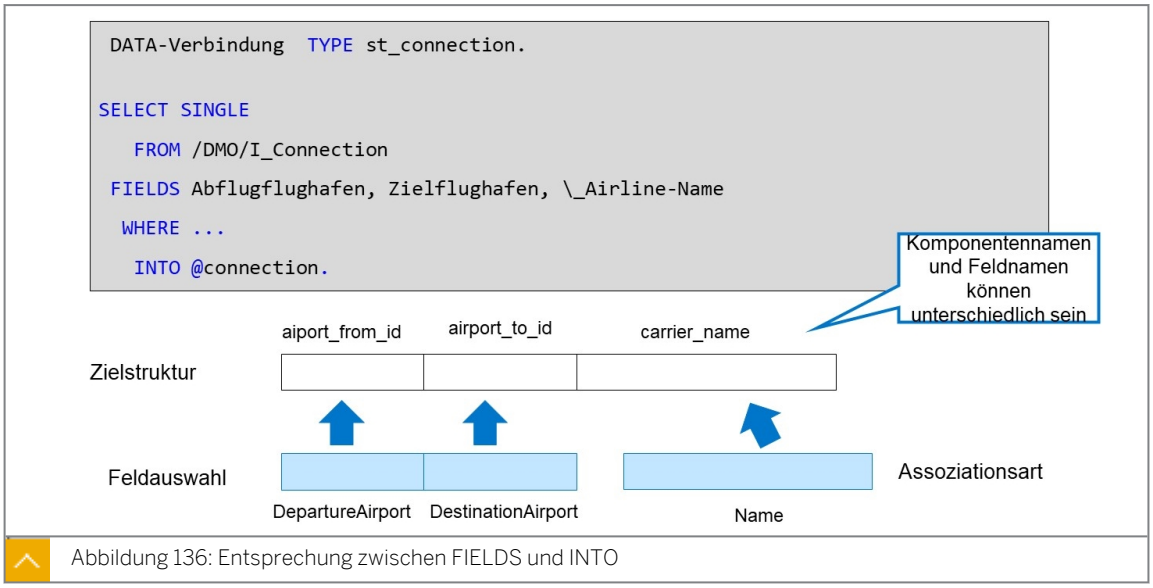


Abbildung 136: Entsprechung zwischen FIELDS und INTO

Die INTO-Klausel der SELECT-Anweisung funktioniert nur dann richtig, wenn die Anzahl und die Typen der Strukturkomponenten der Anzahl und den Typen, die in der FIELDS-Klausel festgelegt sind, entsprechen. Im oben gezeigten Beispiel kann die Anweisung nur funktionieren, wenn die Struktur connection drei Komponenten mit dem gleichen Typ und der gleichen Länge aufweist wie die in der FIELDS-Klausel aufgeführten Spalten DepartureAirport, DestinationAirport und _Airline-Name. Beachten Sie, dass die Namen in diesem Fall nicht identisch sein müssen – das System füllt die Zielstruktur von links nach rechts.

Notiz:
Wenn die Feldliste in der FIELDS-Klausel nicht mit der Struktur oder dem Tabellenzeilentyp in der INTO-Klausel übereinstimmt, kommt es zu einem Laufzeitfehler.

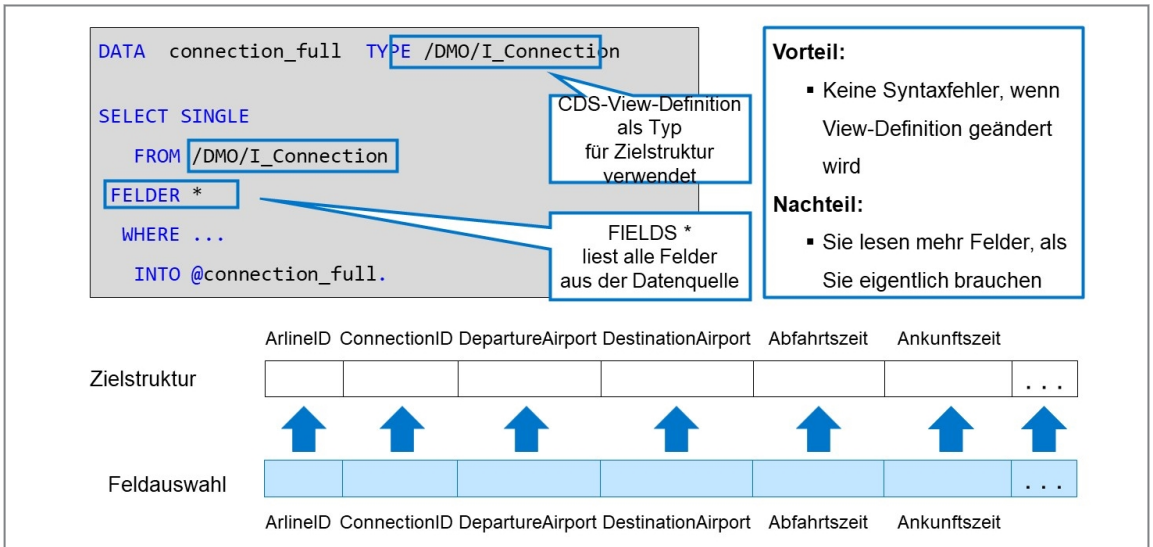


Abbildung 137: FIELDS * zum Lesen aller Felder aus einer Datenquelle

© Copyright. Alle Rechte vorbehalten.

Das Beispiel zeigt eine einfache Technik, um sicherzustellen, dass die Zielstruktur mit der Feldauswahl übereinstimmt:

- Die Zielstruktur ist mit der CDS-View /DMO/I_Connection typisiert, die die Datenquelle in der FROM-Klausel darstellt.
- Das Sternchen (*) nach dem Schlüsselwort FIELDS ist eine kurze Notation, um sicherzustellen, dass alle Felder der View Teil der Feldauswahl sind. Exponierte Assoziationen werden ignoriert.

> Notiz:
> Diese Technik steht auch beim direkten Lesen aus einer Datenbanktabelle zur Verfügung. Wie CDS-View-Definitionen können auch Definitionen von Datenbanktabellen als globale Strukturtypen in ABAP dienen.

Der Hauptvorteil dieser Technik besteht darin, dass die SELECT-Anweisung syntaktisch intakt bleibt, auch wenn Sie oder eine andere Person Änderungen an einer CDS-View oder Datenbanktabelle vornehmen. Der größte Nachteil besteht darin, dass immer alle Felder aus der Datenbank gelesen werden, unabhängig davon, ob sie tatsächlich benötigt werden oder nicht.

> Notiz:
> Verwenden Sie diese Technik nur für Views und Tabellen mit wenigen Feldern und nur dann, wenn Sie tatsächlich alle Felder benötigen. Wenn unnötig Daten aus der Datenbank gelesen werden, kommt es häufig zu Performanceproblemen.

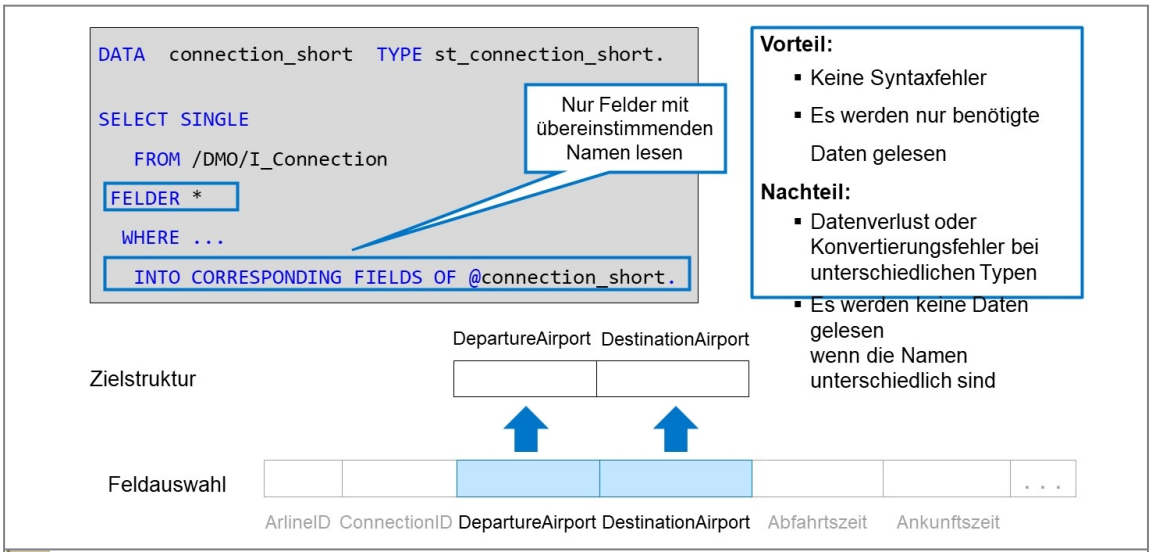


Abbildung 138: INTO CORRESPONDING FIELDS OF zum ausschließlichen Lesen von Feldern mit entsprechenden Namen

Eine weitere Möglichkeit, Syntaxfehler zu vermeiden, ist die Variante INTO CORRESPONDING FIELDS. Diese Variante hat die gleiche Wirkung wie der Operator CORRESPONDING #( ), den Sie bereits kennengelernt haben. Sie stellt sicher, dass Daten zwischen gleichnamigen Komponenten kopiert werden. Indem Sie den Strukturtyp Ihren Anforderungen entsprechend definieren, können Sie sicherstellen, dass nur die benötigten Daten gelesen werden.

© Copyright. Alle Rechte vorbehalten.

Zur Erinnerung: Nur die Namen müssen identisch sein. Um Probleme zu vermeiden, sollten Sie jedoch darauf achten, dass gleichnamige Komponenten kompatible Typen haben. Anderenfalls versucht das System, die Inhalte des Quellfelds in den Typen des Zielfelds zu konvertieren. Dies kann zu Datenverlust oder (behandelbaren) Laufzeitfehlern führen.

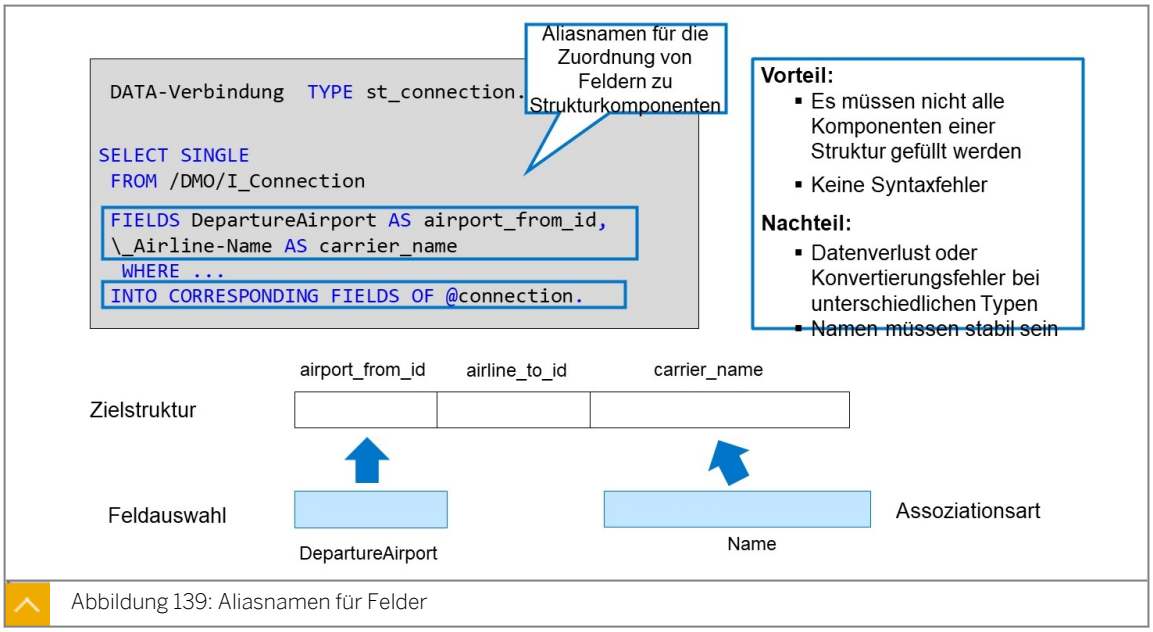


Abbildung 139: Aliasnamen für Felder

Wenn die Feldnamen in der Datenquelle und die Komponentennamen in der Zielstruktur nicht übereinstimmen, funktioniert die Kombination von FIELDS * und INTO CORRESPONDING FIELDS OF nicht.

Wenn Sie die Variante INTO CORRESPONDING FIELDS OF beibehalten möchten, können Sie für die ausgewählten Felder in der Feldliste Aliasnamen definieren. Fügen Sie dazu hinter dem Feldnamen den Zusatz AS sowie anschließend den Aliasnamen hinzu. Im Beispiel lautet der Aliasname für das View-Feld DepartureAirport „airport_from_id“ und der Aliasname für den Pfadausdruck _Airline-Name „carrier_name“. Basierend auf diesen Aliasnamen identifiziert INTO CORRESPONDING FIELDS OF korrekt die Strukturkomponente, in der die abgerufenen Daten gespeichert werden sollen.

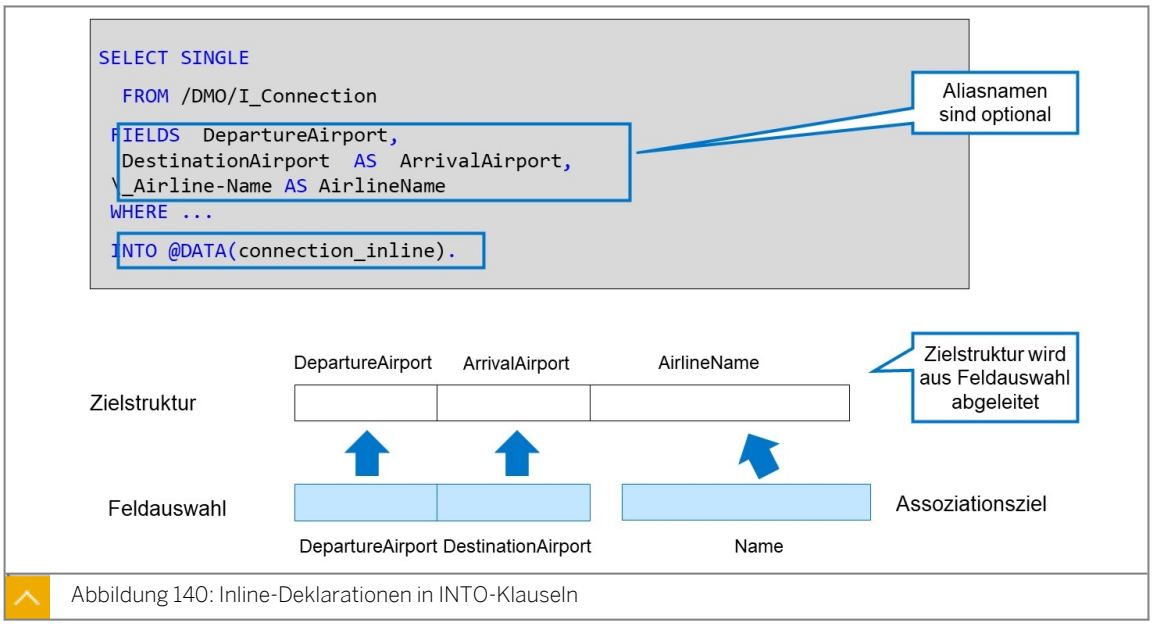


Abbildung 140: Inline-Deklarationen in INTO-Klauseln

© Copyright. Alle Rechte vorbehalten.

Konflikte zwischen der Feldauswahl und der Zielstruktur lassen sich am einfachsten durch eine Inline-Deklaration in der INTO -Klausel vermeiden. Die Reihenfolge, der Typ und der Name der inline deklarierten Struktur werden aus der FIELDS-Klausel abgeleitet. Daher passt die Zielstruktur immer zur Feldauswahl.

> Notiz:
> Inline-Deklarationen werden nur hinter INTO unterstützt. Hinter INTO CORRESPONDING FIELDS OF können keine Inline-Deklarationen verwendet werden.

Im Beispiel gibt es keinen Alias für das Feld `DepartureAirport`. Der Name des Feldes wird als Komponentenname in der Struktur `connection_inline` verwendet. Das Feld `DestinationAirport`sowie der Pfadausdruck `\_Airline-Name` haben optionale Aliasnamen (`ArrivalAirport` und `AirlineName`). Hier werden die Aliasnamen als Komponentennamen verwendet.

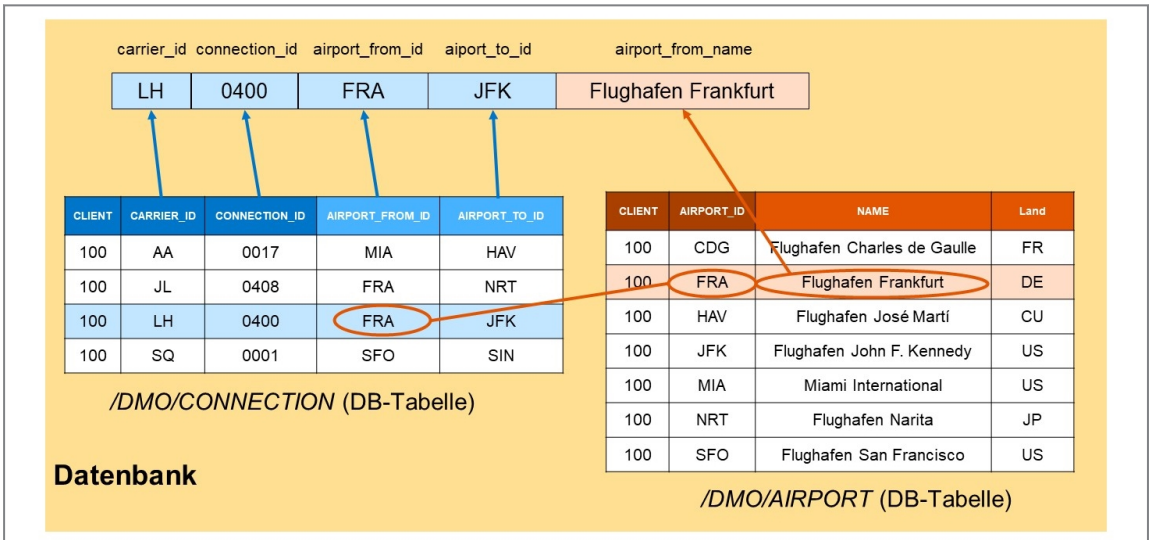


Abbildung 141: Daten aus verschiedenen Quellen kombinieren (Joins)

Bei der Arbeit mit einer relationalen Datenbank tritt häufig das Problem auf, dass zugehörige Daten aus verschiedenen Datenbanktabellen gelesen werden müssen. Wir hatten bereits gelernt, dass sich diese Aufgabe durch Assoziationen in CDS-Views elegant lösen lässt.

Wenn keine CDS-View mit geeigneten Assoziationen vorhanden ist, können stattdessen SQL-Joins implementiert werden. Das obige Beispiel veranschaulicht das Prinzip von Joins:

Wir interessieren uns für Flugverbindungen und die Flughäfen, die dadurch miteinander verbunden werden. Die dreistelligen IDs der Flughäfen finden Sie in der Datenbanktabelle /DMO/CONNECTION. Die vollständigen Namen der Flughäfen sind in der Datenbanktabelle /DMO/AIRPORT gespeichert.

Um eine Verbindung mit dem Namen des Abflughafens abzurufen, werden in einer SELECT-Anweisung die Verbindungsdaten aus der Datenbanktabelle /DMO/CONNECTION gelesen und mit der Datenbanktabelle /DMO/AIRPORT verknüpft.

Ein Join besteht aus folgenden Bestandteilen:

Datenquellen

© Copyright. Alle Rechte vorbehalten.

Die Datenbanktabellen und Views, die miteinander verknüpft werden sollen. Ein einzelner Join verknüpft immer eine linke Datenquelle mit einer rechten Datenquelle. Im obigen Beispiel ist die Tabelle /DMO/CONNECTION die linke Datenquelle und die Tabelle /DMO/AIRPORT die rechte Datenquelle. ABAP SQL unterstützt auch Joins von Joins (geschachtelte Joins).

Join-Bedingung

Die Join-Bedingung gibt an, welche Datensätze der rechten Datenquelle zu einem Datensatz aus der linken Datenquelle gehören. Im obigen Beispiel wird der zugehörige Abflughafen durch den Wert in den Spalten CLIENT und AIRPORT_ID identifiziert. Die Join-Bedingung lautet:

/DMO/CONNECTION~CLIENT = /DMO/AIRPORT~CLIENT

AND

/DMO/CONNECTION~AIRPORT_FROM_ID = /DMO/AIRPORT~AIRPORT_ID

Join-Typ

Der Join-Typ hat Einfluss auf das Ergebnis, wenn eine der Datenquellen keine übereinstimmenden Datensätze enthält. ABAP SQL unterstützt derzeit INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN und CROSS JOIN. Der häufigste Join-Typ ist ein LEFT OUTER JOIN.

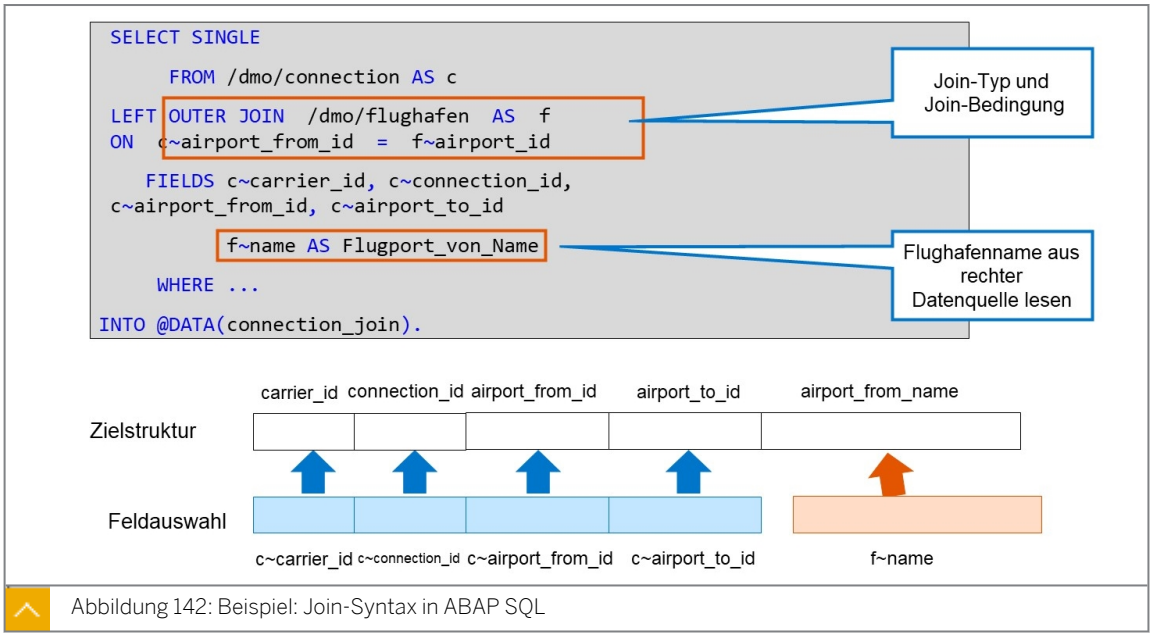


Abbildung 142: Beispiel: Join-Syntax in ABAP SQL

Die Abbildung zeigt die ABAP-SQL-Syntax für einen Join. In der FROM-Klausel wird der Join-Typ durch die Schlüsselwörter LEFT OUTER JOIN zwischen der linken Datenquelle /dmo/connection und der rechten Datenquelle /dmo/airport angegeben. Über die Syntax werden die Aliasnamen c und f für die Datenquellen eingebracht. Aliasnamen für Datenquellen sind optional, es sei denn, eine Datenquelle kommt mehr als einmal im Join vor.

Die Join-Bedingung folgt auf das Schlüsselwort ON. Das Trennzeichen zwischen der Datenquelle oder ihrem Alias und dem Feld ist das Tildezeichen (~).

© Copyright. Alle Rechte vorbehalten.

Notiz:
In ABAP SQL ist es nicht notwendig, die Mandantenfelder zu erwähnen. Sie werden von der Datenbankschnittstelle zur Join-Bedingung hinzugefügt, bevor die Anweisung an die Datenbank gesendet wird. Wenn die FROM-Klausel einen Join definiert, können Sie Felder aus beiden Datenquellen in den FIELDS- und WHERE-Klauseln verwenden.

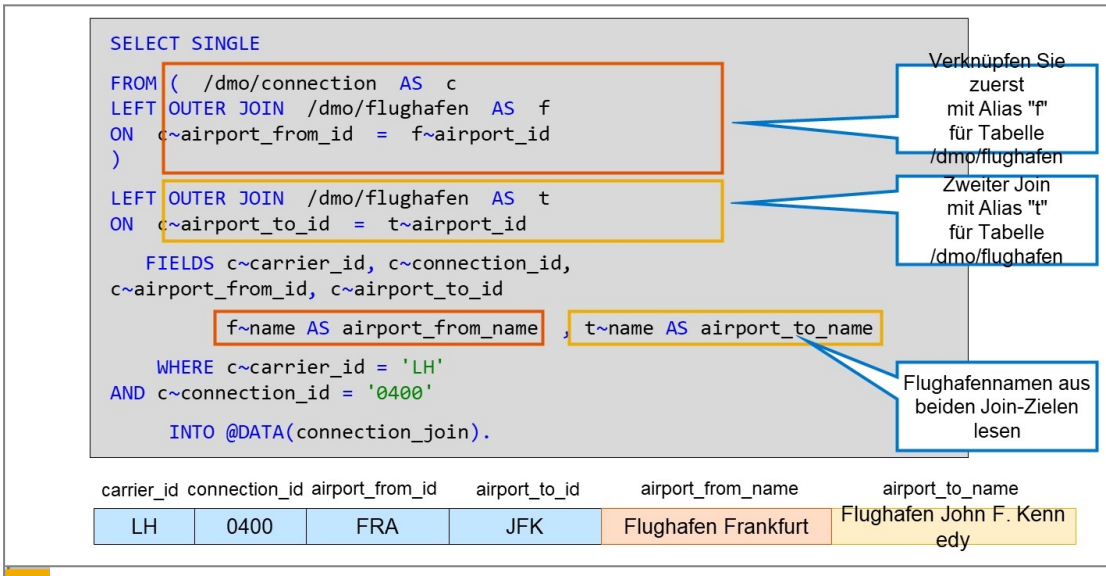


Abbildung 143: Beispiel: Geschachtelter Join

In diesem Beispiel liest die SELECT -Anweisung nicht nur den Namen des Abflughafens, sondern auch den Namen des Zielflughafens. Hierfür definiert die FROM-Klausel einen verschachtelten Join:

Der erste Join ist ein Left Outer Join der Tabellen /dmo/connection und /dmo/airport, der den Alias „f“ (wie „from“) für die rechte Datenquelle einführt. Dieser erste Join wird dann als linke Datenquelle für einen zweiten Left Outer Join verwendet, der die Tabelle /dmo/airport als rechte Datenquelle hat. Beachten Sie, dass in diesem Fall der Alias „t“ (für „to“) entscheidend ist, um dieses Erscheinungsbild der Tabelle /dmo/airport von der vorherigen zu unterscheiden.

Die FIELDS-Klausel listet die Flughafennamen aus beiden Datenquellen auf und führt die Aliase airport_from_name und airport_to_name ein, um sie voneinander zu unterscheiden.

Hinweis:
Die Klammern um den ersten Join sind optional. Wenn sie ausgelassen werden, werden die Joins in der FROM-Klausel von links nach rechts ausgewertet.

© Copyright. Alle Rechte vorbehalten.

Kapitel 5
Übung 14

Ein strukturiertes Datenobjekt verwenden

In dieser Übung deklarieren Sie ein strukturiertes Attribut, füllen es mittels einer SELECT-Anweisung und greifen auf die Strukturkomponenten zu.

Vorlage:

/LRN/CL_S4D400_DBS_CDS (globale Klasse)

Lösung:

/LRN/CL_S4D400_STS_STRUCTURE (globale Klasse)

Aufgabe 1: Vorlage kopieren
Kopieren Sie die Vorlageklasse. Alternativ können Sie Ihre Lösung aus der vorherigen Übung kopieren.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_DBS_CDS`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_STRUCTURE`**, wobei ## für Ihre Gruppennummer steht).

Aufgabe 2: Ein strukturiertes Datenobjekt deklarieren
Deklarieren Sie in der lokalen Klasse ein strukturiertes Attribut **`details`**, um die skalaren Attribute **`airport_from_id`**, **`airport_to_id`** und **`carrier_name`** zu ersetzen. Beginnen Sie mit der Definition eines privaten Strukturtyps **`st_details`** in der lokalen Klasse.

1. Wechseln Sie in die lokale Klasse **`lcl_connection`**.
2. Definieren Sie einen privaten Strukturtyp **`st_details`** mit folgenden Komponenten:

   Tabelle 4: Komponenten des Strukturtyps st_details:

Komponentenname	Datentyp
`DepartureAirport`	**`/dmo/airport_from_id`**
`DestinationAirport`	**`/dmo/airport_to_id`**
`AirlineName`	**`/dmo/carrier_name`**

3. Kommentieren oder entfernen Sie die Deklaration der Attribute **`airport_from_id`**, **`airport_to_id`** und **`carrier_name`**.
4. Deklarieren Sie ein neues privates Instanzattribut **`details`**, und typisieren Sie es mit dem Strukturtyp **`st_details`**.

Aufgabe 3: Auf Strukturkomponenten zugreifen
Verwenden Sie die Komponenten des strukturierten Attributs **`details`** in der Methode **`get_output`**.

© Copyright. Alle Rechte vorbehalten.

1. Passen Sie die Implementierung der Methode **`get_output`** an. Ersetzen Sie den Zugriff auf die Attribute **`airport_from_id`**, **`airport_to_id`** und **`carrier_name`** durch die entsprechende Komponente des Attributs **`details`**.

Hinweis:
Geben Sie die Komponentennamen nicht manuell ein! Nachdem Sie den Strukturkomponentenselektor (-) eingegeben haben, drücken Sie **`Strg + Leertaste`**, um eine Liste aller Komponenten aufzurufen.

Aufgabe 4: Das strukturierte Attribut in der SELECT-Anweisung füllen
Verwenden Sie das strukturierte Attribut als Ziel der SELECT -Anweisung in der Methode **`constructor`**.

1. Passen Sie die SELECT-Anweisung in der Implementierung der Methode **`constructor`** an. Ersetzen Sie die Liste der Datenobjekte in der INTO-Klausel durch das strukturierte Attribut **`details`**.
2. Optional: Verwenden Sie die Syntaxvariante `INTO CORRESPONDING FIELDS OF @details.`.
3. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe. Prüfen Sie, ob die Ausgabe Daten für alle Attribute anzeigt.

© Copyright. Alle Rechte vorbehalten.

Kapitel 5
Lösung 14

Ein strukturiertes Datenobjekt verwenden

In dieser Übung deklarieren Sie ein strukturiertes Attribut, füllen es mittels einer SELECT-Anweisung und greifen auf die Strukturkomponenten zu.

Vorlage:

/LRN/CL_S4D400_DBS_CDS (globale Klasse)

Lösung:

/LRN/CL_S4D400_STS_STRUCTURE (globale Klasse)

Aufgabe 1: Vorlage kopieren
Kopieren Sie die Vorlageklasse. Alternativ können Sie Ihre Lösung aus der vorherigen Übung kopieren.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_DBS_CDS`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_STRUCTURE`**, wobei ## für Ihre Gruppennummer steht).
   a) Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_DBS_CDS`**.
   b) Verknüpfen Sie die View *Project Explorer* mit dem Editor.
   c) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Klasse **`/LRN/CL_S4D400_DBS_CDS`**, um das Kontextmenü zu öffnen.
   d) Wählen Sie im Kontextmenü *Duplicate ...*
   e) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_STRUCTURE`** ein, wobei ## für Ihre Gruppennummer steht.
   f) Passen Sie die Beschreibung an, und wählen Sie *Next*.
   g) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

Aufgabe 2: Ein strukturiertes Datenobjekt deklarieren
Deklarieren Sie in der lokalen Klasse ein strukturiertes Attribut **`details`**, um die skalaren Attribute **`airport_from_id`**, **`airport_to_id`** und **`carrier_name`** zu ersetzen. Beginnen Sie mit der Definition eines privaten Strukturtyps **`st_details`** in der lokalen Klasse.

1. Wechseln Sie in die lokale Klasse **`lcl_connection`**.
   a) Wählen Sie in der globalen Klasse *Local Types*.
2. Definieren Sie einen privaten Strukturtyp **`st_details`** mit folgenden Komponenten:

   Tabelle 4: Komponenten des Strukturtyps st_details:

Komponentenname	Datentyp
`DepartureAirport`	**`/dmo/airport_from_id`**
`DestinationAirport`	**`/dmo/airport_to_id`**

© Copyright. Alle Rechte vorbehalten.

Komponentenname	Datentyp
AirlineName	**/dmo/carrier_name**

a) Fügen Sie nach der Zeile `PRIVATE SECTION.` den folgenden Code hinzu:

```
  TYPES:
    BEGIN OF st_details,
      DepartureAirport   TYPE /dmo/airport_from_id,
      DestinationAirport TYPE   /dmo/airport_to_id,
      AirlineName        TYPE   /dmo/carrier_name,
    END OF st_details.
```

3. Kommentieren oder entfernen Sie die Deklaration der Attribute **airport_from_id**, **airport_to_id** und **carrier_name**.
   a) Wählen Sie die Zeilen mit den drei DATA-Anweisungen aus.
   b) Drücken Sie **Strg + <**, um vor jeder ausgewählten Zeile ein Sternchen (*) hinzuzufügen.
4. Deklarieren Sie ein neues privates Instanzattribut **details**, und typisieren Sie es mit dem Strukturtyp **st_details**.
   a) Passen Sie den Quelltext wie folgt an:

```
*    DATA airport_from_id TYPE /dmo/airport_from_id.
*    DATA airport_to_id   TYPE /dmo/airport_to_id.
*
*    DATA carrier_name    TYPE /dmo/carrier_name.

    DATA details TYPE st_details.
```

Aufgabe 3: Auf Strukturkomponenten zugreifen

Verwenden Sie die Komponenten des strukturierten Attributs **details** in der Methode **get_output**.

1. Passen Sie die Implementierung der Methode **get_output** an. Ersetzen Sie den Zugriff auf die Attribute **airport_from_id**, **airport_to_id** und **carrier_name** durch die entsprechende Komponente des Attributs **details**.

> Hinweis:
> Geben Sie die Komponentennamen nicht manuell ein! Nachdem Sie den Strukturkomponentenselektor (-) eingegeben haben, drücken Sie **Strg + Leertaste**, um eine Liste aller Komponenten aufzurufen.

   a) Navigieren Sie zur Implementierung der Methode **get_output**.
   b) Passen Sie die APPEND-Anweisungen wie folgt an:

```
*    APPEND |--------------------------------|             TO r_output.
*    APPEND |Carrier:     { carrier_id } { carrier_name }| TO r_output.
*    APPEND |Connection:  { connection_id   }|             TO r_output.
*    APPEND |Departure:   { airport_from_id }|             TO r_output.
```


© Copyright. Alle Rechte vorbehalten.

```
*    APPEND |Destination: { airport_to_id   }|             TO r_output.

    APPEND |--------------------------------|                    TO
r_output.
    APPEND |Carrier:     { carrier_id } { details-airlinename }| TO
r_output.
    APPEND |Connection:  { connection_id   }|                    TO
r_output.
    APPEND |Departure:   { details-departureairport     }|       TO
r_output.
    APPEND |Destination: { details-destinationairport   }|       TO
r_output.
```

Aufgabe 4: Das strukturierte Attribut in der SELECT-Anweisung füllen
Verwenden Sie das strukturierte Attribut als Ziel der SELECT -Anweisung in der Methode **`constructor`**.

1. Passen Sie die SELECT-Anweisung in der Implementierung der Methode **`constructor`** an. Ersetzen Sie die Liste der Datenobjekte in der INTO-Klausel durch das strukturierte Attribut **`details`**.
   a) Navigieren Sie zur Implementierung der Methode **`constructor`**.
   b) Passen Sie die SELECT-Anweisung wie folgt an:

```
SELECT SINGLE
      FROM /DMO/I_Connection
    FIELDS DepartureAirport, DestinationAirport, \_Airline-Name
     WHERE AirlineID    = @i_carrier_id
       AND ConnectionID = @i_connection_id
*       INTO ( @airport_from_id, @airport_to_id, @carrier_name  ).
      INTO @details.
```

2. Optional: Verwenden Sie die Syntaxvariante `INTO CORRESPONDING FIELDS OF @details.`.
   a) Ersetzen Sie `INTO @details.` in der SELECT-Anweisung durch den folgenden Code:

```
  INTO CORRESPONDING FIELDS OF @details.
```

   b) Fügen Sie den Aliasnamen **`AirlineName`** für den Pfadausdruck hinzu.
   c) Die SELECT-Anweisung sollte nun wie folgt aussehen:

```
SELECT SINGLE
      FROM /DMO/I_Connection
    FIELDS DepartureAirport, DestinationAirport, \_Airline-Name as
AirlineName
     WHERE AirlineID    = @i_carrier_id
       AND ConnectionID = @i_connection_id
      INTO CORRESPONDING FIELDS OF @details.
```

3. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe. Prüfen Sie, ob die Ausgabe Daten für alle Attribute anzeigt.
   a) Drücken Sie **`Strg + F3`**, um die Klasse zu aktivieren.
   b) Drücken Sie **`F9`**, um die Klasse auszuführen.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Arbeiten Sie mit strukturierten Datenobjekten.
- Verwenden Sie strukturierte Datenobjekte in ABAP SQL.

© Copyright. Alle Rechte vorbehalten.

Kapitel 5

Überprüfung des Lernerfolgs

1. Eine Variable wird mit der Anweisung DATA struct TYPE <Typ> deklariert. Womit können Sie eine Struktur deklarieren?

   Wählen Sie die richtigen Antworten.

   - ☐ **A** Datenbanktabelle
   - ☐ **B** Datenelement
   - ☐ **C** CDS-View
   - ☐ **D** Eingebauter ABAP-Typ

2. Eine Struktur struct enthält eine Komponente comp. Wie adressieren Sie die Komponente?

   Wählen Sie die richtige Antwort.

   - ☐ **A** struct.comp
   - ☐ **B** struct-comp
   - ☐ **C** struct->comp
   - ☐ **D** struct=>comp

3. Was müssen Sie beim Kopieren von Daten zwischen Strukturen mit dem Ausdruck CORRESPONDING beachten?

   Wählen Sie die richtige Antwort.

   - ☐ **A** Es werden nur Komponenten mit identischen Namen kopiert.
   - ☐ **B** Komponenten werden entsprechend ihrer Positionen kopiert.
   - ☐ **C** Es werden nur Komponenten mit identischen Typen kopiert.

© Copyright. Alle Rechte vorbehalten.

Kapitel 5

Überprüfung des Lernerfolgs - Antworten

1. Eine Variable wird mit der Anweisung DATA struct TYPE <Typ> deklariert. Womit können Sie eine Struktur deklarieren?

   Wählen Sie die richtigen Antworten.

   - [X] A Datenbanktabelle
   - [ ] B Datenelement
   - [X] C CDS-View
   - [ ] D Eingebauter ABAP-Typ

   Das ist richtig.

2. Eine Struktur struct enthält eine Komponente comp. Wie adressieren Sie die Komponente?

   Wählen Sie die richtige Antwort.

   - [ ] A struct.comp
   - [X] B struct-comp
   - [ ] C struct->comp
   - [ ] D struct=>comp

   Das ist richtig.

3. Was müssen Sie beim Kopieren von Daten zwischen Strukturen mit dem Ausdruck CORRESPONDING beachten?

   Wählen Sie die richtige Antwort.

   - [X] A Es werden nur Komponenten mit identischen Namen kopiert.
   - [ ] B Komponenten werden entsprechend ihrer Positionen kopiert.
   - [ ] C Es werden nur Komponenten mit identischen Typen kopiert.

© Copyright. Alle Rechte vorbehalten.

KAPITEL 6 Mit komplexen internen Tabellen arbeiten

Lektion 1

Eine komplexe interne Tabelle deklarieren 267

Lektion 2

Mit komplexen internen Tabellen arbeiten 274

Übung 15 : Komplexe interne Tabelle verwenden 287

LERNZIELE

- Deklarieren Sie eine komplexe interne Tabelle.
- Komplexe interne Tabellen mit Daten füllen.
- Zugriff auf den Inhalt komplexer interner Tabellen.
- Verwenden Sie komplexe interne Tabellen in ABAP SQL.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 6
Lektion 1

Eine komplexe interne Tabelle deklarieren

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Deklarieren Sie eine komplexe interne Tabelle.

Eigenschaften komplexer interner Tabellen

TYPE TABLE OF zur Deklaration einer internen Tabelle

```
DATA <interne_Tabelle> TYPE TABLE OF <Zeilentyp>.
```

Beispiel für ABAP-Quelltext

```
DATA Zahl   TYPE i  VALUE 1234.
DATA Zahlen  TYPE TABLE OF i.

APPEND 1234  TO Zahlen.
APPEND 456 TO Zahlen.
APPEND 12 TO Zahlen.
APPEND 8976 TO Zahlen.
...
```

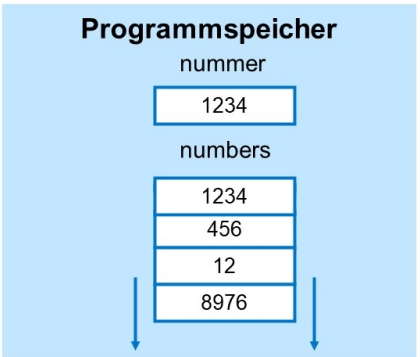


Abbildung 144: Wiederholung: einfache interne Tabellen

Die bisher verwendeten internen Tabellen hatten einen skalaren Datentyp als Zeilentyp. In dem hier in den Abbildungen gezeigten Beispiel ist der Zeilentyp der internen Tabelle numbers der eingebaute ABAP-Typ I.

Solche internen Tabellen werden als „einfache interne Tabellen“ bezeichnet.

© Copyright. Alle Rechte vorbehalten.

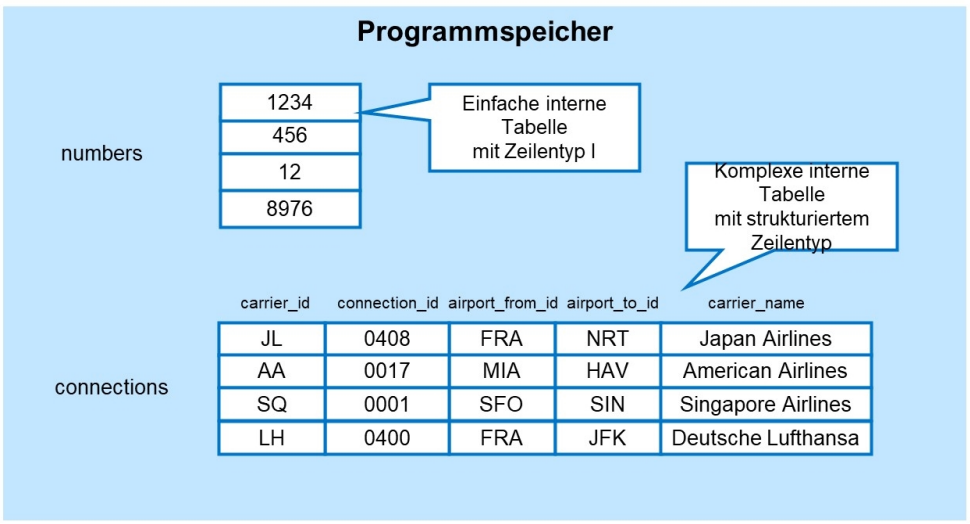


Abbildung 145: Interne Tabellen mit strukturiertem Zeilentyp

Wir sprechen von einer komplexen internen Tabelle, wenn der Zeilentyp ein strukturierter Datentyp ist.

Während eine einfache interne Tabelle nur eine namenlose Spalte hat, besteht eine komplexe interne Tabelle aus mehreren Spalten. Jede davon weist dabei jeweils den Namen und den Typ der entsprechenden Komponente des strukturierten Zeilentyps auf. Im Beispiel ist der Zeilentyp der internen Tabelle `connections` ein strukturierter Typ mit fünf Komponenten: `carrier_id`, `connection_id`, `airport_from_id`, `airport_to_id` und `carrier_name`. Folglich hat die interne Tabelle `connections` fünf Spalten mit diesen Namen.

Notiz:
Die Spalten in den Beispielen in der Abbildung *„Erinnerung: Einfache interne Tabellen“* und *„Interne Tabellen mit strukturiertem Zeilentyp“* haben alle skalare Typen. Generell könnte eine Spalte einer internen Tabelle auch vom strukturierten Typ sein oder sogar einen Tabellentyp haben. Im letzteren Fall spricht man von einer geschachtelten internen Tabelle.

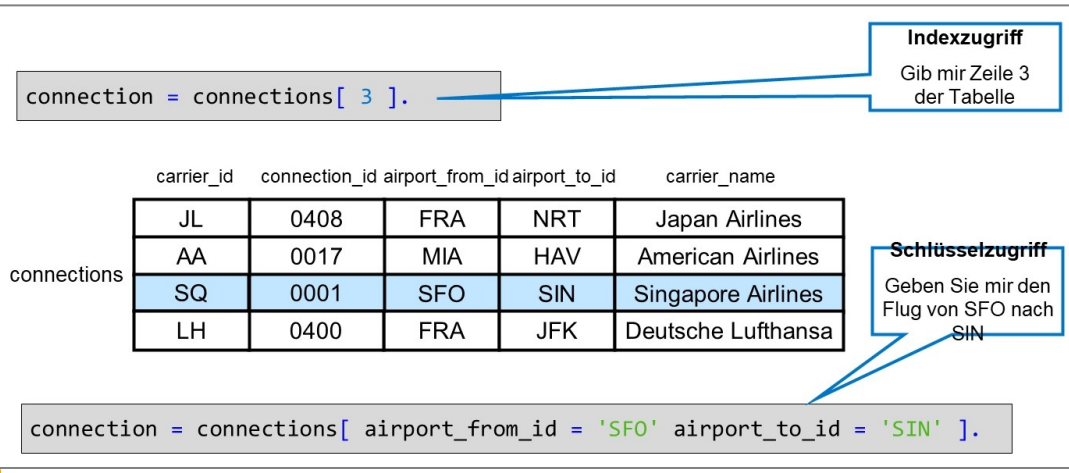


Abbildung 146: Interne Tabellen adressieren – Indexzugriff und Schlüsselzugriff

© Copyright. Alle Rechte vorbehalten.

Bisher wurden die Zeilen einer internen Tabelle durch ihre Position adressiert. Dies wird als „Indexzugriff“ bezeichnet.

Mit den benannten Spalten einer komplexen internen Tabelle wird der Schlüsselzugriff wichtiger. Bei einem Schlüsselzugriff wird eine Zeile der internen Tabelle über die Suche nach bestimmten Werten in bestimmten Spalten adressiert. Die Spalten, in denen Sie suchen, können beliebige Spalten der internen Tabelle sein.

Der Indexzugriff auf eine interne Tabelle erfolgt immer sehr schnell, auch wenn die interne Tabelle viele Zeilen enthält. Der Schlüsselzugriff kann jedoch sehr langsam werden, wenn die Tabelle viele Zeilen enthält. Die Auswahl der richtigen Zugriffsart für die interne Tabelle kann die Performance eines Schlüsselzugriffs verbessern.

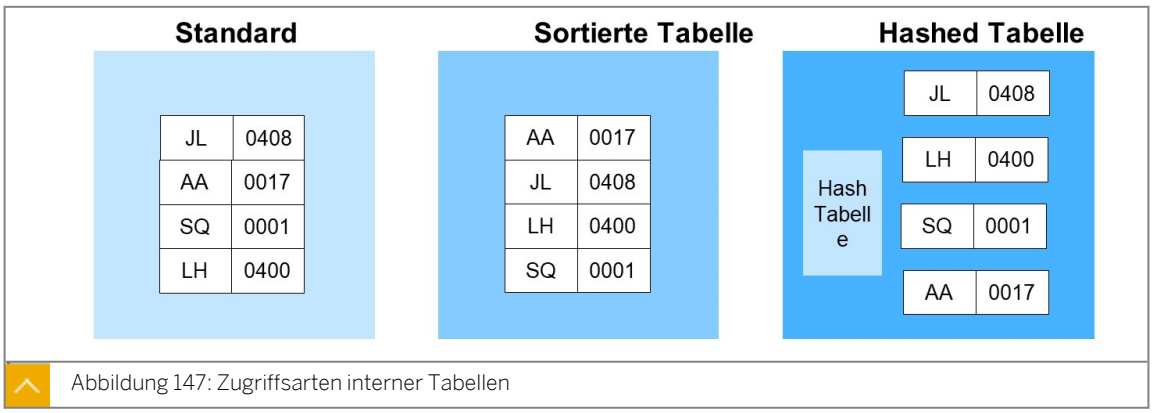


Abbildung 147: Zugriffsarten interner Tabellen

Jede interne Tabelle besitzt eine der folgenden drei Zugriffsarten. Die Zugriffsart legt fest, auf welche Weise Daten in der Tabelle gespeichert werden und, je nachdem, wie das System die Tabelle beim Lesen der Daten durchsucht.

Es gibt folgende Tabellentypen:

Standardtabelle

Bei einer Standardtabelle erfolgt das Speichern der Inhalte in keiner bestimmten Reihenfolge. Neue Einträge werden standardmäßig am Tabellenende hinzugefügt. Um Daten nach Schlüsseln abzurufen, muss das System die Tabelle sequentiell durchsuchen. Bei großen Tabellen kann dies zu langen Antwortzeiten führen. Die bisher verwendeten einfachen internen Tabellen waren Standardtabellen.

Sortierte Tabellen

In einer sortierten Tabelle werden die Inhalte entsprechend den Schlüsselfeldern stets in aufsteigender Reihenfolge sortiert. Wenn Sie einen neuen Datensatz in die Tabelle einfügen, stellt das System sicher, dass dies an der richtigen Position erfolgt. Da die Daten immer sortiert sind, erfolgt das Abrufen von Datensätzen durch das System effizienter als bei Standardtabellen (sofern Sie bestimmte Regeln einhalten).

Hash-Tabelle

Hash-Tabellen werden durch einen speziellen Hash-Algorithmus verwaltet. Auf diese Weise ist sichergestellt, dass das System Datensätze selbst bei extrem großen Tabellen sehr schnell abrufen kann. Diese Performancesteigerung funktioniert jedoch nur in ganz bestimmten Fällen.

© Copyright. Alle Rechte vorbehalten.

Notiz:
In dieser Lektion konzentrieren wir uns auf Standardtabellen. Weitere Informationen zu sortierten Tabellen und Hash-Tabellen finden Sie in der ABAP-Dokumentation.

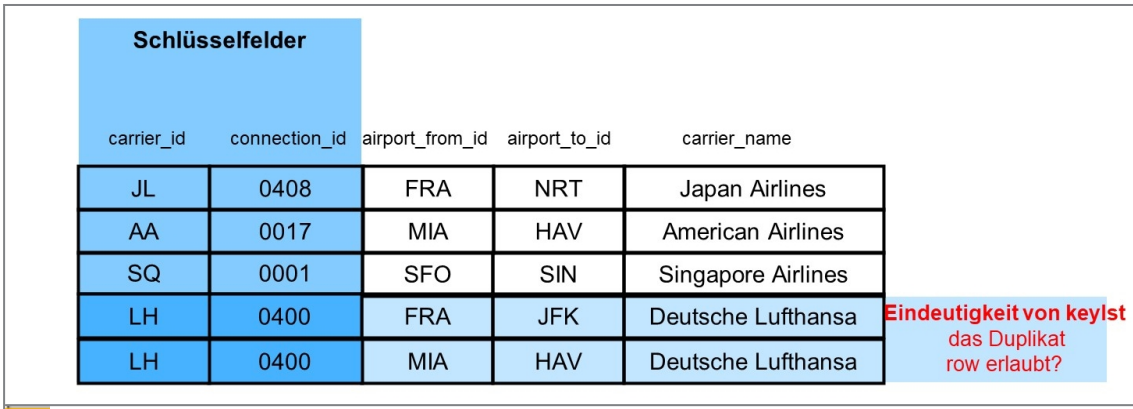


Abbildung 148: Schlüsseldefinition einer internen Tabelle

Jede interne Tabelle besitzt einen Schlüssel. Bei Standardtabellen spielt der Schlüssel keine besonders große Rolle. Bei sortierten Tabellen und Hash-Tabellen ist der Schlüssel dagegen sehr wichtig, da er bestimmt, auf welche Weise die Daten in der Tabelle verwaltet werden. Grundsätzlich sind sortierte Tabellen und Hash-Tabellen nur für Schlüsselzugriffe schneller, die alle oder zumindest eine Teilmenge der Schlüsselfelder adressieren.

Ein weiteres Attribut des Tabellenschlüssels ist seine Eindeutigkeit. Manchmal kann es erforderlich sein, doppelte Einträge in einer internen Tabelle zuzulassen. In anderen Fällen müssen Sie sicherstellen, dass der Schlüssel eindeutig ist. Hier gelten folgende Regeln:

- In Standardtabellen sind Dubletten immer zulässig.
- In Hash-Tabellen sind Dubletten niemals zulässig.
- Bei sortierten Tabellen legen Sie in der Definition fest, ob der Schlüssel eindeutig sein soll oder nicht.

Notiz:
Interne Tabellen können auch Sekundärschlüssel haben. Mit Sekundärschlüsseln kann die Performance von Schlüsselzugriffen auf interne Tabellen, die unterschiedliche Feldkombinationen verwenden, verbessert werden. Weitere Informationen finden Sie in der Dokumentation zur ABAP-Syntax.

© Copyright. Alle Rechte vorbehalten.

Komplexe Tabellentypen

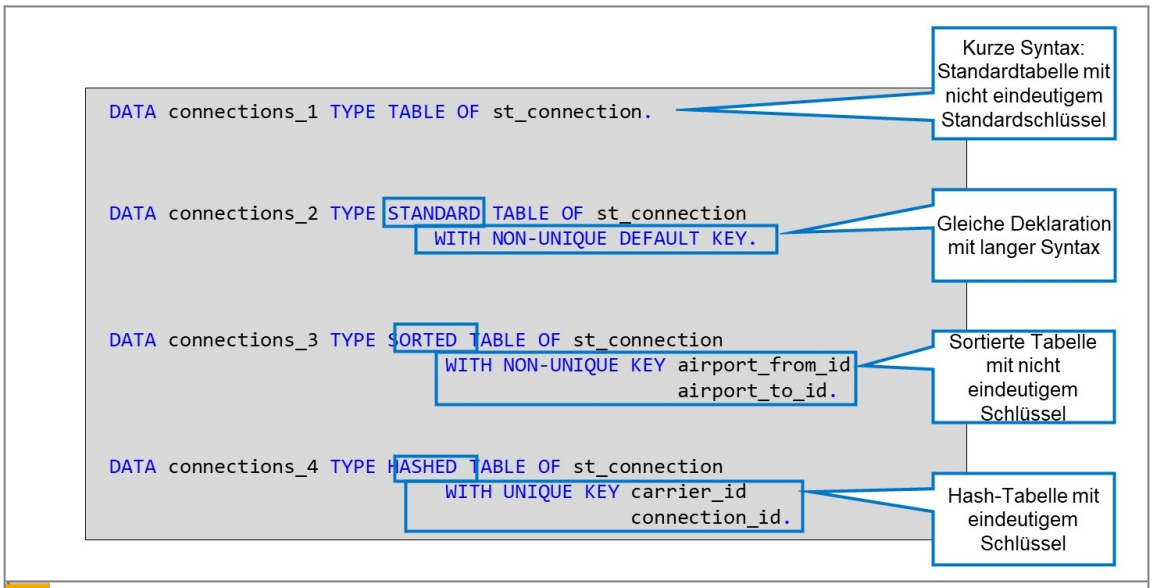


Abbildung 149: Beispiele: komplexe interne Tabellen

Die Abbildung „Beispiele: komplexe interne Tabellen“ zeigt einige Beispiele für die Deklaration komplexer interner Tabellen.

Im ersten Beispiel werden weder die Zugriffsart noch die Schlüsselattribute angegeben. Der ABAP-Compiler deklariert implizit eine Standardtabelle mit einem nicht eindeutigen Standardschlüssel. Der Standardschlüssel besteht aus allen nicht numerischen Spalten des Tabellenzeilentyps.

Für einfache interne Tabellen, bei denen Zugriffsart und Schlüsseldefinition keine Rolle spielen, ist die Kurzsyntax akzeptabel. Bei komplexen internen Tabellen sollten Zugriffsart und Schlüssel eindeutig angegeben werden. Im zweiten Beispiel wird eine interne Tabelle des gleichen Typs wie im ersten Beispiel definiert, aber statt der Kurzform werden explizit die Zugriffsart und der Schlüssel angegeben.

Die dritte und vierte interne Tabelle definieren interne Tabellen der Zugriffsart „sortiert“ und „Hash“. Für diese Zugriffsarten ist die Definition des Schlüssels obligatorisch.

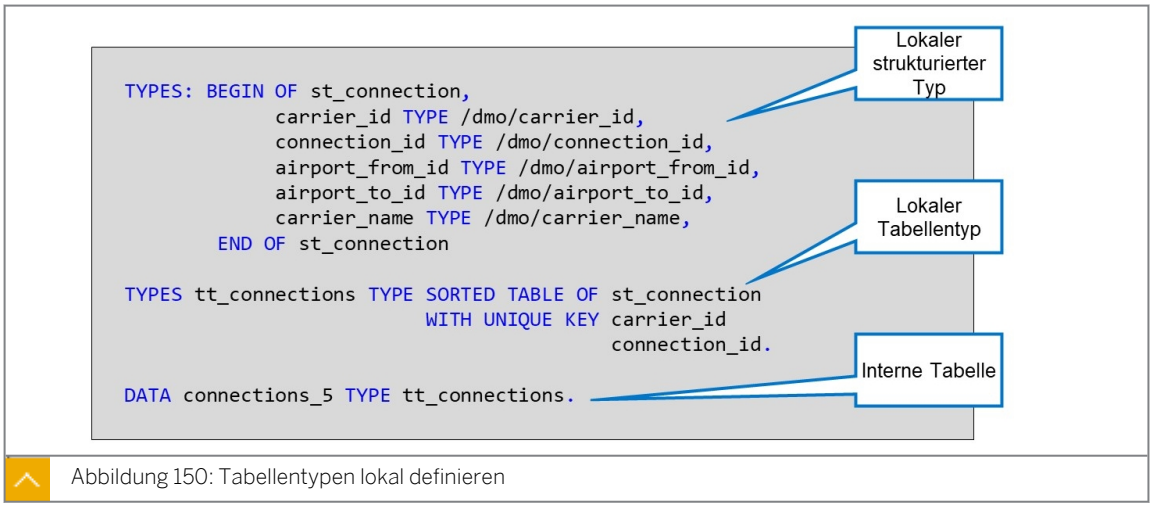


Abbildung 150: Tabellentypen lokal definieren

© Copyright. Alle Rechte vorbehalten.

Des Weiteren empfiehlt es sich für einen guten Programmierstil, erst den Datentyp festzulegen und dann eine Variable anzulegen, die auf den Typ verweist.

Anstelle der Angabe von Zugriffsart und Schlüssel einer internen Tabelle in der DATA-Anweisung sollte ein benannter Tabellentyp verwendet werden. Wenn Sie den Tabellentyp nur lokal in einer Methode oder in Verbindung mit einer bestimmten Klasse benötigen, können Sie ihn mit der TYPES-Anweisung definieren.

Im Beispiel wird zuerst ein Strukturtyp st_connection definiert. Mit diesem Strukturtyp als Zeilentyp wird dann der Tabellentyp tt_connections definiert. Abschließend bezieht sich die Deklaration der internen Tabelle connections_5 auf den Tabellentyp.

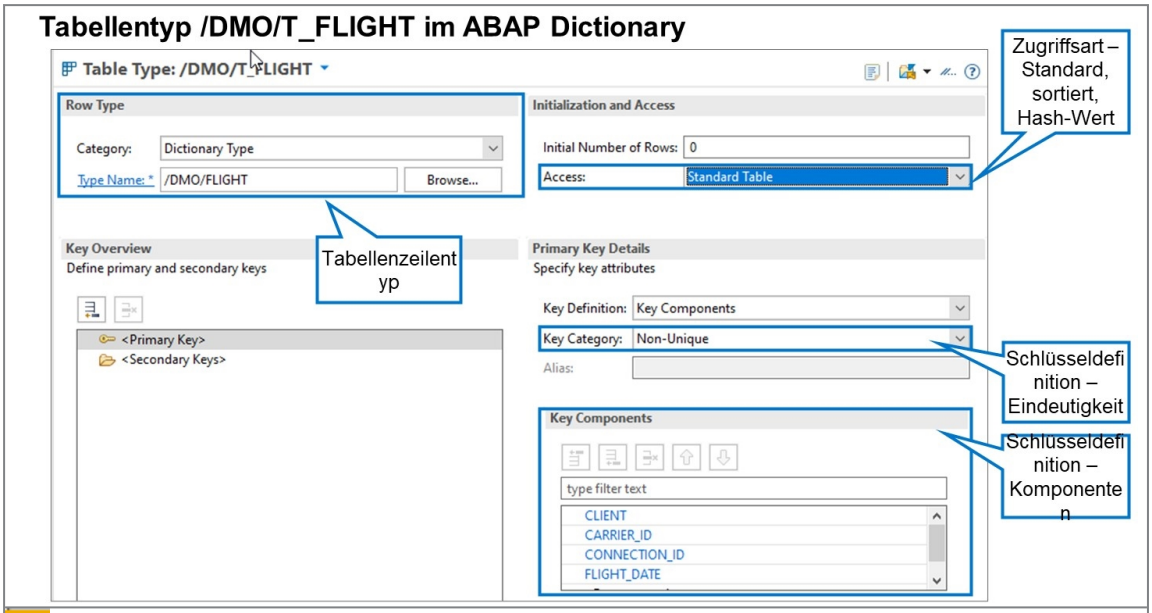


Abbildung 151: Globale Tabellentypen

Wenn Sie den Tabellentyp global benötigen, können Sie einen globalen Tabellentyp verwenden.

Ein globaler Tabellentyp ist ein Repository-Objekt, das an beliebiger Stelle im System als Datentyp verwendet werden kann. ADT stellt einen gesonderten Editor für diese Art von Repository-Objekt bereit. Das Werkzeug besteht aus den folgenden Bereichen:

Zeilentyp

Dieser Bereich enthält die Herkunft und den Namen des Zeilentyps. Im Beispiel wird der globale Typ /DMO/FLIGHT verwendet.

Initialisierung und Zugriff

Dieser Bereich enthält unter anderem die Definition der Zugriffsart. Im Beispiel ist die Zugriffsart „Standardtabelle“ ausgewählt.

Schlüsselübersicht

Dieser Bereich gibt eine Übersicht über die Schlüssel. Wählen Sie den Schlüssel aus, für den Sie Details auf der rechten Seite anzeigen möchten.

> Notiz:
> Jeder Tabellentyp enthält die Definition eines Primärschlüssels. Sekundärschlüssel sind optional. Der Tabellentyp im Beispiel enthält keine Sekundärschlüssel.

© Copyright. Alle Rechte vorbehalten.

Schlüsseldetails

Dieser Rahmen zeigt Details für den ausgewählten Schlüssel an: die Schlüsselart, d.h. ob der Schlüssel eindeutig oder nicht eindeutig ist, und die Liste der Schlüsselkomponenten. Im Beispiel ist der Primärschlüssel nicht eindeutig und besteht aus den Komponenten CLIENT, CARRIER_ID, CONNECTION_ID und FLIGHT_DATE.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Deklarieren Sie eine komplexe interne Tabelle.

© Copyright. Alle Rechte vorbehalten.

Kapitel 6
Lektion 2

Mit komplexen internen Tabellen arbeiten

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Komplexe interne Tabellen mit Daten füllen.
- Zugriff auf den Inhalt komplexer interner Tabellen.
- Verwenden Sie komplexe interne Tabellen in ABAP SQL.

Komplexe interne Tabellen füllen

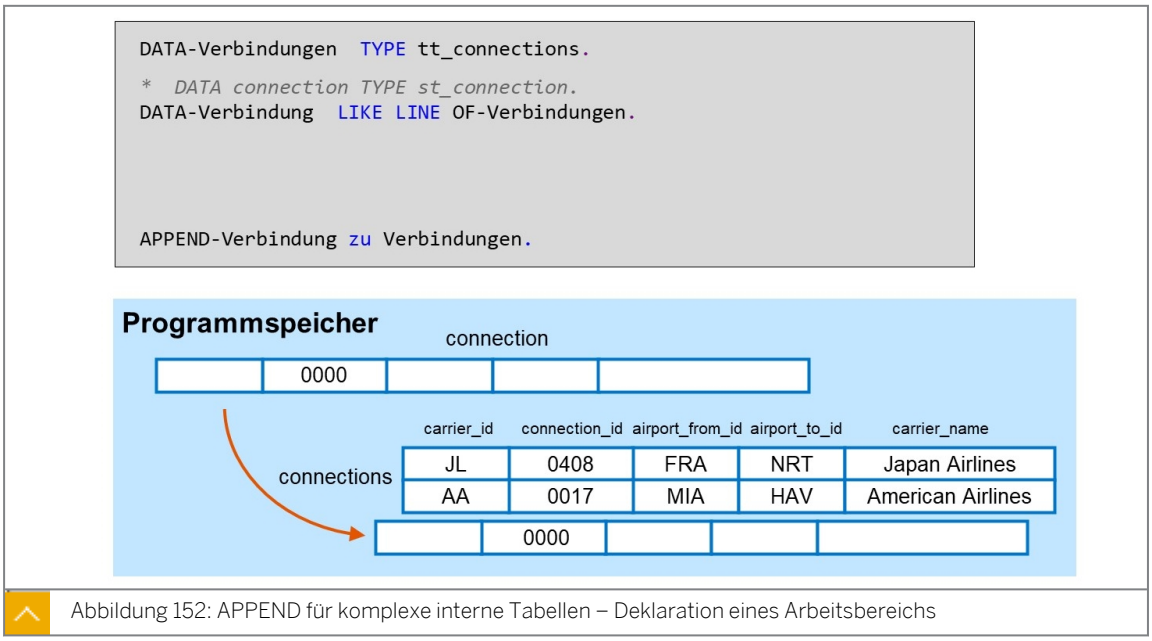


Abbildung 152: APPEND für komplexe interne Tabellen – Deklaration eines Arbeitsbereichs

Wie Sie bereits gelernt haben, lässt sich eine neue Zeile am einfachsten durch die Anweisung APPEND mit einem Datenobjekt, dessen Typ dem Zeilentyp der internen Tabelle entspricht, in eine interne Tabelle einfügen. Dieses Datenobjekt wird auch als „Arbeitsbereich“ bezeichnet.

Bei einfachen internen Tabellen kann der in APPEND verwendete Arbeitsbereich eine skalare Variable, eine Konstante oder ein Literal sein. Bei komplexen internen Tabellen muss der Arbeitsbereich strukturiert sein.

Im Beispiel wird die strukturierte Variable connection verwendet, um die interne Tabelle connections zu füllen.

Grundsätzlich gibt es zwei Möglichkeiten, einen Arbeitsbereich connection zu deklarieren:

- Direkte Referenzierung des Zeilentyps st_connection

© Copyright. Alle Rechte vorbehalten.

- Indirekte Referenzierung des Zeilentyps über LIKE LINE OF <internal_table>

Die Definition von Arbeitsbereichen mit LIKE LINE OF hat zwei Vorteile:

- Sie zeigt den Zweck der strukturierten Variablen als Arbeitsbereich für die interne Tabelle.
- Sie sorgt dafür, dass der Arbeitsbereich zur internen Tabelle passt, auch wenn sich die Definition der internen Tabelle ändert.

```
DATA-Verbindungen  TYPE tt_connections.
DATA-Verbindung  LIKE LINE OF-Verbindungen.

connection  = VALUE #(  carrier_id  = 'NN'
                        connection_id = '1234'
                        airport_from_id = 'ABC'
                        airport_to_id = 'XYZ'
                        carrier_name = 'Meine Fluggesellschaft' ).
Verbindung an Verbindungen anhängen.
```

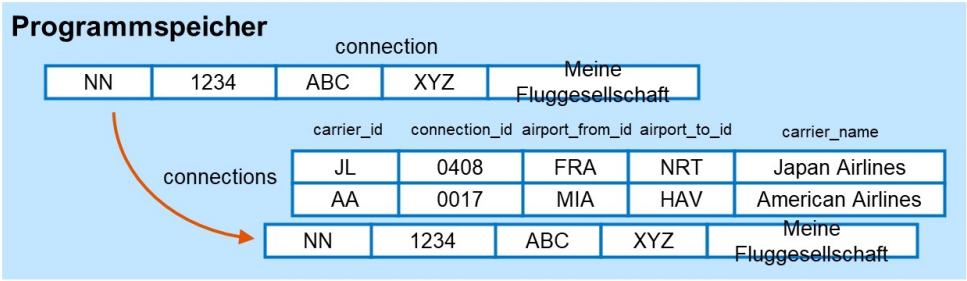


Abbildung 153: APPEND für komplexe interne Tabellen – den Arbeitsbereich füllen

Wenn Sie den Arbeitsbereich nicht vor der APPEND-Anweisung füllen, wird die neue Zeile der internen Tabelle mit typspezifischen Initialwerten gefüllt.

Hinweis:
Das gleiche Ergebnis erhalten Sie mit der Sondervariante APPEND INITIAL LINE TO <internal_table>. Für diese Variante ist nicht einmal ein Arbeitsbereich erforderlich.

Um den strukturierten Arbeitsbereich zu füllen, können Sie entweder die einzelnen Komponenten füllen oder, wie Sie im Beispiel sehen können, einen Ausdruck VALUE #( ) verwenden.

© Copyright. Alle Rechte vorbehalten.

```
DATA-Verbindungen  TYPE tt_connections.
APPEND VALUE #( carrier_id = 'NN'
                connection_id = '1234'
                airport_from_id  = 'ABC'
                airport_to_id = 'XYZ'
 carrier_name = 'Meine Fluggesellschaft'
 )
    TO-Verbindungen.
```

Programmspeicher

connections

carrier_id	connection_id	airport_from_id	airport_to_id	carrier_name
JL	0408	FRA	NRT	Japan Airlines
AA	0017	MIA	HAV	American Airlines
NN	1234	ABC	XYZ	Meine Fluggesellschaft

Abbildung 154: APPEND für komplexe interne Tabellen – direkte Verwendung von VALUE #( )

Wie im Beispiel zu sehen ist, können Sie auch einen Ausdruck VALUE #( ) direkt in der APPEND-Anweisung verwenden. In diesem Fall benötigen Sie keinen Arbeitsbereich.

Notiz:
Dies kann sich positiv auf den gesamten Speicherverbrauch Ihres Programms auswirken.

VALUE #( ) für interne Tabellen

```
  <internal_table> = VALUE #( ( <Werte für zeile1>)
(<values for row2>)
...
).
```

Codebeispiel

```
DATA carrier  TYPE tt_carrier.
Carrier = VALUE #( ( carrier_id = 'AA' carrier_name = 'American Airlines' )
                   (  carrier_id  = 'JL'  carrier_name = 'Japan Airlines' )
 (  carrier_id  = 'NN'  carrier_name  = 'My Airline' )
                  ).
```

Programmspeicher

Spediteure

carrier_id	carrier_name	currency_code
AA	American Airlines	
JL	Japan Airlines	
NN	Meine Fluggesellschaft	

Abbildung 155: Eine interne Tabelle mit mehreren Zeilen füllen

Es gibt eine Variante des Ausdrucks VALUE #( ), die Sie einer internen Tabelle direkt zuweisen können. In dieser Variante von VALUE #( ) werden zusätzliche Klammerpaare verwendet, um die Tabellenzeilen voneinander zu trennen.

© Copyright. Alle Rechte vorbehalten.

Das Code-Beispiel füllt die interne Tabelle carrier mit drei Zeilen, die jeweils einen anderen Wert für carrier_id und carrier_name aufweisen. Dadurch wird die Spalte currency_code nicht erwähnt, sondern mit dem typspezifischen Initialwert gefüllt.

Notiz:
Mit der obigen Zuordnung werden alle vorhandenen Tabellenzeilen entfernt, bevor die Tabelle mit den neuen Zeilen gefüllt wird.

CORRESPONDING für interne Tabellen

```
<Zieltabelle> = CORRESPONDING #( <Quelltabelle> ).
```

Codebeispiel

```
connections  = CORRESPONDING #(  carrier  ).
```

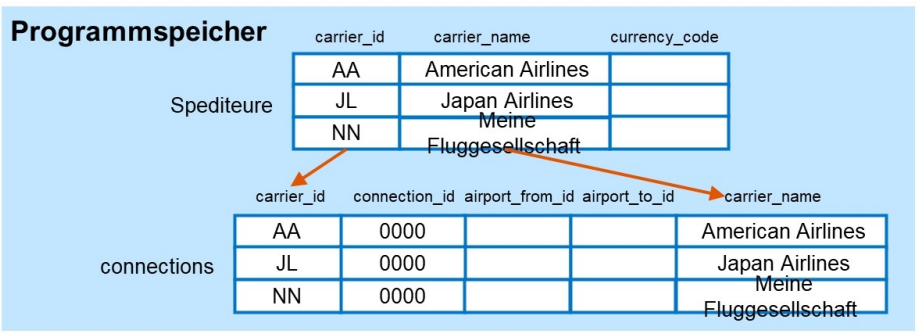


Abbildung 156: Eine interne Tabelle aus einer anderen füllen

Um Daten zwischen namensgleichen Feldern zweier interner Tabellen zu kopieren, verwenden Sie den CORRESPONDING-Operator. Dies funktioniert ähnlich wie CORRESPONDING für Strukturen: Für jede Zeile der internen Quelltabelle legt das System eine neue Zeile in der internen Zieltabelle an und kopiert Daten zwischen namensgleichen Feldern. Quellfelder, für die es kein gleichnamiges Feld im Ziel gibt, werden nicht kopiert. Zielfelder, für die es kein gleichnamiges Feld in der Quelle gibt, werden mit typspezifischen Initialwerten gefüllt.

Im Beispiel enthält die interne Quelltabelle carrier drei Zeilen. Daher enthält die interne Zieltabelle connections nach der Wertzuweisung auch drei Zeilen.

Die Felder carrier_id und carrier_name sind in beiden internen Tabellen vorhanden. Sie werden von der Quelle in das Ziel kopiert. Das Feld currency_code ist nur in der Quelle vorhanden. Es wird nicht kopiert. Die Felder connection_id, airport_from_id und airport_to_id sind nur im Ziel vorhanden. Sie werden mit Initialwerten gefüllt.

Notiz:
Wenn die interne Tabelle, die als Ziel dient, vor der Zuweisung Daten enthält, werden diese vom System gelöscht.

© Copyright. Alle Rechte vorbehalten.

Zugriff auf komplexe interne Tabellen

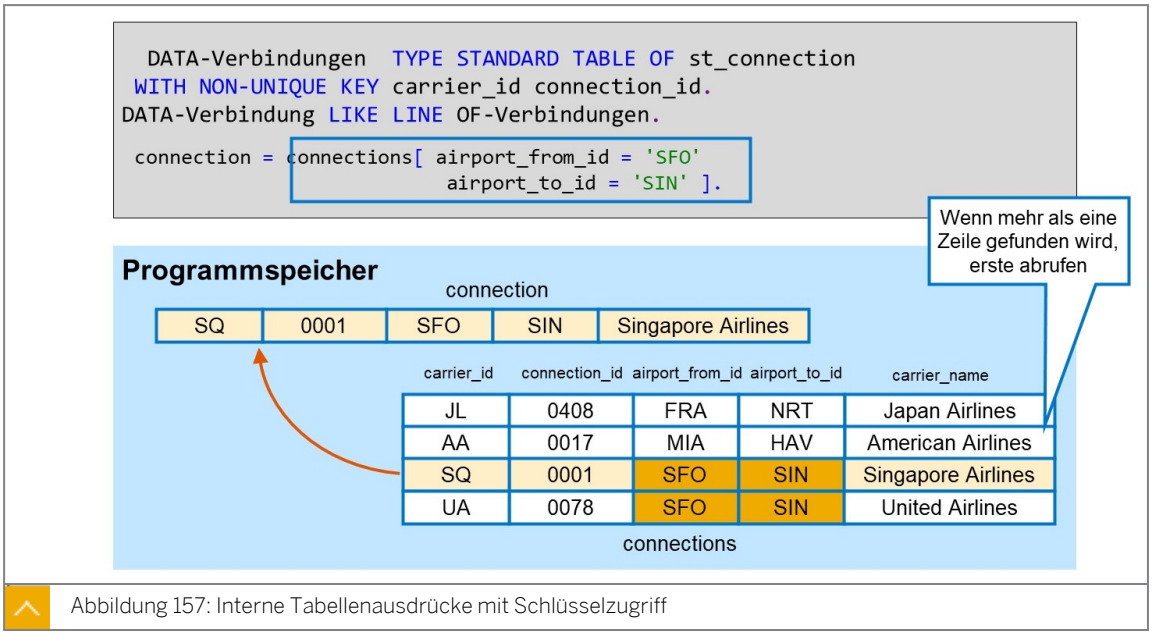


Abbildung 157: Interne Tabellenausdrücke mit Schlüsselzugriff

Sie haben in diesem Kurs bereits gelernt, wie Sie mithilfe eines internen Tabellenausdrucks eine einzelne Zeile aus einer einfachen internen Tabelle abrufen. Dort wurde ein Indexzugriff verwendet, d.h., die Zeile wurde durch ihre Position in der internen Tabelle identifiziert. Dieser Indexzugriff funktioniert für komplexe interne Tabellen genauso. Für komplexe interne Tabellen werden dagegen interne Tabellenausdrücke mit Schlüsselzugriff wichtig, bei denen die Zeile durch ihren Inhalt identifiziert wird.

Notiz:
Obwohl dies als Schlüsselzugriff bezeichnet wird, können beliebige Felder für die Selektion verwendet werden, nicht nur Schlüsselfelder der internen Tabelle. Wenn mehr als eine Zeile die Anforderung erfüllt, geben STANDARD-Tabellen die erste Zeile zurück, d.h. die Zeile mit dem niedrigsten Index.

Im Beispiel wird eine einzelne Zeile aus der internen Tabelle connections gelesen. Der Schlüssel dieser internen Tabelle besteht aus den Feldern carrier_id und connection_id, aber der Schlüsselzugriff verwendet airport_from_id und airport_to_id, um die Zeile zu identifizieren. Die interne Tabelle enthält zwei Verbindungen vom Flughafen SFO zu SIN, sodass die erste davon zurückgegeben wird.

Notiz:
Beachten Sie, dass die ABAP-Laufzeit die Ausnahme CX_SY_ITAB_LINE_NOT_FOUND auslöst, wenn keine der Zeilen die Bedingung erfüllt. Behandeln Sie diese Ausnahme zur Vermeidung von Laufzeitfehlern in einer Struktur TRY ... CATCH ... ENDTRY.

© Copyright. Alle Rechte vorbehalten.

```
LOOP AT VerbindungenINTO-Verbindung
                    WHERE airport_from_id <> 'MIA'.

* etwas mit dem Inhalt von connection tun

   ENDLOOP.
```

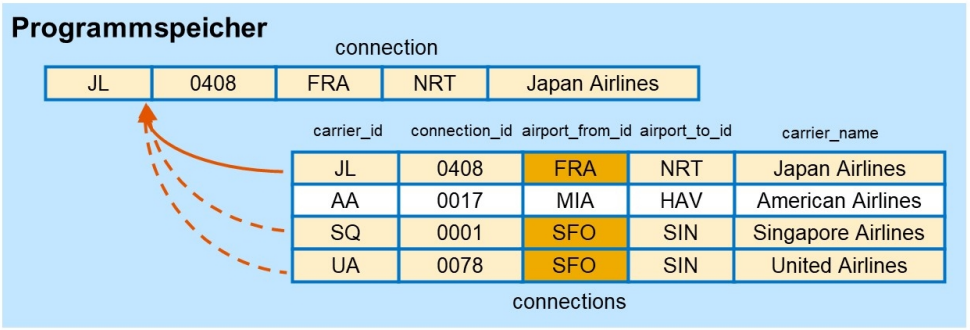


Abbildung 158: LOOP ... ENDLOOP mit Schlüsselzugriff

Wenn Sie mehrere Zeilen einer internen Tabelle verarbeiten möchten und diese unter Angabe von Feldern identifizieren möchten, verwenden Sie LOOP AT <interne Tabelle> INTO <Ziel> WHERE <Bedingung>. Die WHERE-Bedingung kann eine beliebige Anzahl konstituierender Ausdrücke enthalten, die durch AND und OR verknüpft sind. Innerhalb der Ausdrücke können Sie nicht nur das Gleichheitszeichen (=), sondern auch die Operatoren >, >=, <, <=, <> und BETWEEN verwenden.

MODIFY TABLE-Anweisung

```
MODIFY TABLE <interne Tabelle> FROM <Arbeitsbereich>.
```

Beispiel

```
Datenträger TYPE STANDARD TABLE OF st_carrier
                    WITH NON-UNIQUE KEY carrier_id.
DATA-Träger LIKE LINE OF Carrier.

 carrier = carrier[ carrier_id = 'JL' ].

 carrier-currency_code = 'JPY'.

MODIFY TABLE Carrier FROM carrier.
```

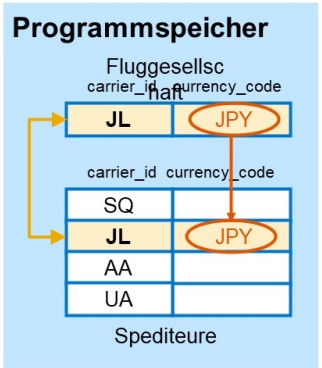


Abbildung 159: Eine Tabellenzeile mit Schlüsselzugriff aktualisieren

Nach dem Einlesen des Inhalts einer Tabellenzeile in einen Arbeitsbereich möchten Sie unter Umständen Änderungen aus dem Arbeitsbereich zurück in die interne Tabelle schreiben. Eine Möglichkeit hierfür ist die Anweisung MODIFY TABLE.

Diese Anweisung ist ein Schlüsselzugriff, da das System den Inhalt der Schlüsselfelder im Arbeitsbereich verwendet, um die zu ändernde Tabellenzeile zu identifizieren. Sie überschreibt dann diese Tabellenzeile mit den Inhalten des Arbeitsbereichs.

© Copyright. Alle Rechte vorbehalten.

Im Beispiel enthält der Arbeitsbereich carrier den Wert 'JL' im Schlüsselfeld carrier_id. Anhand dieses Werts identifiziert das System die zweite zu aktualisierende Zeile. Diese Zeile wird dann mit den Werten aus dem Arbeitsbereich aktualisiert.

> Notiz:
> Mit MODIFY TABLE können nur Nicht-Schlüsselfelder geändert werden. Die Anweisung unterstützt keine Änderungen an Schlüsselfeldern.

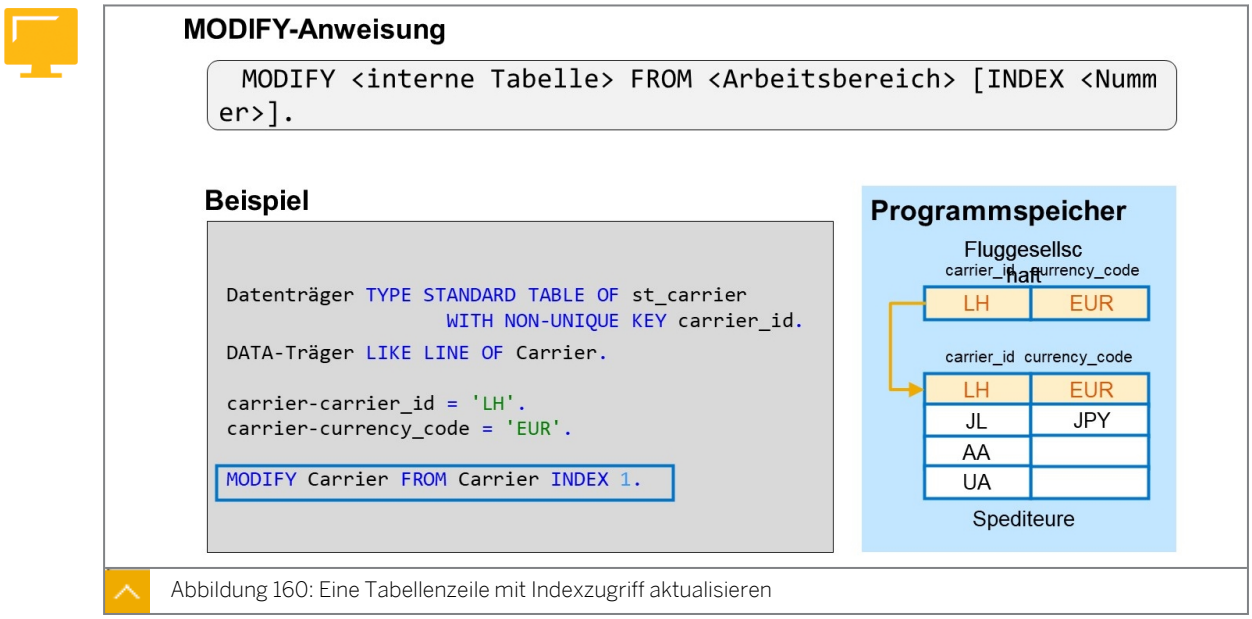


Abbildung 160: Eine Tabellenzeile mit Indexzugriff aktualisieren

Die MODIFY -Anweisung (ohne Schlüsselwort TABLE!) unterscheidet nicht zwischen Schlüsselfeldern und Nicht-Schlüsselfeldern. Sie überschreibt die gesamte Tabellenzeile mit neuen Werten aus dem Arbeitsbereich. Diese Anweisung gilt als Indexzugriff, da die zu aktualisierende Zeile durch ihre Position in der internen Tabelle identifiziert wird. In der Regel wird der Index explizit mit dem Zusatz INDEX, gefolgt von einem ganzzahligen Argument (Literal, Konstante, Variable, Ausdruck), angegeben.

> Notiz:
> Es gibt auch eine spezielle Variante ohne den Zusatz INDEX. Wir werden diese Variante als Nächstes besprechen.

Im Beispiel verwendet die MODIFY -Anweisung den Zusatz INDEX, um die erste Tabellenzeile zu adressieren. In dieser Zeile werden alle Felder mit den Werten aus dem Arbeitsbereich überschrieben, auch das Schlüsselfeld carrier_id.

© Copyright. Alle Rechte vorbehalten.

Kurzform von MODIFY (ohne INDEX)

```
MODIFY <interne Tabelle> FROM <Arbeitsbereich>.
```

Beispiel

```
Datenträger TYPE STANDARD TABLE OF st_carrier
                    WITH NON-UNIQUE KEY carrier_id.
DATA-Träger LIKE LINE OF Carrier.

 LOOP AT Carrier INTO Carrier
WHERE currency_code IS INITIAL.

     carrier-currency_code = 'USD'.

    MODIFY Carrier FROM Carrier.

 ENDLOOP.
```

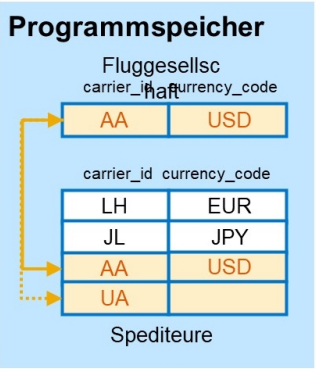


Abbildung 161: Mehrere Tabellenzeilen in einer Schleife aktualisieren

Es treten häufig Situationen auf, in denen Sie die Inhalte mehrerer oder sogar aller Zeilen einer internen Tabelle bearbeiten müssen. Dazu implementieren Sie eine Schleife über die Tabelle, die jede zu ändernde Zeile nacheinander in einen Arbeitsbereich stellt. Innerhalb der Schleife ändern Sie zunächst den Inhalt des Arbeitsbereichs. Anschließend schreiben Sie die Änderungen mit der MODIFY-Anweisung in die interne Tabelle zurück.

Notiz:
Wenn Sie Ihre Änderungen nicht zurück in die Tabelle schreiben, gehen die Änderungen verloren, wenn der Arbeitsbereich mit den Daten aus der nächsten Zeile gefüllt wird.

Im Beispiel liest die Schleife alle Zeilen der internen Tabelle carriers, für die das Feld currency_code noch nicht gefüllt ist. Dies ist bei den letzten beiden Zeilen der Fall. Für jede dieser Zeilen ersetzt das Programm den Initialwert im Feld currency_code durch den neuen Wert „USD“. Schließlich wird die aktuelle Zeile mithilfe der MODIFY-Anweisung mit den aktualisierten Werten überschrieben.

Anstatt den Index explizit anzugeben, wird im Quelltextbeispiel eine Kurzform der MODIFY-Anweisung verwendet, bei der der Zusatz INDEX fehlt. Diese Kurzform ist nur zwischen LOOP ... ENDLOOP erlaubt. Nur dort kann das System implizit die Zeile aktualisieren, an der es gerade arbeitet.

Notiz:
Wenn Sie versuchen, MODIFY ohne INDEX außerhalb von LOOP ... ENDLOOP zu verwenden, weiß das System nicht, welche Zeile bearbeitet werden soll, und löst einen nicht abfangbaren Laufzeitfehler aus. Um solche Laufzeitfehler zu vermeiden, achten Sie darauf, die zugehörige Warnung der Syntaxprüfung nicht zu ignorieren.

© Copyright. Alle Rechte vorbehalten.

Komplexe interne Tabellen in ABAP SQL

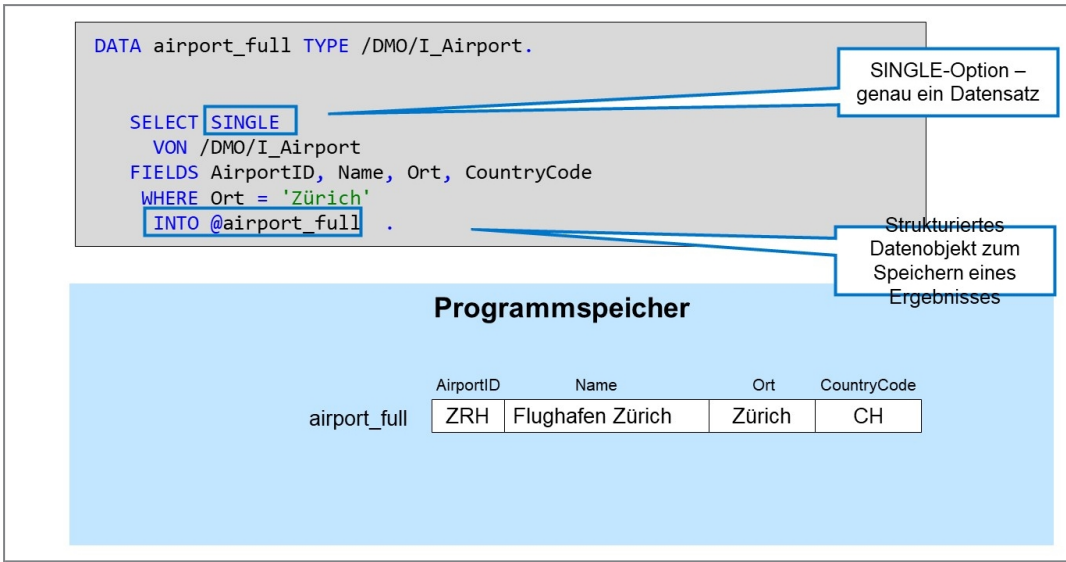


Abbildung 162: Erinnerung: Strukturierte Variablen in SELECT SINGLE ... INTO ...

Die ABAP-SQL-Anweisung SELECT liest Daten aus einer Datenbanktabelle oder einer CDS-View. Wenn Sie die Option SINGLE verwenden, wird genau ein Datensatz aus der Datenbank gelesen, auch wenn mehr Daten vorhanden sind, die die Bedingungen in der WHERE-Klausel erfüllen.

Wie Sie bereits gelernt haben, ist eine Möglichkeit, dieses Einzelsatzergebnis zu erhalten, eine strukturierte Variable nach dem Schlüsselwort INTO.

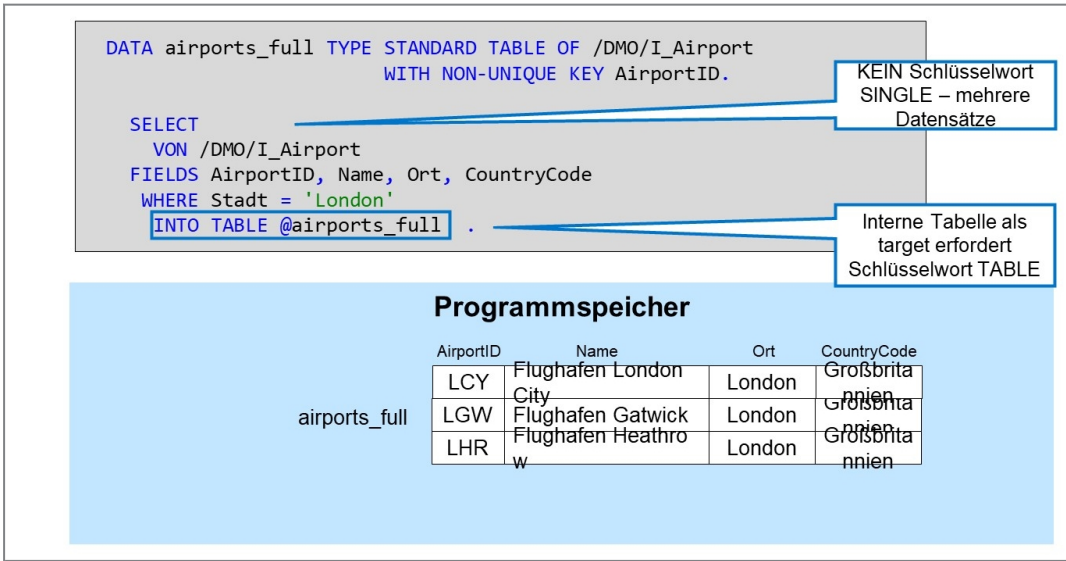


Abbildung 163: Interne Tabelle bei SELECT ... INTO TABLE ...

Wenn Sie SELECT ohne SINGLE verwenden, geben Sie an, dass Sie an allen Datensätzen interessiert sind, die den Bedingungen in der WHERE-Klausel entsprechen. Sie müssen dann sicherstellen, dass Sie mehrere Datensätze empfangen und speichern können. Der offensichtliche Weg, dies zu tun, ist die Verwendung einer komplexen internen Tabelle als Ziel

© Copyright. Alle Rechte vorbehalten.

der SELECT -Anweisung. Dies ist möglich, erfordert aber den Zusatz TABLE zwischen dem Schlüsselwort INTO und dem Namen der internen Tabelle.

Im Beispiel möchten wir alle drei Flughäfen lesen, die mit London in Verbindung stehen, und nicht nur einen einzigen von ihnen. Daher wird das Schlüsselwort SINGLE nach SELECT weggelassen, das Schlüsselwort TABLE hinter INTO hinzugefügt und die interne Tabelle airports_full als Ziel der SELECT-Anweisung verwendet.

Das Beispiel verwendet eine explizite Feldliste hinter FIELDS, die den Spalten der internen Tabelle airports_full entspricht. Natürlich können Sie auch FIELDS *, INTO CORRESPONDING FIELDS OF TABLE und Aliasnamen in der Feldliste verwenden.

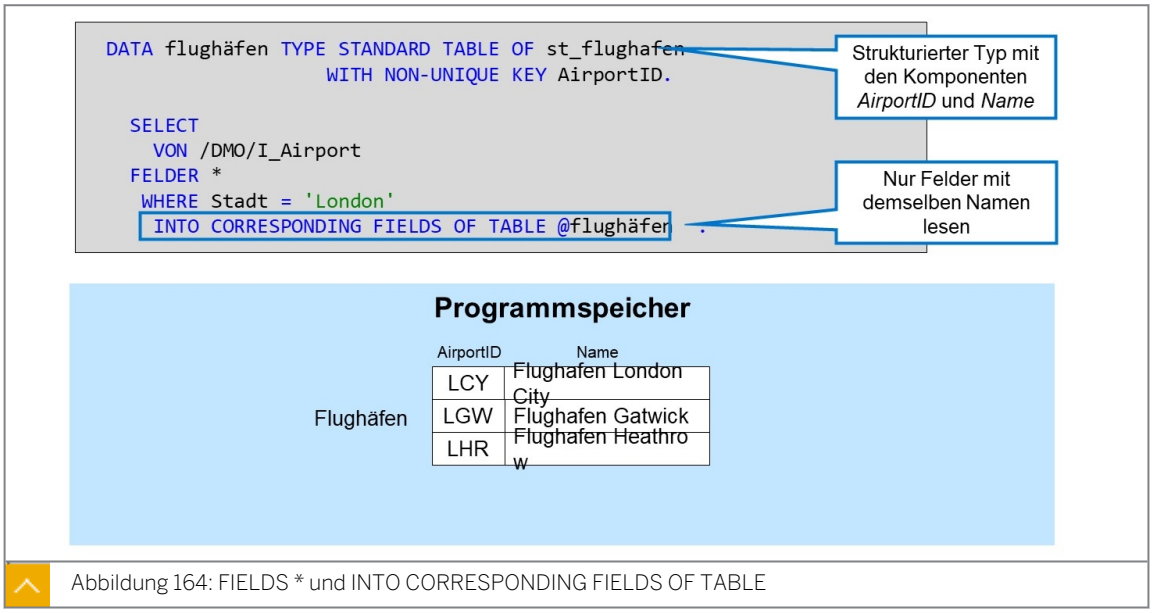


Abbildung 164: FIELDS * und INTO CORRESPONDING FIELDS OF TABLE

Das Beispiel verwendet FIELDS * statt einer expliziten Feldliste und INTO CORRESPONDING FIELDS OF TABLE statt INTO TABLE.

Da der Zeilentyp der internen Tabelle flughäfen nur zwei Komponenten AirportID und Name enthält, werden nur die gleichnamigen Felder von der Datenbank gelesen.

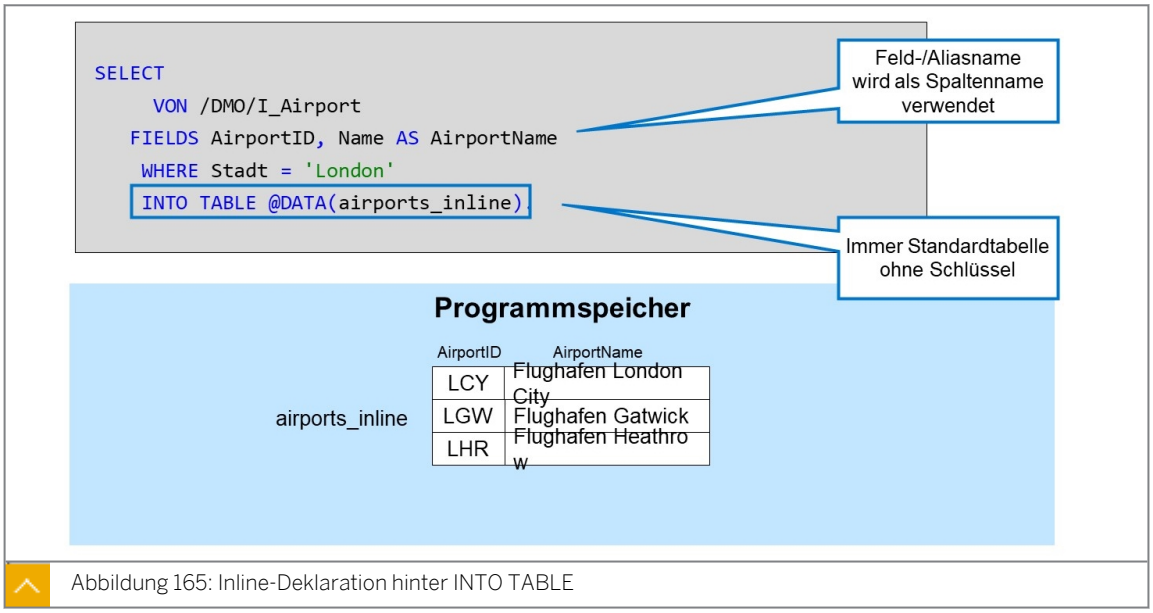


Abbildung 165: Inline-Deklaration hinter INTO TABLE

© Copyright. Alle Rechte vorbehalten.

Wenn Sie DATA( ) in einer SELECT-Anweisung hinter dem Zusatz INTO TABLE verwenden, deklarieren Sie inline eine interne Tabelle. Der Zeilentyp dieser internen Tabelle wird aus der FIELDS -Klausel abgeleitet. Für Tabellenfelder und View-Elemente ist ein Aliasname optional. Für Ausdrücke in der FIELDS -Klausel ist ein Aliasname obligatorisch, wenn die INTO -Klausel eine Inline-Deklaration enthält.

Notiz:
Inline-Deklarationen interner Tabellen werden nur hinter INTO TABLE unterstützt. Inline-Deklarationen können nicht hinter INTO CORRESPONDING FIELDS OF TABLE verwendet werden.

Notiz:
Inline deklarierte interne Tabellen sind immer Standardtabellen ohne Schlüssel. Sortierte Tabellen oder Hash-Tabellen können nicht mit Inline-Deklarationen deklariert werden. Dies kann zu Performanceproblemen führen, wenn Sie die interne Tabelle mit vielen Zeilen füllen und den Schlüsselzugriff sehr häufig verwenden.

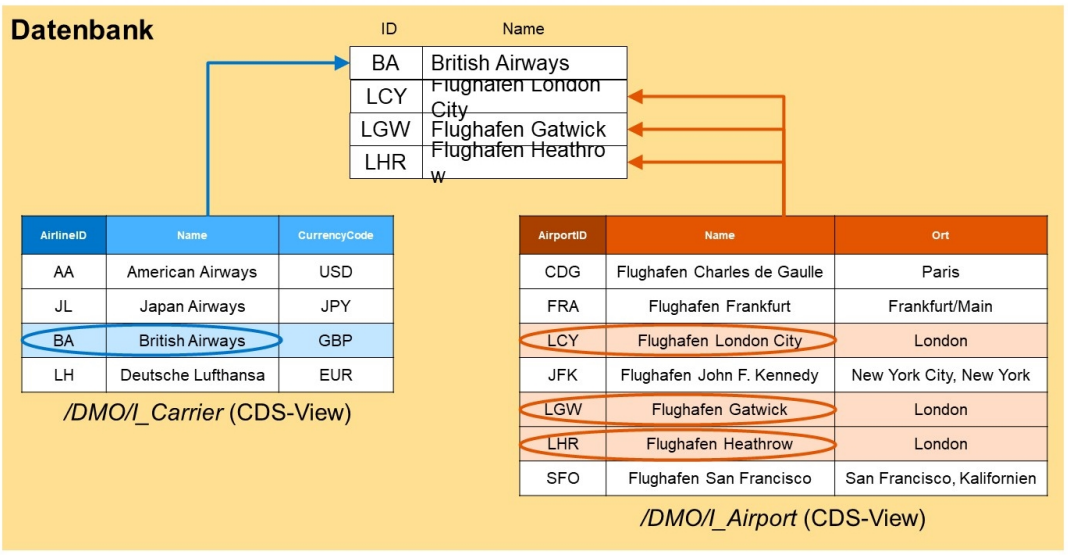


Abbildung 166: Kombinieren von Ergebnissen aus mehreren SELECTs (Unions)

Wenn Sie mehrere Datensätze aus der Datenbank lesen, werden einige spezielle SQL-Techniken besonders interessant. Eine dieser Techniken ist die UNION-Richtlinie, um die Ergebnisse mehrerer SELECT-Anweisungen zu kombinieren.

Die Abbildung zeigt die Kombination zweier SELECT-Ergebnisse:

Das erste SELECT -Ergebnis liest ID und NAME aller Carrier mit `CurrencyCode = 'GBP'`. Der zweite SELECT liest ID und NAME aller Flughäfen mit `City = 'London'`. Der erste SELECT gibt einen Datensatz zurück, der zweite SELECT gibt drei Datensätze zurück. Anstatt diese Ergebnisse separat abzurufen, werden sie zu einem Ergebnis mit vier Datensätzen kombiniert. Es ist wichtig, darauf hinzuweisen, dass dies innerhalb der Datenbank geschieht.

© Copyright. Alle Rechte vorbehalten.

Voraussetzung für diese Technik ist natürlich, dass die beiden Ergebnisse miteinander kompatibel sind, d.h. dass sie die gleiche Anzahl von Feldern und die gleichen Feldnamen haben. Es ist jedoch nicht notwendig, dass die Typen der Felder identisch sind.

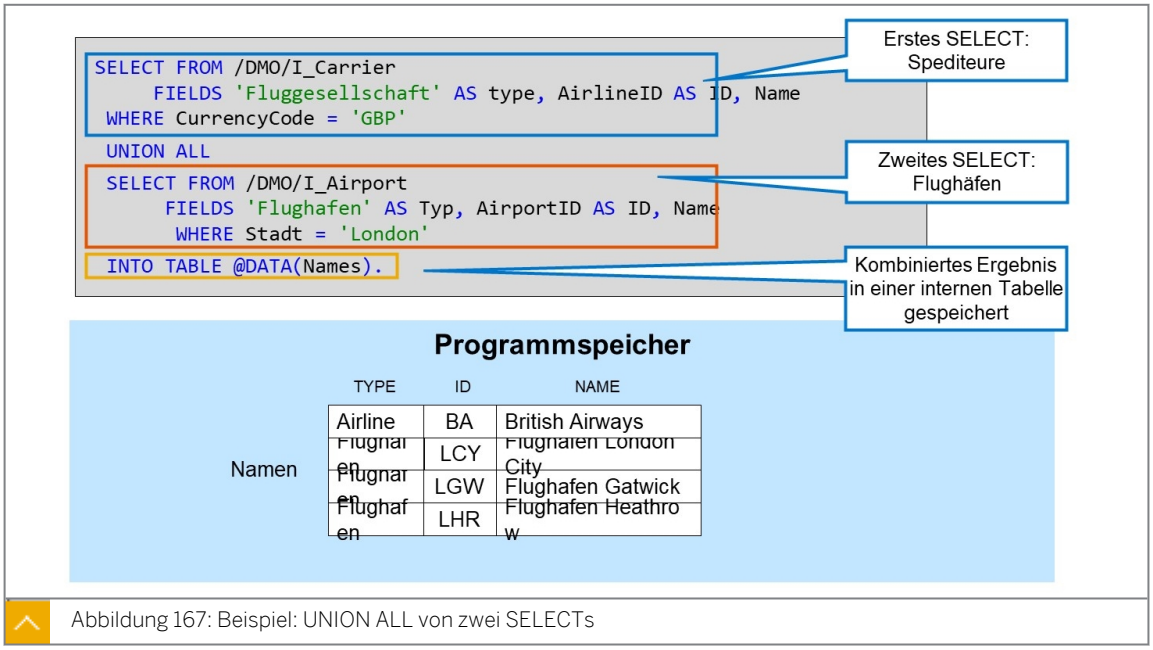


Abbildung 167: Beispiel: UNION ALL von zwei SELECTs

Die ABAP-SQL-Syntax für dieses Beispiel besteht aus zwei SELECT -Anweisungen. Jede SELECT-Anweisung hat ihre eigene FROM-Klausel, FIELDS-Klausel und WHERE-Klausel, aber ganz am Ende gibt es nur eine INTO-Klausel. Die beiden SELECT -Anweisungen sind durch Schlüsselwörter UNION ALL verbunden.

Notiz:
Bei UNION statt UNION ALL würde die Datenbank nach Duplikaten suchen und diese eliminieren, bevor das Ergebnis zurückgegeben wird. Wir verwenden UNION ALL, um diese unnötige zusätzliche Last auf der Datenbank zu vermeiden.

Beide Feldlisten bestehen aus drei Elementen, wobei das erste und das zweite Element in beiden FIELDS -Klauseln identische Aliasnamen haben. Das dritte Feld benötigt keinen Alias, da der Feldname in beiden CDS-Views gleich ist.

Notiz:
Das erste Element in FIELDS ist ein literaler Text, mit dem wir im kombinierten Ergebnis zwischen Fluggesellschaften und Flughäfen unterscheiden können.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 6
Übung 15

Komplexe interne Tabelle verwenden

In dieser Übung deklarieren Sie ein tabellenartiges Attribut, füllen es mit einer SELECT-Anweisung und greifen auf den Inhalt zu.

Vorlage:

/LRN/CL_S4D400_STS_STRUCTURE (globale Klasse)

Lösung:

/LRN/CL_S4D400_ITS_ITAB (globale Klasse)

Aufgabe 1: Vorlage kopieren
Kopieren Sie die Vorlageklasse. Alternativ können Sie Ihre Lösung aus der vorherigen Übung kopieren.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_STS_STRUCTURE`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_ITAB`**, wobei ## für Ihre Gruppennummer steht).

Aufgabe 2: Interne Tabelle deklarieren
Deklarieren Sie in der lokalen Klasse ein statisches, tabellenartiges Attribut **`flughäfen`**, um Detailinformationen zu allen verfügbaren Flughäfen zu puffern. Definieren Sie zunächst einen privaten Strukturtyp **`st_flughafen`** und einen Tabellentyp **`tt_flughäfen`** innerhalb der lokalen Klasse.

1. Wechseln Sie in die lokale Klasse **`lcl_connection`**.
2. Definieren Sie einen privaten Strukturtyp **`st_flughafen`** mit den folgenden Komponenten:

   Tabelle 5: Komponenten des Strukturtyps st_flughafen:

Komponentenname	Datentyp
`AirportID`	**`/dmo/airport_id`**
`Name`	**`/dmo/airport_name`**

3. Definieren Sie einen privaten Tabellentyp **`tt_flughäfen`** mit den folgenden Eigenschaften:

   Tabelle 6: Eigenschaften des Tabellentyps tt_flughäfen:

Eigenschaft	Wert
Zeilentyp	**`st_flughafen`**
Tabellenart	**`STANDARD TABLE`**

© Copyright. Alle Rechte vorbehalten.

Eigenschaft	Wert
Schlüsseldefinition	**NICHT-EINHEITLICHER STANDARD-SCHLÜSSEL**

4. Deklarieren Sie ein neues privates statisches Attribut **flughäfen** und typisieren Sie es mit dem Tabellentyp **tt_flughäfen**.

Aufgabe 3: Statisches Attribut in einem Klassenkonstruktor füllen
Definieren Sie einen Klassenkonstruktor und implementieren Sie eine SELECT-Anweisung, die alle verfügbaren Flughäfen aus der CDS-View-Entität **/DMO/I_Airport** in das statische Attribut **flughäfen** liest.

1. Fügen Sie der lokalen Klasse **lcl_connection** über einen Quickfix einen Klassenkonstruktor hinzu.
2. Implementieren Sie im Klassenkonstruktor eine SELECT -Anweisung, die alle Datensätze aus der CDS-View-Entität **/DMO/I_Airport** in das statische Attribut **flughäfen** liest.

Aufgabe 4: Zugriff auf den Inhalt der internen Tabelle
Verwenden Sie den Inhalt der internen Tabelle **flughäfen** in der Methode **get_output**, um die Flughafennamen zur Ausgabe hinzuzufügen.

1. Navigieren Sie zur Implementierung der Methode **get_output**.
2. Lesen Sie zu Beginn der Methode die Details des Abflugflughafens in ein strukturiertes Datenobjekt **flight**.

Hinweis:
Verwenden Sie einen Tabellenausdruck `airports[ ... ]` und eine Inline-Deklaration für das Datenobjekt **escape**.

3. Ebenso lesen Sie die Details des Zielflughafens in ein strukturiertes Datenobjekt **destination**.
4. Verwenden Sie den **Komponentennamen** der beiden Strukturen, um die Flughafennamen zur Ausgabe hinzuzufügen.
5. Optional: Verzichten Sie auf das strukturierte Datenobjekt und verwenden Sie die Tabellenausdrücke direkt in den Zeichenketten-Templates.
6. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe. Prüfen Sie, ob die Ausgabe Daten für alle Attribute anzeigt.

© Copyright. Alle Rechte vorbehalten.

Kapitel 6
Lösung 15

Komplexe interne Tabelle verwenden

In dieser Übung deklarieren Sie ein tabellenartiges Attribut, füllen es mit einer SELECT-Anweisung und greifen auf den Inhalt zu.

Vorlage:

/LRN/CL_S4D400_STS_STRUCTURE (globale Klasse)

Lösung:

/LRN/CL_S4D400_ITS_ITAB (globale Klasse)

Aufgabe 1: Vorlage kopieren
Kopieren Sie die Vorlageklasse. Alternativ können Sie Ihre Lösung aus der vorherigen Übung kopieren.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_STS_STRUCTURE`** in eine Klasse in Ihrem eigenen Paket (Namensvorschlag: **`ZCL_##_ITAB`**, wobei ## für Ihre Gruppennummer steht).
   a) Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_STS_STRUCTURE`**.
   b) Verknüpfen Sie die View *Project Explorer* mit dem Editor.
   c) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Klasse **`/LRN/CL_S4D400_STS_STRUCTURE`**, um das Kontextmenü zu öffnen.
   d) Wählen Sie im Kontextmenü *Duplicate ...*
   e) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_ITAB`** ein, wobei ## für Ihre Gruppennummer steht.
   f) Passen Sie die Beschreibung an, und wählen Sie *Next*.
   g) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

Aufgabe 2: Interne Tabelle deklarieren
Deklarieren Sie in der lokalen Klasse ein statisches, tabellenartiges Attribut **`flughäfen`**, um Detailinformationen zu allen verfügbaren Flughäfen zu puffern. Definieren Sie zunächst einen privaten Strukturtyp **`st_flughafen`** und einen Tabellentyp **`tt_flughäfen`** innerhalb der lokalen Klasse.

1. Wechseln Sie in die lokale Klasse **`lcl_connection`**.
   a) Wählen Sie in der globalen Klasse *Local Types*.
2. Definieren Sie einen privaten Strukturtyp **`st_flughafen`** mit den folgenden Komponenten:

© Copyright. Alle Rechte vorbehalten.

Tabelle 5: Komponenten des Strukturtyps st_flughafen:

Komponentenname	Datentyp
AirportID	**/dmo/airport_id**
Name	**/dmo/airport_name**

a) Passen Sie den Quelltext wie folgt an:

```
TYPES:
   BEGIN OF st_details,
     DepartureAirport   TYPE /dmo/airport_from_id,
     DestinationAirport TYPE /dmo/airport_to_id,
     AirlineName        TYPE /dmo/carrier_name,
   END OF st_details.

 TYPES:
   BEGIN OF st_airport,
     AirportId TYPE /dmo/airport_id,
     Name      TYPE /dmo/airport_name,
   END OF st_airport.
```

3. Definieren Sie einen privaten Tabellentyp **tt_flughäfen** mit den folgenden Eigenschaften:

Tabelle 6: Eigenschaften des Tabellentyps tt_flughäfen:

Eigenschaft	Wert
Zeilentyp	**st_flughafen**
Tabellenart	**STANDARD TABLE**
Schlüsseldefinition	**NICHT-EINHEITLICHER STANDARD-SCHLÜSSEL**

a) Passen Sie den Quelltext wie folgt an:

```
TYPES:
     BEGIN OF st_airport,
       AirportId TYPE /dmo/airport_id,
       Name      TYPE /dmo/airport_name,
     END OF st_airport.

   TYPES tt_airports TYPE STANDARD TABLE OF st_airport
                          WITH NON-UNIQUE DEFAULT KEY.
```

4. Deklarieren Sie ein neues privates statisches Attribut **flughäfen** und typisieren Sie es mit dem Tabellentyp **tt_flughäfen**.

a) Fügen Sie am Ende der Klassendefinition den folgenden Quelltext hinzu:

```
 CLASS-DATA airports TYPE tt_airports.
```

Aufgabe 3: Statisches Attribut in einem Klassenkonstruktor füllen

Definieren Sie einen Klassenkonstruktor und implementieren Sie eine SELECT-Anweisung, die alle verfügbaren Flughäfen aus der CDS-View-Entität **/DMO/I_Airport** in das statische Attribut **flughäfen** liest.

© Copyright. Alle Rechte vorbehalten.

1. Fügen Sie der lokalen Klasse **`lcl_connection`** über einen Quickfix einen Klassenkonstruktor hinzu.
   a) Setzen Sie den Cursor auf den Namen der Klasse, und drücken Sie **`Strg + 1`**.
   b) Doppelklicken Sie auf *Klassenkonstruktor generieren*.
2. Implementieren Sie im Klassenkonstruktor eine SELECT -Anweisung, die alle Datensätze aus der CDS-View-Entität **`/DMO/I_Airport`** in das statische Attribut **`flughäfen`** liest.
   a) Navigieren Sie zur Implementierung der Methode **`class_constructor`**.
   b) Fügen Sie in der Methodenimplementierung den folgenden Quelltext hinzu:

```
    SELECT FROM /DMO/I_Airport
         FIELDS AirportID, Name
           INTO TABLE @airports.
```

Aufgabe 4: Zugriff auf den Inhalt der internen Tabelle
Verwenden Sie den Inhalt der internen Tabelle **`flughäfen`** in der Methode **`get_output`**, um die Flughafennamen zur Ausgabe hinzuzufügen.

1. Navigieren Sie zur Implementierung der Methode **`get_output`**.
   a) Fahren Sie wie in den vorangegangenen Übungen fort.
2. Lesen Sie zu Beginn der Methode die Details des Abflugflughafens in ein strukturiertes Datenobjekt **`flight`**.

Hinweis:
Verwenden Sie einen Tabellenausdruck `airports[ ... ]` und eine Inline-Deklaration für das Datenobjekt **`escape`**.

   a) Fügen Sie am Anfang der Methode den folgenden Code hinzu:

```
  DATA(departure)   = airports[ airportID = details-departureairport ].
```

3. Ebenso lesen Sie die Details des Zielflughafens in ein strukturiertes Datenobjekt **`destination`**.
   a) Passen Sie den Quelltext wie folgt an:

```
  DATA(departure)   = airports[ airportID = details-
departureairport   ].
  DATA(destination) = airports[ airportID = details-
destinationairport ].
```

4. Verwenden Sie den **`Komponentennamen`** der beiden Strukturen, um die Flughafennamen zur Ausgabe hinzuzufügen.
   a) Passen Sie den Quelltext wie folgt an:

```
    APPEND |Departure:   { details-departureairport   } { departure-
name   }| TO r_output.
    APPEND |Destination: { details-destinationairport } { destination-
name }| TO r_output.
```

5. Optional: Verzichten Sie auf das strukturierte Datenobjekt und verwenden Sie die Tabellenausdrücke direkt in den Zeichenketten-Templates.

© Copyright. Alle Rechte vorbehalten.

a) Kommentieren Sie die Codezeilen, in denen Sie die Strukturen **`für Abfahrt`** und **`Ziel`** füllen.

b) Ersetzen Sie im Zeichenketten-Template `departure-name` durch `airports[ airportid = details-departureairport ]-name`.

c) Ersetzen Sie `destination-name` analog durch `airports[ airportid = details-destinationairport ]-name`.

6. Aktivieren Sie die Klasse. Führen Sie sie aus, und analysieren Sie die Konsolenausgabe. Prüfen Sie, ob die Ausgabe Daten für alle Attribute anzeigt.

a) Drücken Sie **`Strg + F3`**, um die Klasse zu aktivieren.

b) Drücken Sie **`F9`**, um die Klasse auszuführen.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Komplexe interne Tabellen mit Daten füllen.
- Zugriff auf den Inhalt komplexer interner Tabellen.
- Verwenden Sie komplexe interne Tabellen in ABAP SQL.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 6

Überprüfung des Lernerfolgs

1. Welche der folgenden Aussagen trifft auf eine interne Standardtabelle zu?

   Wählen Sie die richtige Antwort.

   - ☐ A Der Schlüssel ist immer eindeutig
   - ☐ B Der Schlüssel ist immer nicht eindeutig
   - ☐ C Man kann wählen, ob der Schlüssel eindeutig oder nicht eindeutig sein soll.

2. Der Typ tt_table wird wie folgt definiert: TYPES tt_table TYPE STANDARD TABLE OF st_connection WITH NON-UNIQUE KEY carrier_id connection_id. Mit welcher DATA-Anweisung würden Sie eine interne Tabelle dieses Typs anlegen?

   Wählen Sie die richtige Antwort.

   - ☐ A DATA table TYPE tt_table.
   - ☐ B DATA table TYPE TABLE OF tt_table.
   - ☐ C DATA table TYPE REF TO tt_table.

3. Welche der folgenden Elemente können Sie zum Füllen einer internen Tabelle verwenden?

   Wählen Sie die richtigen Antworten.

   - ☐ A READ
   - ☐ B VALUE #( )
   - ☐ C LOOP...ENDLOOP
   - ☐ D APPEND

© Copyright. Alle Rechte vorbehalten.

4. Sie möchten Daten aus zwei Datenbanktabellen lesen, sodass die SELECT-Anweisung eine einzelne Ergebnismenge zurückgibt, die keine doppelten Einträge enthält. Welche der folgenden Techniken würden Sie verwenden?

   Wählen Sie die richtige Antwort.

   - [ ] **A** LEFT OUTER JOIN
   - [ ] **B** UNION
   - [ ] **C** INNERER JOIN
   - [ ] **D** UNION ALL

© Copyright. Alle Rechte vorbehalten.

Überprüfung des Lernerfolgs - Antworten

1. Welche der folgenden Aussagen trifft auf eine interne Standardtabelle zu?

   Wählen Sie die richtige Antwort.

   - [ ] A Der Schlüssel ist immer eindeutig
   - [x] B Der Schlüssel ist immer nicht eindeutig
   - [ ] C Man kann wählen, ob der Schlüssel eindeutig oder nicht eindeutig sein soll.

   Das ist richtig.

2. Der Typ tt_table wird wie folgt definiert: TYPES tt_table TYPE STANDARD TABLE OF st_connection WITH NON-UNIQUE KEY carrier_id connection_id. Mit welcher DATA-Anweisung würden Sie eine interne Tabelle dieses Typs anlegen?

   Wählen Sie die richtige Antwort.

   - [x] A DATA table TYPE tt_table.
   - [ ] B DATA table TYPE TABLE OF tt_table.
   - [ ] C DATA table TYPE REF TO tt_table.

   Das ist richtig.

3. Welche der folgenden Elemente können Sie zum Füllen einer internen Tabelle verwenden?

   Wählen Sie die richtigen Antworten.

   - [ ] A READ
   - [x] B VALUE #( )
   - [ ] C LOOP...ENDLOOP
   - [x] D APPEND

© Copyright. Alle Rechte vorbehalten.

4. Sie möchten Daten aus zwei Datenbanktabellen lesen, sodass die SELECT-Anweisung eine einzelne Ergebnismenge zurückgibt, die keine doppelten Einträge enthält. Welche der folgenden Techniken würden Sie verwenden?

   Wählen Sie die richtige Antwort.

   - [ ] A LEFT OUTER JOIN
   - [x] B UNION
   - [ ] C INNERER JOIN
   - [ ] D UNION ALL

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

KAPITEL 7 Datenbankänderungen mit Business Objects implementieren

Lektion 1

Analysieren eines Business-Objekts 300
Übung 16 : Ein Business-Objekt analysieren 307

Lektion 2

Verwendung der Entity Manipulation Language 315
Übung 17 : Daten über EML modifizieren 321

LERNZIELE

- Analysieren Sie ein Business-Objekt.
- Implementieren Sie eine EML-Anweisung.

© Copyright. Alle Rechte vorbehalten.

Kapitel 7
Lektion 1

Analysieren eines Business-Objekts

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Analysieren Sie ein Business-Objekt.

Business-Objekte

Im **ABAP-RESTful-Anwendungsprogrammiermodell** definiert ein Business-Objekt eine bestimmte Entität, z.B. ein Reisebüro. Seine Definition besteht aus zwei Teilen: einer oder mehreren CDS-Views, die die Struktur des Objekts definieren, d.h. die darin enthaltenen Felder, und einer Verhaltensdefinition, die beschreibt, was Sie mit dem Business-Objekt tun können.

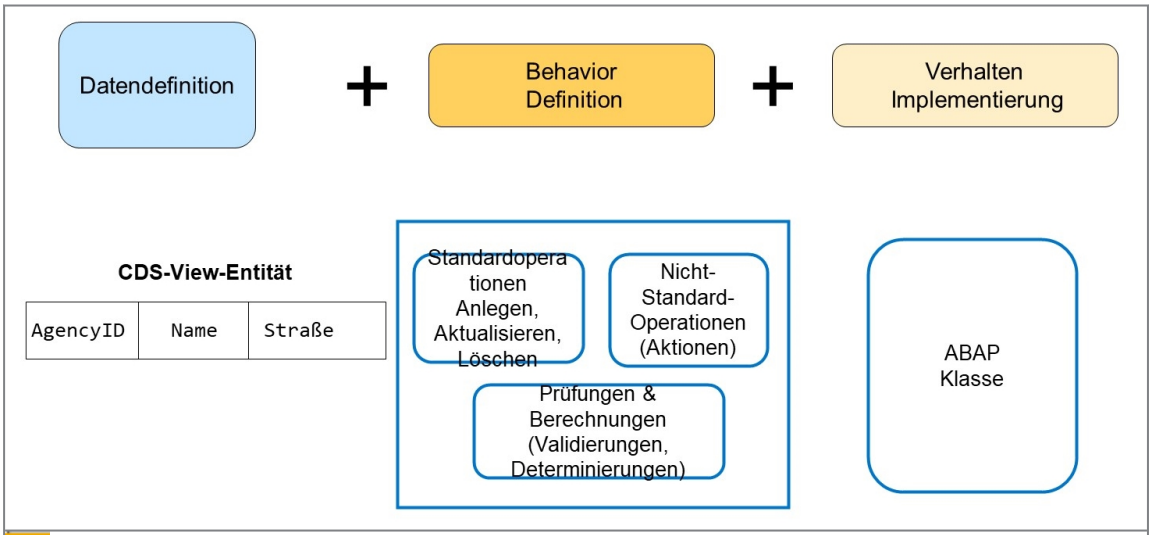


Abbildung 168: Business-Objekte

Die **Verhaltensdefinition** legt fest, welche der Standardoperationen create, update, delete erlaubt sind. Sie kann auch die Definition von Validierungen, Determinierungen und Aktionen enthalten. Validierungen prüfen, ob die Daten korrekt sind, wenn Sie einen Datensatz anlegen oder aktualisieren. Ermittlungen ändern Instanzen von Business-Objekten basierend auf Auslösebedingungen. Aktionen sind Nicht-Standardoperationen, die Sie verwenden, um benutzerdefiniertes, geschäftslogisches Verhalten bereitzustellen. Das Genehmigen einer Bestellung oder das Stornieren eines Flugs sind Aktivitäten, die Sie als Aktion implementieren würden.

Die **Verhaltensimplementierung** besteht aus einer oder mehreren ABAP-Klassen. Hier werden die Validierungen, Determinierungen und Aktionen implementiert. Bei den Standardoperationen unterscheidet das Modell zwei Implementierungsszenarien: Im **nicht verwalteten** Implementierungsszenario werden create, update und delete in der

© Copyright. Alle Rechte vorbehalten.

Verhaltensimplementierung implementiert. Im **verwalteten** Implementierungsszenario übernimmt die Laufzeit diese.

Business-Objekte werden häufig verwendet, um die Transaktionslogik für Fiori-Elements-Apps oder Web-APIs bereitzustellen. Sie können jedoch auch aus dem ABAP-Coding heraus über die **Entity Manipulation Language (EML)** darauf zugreifen. Dies ist eine Reihe von ABAP-Anweisungen, mit denen Sie Daten mithilfe von Business-Objekten anlegen, lesen, aktualisieren und löschen können.

Notiz:
EML kann auch von der Verhaltensimplementierung eines Business-Objekts auf die Anwendungsdaten zugreifen.

```
define root view entity /DMO/R_AgencyTP
  as select from /DMO/I_Agency as Agency
  [...]
{

  key AgencyID,
      Name,
      Street,
      PostalCode,
      City,
      CountryCode,
      PhoneNumber,
      EMailAddress,
      WebAddress,
      Attachment,
      MimeType,
      Filename,
      [...]
}
```

Felder der Entität

Abbildung 169: CDS-View-Entität /DMO/R_AgencyTP

In diesem Kapitel legen Sie eine Klasse an, die ein Business-Objekt zum Ändern von Reisebürodaten verwendet. Die View-Entität enthält das Schlüsselfeld AgencyID und verschiedene weitere Felder, die Informationen über das Reisebüro enthalten.

© Copyright. Alle Rechte vorbehalten.

Verhaltensdefinition und -implementierung

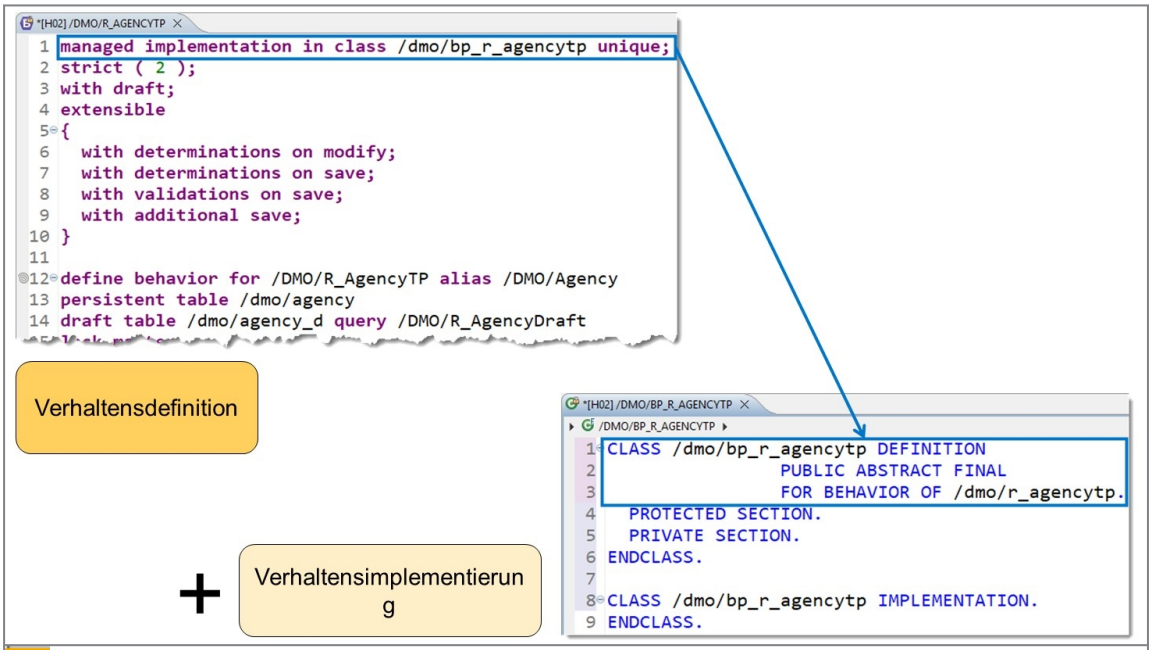


Abbildung 170: Verhaltensdefinition und -implementierung

Das Verhalten eines Business-Objekts besteht aus zwei Teilen: der Verhaltensdefinition und der Verhaltensimplementierung. Die Verhaltensdefinition enthält Informationen darüber, was das Business-Objekt tun kann, während die Verhaltensimplementierung das eigentliche Coding enthält, das das System ausführt.

Die Verhaltensimplementierung ist eine ABAP-Klasse. Sie deklarieren die Klasse in der Verhaltensdefinition in der Anweisung `managed implementation in class <class> unique`. Das eigentliche Coding der Verhaltensimplementierung ist in einer lokalen Klasse innerhalb der von Ihnen angegebenen globalen Klasse enthalten.

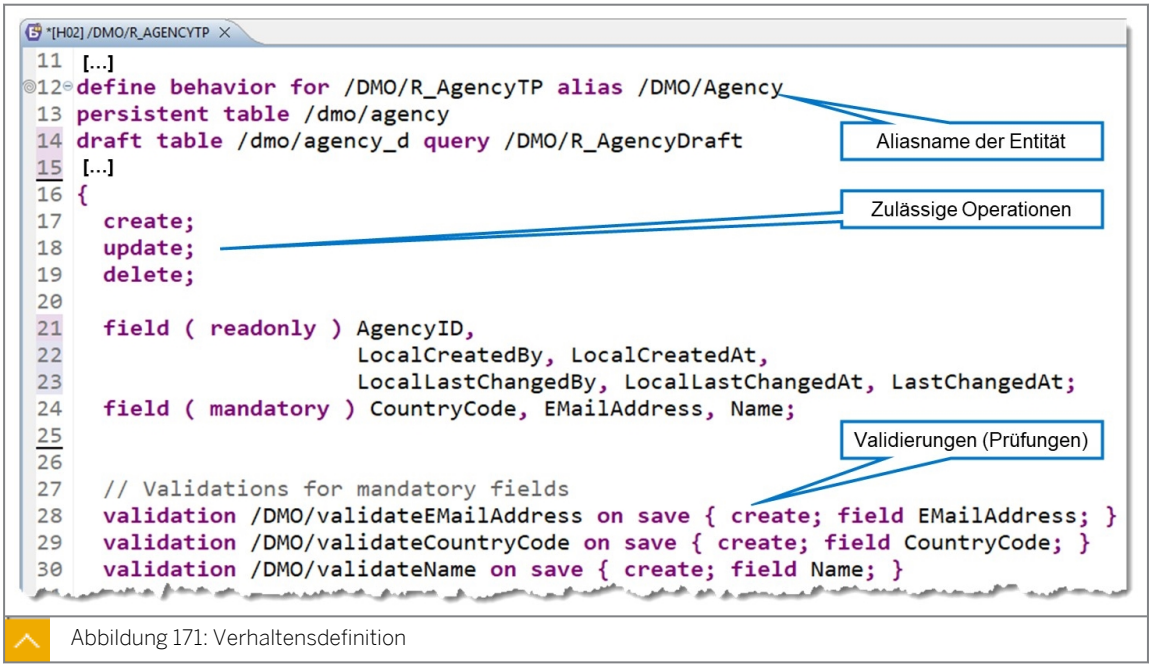


Abbildung 171: Verhaltensdefinition

© Copyright. Alle Rechte vorbehalten.

Eine Verhaltensdefinition ist ein wesentlicher Bestandteil eines Business-Objekts. Sie beschreibt, welche der Standardoperationen erlaubt sind, z.B. Anlegen, Aktualisieren, Löschen. Außerdem werden Prüfungen (Validierungen) definiert, die beim Anlegen oder Ändern von Daten durchgeführt werden.

Das Beispiel hier ist die Verhaltensdefinition für das Reisebüro. Am Anfang der Definition sehen Sie den Namen der CDS-View, die wir gerade angesehen haben, und dass dafür ein Alias definiert ist. Dies ist wichtig, da es sich um den Aliasnamen handelt, über den die Entität mit EML adressiert wird.

Die Verhaltensdefinition verknüpft die CDS-Entität auch mit der Datenbanktabelle, in der die Daten abgelegt sind. In diesem Fall verwendet das Business-Objekt zwei Tabellen: eine für aktive Daten und eine für Entwürfe (Daten, die unvollständig sind und nicht geprüft wurden). Es gibt auch Informationen, die für das Sperren der Daten, für Berechtigungsprüfungen und für die Parallelitätskontrolle relevant sind. Wir werden uns diese Informationen nicht im Detail ansehen. Sie müssen nur wissen, dass sich die Laufzeit um diese Probleme kümmern kann. Sie können auch CDS-Entitäten und Verhaltensdefinitionen basierend auf der Definition einer Datenbanktabelle generieren. In diesem Fall werden Sperren, Berechtigungsprüfungen und Parallelitätskontrollen automatisch behandelt.

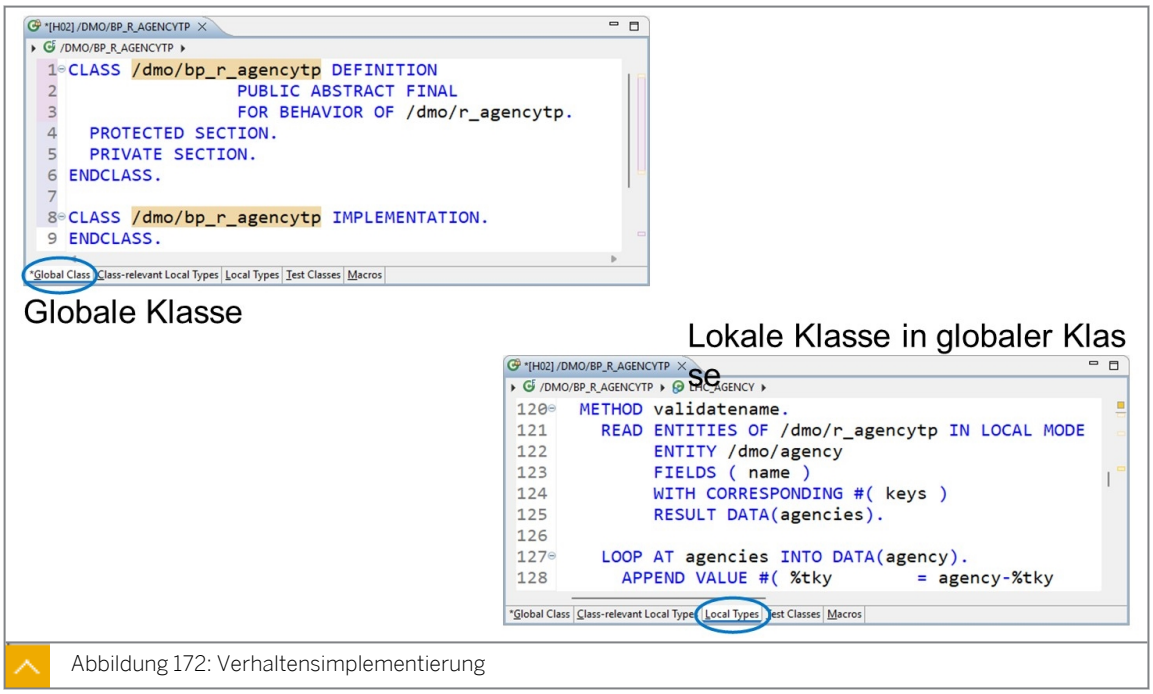


Abbildung 172: Verhaltensimplementierung

Die globale Klasse der Verhaltensimplementierung (auch Behavior-Pool genannt) ist nur eine leere Klassendefinition mit dem speziellen Zusatz FOR BEHAVIOR OF gefolgt vom Namen der Verhaltensdefinition. Die eigentliche Implementierung der Verhaltensdefinition ist eine lokale Klasse innerhalb der globalen Klassendefinition. Sie greifen auf die Klasse zu, indem Sie auf die Registerkarte *Lokale Typen* klicken.

Die Verhaltensimplementierung enthält Business-Objekt-spezifischen Code, z.B. die Implementierung für Validierungen, Determinierungen und Aktionen. Ob sie auch Code für die Standardoperationen (create, update, delete und lock) enthält, hängt von den Details der Verhaltensdefinition ab. Die Verhaltensimplementierung für unser Business-Objekt enthält keinen Quelltext für die Standardoperationen. Dies liegt daran, dass das Business-Objekt die verwaltete Implementierungsart verwendet, in der sich die Laufzeit mit den Standardoperationen befasst.

© Copyright. Alle Rechte vorbehalten.

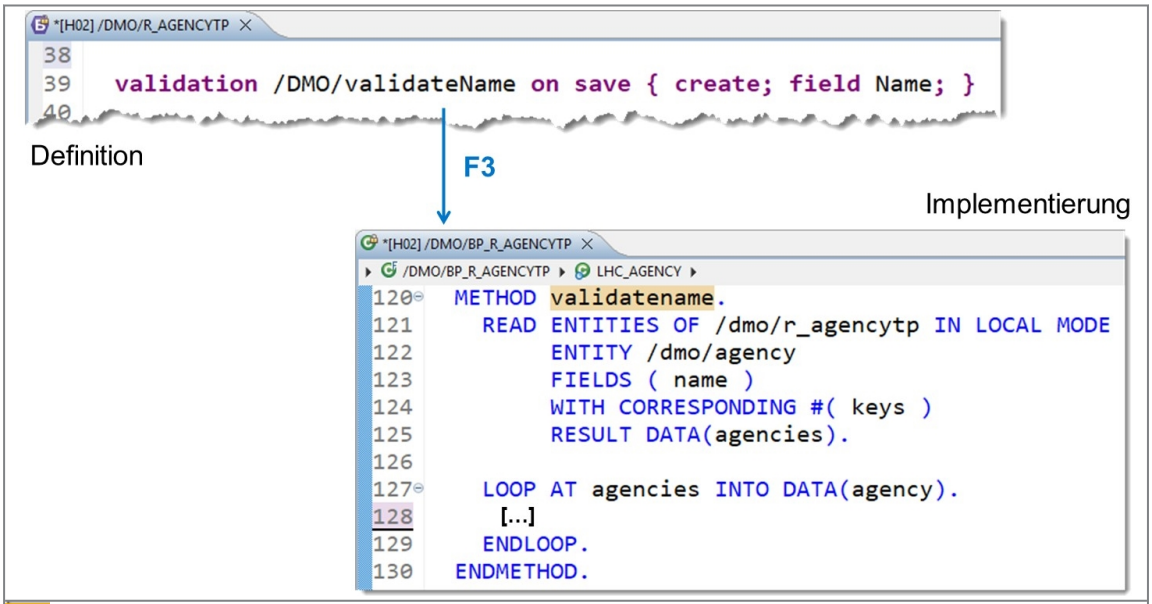


Abbildung 173: Validierungen

Eine Validierung ist eine Prüfung, die die Laufzeit durchführt, wenn Daten geändert werden. Hier wird die Validierung immer beim Anlegen eines neuen Datensatzes durchgeführt (Trigger `create;`). Wenn ein vorhandener Datensatz geändert wird, wird die Validierung nur durchgeführt, wenn das Feld *Name* geändert wurde (Trigger `field Name;`).

Validierungen werden in der Verhaltensdefinition definiert. Für jede Validierung gibt es eine entsprechende Methode in der Verhaltensimplementierung.

BO-Projektionen und BO-Interfaces

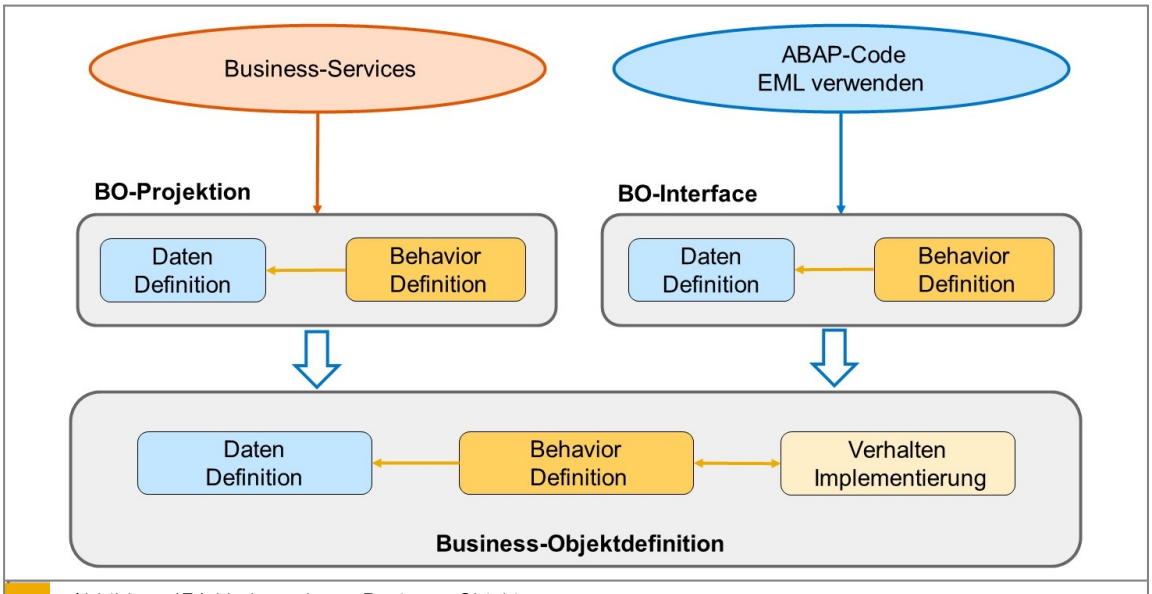


Abbildung 174: Verbrauch von Business-Objekten

Im ABAP-RESTful-Anwendungsprogrammiermodell gibt es zwei wichtige Möglichkeiten, ein Business-Objekt zu konsumieren:

- Über einen **Business-Service**, z.B. einen OData-UI-Service für eine SAP-Fiori-Anwendung.
- Aus ABAP-Code unter Verwendung der **Entity Manipulation Language (EML)**.

© Copyright. Alle Rechte vorbehalten.

Obwohl es technisch möglich ist, sollte ein Business-Objekt nicht direkt konsumiert werden. Stattdessen sollten Konsumenten wie folgt auf **Business-Objekt-Projektionen** (BO-Projektionen) und **Business-Objekt-Interfaces** (BO-Interfaces) zugreifen:

BO-Projektion

Business Services sollten immer eine servicespezifische Projektion des Business-Objekts definieren. Die BO-Projektion gibt an, welche Teilmenge der Daten und Operationen des Business-Objekts über diesen Service verfügbar sind. Eine BO-Projektion kann auch die Definition und Implementierung servicespezifischer Daten und Verhaltensweisen enthalten.

> Notiz:
> Eine BO-Projektion basiert entweder direkt auf dem Business-Objekt oder auf einem BO-Interface.

BO-Interface

Ein Business-Objekt-Interface bietet einen stabilen Zugriff auf die Daten und Operationen eines Business-Objekts. BO-Interfaces sind in der Regel für die Verwendung in anderen Softwarekomponenten freigegeben. Wenn für ein bestimmtes Business-Objekt ein BO-Interface vorhanden ist, sollte immer ABAP-Code mit EML verwendet werden, um auf das BO-Interface zuzugreifen.

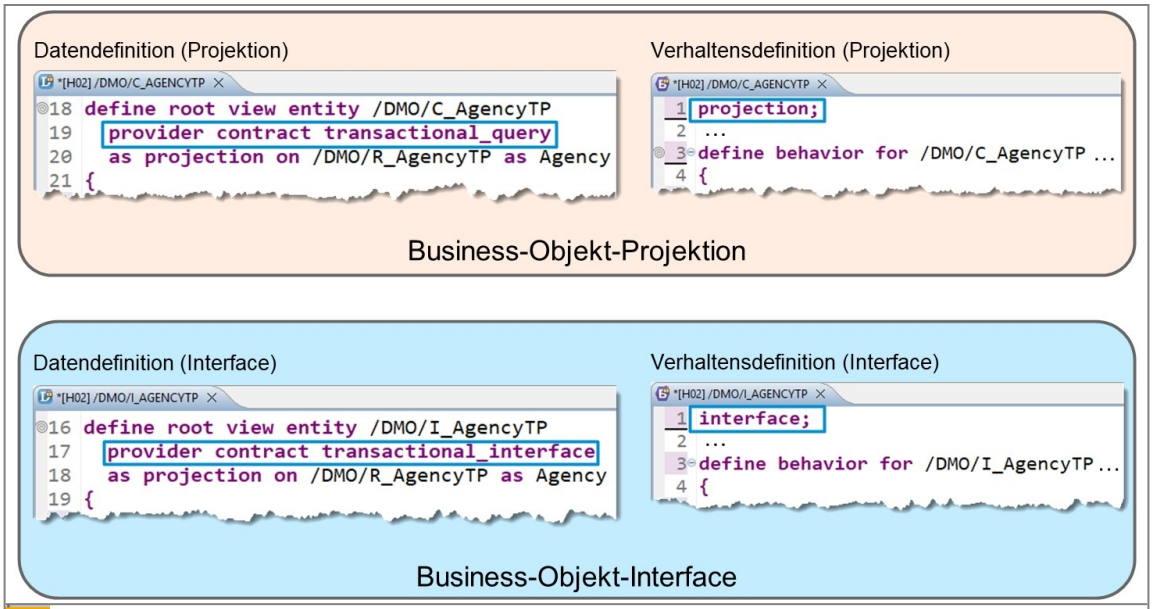


Abbildung 175: BO-Interfaces und BO-Projektionen

Wie ein Business-Objekt besteht eine BO-Projektion aus zwei Teilen: einer oder mehreren CDS-View-Entitäten (definiert in Datendefinitionen) und einer Verhaltensdefinition. Dasselbe gilt für BO-Interfaces. Sie bestehen auch aus einer oder mehreren CDS-Views und einer Verhaltensdefinition.

Die einfachste Möglichkeit, Projektionen und Interfaces zu identifizieren, besteht darin, ihre Verhaltensdefinitionen zu betrachten: Verhaltensdefinitionen für Projektionen beginnen mit dem Schlüsselwort `projection`, Verhaltensdefinitionen für Interfaces beginnen mit dem Schlüsselwort `interface`.

© Copyright. Alle Rechte vorbehalten.

Die CDS-Views von BO-Projektionen und die CDS-Views von BO-Interfaces sind immer **CDS-Projektions-Views**. D.h. ihre Definitionen enthalten den Zusatz `as projection on`, wobei die Definitionen gewöhnlicher CDS-Views `as select from` verwenden. In neueren Releases wird der Anwendungsfall einer Projektions-View mit dem Zusatz `provider contract` hinter dem View-Entitätsnamen wie folgt angegeben:

- `provider contract transactional_interface` für BO-Schnittstellen
- `provider contract transactional_query` für BO-Projektionen

Hinweis:
Für SAP-Entwicklungen gilt folgende Namenskonvention:

- <namespace>C_<...> für BO-Projektionen
- <Namensraum>I_<...> für BO-Interfaces
- <Namensraum>R_<...> für BO-Definitionen

© Copyright. Alle Rechte vorbehalten.

Kapitel 7
Übung 16

Ein Business-Objekt analysieren

In dieser Übung analysieren Sie das Business-Objekt-Interface /DMO/I_AgencyTP, um sich über seine Struktur und die von ihm angebotenen Datenmanipulationsvorgänge zu informieren.

Aufgabe 1: Schnittstellenverhalten analysieren
Analysieren Sie die Verhaltensdefinition des Business-Objekt-Interface **`/DMO/I_AGENCYTP`**.

1. Öffnen Sie die Verhaltensdefinition **`/DMO/I_AGENCYTP`** im Editor.
2. Analysieren Sie den Quelltext. Öffnen Sie die ABAP-Sprachhilfe, um weitere Informationen zu erhalten.

   Wozu dienen die Aussagen `interface;`, `use draft;` und `extensible`?

   Aus wie vielen Entitäten besteht die Business-Objekt-Schnittstelle? Wie lauten die Aliasnamen dieser Entitäten?

   Auf welche Standardoperationen kann über diese Schnittstelle zugegriffen werden?

   Wo finden Sie die Nicht-Standard-Operationen, die die Schnittstelle anbietet?

Aufgabe 2: Datenstruktur analysieren
Analysieren Sie die CDS-View-Entität, die die Datenstruktur der Business-Objekt-Schnittstelle definiert. Navigieren Sie zu ihrer Datenquelle, bis Sie eine CDS-View-Entität erreichen, die keine Projektion ist.

1. Navigieren Sie zur CDS-View-Entität **`/DMO/I_AgencyTP`**, und analysieren Sie den Quelltext.

© Copyright. Alle Rechte vorbehalten.

Welche sind die Schlüsselelemente der CDS-View-Entität?

Ist die Datenstruktur des Business-Objekt-Interfaces auch erweiterbar?

Was ist die Datenquelle der CDS-View-Entität **`/DMO/I_AgencyTP`**?

2. Navigieren Sie zur Datenquelle der CDS-View-Entität **`/DMO/I_AgencyTP`**, und analysieren Sie den Quelltext.

   Ist die CDS-View-Entität **`/DMO/R_AgencyTP`** auch erweiterbar?

   Was ist die Datenquelle der CDS-View-Entität **`/DMO/R_AgencyTP`**?

   Gehört die CDS-View-Entität **`/DMO/R_AgencyTP`** zu einem Business-Objekt-Interface oder einer Business-Objekt-Projektion?

Aufgabe 3: Analysieren des Business-Objekt-Verhaltens

Analysieren Sie die Verhaltensdefinition des Business-Objekts **`/DMO/R_AGENCYTP`**, das unter dem Business-Objekt-Interface **`/DMO/I_AGENCYTP`** liegt.

1. Öffnen Sie die Verhaltensdefinition **`/DMO/R_AGENCYTP`** im Editor.
2. Analysieren Sie den Quelltext. Öffnen Sie die ABAP-Sprachhilfe, um weitere Informationen zu erhalten.

© Copyright. Alle Rechte vorbehalten.

Wozu dient die Aussage `managed ...;`?

Wozu dient der Zusatz? `... implementation in class /dmo/bp_r_agencytp unique;`

Gibt es Elemente, die nicht geändert werden dürfen?

Wozu dient die Aussage? `validation /DMO/validateName on save ... ;`

Wozu dient der Zusatz? `{ create; field Name; }`

Aufgabe 4: Verhaltensimplementierung analysieren

Analysieren Sie die Verhaltensimplementierung des Business-Objekts **`/DMO/R_AGENCYTP`**.

1. Navigieren Sie zur ABAP-Klasse **`/DMO/BP_R_AGENCYTP`**, die die Verhaltensimplementierung des Business-Objekts **`/DMO/R_AGENCYTP`** enthält.

   Warum sind die Definition und Implementierung der globalen Klasse **`/dmo/bp_r_agencytp`** beide leer?

2. Navigieren Sie zur lokalen Klasse **`lhc_Agency`**, und analysieren Sie die Definition.

   Wie viele Methoden sind im privaten Bereich der lokalen Klasse **`lhc_Agency`** definiert? Wozu sollen sie dienen?

© Copyright. Alle Rechte vorbehalten.

Kapitel 7
Lösung 16

Ein Business-Objekt analysieren

In dieser Übung analysieren Sie das Business-Objekt-Interface /DMO/I_AgencyTP, um sich über seine Struktur und die von ihm angebotenen Datenmanipulationsvorgänge zu informieren.

Aufgabe 1: Schnittstellenverhalten analysieren
Analysieren Sie die Verhaltensdefinition des Business-Objekt-Interface **`/DMO/I_AGENCYTP`**.

1. Öffnen Sie die Verhaltensdefinition **`/DMO/I_AGENCYTP`** im Editor.
2. Analysieren Sie den Quelltext. Öffnen Sie die ABAP-Sprachhilfe, um weitere Informationen zu erhalten.

   Wozu dienen die Aussagen `interface;`, `use draft;` und `extensible`?

   `interface;` besagt, dass diese Verhaltensdefinition nicht zum Business-Objekt selbst gehört, sondern zu einer freigegebenen Schnittstelle auf dem Business-Objekt. `use draft;` legt fest, dass diese Schnittstelle die Behandlung von Zwischenversionen von Daten (Draft-Instanzen) unterstützt. `extensible`gibt an, dass Erweiterungen dieser Schnittstelle durch Partner und Kunden unterstützt werden.

   Aus wie vielen Entitäten besteht die Business-Objekt-Schnittstelle? Wie lauten die Aliasnamen dieser Entitäten?

   Das Business-Objekt-Interface besteht aus nur einer Entität, da die Verhaltensdefinition nur eine `DEFINE BEHAVIOR FOR ...` -Anweisung enthält. Der Aliasname ist **`/DMO/Agency`**.

   Auf welche Standardoperationen kann über diese Schnittstelle zugegriffen werden?

   Die Schnittstelle bietet die Operationen create, update, delete, da das Verhalten die Anweisungen `use create;`, `use update;` und `use delete;` enthält.

   Wo finden Sie die Nicht-Standard-Operationen, die die Schnittstelle anbietet?

   Nicht-Standard-Operationen sind nach dem Schlüsselwort `use action` aufgeführt. Beachten Sie, dass es sich bei allen Aktionen in diesem Interface um **Entwurfsaktionen** handelt. Sie beziehen sich auf die Handhabung von Zwischenversionen von Daten.

© Copyright. Alle Rechte vorbehalten.

Aufgabe 2: Datenstruktur analysieren
Analysieren Sie die CDS-View-Entität, die die Datenstruktur der Business-Objekt-Schnittstelle definiert. Navigieren Sie zu ihrer Datenquelle, bis Sie eine CDS-View-Entität erreichen, die keine Projektion ist.

1. Navigieren Sie zur CDS-View-Entität **`/DMO/I_AgencyTP`**, und analysieren Sie den Quelltext.
   a) Gehen Sie zur Anweisung `define behavior for /DMO/I_AgencyTP`.
   b) Halten Sie die **Strg-Taste** gedrückt, und klicken Sie mit der linken Maustaste auf `/DMO/I_AgencyTP`. Alternativ positionieren Sie den Cursor auf `/DMO/I_AgencyTP` und drücken **F3**.

   Welche sind die Schlüsselelemente der CDS-View-Entität?

   Es gibt nur ein Element mit dem Präfix `key`. Der Name dieses View-Elements ist **`AgencyID`**.

   Ist die Datenstruktur des Business-Objekt-Interfaces auch erweiterbar?

   Ja, es gibt eine Gruppe von Annotationen, die mit `@AbapCatalog.extensibility` mit der Annotation `extensible: true` beginnen.

   Was ist die Datenquelle der CDS-View-Entität **`/DMO/I_AgencyTP`**?

   Die CDS-View-Entität **`/DMO/I_AgencyTP`** ist eine Projektion auf die CDS-View-Entität **`/DMO/R_AgencyTP`**.

2. Navigieren Sie zur Datenquelle der CDS-View-Entität **`/DMO/I_AgencyTP`**, und analysieren Sie den Quelltext.
   a) Navigieren Sie zum Quelltextfragment `as projection on /DMO/R_AgencyTP`.
   b) Halten Sie die **Strg-Taste** gedrückt, und klicken Sie mit der linken Maustaste auf `/DMO/R_AgencyTP`. Alternativ positionieren Sie den Cursor auf `/DMO/R_AgencyTP` und drücken **F3**.

   Ist die CDS-View-Entität **`/DMO/R_AgencyTP`** auch erweiterbar?

   Ja, sie enthält auch die Annotation `extensible : true`.

© Copyright. Alle Rechte vorbehalten.

Was ist die Datenquelle der CDS-View-Entität **/DMO/R_AgencyTP**?

Die CDS-View-Entität **/DMO/R_AgencyTP** liest aus der CDS-View-Entität **/DMO/I_Agency**.

Gehört die CDS-View-Entität **/DMO/R_AgencyTP** zu einem Business-Objekt-Interface oder einer Business-Objekt-Projektion?

Nr. Die CDS-View-Entität **/DMO/R_AgencyTP** liest ihre Daten mit dem Schlüsselwort `as select from` und nicht mit `as projection on`. Die CDS-View-Entität **/DMO/R_AgencyTP** gehört zur Definition des Business-Objekts selbst.

Aufgabe 3: Analysieren des Business-Objekt-Verhaltens
Analysieren Sie die Verhaltensdefinition des Business-Objekts **/DMO/R_AGENCYTP**, das unter dem Business-Objekt-Interface **/DMO/I_AGENCYTP** liegt.

1. Öffnen Sie die Verhaltensdefinition **/DMO/R_AGENCYTP** im Editor.
2. Analysieren Sie den Quelltext. Öffnen Sie die ABAP-Sprachhilfe, um weitere Informationen zu erhalten.

   Wozu dient die Aussage `managed ...;`?

   `managed` legt fest, dass es sich um ein Business-Objekt und nicht um ein Interface oder eine Projektion handelt. Er gibt auch an, dass für dieses Business-Objekt die Standardoperationen create, update, delete vom Framework verarbeitet werden.

   Wozu dient der Zusatz? `... implementation in class /dmo/bp_r_agencytp unique;`

   `... implementation in class /dmo/bp_r_agencytp` gibt die ABAP-Klasse für die Verhaltensimplementierung dieses Business-Objekts an. Beachten Sie, dass der Zusatz `unique` obligatorisch ist und festlegt, dass jede Operation genau einmal implementiert werden kann.

   Gibt es Elemente, die nicht geändert werden dürfen?

   Ja, die Anweisung `field ( readonly ) ...` deklariert, dass die Elemente **AgencyID**, **LocalCreatedBy**, **LocalCreatedAt**, **LocalLastChangedBy**, **LocalLastChangedAt** und **LastChangedAt** nicht geändert werden dürfen.

© Copyright. Alle Rechte vorbehalten.

Wozu dient die Aussage? `validation /DMO/validateName on save ... ;`

Diese Anweisung definiert eine Konsistenzprüfung (Validierung) für das Geschäftsobjekt. Der Name gibt an, dass der Inhalt des Elements **Name** der CDS-View-Entität **/DMO/R_AgencyTP** geprüft wird.

Wozu dient der Zusatz? `{ create; field Name; }`

Der Zusatz definiert die Auslösebedingungen: Beim Anlegen eines neuen Datensatzes (Standardoperation create) wird die Prüfung immer durchgeführt. Wenn ein Datenset geändert wird (Standardoperation update), wird die Prüfung nur durchgeführt, wenn sich der Wert des Elements **Name** geändert hat.

Aufgabe 4: Verhaltensimplementierung analysieren

Analysieren Sie die Verhaltensimplementierung des Business-Objekts **/DMO/R_AGENCYTP**.

1. Navigieren Sie zur ABAP-Klasse **/DMO/BP_R_AGENCYTP**, die die Verhaltensimplementierung des Business-Objekts **/DMO/R_AGENCYTP** enthält.
   a) Gehen Sie zur Anweisung `managed implementation in class /dmo/bp_r_agencytp unique;`.
   b) Halten Sie die **Strg-Taste** gedrückt, und klicken Sie mit der linken Maustaste auf `/dmo/bp_r_agencytp`. Alternativ positionieren Sie den Cursor auf `/dmo/bp_r_agencytp` und drücken **F3**.

   Warum sind die Definition und Implementierung der globalen Klasse **/dmo/bp_r_agencytp** beide leer?

   Die globale Klasse **/dmo/bp_r_agencytp** ist ein Behavior-Pool. Sie dient als Verschalung für eine oder mehrere lokale Klassen. Die Verhaltensimplementierung selbst befindet sich in diesen lokalen Klassen.

2. Navigieren Sie zur lokalen Klasse **lhc_Agency**, und analysieren Sie die Definition.
   a) Wechseln Sie zur Registerkarte *Lokale Typen*. Expandieren Sie alternativ den Wurzelknoten */DMO/BP_R_AGENCYTP* in der Sicht *Gliederung*, und wählen Sie *LHC_AGENCY*.

   Wie viele Methoden sind im privaten Bereich der lokalen Klasse **lhc_Agency** definiert? Wozu sollen sie dienen?

   Es gibt fünf Methoden. Vier Methoden implementieren Validierungen (Zusatz `FOR VALIDATE ON SAVE`). Eine Methode implementiert Berechtigungsprüfungen (Zusatz `FOR GLOBAL AUTHORIZATION`).

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Analysieren Sie ein Business-Objekt.

© Copyright. Alle Rechte vorbehalten.

Verwendung der Entity Manipulation Language

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Implementieren Sie eine EML-Anweisung.

Entity Manipulation Language (EML)

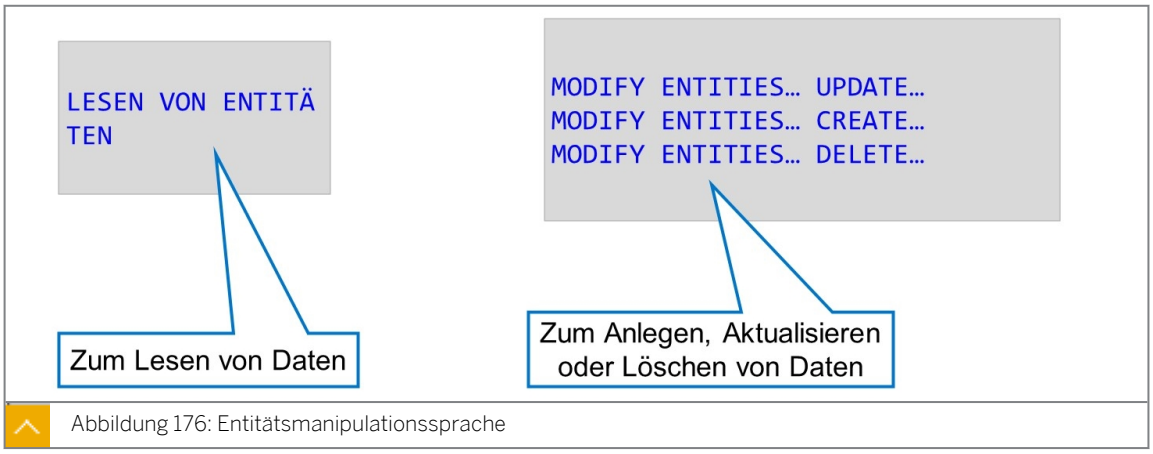


Abbildung 176: Entitätsmanipulationssprache

EML besteht aus Anweisungen, mit denen Sie die Daten eines Business-Objekts manipulieren können. Zum Lesen von Daten verwenden Sie die Anweisung READ ENTITIES; für alle anderen Operationen verwenden Sie die MODIFY ENTITIES -Anweisung mit dem entsprechenden Zusatz UPDATE, CREATE oder DELETE.

Notiz:
Sie können die Operationen CREATE, UPDATE und DELETE nur verwenden, wenn die Verhaltensdefinition des Business-Objekt-Interface die entsprechende `use create`, `use update` oder `use delete` -Direktive enthält. Der Versuch, eine verbotene Operation zu verwenden, verursacht einen Syntaxfehler.

© Copyright. Alle Rechte vorbehalten.

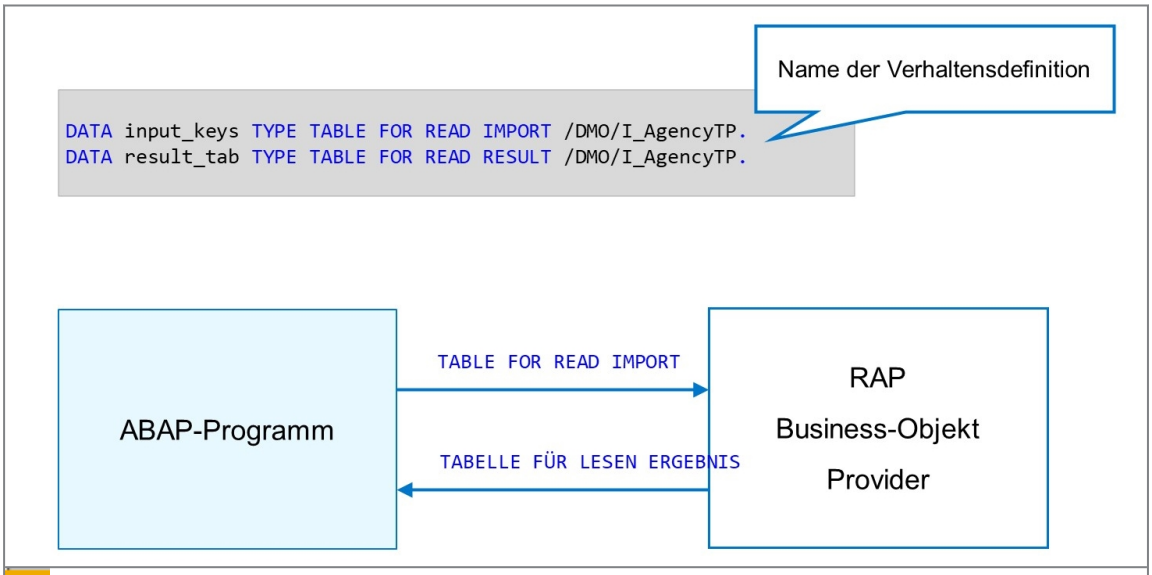


Abbildung 177: Daten werden gelesen

Um Daten aus einem Business-Objekt zu lesen, verwenden Sie die Anweisung READ ENTITIES. Die Anweisung hat zwei wichtige Parameter: eine interne Tabelle mit den Schlüsseln der Daten, die Sie lesen möchten, und eine weitere Tabelle, die die Ergebnisse der Query enthält.

Diese internen Tabellen haben spezielle Datentypen, die als abgeleitete Verhaltensdefinitionstypen bezeichnet werden. Das System legt sie automatisch an, wenn ein Entwickler eine Verhaltensdefinition anlegt, und sie enthalten einige oder alle Felder des Business-Objekts zusammen mit weiteren Feldern, die steuern, wie das System eine bestimmte Anforderung verarbeitet. Sie deklarieren die internen Tabellen mit dem neuen Zusatz `TYPE TABLE FOR <operation>` in der DATA-Anweisung.

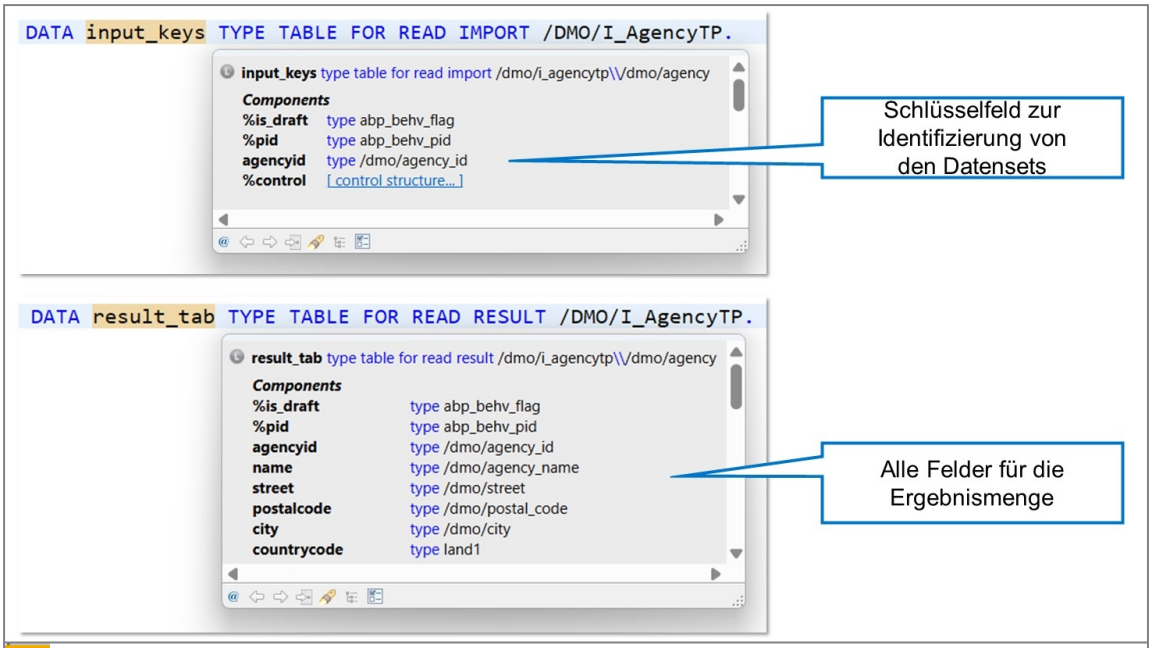


Abbildung 178: Abgeleitete Typen für Leseoperationen

Der Typ `TABLE FOR READ IMPORT` enthält das oder die Schlüsselfelder des Business-Objekts. Die *%control* -Struktur ist eine generierte Struktur, die angibt, welche Felder des

© Copyright. Alle Rechte vorbehalten.

Business-Objekts tatsächlich in der aktuellen Operation verwendet werden. In unserem Beispiel füllt das System die Struktur automatisch und Sie müssen sich nicht darum kümmern.

Der Typ `TABLE FOR READ RESULT` enthält alle Felder des Business-Objekts. Diese Tabelle enthält die Ergebnismenge nach Ausführung der Leseanweisung.

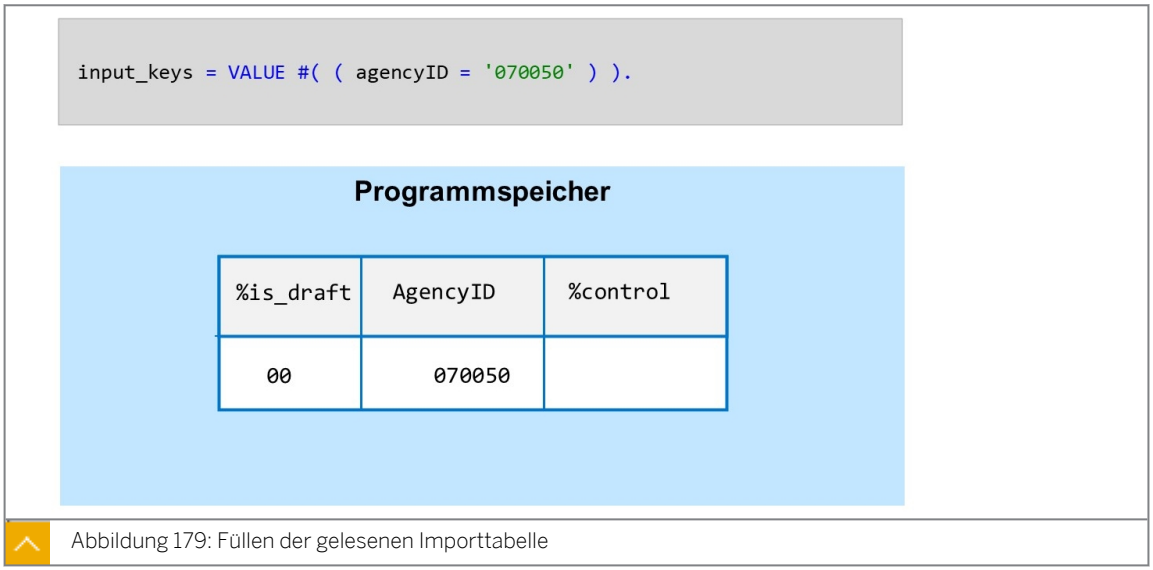


Abbildung 179: Füllen der gelesenen Importtabelle

Die gelesene Importtabelle enthält eine Spalte für jedes Schlüsselfeld des Business-Objekts, in diesem Fall *agencyID*. Um eine bestimmte Agentur zu lesen, fügen Sie der internen Tabelle eine Zeile mit ihrem Schlüssel hinzu. Neben dem oder den Schlüsselfeldern enthält die Tabelle die Spalten *%is_draft* und *%control*. Mit *%is_draft* können Sie angeben, ob Sie Entwurfsdaten oder aktive Daten lesen möchten. Mit *der %control* -Struktur wird angegeben, welche Felder gelesen werden sollen.

Notiz:
In unserem Beispiel lesen wir nur aktive Daten, und die *%control* -Struktur wird vom Framework gefüllt, basierend auf der Feldliste nach dem Zusatz FIELDS.

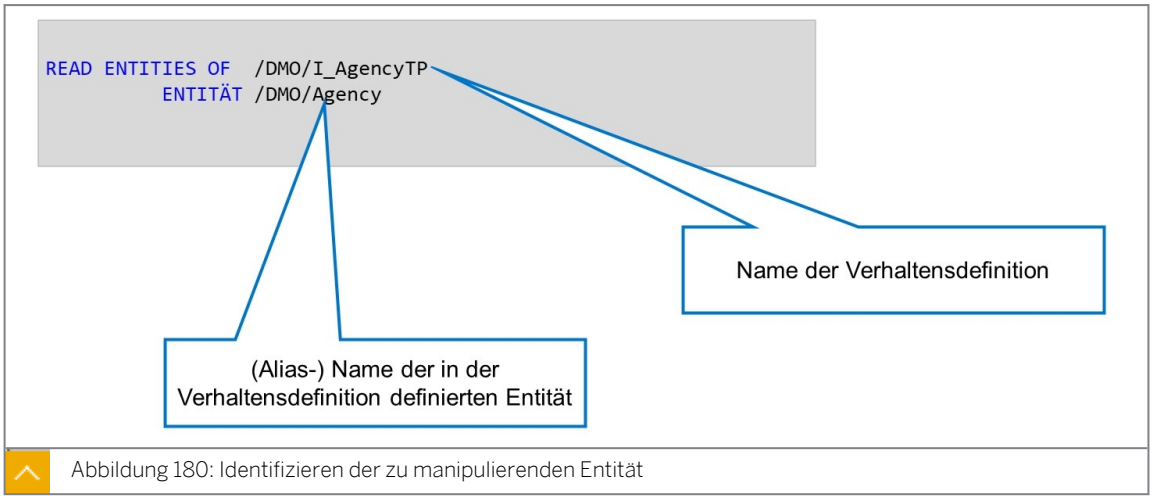


Abbildung 180: Identifizieren der zu manipulierenden Entität

Wenn Sie ein Business-Objekt mit der Anweisung READ ENTITIES OF oder MODIFY ENTITIES OF bearbeiten, müssen Sie zuerst den Namen der Verhaltensdefinition angeben. Darauf folgt

© Copyright. Alle Rechte vorbehalten.

das Schlüsselwort ENTITY und der Name der Entität, mit der Sie arbeiten möchten. Wenn die Entität einen Aliasnamen hat, sollten Sie ihn hier verwenden.

> Notiz:
> Der Aliasname kann nicht hinter dem Zusatz OF verwendet werden. Dies liegt daran, dass Sie an dieser Stelle technisch nicht den Namen der Entität, sondern den Namen der Verhaltensdefinition angeben.

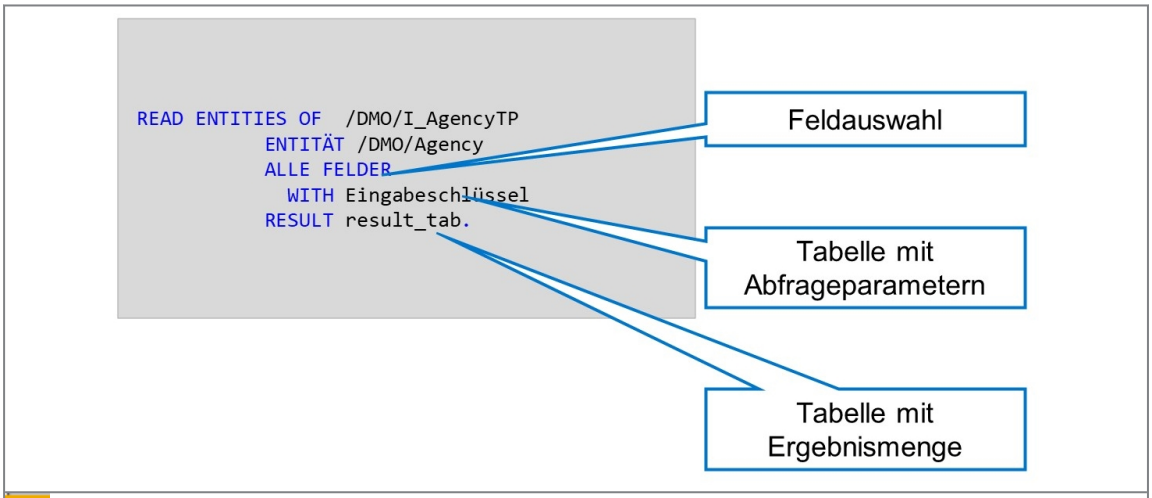


Abbildung 181: READ ENTITIES -Anweisung

Die READ ENTITIES -Anweisung liest Business-Objekt-Daten gemäß den Schlüsseln, die Sie im WITH -Zusatz übergeben. Sie gibt das Ergebnis im Zusatz RESULT in der internen Tabelle zurück. In der -Anweisung können Sie auch angeben, welche Felder des Business-Objekts Sie benötigen. In diesem Beispiel haben wir den Zusatz ALL FIELDS verwendet, um alle Felder zurückzugeben. Wenn Sie jedoch nur eine Teilmenge der Felder benötigen, können Sie die Variante FIELDS (*field1 field2* ...) verwenden, um die Menge der gelesenen Daten einzuschränken.

> Notiz:
> Anders als bei einer SELECT -Anweisung ist die Feldliste nicht kommasepariert.

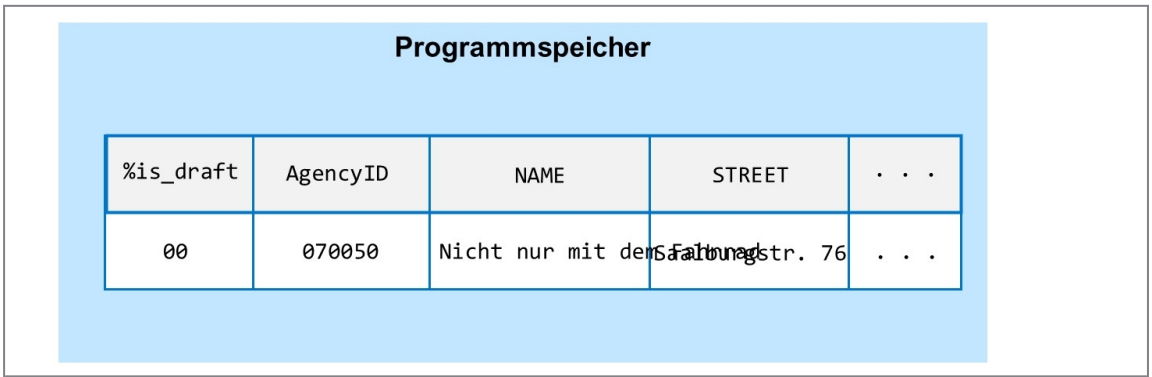


Abbildung 182: Ergebnistabelle

© Copyright. Alle Rechte vorbehalten.

Die Ergebnistabelle enthält alle Felder des Business-Objekts zusammen mit dem Steuerungsfeld *%is_draft*. Wenn Ihre READ ENTITIES -Anweisung die Variante ALL FIELDS enthält, stellt das System die Werte aller Felder bereit. Wenn Sie die Variante FIELDS (*f1* ... *fn*) verwenden, werden nur die Felder gefüllt, die Sie angefordert haben.

> Notiz:
> Schlüsselfelder werden immer gefüllt, auch wenn sie nicht in der Variante FIELDS (*f1* ... *fn*) angegeben sind.

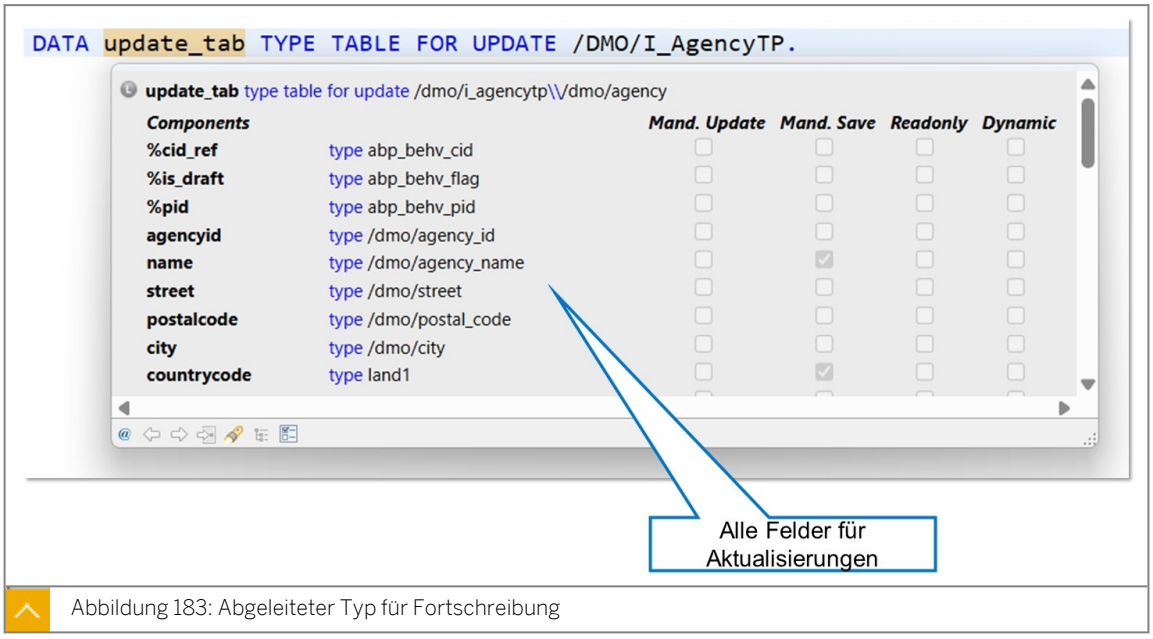


Abbildung 183: Abgeleiteter Typ für Fortschreibung

Wenn Sie Daten aktualisieren möchten, deklarieren Sie eine interne Tabelle mit `TYPE TABLE FOR UPDATE`. Diese enthält alle Felder des Business-Objekts sowie die *%control* -Struktur. In unserer Variante der MODIFY ENTITIES -Anweisung wird die *%control* -Struktur automatisch gefüllt.

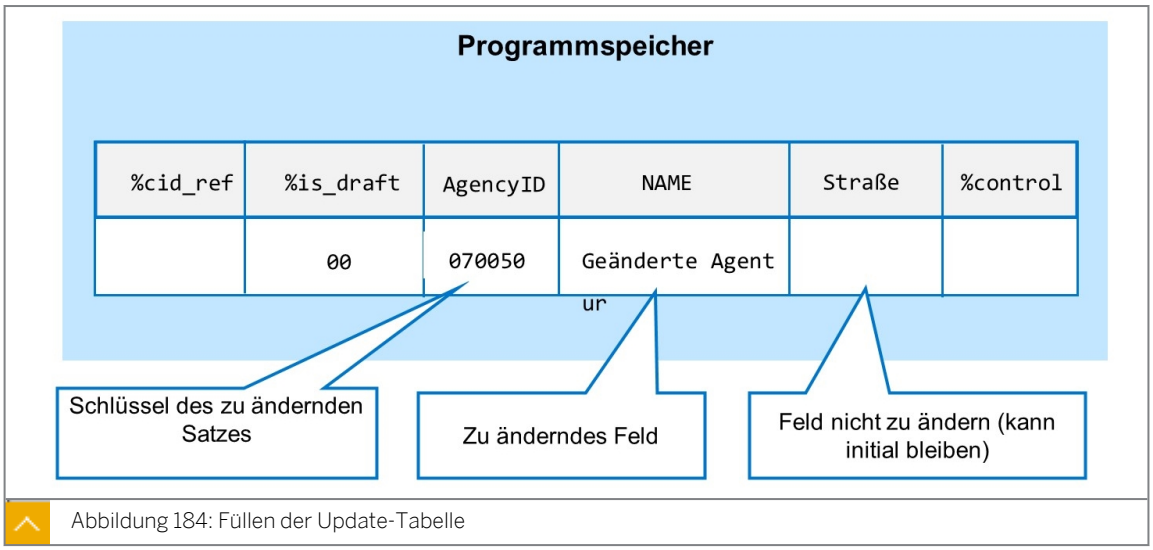


Abbildung 184: Füllen der Update-Tabelle

© Copyright. Alle Rechte vorbehalten.

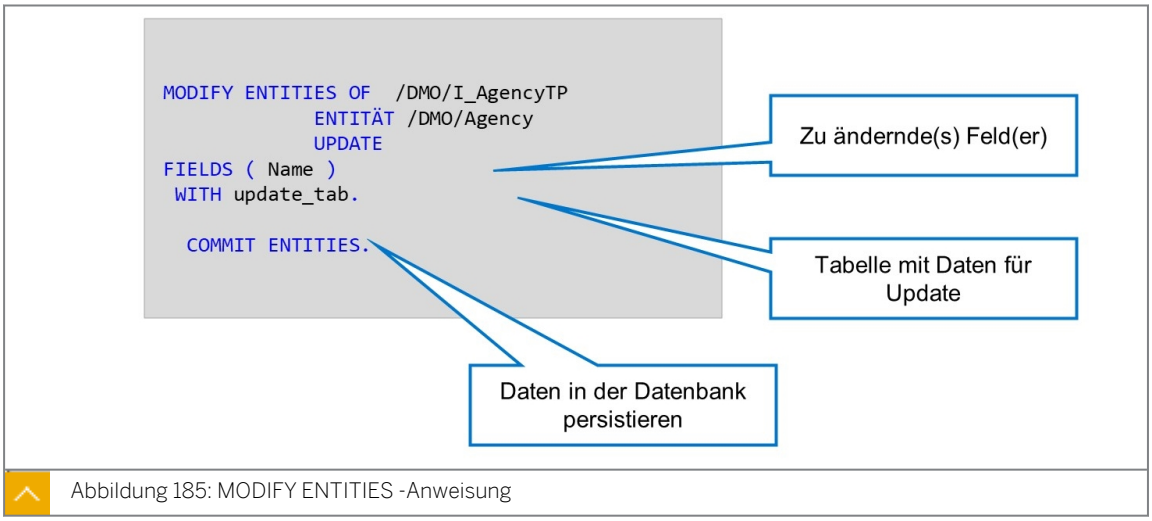


Abbildung 185: MODIFY ENTITIES -Anweisung

Die MODIFY ENTITIES -Anweisung aktualisiert Daten im Transaktionspuffer. Im Zusatz FIELDS legen Sie fest, welche Felder geändert werden sollen. Im Zusatz WITH übergeben Sie die interne Tabelle mit den Daten, die Sie aktualisieren möchten.

Wenn Sie EML außerhalb des Business-Objekts verwenden, müssen Sie die Anweisung COMMIT ENTITIES verwenden, um die Sicherungssequenz auszulösen und die Daten in der Datenbank zu persistieren.

Notiz:
Später in diesem Kurs verwenden wir EML in der Verhaltensimplementierung des Business-Objekts. Innerhalb der Verhaltensimplementierung ist es weder erforderlich noch zulässig, den Commit mit COMMIT ENTITIES auszulösen.

© Copyright. Alle Rechte vorbehalten.

Kapitel 7
Übung 17

Daten über EML modifizieren

In dieser Übung verwenden Sie EML, um die Standardoperation *update* für die Wurzelentität des Business-Objekts **`/DMO/R_AGENCYTP`** durchzuführen. Um spätere Inkonsistenzen zu vermeiden, greifen Sie nicht direkt auf das Business-Objekt zu, sondern über die freigegebene stabile Schnittstelle **`/DMO/I_AGENCYTP.`**

Vorlage:

keine

Lösung:

/LRN/CL_S4D400_BOS_EML (globale Klasse)

Aufgabe 1: Aktuelle Daten analysieren
Verwenden Sie das Tool *Data Preview*, um den aktuellen Namen des Reisebüros mit der ID „0700##" zu prüfen, wobei ## für Ihre Gruppennummer steht.

1. Öffnen Sie die Definition der CDS-View-Entität **`/DMO/I_AgencyTP`**.
2. Öffnen Sie das Tool *Datenvorschau*.
3. Prüfen Sie den aktuellen Namen des Reisebüros mit der ID „0700##", wobei ## für Ihre Gruppennummer steht.

Aufgabe 2: Globale Klasse anlegen
Legen Sie in Ihrem eigenen Paket eine neue ABAP-Klasse an.

1. Legen Sie eine neue ABAP-Klasse mit dem Namen **`ZCL_##_EML`** an, wobei **`##`** für Ihre Gruppennummer steht. Stellen Sie sicher, dass die Klasse das Interface **`IF_OO_ADT_CLASSRUN`** implementiert.

Aufgabe 3: Daten aktualisieren
Implementieren Sie eine EML-Anweisung, um den Namen des Reisebüros mit der ID „0700##" (## steht für Ihre Gruppennummer) zu ändern.

1. Deklarieren Sie in der Methode **`IF_OO_ADT_CLASSRUN~MAIN`** eine interne Tabelle mit dem korrekten abgeleiteten Typ für eine EML-Aktualisierung der Entität **`/DMO/I_AgencyTP`** (Namensvorschlag: **`agencies_upd)`**.
2. Füllen Sie die interne Tabelle mit einer einzelnen Zeile mit „0700##" als Wert für die Spalte **`AGENCYID`** (wobei ## für Ihre Gruppennummer steht) und einem beliebigen neuen Wert für die Spalte **`NAME`**.
3. Implementieren Sie die EML-Anweisung MODIFY ENTITIES für das Business-Objekt-Interface **`/DMO/I_AgencyTP`**. Aktualisieren Sie die Daten der Wurzelentität, um sicherzustellen, dass nur das Feld **`NAME`** geändert wird.

© Copyright. Alle Rechte vorbehalten.

Hinweis:
Beachten Sie, dass das Interface-Verhalten den Aliasnamen **`/DMO/Agency`** für die Wurzelentität **`/DMO/I_AgencyTP`** definiert.

4. Um ein sichtbares Ergebnis aus der Anwendung zu erhalten, schreiben Sie mit der Methode **`out->write`** ein Textliteral in die Konsole.
5. Aktivieren und testen Sie die Klasse.
6. Prüfen Sie, ob der neue Name auf die Datenbank geschrieben wurde.
7. Fügen Sie Ihrem Quelltext die Anweisung COMMIT ENTITIES hinzu, um die Änderungen zu bestätigen.
8. Aktivieren und testen Sie die Klasse erneut. Bestätigen Sie, dass die Änderungen nun in der Datenbank vorhanden sind.

© Copyright. Alle Rechte vorbehalten.

Daten über EML modifizieren

In dieser Übung verwenden Sie EML, um die Standardoperation *update* für die Wurzelentität des Business-Objekts **`/DMO/R_AGENCYTP`** durchzuführen. Um spätere Inkonsistenzen zu vermeiden, greifen Sie nicht direkt auf das Business-Objekt zu, sondern über die freigegebene stabile Schnittstelle **`/DMO/I_AGENCYTP.`**

Vorlage:

keine

Lösung:

/LRN/CL_S4D400_BOS_EML (globale Klasse)

Aufgabe 1: Aktuelle Daten analysieren
Verwenden Sie das Tool *Data Preview*, um den aktuellen Namen des Reisebüros mit der ID „0700##" zu prüfen, wobei ## für Ihre Gruppennummer steht.

1. Öffnen Sie die Definition der CDS-View-Entität **`/DMO/I_AgencyTP`**.
   a) Wenn Sie die Datendefinition **`/DMO/I_AGENCYTP`** aus der vorherigen Übung noch geöffnet haben, wechseln Sie zur entsprechenden Registerkarte. Drücken Sie andernfalls **`Strg + Umschalt + A`**, um das ABAP-Entwicklungsobjekt zu öffnen.
2. Öffnen Sie das Tool *Datenvorschau*.
   a) Klicken Sie mit der rechten Maustaste auf eine beliebige Stelle im Quelltext, um das Kontextmenü zu öffnen.
   b) Wählen Sie im Kontextmenü *Open With* → *Data Preview*.
3. Prüfen Sie den aktuellen Namen des Reisebüros mit der ID „0700##", wobei ## für Ihre Gruppennummer steht.
   a) Blättern Sie nach unten zum Eintrag mit dem Wert „0700##" in der Spalte **`AgencyID`**, und notieren Sie sich den Wert in der Spalte **`Name`**.

Aufgabe 2: Globale Klasse anlegen
Legen Sie in Ihrem eigenen Paket eine neue ABAP-Klasse an.

1. Legen Sie eine neue ABAP-Klasse mit dem Namen **`ZCL_##_EML`** an, wobei **`##`** für Ihre Gruppennummer steht. Stellen Sie sicher, dass die Klasse das Interface **`IF_OO_ADT_CLASSRUN`** implementiert.
   a) Wählen Sie *File* → *New* → *ABAP Class*.
   b) Geben Sie den Namen Ihres Pakets in das Feld *Package* ein. Geben Sie im Feld *Name* den Namen **`ZCL_##_EML`** ein, wobei **`##`** für Ihre Gruppennummer steht. Geben Sie eine Beschreibung ein.
   c) Wählen Sie im Gruppenrahmen *Interfaces* die Option *Add*.

© Copyright. Alle Rechte vorbehalten.

d) Geben Sie **IF_OO_ADT_CLASSRUN** ein. Wenn das Interface in der Trefferliste angezeigt wird, fügen Sie es mit Doppelklick zur Klassendefinition hinzu.

e) Wählen Sie *Next*.

f) Wählen Sie Ihren Transportauftrag aus, und wählen Sie *Finish*.

Aufgabe 3: Daten aktualisieren

Implementieren Sie eine EML-Anweisung, um den Namen des Reisebüros mit der ID „0700##" (## steht für Ihre Gruppennummer) zu ändern.

1. Deklarieren Sie in der Methode **IF_OO_ADT_CLASSRUN~MAIN** eine interne Tabelle mit dem korrekten abgeleiteten Typ für eine EML-Aktualisierung der Entität **/DMO/I_AgencyTP** (Namensvorschlag: **agencies_upd)** .
   a) Fügen Sie folgenden Code hinzu:

```
DATA agencies_upd TYPE TABLE FOR UPDATE /DMO/I_AgencyTP.
```

2. Füllen Sie die interne Tabelle mit einer einzelnen Zeile mit „0700##" als Wert für die Spalte **AGENCYID** (wobei ## für Ihre Gruppennummer steht) und einem beliebigen neuen Wert für die Spalte **NAME**.
   a) Fügen Sie nach der Datendeklaration einen Quelltext hinzu, der dem folgenden Code ähnelt:

```
agencies_upd = VALUE #( ( agencyid = '0700##' name = 'Some fancy
new name' ) ).
```

3. Implementieren Sie die EML-Anweisung MODIFY ENTITIES für das Business-Objekt-Interface **/DMO/I_AgencyTP**. Aktualisieren Sie die Daten der Wurzelentität, um sicherzustellen, dass nur das Feld **NAME** geändert wird.

Hinweis:
Beachten Sie, dass das Interface-Verhalten den Aliasnamen **/DMO/Agency** für die Wurzelentität **/DMO/I_AgencyTP** definiert.

   a) Fügen Sie am Ende der Methode den folgenden Code hinzu:

```
MODIFY ENTITIES OF /dmo/i_agencytp
   ENTITY /dmo/agency
   UPDATE FIELDS ( name )
     WITH agencies_upd.
```

4. Um ein sichtbares Ergebnis aus der Anwendung zu erhalten, schreiben Sie mit der Methode **out->write** ein Textliteral in die Konsole.
   a) Geben Sie nach der `MODIFY ENTITIES`-Anweisung folgenden Quelltext ein:

```
out->write( `Method execution finished!`  ).
```

5. Aktivieren und testen Sie die Klasse.
   a) Drücken Sie **Strg + F3**, um die Klasse zu aktivieren.
   b) Drücken Sie **F9**, um die Klasse auszuführen.
6. Prüfen Sie, ob der neue Name auf die Datenbank geschrieben wurde.

© Copyright. Alle Rechte vorbehalten.

a) Kehren Sie zur Datenvorschau für die CDS-View-Entität **/DMO/I_AgencyTP** zurück.

b) Wählen Sie in der Symbolleiste der Registerkarte *Datenvorschau* die Option *Aktualisieren*.

Warum ist Ihre Datenänderung noch nicht in der Datenbank angekommen?

Ihr Programm speichert die Änderungen im Anwendungspuffer, sie werden jedoch erst mit der Anweisung COMMIT ENTITIES in die Datenbank geschrieben.

7. Fügen Sie Ihrem Quelltext die Anweisung COMMIT ENTITIES hinzu, um die Änderungen zu bestätigen.

a) Kehren Sie zur Methode **IF_OO_ADT_CLASSRUN~MAIN** in Ihrer globalen Klasse **ZCL_##_EML** zurück.

b) Passen Sie den Quelltext wie folgt an:

```
METHOD if_oo_adt_classrun~main.

    DATA agencies_upd TYPE TABLE FOR UPDATE /dmo/i_agencytp.

    agencies_upd = VALUE #( ( agencyid = '0700##' name = 'Some fancy
new name' ) ).

    MODIFY ENTITIES OF /dmo/i_agencytp
      ENTITY /dmo/agency
      UPDATE FIELDS ( name )
        WITH agencies_upd.

    COMMIT ENTITIES.

    out->write( `Method execution finished!`  ).

  ENDMETHOD.
```

8. Aktivieren und testen Sie die Klasse erneut. Bestätigen Sie, dass die Änderungen nun in der Datenbank vorhanden sind.

a) Drücken Sie **Strg + F3**, um die Klasse zu aktivieren.

b) Drücken Sie **F9**, um die Klasse auszuführen.

c) Kehren Sie zur Datenvorschau für die CDS-View-Entität **/DMO/I_AgencyTP** zurück.

d) Wählen Sie in der Symbolleiste der Registerkarte *Datenvorschau* die Option *Aktualisieren*.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Implementieren Sie eine EML-Anweisung.

© Copyright. Alle Rechte vorbehalten.

Kapitel 7

Überprüfung des Lernerfolgs

1. Was ist in unserem Szenario in einer Verhaltensimplementierung kodiert?

   Wählen Sie die richtigen Antworten.

   - ☐ A Validierungen
   - ☐ B Ermittlungen
   - ☐ C Update (Aktualisieren)

2. In der READ ENTITIES -Anweisung verwenden Sie eine interne Tabelle mit einem speziellen Typ (TYPE TABLE FOR READ RESULT). Wie wird dieser Typ angelegt?

   Wählen Sie die richtige Antwort.

   - ☐ A Der Entwickler des Business-Objekts legt es als globalen Typ an.
   - ☐ B Der Entwickler des Business-Objekts legt es in der Verhaltensimplementierung an.
   - ☐ C Das System legt sie automatisch an.

© Copyright. Alle Rechte vorbehalten.

Kapitel 7

Überprüfung des Lernerfolgs - Antworten

1. Was ist in unserem Szenario in einer Verhaltensimplementierung kodiert?

   Wählen Sie die richtigen Antworten.

   - [X] A Validierungen
   - [X] B Ermittlungen
   - [ ] C Update (Aktualisieren)

   Das ist richtig. Validierungen und Determinierungen werden in der Verhaltensimplementierung kodiert. Im verwalteten Implementierungsszenario kümmert sich das Framework jedoch um die Standardoperationen create, update und delete.

2. In der READ ENTITIES -Anweisung verwenden Sie eine interne Tabelle mit einem speziellen Typ (TYPE TABLE FOR READ RESULT). Wie wird dieser Typ angelegt?

   Wählen Sie die richtige Antwort.

   - [ ] A Der Entwickler des Business-Objekts legt es als globalen Typ an.
   - [ ] B Der Entwickler des Business-Objekts legt es in der Verhaltensimplementierung an.
   - [X] C Das System legt sie automatisch an.

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

KAPITEL 8 Das ABAP-RESTful-Anwendungsprogrammiermodell beschreiben

Lektion 1

Einführung in das Programmiermodell 331

Lektion 2

Die Architektur erkunden 335
Übung 18 : Datenbanktabelle kopieren 343
Übung 19 : OData-UI-Service generieren und Vorschau anzeigen 357

Lektion 3

ABAP-Logik hinzufügen 366
Übung 20 : Flugpreis validieren 381

Lektion 4

Verbesserung der Benutzerfreundlichkeit 389
Übung 21 : Benutzungsoberfläche anpassen 395

LERNZIELE

- Beschreiben Sie den Prozess zur Entwicklung eines OData-Service mit dem ABAP-RESTful-Anwendungsprogrammiermodell.
- Legen Sie eine Datenbanktabelle an.
- Generieren Sie die Entwicklungsobjekte für einen OData-UI-Service.
- Implementieren Sie das Verhalten eines Business-Objekts.
- Ordnen Sie Felder in der App an.
- Stellen Sie eine Eingabehilfe bereit.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Lektion 1

Einführung in das Programmiermodell

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Beschreiben Sie den Prozess zur Entwicklung eines OData-Service mit dem ABAP-RESTful-Anwendungsprogrammiermodell.

Überblick

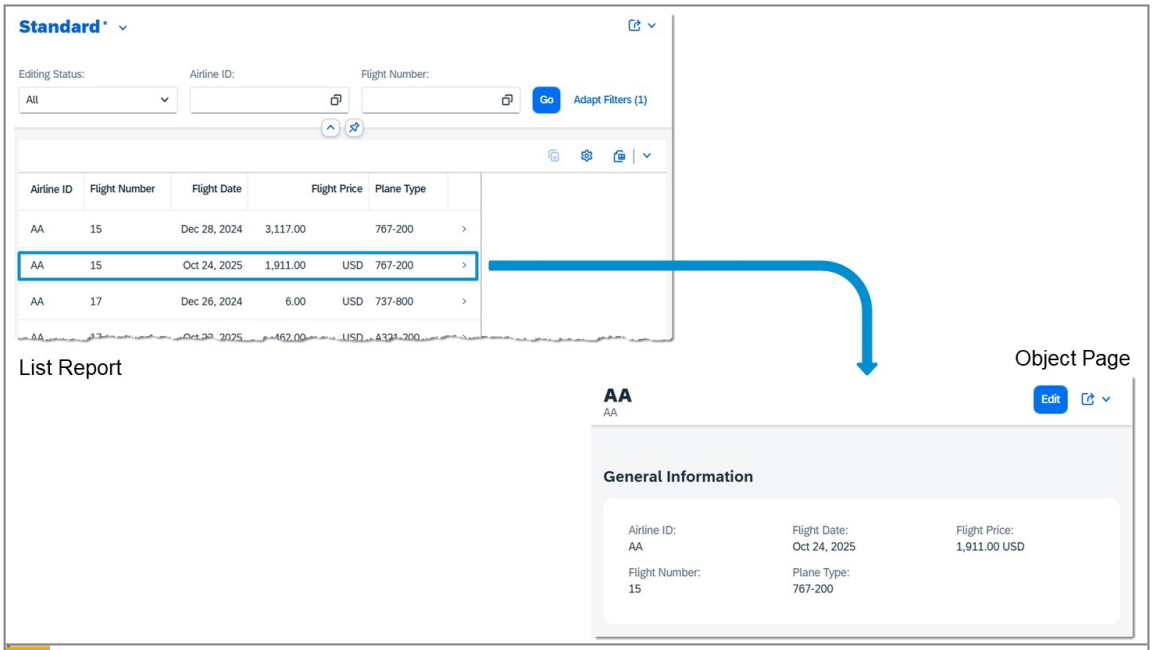


Abbildung 186: Musterantrag

In diesem Kapitel erstellen Sie eine Anwendung mit dem ABAP-RESTful-Anwendungsprogrammiermodell. Die Anwendung zeigt Flugdetails an und ermöglicht es Ihnen, den Preis für einen bestimmten Flug zu ändern.

© Copyright. Alle Rechte vorbehalten.

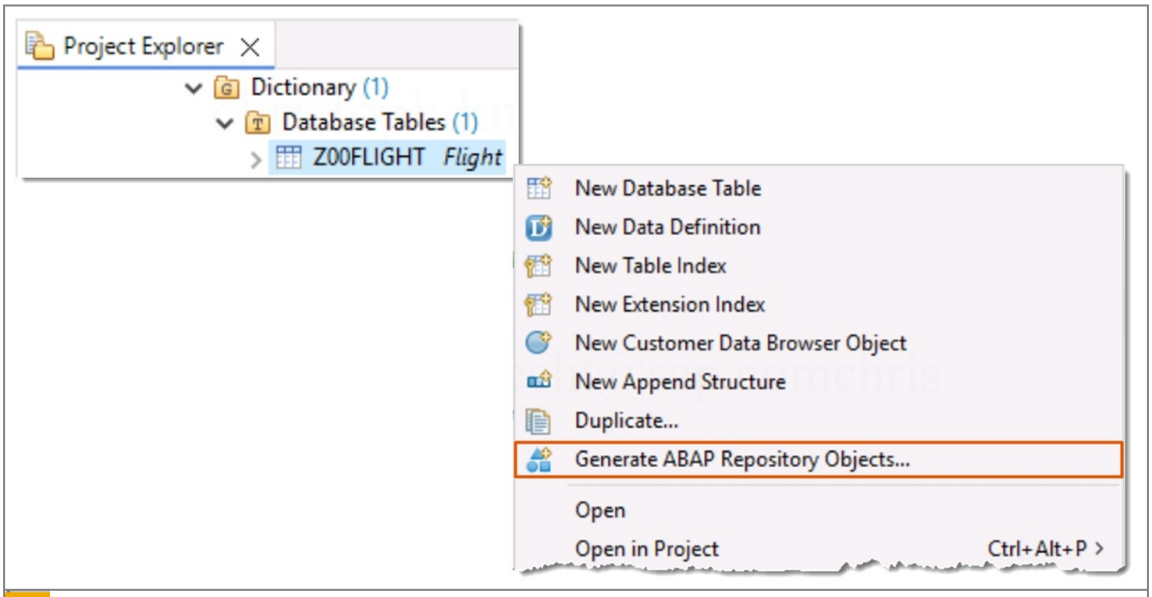


Abbildung 187: Generierung der benötigten Objekte

Sie beginnen mit dem Anlegen einer Datenbanktabellendefinition. Basierend auf der Definition Ihrer Tabelle verwenden Sie den Objektgenerator, um alle weiteren Objekte anzulegen, die Sie benötigen.

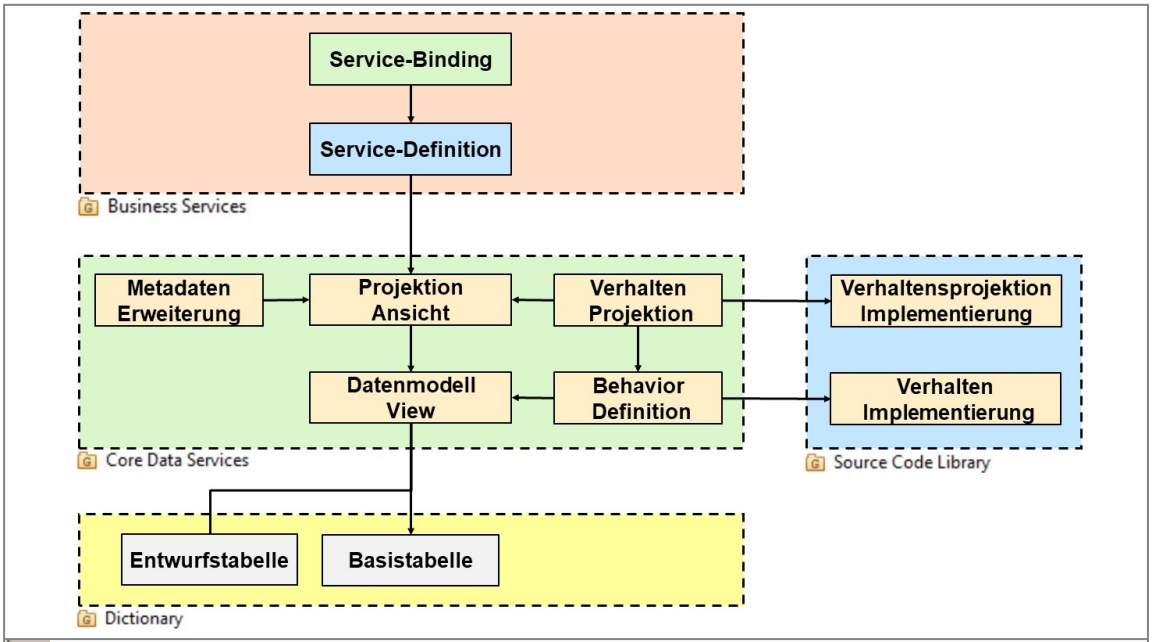


Abbildung 188: Repository-Objekte des generierten OData-UI-Service

Mit dem Generator legen Sie einen OData-UI-Service an, der die folgenden Objekte umfasst:

Ein Business-Objekt, das aus Folgendem besteht:

- Eine *CDS-View-Entität*
- Eine *CDS-Verhaltensdefinition*
- Eine *ABAP-Klasse* für die Verhaltensimplementierung (*Behavior Pool*)

© Copyright. Alle Rechte vorbehalten.

- Zwei *Datenbanktabellen*, eine für aktive Daten und eine für Entwurfsdaten

Eine Business-Objekt-Projektion, bestehend aus:

- Eine *CDS-View-Entität* (*Projektions-View*)
- Eine *CDS-Verhaltensdefinition* (*Behavior Projection*)
- Eine *ABAP-Klasse* für die Verhaltensprojektionsimplementierung (*Behavior Pool*)
- Eine *CDS-Metadatenerweiterung* für die servicespezifischen UI-Metadaten

Ein Business-Service, der aus Folgendem besteht:

- Eine *Service-Definition*
- Eine *Service-Bindung*

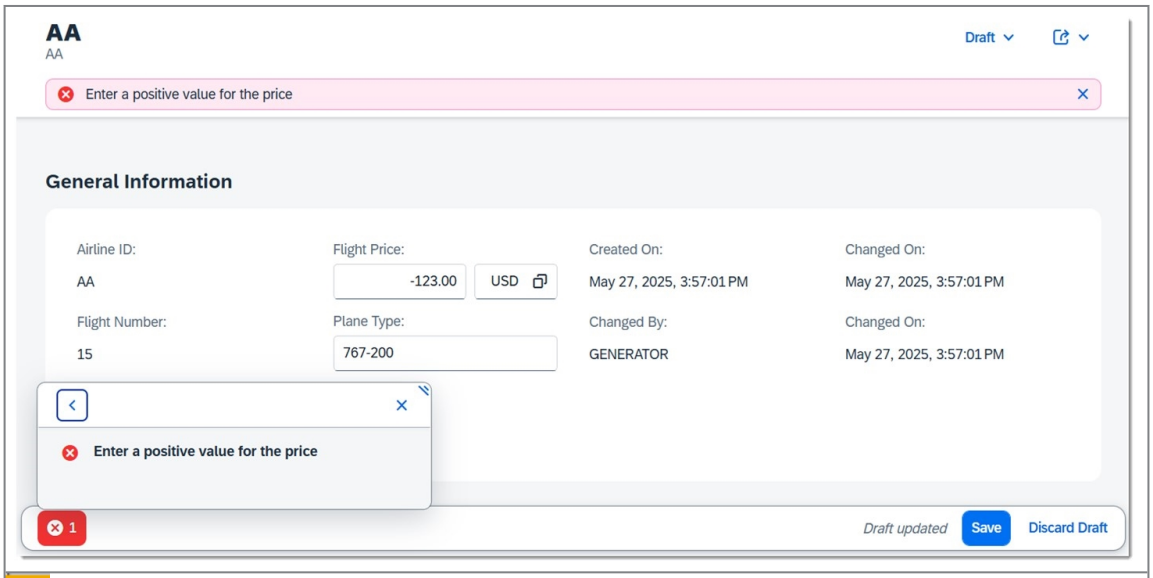


Abbildung 189: Validieren des Flugpreises

Anschließend fügen Sie Ihrer Anwendung eine Validierung hinzu, um sicherzustellen, dass der Preis für den Flug nicht null oder negativ ist.

© Copyright. Alle Rechte vorbehalten.

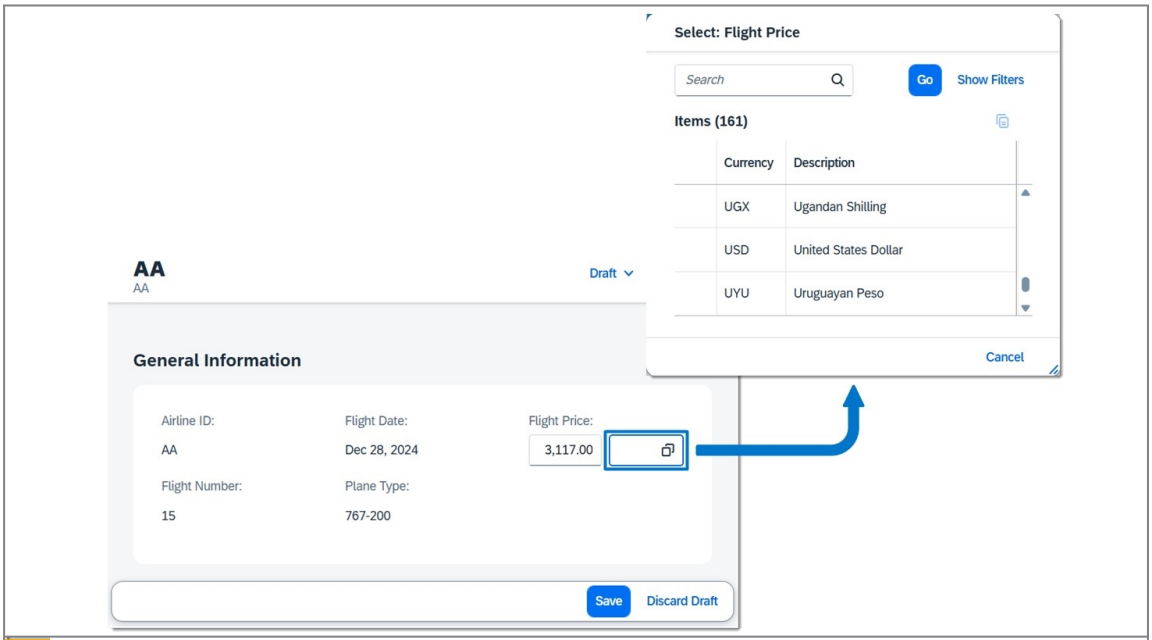


Abbildung 190: Benutzungsoberfläche anpassen

Schließlich erkunden Sie die Wertehilfe, die in der Anwendung bereitgestellt wird. Darüber hinaus passen Sie die Benutzungsoberfläche der generierten App an, indem Sie die UI-Metadaten bearbeiten und das Business-Objekt-Verhalten anpassen.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Beschreiben Sie den Prozess zur Entwicklung eines OData-Service mit dem ABAP-RESTful-Anwendungsprogrammiermodell.

© Copyright. Alle Rechte vorbehalten.

Die Architektur erkunden

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Legen Sie eine Datenbanktabelle an.
- Generieren Sie die Entwicklungsobjekte für einen OData-UI-Service.

Anlegen des Datenmodells

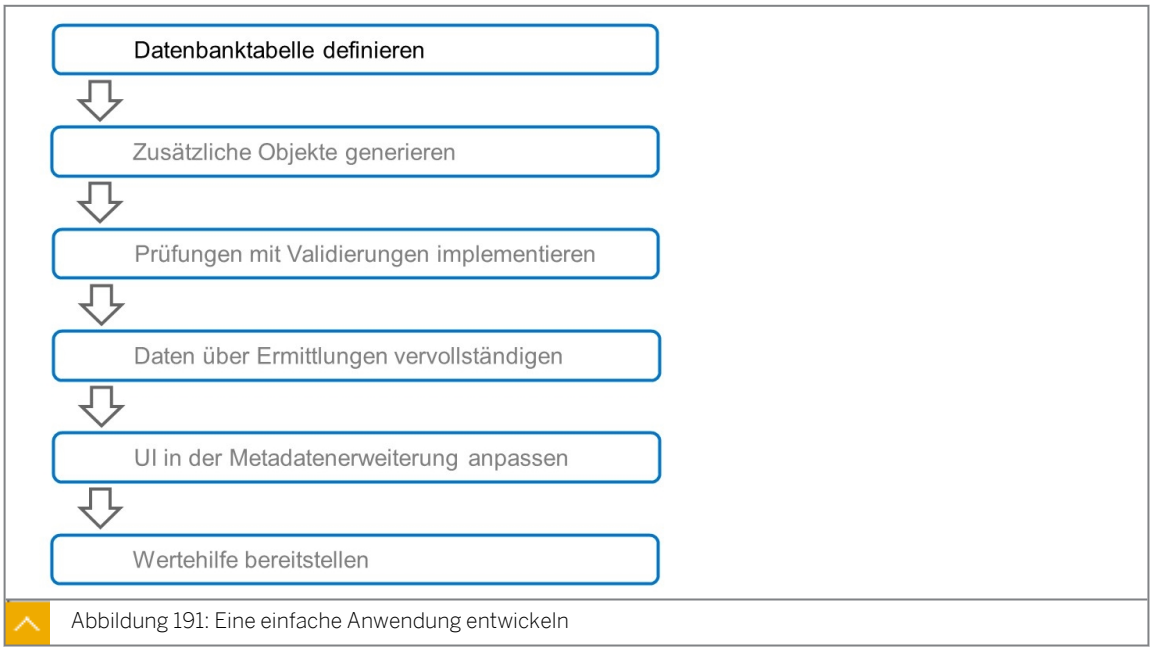


Abbildung 191: Eine einfache Anwendung entwickeln

Beispieldatenmodell

Mandant	UUID	carrier_id	connection_id	city_from	airport_from_id	Länder_von	...
100	<UUID_1>	LH	0400	Frankfurt	FRA	DE	
100	<UUID_2>	LH	0401	New York	JFK	US	
100	<UUID_3>	AA	0017	San Francisco	SFO	US	

Abbildung 192: Beispieldatenmodell

Das Datenmodell in diesem Beispiel ist sehr einfach - es gibt eine einzige Datenbanktabelle, die Details zu Flugverbindungen enthält - eine Fluggesellschaft, eine Flugnummer, Details zum Abflugflughafen und zum Ziel.

© Copyright. Alle Rechte vorbehalten.

> Notiz:
> Nicht in der Abbildung enthalten sind zusätzliche Felder, die die Laufzeit benötigt, um die Parallelitätskontrolle zu verwalten. Diese werden wir uns wenig später ansehen.

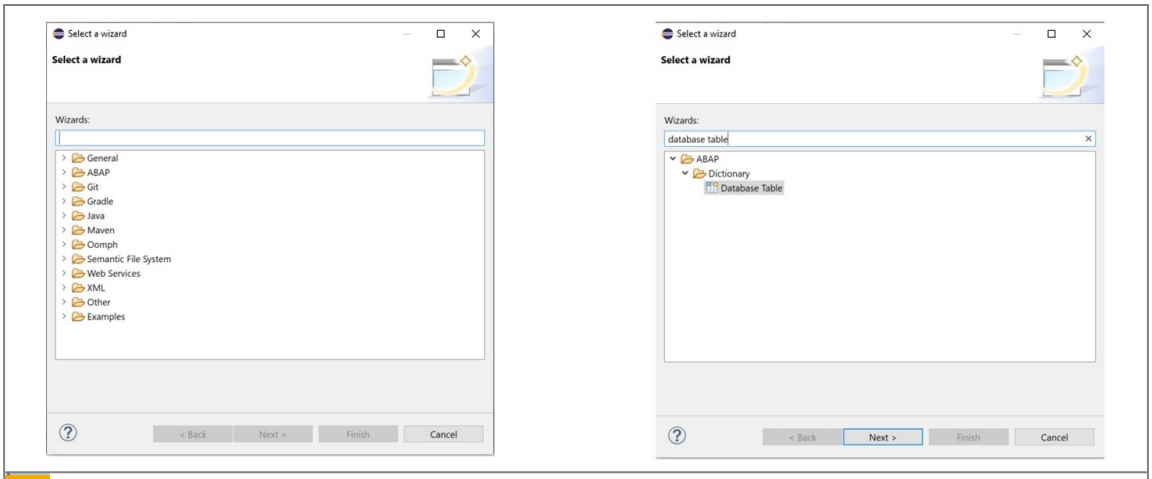


Abbildung 193: Eine Datenbanktabelle anlegen

Um eine Datenbanktabelle anzulegen, wählen Sie *File* → *New* → *Other*. Ein Dialogfenster wird angezeigt.

Geben Sie **`Datenbanktabelle`** in das Filterfeld ein. ADT schränkt die Liste der Objekte ein, die Sie auswählen können.

Doppelklicken Sie auf den Eintrag *Datenbanktabelle*, um die Tabelle anzulegen.

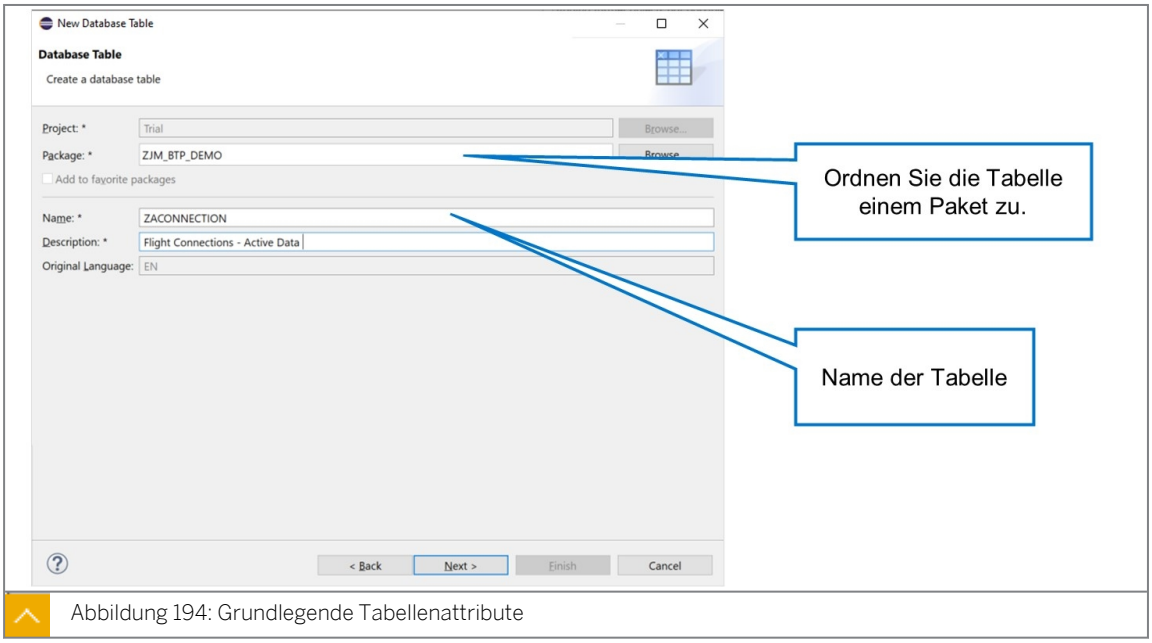


Abbildung 194: Grundlegende Tabellenattribute

Wenn Sie eine Tabelle anlegen, müssen Sie ihr einen Namen geben und sie einem Paket zuordnen.

© Copyright. Alle Rechte vorbehalten.

In der Abbildung „Grundlegende Tabellenattribute“ steht das Präfix Z für den Kundennamensraum. Dies ist die Tabelle, in der gesicherte Daten gespeichert werden. Später wird es eine ähnliche Tabelle *ZCONNECTION_D* geben, in der D für Entwurfsdaten steht. Dadurch können Benutzer unvollständige Daten speichern und zu einem späteren Zeitpunkt mit der Verarbeitung fortfahren.

Wählen Sie *Next*. ADT fordert Sie zur Eingabe eines Transportauftrags auf. Wählen Sie einen geeigneten Auftrag aus, und wählen Sie *Fertigstellen*.

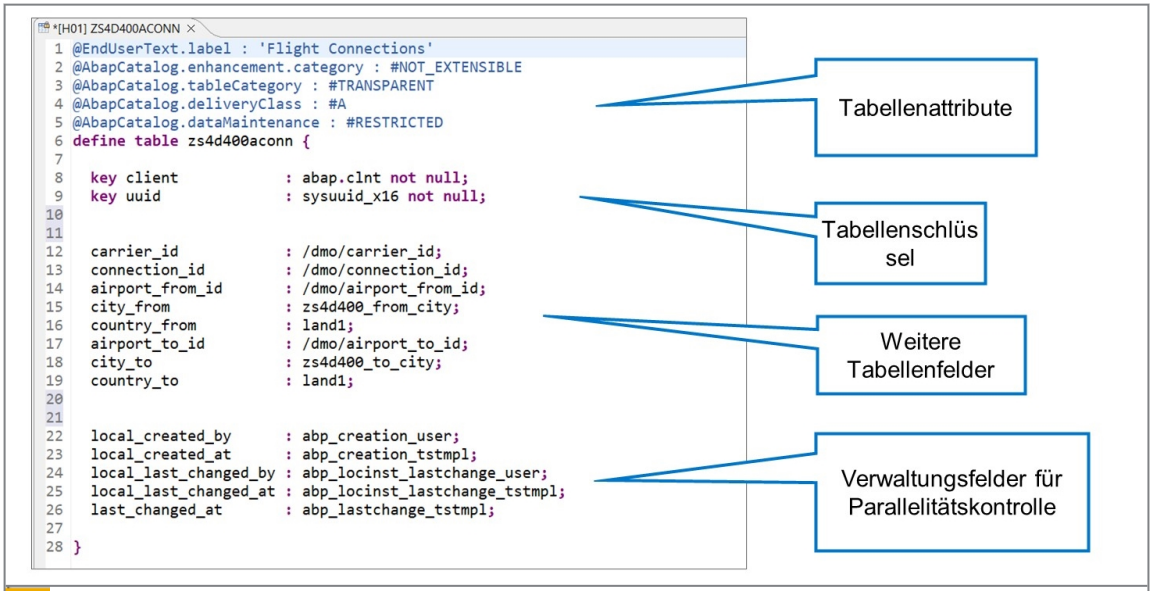


Abbildung 195: Tabellendefinition

In der Tabellendefinition geben Sie die Namen und Datentypen der einzelnen Spalten der Tabelle an. Dazu verwenden Sie die Notation `field_name : data_type ;`.

Der Datentyp kann ein eingebauter Typ des ABAP Dictionary sein - hierfür ist die Notation *abap.data_type*. Wenn der Datentyp unvollständig ist, geben Sie die Länge in Klammern hinter dem Datentyp an, z.B. *abap.char(10)*. Für numerische Typen mit einer festen Anzahl von Dezimalstellen würden Sie z.B. *abap.dec(15,2)* schreiben. Hiermit wird eine Spalte vom Typ DEC mit 15 Stellen inklusive zwei Nachkommastellen definiert.

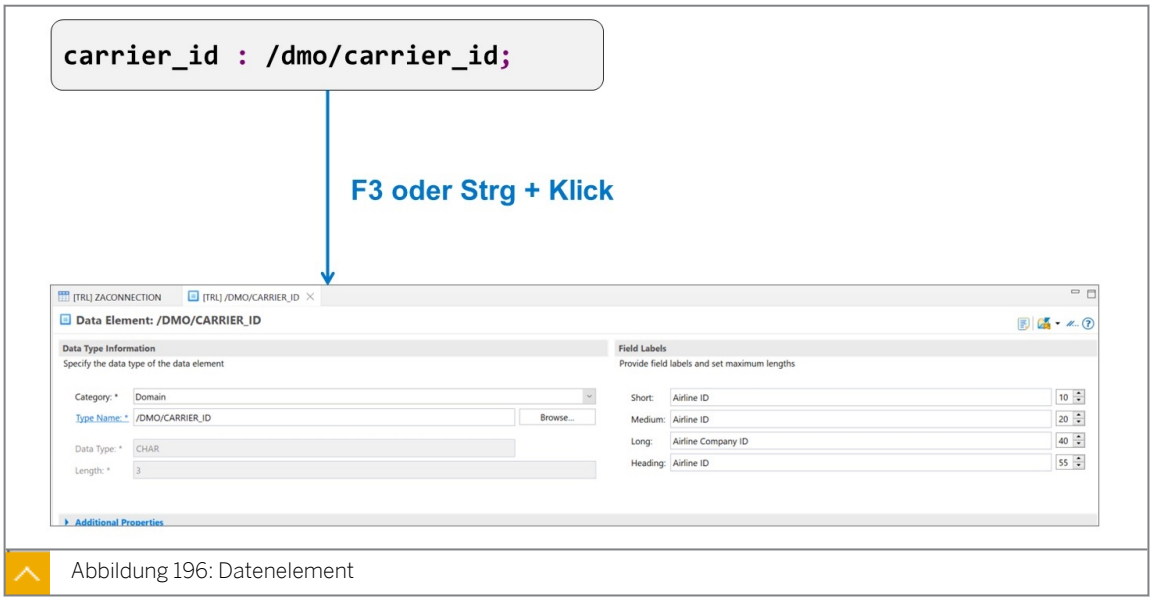


Abbildung 196: Datenelement

© Copyright. Alle Rechte vorbehalten.

Wenn Sie keinen eingebauten Typ des ABAP Dictionary verwenden, um den Typ einer Tabellenspalte anzugeben, verwenden Sie stattdessen ein Datenelement. Datenelemente sind ABAP-Dictionary-Objekte, die den Typ eines einzelnen Felds wie UUID oder Airline Code definieren. Datenelemente enthalten nicht nur eine technische Typbeschreibung, sondern auch semantische Informationen wie Feldbezeichner. Wenn Sie Datenelemente verwenden, um die Typen der Tabellenspalten zu definieren, kann das System die Feldbezeichner automatisch auf der generierten Benutzungsoberfläche verwenden.

Das Datenelement in der Abbildung „Datenelement“ beschreibt das Feld „Fluggesellschaft“. Es handelt sich um ein Zeichenfeld der Länge 3. Das System verwendet die Feldbezeichner automatisch auf der generierten Benutzungsoberfläche.

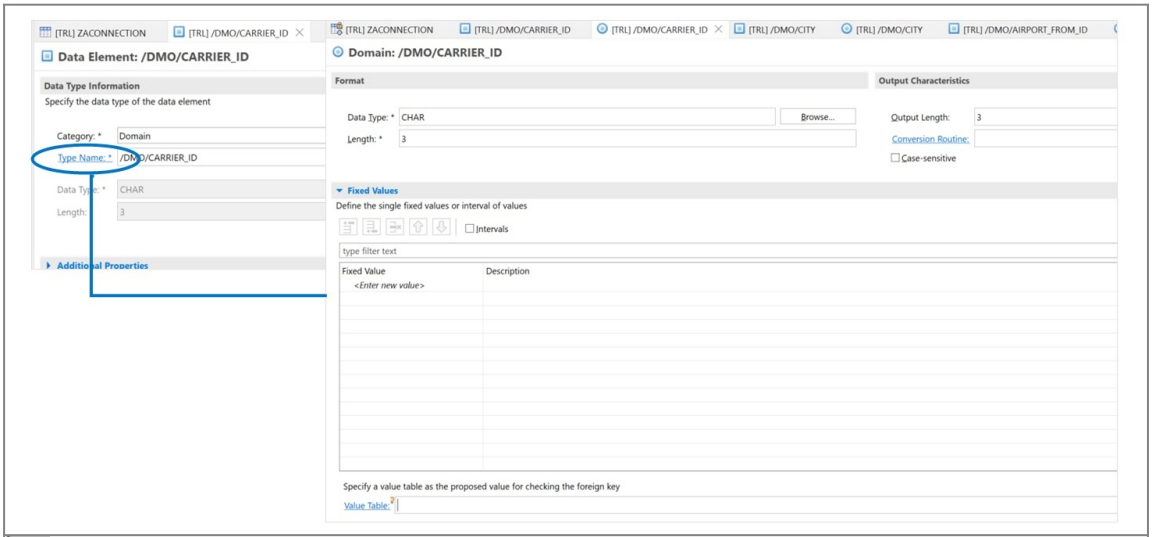


Abbildung 197: Domäne

Der technische Typ eines Datenelements wird in der Regel durch eine Domäne festgelegt. Hier sehen Sie, dass die Domäne für einen Airline Code ein Zeichenfeld mit einer Länge von 3 Zeichen definiert.

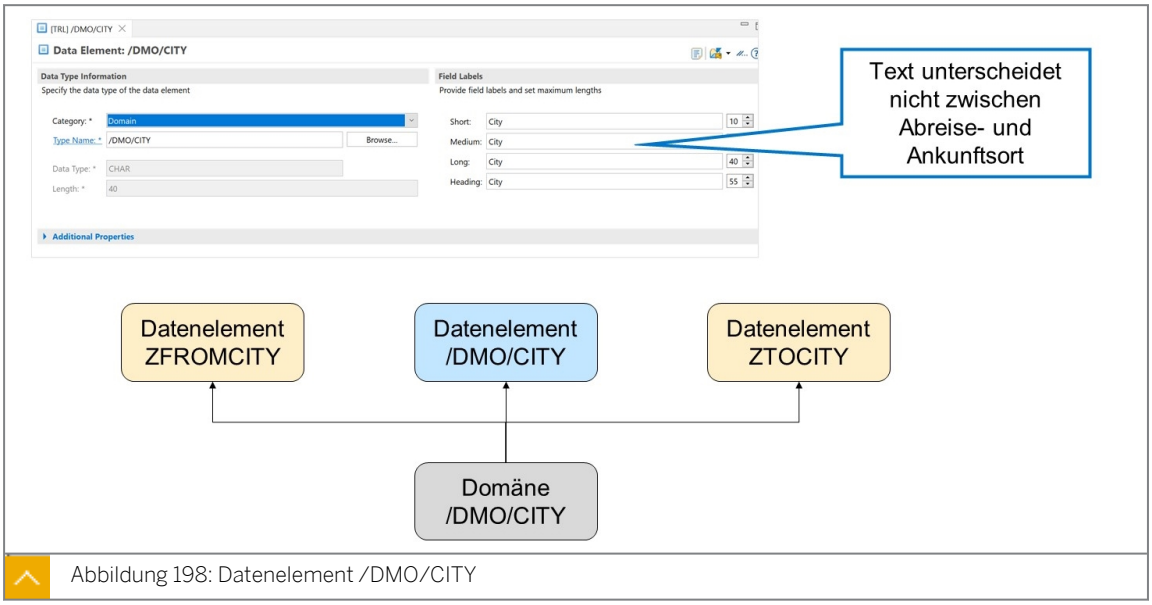


Abbildung 198: Datenelement /DMO/CITY

Das Datenelement */DMO/CITY* beschreibt den Datentyp für die Spalten *city_from* und *city_to* der Datenbanktabelle. Sein Feldbezeichner - der Text, der auf der Benutzungsoberfläche

© Copyright. Alle Rechte vorbehalten.

angezeigt wird - ist jedoch nur das Wort "Stadt". Dadurch kann nicht unterschieden werden, welche Stadt die Abflugstadt des Flugs und welche die Zielstadt ist.

Um die Benutzerfreundlichkeit zu verbessern, ist es besser, in diesem Fall nicht das Datenelement */DMO/CITY* zu verwenden. Stattdessen legen Sie ein neues Datenelement für den Abfahrtsort und ein weiteres für das Ziel an und definieren jeweils die entsprechenden Feldbezeichner. Neben den semantischen Informationen müssen Sie einen Datentyp für die neuen Datenelemente angeben. Dazu ordnen Sie eine Domäne zu. Hier können Sie die Domäne */DMO/CITY* wiederverwenden, da Abflugort und Zielort dieselben technischen Attribute haben sollten.

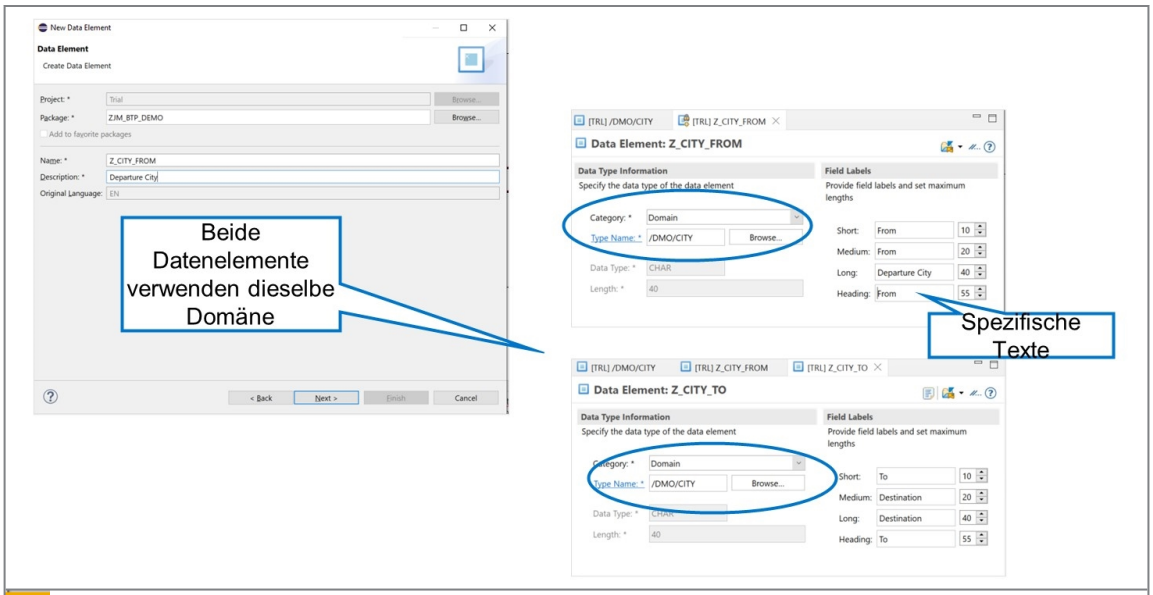


Abbildung 199: Datenelement anlegen

Um ein neues Datenelement anzulegen, wählen Sie *File* → *New* → *Other*, und geben Sie dann data element in das Feld *Filter Text* ein. Geben Sie einen Paketnamen und den Namen des Datenelements ein, und wählen Sie *Weiter*. Wählen Sie einen Transportauftrag aus, und wählen Sie *Fertigstellen*.

Anschließend müssen Sie einen Datentyp für das Datenelement angeben. Dies sollte eine Domäne sein. Im Beispiel wird die vorhandene Domäne */DMO/CITY* wiederverwendet. Beachten Sie, dass beide Datenelemente dieselbe Domäne verwenden. Das bedeutet, wenn jemand die Länge der Domäne nachträglich ändert, gilt die Änderung für alle Datenelemente, die sie verwenden.

Abschließend geben Sie kurze, mittlere und lange Feldbezeichner sowie einen Bezeichner ein, der für Spaltenüberschriften verwendet wird. Wenn Sie später die Fiori-Elements-App generieren, verwendet das System diese Texte automatisch.

```
key  client  :  abap.clnt not null;
key  uuid  :  sysuuid_x16  not null;
```

Abbildung 200: Tabellenschlüssel

© Copyright. Alle Rechte vorbehalten.

Jede Tabelle, die Sie definieren, muss einen Primärschlüssel haben. Dies ist eine Abfolge von Feldern am Anfang der Tabellenbeschreibung, die jeden Eintrag in der Tabelle eindeutig identifiziert.

Das erste Feld der Tabelle (und damit auch das erste Feld in der Schlüsseldefinition) muss das Mandantenfeld sein, das den Datentyp *abap.clnt* hat. Darüber hinaus ist es für unser Beispiel praktisch, eine UUID für den eindeutigen Schlüssel zu verwenden. Wenn Sie das Datenelement sysuuid_x16 verwenden, um den Typ dieses Feldes anzugeben, können Sie die UUID von der Laufzeit automatisch zuordnen lassen, wenn Sie einen neuen Datensatz anlegen.

```
local_created_by : abp_creation_user;
local_created_at  : abp_creation_tstmpl;
local_last_changed_by  : abp_locinst_lastchange_user;
local_last_changed_at  : abp_locinst_lastchange_tstmpl;
last_changed_at : abp_lastchange_tstmpl;
```

Abbildung 201: Felder für Administration und Concurrency Control

Aus Gründen der Rückverfolgbarkeit wird empfohlen, einige administrative Informationen mit den Daten zu speichern, z.B. dem Benutzer, der die Daten und Zeitstempel für das Anlegen und die letzte Änderung angelegt oder geändert hat. Die Datenbanktabelle, die Sie anlegen, sollte die in der Abbildung dargestellten Felder enthalten.

Notiz:
Diese Felder sind für den Generator, den wir später verwenden werden, obligatorisch. Damit dieser Generator ordnungsgemäß funktioniert, müssen die Felder auch mit den hier aufgeführten Datentypen definiert werden.

Insbesondere *fieldslocal_last_changed_at* und *last_changed_at* sind nicht nur administrative Felder. Im ABAP-RESTful-Anwendungsprogrammiermodell werden sie auch für die Parallelitätskontrolle verwendet.

Da RESTful-Anwendungen zustandslos sind, kann die Datenkonsistenz nicht allein durch exklusive Sperren sichergestellt werden. Das ABAP-RESTful-Anwendungsprogrammiermodell verwendet eine Mischung aus exklusiven Sperren und **ETags**, um Dateninkonsistenzen zu vermeiden. Ein ETag ist ein Feld, das seinen Wert ändert, wenn ein Datensatz aktualisiert wird. Durch den Vergleich des Werts des ETag-Felds kann das Framework sicherstellen, dass ein Datenset seit dem letzten Lesen nicht geändert wurde. Zeitstempel der letzten Änderung machen perfekte ETag-Felder.

© Copyright. Alle Rechte vorbehalten.

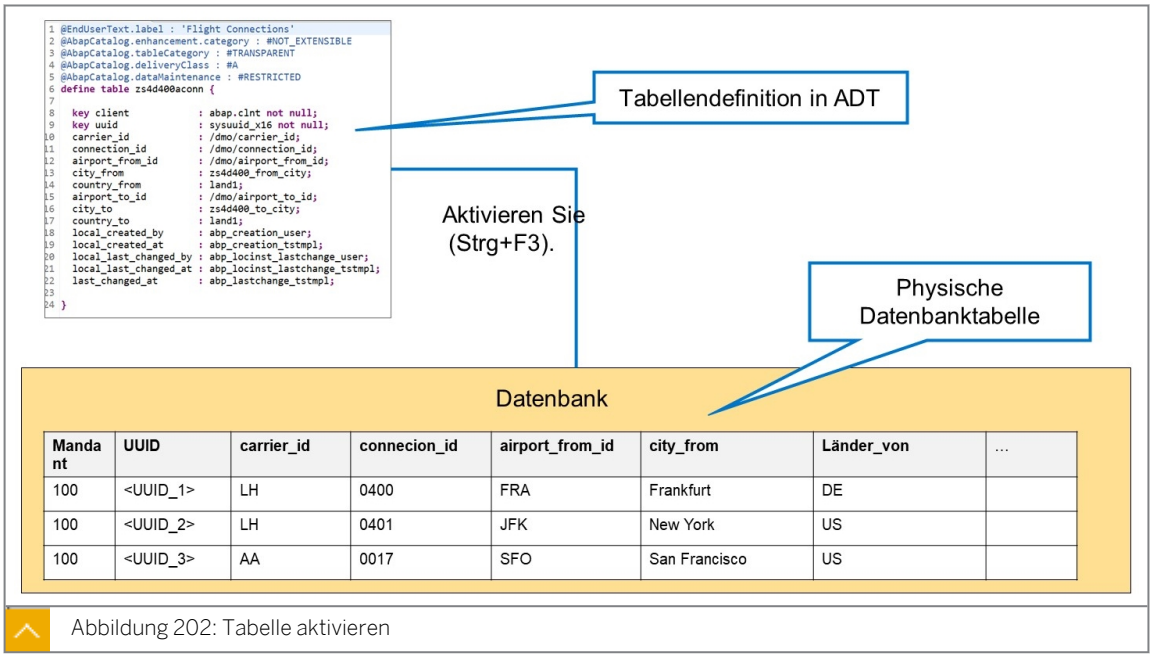


Abbildung 202: Tabelle aktivieren

Wenn Sie eine Datenbanktabelle in ADT aktivieren, legt das System die entsprechende physische Tabelle in der Datenbank an. Anschließend können Sie die zusätzlichen Objekte, die Sie für Ihre App benötigen, automatisch generieren.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Übung 18

Datenbanktabelle kopieren

In den Übungen dieses Kapitels legen Sie einen OData-UI-Service an. Dieser UI-Service bildet die Grundlage für eine Anwendung, mit der Sie den Preis von Flügen ändern können. Zunächst legen Sie ein dediziertes Paket für die erforderlichen Entwicklungsobjekte an. Anschließend kopieren Sie eine Datenbanktabelle und füllen diese mit Daten.

Vorlage:

/LRN/S4D400_APT (Datenbanktabelle)
/LRN/CL_S4D400_APT_COPY (globale Klasse)

Lösung:

/LRN/S4D400AFLGT (Datenbanktabelle)
/LRN/CL_S4D400_APS_COPY (globale Klasse)

Aufgabe 1: Unterpaket anlegen
Legen Sie ein neues Unterpaket unter Ihrem eigenen Paket ZS4D400_## an (wobei ## für Ihre Gruppennummer steht).

Das Unterpaket sollte die folgenden Attribute haben:

Attribut	Wert
Name	**`ZS4D400_##_RAP`**, wobei **`##`** für Ihre Gruppennummer steht
Beschreibung	Objekte für OData-UI-Service
Zu bevorzugten Paketen hinzufügen	Markiert
Superpackage	**`ZS4D400_##`**, wobei **`##`** für Ihre Gruppennummer steht

1. Legen Sie ein Unterpaket unter Ihrem eigenen Paket ZS4D400_## an (wobei ## für Ihre Gruppennummer steht).

Aufgabe 2: Datenbanktabelle kopieren
Legen Sie im neuen Paket eine Kopie der Datenbanktabelle **`/LRN/S4D400_APT`** an. Nennen Sie die neue Datenbanktabelle **`Z##FLIGHT`**, wobei ## für Ihre Gruppennummer steht.

1. Kopieren Sie die Datenbanktabelle **`/LRN/S4D400_APT`** in Ihr eigenes Paket **`ZS4D400_##_RAP`**, wobei ## für Ihre Gruppennummer steht.
2. Aktivieren Sie die Datenbanktabellendefinition.

© Copyright. Alle Rechte vorbehalten.

Aufgabe 3: Datenbanktabelle füllen
Legen Sie in Ihrem Paket *ZS4D400_##* eine Kopie der globalen Klasse **`/LRN/CL_S4D400_APT_COPY`** an (Namensvorschlag: **`ZCL_##_COPY`**). Ändern Sie in dieser Kopie den Wert der Konstante **`table_name`** in den Namen Ihrer eigenen Datenbanktabelle. Führen Sie dann die Klasse als Konsolen-App aus. Dadurch wird Ihre Datenbanktabelle mit Daten gefüllt.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_APT_COPY`** in eine Klasse in Ihrem eigenen Paket **`ZS4D400_##`**.
2. Passen Sie in der Kopie das Coding der Methode **`IF_OO_ADT_CLASSRUN~MAIN`** an. Ändern Sie den Wert der Konstante **`table_name`** in den Namen Ihrer eigenen Datenbanktabelle.
3. Aktivieren Sie die Klasse, und führen Sie sie als Konsolen-App aus.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Lösung 18

Datenbanktabelle kopieren

In den Übungen dieses Kapitels legen Sie einen OData-UI-Service an. Dieser UI-Service bildet die Grundlage für eine Anwendung, mit der Sie den Preis von Flügen ändern können. Zunächst legen Sie ein dediziertes Paket für die erforderlichen Entwicklungsobjekte an. Anschließend kopieren Sie eine Datenbanktabelle und füllen diese mit Daten.

Vorlage:

/LRN/S4D400_APT (Datenbanktabelle)
/LRN/CL_S4D400_APT_COPY (globale Klasse)

Lösung:

/LRN/S4D400AFLGT (Datenbanktabelle)
/LRN/CL_S4D400_APS_COPY (globale Klasse)

Aufgabe 1: Unterpaket anlegen
Legen Sie ein neues Unterpaket unter Ihrem eigenen Paket ZS4D400_## an (wobei ## für Ihre Gruppennummer steht).

Das Unterpaket sollte die folgenden Attribute haben:

Attribut	Wert
Name	**`ZS4D400_##_RAP`**, wobei **##** für Ihre Gruppennummer steht
Beschreibung	Objekte für OData-UI-Service
Zu bevorzugten Paketen hinzufügen	Markiert
Superpackage	**`ZS4D400_##`**, wobei **##** für Ihre Gruppennummer steht

1. Legen Sie ein Unterpaket unter Ihrem eigenen Paket ZS4D400_## an (wobei ## für Ihre Gruppennummer steht).
   a) Klicken Sie im *Project Explorer* mit der rechten Maustaste auf Ihr ABAP-Paket *ZS4D400_##*, und wählen Sie *New → ABAP Package*.
   b) Geben Sie den Paketnamen **`ZS4D400_##_RAP`** ein, wobei **##** für Ihre Gruppennummer steht.
   c) Geben Sie die Beschreibung **`Objekte für OData-UI-Service ein`**.
   d) Markieren Sie das Ankreuzfeld *Zu Favoritenpaketen hinzufügen*.

© Copyright. Alle Rechte vorbehalten.

e) Stellen Sie sicher, dass das Feld *Oberpaket* den Namen Ihres bereits vorhandenen Pakets enthält.

f) Wählen Sie *Next*.

g) Stellen Sie sicher, dass die *Softwarekomponente* auf *ZSTUDENTS* gesetzt ist.

h) Wählen Sie *Next*.

i) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

Aufgabe 2: Datenbanktabelle kopieren
Legen Sie im neuen Paket eine Kopie der Datenbanktabelle **`/LRN/S4D400_APT`** an. Nennen Sie die neue Datenbanktabelle **`Z##FLIGHT`**, wobei ## für Ihre Gruppennummer steht.

1. Kopieren Sie die Datenbanktabelle **`/LRN/S4D400_APT`** in Ihr eigenes Paket **`ZS4D400_##_RAP`**, wobei ## für Ihre Gruppennummer steht.
   a) Öffnen Sie die Definition der Datenbanktabelle **`/LRN/S4D400_APT`**.
   b) Verknüpfen Sie die View *Project Explorer* mit dem Editor.
   c) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Datenbanktabelle **`/LRN/S4D400_APT`**, um das Kontextmenü zu öffnen.
   d) Wählen Sie im Kontextmenü *Duplizieren...*.
   e) Geben Sie den Namen Ihres Pakets **`ZS4D400_##_RAP`** in das Feld *Paket* ein. Geben Sie im Feld *Name***`Z##FLIGHT`** ein, wobei ## für Ihre Gruppennummer steht.
   f) Wählen Sie *Next*.
   g) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.
2. Aktivieren Sie die Datenbanktabellendefinition.
   a) Drücken Sie **`Strg + F3`**, um das Entwicklungsobjekt zu aktivieren.

Aufgabe 3: Datenbanktabelle füllen
Legen Sie in Ihrem Paket *ZS4D400_##* eine Kopie der globalen Klasse **`/LRN/CL_S4D400_APT_COPY`** an (Namensvorschlag: **`ZCL_##_COPY`**). Ändern Sie in dieser Kopie den Wert der Konstante **`table_name`** in den Namen Ihrer eigenen Datenbanktabelle. Führen Sie dann die Klasse als Konsolen-App aus. Dadurch wird Ihre Datenbanktabelle mit Daten gefüllt.

1. Kopieren Sie die Klasse **`/LRN/CL_S4D400_APT_COPY`** in eine Klasse in Ihrem eigenen Paket **`ZS4D400_##`**.
   a) Öffnen Sie den Quelltext der globalen Klasse **`/LRN/CL_S4D400_APT_COPY`**.
   b) Verknüpfen Sie bei Bedarf die View *Project Explorer* mit dem Editor.
   c) Klicken Sie in der View *Project Explorer* mit der rechten Maustaste auf die Klasse **`/LRN/CL_S4D400_APT_COPY`**, um das Kontextmenü zu öffnen.
   d) Wählen Sie im Kontextmenü *Duplizieren...*.
   e) Geben Sie **`ZS4D400_##`** in das Feld *Paket* ein. Geben Sie im Feld *Name***`ZCL_##_COPY`** ein, wobei ## für Ihre Gruppennummer steht.
   f) Wählen Sie *Next*.

© Copyright. Alle Rechte vorbehalten.

g) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

2. Passen Sie in der Kopie das Coding der Methode **`IF_OO_ADT_CLASSRUN~MAIN`** an. Ändern Sie den Wert der Konstante **`table_name`** in den Namen Ihrer eigenen Datenbanktabelle.

   a) Stellen Sie sicher, dass der Wert der Konstante dem Namen Ihrer Datenbanktabelle entspricht.

3. Aktivieren Sie die Klasse, und führen Sie sie als Konsolen-App aus.

   a) Drücken Sie **`Strg + F3`**, um die Klasse zu aktivieren.

   b) Drücken Sie **`F9`**, um die Klasse auszuführen.

   c) Prüfen Sie die Sicht *Konsole*. Die letzte Ausgabe sollte lauten: *<...> wurde mit Daten gefüllt*. Wenn *<...> nicht vom richtigen Typ ist*, stellen Sie sicher, dass eine Datenbanktabelle mit diesem Namen vorhanden, aktiv und eine exakte Kopie der Vorlagetabelle ist.

© Copyright. Alle Rechte vorbehalten.

Der Objektgenerator

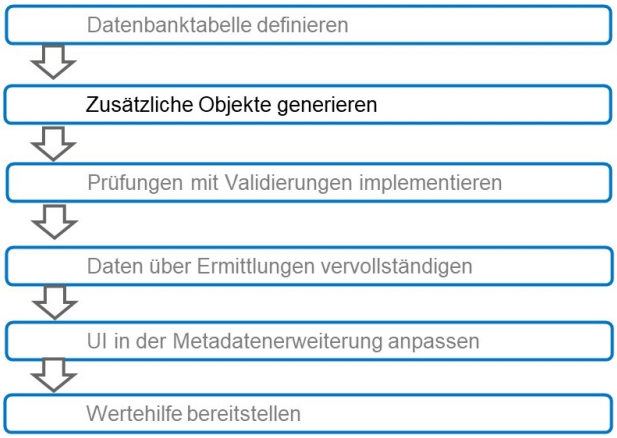


Abbildung 203: Eine einfache Anwendung entwickeln

Zusätzliche Objekte generieren

Wir haben gerade eine Datenbanktabelle für unsere Daten definiert, aber die App, die wir anlegen werden, benötigt mehr Objekte als nur diese Tabelle. Diese Objekte können mithilfe eines Assistenten in ADT generiert werden.

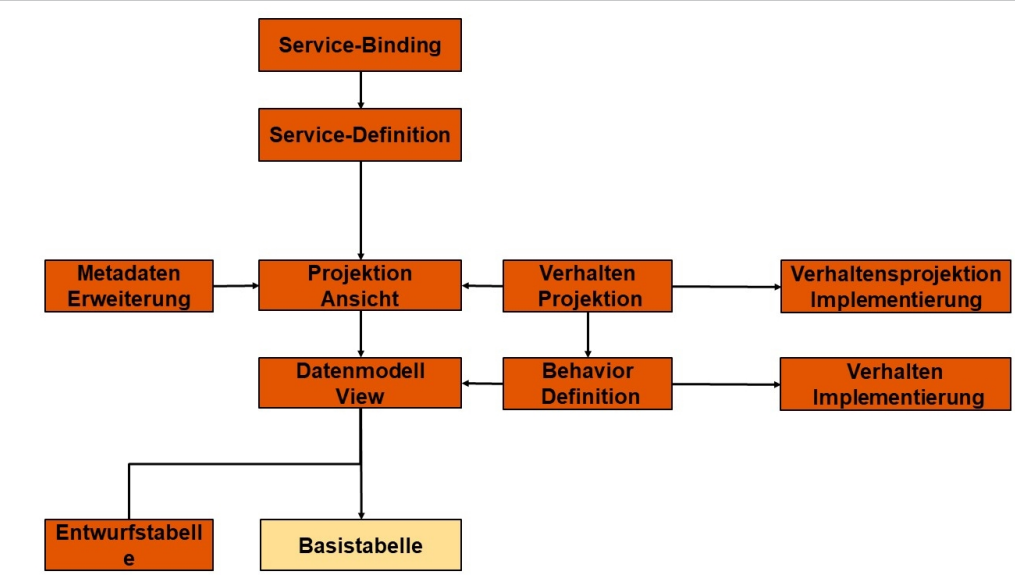


Abbildung 204: Zusätzliche Objekte generieren

Die generierten Objekte enthalten alle Informationen, die erforderlich sind, um eine funktionierende App mit der Anlege-, Lese-, Aktualisierungs- und Löschfunktion bereitzustellen. Später werden wir auch einige dieser Objekte anpassen und erweitern, um das Erscheinungsbild der Benutzungsoberfläche zu ändern und einige Prüfungen und Berechnungen zu implementieren.

© Copyright. Alle Rechte vorbehalten.

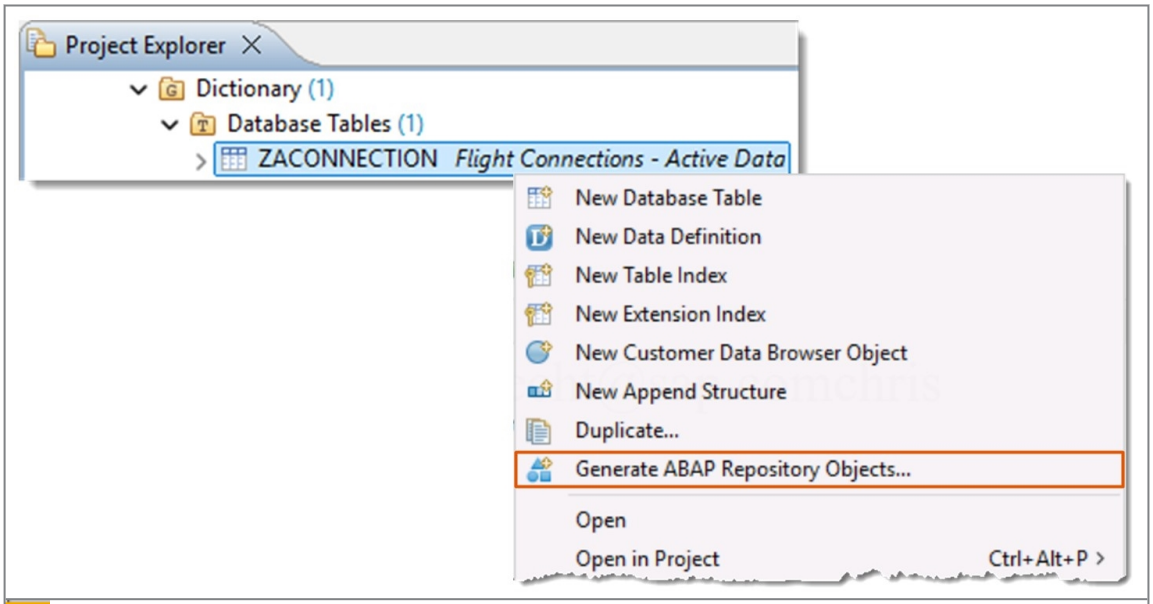


Abbildung 205: Starten des Wizards

Um den Assistenten zu starten, klicken Sie im *Project Explorer* mit der rechten Maustaste auf den Tabellennamen, und wählen Sie *Generate ABAP Repository Objects...*.

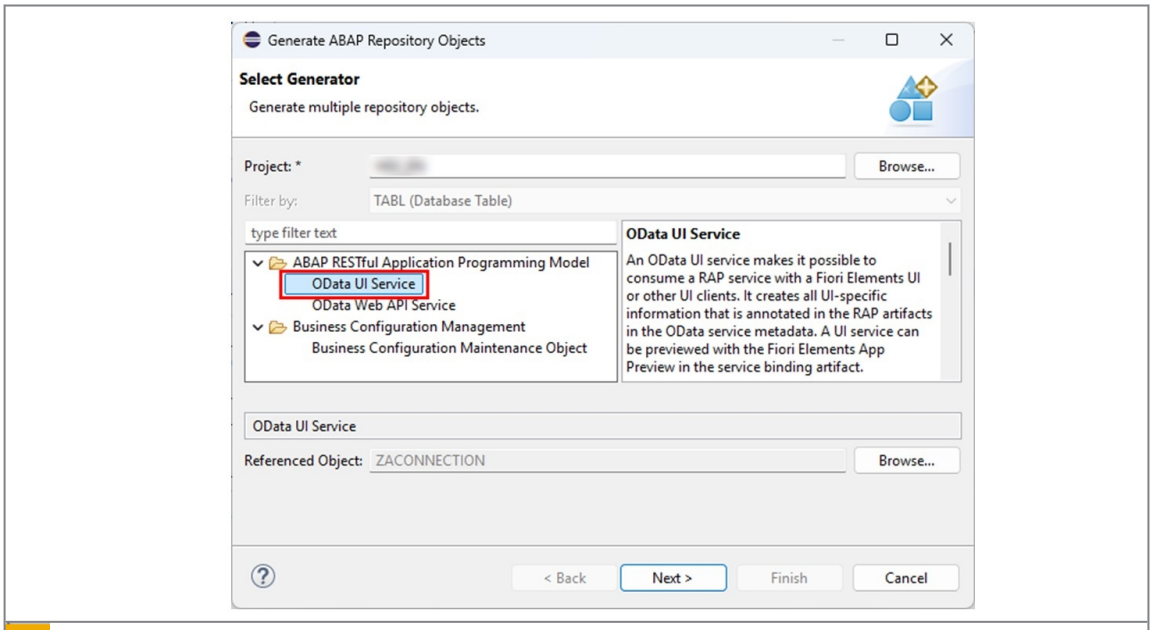


Abbildung 206: Auswahl des Generators

Zuerst müssen Sie den Generator auswählen. In unserem Beispiel möchten wir einen *OData-UI-Service* generieren. Im nächsten Dialogfenster müssen Sie ein Paket eingeben, dem alle neuen Objekte zugeordnet werden.

© Copyright. Alle Rechte vorbehalten.

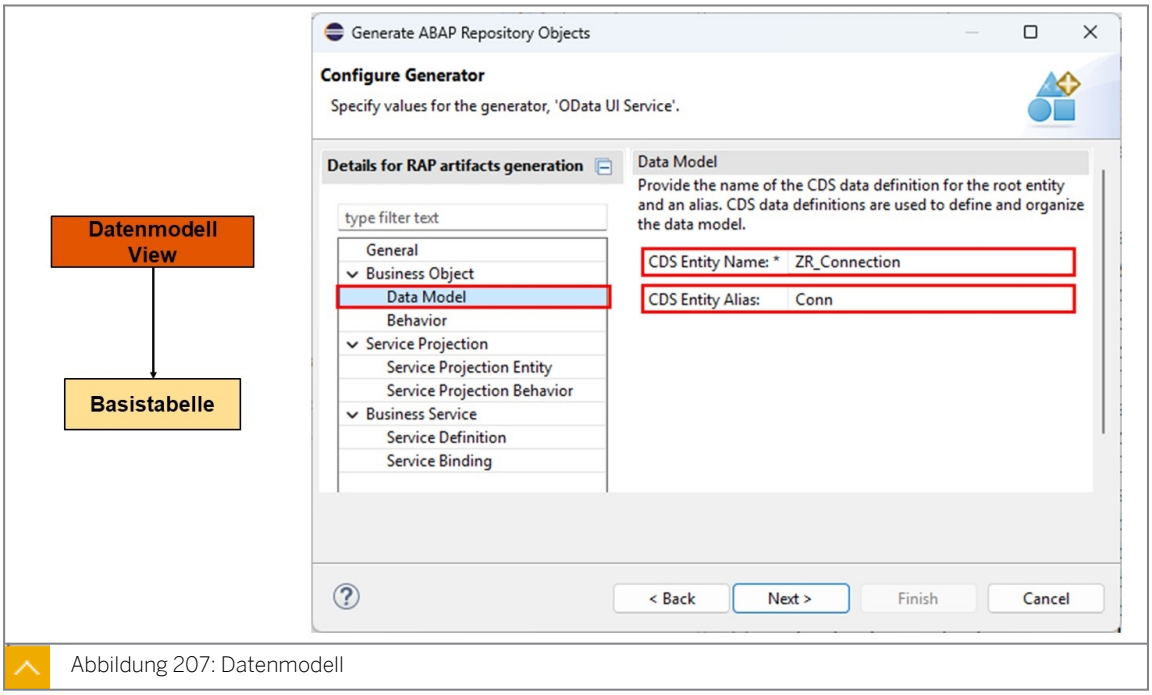


Abbildung 207: Datenmodell

Im RESTful-Anwendungsprogrammiermodell greifen Sie nicht direkt auf Datenbanktabellen zu. Stattdessen verwenden Sie CDS-Entitäten, um das Datenmodell zu definieren. Zu diesem Zeitpunkt schlägt der Generator einen Namen für die CDS-Entität vor. Da der Name der Datenbanktabelle mit Z beginnt, also im Kundennamensraum liegt, schlägt der Generator einen Namen im gleichen Namensraum vor. Eine weitere Konvention besteht darin, dass dem Namensraum der Buchstabe R und dann ein Unterstrich folgt.

Optional können Sie einen Aliasnamen eingeben, der in der generierten Anwendung verwendet wird, um die generierte CDS-Entität zu identifizieren.

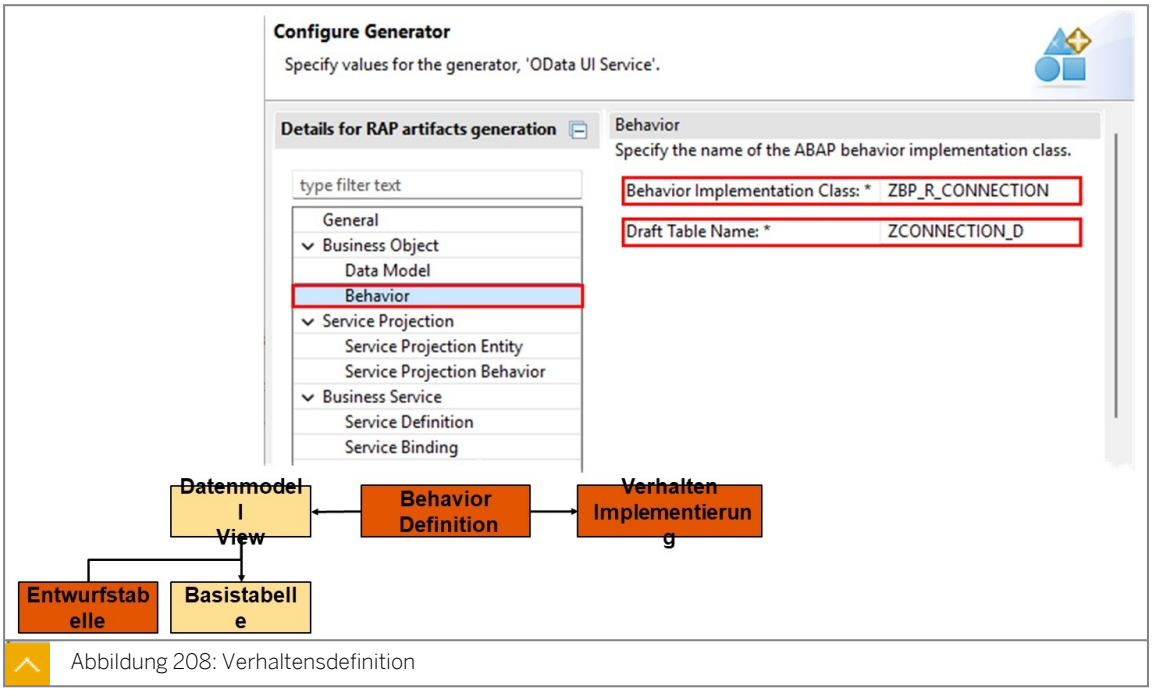


Abbildung 208: Verhaltensdefinition

Mit der Datenmodellsicht, die Sie anlegen, können Sie Daten aus der Datenbank lesen. Unsere App sollte jedoch auch in der Lage sein, Daten anzulegen, zu ändern und zu löschen. Um dies

© Copyright. Alle Rechte vorbehalten.

zu ermöglichen, müssen Sie eine Verhaltensdefinition definieren. Dies ist mit der Datenmodellsicht verknüpft und gibt an, welche der Anlege-, Aktualisierungs- und Löschaktionen zulässig sind. Beispielsweise dürfen einige Arten von Dokumenten nach ihrer Erstellung nicht mehr geändert werden. In diesem Fall würden Sie den Anlegevorgang zulassen, aber nicht aktualisieren oder löschen.

Neben der Angabe, welche der Anlege-, Aktualisierungs- und Löschaktionen verfügbar sein sollen, kann die Verhaltensdefinition auch die folgenden Definitionen enthalten:

- Entwurfsaktivierung
- Aktivierung der automatischen Nummerierung für das UUID-Feld
- Validierungen: Hierbei handelt es sich um Prüfungen, die ausgeführt werden, wenn der Benutzer Daten in die App eingibt.
- Ermittlungen: Eine Ermittlung führt eine Berechnung durch, um Felder im Datensatz zu füllen.

Der Generator legt die Verhaltensdefinition an und schaltet die automatische Nummerierung für das Feld UUID ein. Wenn Sie Validierungen und Ermittlungen verwenden müssen, müssen Sie diese manuell hinzufügen.

Die Verhaltensdefinition deklariert, welche Validierungen und Determinierungen vorhanden sind. Sie benötigen jedoch auch eine ABAP-Implementierung. Die Implementierungen sind Methoden, daher benötigen Sie eine Implementierungsklasse. Die Namenskonvention für diese Klasse besteht darin, das Präfix für den Namensraum zu verwenden, gefolgt von **`BP_R_`**. **`BP`** steht für Behavior-Pool.

Die Verhaltensdefinition definiert auch die Entwurfsaktivierung für die Entität. Basierend auf der Definition der Datenmodellsicht legt der Generator eine entsprechende Tabelle an, die die Entwurfsdaten enthält. Daher müssen Sie an dieser Stelle den Namen der Entwurfstabelle angeben. Der Generator schlägt einen Namen vor, der mit dem Namen der Basistabelle identisch ist, jedoch mit einem zusätzlichen **`_D`** am Ende.

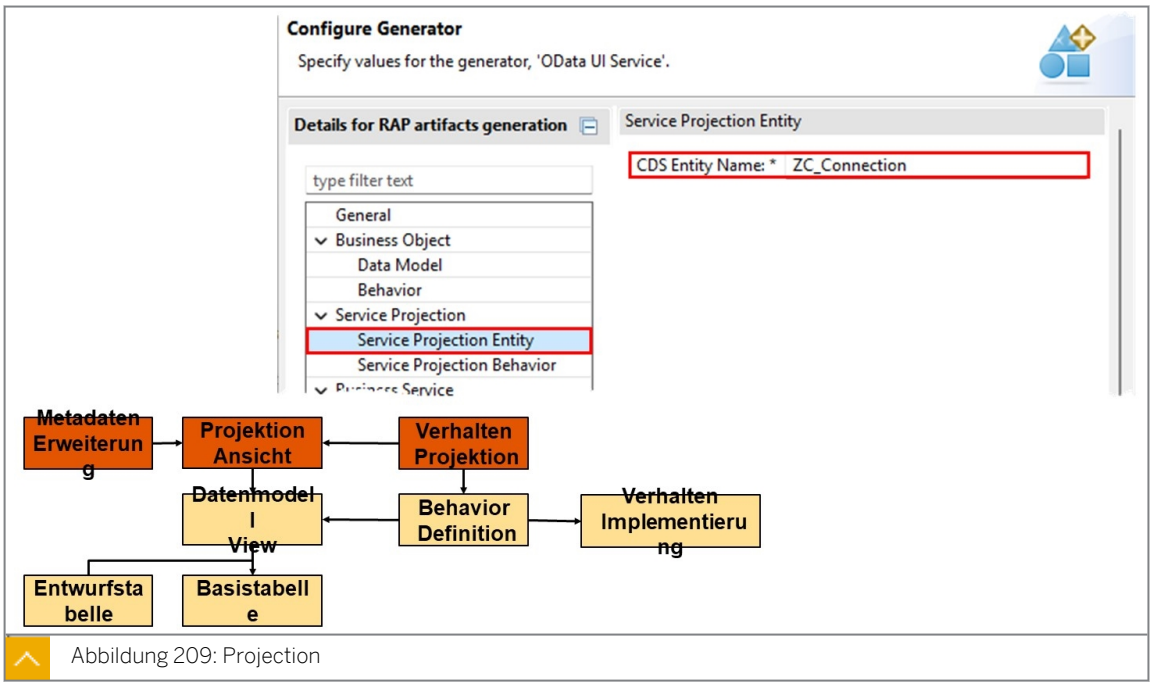


Abbildung 209: Projection

© Copyright. Alle Rechte vorbehalten.

Das Datenmodell und seine Verhaltensdefinition sind eine wiederverwendbare Implementierung einer bestimmten Geschäftsentität. Der nächste Schritt bei der Objektgenerierung besteht darin, den Namen der Serviceprojektion anzugeben. Die Projektion enthält eine View mit genau den Feldern, die für eine bestimmte App erforderlich sind, eine Verhaltensdefinition, die angibt, welches der definierten Verhaltensweisen tatsächlich in der App verfügbar sein soll, und eine Metadatenerweiterung. Die Metadatenerweiterung enthält CDS-Annotationen, die definieren, wie die Benutzungsoberfläche der App aussehen soll.

Die Namenskonvention für die Projektionsschicht lautet **`ZC_<Name>`**. Die Projektions-View, die Verhaltensprojektion und die Metadatenerweiterung haben alle denselben Namen.

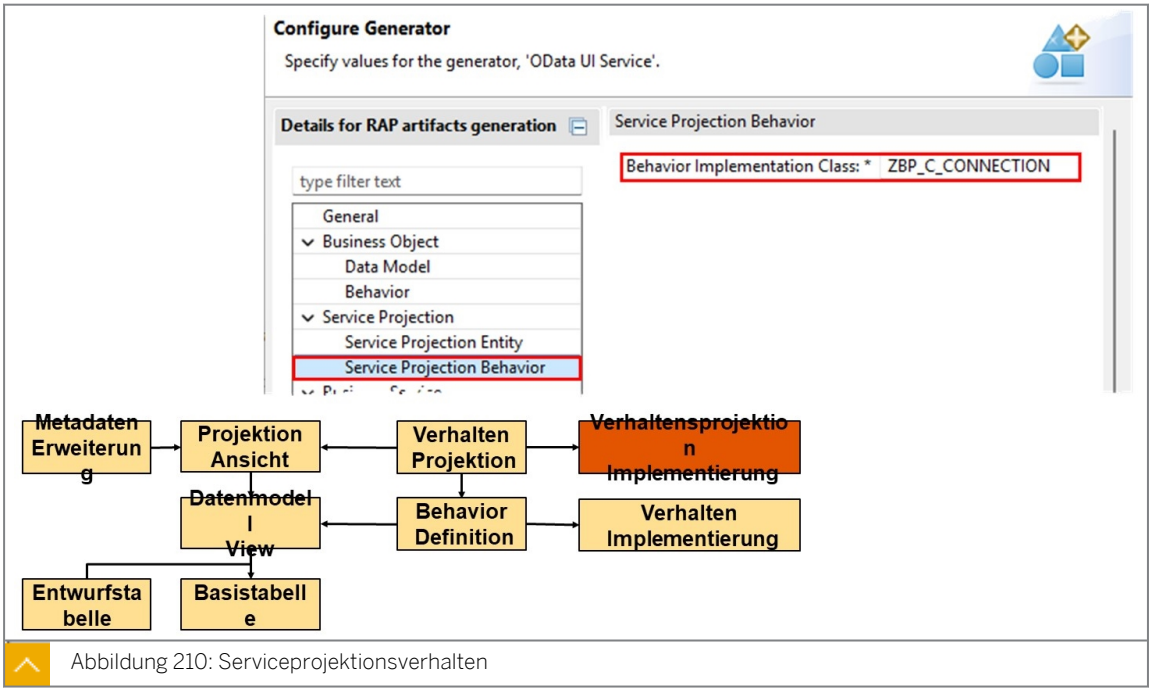


Abbildung 210: Serviceprojektionsverhalten

Manchmal kann es erforderlich werden, servicespezifische Implementierungen bereitzustellen. Die Namenskonvention für die Verhaltensimplementierungsklasse auf Projektionsebene besteht darin, das Präfix für den Namensraum zu verwenden, gefolgt von **`BP_C_`**.

© Copyright. Alle Rechte vorbehalten.

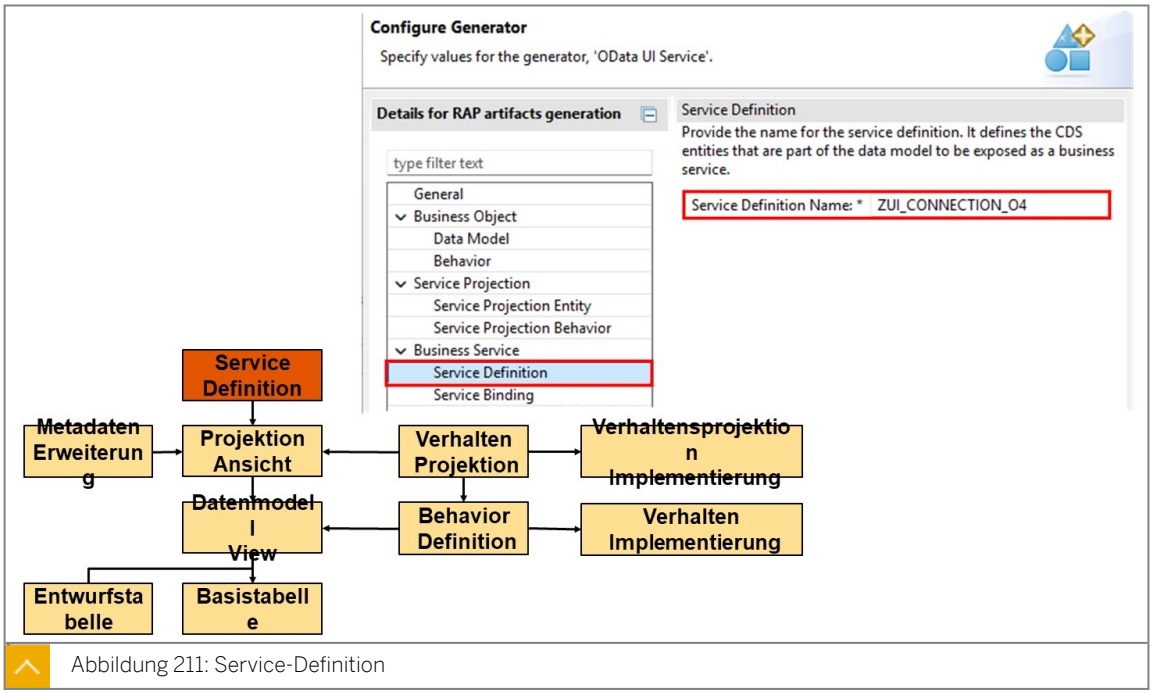


Abbildung 211: Service-Definition

Um die App bereitzustellen, müssen Sie eine Service-Definition und ein Service-Binding anlegen. Die Service-Definition gibt die Projektions-View an, die in diesem Service exponiert werden soll.

Der Generator schlägt einen Namen für die Service-Definition in der Form **`ZUI_<...vor.>_O4`**. Dies gibt an, dass der Service für eine Fiori-Elements-App vorgesehen ist und Version 4 des OData-Protokolls verwendet.

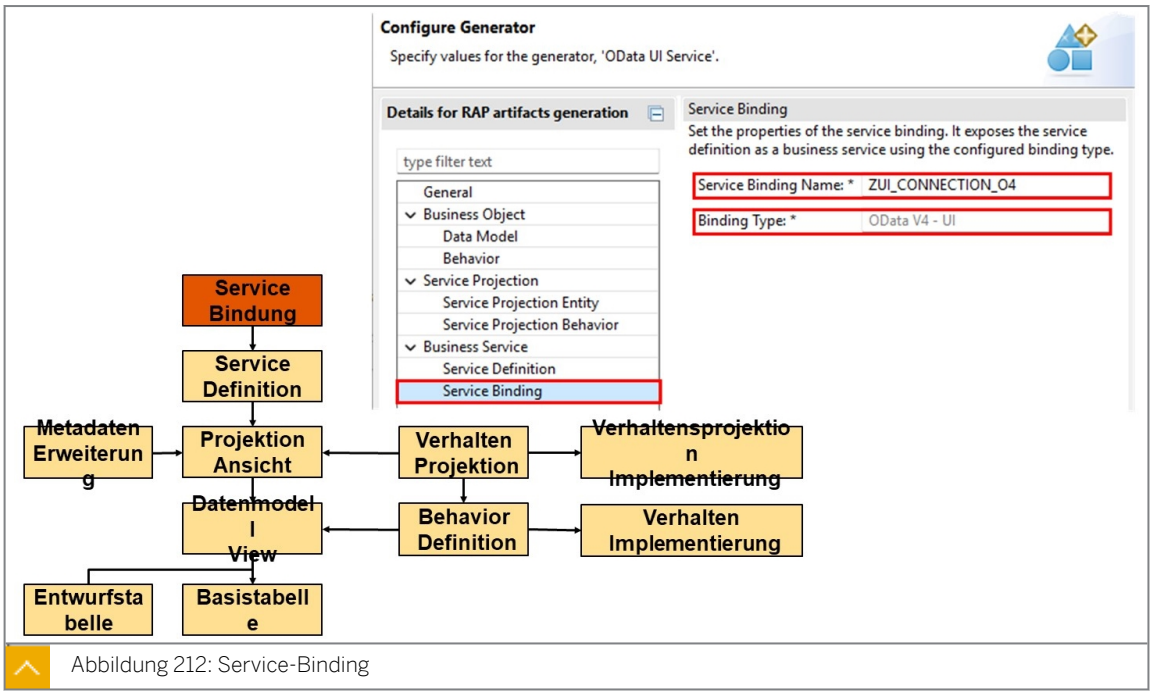


Abbildung 212: Service-Binding

Die Service-Bindung gibt tatsächlich das zu verwendende Protokoll an. Der vorgeschlagene Name für die Service-Bindung ist derselbe wie für die Service-Definition: **`ZUI_<...>_O4`**.

© Copyright. Alle Rechte vorbehalten.

Am Ende des Wizards zeigt das System eine Liste aller Objekte an, die es generieren wird. Sie können Ihre Eingaben gegen die Namenskonventionen prüfen und nach Tippfehlern Ausschau halten und ggf. zurückgehen und fehlerhafte Einträge ändern.

Vorschau des OData-UI-Service

Service veröffentlichen und Vorschau anzeigen

Mit Ihrer Datenbanktabelle und dem Objektgenerator können Sie alle Objekte generieren, die Sie benötigen, um eine funktionierende Fiori-Elements-App zu erhalten. Der einzige manuelle Schritt besteht darin, den Service zu veröffentlichen.

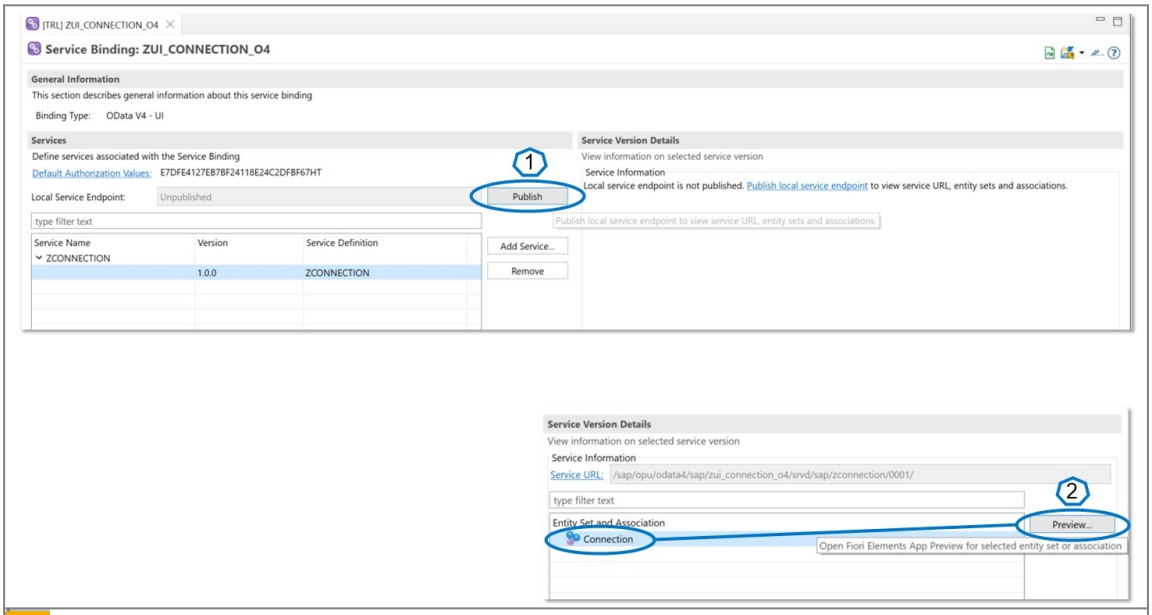


Abbildung 213: Service veröffentlichen und Vorschau anzeigen

Sie müssen den Service veröffentlichen, bevor Sie die App testen können. Öffnen Sie dazu das Service-Binding, und wählen Sie *Publizieren*. Anschließend wird die von Ihnen angelegte Entität (in diesem Fall Verbindung) in der Liste der Entitäten angezeigt. Markieren Sie sie, und wählen Sie *Vorschau*, um die App zu testen.

© Copyright. Alle Rechte vorbehalten.

App testen

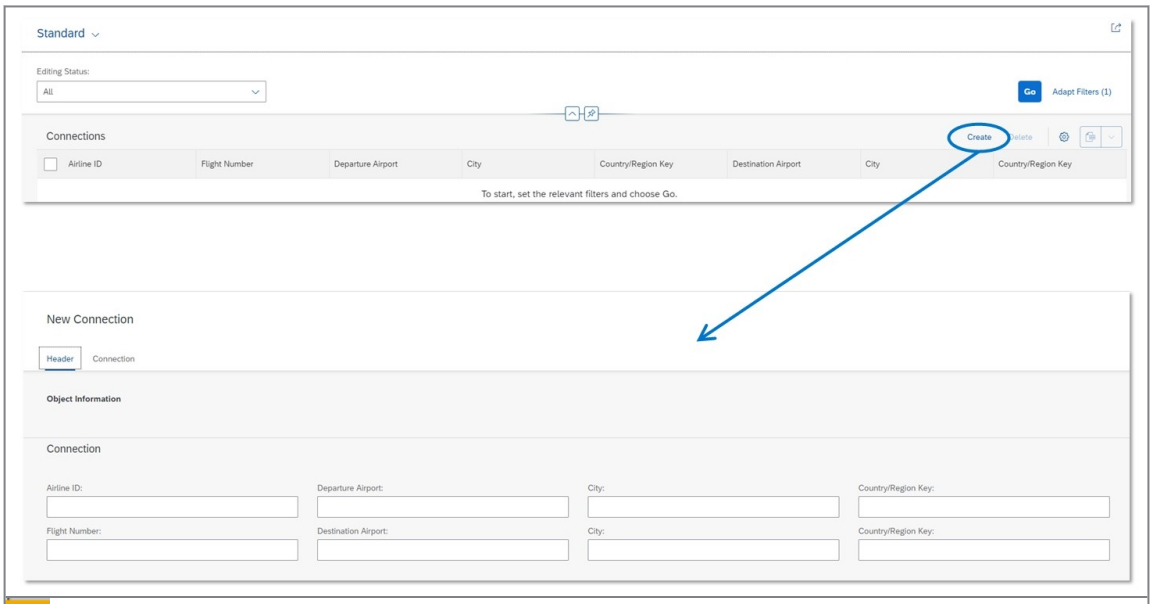


Abbildung 214: App testen

Wenn Sie *Vorschau* wählen, wird ein Browserfenster geöffnet, in dem Sie eine neue Flugverbindung anlegen können. Die App implementiert jedoch keine anderen Prüfungen als Typprüfungen (z.B. sind im Feld *Flugnummer* nur Ziffern zulässig).

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Übung 19

OData-UI-Service generieren und Vorschau anzeigen

In dieser Übung generieren Sie die Repository-Objekte für einen OData-UI-Service und zeigen eine Vorschau des Service als SAP-Fiori-App an.

Vorlage:

keine

Lösung:

Die generierten Entwicklungsobjekte im Paket **`/LRN/S4D400_EXERCISE_RAP`**.

Notiz:
Beachten Sie, dass sich die Implementierung des verwendeten Generators ändern kann. Daher können die in der Übung generierten Entwicklungsobjekte von denen in der Lösung abweichen.

Voraussetzungen

Sie haben die vorherige Übung abgeschlossen. Sie haben die Datenbanktabelle **`Z##FLIGHT`** angelegt (wobei ## für Ihre Gruppennummer steht), und die Datenbanktabelle enthält Daten.

Aufgabe 1: Generator verwenden

Verwenden Sie den *Generator-OData-UI-Service des → ABAP-RESTful-Anwendungsprogrammiermodells*, um die erforderlichen Entwicklungsobjekte in Ihrem Unterpaket **`ZS4D400_##_RAP`** zu generieren. Geben Sie dazu im Assistenten folgende Werte an:

Feld	Wert
Allgemeines → referenziertes Objekt	**`Z##FLIGHT`**
Allgemeiner → Projektname	*<leer>*
Allgemeines → Artefaktpräfix	*<leer>*
Suffix → für allgemeine Artefakte	*<leer>*
CDS-Entitätsname des → → Business-Objekt-Datenmodells	**`ZR_##Flug`**
CDS-Entitätsalias des → → Business-Objekt-Datenmodells	**`Flight`**
→ → Business-Objekt-Verhaltensverhaltensimplementierungsklasse	**`ZBP_R_##FLIGHT`**

© Copyright. Alle Rechte vorbehalten.

Feld	Wert
Name der Entwurfstabelle für → → Business-Objekt-Verhalten	**Z##FLIGHT_D**
Service Projektionsservice → Projektionsentität → CDS-Entitätsname	**ZC_##Flug**
Service Projektion → Service Projektionsverhalten → Verhaltensimplementierungsklasse	**ZBP_C_##FLIGHT**
Name der → → Business-Service-Service-Definition	**ZUI_##FLIGHT_O4**
→ → Business-Service-Binding-Service-Binding-Name	**ZUI_##FLIGHT_O4**
→ → Business-Service-Binding-Binding-Typ	**OData V4 - UI**

1. Starten Sie den Objektgenerator für Ihre Datenbanktabelle **Z##FLIGHT** (wobei ## für Ihre Gruppennummer steht).
2. Wählen Sie im Dialogfenster den Generator aus.
3. Stellen Sie sicher, dass die Entwicklungsobjekte in Ihrem Unterpaket **ZS4D400_##_RAP** angelegt werden.
4. Konfigurieren Sie den Generator mit den Daten aus der Tabelle.

> Achtung:
> Wählen Sie nicht *Weiter*, bevor Sie alle Daten eingegeben haben.

5. Prüfen Sie die Repository-Objekte, die generiert werden sollen, und generieren Sie die Objekte.

> Notiz:
> Haben Sie ein wenig Geduld. Die Generierung der Objekte dauert einige Zeit. Am Ende des Generierungsprozesses wird ein Dialogfenster angezeigt, in dem Sie gefragt werden, ob Sie die generierte Service-Bindung im Editor öffnen möchten. Bestätigen Sie diesen Dialog, indem Sie auf *Öffnen* klicken.

6. Analysieren Sie die Generatorausgabe im *Projekt-Explorer*.

Aufgabe 2: Vorschau der App anzeigen
Veröffentlichen Sie den OData-UI-Service, und zeigen Sie eine Vorschau des Ergebnisses als SAP-Fiori-App an.

1. Veröffentlichen Sie den lokalen Service-Endpunkt.

© Copyright. Alle Rechte vorbehalten.

Notiz:
Das Veröffentlichen des Service kann einige Zeit in Anspruch nehmen.

2. Zeigen Sie eine Vorschau des OData-UI-Service als SAP-Fiori-App an.
3. Zeigen Sie die Liste der Flüge an.

   Welche Drucktasten sind in der Symbolleiste der angezeigten Tabelle verfügbar?

4. Navigieren Sie für einen der Flüge zur Detailsicht.

   Welche Drucktasten sind im Objektseitenkopf verfügbar?

5. Wechseln Sie für diesen Flug in den Änderungsmodus.

   Welche Felder können Sie im Änderungsmodus bearbeiten?

   Warum ist es nicht möglich, die Felder *Fluggesellschafts-ID*, *Flugnummer* und *Flugdatum* zu bearbeiten?

6. Geben Sie im Feld *Flugpreis* einen Text ein, z.B. .

   Ist es möglich, die Änderungen zu sichern?

7. Geben Sie einen negativen Wert für den *Flugpreis ein*.

   Können Sie die Änderungen jetzt sichern?

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Lösung 19

OData-UI-Service generieren und Vorschau anzeigen

In dieser Übung generieren Sie die Repository-Objekte für einen OData-UI-Service und zeigen eine Vorschau des Service als SAP-Fiori-App an.

Vorlage:

keine

Lösung:

Die generierten Entwicklungsobjekte im Paket **`/LRN/S4D400_EXERCISE_RAP`**.

Notiz:
Beachten Sie, dass sich die Implementierung des verwendeten Generators ändern kann. Daher können die in der Übung generierten Entwicklungsobjekte von denen in der Lösung abweichen.

Voraussetzungen

Sie haben die vorherige Übung abgeschlossen. Sie haben die Datenbanktabelle **`Z##FLIGHT`** angelegt (wobei ## für Ihre Gruppennummer steht), und die Datenbanktabelle enthält Daten.

Aufgabe 1: Generator verwenden

Verwenden Sie den *Generator-OData-UI-Service des → ABAP-RESTful-Anwendungsprogrammiermodells*, um die erforderlichen Entwicklungsobjekte in Ihrem Unterpaket **`ZS4D400_##_RAP`** zu generieren. Geben Sie dazu im Assistenten folgende Werte an:

Feld	Wert
Allgemeines → referenziertes Objekt	**`Z##FLIGHT`**
Allgemeiner → Projektname	*<leer>*
Allgemeines → Artefaktpräfix	*<leer>*
Suffix → für allgemeine Artefakte	*<leer>*
CDS-Entitätsname des → → Business-Objekt-Datenmodells	**`ZR_##Flug`**
CDS-Entitätsalias des → → Business-Objekt-Datenmodells	**`Flight`**
→ → Business-Objekt-Verhaltensverhaltensimplementierungsklasse	**`ZBP_R_##FLIGHT`**

© Copyright. Alle Rechte vorbehalten.

Feld	Wert
Name der Entwurfstabelle für → → Business-Objekt-Verhalten	**Z##FLIGHT_D**
Service Projektionsservice → Projektionsentität → CDS-Entitätsname	**ZC_##Flug**
Service Projektion → Service Projektionsverhalten → Verhaltensimplementierungsklasse	**ZBP_C_##FLIGHT**
Name der → → Business-Service-Service-Definition	**ZUI_##FLIGHT_O4**
→ → Business-Service-Binding-Service-Binding-Name	**ZUI_##FLIGHT_O4**
→ → Business-Service-Binding-Binding-Typ	**OData V4 - UI**

1. Starten Sie den Objektgenerator für Ihre Datenbanktabelle **Z##FLIGHT** (wobei ## für Ihre Gruppennummer steht).
   a) Expandieren Sie im *Project Explorer* den Pfad *Favorite Packages → ZS4D400_## → ZS4D400_##_RAP → Dictionary → Database Tables* (wobei ## für Ihre Gruppennummer steht). Alternativ können Sie die Definition der Datenbanktabelle **Z##FLIGHT** öffnen und bei Bedarf die View *Project Explorer* mit dem Editor verknüpfen.

   b) Klicken Sie in der *View Project Explorer* mit der rechten Maustaste auf die Datenbanktabelle **Z##FLIGHT**, um das Kontextmenü zu öffnen.

   c) Wählen Sie im Kontextmenü *Generate ABAP Repository Objects...*.

2. Wählen Sie im Dialogfenster den Generator aus.
   a) Wählen Sie auf der linken Seite des Dialogfensters *ABAP RESTful Application Programming Model → OData UI Service*.

   b) Wählen Sie *Weiter*, um zum nächsten Schritt zu navigieren.

3. Stellen Sie sicher, dass die Entwicklungsobjekte in Ihrem Unterpaket **ZS4D400_##_RAP** angelegt werden.
   a) Wenn das Fenster oben *Paket eingeben* lautet, stellen Sie sicher, dass das Feld *Paket* den Wert **ZS4D400_##_RAP** enthält.

   b) Wählen Sie *Weiter*, um zum nächsten Schritt zu navigieren.

4. Konfigurieren Sie den Generator mit den Daten aus der Tabelle.

Achtung:
Wählen Sie nicht *Weiter*, bevor Sie alle Daten eingegeben haben.

© Copyright. Alle Rechte vorbehalten.

a) Wählen Sie die Positionen auf der linken Seite des Dialogfensters nacheinander aus, und füllen Sie die entsprechenden Felder im rechten Bereich aus. Die erforderlichen Werte werden in der obigen Tabelle aufgeführt.

b) Nachdem Sie alle Daten eingegeben haben, wählen Sie *Weiter*.

5. Prüfen Sie die Repository-Objekte, die generiert werden sollen, und generieren Sie die Objekte.

> Notiz:
> Haben Sie ein wenig Geduld. Die Generierung der Objekte dauert einige Zeit. Am Ende des Generierungsprozesses wird ein Dialogfenster angezeigt, in dem Sie gefragt werden, ob Sie die generierte Service-Bindung im Editor öffnen möchten. Bestätigen Sie diesen Dialog, indem Sie auf *Öffnen* klicken.

a) Prüfen Sie die Liste der angezeigten Repository-Objekte.

b) Wenn Sie sicher sind, dass Sie alle Namen korrekt eingegeben haben, wählen Sie *Weiter*.

c) Bestätigen Sie den Transportauftrag, und wählen Sie *Finish*.

d) Bestätigen Sie das Dialogfenster am Ende des Generierungsprozesses, indem Sie *Öffnen* wählen.

6. Analysieren Sie die Generatorausgabe im *Projekt-Explorer*.
   a) Positionieren Sie im *Project Explorer* den Cursor auf Ihrem Unterpaket **`ZS4D400_##_RAP`**, und drücken Sie **`F5`**, um den Inhalt des Pakets neu zu laden.

   b) Expandieren Sie alle Unterknoten des Pakets, um die Namen der generierten Entwicklungsobjekte anzuzeigen.

Aufgabe 2: Vorschau der App anzeigen

Veröffentlichen Sie den OData-UI-Service, und zeigen Sie eine Vorschau des Ergebnisses als SAP-Fiori-App an.

1. Veröffentlichen Sie den lokalen Service-Endpunkt.

> Notiz:
> Das Veröffentlichen des Service kann einige Zeit in Anspruch nehmen.

a) Falls noch nicht geschehen, öffnen Sie die generierte Service-Bindung *ZUI_##FLIGHT_O4*.

b) Wenn die Service-Bindung inaktiv ist, drücken Sie **`Strg + F3`**, um sie zu aktivieren.

c) Um den Service zu publizieren, wählen Sie im Center des Editors *Publizieren*.

2. Zeigen Sie eine Vorschau des OData-UI-Service als SAP-Fiori-App an.
   a) Wählen Sie im Abschnitt *Service Version Details* auf der rechten Seite *Flight* und dann *Preview...*.

© Copyright. Alle Rechte vorbehalten.

Ergebnis

Ein Browserfenster mit einer SAP-Fiori-Anwendung wird geöffnet. Wenn Sie dazu aufgefordert werden, melden Sie sich mit Ihrem ABAP-Systembenutzer an der Anwendung an.

3. Zeigen Sie die Liste der Flüge an.
   a) Wählen Sie *Start*, um die Liste der Flüge anzuzeigen, die in Ihrer Datenbanktabelle **`Z##FLIGHT`** gespeichert sind.

   Welche Drucktasten sind in der Symbolleiste der angezeigten Tabelle verfügbar?

   Anlegen, *Löschen*, *In Zwischenablage kopieren*, *Mehr/Weniger je Zeile anzeigen*, *Einstellungen*, *Tabelle exportieren*

4. Navigieren Sie für einen der Flüge zur Detailsicht.
   a) Wählen Sie einen der Listeneinträge, um zur Object Page mit den Details dieses Flugs zu navigieren.

   Welche Drucktasten sind im Objektseitenkopf verfügbar?

   Bearbeiten, *Löschen*, *Freigeben*

5. Wechseln Sie für diesen Flug in den Änderungsmodus.
   a) Wählen Sie *Bearbeiten*.

   Welche Felder können Sie im Änderungsmodus bearbeiten?

   Flugpreis, *Währungscode* und *Flugzeugtyp*

   Warum ist es nicht möglich, die Felder *Fluggesellschafts-ID*, *Flugnummer* und *Flugdatum* zu bearbeiten?

   Die Felder *Fluggesellschafts-ID*, *Flugnummerund Flugdatum* gehören zum Primärschlüssel der Datenbanktabelle. Eine Änderung des Primärschlüssels ist nie sinnvoll. Standardmäßig setzt der Generator alle Schlüsselfelder während der Aktualisierung auf schreibgeschützt.

6. Geben Sie im Feld *Flugpreis* einen Text ein, z.B. **.**

   Ist es möglich, die Änderungen zu sichern?

   Nr. Das Feld *Flugpreis* basiert auf einem numerischen Typ, und die generierte Anwendung enthält typbasierte Eingabeprüfungen.

7. Geben Sie einen negativen Wert für den *Flugpreis ein*.

© Copyright. Alle Rechte vorbehalten.

Können Sie die Änderungen jetzt sichern?

Ja, die Prüfung auf positive Preise ist die Anwendungslogik. Sie muss vom Entwickler im Verhalten des Business-Objekts implementiert werden.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Legen Sie eine Datenbanktabelle an.
- Generieren Sie die Entwicklungsobjekte für einen OData-UI-Service.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Lektion 3

ABAP-Logik hinzufügen

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Implementieren Sie das Verhalten eines Business-Objekts.

Validierungen

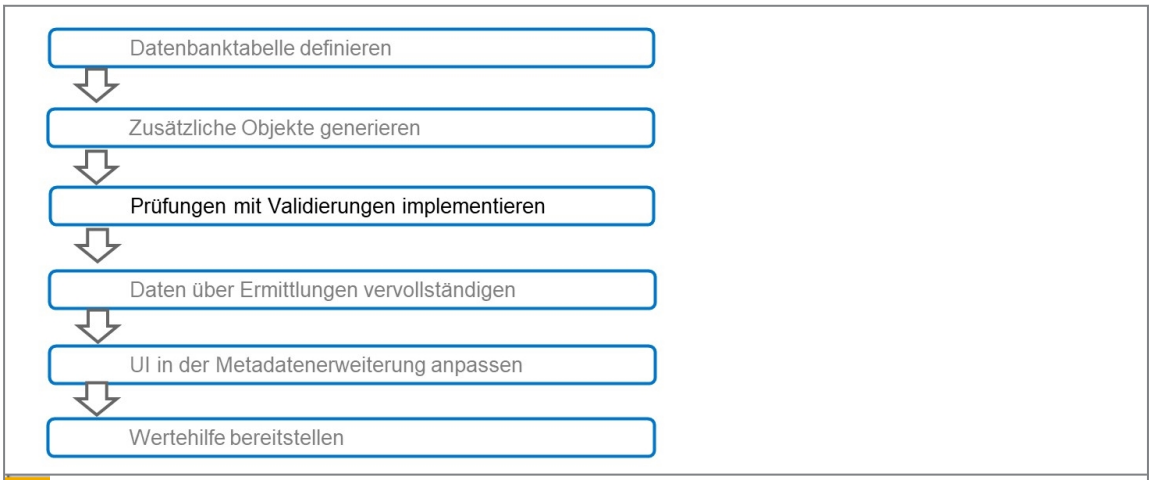
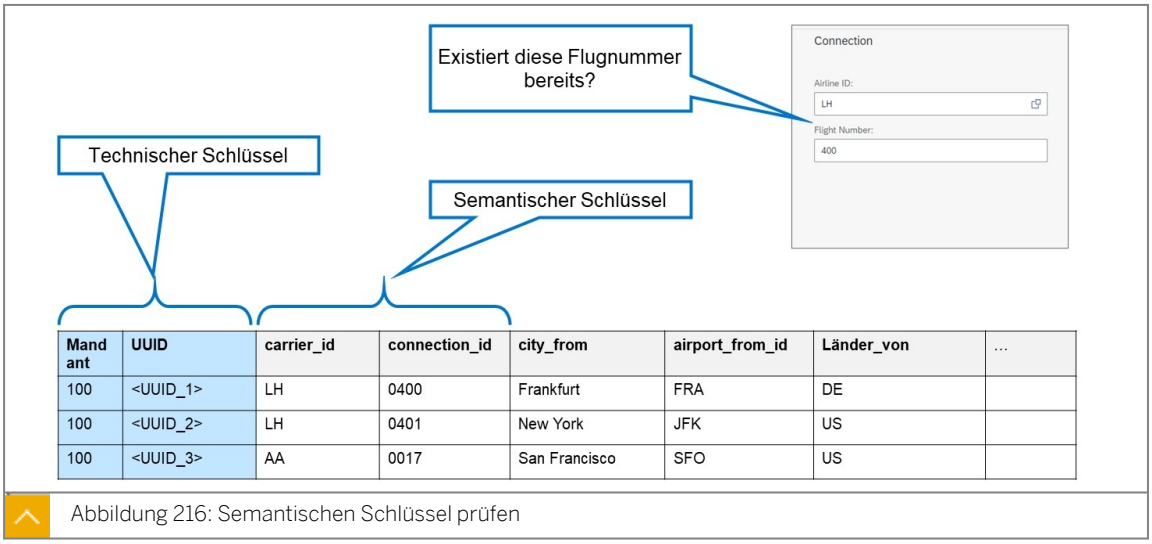


Abbildung 215: Eine einfache Anwendung entwickeln

Semantischen Schlüssel prüfen



Mandant	UUID	carrier_id	connection_id	city_from	airport_from_id	Länder_von	...
100	<UUID_1>	LH	0400	Frankfurt	FRA	DE	
100	<UUID_2>	LH	0401	New York	JFK	US	
100	<UUID_3>	AA	0017	San Francisco	SFO	US	

Abbildung 216: Semantischen Schlüssel prüfen

© Copyright. Alle Rechte vorbehalten.

Im ABAP-RESTful-Anwendungsprogrammiermodell setzt sich der Schlüssel einer Datenbanktabelle häufig aus dem Mandantenfeld und einem UUID-Feld zusammen, dessen Wert automatisch von der Laufzeit vergeben wird, wenn Sie eine neue Instanz des Business-Objekts anlegen. Diese Feldkombination ist ausreichend, um sicherzustellen, dass das System jeden Datensatz in der Tabelle eindeutig identifizieren kann. Neben diesem technischen Schlüssel hat unser Objekt jedoch auch einen semantischen Schlüssel - in diesem Fall die Kombination aus Fluggesellschaft und Flugnummer, die auch gemäß der Business-Logik eindeutig sein muss. Um die Eindeutigkeit dieser Feldkombination zu gewährleisten, müssen Sie eine eigene Prüfung in Form einer Validierung implementieren.

Sie deklarieren Validierungen in der Verhaltensdefinition der CDS-View-Entität und implementieren sie in der Verhaltensimplementierungsklasse.

Eingabeprüfungen in der App

Neben der Prüfung des semantischen Schlüssels gibt es weitere Prüfungen, die Sie durchführen müssen. Obwohl die generierte App es Ihnen ermöglicht, Daten anzulegen, zu lesen, zu aktualisieren und zu löschen, enthält sie noch keine Konsistenzprüfungen. Folglich können Sie Flugverbindungen für Fluggesellschaften anlegen, die nicht vorhanden sind oder bei denen Abflug- und Zielflughafen identisch sind.

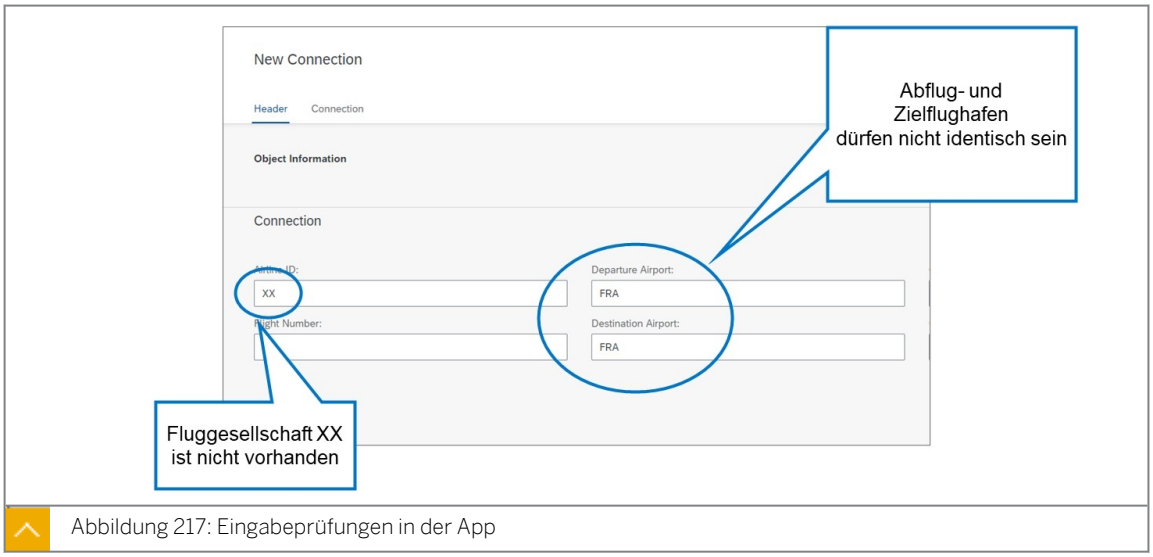


Abbildung 217: Eingabeprüfungen in der App

Um dies zu verhindern, definieren Sie weitere Validierungen in der Verhaltensdefinition und implementieren sie in der Verhaltensimplementierungsklasse.

Nachrichtentexte anlegen

Bevor Sie die Validierung anlegen, müssen Sie die Texte anlegen, die Sie anzeigen möchten. Dies geschieht über eine Nachrichtenklasse. Eine Nachrichtenklasse ist eine Sammlung von bis zu 1000 Nachrichten, die zu einem bestimmten Anwendungsbereich gehören. Wie in der Abbildung dargestellt, hat jeder Text eine Nummer, die die Nachricht innerhalb der Nachrichtenklasse eindeutig identifiziert.

© Copyright. Alle Rechte vorbehalten.

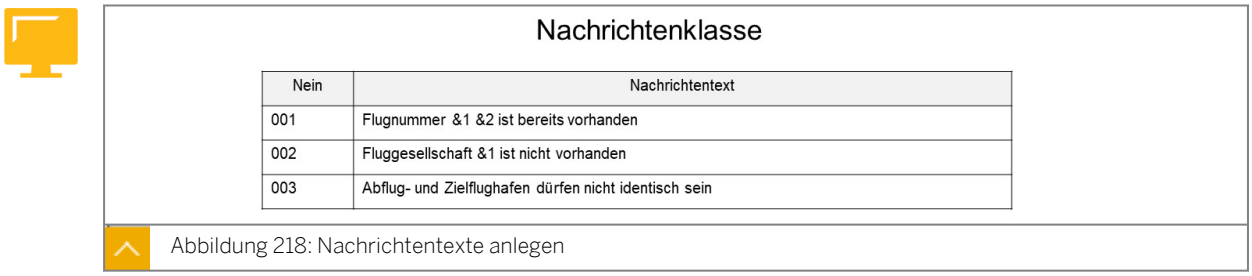


Abbildung 218: Nachrichtentexte anlegen

Um eine neue Nachrichtenklasse anzulegen, gehen Sie wie folgt vor:

1. Wählen Sie *File* → *New* → *Other...*, und geben Sie message in das Filterfeld ein.
2. Doppelklicken Sie auf den Eintrag *Nachrichtenklasse* in der Trefferliste, und geben Sie dann ein Paket, einen Namen und eine Beschreibung für die neue Nachrichtenklasse ein. Wählen Sie *Next*.
3. Ordnen Sie die Nachrichtenklasse einem Transportauftrag zu, und wählen Sie *Fertigstellen*.

Nachrichten können auch Platzhalter enthalten, die bei der Anzeige der Nachricht durch konkrete Werte ersetzt werden. Platzhalter werden durch das Et-Zeichen gefolgt von einer Zahl gekennzeichnet. Sie können in jeder Nachricht bis zu vier Platzhalter verwenden.

Validierung definieren

Um eine Validierung zu definieren, fügen Sie der Verhaltensdefinition Ihres Business-Objekts eine Validierungsdeklaration hinzu. In diesem Beispiel sollte die Validierung immer dann durchgeführt werden, wenn der Benutzer einen Datensatz sichert. Dies kann entweder beim Anlegen des Datensatzes oder bei einer nachträglichen Änderung des Datensatzes der Fall sein.

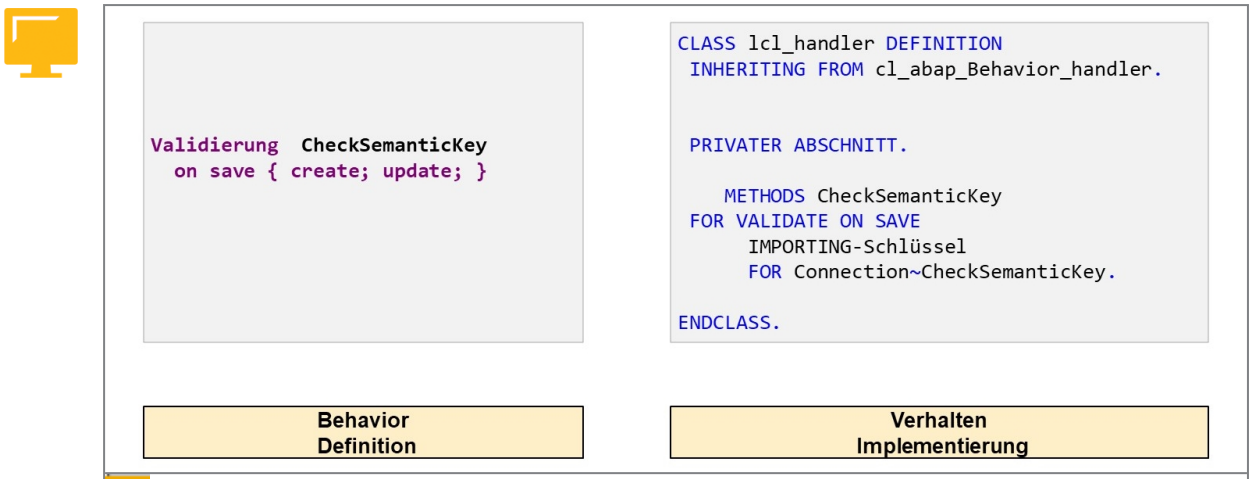


Abbildung 219: Validierung definieren

Wenn Sie die Validierung in der Verhaltensdefinition definieren, erhalten Sie eine Warnung, dass die entsprechende Methode nicht vorhanden ist. Verwenden Sie einen Quickfix (Tastenkombination `STRG + 1`), um die Methode zur Verhaltensimplementierung hinzuzufügen. Die Verhaltensimplementierung ist eine lokale Klasse innerhalb Ihres Behavior-Pools. Die Methodendefinition enthält den Zusatz `FOR VALIDATE ON SAVE`, der ihn als Implementierung der Validierung kennzeichnet. Sie hat den Importing-Parameter *KEYS*. Hierbei handelt es sich um eine interne Tabelle, die die Schlüssel der angelegten oder

© Copyright. Alle Rechte vorbehalten.

geänderten Objekte enthält. Sie verwenden diese, um die tatsächlichen Daten zu lesen, die der Benutzer eingegeben hat.

Der Zusatz `FOR Connection~CheckSemanticKey` verknüpft die Methode mit der Validierung *CheckSemanticKey* aus der Verhaltensdefinition. *Verbindung* ist hier der Aliasname der View-Entität *Z_R_CONNECTION*.

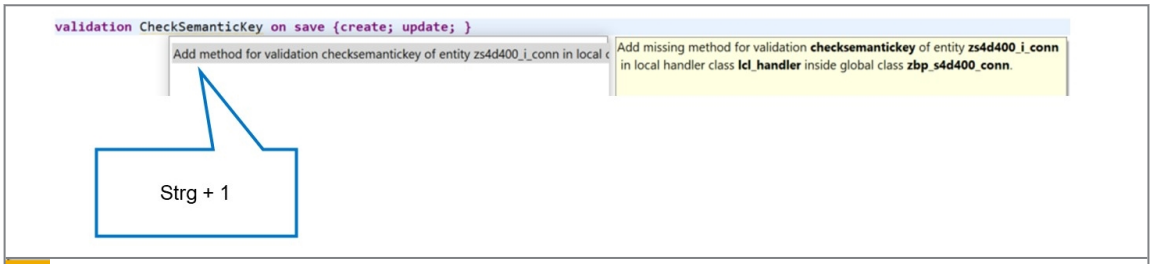


Abbildung 220: Verwenden einer Schnellkorrektur

Wenn Sie eine Validierung definieren, müssen Sie auch deren Implementierung anlegen. Dies ist eine Methode im Behavior-Pool. Der einfachste Weg, dies zu tun, ist die Verwendung einer Schnellkorrektur. Positionieren Sie den Cursor auf den Namen der Validierung, und drücken Sie `STRG + 1`. ADT schlägt vor, die Methode anzulegen. Doppelklicken Sie auf den Vorschlag, um die Methode anzulegen.

Der Validierungsprozess

Notiz:
Einige Codebeispiele in diesem Abschnitt verwenden SELECT-Anweisungen innerhalb von Schleifen. Dies wurde vorgenommen, um die Beispiele einfach zu halten. Beachten Sie, dass SELECTs in Schleifen zu Performanceproblemen führen können und vermieden werden sollten.

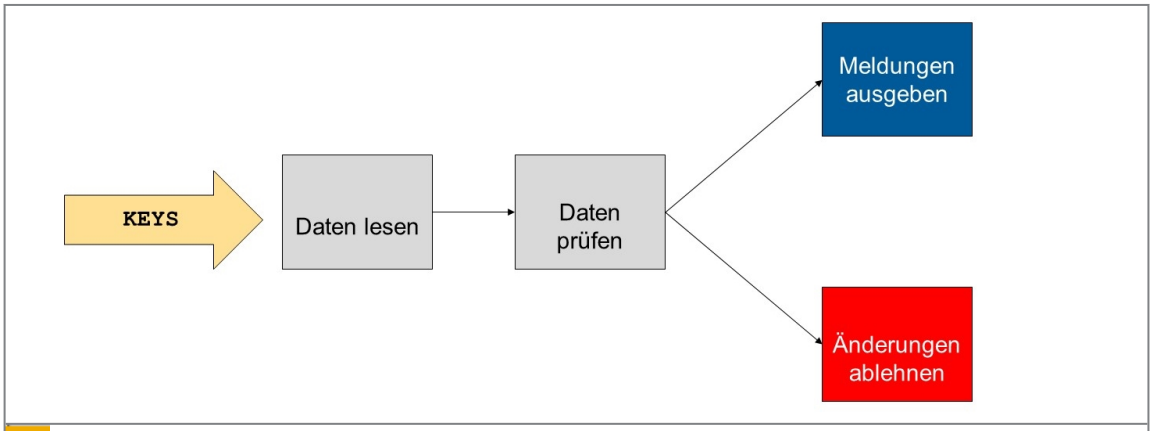


Abbildung 221: Validierungsprozess

Wenn das System eine Validierung auslöst, ruft es die entsprechende Implementierung auf. Der Importing-Parameter *KEYS* enthält die Schlüssel der geänderten Datensätze. Sie verwenden die Schlüssel, um die Felder der Datensätze zu lesen, die Sie mit Entity Manipulation Language (EML) benötigen. EML ist ein spezieller Satz von Anweisungen in ABAP, mit dem Sie Business-Objekte adressieren können.

Nachdem Sie die Daten gelesen haben, können Sie die von Ihnen benötigten Prüfungen durchführen. Wenn die Prüfung fehlschlägt, müssen Sie eine entsprechende Fehlermeldung

© Copyright. Alle Rechte vorbehalten.

ausgeben und vor allem dem Framework mitteilen, die Änderungen nicht in die Datenbank zu schreiben.

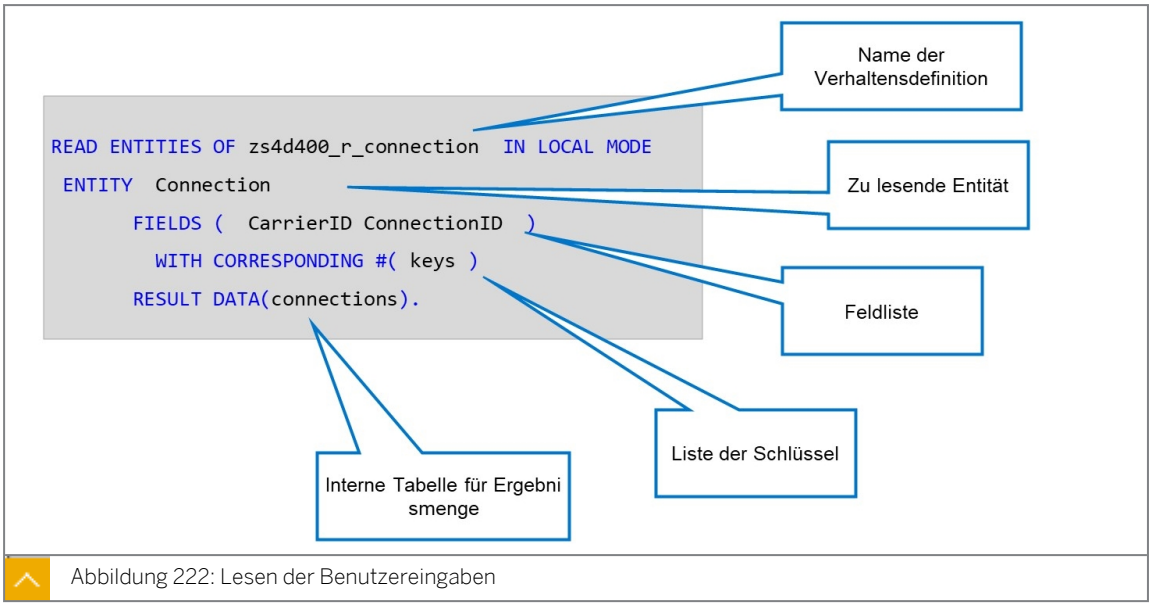


Abbildung 222: Lesen der Benutzereingaben

Die erste Aufgabe in einer Validierung besteht darin, die Benutzereingaben zu lesen. Dazu verwenden Sie die EML-Anweisung (Entity Manipulation Language) `READ ENTITIES`. Die Schlüssel der entsprechenden Datensätze werden mit den *Importparameterschlüsseln* an die Validierung übergeben.

Die Felder, die Sie zur Validierung des semantischen Schlüssels benötigen, sind *Spediteur-ID* für die Fluggesellschaft und *ConnectionID* für die Flugnummer.

Der Quelltextausschnitt verwendet das Schlüsselwort CORRESPONDING und eine Inline-Deklaration für die Ergebnismenge. Nachfolgend sehen Sie den äquivalenten Code, der explizit definierte Variablen verwendet, was das Verständnis der verwendeten Typen erleichtert.

```
DATA read_keys   TYPE TABLE FOR READ IMPORT zs4d400_r_connection.
DATA connections TYPE TABLE FOR READ RESULT zs4d400_r_connection.

read_keys = CORRESPONDING #( keys ).

READ ENTITIES OF zs4d400_r_connection IN LOCAL MODE
       ENTITY Connection
       FIELDS ( CarrierID ConnectionID )
         WITH read_keys
       RESULT connections.
```

Nachdem Sie die Benutzereingaben gelesen haben, können Sie die Werte von *CarrierID* und *ConnectionID* verwenden, um zu sehen, ob dieser semantische Schlüssel bereits in einem anderen Datensatz verwendet wurde als dem, den Sie gerade bearbeiten. Da sich die Schlüsselkombination entweder in der aktiven Tabelle oder in der Entwurfstabelle befinden kann, müssen Sie beides durchsuchen. Am effizientesten ist dies mit einer **Union**.

© Copyright. Alle Rechte vorbehalten.

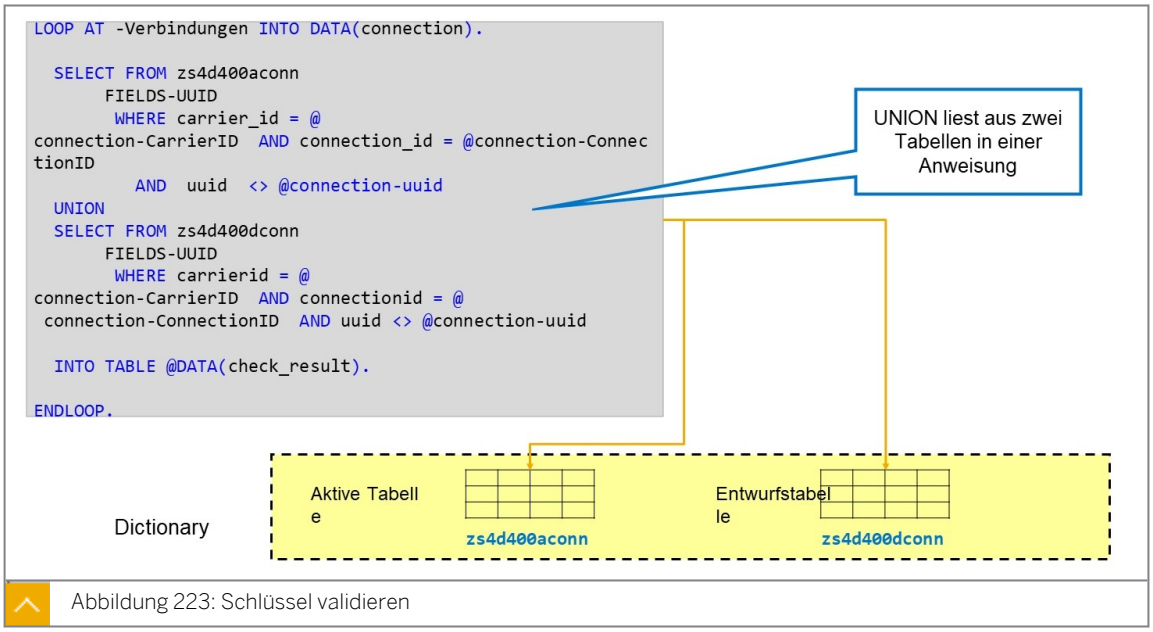


Abbildung 223: Schlüssel validieren

Die Ergebnismenge dieser Abfrage sollte immer leer sein. Ist dies nicht der Fall, gibt es mehr Datensätze mit derselben Kombination aus *Spediteur-ID* und *Verbindungs-ID*. Das bedeutet, dass der Datensatz, den der Benutzer gerade anlegen möchte, eine Dublette ist und abgelehnt werden muss.

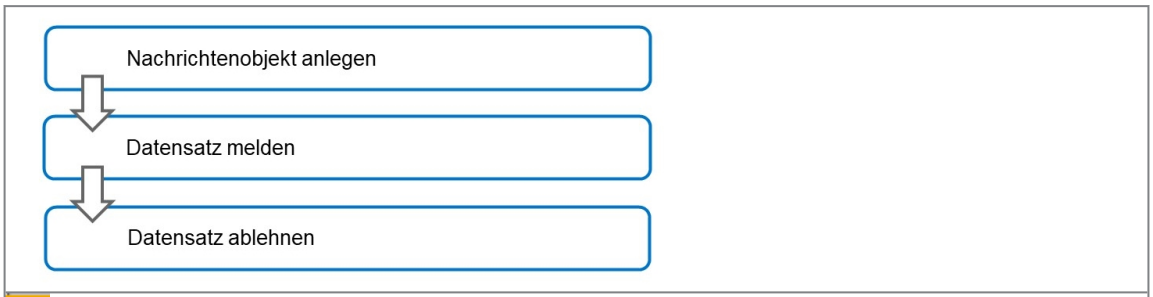


Abbildung 224: Vorgehen bei fehlgeschlagener Validierung

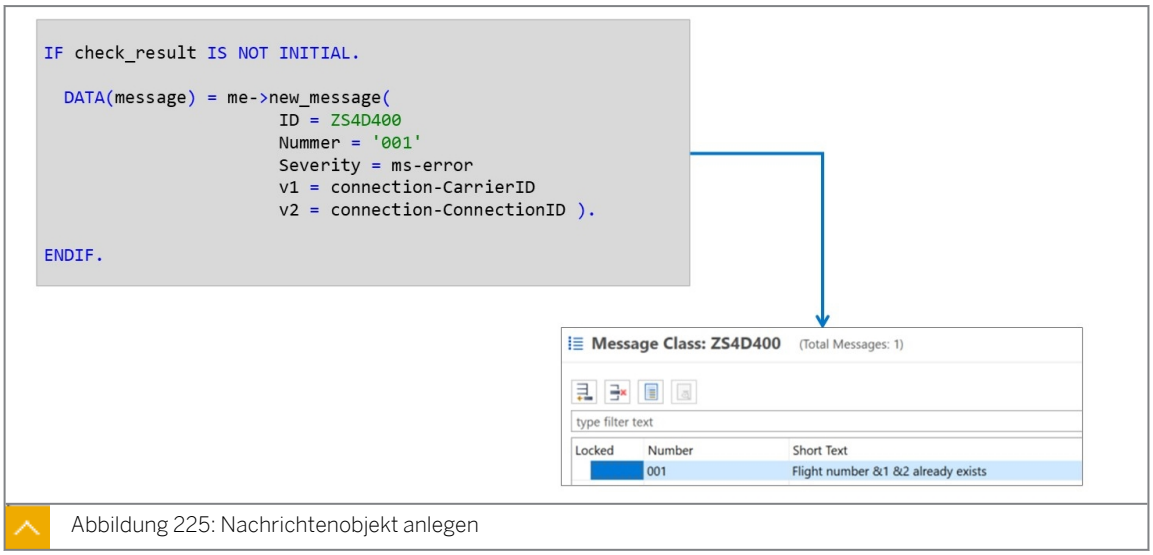


Abbildung 225: Nachrichtenobjekt anlegen

Wenn die Kombination aus Spediteur-ID und Verbindungs-ID bereits vorhanden ist, gibt es einen Eintrag in der Tabelle *check_result*. In diesem Fall müssen Sie eine Meldung ausgeben.

© Copyright. Alle Rechte vorbehalten.

Der erste Schritt besteht darin, ein Nachrichtenobjekt anzulegen. Dazu verwenden Sie die Selbstreferenz me und rufen die Methode *new_message( )* auf. Die Parameter *ID*, *Nummer* und *Schweregrad* sind obligatorisch. *ID* ist der Name der Nachrichtenklasse, die die Nachricht enthält; *Nummer* ist die Nachrichtennummer. *Der Schweregrad* klassifiziert die Meldung als Erfolgs-, Informations-, Warn- oder Fehlermeldung. Die Verhaltensimplementierungsklasse enthält eine strukturierte Konstante *ms*, deren Komponenten die verschiedenen Schweregrade darstellen. In diesem Fall benötigen Sie den Schweregrad *ms-error*.

Die Methode hat auch die optionalen Importparameter *v1*, *v2*, *v3* und *v4*. Sie verwenden diese, um Platzhalter durch konkrete Werte zu ersetzen. In diesem Beispiel wird der Platzhalter &1 durch den Fluggesellschaftscode und der Platzhalter &2 durch die Flugnummer ersetzt.

Das Ergebnis des Methodenaufrufs ist eine Objektreferenz. Im nächsten Schritt übergeben Sie das Objekt an die Laufzeit, sodass die Fehlermeldung an den OData-Service zurückgegeben und in der App-Vorschau angezeigt wird.

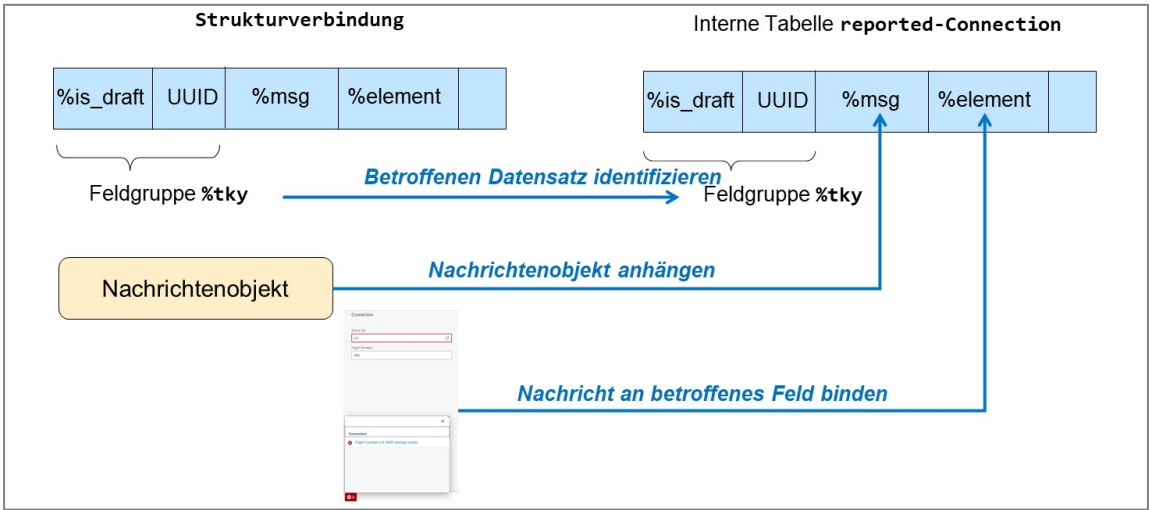


Abbildung 226: Meldung melden

Damit die Laufzeit eine Meldung anzeigt, müssen Sie sie mit der Struktur *reported* melden. Dies ist ein impliziter Changing-Parameter aller Validierungsmethoden und eine tiefe Struktur. Sie enthält eine Komponente mit dem Aliasnamen der Entität. Diese Komponente ist eine interne Tabelle.

Um die Meldung zu melden, müssen Sie drei Schritte ausführen:

1. Ergänzen Sie die interne Tabelle um den Schlüssel des betroffenen Satzes. Dies ist über die Feldgruppe *%tky* möglich. Wenn Sie solche Felder gruppieren, können Sie den Namen der Gruppe adressieren, anstatt jedes Feld einzeln ansprechen zu müssen.
2. Hängen Sie das Nachrichtenobjekt an die Tabelle an. Dazu ordnen Sie die Objektreferenz des Nachrichtenobjekts der Komponente *%msg* der internen Tabelle zu.
3. Binden Sie die Nachricht an das betroffene Feld. Dadurch wird sichergestellt, dass das Feld in der App hervorgehoben wird. Dies wiederum hilft dem Benutzer, besser in der App zu navigieren. Dazu verwenden Sie die Komponente *%element* der internen Tabelle.

© Copyright. Alle Rechte vorbehalten.

```
DATA reported_record LIKE LINE OF reported-connection.
```

```
reported_record-%tky   = connection-%tky.
reported_record-%msg   =  Nachricht.
reported_record-%element-CarrierID =  if_abap_behv=>mk-on.
reported_record-%element-ConnectionID =  if_abap_behv=>mk-on.

APPEND reported_record TO reported-connection.
```

Abbildung 227: Meldung melden

In diesem Beispiel ist *reported_record* eine Struktur mit dem Zeilentyp der internen Tabelle *reported-connection*. Sie füllen die Komponente *%tky* mit dem Inhalt der Feldgruppe *%tky* in der Struktur *connection*. Diese *Verbindungsstruktur* dient als Arbeitsbereich für die interne Tabelle, die die vom Benutzer eingegebenen Daten enthält. Anschließend ordnen Sie das Nachrichtenobjekt, das Sie mit der Methode *new_message( )* angelegt haben, der Komponente *%msg* zu. Zum Schluss verwenden Sie die Struktur *%element*, um die Nachricht an die Felder *CarrierID* und *ConnectionID* zu binden. *%element* enthält eine Komponente für jedes Feld in der Entität. Wenn Sie eine Komponente auf *wahr* setzen, wird das entsprechende Eingabefeld in der App hervorgehoben. Dazu verwenden Sie die strukturierte Konstante *if_abap_behv=>mk*. Dies hat die Komponente *an* für markiert/wahr und *aus* für nicht markiert/falsch.

Die globalen Konstanten *abap_true* und *abap_false* können an dieser Stelle nicht verwendet werden, da deren Datentypen nicht kompatibel sind.

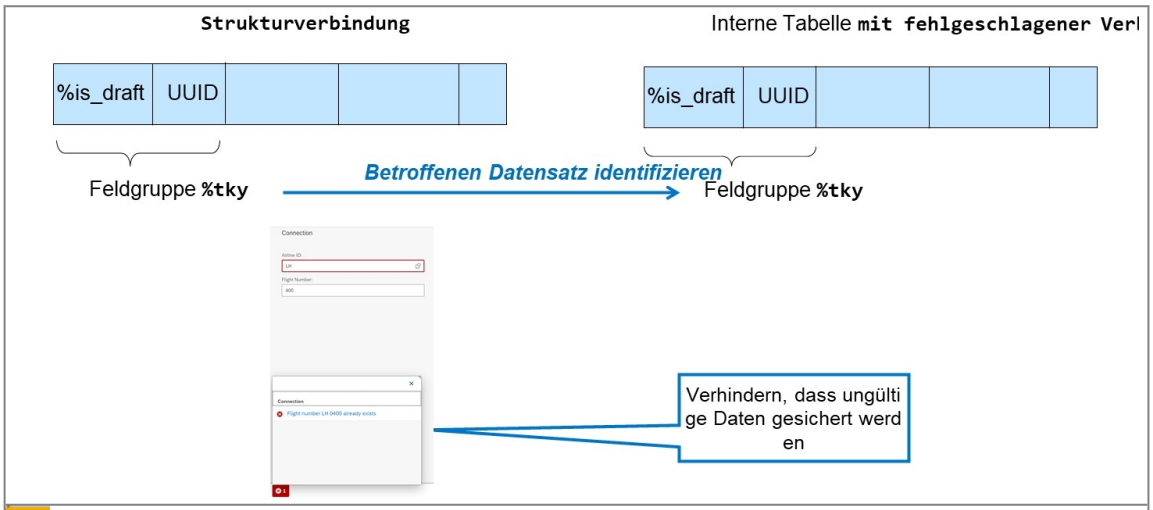


Abbildung 228: Änderungen ablehnen

Zusätzlich zur Ausgabe der Meldung müssen Sie der Laufzeit mitteilen, dass die fehlerhaften Daten nicht gesichert werden sollen. Dazu verwenden Sie die *fehlgeschlagene* Struktur der Validierungsmethode. *Fehlgeschlagen* ist ein impliziter Änderungsparameter, der in allen Validierungsmethoden vorhanden ist.

Um einen Datensatz als *fehlgeschlagen* zu melden, fügen Sie seine Feldgruppe *%tky* der Feldgruppe *%tky* der internen Tabelle *failed-Connection* hinzu.

© Copyright. Alle Rechte vorbehalten.

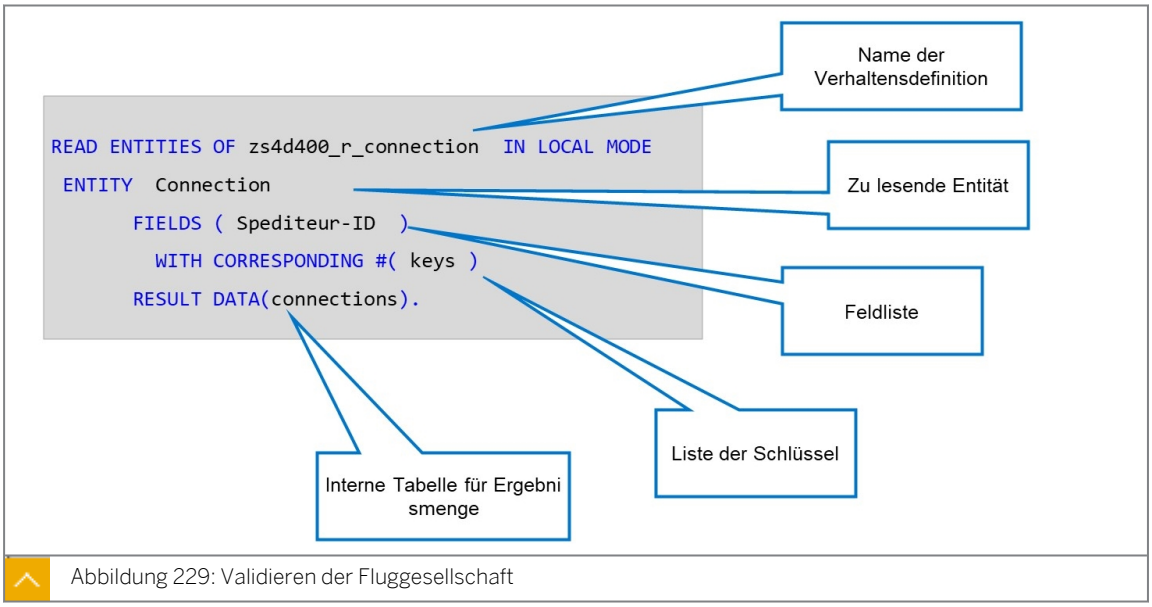


Abbildung 229: Validieren der Fluggesellschaft

Die nächste Validierung prüft, ob die Fluggesellschaft, die der Benutzer eingegeben hat, tatsächlich vorhanden ist. Der erste Schritt besteht darin, die Benutzereingaben mit der EML-Anweisung READ ENTITIES zu lesen. Dieses Mal müssen Sie nur das Feld *Spediteur-ID* lesen.

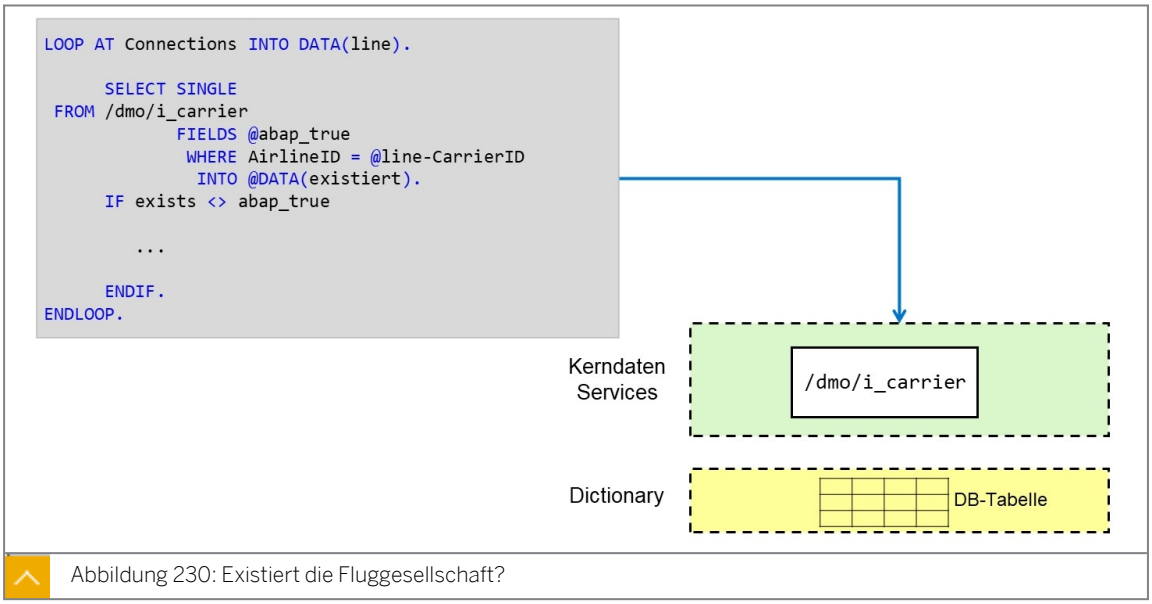


Abbildung 230: Existiert die Fluggesellschaft?

Die SELECT SINGLE -Anweisung liest Daten über die CDS-View-Entität /dmo/i_carrier und prüft, ob die angegebene Fluggesellschaft vorhanden ist. Ist dies der Fall, wird der Wert der globalen Konstante abap_true ('X') in das Feld exists gestellt. Wenn nach der SELECT-Anweisung initial ist, müssen Sie eine Meldung ausgeben, melden und den Datensatz wie im vorherigen Beispiel zur fehlgeschlagenen Struktur hinzufügen.

© Copyright. Alle Rechte vorbehalten.

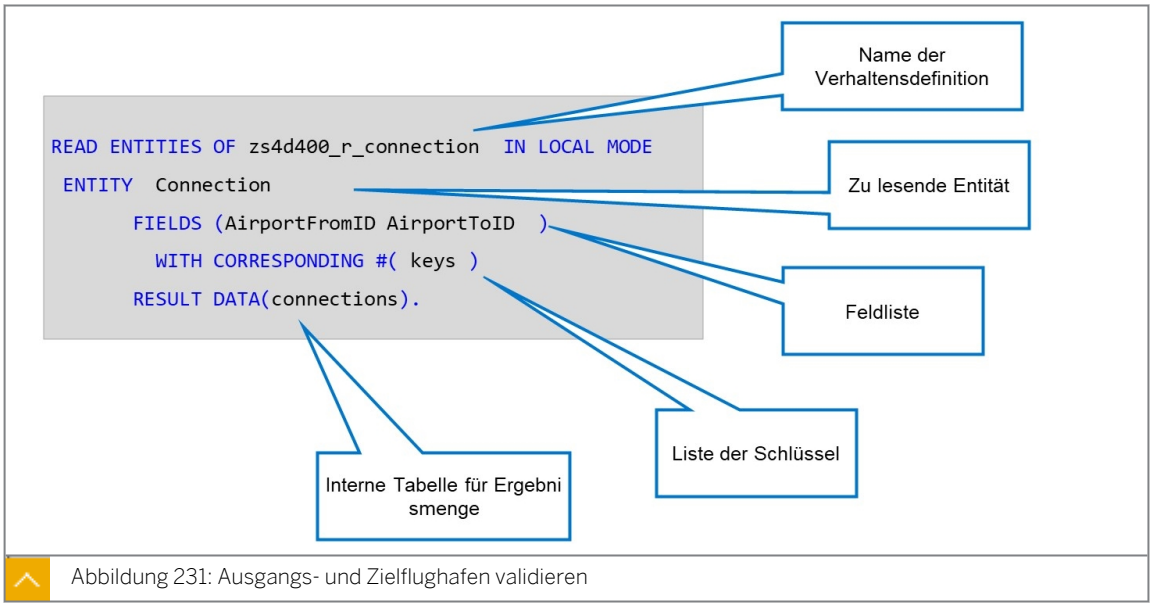


Abbildung 231: Ausgangs- und Zielflughafen validieren

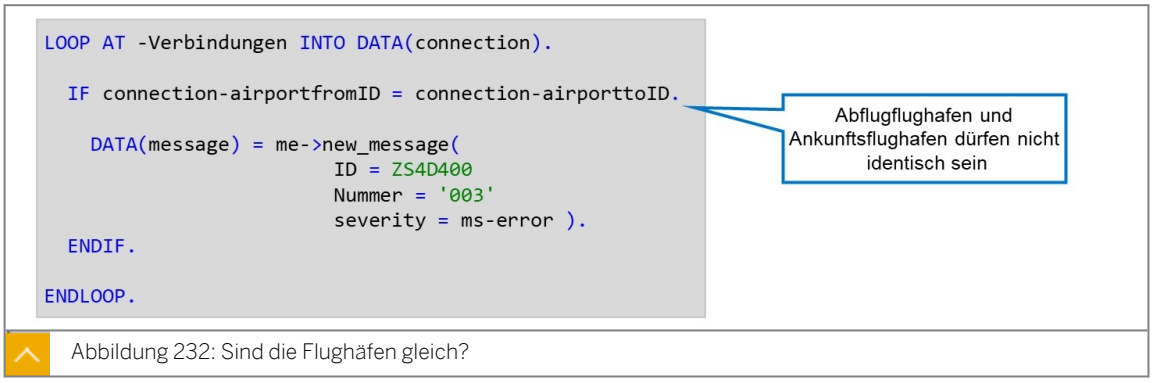


Abbildung 232: Sind die Flughäfen gleich?

Wenn Abflug- und Ankunftsflughafen identisch sind, müssen Sie die entsprechende Meldung ausgeben und die gemeldeten und fehlgeschlagenen Strukturen füllen. Der Quelltextauszug zeigt den relevanten Quelltext zum Anlegen der Nachricht an. Der Quelltext zum Füllen der gemeldeten und fehlgeschlagenen Strukturen ist derselbe wie in den vorherigen Beispielen.

Ermittlungen

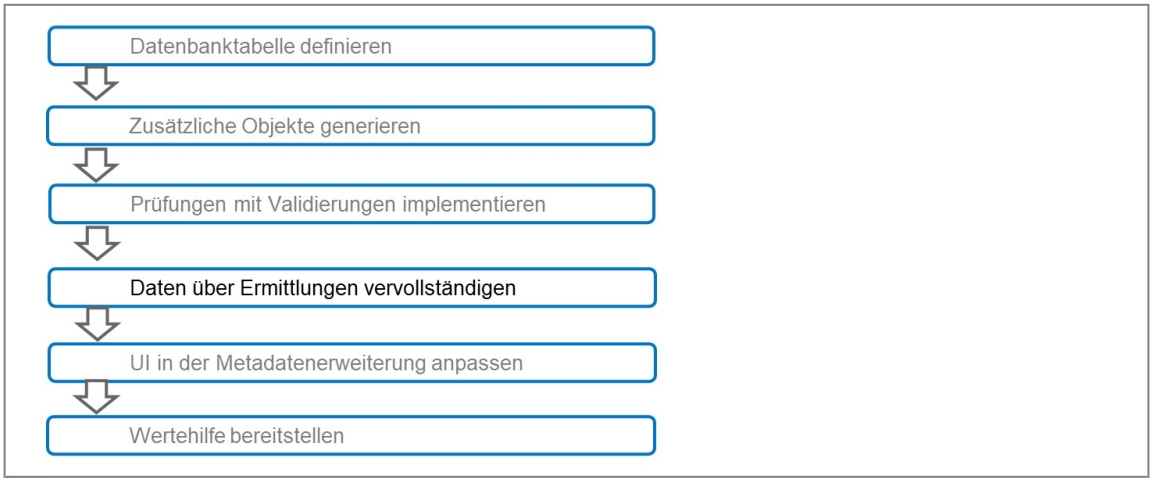


Abbildung 233: Eine einfache Anwendung entwickeln

© Copyright. Alle Rechte vorbehalten.

In diesem Abschnitt erfahren Sie, wie Sie die Daten mithilfe von Ermittlungen vervollständigen.

Orte anhand von Flughafencodes ermitteln

In der Beispiel-App enthält die Flugverbindungsentität einen Abflugflughafen, eine Stadt und ein Land sowie einen Ankunftsflughafen, eine Stadt und ein Land. Obwohl es möglich wäre, den Benutzer zur Eingabe all dieser Informationen zu zwingen, ist es in Bezug auf Benutzerfreundlichkeit und Datenkonsistenz besser, dass der Benutzer nur die Flughafencodes eingibt und die App die entsprechenden Orts- und Länderinformationen aus der Datenbank liest. Im ABAP-RESTful-Anwendungsprogrammiermodell können Sie diese Art von Aufgabe mithilfe einer **Ermittlung** ausführen.

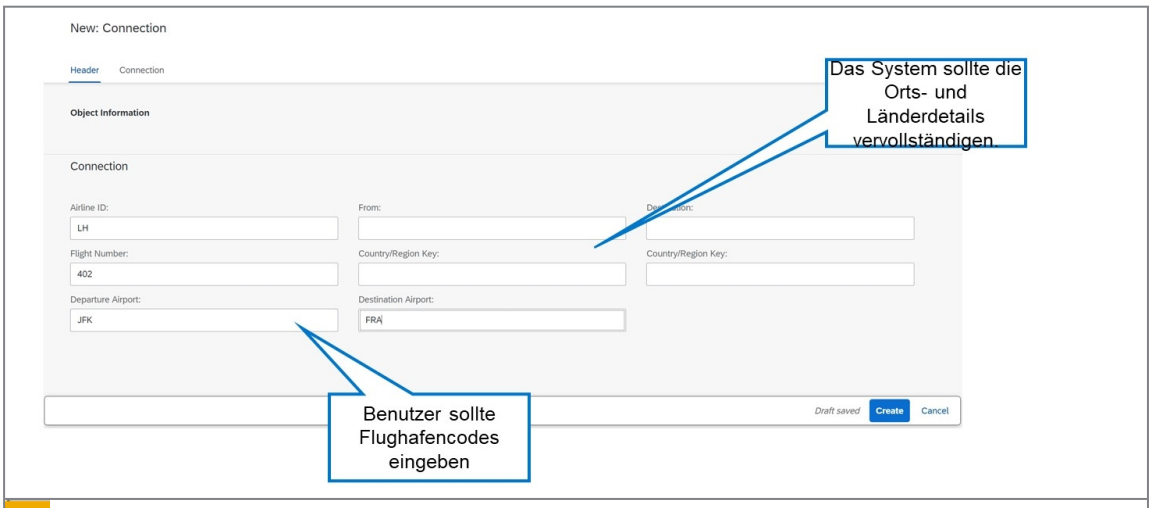


Abbildung 234: Orte anhand von Flughafencodes ermitteln

Sie implementieren zunächst die Ermittlung. Anschließend erfahren Sie, wie Sie die Eingabe für die Felder deaktivieren, die automatisch gefüllt werden.

Findung definieren

Sie definieren eine Determinierung in der Verhaltensdefinition eines Business-Objekts. Die Ermittlung heißt hier getCities, sie wird immer dann aufgerufen, wenn das Business-Objekt gesichert wird und sich mindestens eines der Felder *AirportFromID* und *AirportToID* geändert hat. Sie können einen Quickfix in der Verhaltensdefinition verwenden, um die entsprechende Methode in der Verhaltensimplementierung anzulegen.

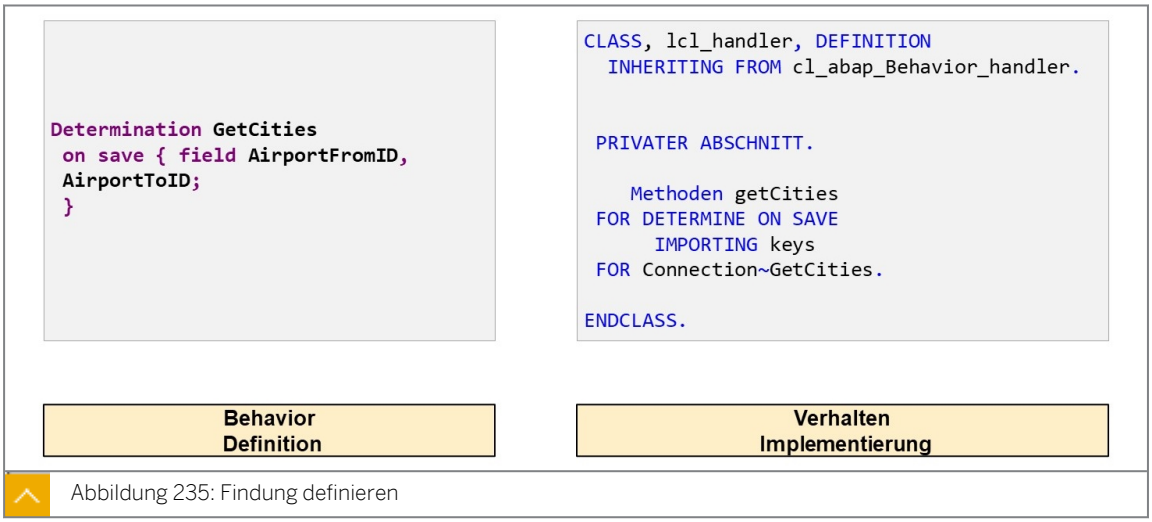


Abbildung 235: Findung definieren

© Copyright. Alle Rechte vorbehalten.

Der Findungsprozess

Sehen wir uns die einzelnen Schritte des Findungsprozesses an.

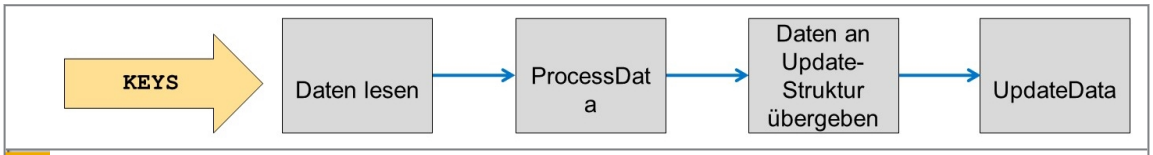


Abbildung 236: Der Findungsprozess

Wenn das System eine Ermittlung anstößt, ruft es die entsprechende Implementierung auf. Der Importing-Parameter KEYS enthält die Schlüssel der geänderten Datensätze. In der Ermittlungsmethode verwenden Sie EML, um die Daten basierend auf den Schlüsseln genauso zu lesen wie in den Validierungen. Bei einer Ermittlung manipulieren Sie jedoch auch die Daten in der Methode und müssen folglich die vom Framework gehaltenen Daten mit der EML-Anweisung UPDATE aktualisieren.

Notiz:
Einige Codebeispiele in diesem Abschnitt verwenden SELECT-Anweisungen innerhalb von Schleifen. Dies wurde vorgenommen, um die Beispiele einfach zu halten. Beachten Sie, dass SELECTs in Schleifen zu Performanceproblemen führen können und vermieden werden sollten.

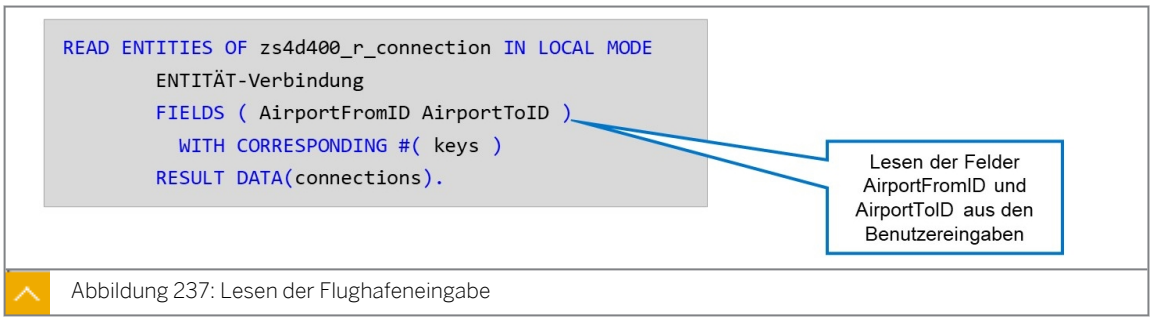


Abbildung 237: Lesen der Flughafeneingabe

Zu Beginn der Ermittlung lesen Sie die Benutzereingaben über EML. Sie benötigen die Felder *AirportFromID* und *AirportToID* und verwenden diese, um die Stadt- und Länderinformationen zu vervollständigen.

© Copyright. Alle Rechte vorbehalten.

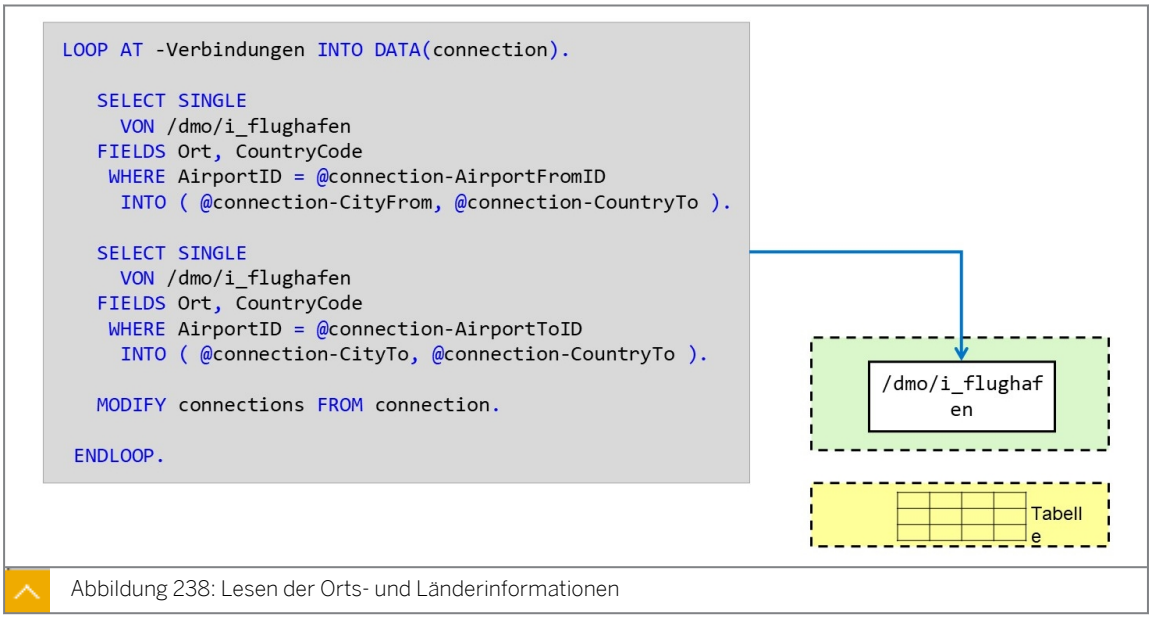


Abbildung 238: Lesen der Orts- und Länderinformationen

Das Demonstrationsdatenmodell stellt die CDS-View-Entität */dmo/i_flughafen* bereit, mit der Sie die Stadt und das Land lesen können, in denen sich ein bestimmter Flughafen befindet. Das Beispiel verwendet die Variante der INTO -Klausel, in der Sie die Felder der Struktur, die Sie füllen möchten, explizit angeben. Beachten Sie, dass die Änderungen an den Daten im Arbeitsbereich der internen Tabelle liegen und dass sie mit der MODIFY -Anweisung an die Tabelle selbst zurückgegeben werden müssen.

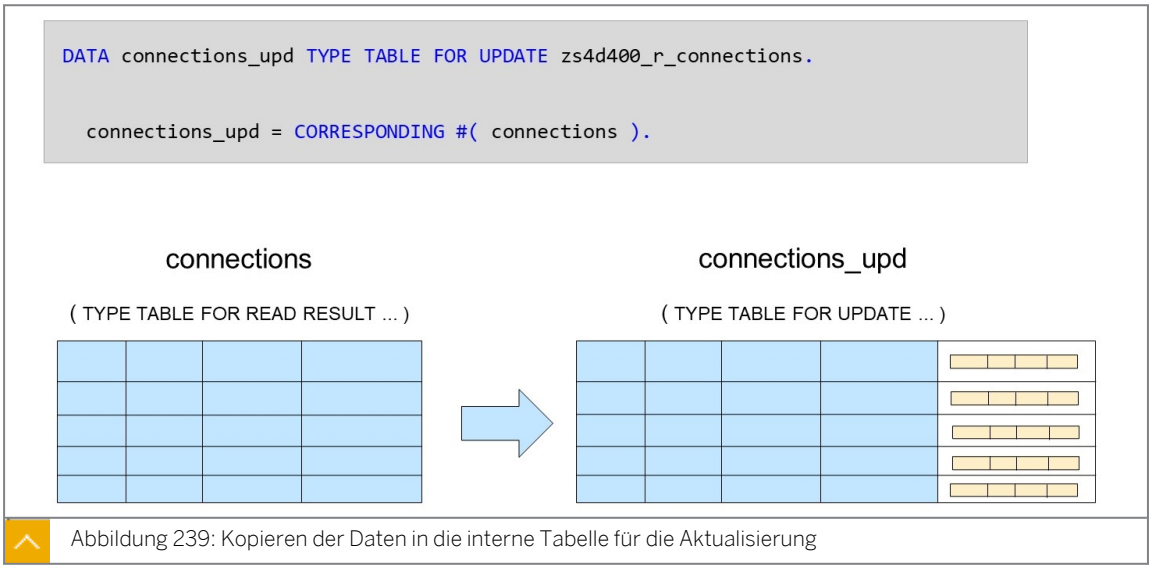


Abbildung 239: Kopieren der Daten in die interne Tabelle für die Aktualisierung

Die Anweisung READ ENTITIES gibt eine interne Tabelle mit dem abgeleiteten Typ FOR READ RESULT zurück. Um die Daten im Transaktionspuffer zu ändern, benötigen Sie eine MODIFY ENTITIES -Anweisung. Die Daten, die Sie ändern möchten, übergeben Sie an diese Anweisung über eine interne Tabelle mit dem abgeleiteten Typ FOR UPDATE. Die Datenfelder sind in beiden Typen identisch, aber die FOR UPDATE-Tabelle hat eine zusätzliche Struktur namens %control, die Verwaltungsinformationen enthält.

Sie können die *Verbindungstabelle* nicht an die MODIFY ENTITIES -Anweisung übergeben. Daher müssen Sie Ihre Daten in eine entsprechend typisierte interne Tabelle (*connections_upd*) kopieren, bevor Sie die eigentliche Modifikation durchführen.

© Copyright. Alle Rechte vorbehalten.

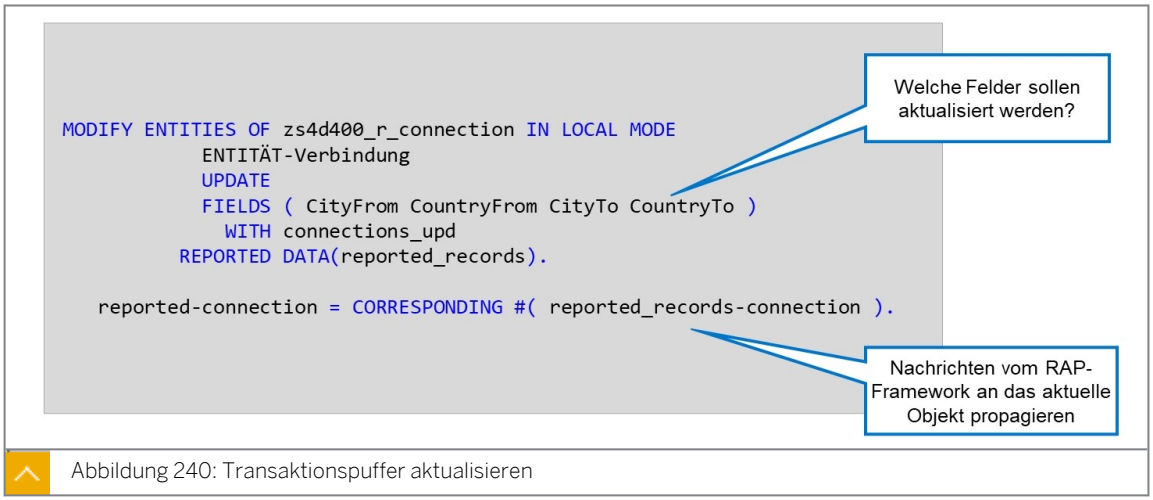


Abbildung 240: Transaktionspuffer aktualisieren

Um die Daten mit den Feldern zu aktualisieren, die Sie in der Ermittlung gefüllt haben, verwenden Sie die MODIFY ENTITIES -Anweisung. Darin geben Sie an, welche Felder in der FIELDS-Klausel aktualisiert werden sollen, und übergeben die Daten mit dem Zusatz WITH an eine interne Tabelle. Diese Tabelle muss den korrekten abgeleiteten Datentyp haben, der in diesem Fall `TYPE TABLE FOR UPDATE zsd4d400_r_connection` wäre.

Die MODIFY ENTITIES -Anweisung kann Nachrichten zurückgeben, die Sie über die REPORTED -Klausel erhalten. Anschließend propagieren Sie diese Nachrichten an Ihr eigenes Business-Objekt, indem Sie den Inhalt der internen Tabelle in die REPORTED-Struktur der Determinierungsmethode kopieren.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Übung 20

Flugpreis validieren

In dieser Übung definieren und implementieren Sie eine Validierung für den Flugpreis.

Vorlage:

keine

Lösung:

/LRN/S4D400_R_FLIGHT (Verhaltensdefinition)
/LRN/BP_S4D400_R_FLIGHT (globale Klasse)

Voraussetzungen

Sie haben die vorherigen Übungen abgeschlossen. Sie haben die Datenbanktabelle **`Z##FLIGHT`** (wobei ## für Ihre Gruppennummer steht) angelegt und gefüllt und die Entwicklungsobjekte für einen OData-UI-Service generiert.

Aufgabe 1: Preis validieren

Definieren und implementieren Sie eine Validierung, um zu prüfen, ob das Feld **`Preis`** einen positiven Wert hat (Namensvorschlag: **`validatePrice`**). Wenn der Wert negativ oder gleich null ist, lehnen Sie die Änderung ab, und melden Sie eine geeignete Fehlermeldung aus der Nachrichtenklasse **`/LRN/S4D400`**.

1. Definieren Sie in der Verhaltensdefinition **`ZR_##FLIGHT`** eine neue Validierung **`validatePrice`**. Stellen Sie sicher, dass die Validierung immer während der Standardoperation **`Anlegen`** ausgeführt wird, jedoch nur, wenn sich der Wert von **`Preis`** für alle anderen Vorgänge geändert hat.

   Hinweis:
   Verwenden Sie nach Möglichkeit die Code-Vervollständigung, um den Code einzugeben.

2. Aktivieren Sie die Verhaltensdefinition.

3. Verwenden Sie eine Schnellkorrektur, um die Validierungsimplementierungsmethode in der Verhaltens-Handler-Klasse anzulegen.

   Wo wird die Methode angelegt?

   _____

   _____

   _____

4. Deklarieren Sie zu Beginn der Methode **`validatePrice`** ein strukturiertes Datenobjekt, das Sie mit dem Zeilentyp **`failed-flight`** eingeben (Namensvorschlag:

© Copyright. Alle Rechte vorbehalten.

failed_record). Deklarieren Sie analog dazu ein strukturiertes Datenobjekt, das Sie mit dem Zeilentyp **reported-flight** (Namensvorschlag: **reported_record**) typisieren.

Notiz:
Diese Strukturen werden verwendet, um Zeilen zu **Failed-Flight** und **Reported-Flight** hinzuzufügen, wenn die Validierung einen Fehler findet.

5. Verwenden Sie eine READ ENTITIES -Anweisung, um die Benutzereingaben aus dem transaktionalen Puffer zu lesen. Verwenden Sie den Zusatz IN LOCAL MODE und stellen Sie sicher, dass nur die Schlüsselfelder und das Feld **Price** gelesen werden. Verwenden Sie eine Inline-Deklaration für die Ergebnismenge (Namensvorschlag **Flüge**).

Notiz:
Eine Auflistung der Schlüsselfelder nach dem Zusatz FIELDS ist nicht notwendig. READ ENTITIES liest immer die Schlüsselfelder.

6. Implementieren Sie eine Schleife über die Daten, die Sie gerade lesen. Verwenden Sie eine Inline-Deklaration für den Arbeitsbereich (Namensvorschlag: **Flug**).
7. Prüfen Sie innerhalb der Schleife, ob die **Preiskomponente** größer als null ist. Falls nicht, füllen Sie die Struktur **failed_record** mit dem Schlüssel des aktuellen Fluges und fügen Sie sie als neue Zeile in die Tabelle **failed-flight** ein. Füllen Sie analog dazu die Struktur **reported_record** mit dem Schlüssel des aktuellen Fluges und fügen Sie diese als neue Zeile in die Tabelle **reported-flight** ein.

Hinweis:
Bei entwurfsfähigen Business-Objekten wird die Verwendung der Komponente **%tky** für die Schlüsselvergabe empfohlen.

8. Füllen Sie vor der APPEND-Anweisung die Komponente **%msg** der Struktur **reported_record** mit einer Referenz auf ein Nachrichtenobjekt. Um das Nachrichtenobjekt anzulegen, rufen Sie die Methode **new_message** mit folgender Eingabe auf:

Parametername	Wert
ID	/LRN/S4D400
nummer	101
Severity	ms-error

9. Aktivieren Sie die Klasse.

Aufgabe 2: Testen und debuggen
Setzen Sie einen Breakpoint in der Validierungsimplementierung. Starten Sie die Vorschau des OData-UI-Service neu, und ändern Sie den Preis eines vorhandenen Flugs. Nehmen Sie gültige und ungültige Einträge vor, und debuggen Sie die Validierung.

© Copyright. Alle Rechte vorbehalten.

1. Setzen Sie einen Breakpoint bei der READ ENTITIES -Anweisung der Methode **`validatePrice`**.
2. Starten Sie die Vorschau des OData-UI-Service neu.
3. Zeigen Sie in der App die Liste der Flüge an, öffnen Sie die Details zu einem der Flüge, und wechseln Sie in den Änderungsmodus.
4. Nehmen Sie einige Änderungen am Preis vor, und wählen Sie dann *Sichern*. Analysieren Sie die Validierung im Debugger.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Lösung 20

Flugpreis validieren

In dieser Übung definieren und implementieren Sie eine Validierung für den Flugpreis.

Vorlage:

keine

Lösung:

/LRN/S4D400_R_FLIGHT (Verhaltensdefinition)
/LRN/BP_S4D400_R_FLIGHT (globale Klasse)

Voraussetzungen

Sie haben die vorherigen Übungen abgeschlossen. Sie haben die Datenbanktabelle **Z##FLIGHT** (wobei ## für Ihre Gruppennummer steht) angelegt und gefüllt und die Entwicklungsobjekte für einen OData-UI-Service generiert.

Aufgabe 1: Preis validieren

Definieren und implementieren Sie eine Validierung, um zu prüfen, ob das Feld **Preis** einen positiven Wert hat (Namensvorschlag: **validatePrice**). Wenn der Wert negativ oder gleich null ist, lehnen Sie die Änderung ab, und melden Sie eine geeignete Fehlermeldung aus der Nachrichtenklasse **/LRN/S4D400**.

1. Definieren Sie in der Verhaltensdefinition **ZR_##FLIGHT** eine neue Validierung **validatePrice**. Stellen Sie sicher, dass die Validierung immer während der Standardoperation **Anlegen** ausgeführt wird, jedoch nur, wenn sich der Wert von **Preis** für alle anderen Vorgänge geändert hat.

   Hinweis:
   Verwenden Sie nach Möglichkeit die Code-Vervollständigung, um den Code einzugeben.

   a) Öffnen Sie die Verhaltensdefinition **ZR_##FLIGHT**.

   b) Passen Sie den Quelltext wie folgt an:

```
create;
update;
delete;

validation validatePrice on save { create; field Price; }
```

2. Aktivieren Sie die Verhaltensdefinition.
   a) Drücken Sie **Strg + F3**, um die Verhaltensdefinition zu aktivieren.
3. Verwenden Sie eine Schnellkorrektur, um die Validierungsimplementierungsmethode in der Verhaltens-Handler-Klasse anzulegen.

© Copyright. Alle Rechte vorbehalten.

a) Positionieren Sie den Cursor auf den Namen der Validierung, und wählen Sie *Strg + 1*.

b) Doppelklicken Sie auf den Eintrag *Methode für Validierung hinzufügen...*.

Wo wird die Methode angelegt?

In der lokalen Klasse **lhc_zr_##flight** der globalen Klasse **ZBP_R_##FLIGHT**.

4. Deklarieren Sie zu Beginn der Methode **validatePrice** ein strukturiertes Datenobjekt, das Sie mit dem Zeilentyp **failed-flight** eingeben (Namensvorschlag: **failed_record**). Deklarieren Sie analog dazu ein strukturiertes Datenobjekt, das Sie mit dem Zeilentyp **reported-flight** (Namensvorschlag: **reported_record**) typisieren.

Notiz:
Diese Strukturen werden verwendet, um Zeilen zu **Failed-Flight** und **Reported-Flight** hinzuzufügen, wenn die Validierung einen Fehler findet.

a) Fügen Sie folgenden Code hinzu:

```
DATA failed_record   LIKE LINE OF failed-flight.
DATA reported_record LIKE LINE OF reported-flight.
```

5. Verwenden Sie eine READ ENTITIES -Anweisung, um die Benutzereingaben aus dem transaktionalen Puffer zu lesen. Verwenden Sie den Zusatz IN LOCAL MODE und stellen Sie sicher, dass nur die Schlüsselfelder und das Feld **Price** gelesen werden. Verwenden Sie eine Inline-Deklaration für die Ergebnismenge (Namensvorschlag **Flüge**).

Notiz:
Eine Auflistung der Schlüsselfelder nach dem Zusatz FIELDS ist nicht notwendig. READ ENTITIES liest immer die Schlüsselfelder.

a) Fügen Sie nach den Deklarationen den folgenden Code hinzu, und ersetzen Sie **##** durch Ihre Gruppennummer:

```
READ ENTITIES OF ZR_##Flight IN LOCAL MODE
  ENTITY Flight
    FIELDS ( Price )
    WITH CORRESPONDING #(  keys )
    RESULT DATA(flights).
```

6. Implementieren Sie eine Schleife über die Daten, die Sie gerade lesen. Verwenden Sie eine Inline-Deklaration für den Arbeitsbereich (Namensvorschlag: **Flug**).

a) Fügen Sie nach der EML-Anweisung den folgenden Quelltext hinzu:

```
LOOP AT flights INTO DATA(flight).

ENDLOOP.
```

7. Prüfen Sie innerhalb der Schleife, ob die **Preiskomponente** größer als null ist. Falls nicht, füllen Sie die Struktur **failed_record** mit dem Schlüssel des aktuellen Fluges und fügen Sie sie als neue Zeile in die Tabelle **failed-flight** ein. Füllen Sie analog dazu die Struktur **reported_record** mit dem Schlüssel des aktuellen Fluges und fügen Sie diese als neue Zeile in die Tabelle **reported-flight** ein.

© Copyright. Alle Rechte vorbehalten.

Hinweis:
Bei entwurfsfähigen Business-Objekten wird die Verwendung der Komponente **%tky** für die Schlüsselvergabe empfohlen.

a) Fügen Sie in der Schleife den folgenden Quelltext hinzu:

```
IF flight-price <= 0.

  failed_record-%tky = flight-%tky.
  APPEND failed_record TO failed-flight.

  reported_record-%tky = flight-%tky.
  APPEND reported_record TO reported-flight.
ENDIF.
```

8. Füllen Sie vor der APPEND-Anweisung die Komponente **%msg** der Struktur **reported_record** mit einer Referenz auf ein Nachrichtenobjekt. Um das Nachrichtenobjekt anzulegen, rufen Sie die Methode **new_message** mit folgender Eingabe auf:

Parametername	Wert
ID	/LRN/S4D400
nummer	101
Severity	ms-error

a) Passen Sie den Quelltext wie folgt an:

```
LOOP AT flights INTO DATA(flight).
  IF flight-price <= 0.

    failed_record-%tky = flight-%tky.
    APPEND failed_record TO failed-flight.

    reported_record-%tky = flight-%tky.
    reported_record-%msg = new_message(
                      id       = '/LRN/S4D400'
                      number   = '101'
                      severity = ms-error ).
    APPEND reported_record TO reported-flight.

  ENDIF.
ENDLOOP.
```

9. Aktivieren Sie die Klasse.

a) Drücken Sie **Strg + F3**, um die Klasse zu aktivieren.

Aufgabe 2: Testen und debuggen

Setzen Sie einen Breakpoint in der Validierungsimplementierung. Starten Sie die Vorschau des OData-UI-Service neu, und ändern Sie den Preis eines vorhandenen Flugs. Nehmen Sie gültige und ungültige Einträge vor, und debuggen Sie die Validierung.

1. Setzen Sie einen Breakpoint bei der READ ENTITIES -Anweisung der Methode **validatePrice**.

© Copyright. Alle Rechte vorbehalten.

a) Suchen Sie in der Implementierung der Methode **`validatePrice`** die Anweisung READ ENTITIES, und doppelklicken Sie auf den Bereich links neben der Zeilennummer, um einen Breakpoint zu setzen.

2. Starten Sie die Vorschau des OData-UI-Service neu.
   a) Schließen Sie das Browserfenster oder die Browserregisterkarte, die die Vorschau enthält.
   b) Öffnen Sie die Service-Bindung **`ZUI_##FLIGHT_O4`**. Wählen Sie in der Liste *Entitätsmenge und Assoziation* auf der rechten Seite zunächst den Eintrag *Flug* und dann *Vorschau...*.
3. Zeigen Sie in der App die Liste der Flüge an, öffnen Sie die Details zu einem der Flüge, und wechseln Sie in den Änderungsmodus.
   a) Fahren Sie wie in den vorangegangenen Übungen fort.
4. Nehmen Sie einige Änderungen am Preis vor, und wählen Sie dann *Sichern*. Analysieren Sie die Validierung im Debugger.
   a) Fahren Sie wie in den vorangegangenen Übungen fort.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Implementieren Sie das Verhalten eines Business-Objekts.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Lektion 4

Verbesserung der Benutzerfreundlichkeit

LERNZIELE DER LEKTION
Am Ende dieser Lektion können Sie:

- Ordnen Sie Felder in der App an.
- Stellen Sie eine Eingabehilfe bereit.

UI-Anpassung

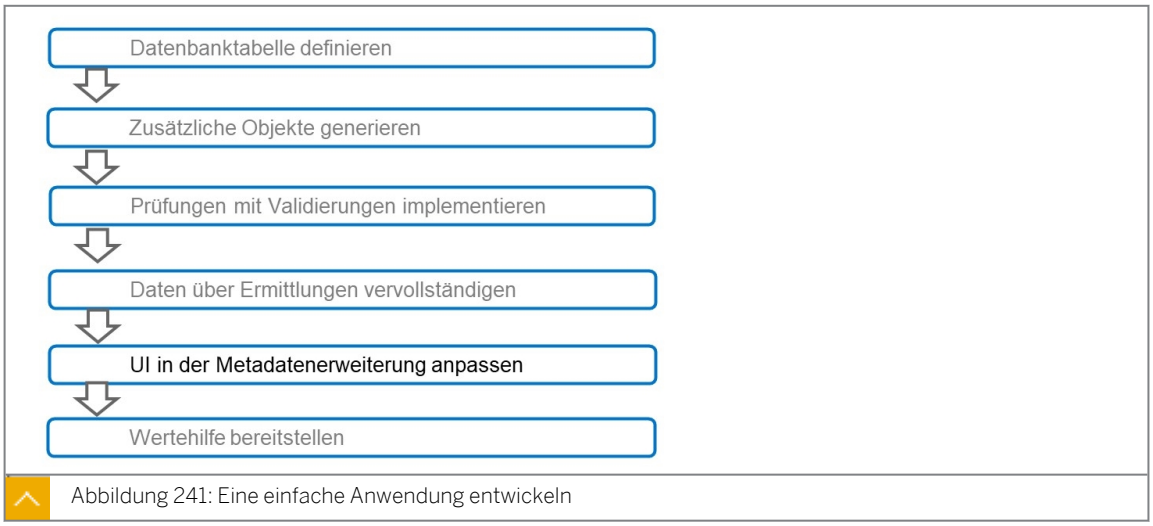


Abbildung 241: Eine einfache Anwendung entwickeln

In diesem Abschnitt erfahren Sie, wie Sie Feldeigenschaften in der Verhaltensdefinition steuern und die Benutzungsoberfläche in der Metadatenerweiterung anpassen.

© Copyright. Alle Rechte vorbehalten.

Felder auf der Benutzungsoberfläche anordnen

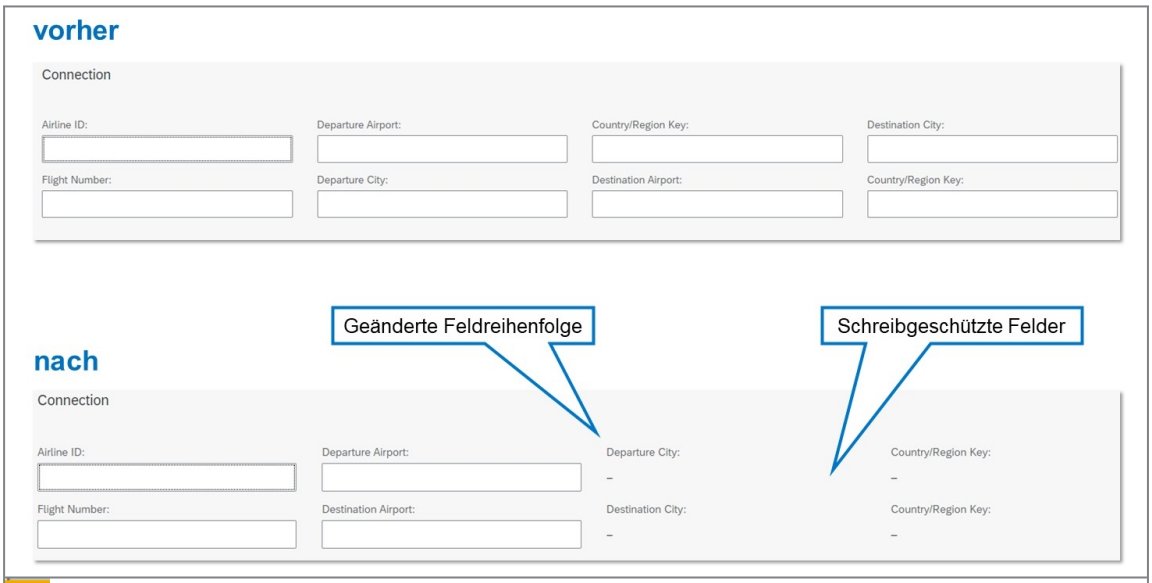


Abbildung 242: UI anpassen

Wenn Sie eine SAP-Fiori-Elements-App generieren, legt das System eine Standardbenutzungsoberfläche an. Die Felder werden in der Reihenfolge eingabebereit dargestellt, in der sie in der CDS-Entität erscheinen.

Im Beispiel zur Flugverbindung können Sie die Informationen leichter lesbar machen, indem Sie die Reihenfolge der Felder ändern. Sie können die Felder, die von der Ermittlung automatisch gefüllt werden, auch schreibgeschützt machen.

Felder schreibgeschützt machen

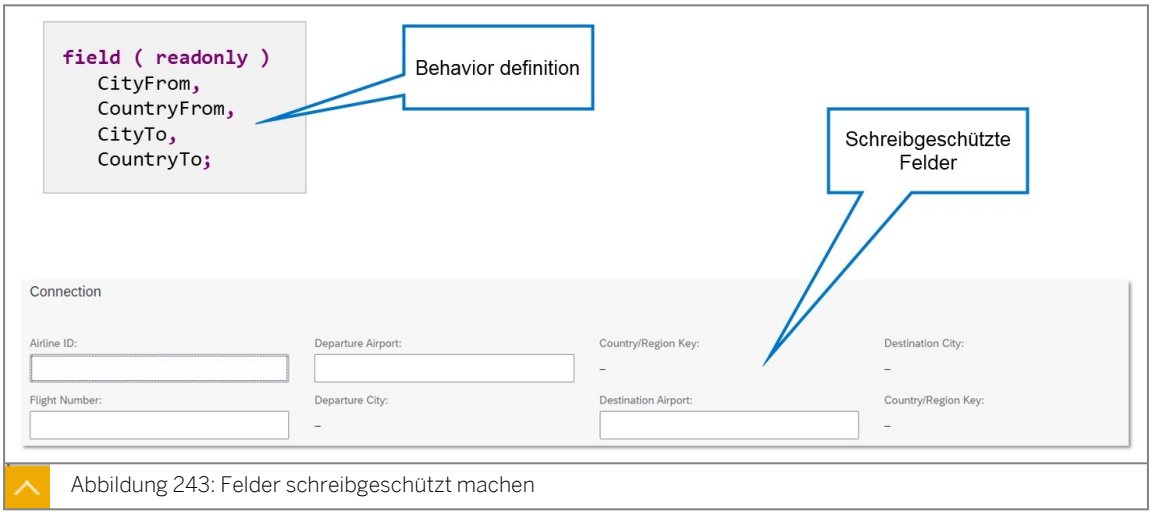


Abbildung 243: Felder schreibgeschützt machen

Sie machen Felder in der Verhaltensdefinition oder der Verhaltensprojektion schreibgeschützt. Es hängt davon ab, ob Sie möchten, dass sie allgemein oder nur für Ihren bestimmten Service schreibgeschützt sind.

In unserem Beispiel machen wir die Felder „Ort“ und „Land“ auf Datenmodellebene schreibgeschützt, da ihre Werte immer von der Ermittlung bereitgestellt werden sollten. Wenn wir sie weiterhin bearbeitbar halten, müssen wir eine Validierung implementieren, um sicherzustellen, dass die Städte und Länder mit den Flughafen-IDs übereinstimmen.

© Copyright. Alle Rechte vorbehalten.

Um eine Liste von Feldern als schreibgeschützt zu definieren, verwenden Sie die Anweisung field (readonly) mit einer kommagetrennten Liste von Feldern. Sie beenden die Liste mit einem Semikolon nach dem letzten Feld.

Sobald Sie die Felder schreibgeschützt gemacht haben, können Sie sehen, dass es sinnvoll ist, sie neu anzuordnen. Dazu annotieren Sie die entsprechenden Felder der Projektion. In den meisten Fällen tun Sie dies in einer Metadatenerweiterung.

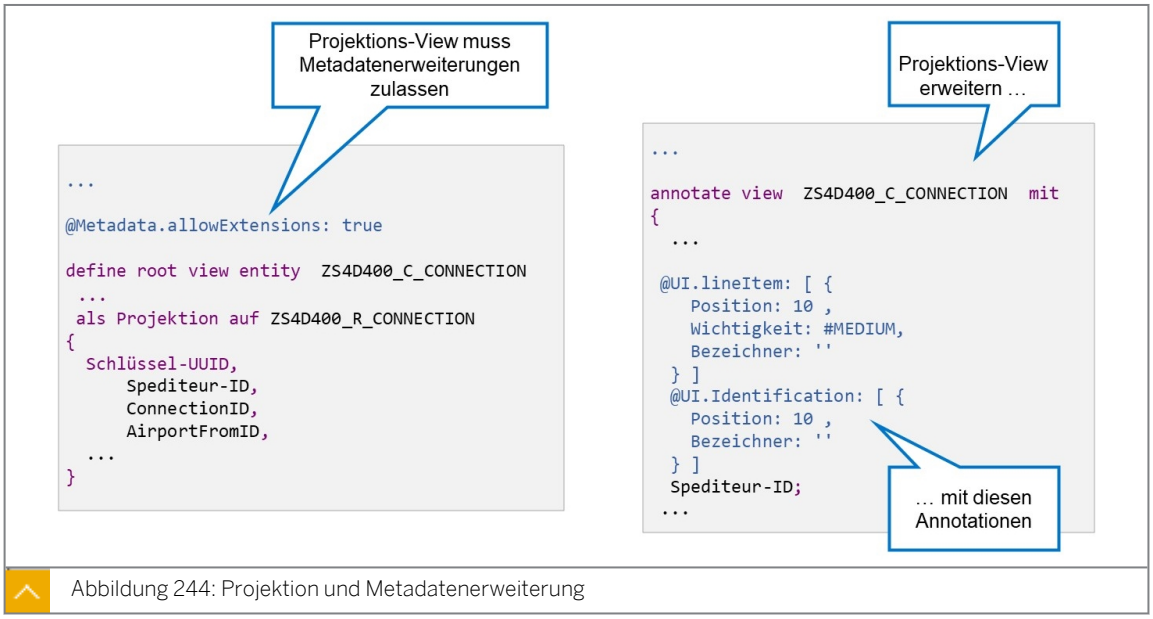


Abbildung 244: Projektion und Metadatenerweiterung

Metadatenerweiterungen sind eigenständige Repository-Objekte, die die zu einer bestimmten CDS-Entität gehörenden Annotationen enthalten. Die Idee dahinter ist, sowohl die Definition der CDS-Entität als auch die Annotationen durch deren Trennung verständlicher zu machen. Andernfalls würde die CDS-Definition sehr lang und schwer lesbar werden.

Wenn Sie eine Metadatenerweiterung in Verbindung mit einer CDS-View verwenden möchten, muss die CDS-View die Annotation `@Metadata.allowExtensions: true` enthalten. Der Generator übernimmt dies für Sie und generiert automatisch sowohl eine Metadatenerweiterung für die Projektions-View als auch die entsprechende Annotation in der Projektions-View selbst.

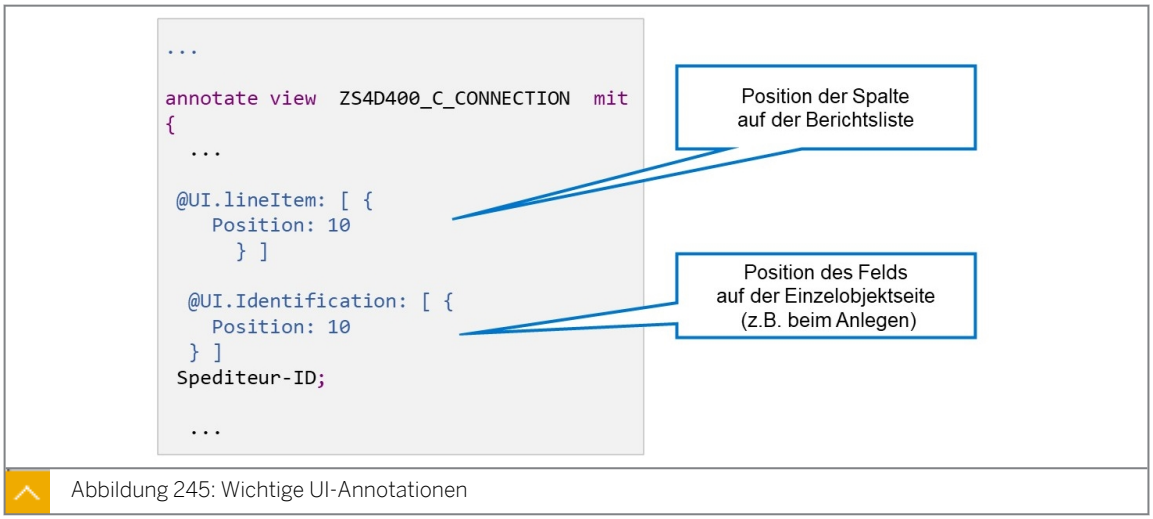


Abbildung 245: Wichtige UI-Annotationen

© Copyright. Alle Rechte vorbehalten.

Es gibt zwei wichtige UI-Annotationen, die Sie möglicherweise in der Metadatenerweiterung ändern müssen. Die erste ist `@UI.LineItem`. Dies ist ein Array, in dem Sie die *Attributposition* festlegen. Dies bestimmt die Position der Spalte auf der Berichtslistenseite der App.

Die zweite Annotation ist `@UI.identification`. Dies ist wiederum ein Array, in dem Sie auch die *Attributposition* festlegen. Dies bestimmt die Position des Felds auf der Einzelobjektseite, die während des Anlege- oder Aktualisierungsvorgangs angezeigt wird.

Felder neu anordnen

Abbildung 246: Felder neu anordnen

Um die Position der Felder zu ändern, ordnen Sie dem Attribut`@UI.identification: [ { position: } ]` einen neuen Wert zu. Um die Felder so anzuordnen, dass sie wie in der Abbildung "Nach" angezeigt werden, müssen Sie die Position des Feldes *AirportToID* von 60 auf 40, die Position des Felds *CityFrom* von 40 auf 50, die Position des Feldes *CountryFrom* von 50 auf 70 usw. ändern.

Eingabehilfe

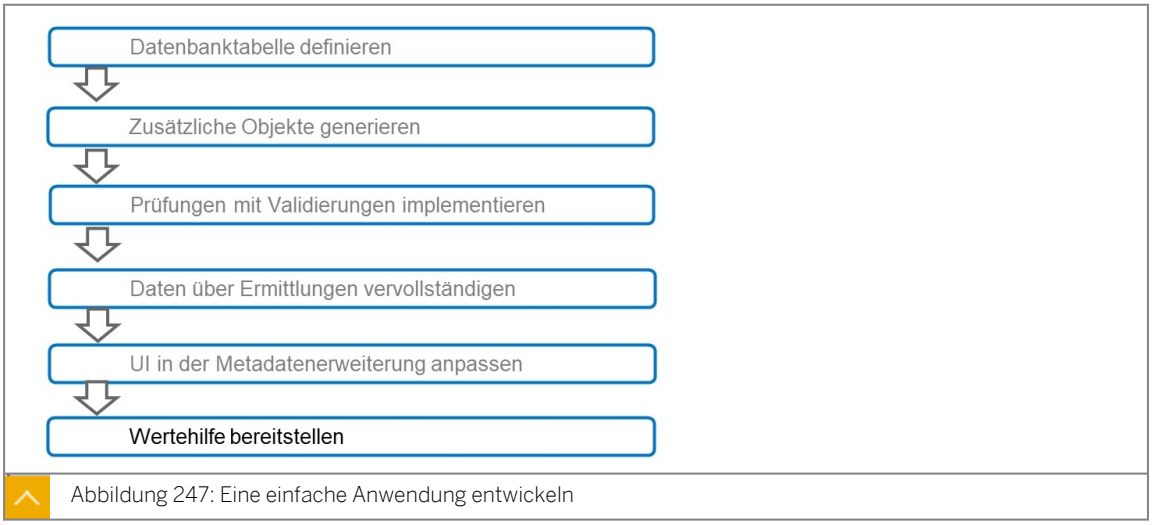


Abbildung 247: Eine einfache Anwendung entwickeln

© Copyright. Alle Rechte vorbehalten.

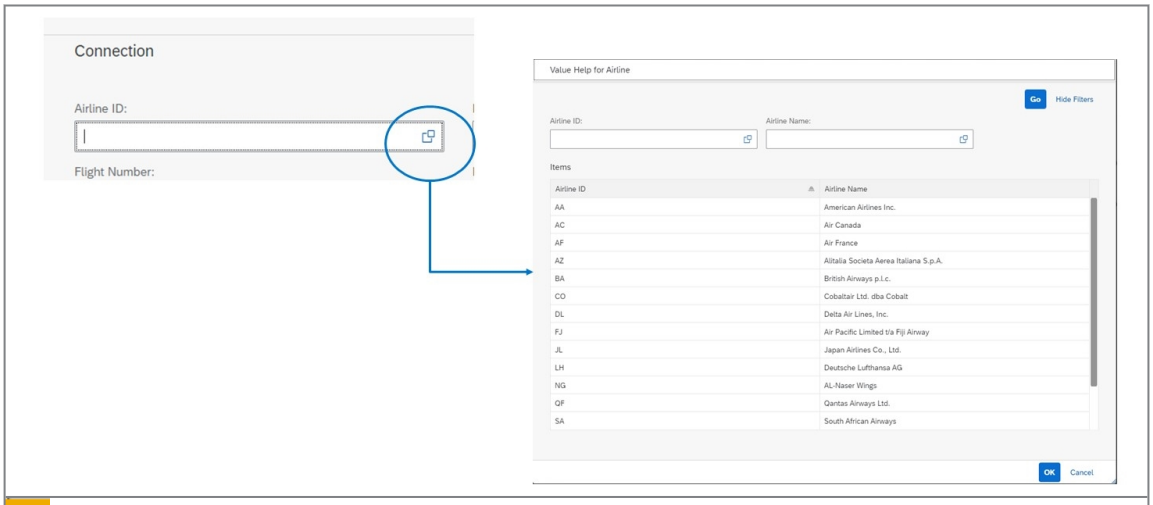


Abbildung 248: Wertehilfe bereitstellen

Eine Funktion aller SAP-Benutzungsoberflächenlösungen im Laufe der Jahre ist eine Wertehilfe, die dem Benutzer die Möglichkeit bietet, eine Liste möglicher Werte anzuzeigen, die in ein Feld eingegeben werden können. Dies wird in der Regel durch das Lesen von Daten aus der Datenbank erreicht. In Fiori Elements können Sie eine Wertehilfe bereitstellen, indem Sie eine CDS-View definieren, die die möglichen Werte anzeigt, und sie an ein Feld in der App anhängen.

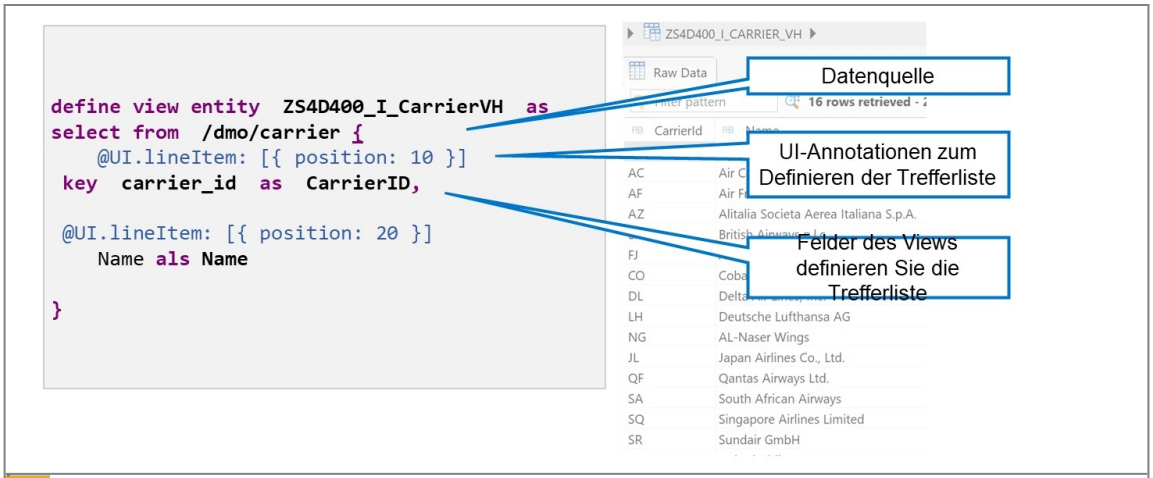


Abbildung 249: CDS-View ZS4D400_I_CARRIERVH

Diese CDS-View wählt Daten aus der Datenbanktabelle /DMO/CARRIER aus, die eine Liste der Fluggesellschaften enthält. Sie gibt zwei Spalten zurück: den Fluggesellschaftscode und seinen Namen. Diese Spalten bilden die Trefferliste der Wertehilfe.

Die View enthält auch `@UI.lineItem` Annotationen. Diese legen fest, wie die Trefferliste auf dem Bildschirm aussehen wird - in diesem Fall mit dem Airline Code zuerst und dessen Namen als zweiter.

© Copyright. Alle Rechte vorbehalten.

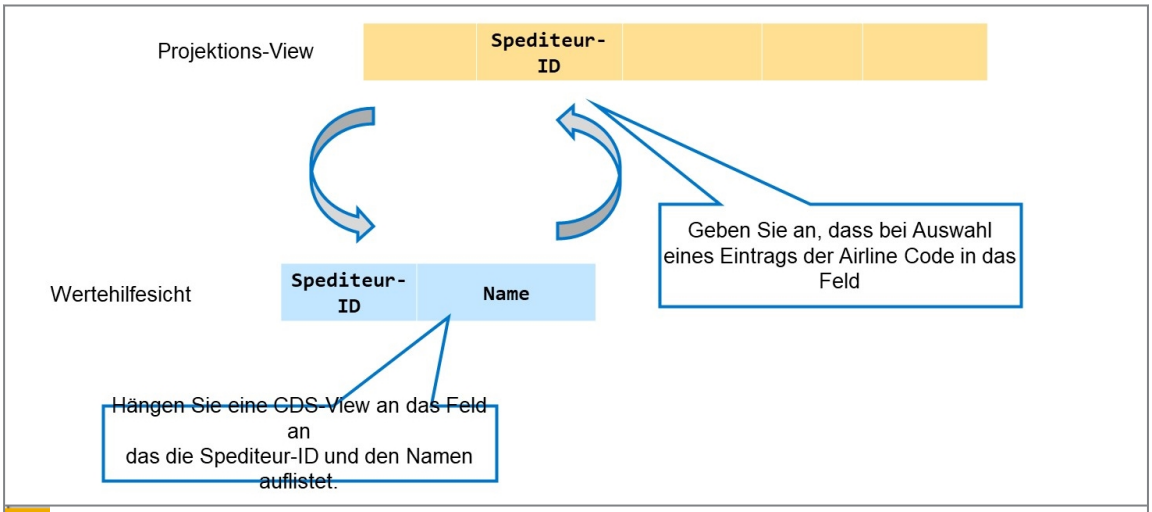


Abbildung 250: Wertehilfe an ein Feld anhängen

Um eine Wertehilfe bereitzustellen, hängen Sie die View, die die Trefferliste definiert, über eine Annotation an ein Feld Ihrer Projektions-View an. Das System zeigt das Feld dann mit einer Wertehilfe-Drucktaste an. Wenn der Benutzer die Drucktaste wählt, liest das System die möglichen Werte über die Sicht und zeigt die Trefferliste entsprechend an.

Wenn der Benutzer einen Eintrag aus der Trefferliste auswählt, muss das System einen Einzelwert in das Feld zurückgeben. In der Annotation muss zusätzlich angegeben werden, welches Feld der Trefferliste der Wertehilfe in das Eingabefeld gestellt werden soll.

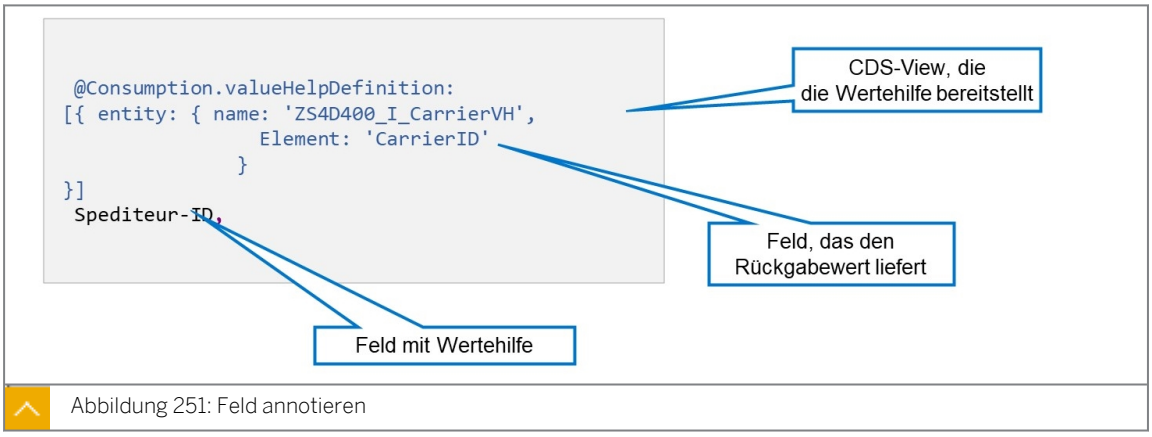


Abbildung 251: Feld annotieren

Um die Wertehilfe an das Feld anzuhängen, verwenden Sie die Annotation `@Consumption.valueHelpDefinition`. Dies ist ein Array, das eine *Objektentität* enthält. *entity* hat zwei Attribute: *name* ist der Name der CDS-View, mit der das System die Wertehilfe bereitstellen soll, *element* ist der Name des Elements der CDS-View, das den Wert enthält, der verwendet werden soll, wenn der Benutzer einen Eintrag aus der Wertehilfe auswählt.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Übung 21

Benutzungsoberfläche anpassen

In dieser Übung lernen Sie die in der Anwendung bereitgestellte Wertehilfe kennen. Darüber hinaus passen Sie die Benutzungsoberfläche der generierten App an: entweder direkt durch Bearbeiten der UI-Metadaten oder indirekt durch Anpassung des Business-Objekt-Verhaltens.

Vorlage:

keine

Lösung:

/LRN/S4D400_C_FLIGHT (Verhaltensprojektion)
/LRN/S4D400_C_FLIGHT (Metadatenerweiterung)

Voraussetzungen

Sie haben die vorherigen Übungen abgeschlossen. Sie haben die Datenbanktabelle **`Z##FLIGHT`** (wobei ## für Ihre Gruppennummer steht) angelegt und gefüllt und die Entwicklungsobjekte für einen OData-UI-Service generiert.

Aufgabe 1: Wertehilfe analysieren

Stellen Sie sicher, dass für das Feld *Währung* auf der Objektseite eine Wertehilfe angeboten wird. Untersuchen Sie, wo und wie diese Wertehilfe definiert ist.

1. Starten Sie die Vorschau des OData-UI-Service.
2. Öffnen Sie die Objektseite für einen Flug im Bearbeitungsmodus.
3. Stellen Sie sicher, dass für das Feld *Währung* eine Wertehilfe verfügbar ist.
4. Kehren Sie zur Entwicklungsumgebung zurück, und erkunden Sie die Definition der Wertehilfe.

   Welche Annotation wird in der Projektions-View *ZC_##Flight* verwendet, um die Wertehilfe zu definieren?

   _____
   _____
   _____

   Welche CDS-View stellt die Wertehilfe bereit?

   _____
   _____
   _____

© Copyright. Alle Rechte vorbehalten.

Aufgabe 2: Verhalten anpassen
Schließen Sie die Standardoperationen *create* und *delete* aus, und setzen Sie das Feld **`PlaneTypeID`** auf schreibgeschützt.

> Notiz:
> Diese Änderungen sollten nur für Ihren OData-UI-Service gelten. Das Business-Objekt selbst sollte weiterhin das Anlegen und Löschen von Flügen sowie das Ändern des Flugzeugtyps unterstützen.

1. Deaktivieren Sie in der Verhaltensprojektion für Ihren Service die Standardoperationen *create* und *delete*.
2. Setzen Sie in der Verhaltensprojektion Ihres Business-Objekts das Feld **`PlaneTypeID`** auf schreibgeschützt.
3. Aktivieren Sie die Verhaltensprojektion.

Aufgabe 3: UI-Metadaten anpassen
Blenden Sie die administrativen Felder auf der Benutzungsoberfläche aus. Stellen Sie sicher, dass nur **`Spediteur-ID`** und **`Verbindungs-ID`** als Selektionsfelder auf der Berichtslistenseite angezeigt werden. Ändern Sie die Feldreihenfolge auf der Objektseite, sodass **`PlaneTypeID`** vor **`Price`** angezeigt wird.

1. Bearbeiten Sie die Metadatenerweiterung der Projektions-View **`ZC_##Flight`**. Entfernen Sie alle UI-Annotationen für die Felder **`LocalCreatedBy`**, **`LocalCreatedAt`**, **`LocalLastChangedBy`**, **`LocalLastChangedAt`** und **`LastChangedAt`**, und ersetzen Sie sie durch **`@UI.hidden: true`**.
2. Entfernen Sie die Annotation **`@UI.selectionField`** für alle Elemente außer den Elementen **`CarrierID`** und **`ConnectionID`**.
3. Ändern Sie die Position des Feldes **`PlaneTypeID`** auf der Object Page. Platzieren Sie es zwischen **`FlightDate`** und **`Price`**.

> Hinweis:
> Die Position auf der Object Page wird mit der Annotation **`@UI.identifier`** angegeben.

4. Aktivieren Sie die Metadatenerweiterung.

Aufgabe 4: Ergebnis überprüfen
Starten Sie die Vorschau des OData-UI-Service neu, und überprüfen Sie die Änderungen an der Benutzungsoberfläche.

1. Starten Sie die Vorschau des OData-UI-Service neu.
2. Stellen Sie auf der Berichtslistenseite sicher, dass die Drucktasten *Anlegen* und *Löschen* verschwunden sind.
3. Stellen Sie sicher, dass auf der Berichtslistenseite keine administrativen Felder mehr angezeigt werden.

© Copyright. Alle Rechte vorbehalten.

4. Stellen Sie sicher, dass neben dem Feld *Bearbeitungsstatus* oben auf der Berichtslistenseite nur zwei Auswahlfelder angezeigt werden. Testen Sie deren Funktionalität.

Hinweis:
Beachten Sie, dass die Selektionsfelder die Eingabe mehrerer Werte und Bereiche unterstützen.

5. Navigieren Sie für jedes Datenset zur Object Page. Stellen Sie sicher, dass die Drucktaste *Löschen* verschwunden ist.
6. Stellen Sie sicher, dass auf der Object Page keine administrativen Felder mehr angezeigt werden.
7. Stellen Sie sicher, dass das Feld *Flugzeugtyp* nach dem Feld *Flugdatum* und vor dem Feld *Flugpreis* angezeigt wird.
8. Wechseln Sie in den Bearbeitungsmodus, und stellen Sie sicher, dass der Wert des Felds *Flugzeugtyp* nicht geändert werden kann.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8
Lösung 21

Benutzungsoberfläche anpassen

In dieser Übung lernen Sie die in der Anwendung bereitgestellte Wertehilfe kennen. Darüber hinaus passen Sie die Benutzungsoberfläche der generierten App an: entweder direkt durch Bearbeiten der UI-Metadaten oder indirekt durch Anpassung des Business-Objekt-Verhaltens.

Vorlage:

keine

Lösung:

/LRN/S4D400_C_FLIGHT (Verhaltensprojektion)
/LRN/S4D400_C_FLIGHT (Metadatenerweiterung)

Voraussetzungen

Sie haben die vorherigen Übungen abgeschlossen. Sie haben die Datenbanktabelle **`Z##FLIGHT`** (wobei ## für Ihre Gruppennummer steht) angelegt und gefüllt und die Entwicklungsobjekte für einen OData-UI-Service generiert.

Aufgabe 1: Wertehilfe analysieren

Stellen Sie sicher, dass für das Feld *Währung* auf der Objektseite eine Wertehilfe angeboten wird. Untersuchen Sie, wo und wie diese Wertehilfe definiert ist.

1. Starten Sie die Vorschau des OData-UI-Service.
   a) Fahren Sie wie in den vorangegangenen Übungen fort.
2. Öffnen Sie die Objektseite für einen Flug im Bearbeitungsmodus.
   a) Fahren Sie wie in den vorangegangenen Übungen fort.
3. Stellen Sie sicher, dass für das Feld *Währung* eine Wertehilfe verfügbar ist.
   a) Öffnen Sie die Wertehilfe für das Feld *Währung*, das für den Flugpreis angezeigt wird.
4. Kehren Sie zur Entwicklungsumgebung zurück, und erkunden Sie die Definition der Wertehilfe.

   Welche Annotation wird in der Projektions-View *ZC_##Flight* verwendet, um die Wertehilfe zu definieren?

   `@Consumption.valueHelpDefinition`

© Copyright. Alle Rechte vorbehalten.

Welche CDS-View stellt die Wertehilfe bereit?

I_CurrencyStdVH

Aufgabe 2: Verhalten anpassen

Schließen Sie die Standardoperationen *create* und *delete* aus, und setzen Sie das Feld **PlaneTypeID** auf schreibgeschützt.

> Notiz:
> Diese Änderungen sollten nur für Ihren OData-UI-Service gelten. Das Business-Objekt selbst sollte weiterhin das Anlegen und Löschen von Flügen sowie das Ändern des Flugzeugtyps unterstützen.

1. Deaktivieren Sie in der **Verhaltensprojektion** für Ihren Service die Standardoperationen *create* und *delete*.
   a) Öffnen Sie die Verhaltensprojektion **ZC_##FLIGHT** (wobei **##** für Ihre Gruppennummer steht).
   b) Fügen Sie Kommentarmarkierungen vor den Anweisungen `use create;` und `use delete;` hinzu.

      Um die Kommentarmarkierungen hinzuzufügen, gehen Sie wie gewohnt vor: Markieren Sie die Codezeilen, die Sie kommentieren möchten, und drücken Sie **Strg + <**.

      Ihr Code sollte dann wie folgt aussehen:

```
//  use create;
use update;
//  use delete;
```

2. Setzen Sie in der **Verhaltensprojektion** Ihres Business-Objekts das Feld **PlaneTypeID** auf schreibgeschützt.
   a) Fügen Sie in der Verhaltensprojektion den folgenden Quelltext hinzu:

```
field ( readonly ) PlaneTypeID;
```

> Notiz:
> Die genaue Position der Anweisung spielt keine Rolle. Wir empfehlen, dass Sie es vor der **Verwendung platzieren...** Anweisungen.

3. Aktivieren Sie die Verhaltensprojektion.
   a) Drücken Sie **Strg + F3**, um die Verhaltensprojektion zu aktivieren.

Aufgabe 3: UI-Metadaten anpassen

Blenden Sie die administrativen Felder auf der Benutzungsoberfläche aus. Stellen Sie sicher, dass nur **Spediteur-ID** und **Verbindungs-ID** als Selektionsfelder auf der Berichtslistenseite angezeigt werden. Ändern Sie die Feldreihenfolge auf der Objektseite, sodass **PlaneTypeID** vor **Price** angezeigt wird.

1. Bearbeiten Sie die Metadatenerweiterung der Projektions-View **ZC_##Flight**. Entfernen Sie alle UI-Annotationen für die Felder **LocalCreatedBy**, **LocalCreatedAt**,

© Copyright. Alle Rechte vorbehalten.

LocalLastChangedBy, **LocalLastChangedAt** und **LastChangedAt**, und ersetzen Sie sie durch **@UI.hidden: true**.

a) Öffnen Sie die Metadatenerweiterung **ZC_##FLIGHT**, wobei **##** für Ihre Gruppennummer steht.

b) Passen Sie den Quelltext wie folgt an:

```
@UI.hidden: true
LocalCreatedBy;

@UI.hidden: true
LocalCreatedAt;

@UI.hidden: true
LocalLastChangedBy;

@UI.hidden: true
LocalLastChangedAt;

@UI.hidden: true
LastChangedAt;
```

2. Entfernen Sie die Annotation **@UI.selectionField** für alle Elemente außer den Elementen **CarrierID** und **ConnectionID**.

a) Passen Sie den Quelltext wie folgt an:

```
@UI.identification: [ {
  position: 30
} ]
@UI.lineItem: [ {
  position: 30
} ]
FlightDate;

@UI.identification: [ {
  position: 40
} ]
@UI.lineItem: [ {
  position: 40
} ]
Price;

@UI.identification: [ {
  position: 50
} ]
@UI.lineItem: [ {
  position: 50
} ]
PlaneTypeID;
```

3. Ändern Sie die Position des Feldes **PlaneTypeID** auf der Object Page. Platzieren Sie es zwischen **FlightDate** und **Price**.

Hinweis:
Die Position auf der Object Page wird mit der Annotation **@UI.identifier** angegeben.

© Copyright. Alle Rechte vorbehalten.

a) Ändern Sie den Wert der Annotation **`@UI.identifier: [ { position: } ]`** für das Element **`PlaneTypeID`**. Wählen Sie einen Wert, der zwischen den Werten für die Elemente **`FlightDate`** und **`Price`** liegt.

b) Passen Sie den Quelltext wie folgt an:

```
@UI.identification: [ {
  position: 35
} ]
@UI.lineItem: [ {
  position: 50
} ]
PlaneTypeID;
```

4. Aktivieren Sie die Metadatenerweiterung.
   a) Drücken Sie **`Strg + F3`**, um die Metadatenerweiterung zu aktivieren.

Aufgabe 4: Ergebnis überprüfen

Starten Sie die Vorschau des OData-UI-Service neu, und überprüfen Sie die Änderungen an der Benutzungsoberfläche.

1. Starten Sie die Vorschau des OData-UI-Service neu.
   a) Schließen Sie das Browserfenster oder die Browserregisterkarte, die die Vorschau enthält.
   b) Öffnen Sie die Service-Bindung **`ZUI_##FLIGHT_O4`**. Wählen Sie in der Liste *Entitätsmenge und Assoziation* auf der rechten Seite zunächst den Eintrag *Flug* und dann *Vorschau....*
2. Stellen Sie auf der Berichtslistenseite sicher, dass die Drucktasten *Anlegen* und *Löschen* verschwunden sind.
   a) Fahren Sie wie in den vorangegangenen Übungen fort.
3. Stellen Sie sicher, dass auf der Berichtslistenseite keine administrativen Felder mehr angezeigt werden.
   a) Fahren Sie wie in den vorangegangenen Übungen fort.
4. Stellen Sie sicher, dass neben dem Feld *Bearbeitungsstatus* oben auf der Berichtslistenseite nur zwei Auswahlfelder angezeigt werden. Testen Sie deren Funktionalität.

Hinweis:
Beachten Sie, dass die Selektionsfelder die Eingabe mehrerer Werte und Bereiche unterstützen.

   a) Geben Sie im Auswahlfeld *Fluggesellschafts-ID* einen Wert ein (z.B. **`LH`**), und wählen Sie *Start*.
   b) Wiederholen Sie dies mit anderen Werten.
5. Navigieren Sie für jedes Datenset zur Object Page. Stellen Sie sicher, dass die Drucktaste *Löschen* verschwunden ist.
   a) Fahren Sie wie in den vorangegangenen Übungen fort.
6. Stellen Sie sicher, dass auf der Object Page keine administrativen Felder mehr angezeigt werden.

© Copyright. Alle Rechte vorbehalten.

a) Fahren Sie wie in den vorangegangenen Übungen fort.

7. Stellen Sie sicher, dass das Feld *Flugzeugtyp* nach dem Feld *Flugdatum* und vor dem Feld *Flugpreis* angezeigt wird.
   a) Fahren Sie wie in den vorangegangenen Übungen fort.
8. Wechseln Sie in den Bearbeitungsmodus, und stellen Sie sicher, dass der Wert des Felds *Flugzeugtyp* nicht geändert werden kann.
   a) Fahren Sie wie in den vorangegangenen Übungen fort.

© Copyright. Alle Rechte vorbehalten.

ZUSAMMENFASSUNG DER LEKTION

Nun können Sie

- Ordnen Sie Felder in der App an.
- Stellen Sie eine Eingabehilfe bereit.

© Copyright. Alle Rechte vorbehalten.

© Copyright. Alle Rechte vorbehalten.

Kapitel 8

Überprüfung des Lernerfolgs

1. Sie möchten mithilfe des Objektgenerators eine Anwendung gemäß dem ABAP-RESTful-Anwendungsprogrammiermodell anlegen. Welche Art von Entwicklungsobjekt benötigen Sie als Ausgangspunkt?

   Wählen Sie die richtige Antwort.

   - ☐ A Datenbanktabelle
   - ☐ B CDS-Entität
   - ☐ C Verhaltensdefinition
   - ☐ D Metadatenerweiterung
   - ☐ E Service-Definition
   - ☐ F Service-Binding

2. Mit welchem der folgenden Elemente können Sie den Datentyp einer Spalte in einer Datenbanktabelle angeben?

   Wählen Sie die richtige Antwort.

   - ☐ A Domäne
   - ☐ B Datenelement
   - ☐ C Lokaler Typ

3. Sie möchten die Entwicklungsobjekte für eine einfache ABAP-RESTful-Anwendung generieren. Bei der Definition der Datenbanktabelle müssen Sie ein Mandantenfeld anlegen. Welchen Datentyp muss er haben?

   Wählen Sie die richtige Antwort.

   - ☐ A Der eingebaute ABAP-Dictionary-Typ abap.char(3)
   - ☐ B Der eingebaute ABAP-Dictionary-Typ abap.clnt.
   - ☐ C Das Datenelement MANDT.

© Copyright. Alle Rechte vorbehalten.

4. Welche der folgenden Teile einer Verhaltensdefinition werden automatisch generiert?

   Wählen Sie die richtigen Antworten.

   - ☐ A Entwurfsaktivierung
   - ☐ B Validierungen
   - ☐ C Ermittlungen
   - ☐ D Anlege-, Aktualisierungs- und Löschvorgänge

5. Sie müssen einen Fehlertext anzeigen. Wo erstellen Sie es?

   Wählen Sie die richtige Antwort.

   - ☐ A In einem Textpool.
   - ☐ B In einer globalen Klasse.
   - ☐ C In einer Nachrichtenklasse.

6. Was generiert das System beim Anlegen einer Validierung automatisch?

   Wählen Sie die richtige Antwort.

   - ☐ A Eine leere Methode in der lokalen Klasse der Verhaltensimplementierung.
   - ☐ B Eine leere Methode in der globalen Klasse der Verhaltensimplementierung.
   - ☐ C Eine vollständig implementierte Methode in der lokalen Klasse der Verhaltensimplementierung.
   - ☐ D Eine vollständig implementierte Methode in der globalen Klasse der Verhaltensimplementierung.

7. In der Validierung verwenden Sie die READ ENTITIES -Anweisung, um die vom Benutzer eingegebenen Daten zu lesen. Welchen Parameter der Validierungsmethode verwenden Sie, um sicherzustellen, dass die korrekten Daten abgerufen werden?

   Wählen Sie die richtige Antwort.

   - ☐ A BERICHTIGT
   - ☐ B DURCHGEFALLEN
   - ☐ C KEYS

© Copyright. Alle Rechte vorbehalten.

8. Wo deklarieren Sie Felder als schreibgeschützt?

   Wählen Sie die richtige Antwort.

   - ☐ A In der Metadatenerweiterung
   - ☐ B In der Verhaltensimplementierung
   - ☐ C In der Verhaltensdefinition

9. Was gibt die Annotation @Consumption.ValueHelpDefiniton an?

   Wählen Sie die richtigen Antworten.

   - ☐ A eine Liste von Eingabemöglichkeiten
   - ☐ B CDS-View, die die Trefferliste bereitstellt
   - ☐ C Liste der Felder in der Trefferliste
   - ☐ D Das Feld, das den ausgewählten Wert bereitstellt

© Copyright. Alle Rechte vorbehalten.

Kapitel 8

Überprüfung des Lernerfolgs - Antworten

1. Sie möchten mithilfe des Objektgenerators eine Anwendung gemäß dem ABAP-RESTful-Anwendungsprogrammiermodell anlegen. Welche Art von Entwicklungsobjekt benötigen Sie als Ausgangspunkt?

   Wählen Sie die richtige Antwort.

   - [X] A Datenbanktabelle
   - [ ] B CDS-Entität
   - [ ] C Verhaltensdefinition
   - [ ] D Metadatenerweiterung
   - [ ] E Service-Definition
   - [ ] F Service-Binding

   Richtig! Sie benötigen eine Datenbanktabelle als Ausgangspunkt.

2. Mit welchem der folgenden Elemente können Sie den Datentyp einer Spalte in einer Datenbanktabelle angeben?

   Wählen Sie die richtige Antwort.

   - [ ] A Domäne
   - [X] B Datenelement
   - [ ] C Lokaler Typ

   Das ist richtig.

3. Sie möchten die Entwicklungsobjekte für eine einfache ABAP-RESTful-Anwendung generieren. Bei der Definition der Datenbanktabelle müssen Sie ein Mandantenfeld anlegen. Welchen Datentyp muss er haben?

   Wählen Sie die richtige Antwort.

   - [ ] A Der eingebaute ABAP-Dictionary-Typ abap.char(3)
   - [X] B Der eingebaute ABAP-Dictionary-Typ abap.clnt.
   - [ ] C Das Datenelement MANDT.

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

4. Welche der folgenden Teile einer Verhaltensdefinition werden automatisch generiert?

   Wählen Sie die richtigen Antworten.

   - [X] A Entwurfsaktivierung
   - [ ] B Validierungen
   - [ ] C Ermittlungen
   - [X] D Anlege-, Aktualisierungs- und Löschvorgänge

   Das ist richtig.

5. Sie müssen einen Fehlertext anzeigen. Wo erstellen Sie es?

   Wählen Sie die richtige Antwort.

   - [ ] A In einem Textpool.
   - [ ] B In einer globalen Klasse.
   - [X] C In einer Nachrichtenklasse.

   Das ist richtig.

6. Was generiert das System beim Anlegen einer Validierung automatisch?

   Wählen Sie die richtige Antwort.

   - [X] A Eine leere Methode in der lokalen Klasse der Verhaltensimplementierung.
   - [ ] B Eine leere Methode in der globalen Klasse der Verhaltensimplementierung.
   - [ ] C Eine vollständig implementierte Methode in der lokalen Klasse der Verhaltensimplementierung.
   - [ ] D Eine vollständig implementierte Methode in der globalen Klasse der Verhaltensimplementierung.

   Das ist richtig.

7. In der Validierung verwenden Sie die READ ENTITIES -Anweisung, um die vom Benutzer eingegebenen Daten zu lesen. Welchen Parameter der Validierungsmethode verwenden Sie, um sicherzustellen, dass die korrekten Daten abgerufen werden?

   Wählen Sie die richtige Antwort.

   - [ ] A BERICHTIGT
   - [ ] B DURCHGEFALLEN
   - [X] C KEYS

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.

8. Wo deklarieren Sie Felder als schreibgeschützt?

   Wählen Sie die richtige Antwort.

   - [ ] A In der Metadatenerweiterung
   - [ ] B In der Verhaltensimplementierung
   - [x] C In der Verhaltensdefinition

   Das ist richtig.

9. Was gibt die Annotation @Consumption.ValueHelpDefiniton an?

   Wählen Sie die richtigen Antworten.

   - [ ] A eine Liste von Eingabemöglichkeiten
   - [x] B CDS-View, die die Trefferliste bereitstellt
   - [ ] C Liste der Felder in der Trefferliste
   - [x] D Das Feld, das den ausgewählten Wert bereitstellt

   Das ist richtig.

© Copyright. Alle Rechte vorbehalten.